

PART V — THE MESSAGE BACKBONE

25. RabbitMQ Topology for NEXUS-1

The topology is part of the distributed contract. Exchanges express producer-owned event categories; queues express consumer-owned delivery obligations; bindings connect the two without transferring data ownership to the broker.

CHAPTER DECISION

NEXUS-1 uses durable topic exchanges, durable quorum queues for business integrations, manual consumer acknowledgements, explicit dead-letter routes, and policy-managed operational arguments. The declaration is idempotent and executable; delivery claims remain deliberately narrower than end-to-end business correctness.

This chapter is marked ★ because prose is insufficient. It therefore includes a concrete topology model, RabbitMQ.Client declarations, policy commands, passive verification, and acceptance checks. The code is intentionally the architectural spine rather than a complete platform package. Chapter 26 will persist producer intent; Chapter 27 will prove confirmed dispatch and recovery; Chapter 28 will prove atomic consumer idempotency.

25.1 The guarantee ledger comes before the names

Boundary	Mechanism introduced here	Claim allowed	Claim not yet allowed
Producer → exchange	Durable topic exchange and stable routing-key grammar.	A valid route can be declared and inspected.	A committed domain change will always reach the broker.
Exchange → consumer queue	Durable binding to a durable quorum queue.	Matching publications are eligible for that queue.	Every publication is persistent or confirmed.
Queue → consumer	Manual-ack consumption contract.	Deletion is deferred until the consumer acknowledges.	The business effect and acknowledgement are atomic.
Failed delivery → terminal queue	Policy-selected DLX and explicit dead queue.	Terminal failure becomes operator-visible when routing succeeds.	Dead-letter transfer is lossless in every cluster condition.
End-to-end	Correlation/message metadata required by the canonical envelope.	Evidence can join publish, delivery, failure, and replay.	Exactly-once delivery or globally atomic state.

Durability is a conjunction, not a checkbox. A durable exchange survives broker restart. A durable queue's metadata survives. A message must still be published as persistent, and the producer needs publisher confirms before it can treat broker acceptance as evidence. Even then, a retransmission after an ambiguous connection failure can duplicate delivery. The later inbox is therefore not optional.

SENIOR-REVIEW SHORTCUT

Whenever a diagram says “reliable”, ask for the exact tuple: exchange durability, queue durability/type, message delivery mode, confirm outcome, routing result, consumer acknowledgement, local transaction, and replay behavior.

25.2 Naming is an ownership model

Broker names are operational APIs. Renaming a queue can discard backlog; changing a binding changes the audience; changing a queue type requires migration rather than an in-place policy tweak. NEXUS-1 therefore separates producer vocabulary from consumer deployment identity.

Element	Canonical pattern	Owner	Example
Event exchange	nexus.events	Platform declaration; producers own the routes they publish.	nexus.events
Routing key	<producer>.<event-name>.v<major>	Producer contract owner.	alarm-management.alarm-flood-detected.v1
Consumer queue	<consumer>.<subscription>.v<major>	Consuming capability.	rootcause.alarm-events.v1
Dead-letter exchange	nexus.dead	Platform declaration.	nexus.dead
Dead routing key	<live-queue>.dead	Consuming capability.	rootcause.alarm-events.v1.dead
Dead queue	<live-queue>.dead	Consuming capability / operations.	rootcause.alarm-events.v1.dead

The routing key includes the producer and contract major, not a mutable consumer name and not an entity identifier. Consumers select categories with bindings. Payload evolution within a compatible major remains the producer's responsibility; a breaking semantic or structural change gets a new contract major and a deliberate dual-publish or migration decision.

25.3 The canonical topology

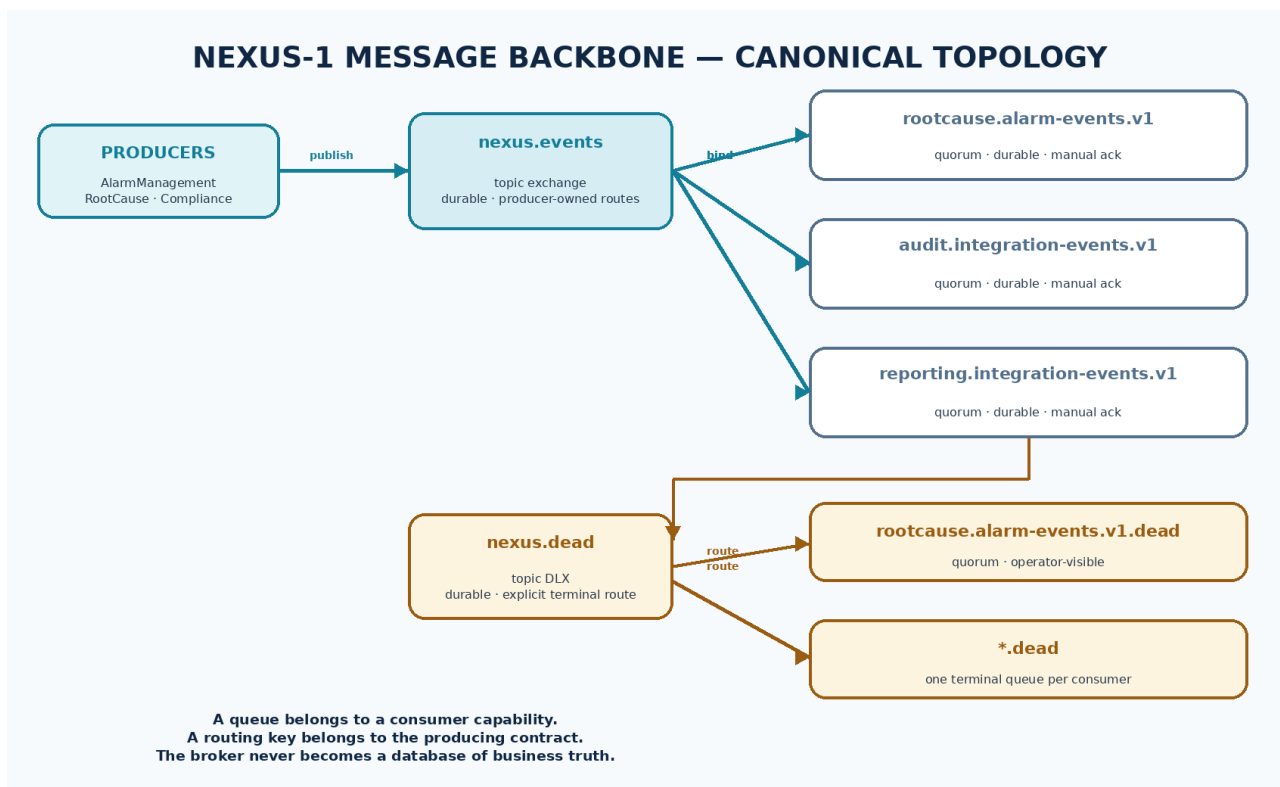


Figure 25.1 — Producer-owned routes feed consumer-owned quorum queues; each live queue has an explicit terminal path.

AlarmManagement publishes an integration event only after its own local decision has created an outbox record. RootCause, Audit, and Reporting do not share a queue: each owns an independent subscription and can progress, pause, retry, or dead-letter without stealing another consumer's delivery. A queue is never shared merely to reduce broker objects.

Queue	Bindings from nexus.events	Queue type	Terminal route
rootcause.alarm-events.v1	alarm-management.alarm-raised.v1; alarm-management.alarm-flood-detected.v1; alarm-management.alarm-cleared.v1	quorum	rootcause.alarm-events.v1.dead
audit.integration-events.v1	alarm-management.;; root-cause.;; compliance.;	quorum	audit.integration-events.v1.dead
compliance.root-cause-events.v1	root-cause.verdict-issued.v1; root-cause.case-closed.v1	quorum	compliance.root-cause-events.v1.dead
reporting.integration-events.v1	alarm-management.;; root-cause.;; compliance.;	quorum	reporting.integration-events.v1.dead

The broad Audit and Reporting bindings are intentional but still versioned. They do not grant those services access to private databases. They receive only producer-approved integration contracts. If a future producer has no approved public event family, the wildcard is narrowed before that producer is added.

25.4 An executable declaration, not a screenshot

The declaration below targets the asynchronous RabbitMQ .NET client API. It is safe to run repeatedly when every existing object has equivalent properties. RabbitMQ closes the channel on an inequivalent redeclaration; that failure is desirable because it exposes configuration drift instead of silently changing semantics.

EXECUTABLE ASSET 25-A — TOPOLOGY MODEL

```
using RabbitMQ.Client;

public static class NexusTopology
{
    public const string Events = "nexus.events";
    public const string Dead = "nexus.dead";

    public sealed record Subscription(
        string Queue,
        string[] Routes,
        string DeadQueue)
    {
        public string DeadRoute => $"{Queue}.dead";
    }

    public static readonly Subscription[] Subscriptions =
    [
        new("rootcause.alarm-events.v1",
            ["alarm-management.alarm-raised.v1",
             "alarm-management.alarm-flood-detected.v1",
             "alarm-management.alarm-cleared.v1"],
            "rootcause.alarm-events.v1.dead"),

        new("audit.integration-events.v1",
            ["alarm-management.#", "root-cause.#", "compliance.#"],
            "audit.integration-events.v1.dead"),

        new("compliance.root-cause-events.v1",
            ["root-cause.verdict-issued.v1",
             "root-cause.case-closed.v1"],
            "compliance.root-cause-events.v1.dead"),

        new("reporting.integration-events.v1",
            ["alarm-management.#", "root-cause.#", "compliance.#"],
            "reporting.integration-events.v1.dead")
    ];
}
```

Queue arguments contain only properties that cannot be supplied safely by a normal policy. NEXUS-1 fixes the queue type as quorum at declaration time. Dead-letter exchange, dead route, delivery limit, overflow, and capacity controls remain policy-managed so operators can change them without deleting and recreating the live queue.

EXECUTABLE ASSET 25-B — IDEMPOTENT RABBITMQ DECLARATION

```
using RabbitMQ.Client;

public static async Task DeclareAsync(
    IChannel channel,
    CancellationToken cancellationToken)
{
    await channel.ExchangeDeclareAsync(
        exchange: NexusTopology.Events,
        type: ExchangeType.Topic,
        durable: true,
        autoDelete: false,
        arguments: null,
        passive: false,
        noWait: false,
        cancellationToken);

    await channel.ExchangeDeclareAsync(
        exchange: NexusTopology.Dead,
        type: ExchangeType.Topic,
        durable: true,
        autoDelete: false,
        arguments: null,
        passive: false,
        noWait: false,
        cancellationToken);

    foreach (var subscription in NexusTopology.Subscriptions)
    {
        var liveArguments = new Dictionary<string, object?>
        {
            ["x-queue-type"] = "quorum"
        };

        await channel.QueueDeclareAsync(
            queue: subscription.Queue,
            durable: true,
            exclusive: false,
            autoDelete: false,
            arguments: liveArguments,
            passive: false,
            noWait: false,
            cancellationToken);

        foreach (var route in subscription.Routes)
            await channel.QueueBindAsync(
                subscription.Queue, NexusTopology.Events, route,
                arguments: null, noWait: false, cancellationToken);

        await channel.QueueDeclareAsync(
            queue: subscription.DeadQueue,
            durable: true,
            exclusive: false,
            autoDelete: false,
            arguments: new Dictionary<string, object?>
            {
                ["x-queue-type"] = "quorum"
            },
            passive: false,
            noWait: false,
            cancellationToken);

        await channel.QueueBindAsync(
            subscription.DeadQueue, NexusTopology.Dead,
            subscription.DeadRoute,
            arguments: null, noWait: false, cancellationToken);
    }
}
```

Why noWait is false

Declaration is control-plane work. The bootstrapper waits for broker responses so an unavailable broker, permission failure, inequivalent object, or invalid argument fails startup or the provisioning job visibly. Throughput is irrelevant here; silent topology drift is the risk.

Why the live queues are quorum queues

The selected subscriptions carry business integration work and need replicated queue semantics. Quorum queues also provide a deliver

the required policy combination is enabled. This does not remove the need for publisher confirms, idempotent consumers, capacity limits, or operational repair.

25.5 Operational policy is separate from application declaration

RabbitMQ policies are the preferred mechanism for dead lettering because hard-coded x-arguments cannot be changed without redeploying applications and deleting/redeclaring queues. The policy below targets only NEXUS-1 live quorum queues and provides an explicit terminal exchange. A production rollout must verify the regular expression against an inventory before applying it.

EXECUTABLE ASSET 25-C — LIVE-QUEUE POLICY

```
rabbitmqctl set_policy nexus-live-queue-safety '^(rootcause\.alarm-events|audit\.integration-events|compliance\.root-cause-events|reporting\.integration-events)\.v1$' '{
  "dead-letter-exchange": "nexus.dead",
  "dead-letter-strategy": "at-least-once",
  "overflow": "reject-publish",
  "delivery-limit": 8,
  "max-length-bytes": 1073741824
}' --priority 50 --apply-to quorum_queues
```

IMPORTANT LIMITATION

At-least-once dead lettering for quorum queues requires the supported feature set and the documented policy combination, including reject-publish overflow. If the target queue lacks an online quorum, transfer can stall. The original live queue therefore remains part of the incident surface; a DLQ is not proof that failure handling cannot lose or block.

The terminal queues deliberately do not dead-letter back into nexus.dead. That prevents an automatic cycle. Replay is an operator-authorized republish with provenance, a new transport attempt, and the original message identifier preserved for inbox deduplication. Chapter 29 will define that process and its tests.

25.6 Acknowledgement is a consumer protocol

The topology cannot configure a consumer acknowledgement mode. The consuming adapter must subscribe with automatic acknowledgement disabled, process one delivery through its inbox and local transaction, commit, and only then acknowledge the delivery. A process failure before acknowledgement allows redelivery. A permanent failure is negatively acknowledged without requeue only after the bounded policy has classified it for the terminal path.

CONSUMER CONTRACT — MANUAL ACKNOWLEDGEMENT

```
var consumer = new AsyncEventingBasicConsumer(channel);

consumer.ReceivedAsync += async (_, delivery) =>
{
    try
    {
        await handler.ProcessAtomicallyAsync(
            delivery.BasicProperties.MessageId!,
            delivery.Body,
            cancellationToken);

        await channel.BasicAckAsync(
            delivery.DeliveryTag,
            multiple: false,
            cancellationToken);
    }
    catch (PermanentMessageException)
    {
        await channel.BasicNackAsync(
            delivery.DeliveryTag,
            multiple: false,
            requeue: false,
            cancellationToken);
    }
    // Transient failure policy is bounded and implemented in Chapter 29.
};

await channel.BasicConsumeAsync(
    queue: NexusTopology.Subscriptions[0].Queue,
    autoAck: false,
    consumer: consumer,
    cancellationToken);
```

This snippet is not yet the Chapter 28 inbox implementation. In particular, `ProcessAtomicallyAsync` must atomically claim the message identifier and commit `RootCause` state in `RootCause`'s own database. The snippet exists here to prevent a topology document from implying that a durable queue alone protects business effects.

25.7 The routing key and envelope solve different problems

The routing key is broker metadata optimized for subscription. The envelope is contract metadata optimized for interpretation, traceability, deduplication, and causality. Consumers must not reconstruct identifiers or occurred time from a routing key.

CANONICAL EVENT ENVELOPE — REPRESENTATIVE INSTANCE

```
{
  "messageId": "019bc4d4-5dc1-7b8d-9d48-542d25315356",
  "eventType": "AlarmFloodDetected",
  "schemaVersion": 1,
  "occurredAtUtc": "2026-07-25T09:42:17.184Z",
  "correlationId": "corr-site-17-window-884",
  "causationId": "cmd-evaluate-window-884",
  "producer": {
    "service": "AlarmManagement",
    "instance": "alarm-management-7d9fd6"
  },
  "data": {
    "siteId": "site-17",
    "windowId": "window-884",
    "alarmCount": 137
  }
}
```

Field	Invariant	Failure if absent
messageId	Immutable across redelivery and authorized replay of the same logical publication.	Inbox cannot deduplicate reliably.
eventType + schemaVersion	Producer-owned, explicit, and validated before dispatch.	Consumer guesses payload meaning.

Field	Invariant	Failure if absent
occurredAtUtc	The producer's fact time, not broker arrival time.	Latency and event-time reasoning collapse.
correlationId	Propagated across the distributed business flow.	Cross-service evidence cannot be joined.
causationId	Names the command/message that directly caused this event.	Causal chain becomes narrative only.
producer	Stable service identity plus runtime instance where available.	Ownership and operational origin become ambiguous.

25.8 Provisioning and application startup are distinct

Production permissions should not require every service instance to create every queue. A deployment-owned topology job can declare shared exchanges and consumer queues with configure rights. Runtime producers need write access to `nexus.events`; runtime consumers need read access to their queues and, where the broker validates DLX permissions during declaration, appropriate write access to `nexus.dead`. The exact least-privilege split is environment-specific and must be verified against broker behavior.

Principal	Configure	Write	Read
nexus-topology	nexus.events, nexus.dead, approved queues/bindings	nexus.dead where declaration validation requires it	live queues where DLX declaration validation requires it
alarm-management	none in production runtime	nexus.events	none
rootcause	none in production runtime	nexus.events for its own produced events	rootcause.alarm-events.v1
operations-replay	none	nexus.events via audited replay tool	*.dead with explicit authorization

DO NOT HIDE PROVISIONING FAILURE

If topology is managed out of process, the service still performs passive checks for the objects it requires. Readiness remains false when a required exchange or queue is absent or inequivalent; liveness does not fail merely because a dependency is temporarily unavailable.

25.9 Passive verification makes drift executable

EXECUTABLE ASSET 25-D — PASSIVE EXISTENCE CHECK

```
public static async Task VerifyRequiredObjectsAsync(
    IChannel channel,
    CancellationToken cancellationToken)
{
    await channel.ExchangeDeclarePassiveAsync(
        NexusTopology.Events, cancellationToken);
    await channel.ExchangeDeclarePassiveAsync(
        NexusTopology.Dead, cancellationToken);

    foreach (var subscription in NexusTopology.Subscriptions)
    {
        await channel.QueueDeclarePassiveAsync(
            subscription.Queue, cancellationToken);
        await channel.QueueDeclarePassiveAsync(
            subscription.DeadQueue, cancellationToken);
    }
}
```

Passive declaration proves existence, not every binding, policy, permission, or effective argument. The deployment gate therefore also queries the management API or definitions export and compares a normalized topology manifest. Secrets and connection strings are excluded from the evidence.

EXECUTABLE ASSET 25-E — NORMALIZED TOPOLOGY MANIFEST

```

{
  "exchanges": {
    "nexus.events": { "type": "topic", "durable": true },
    "nexus.dead": { "type": "topic", "durable": true }
  },
  "requiredBindings": [
    {
      "source": "nexus.events",
      "destination": "rootcause.alarm-events.v1",
      "routingKey": "alarm-management.alarm-flood-detected.v1"
    },
    {
      "source": "nexus.dead",
      "destination": "rootcause.alarm-events.v1.dead",
      "routingKey": "rootcause.alarm-events.v1.dead"
    }
  ],
  "queueDefaults": {
    "durable": true,
    "exclusive": false,
    "autoDelete": false,
    "type": "quorum"
  }
}

```

25.10 Acceptance tests for the topology

A broker-backed test suite creates a disposable vhost, applies the policy, runs the declaration, and inspects behavior. Each test must retain broker version, client version, vhost identity, topology manifest hash, timestamps, and result. Passing a mocked IChannel is not broker evidence.

1. Declare twice. Both runs succeed and produce the same normalized exchanges, queues, bindings, and effective policies.
2. Change nexus.events from topic to direct in the candidate declaration. The broker rejects the inequivalent redeclaration and the deployment gate fails.
3. Publish a persistent message with the AlarmFloodDetected routing key. RootCause, Audit, and Reporting each receive an independent copy; Compliance does not.
4. Publish an unknown routing key with mandatory publication enabled. The producer observes the unroutable return and does not mark the outbox item dispatched.
5. Stop a consumer after delivery but before acknowledgement. On recovery the message is redelivered and retains its messageld.
6. Reject a delivery without requeue after the delivery limit/classifier reaches terminal state. The message appears on the correct dead queue with dead-letter history.
7. Restart broker nodes under the selected quorum configuration. Durable objects remain and a confirmed persistent publication remains available.
8. Remove or misbind a required queue in the disposable vhost. Passive/manifest verification fails readiness and the release gate.

Evidence ID	Artifact	Current status
MQ-TOPO-001	Normalized exchange/queue/binding/policy inventory.	Specified; broker capture required.
MQ-IDEMP-001	Second declaration transcript and identical manifest hash.	Specified; broker run required.
MQ-ROUTE-001	Positive and negative routing matrix.	Specified; broker run required.
MQ-DLX-001	Terminal routing transcript and x-death inspection.	Specified; broker run required.
MQ-RESTART-001	Node restart/recovery trace for selected cluster profile.	Specified; environment run required.

Evidence ID	Artifact	Current status
MQ-DRIFT-001	Inequivalent declaration and missing binding failures.	Specified; broker run required.

25.11 Failure analysis

Failure	Visible signal	Safe reaction	Unsafe reaction
Exchange absent	Passive declare/channel failure; readiness false.	Stop dispatch; retain outbox rows.	Publish to another exchange silently.
Queue absent or binding missing	Manifest mismatch; mandatory return where applicable.	Fail deployment or retain publication for repair.	Mark dispatch complete on TCP write.
No quorum	Queue operation/confirm/dead-letter failure and broker alarms.	Back off; preserve durable intent; alert.	Loop without bound or drop the original.
Consumer dies before ack	Unacked delivery returns/redelivers.	Run inbox-protected handler again.	Assume one delivery means one effect.
Poison message	Repeated delivery/death metadata and terminal queue.	Bound attempts; preserve evidence; authorized repair/replay.	Requeue forever.
Policy drift	Effective policy differs from manifest.	Block release and reconcile deliberately.	Redeclare queues destructively.

CHAPTER 25 - CODE AUDIT

Broker evidence begins in an isolated virtual host

A disposable vhost prevents acceptance tests from mutating shared topology. Every run records the server image, broker version, client package, node count, feature flags and exact permission set before declarations begin.

EVIDENCE STATUS

These listings are compile-oriented. No RabbitMQ cluster, vhost, policy, publication, delivery, node restart or dead-letter transfer has executed here. All MQ evidence remains PENDING.

EXECUTABLE ASSET 25-F - ENVIRONMENT AND PRINCIPAL IDENTITIES

```
public sealed record BrokerEnvironmentIdentity(
    string ServerImageDigest,
    string ServerVersion,
    string ClientPackageVersion,
    int ClusterNodeCount,
    string FeatureFlagsSha256,
    string EnvironmentSha256);

public sealed record RabbitPermission(
    string Configure,
    string Write,
    string Read);

public static class NexusBrokerPermissions
{
    private const string Queues =
        "(rootcause\\.alarm-events|audit\\.integration-events"
        + "|compliance\\.root-cause-events"
        + "|reporting\\.integration-events)\\.v1(\\.dead)?";

    public static readonly RabbitPermission Topology =
        new($"^(nexus\\.events|dead){Queues}$",
            "^nexus\\.events|dead$",
            $"^{Queues}$");

    public static readonly RabbitPermission AlarmPublisher =
        new("^$", "^nexus\\.events$", "^$");

    public static readonly RabbitPermission RootCauseRuntime =
        new("^$", "^nexus\\.events$",
            "^rootcause\\.alarm-events\\.v1$");
}
```

EXECUTABLE ASSET 25-G - DISPOSABLE-VHOST LEASE

```
public sealed class RabbitVhostLease : IAsyncDisposable
{
    private readonly IRabbitManagementAdmin _admin;
    private readonly List<string> _users = [];
    public string Vhost { get; }
    public BrokerEnvironmentIdentity Identity { get; }

    private RabbitVhostLease(
        IRabbitManagementAdmin admin,
        string vhost,
        BrokerEnvironmentIdentity identity)
    {
        _admin = admin;
        Vhost = vhost;
        Identity = identity;
    }

    public static async Task<RabbitVhostLease> StartAsync(
        IRabbitManagementAdmin admin,
        ITestSecretFactory secrets,
        Guid runId,
        CancellationToken cancellationToken)
    {
        var identity = await admin
            .ReadEnvironmentIdentityAsync(cancellationToken);
        if (identity.ClusterNodeCount < 3)
            throw new InsufficientQuorumTestProfile();

        var lease = new RabbitVhostLease(
            admin, $"nexus-compact-{runId:N}", identity);
        try
        {
            await admin.CreateVhostAsync(
                lease.Vhost, cancellationToken);
            await lease.AddPrincipalAsync(
                "topology", NexusBrokerPermissions.Topology,
                secrets, cancellationToken);
            await lease.AddPrincipalAsync(
                "alarm", NexusBrokerPermissions.AlarmPublisher,
                secrets, cancellationToken);
            await lease.AddPrincipalAsync(
                "rootcause",
                NexusBrokerPermissions.RootCauseRuntime,
                secrets, cancellationToken);
            return lease;
        }
        catch
        {
            await lease.DisposeAsync();
            throw;
        }
    }

    private async Task AddPrincipalAsync(
        string role, RabbitPermission permission,
        ITestSecretFactory secrets, CancellationToken ct)
    {
        var principal =
            await _admin.CreateEphemeralPrincipalAsync(
                Vhost, role, permission, secrets, ct);
        _users.Add(principal.Username);
    }

    public async ValueTask DisposeAsync()
    {
        foreach (var user in _users.AsEnumerable().Reverse())
            await _admin.DeleteUserIfExistsAsync(user);
        await _admin.DeleteVhostIfExistsAsync(Vhost);
    }
}
```

PROVISIONING AND FEATURE GATES

Policy prerequisites are executable startup facts

At-least-once dead lettering is accepted only when the selected server profile has the required feature flag and the effective quorum-queue policy contains the complete reviewed tuple.

EXECUTABLE ASSET 25-H - REVIEWED POLICY MODEL

```
public sealed record RabbitPolicyDefinition(
    string Name,
    string Pattern,
    string ApplyTo,
    int Priority,
    IReadOnlyDictionary<string, JsonElement> Definition);

public static class NexusLiveQueuePolicy
{
    public const string Name = "nexus-live-queue-safety";
    public const string Pattern =
        "^(rootcause\\.alarm-events"
        + "|audit\\.integration-events"
        + "|compliance\\.root-cause-events"
        + "|reporting\\.integration-events)\\.v1$";

    public static RabbitPolicyDefinition Build() => new(
        Name,
        Pattern,
        "quorum_queues",
        50,
        new SortedDictionary<string, JsonElement>(
            StringComparer.Ordinal)
        {
            ["dead-letter-exchange"] =
                JsonValue("nexus.dead"),
            ["dead-letter-strategy"] =
                JsonValue("at-least-once"),
            ["overflow"] = JsonValue("reject-publish"),
            ["delivery-limit"] = JsonValue(8),
            ["max-length-bytes"] =
                JsonValue(1_073_741_824)
        });
}
```

EXECUTABLE ASSET 25-I - PROVISION, DECLARE TWICE AND FREEZE

```
public sealed class BrokerTopologyProvisioner(
    IRabbitManagementClient management,
    INexusTopologyDeclaration declaration,
    ITopologyManifestReader manifests)
{
    public async Task<TopologyProvisioningResult> ApplyAsync(
        IConnection connection,
        CancellationToken cancellationToken)
    {
        var flags = await management
            .GetFeatureFlagsAsync(cancellationToken);
        if (!flags.IsEnabled("stream_queue"))
            throw new MissingRequiredFeatureFlag(
                "stream_queue");

        await management.PutPolicyAsync(
            NexusLiveQueuePolicy.Build(),
            cancellationToken);

        await using var channel =
            await connection.CreateChannelAsync(
                cancellationToken: cancellationToken);
        await declaration.DeclareAsync(
            channel, cancellationToken);
        var first = await manifests
            .CaptureAsync(cancellationToken);

        await declaration.DeclareAsync(
            channel, cancellationToken);
        var second = await manifests
            .CaptureAsync(cancellationToken);

        if (!CryptographicOperations.FixedTimeEquals(
            Convert.FromHexString(first.ManifestSha256),
            Convert.FromHexString(second.ManifestSha256)))
            throw new NonIdempotentTopology();

        return new(first, second, flags.Digest);
    }
}

public sealed class TopologyProvisioningTests(
    BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_IDEMP_001_second_declare_is_identical()
    {
        var result = await fixture.ProvisionAsync();
        result.First.Should().BeEquivalentTo(
            result.Second,
            options => options.WithStrictOrdering());
    }
}
```

SAFER IS NOT LOSSLESS

The policy enables RabbitMQ's safer quorum dead-letter path. Missing target quorum, unroutable transfer or target rejection can still stall and may produce duplicates; those conditions remain part of the acceptance suite.

NORMALIZED MANAGEMENT MANIFEST

Definitions become a typed, formatting-free digest

Passive declaration proves existence only. The deployment gate exports one vhost, removes broker-generated noise, sorts every collection and hashes typed exchange, queue, binding and policy fields.

EXECUTABLE ASSET 25-J - CLOSED NORMALIZED TOPOLOGY

```
public sealed record ExchangeFact(
    string Name, string Type, bool Durable);

public sealed record QueueFact(
    string Name,
    bool Durable,
    bool Exclusive,
    bool AutoDelete,
    string QueueType);

public sealed record BindingFact(
    string Source,
    string Destination,
    string DestinationType,
    string RoutingKey);

public sealed record PolicyFact(
    string Name,
    string Pattern,
    string ApplyTo,
    int Priority,
    IReadOnlyList<KeyValuePair<string, string>> Definition);

public sealed record NormalizedTopology(
    IReadOnlyList<ExchangeFact> Exchanges,
    IReadOnlyList<QueueFact> Queues,
    IReadOnlyList<BindingFact> Bindings,
    IReadOnlyList<PolicyFact> Policies,
    string ManifestProfile,
    string ManifestSha256);

public static class TopologyNormalizer
{
    public const string Profile =
        "nexus.rabbit-topology.v1/len32-utf8+sha256";

    public static NormalizedTopology Normalize(
        RabbitDefinitions source)
    {
        var exchanges = source.Exchanges
            .Where(x => x.Name is "nexus.events"
                or "nexus.dead")
            .Select(x => new ExchangeFact(
                x.Name, x.Type, x.Durable))
            .OrderBy(x => x.Name, StringComparer.Ordinal)
            .ToArray();

        var queues = source.Queues
            .Where(x => NexusNames.IsRequiredQueue(x.Name))
            .Select(x => new QueueFact(
                x.Name, x.Durable, x.Exclusive,
                x.AutoDelete, x.ArgumentString("x-queue-type")))
            .OrderBy(x => x.Name, StringComparer.Ordinal)
            .ToArray();

        var bindings = source.Bindings
            .Where(NexusNames.IsRequiredBinding)
            .Select(x => new BindingFact(
                x.Source, x.Destination,
                x.DestinationType, x.RoutingKey))
            .OrderBy(x => x.Source, StringComparer.Ordinal)
            .ThenBy(x => x.Destination,
                StringComparer.Ordinal)
            .ThenBy(x => x.RoutingKey,
                StringComparer.Ordinal)
            .ToArray();

        var policies = source.Policies
            .Where(x => x.Name == NexusLiveQueuePolicy.Name)
            .Select(x => new PolicyFact(
                x.Name, x.Pattern, x.ApplyTo, x.Priority,
                x.Definition
                .Select(p => KeyValuePair.Create(
                    p.Key, CanonicalScalar(p.Value)))
                .OrderBy(p => p.Key,
                    StringComparer.Ordinal)
                .ToArray()))
            .ToArray();

        var digest = TypedTopologyHash.Compute(
            Profile, exchanges, queues, bindings, policies);
        return new(exchanges, queues, bindings, policies,
            Profile, digest);
    }
}
```

EXECUTABLE ASSET 25-K - CANONICAL POLICY AND DIGEST COMPLETION

```
public static class TopologyManifestGate
{
    public static void AssertExact(
        NormalizedTopology expected,
        NormalizedTopology actual)
    {
        actual.Exchanges.Should().Equal(expected.Exchanges);
        actual.Queues.Should().Equal(expected.Queues);
        actual.Bindings.Should().Equal(expected.Bindings);
        actual.Policies.Should().Equal(expected.Policies);
        actual.ManifestProfile.Should().
            Be(expected.ManifestProfile);
        actual.ManifestSha256.Should().
            Be(expected.ManifestSha256);
    }
}

public sealed class TopologyManifestTests(BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_TOPO_001_management_manifest_is_exact()
    {
        var actual = await fixture.Manifests.CaptureAsync(
            fixture.CancellationTokens);
        TopologyManifestGate.AssertExact(
            ExpectedTopology.Build(), actual);
    }
}
```

CONFIRM-TRACKED PUBLICATION

A publish succeeds only after a correlated broker outcome

The .NET channel enables publisher confirmations and confirmation tracking. Mandatory publication and a persistent message profile are fixed; a return or nack becomes a typed non-success outcome.

EXECUTABLE ASSET 25-L - CONFIRMING PUBLISHER CHANNEL

```
public enum BrokerPublishStatus
{
    Confirmed,
    Unroutable,
    Nackd
}

public sealed record BrokerPublishOutcome(
    BrokerPublishStatus Status,
    string MessageId,
    ulong? PublishSequenceNumber);

public sealed record IntegrationPublication(
    string MessageId,
    string EventType,
    string RoutingKey,
    ReadOnlyMemory<byte> Body,
    DateTimeOffset OccurredAtUtc);

public sealed class ConfirmingPublisher : IAsyncDisposable
{
    private readonly IChannel _channel;
    private readonly SemaphoreSlim _serial = new(1, 1);

    private ConfirmingPublisher(IChannel channel) =>
        _channel = channel;

    public static async Task<ConfirmingPublisher> CreateAsync(
        IConnection connection,
        CancellationToken cancellationToken)
    {
        var options = new CreateChannelOptions(
            publisherConfirmationsEnabled: true,
            publisherConfirmationTrackingEnabled: true);
        var channel = await connection.CreateChannelAsync(
            options, cancellationToken);
        return new(channel);
    }

    public async Task<BrokerPublishOutcome> PublishAsync(
        IntegrationPublication message,
        CancellationToken cancellationToken)
    {
        await _serial.WaitAsync(cancellationToken);
        try
        {
            var properties = new BasicProperties
            {
                MessageId = message.MessageId,
                Type = message.EventType,
                ContentType = "application/json",
                ContentEncoding = "utf-8",
                DeliveryMode = 2,
                Timestamp = new AmqpTimestamp(
                    message.OccurredAtUtc.ToUnixTimeSeconds())
            };

            await _channel.BasicPublishAsync(
                exchange: NexusTopology.Events,
                routingKey: message.RoutingKey,
                mandatory: true,
                basicProperties: properties,
                body: message.Body,
                cancellationToken: cancellationToken);
            return new(BrokerPublishStatus.Confirmed,
                message.MessageId, null);
        }
        catch (PublishException exception)
        {
            return new(
                exception.IsReturn
                    ? BrokerPublishStatus.Unroutable
                    : BrokerPublishStatus.Nackd,
                message.MessageId,
                exception.PublishSequenceNumber);
        }
        finally
        {
            _serial.Release();
        }
    }

    public async ValueTask DisposeAsync()
    {
        _serial.Dispose();
        await _channel.DisposeAsync();
    }
}
```

EXECUTABLE ASSET 25-M - UNKNOWN ROUTE IS OBSERVABLE

```
public sealed class PublishAcceptanceTests(
    BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_RETURN_001_unknown_route_is_not_success()
    {
        var publication = fixture.Publication(
            route: "alarm-management.unknown.v1");
        var result = await fixture.Publisher.PublishAsync(
            publication, fixture.CancellationToken);

        result.Status.Should()
            .Be(BrokerPublishStatus.Unroutable);
        await fixture.Queues.AssertAllEmptyAsync();
    }
}
```

POSITIVE AND NEGATIVE ROUTING

One route produces three independent queue copies

A confirmed `AlarmFloodDetected` publication must reach `RootCause`, `Audit` and `Reporting`, but not `Compliance`. `Basic.Get` is used only as a deterministic acceptance probe; each delivery remains unacked until its bytes are copied.

EXECUTABLE ASSET 25-N - MANUAL-ACK ACCEPTANCE PROBE

```
public sealed record ProbedDelivery(
    string Queue,
    ulong DeliveryTag,
    string MessageId,
    string Exchange,
    string RoutingKey,
    bool Redelivered,
    byte[] Body);

public static class QueueProbe
{
    public static async Task<ProbedDelivery?> TryReadAsync(
        IChannel channel,
        string queue,
        CancellationToken cancellationToken)
    {
        var delivery = await channel.BasicGetAsync(
            queue, autoAck: false,
            cancellationToken: cancellationToken);
        if (delivery is null)
            return null;

        return new(
            queue,
            delivery.DeliveryTag,
            delivery.BasicProperties.MessageId,
            ?? throw new MissingMessageId(),
            delivery.Exchange,
            delivery.RoutingKey,
            delivery.Redelivered,
            delivery.Body.ToArray());
    }
}
```

EXECUTABLE ASSET 25-O - EXACT ROUTING MATRIX

```
public sealed class RoutingAcceptanceTests(
    BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_ROUTE_001_fanout_is_owned()
    {
        var message = fixture.Publication(
            "alarm-management:alarm-flood-detected.v1");
        var outcome = await fixture.Publisher.PublishAsync(
            message, fixture.CancellationToken);
        outcome.Status.Should()
            .Be(BrokerPublishStatus.Confirmed);

        var expected = new[]
        {
            "rootcause:alarm-events.v1",
            "audit:integration-events.v1",
            "reporting:integration-events.v1"
        };

        foreach (var queue in expected)
        {
            var delivery = await QueueProbe.TryReadAsync(
                fixture.Channel, queue,
                fixture.CancellationToken);
            delivery.Should().NotBeNull();
            delivery!.MessageId.Should().Be(message.MessageId);
            delivery.Body.Should().Equal(
                message.Body.ToArray());
            await fixture.Channel.BasicAckAsync(
                delivery.DeliveryTag, multiple: false,
                cancellationToken:
                    fixture.CancellationToken);
        }

        var forbidden = await QueueProbe.TryReadAsync(
            fixture.Channel,
            "compliance:root-cause-events.v1",
            fixture.CancellationToken);
        forbidden.Should().BeNull();
    }
}
```

NO TIMING SLEEP

The publisher confirm completes after routed queues accept the message. The probe therefore does not wait an arbitrary delay; absence and presence are evaluated against the completed publish outcome in the isolated vhost.

REDELIVERY AND TERMINAL ROUTING

Unacked work returns; rejected work gains x-death history

Closing a channel before acknowledgement must redeliver the same logical publication. A bounded negative acknowledgement routes a fresh message to the consumer-owned dead queue, where its most recent x-death entry is inspected.

EXECUTABLE ASSET 25-P - CHANNEL-LOSS REDELIVERY

```
public sealed class RedeliveryAcceptanceTests(
    BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_REDELIVERY_001_unacked_message_returns()
    {
        var message = fixture.Publication(
            "alarm-management.alarm-flood-detected.v1");
        (await fixture.Publisher.PublishAsync(
            message, fixture.CancellationToken))
            .Status.Should().Be(BrokerPublishStatus.Confirmed);

        await using (var first =
            await fixture.Connection.CreateChannelAsync(
                cancellationToken: fixture.CancellationToken))
        {
            var delivery = await QueueProbe.TryReadAsync(
                first, "rootcause.alarm-events.v1",
                fixture.CancellationToken);
            delivery.Should().NotBeNull();
            delivery!.Redelivered.Should().BeFalse();
            // Dispose without ack: broker requeues it.
        }

        await using var second =
            await fixture.Connection.CreateChannelAsync(
                cancellationToken: fixture.CancellationToken);
        var replay = await QueueProbe.TryReadAsync(
            second, "rootcause.alarm-events.v1",
            fixture.CancellationToken);
        replay.Should().NotBeNull();
        replay!.Redelivered.Should().BeTrue();
        replay.MessageId.Should().Be(message.MessageId);
        replay.Body.Should().Equal(message.Body.ToArray());
        await second.BasicAckAsync(
            replay.DeliveryTag, multiple: false,
            cancellationToken: fixture.CancellationToken);
    }
}
```

EXECUTABLE ASSET 25-Q - DEAD-LETTER REASON AND PROVENANCE

```
public sealed class DeadLetterAcceptanceTests(
    BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_DLX_001_rejection_reaches_owned_dead_queue()
    {
        var message = fixture.Publication(
            "alarm-management.alarm-flood-detected.v1");
        await fixture.Publisher.PublishAsync(
            message, fixture.CancellationToken);

        var live = await fixture.Eventually.ReadAsync(
            "rootcause.alarm-events.v1");
        await fixture.Channel.BasicNackAsync(
            live.DeliveryTag,
            multiple: false,
            requeue: false,
            cancellationToken: fixture.CancellationToken);

        var dead = await fixture.Eventually.ReadAsync(
            "rootcause.alarm-events.v1.dead");
        dead.MessageId.Should().Be(message.MessageId);
        var death = XDeathReader.MostRecent(
            dead.BasicProperties.Headers);
        death.Queue.Should()
            .Be("rootcause.alarm-events.v1");
        death.Reason.Should().Be("rejected");
        death.Count.Should().BeGreaterOrEqualTo(1);
        await fixture.Channel.BasicAckAsync(
            dead.DeliveryTag, multiple: false,
            cancellationToken: fixture.CancellationToken);
    }
}
```

BOUNDED POLLING IS EVIDENCE, NOT SLEEP

Eventually.ReadAsync uses a cancellation deadline, records every poll and fails with the source/dead queue counts. It never converts a timeout into an empty success or retries without a bound.

DRIFT, READINESS AND RESTART

Broken topology fails closed and is repaired deliberately

An inequivalent declaration closes only its disposable channel and leaves the existing exchange unchanged. Removing a required binding makes readiness false; a three-node restart must preserve declared objects and a confirmed persistent publication.

EXECUTABLE ASSET 25-R - INEQUIVALENT REDECLARATION AND DRIFT

```
public sealed class DriftAcceptanceTests(
    BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_DRIFT_001_drift_blocks_readiness()
    {
        var baseline = await fixture.Manifests.CaptureAsync(
            fixture.CancellationToken);

        await using var driftChannel =
            await fixture.Connection.CreateChannelAsync(
                cancellationTokens: fixture.CancellationToken);
        await Assert.ThrowsAsync<OperationInterruptedException>(
            () => driftChannel.ExchangeDeclareAsync(
                NexusTopology.Events,
                ExchangeType.Direct,
                ExchangeType.Direct,
                durable: true,
                autoDelete: false,
                arguments: null,
                noWait: false,
                cancellationTokens:
                    fixture.CancellationToken));
        driftChannel.IsOpen.Should().BeFalse();

        (await fixture.Manifests.CaptureAsync(
            fixture.CancellationToken))
            .ManifestSha256.Should()
                .Be(baseline.ManifestSha256);

        await fixture.Management.DeleteBindingAsync(
            source: NexusTopology.Events,
            destination: "rootcause.alarm-events.v1",
            route:
                "alarm-management.alarm-flood-detected.v1",
            cancellationTokens:
                fixture.CancellationToken);
        var readiness = await fixture.Readiness.EvaluateAsync(
            fixture.CancellationToken);
        readiness.IsReady.Should().BeFalse();
        readiness.MissingBindings.Should().ContainSingle();
    }
}
```

EXECUTABLE ASSET 25-S - CLUSTER RESTART PRESERVATION

```
public sealed class RestartAcceptanceTests(
    BrokerFixture fixture)
{
    [Fact]
    public async Task MQ_RESTART_001_confirmed_message_survives()
    {
        fixture.Vhost.Identity.ClusterNodeCount
            .Should().BeGreaterOrEqualTo(3);
        await fixture.Declaration.DeclareAsync(
            fixture.Channel, fixture.CancellationToken);

        var message = fixture.Publication(
            "alarm-management.alarm-flood-detected.v1");
        (await fixture.Publisher.PublishAsync(
            message, fixture.CancellationToken))
            .Status.Should().Be(BrokerPublishStatus.Confirmed);

        var before = await fixture.Manifests.CaptureAsync(
            fixture.CancellationToken);
        await fixture.ClusterFaults.RestartAllNodesAsync(
            fixture.CancellationToken);
        await fixture.ClusterFaults
            .WaitForQuorumAndManagementAsync(
                fixture.CancellationToken);

        var after = await fixture.Manifests.CaptureAsync(
            fixture.CancellationToken);
        after.ManifestSha256.Should()
            .Be(before.ManifestSha256);

        await using var recovered =
            await fixture.ConnectFreshAsync();
        var delivery = await fixture.Eventually.ReadAsync(
            recovered, "rootcause.alarm-events.v1");
        delivery.MessageId.Should().Be(message.MessageId);
        await recovered.BasicAckAsync(
            delivery.DeliveryTag, multiple: false,
            cancellationTokens: fixture.CancellationToken);
    }
}
```

DISPOSABLE-VHOST DESTRUCTION IS SCOPED

All destructive drift and restart controls target only the run's recorded vhost and selected test cluster. Production objects are never inferred from wildcards or used as fault targets.

BROKER ACCEPTANCE EVIDENCE

The chapter closes only on identified broker runs

EXECUTABLE ASSET 25-T - FAIL-CLOSED TOPOLOGY EVIDENCE GATE

```
public enum BrokerEvidenceStatus { Pending, Passed, Failed }

public sealed record BrokerEvidence(
    string EvidenceId,
    BrokerEvidenceStatus Status,
    int DiscoveredTests,
    string ServerImageDigest,
    string ServerVersion,
    string ClientPackageVersion,
    string VhostIdentitySha256,
    string ManifestSha256,
    string ResultSha256);

public static class BrokerEvidenceGate
{
    private static readonly string[] Required =
    [
        "MQ-TOP0-001", "MQ-IDEMP-001",
        "MQ-ROUTE-001", "MQ-RETURN-001",
        "MQ-REDELIVERY-001", "MQ-DLX-001",
        "MQ-RESTART-001", "MQ-DRIFT-001",
        "MQ-SEC-001", "MQ-REVIEW-001"
    ];

    public static void Accept(
        IReadOnlyCollection<BrokerEvidence> evidence)
    {
        var byId = evidence.ToDictionary(
            x => x.EvidenceId, StringComparer.Ordinal);
        var missing = Required
            .Where(id => !byId.ContainsKey(id)).ToArray();
        if (missing.Length != 0)
            throw new MissingBrokerEvidence(missing);

        foreach (var item in Required.Select(id => byId[id]))
        {
            if (item.Status != BrokerEvidenceStatus.Passed)
                throw new BrokerEvidenceNotPassed(
                    item.EvidenceId);
            if (item.DiscoveredTests <= 0)
                throw new ZeroDiscoveryBrokerEvidence(
                    item.EvidenceId);
            if (new[]
            {
                item.ServerImageDigest,
                item.ServerVersion,
                item.ClientPackageVersion,
                item.VhostIdentitySha256,
                item.ManifestSha256,
                item.ResultSha256
            }.Any(String.IsNullOrEmpty))
                throw new UnidentifiedBrokerEvidence(
                    item.EvidenceId);
        }
    }
}
```

EXECUTABLE ASSET 25-U - ACCEPTANCE EVIDENCE RECORD

```
{
  "evidenceId": "MQ-REVIEW-001",
  "chapter": 25,
  "serverImageDigest": null,
  "serverVersion": null,
  "clientPackageVersion": null,
  "clusterNodeCount": null,
  "featureFlagsSha256": null,
  "vhostIdentitySha256": null,
  "manifestProfile":
    "nexus.rabbit-topology.v1/len32-utf8+sha256",
  "manifestSha256": null,
  "tests": {
    "MQ-TOP0-001": "PENDING",
    "MQ-IDEMP-001": "PENDING",
    "MQ-ROUTE-001": "PENDING",
    "MQ-RETURN-001": "PENDING",
    "MQ-REDELIVERY-001": "PENDING",
    "MQ-DLX-001": "PENDING",
    "MQ-RESTART-001": "PENDING",
    "MQ-DRIFT-001": "PENDING",
    "MQ-SEC-001": "PENDING"
  },
  "discoveredTests": null,
  "accepted": false
}
```

Chapter 25 code-audit result

The chapter now exposes one broker-backed route from isolated-vhost identity and least privilege through policy prerequisites, exact management manifest, confirm-tracked publication, routing, redelivery, terminal transfer, drift, restart and fail-closed evidence.

CURRENT VERDICT

The RabbitMQ boundary is implementable in shape, but no cluster-backed result has executed. Topology is not yet broker-verified, and outbox/inbox correctness remains outside this chapter. Every MQ result is explicitly PENDING.

25.12 Decision record

ADR-025 — NEXUS-1 RABBITMQ TOPOLOGY

Field	Decision
Status	Proposed canonical decision; executable declarations supplied; broker evidence pending.
Context	Multiple autonomous consumers require independent progress, durable intent, visible failure, and contract-owned routing without shared databases.
Decision	Use durable topic exchanges <code>nexus.events/nexus.dead</code> ; consumer-owned durable quorum queues; producer-owned versioned routing keys; manual acknowledgements; policy-managed DLX/safety settings; one explicit dead queue per live queue.
Consequences	More queues and evidence, but no competing-consumer theft across capabilities. Duplicate delivery remains possible and is handled by inbox/idempotency. Topology changes become reviewed migrations.
Rejected	One shared integration queue; fanout-only exchange; auto-ack; hard-coded DLX settings; routing by entity identifier; claims of exactly once.

25.13 Official-source validation

The topology and API terminology were checked against current RabbitMQ primary documentation on 25 July 2026. RabbitMQ distinguishes durable queue metadata from persistent messages, treats consumer acknowledgements and publisher confirms as orthogonal mechanisms, recommends policies instead of hard-coded dead-letter `x-arguments`, documents the quorum-queue requirements and caveats for at-least-once dead lettering, and exposes asynchronous `.NET` declarations on `IChannel`.

Sources: <https://www.rabbitmq.com/docs/queues> ; <https://www.rabbitmq.com/docs/confirmations> ;
<https://www.rabbitmq.com/docs/dlx> ; <https://www.rabbitmq.com/docs/quorum-queues> ;
<https://www.rabbitmq.com/docs/reliability> ;
<https://rabbitmq.github.io/rabbitmq-dotnet-client/api/RabbitMQ.Client.IChannel.html>

Technical material still required

The chapter now contains the required diagram and executable declaration shape, but it is not falsely marked as broker-proved. The repository still needs the pinned `RabbitMQ.Client` project, disposable-vhost fixture, RabbitMQ version/feature-flag record, policy application, management-API manifest comparer, routing test matrix, mandatory-return test, quorum interruption, restart recovery, dead-letter safety test, and retained evidence artifacts. Those items are release gates for ★ completion.

Implementation sequence

1. Pin the RabbitMQ server/container image and `RabbitMQ.Client` package; record supported feature flags.
2. Create a disposable NEXUS-1 vhost and least-privilege topology principal.
3. Apply the reviewed quorum/DLX policy and capture its effective definition.
4. Run `DeclareAsync` twice and compare the normalized topology manifest.
5. Execute positive, negative, mandatory-return, redelivery, terminal-routing, quorum-loss, and restart cases.
6. Publish MQ-TOPO-001 through MQ-DRIFT-001 with immutable hashes and environment identity.
7. Only then advance the chapter's claim from specified to broker-verified.

What this chapter has proved

Chapter 25 has established the canonical NEXUS-1 RabbitMQ topology and the ownership logic behind it. It has separated producer routes from consumer queues, made durability and acknowledgement boundaries explicit, defined terminal routes without an automatic replay cycle, and supplied code-level declarations, policies, passive checks, a normalized manifest, and broker acceptance cases.

It has not proved that a committed local transaction survives broker outage, that a publication was confirmed, that consumer effects occur once, or that every dead-letter transfer is lossless. Those claims belong to Chapters 26–29 and require executable evidence.

Chapter checkpoint

Question	Answer
Does the chapter match the official outline?	Yes: exchanges, queues, routing keys, durability, acknowledgements, dead-letter routes, diagram, and executable declarations are present.
Is the topology executable rather than prose-only?	Yes at declaration/policy/manifest level; live broker execution evidence remains explicitly pending.
Are guarantees stated honestly?	Yes. Durable topology, persistence, confirms, acknowledgements, local transactions, and idempotency are not conflated.
Does one consumer affect another's progress?	No. RootCause, Audit, Compliance, and Reporting own independent queues.
May this ★ chapter be called fully verified?	Not yet. The broker-backed evidence catalogue must be produced.

NEXT CHAPTER

Chapter 26, The Transactional Outbox in Each Producer, will place integration-event intent in the same local database transaction as the producer's state change and will prove that broker unavailability cannot erase an already committed event.

PART V - THE MESSAGE BACKBONE

26. The Transactional Outbox in Each Producer

A committed owner fact must not disappear because RabbitMQ is unavailable, and a public event must not exist when the owner transaction rolls back. The outbox makes those two records one local database decision.

CHAPTER DECISION

Every producer stores its owned state change and an immutable serialized public fact in the same service-owned database transaction. Publishing is a later, repeatable activity. No broker call and no second service database participates in the commit.

This chapter is marked ★ because prose is insufficient. Its acceptance surface includes executable C#/EF Core/SQL assets, provider-backed atomicity and rollback tests, deterministic crash/race probes, a fail-closed evidence manifest and ADR-024.

26.1 The outbox solves one exact problem

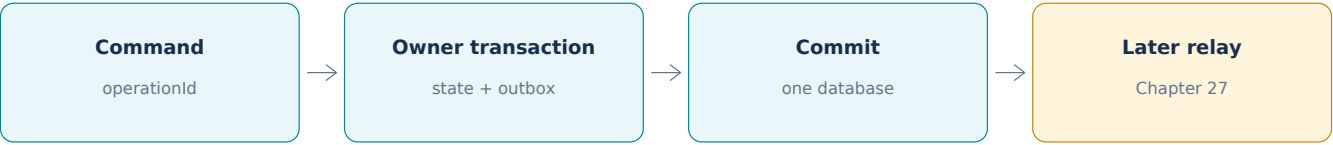
The dangerous implementation is a dual write: commit AlarmManagement truth and then publish AlarmFloodDetected.v1, or publish first and then commit. Either order contains a crash window that can separate the public story from the owner's truth. The outbox does not make the whole distributed flow atomic. It makes one producer's decision and its durable publication intent atomic.

Boundary	Mechanism in Chapter 26	Allowed claim	Not yet allowed
Owner state -> outbox	One local database commit	Both persist or both roll back	The broker has accepted the event
Outbox -> broker	Immutable pending row	Delivery intent survives broker outage	Confirmed, ordered or duplicate-free delivery
Broker -> consumer	Stable message identity	A consumer can deduplicate later	The consumer effect is atomic

THE DUAL-WRITE BOUNDARY

The broker cannot vote on domain truth

The only synchronous durability dependency is the producer's own database. Everything after commit is delayed work with an explicit evidence state.



The broker is deliberately outside the local transaction.

Four outcomes expose the real contract

Observed path	Durable owner state	Outbox	Engineering response
Validation or serialization fails before SaveChanges	Unchanged	Absent	Reject; do not retry as infrastructure failure
Database transaction rolls back	Unchanged	Absent	Retry only under the owning command policy
Commit succeeds; broker is down	Changed	Pending	Return owner outcome; relay catches up later
Commit response is lost	Unknown to caller	Unknown to caller	Resolve by stable operation/fact identity
Relay publishes; mark result is ambiguous	Changed	Still dispatchable	Replay same messageId; Chapter 27 owns proof

NOT A DISTRIBUTED TRANSACTION

No two-phase commit is introduced. RootCause, Audit, Compliance and Reporting remain independent owners. Their progress is eventual and must be made idempotent at each local boundary.

PRODUCER-OWNED DURABLE INTENT

An outbox row is a frozen public fact, not a domain object

The row contains the exact bytes and metadata that a later relay may publish. It never asks the aggregate to re-create history from newer state.

EXECUTABLE ASSET 26-A - IMMUTABLE OUTBOX VALUE

```
public enum OutboxState : byte
{
    Pending = 0,
    InFlight = 1,
    Retryable = 2,
    Dispatched = 3,
    Quarantined = 4
}

public sealed record SerializedPublicFact(
    Guid MessageId,
    string FactKey,
    string Producer,
    string EventType,
    int SchemaVersion,
    string RoutingKey,
    string ContentType,
    DateTimeOffset OccurredAtUtc,
    byte[] EnvelopeBytes,
    byte[] EnvelopeSha256);

public sealed record OutboxHeaders(
    string CorrelationId,
    string CausationId,
    string? TraceParent,
    string? TraceState);
```

EXECUTABLE ASSET 26-B - PENDING ROW CONSTRUCTOR

```
public sealed class OutboxMessageRow
{
    private OutboxMessageRow() { }

    public Guid MessageId { get; private set; }
    public string FactKey { get; private set; } = null!;
    public string Producer { get; private set; } = null!;
    public string EventType { get; private set; } = null!;
    public int SchemaVersion { get; private set; }
    public string RoutingKey { get; private set; } = null!;
    public string ContentType { get; private set; } = null!;
    public DateTimeOffset OccurredAtUtc { get; private set; }
    public DateTimeOffset StoredAtUtc { get; private set; }
    public byte[] EnvelopeBytes { get; private set; } = null!;
    public byte[] EnvelopeSha256 { get; private set; } = null!;
    public string HeadersJson { get; private set; } = null!;
    public OutboxState State { get; private set; }
    public int AttemptCount { get; private set; }
    public DateTimeOffset? NextAttemptAtUtc { get; private set; }
    public string? LockedBy { get; private set; }
    public DateTimeOffset? LockExpiresAtUtc { get; private set; }
    public DateTimeOffset? DispatchedAtUtc { get; private set; }
    public byte[] Version { get; private set; } = null!;

    public static OutboxMessageRow Pending(
        SerializedPublicFact fact,
        string headersJson,
        DateTimeOffset storedAtUtc) => new()
    {
        MessageId = fact.MessageId,
        FactKey = fact.FactKey,
        Producer = fact.Producer,
        EventType = fact.EventType,
        SchemaVersion = fact.SchemaVersion,
        RoutingKey = fact.RoutingKey,
        ContentType = fact.ContentType,
        OccurredAtUtc = fact.OccurredAtUtc,
        StoredAtUtc = storedAtUtc,
        EnvelopeBytes = fact.EnvelopeBytes.ToArray(),
        EnvelopeSha256 = fact.EnvelopeSha256.ToArray(),
        HeadersJson = headersJson,
        State = OutboxState.Pending
    };
}
```


FREEZE BEFORE COMMIT

Serialization belongs before the transaction is accepted

Chapter 20 owns the envelope contract. Chapter 26 consumes that contract and stores its exact UTF-8 bytes, digest and routing metadata before SaveChanges.

EXECUTABLE ASSET 26-C - PRODUCER SERIALIZER PORT

```
public interface IPublicFactSerializer
{
    SerializedPublicFact Serialize<TPayload>(
        PublicFactDescriptor descriptor,
        TPayload payload);
}

public sealed record PublicFactDescriptor(
    Guid MessageId,
    string FactKey,
    string Producer,
    string EventType,
    int SchemaVersion,
    string RoutingKey,
    DateTimeOffset OccurredAtUtc,
    OutboxHeaders Headers);

public sealed class PublicFactSerializer(
    ICanonicalEnvelopeWriter envelopes) : IPublicFactSerializer
{
    public SerializedPublicFact Serialize<TPayload>(
        PublicFactDescriptor descriptor,
        TPayload payload)
    {
        var bytes = envelopes.WriteV1(descriptor, payload);
        if (bytes.Length is 0 or > 262_144)
            throw new PublicFactSizeRejected(bytes.Length);

        return new(
            descriptor.MessageId,
            descriptor.FactKey,
            descriptor.Producer,
            descriptor.EventType,
            descriptor.SchemaVersion,
            descriptor.RoutingKey,
            "application/json",
            descriptor.OccurredAtUtc,
            bytes,
            SHA256.HashData(bytes));
    }
}
```

Why the dispatcher must not serialize again

- New serializer settings cannot silently change an already committed public fact.
- A later aggregate edit cannot rewrite the event that represented an earlier transition.
- The persisted SHA-256 digest gives tests and operational evidence one exact byte identity.
- Replay uses the same messageId, routing key, content type and bytes; only transport attempt metadata changes.

FAILURE IS LOCAL AND EARLY

Unsupported payload, invalid header, oversize envelope or non-canonical serialization fails before owner state is committed. The system never stores a fact that the relay cannot represent.

EF CORE PERSISTENCE

The mapping encodes the outbox invariants

The outbox is infrastructure-owned persistence inside the producer database. Application code sees a staging port, not mutable dispatch columns.

EXECUTABLE ASSET 26-D - SQL SERVER EF CORE MAPPING

```
public sealed class OutboxMessageConfiguration
    : IEntityTypeConfiguration<OutboxMessageRow>
{
    public void Configure(
        EntityTypeBuilder<OutboxMessageRow> builder)
    {
        builder.ToTable("OutboxMessage", "messaging",
            table =>
            {
                table.HasCheckConstraint(
                    "CK_Outbox_State",
                    "[State] BETWEEN 0 AND 4");
                table.HasCheckConstraint(
                    "CK_Outbox_AttemptCount",
                    "[AttemptCount] >= 0");
                table.HasCheckConstraint(
                    "CK_Outbox_SchemaVersion",
                    "[SchemaVersion] >= 1");
                table.HasCheckConstraint(
                    "CK_Outbox_EnvelopeSize",
                    "DATALENGTH([EnvelopeBytes]) BETWEEN 1 AND 262144");
                table.HasCheckConstraint(
                    "CK_Outbox_HeadersJson",
                    "ISJSON([HeadersJson]) = 1");
            });

        builder.HasKey(x => x.MessageId);
        builder.Property(x => x.FactKey)
            .HasMaxLength(96).IsUnicode(false).IsRequired();
        builder.HasIndex(x => x.FactKey).IsUnique();

        builder.Property(x => x.Producer)
            .HasMaxLength(80).IsUnicode(false).IsRequired();
        builder.Property(x => x.EventType)
            .HasMaxLength(160).IsUnicode(false).IsRequired();
        builder.Property(x => x.RoutingKey)
            .HasMaxLength(200).IsUnicode(false).IsRequired();
        builder.Property(x => x.ContentType)
            .HasMaxLength(80).IsUnicode(false).IsRequired();
        builder.Property(x => x.EnvelopeBytes)
            .HasColumnType("varbinary(max)").IsRequired();
        builder.Property(x => x.EnvelopeSha256)
            .HasColumnType("binary(32)").IsRequired();
        builder.Property(x => x.HeadersJson)
            .HasMaxLength(2048).IsRequired();
        builder.Property(x => x.Version)
            .IsRowVersion().IsConcurrencyToken();

        builder.HasIndex(x => new
            {
                x.State,
                x.NextAttemptAtUtc,
                x.StoredAtUtc,
                x.MessageId
            })
            .HasDatabaseName("IX_Outbox_Dispatchable")
            .HasFilter("[State] IN (0, 2)");
    }
}
```

PROVIDER DDL

Every producer owns the same contract, not the same database

AlarmManagementDb, RootCauseDb, AuditDb, ComplianceDb and ReportingDb each apply their own migration. There is no central NEXUS-1 outbox database.

EXECUTABLE ASSET 26-E - SQL SERVER TABLE AND DISPATCH INDEX

```
CREATE SCHEMA messaging AUTHORIZATION dbo;
GO

CREATE TABLE messaging.OutboxMessage
(
    MessageId            uniqueidentifier NOT NULL,
    FactKey              varchar(96)      NOT NULL,
    Producer             varchar(80)      NOT NULL,
    EventType            varchar(160)     NOT NULL,
    SchemaVersion        int              NOT NULL,
    RoutingKey           varchar(200)     NOT NULL,
    ContentType          varchar(80)      NOT NULL,
    OccurredAtUtc        datetimeoffset(7) NOT NULL,
    StoredAtUtc          datetimeoffset(7) NOT NULL,
    EnvelopeBytes        varbinary(max)   NOT NULL,
    EnvelopeSha256       binary(32)       NOT NULL,
    HeadersJson          nvarchar(2048)   NOT NULL,
    State                tinyint          NOT NULL,
    CONSTRAINT DF_Outbox_State DEFAULT (0),
    AttemptCount         int              NOT NULL,
    CONSTRAINT DF_Outbox_Attempts DEFAULT (0),
    NextAttemptAtUtc     datetimeoffset(7) NULL,
    LockedBy             varchar(120)     NULL,
    LockExpiresAtUtc     datetimeoffset(7) NULL,
    DispatchedAtUtc      datetimeoffset(7) NULL,
    Version              rowversion       NOT NULL,

    CONSTRAINT PK_Outbox PRIMARY KEY (MessageId),
    CONSTRAINT UX_Outbox_FactKey UNIQUE (FactKey),
    CONSTRAINT CK_Outbox_State CHECK (State BETWEEN 0 AND 4),
    CONSTRAINT CK_Outbox_Attempts CHECK (AttemptCount >= 0),
    CONSTRAINT CK_Outbox_SchemaVersion CHECK (SchemaVersion >= 1),
    CONSTRAINT CK_Outbox_Size CHECK
        (DATALENGTH(EnvelopeBytes) BETWEEN 1 AND 262144),
    CONSTRAINT CK_Outbox_Headers CHECK (ISJSON(HeadersJson) = 1)
);

CREATE INDEX IX_Outbox_Dispatchable
ON messaging.OutboxMessage
(State, NextAttemptAtUtc, StoredAtUtc, MessageId)
INCLUDE (LockExpiresAtUtc)
WHERE State IN (0, 2);
```

SCHEMA BOUNDARY

Lease and result columns reserve a safe handoff to Chapter 27. Chapter 26 does not define the claiming SQL, publisher confirm state machine or recovery loop.

APPLICATION INTENT

The handler stages owner truth and public intent together

The Application layer chooses the public fact from a completed domain transition. Infrastructure realizes both records through one DbContext.

EXECUTABLE ASSET 26-F - APPLICATION-OWNED STAGING PORT

```
public interface IProducerCommit
{
    Task<ProducerCommitOutcome> CommitAsync(
        OwnedStateChange change,
        SerializedPublicFact publicFact,
        OutboxHeaders headers,
        CancellationToken cancellationToken);
}

public abstract record ProducerCommitOutcome
{
    public sealed record Committed(
        Guid MessageId, string FactKey) : ProducerCommitOutcome;
    public sealed record AlreadyCommitted(
        Guid MessageId, string FactKey) : ProducerCommitOutcome;
    public sealed record Rejected(string Code) : ProducerCommitOutcome;
    public sealed record Unknown(string FactKey) : ProducerCommitOutcome;
}
```

EXECUTABLE ASSET 26-G - ALARM FLOOD COMMAND PATH

```
public sealed class ConfirmAlarmFloodHandler(
    IAlarmFloodRepository floods,
    IPublicFactSerializer serializer,
    IProducerCommit commit,
    IClock clock)
{
    public async Task<ProducerCommitOutcome> Handle(
        ConfirmAlarmFlood command,
        CancellationToken cancellationToken)
    {
        var flood = await floods.LoadAsync(
            command.FloodId, cancellationToken);

        var transition = flood.Confirm(
            command.OperationId,
            command.ConfirmedBy,
            clock.UtcNow);

        var descriptor = AlarmFloodDetectedV1.Describe(
            transition, command.Correlation);
        var fact = serializer.Serialize(
            descriptor,
            AlarmFloodDetectedV1.From(transition));

        return await commit.CommitAsync(
            OwnedStateChange.For(flood),
            fact,
            descriptor.Headers,
            cancellationToken);
    }
}
```

NO PUBLISH CALL

The command can succeed while RabbitMQ is unavailable because acceptance depends only on AlarmManagementDb. The response reports the owner commit, never downstream completion.

INFRASTRUCTURE REALIZATION

One SaveChanges call is the atomic boundary

EF Core wraps one SaveChanges call in a transaction when the provider supports transactions. The producer therefore stages both tracked records and saves once.

EXECUTABLE ASSET 26-H - EXPLICIT PRODUCER COMMIT

```
public sealed class EfProducerCommit<TDbContext>(  
    TDbContext db,  
    IOutboxHeaderWriter headerWriter,  
    IProviderErrorClassifier errors,  
    IClock clock)  
    : IProducerCommit  
    where TDbContext : DbContext, IProducerDbContext  
{  
    public async Task<ProducerCommitOutcome> CommitAsync(  
        OwnedStateChange change,  
        SerializedPublicFact fact,  
        OutboxHeaders headers,  
        CancellationToken cancellationToken)  
    {  
        change.Apply(db);  
        db.OutboxMessages.Add(  
            OutboxMessageRow.Pending(  
                fact,  
                headerWriter.Write(headers),  
                clock.UtcNow));  
  
        try  
        {  
            await db.SaveChangesAsync(cancellationToken);  
            return new ProducerCommitOutcome.Committed(  
                fact.MessageId, fact.FactKey);  
        }  
        catch (DbUpdateException exception)  
            when (errors.IsUniqueFactKey(exception))  
        {  
            db.ChangeTracker.Clear();  
            var existing = await db.OutboxMessages  
                .AsNoTracking()  
                .SingleAsync(  
                    x => x.FactKey == fact.FactKey,  
                    cancellationToken);  
            return new ProducerCommitOutcome.AlreadyCommitted(  
                existing.MessageId, existing.FactKey);  
        }  
        catch (Exception exception)  
            when (errors.CommitMaybeUnknown(exception))  
        {  
            return new ProducerCommitOutcome.Unknown(  
                fact.FactKey);  
        }  
    }  
}
```

A multi-SaveChanges implementation may use an explicit transaction and EF Core savepoints, but this reference path deliberately avoids the extra state space. A transient retry must create a new DbContext and re-run the semantic operation; it must never reuse a poisoned tracker after a failed or unknown commit.

INTERCEPTOR POLICY

A SaveChanges interceptor may collect already-raised events, but it must not invent public semantics, call the broker or hide serialization failure. The reference design keeps staging explicit and testable.

EACH PRODUCER, ONE OWNED TRANSACTION

The pattern repeats without creating shared truth

A common package may provide mapping helpers and value types. Every service still owns its outbox schema, migration, retention and public fact decisions.

Producer	Owned change committed locally	Pending public fact
AlarmManagement	Flood status and accepted operation	AlarmFloodDetected.v1
RootCause	Case/evidence/verdict transition	RootCauseVerdictIssued.v1
Audit	Immutable evidence receipt	AuditEvidenceRecorded.v1 when approved
Compliance	Review state and decision	ComplianceReviewCompleted.v1
Reporting	Projection checkpoint/version	ReportProjectionAdvanced.v1 only if needed

EXECUTABLE ASSET 26-I - ROOTCAUSE PRODUCER PARITY

```
public async Task<ProducerCommitOutcome> IssueVerdictAsync(
    IssueVerdict command,
    CancellationToken cancellationToken)
{
    var rootCauseCase = await cases.LoadAsync(
        command.CaseId, cancellationToken);
    var transition = rootCauseCase.IssueVerdict(
        command.OperationId,
        command.Verdict,
        command.EvidenceIds,
        clock.UtcNow);

    var descriptor = RootCauseVerdictIssuedV1.Describe(
        transition, command.Correlation);
    var fact = serializer.Serialize(
        descriptor,
        RootCauseVerdictIssuedV1.From(transition));

    return await commit.CommitAsync(
        OwnedStateChange.For(rootCauseCase),
        fact,
        descriptor.Headers,
        cancellationToken);
}
```

SHARED MECHANISM, OWNED MEANING

A reusable outbox library may persist and expose rows. It may not choose that a RootCause verdict is public, enrich AlarmManagement payloads or normalize away service-specific semantics.

SEMANTIC IDENTITY

Retries reuse a fact key even when execution is repeated

The operation identity comes from the admitted command. The fact key adds the event family and ordinal so one command may emit a deliberate finite set.

EXECUTABLE ASSET 26-J - LENGTH-PREFIXED FACT IDENTITY

```
public static class PublicFactIdentity
{
    public static string Create(
        string producer,
        Guid operationId,
        string eventType,
        int ordinal)
    {
        string[] fields =
        [
            "nexus-public-fact-v1",
            producer,
            operationId.ToString("D"),
            eventType,
            ordinal.ToString(
                CultureInfo.InvariantCulture)
        ];

        var canonical = string.Concat(fields.Select(value =>
            $"{Encoding.UTF8.GetByteCount(value)}:{value}"));
        return Convert.ToHexString(
            SHA256.HashData(
                Encoding.UTF8.GetBytes(canonical)));
    }
}
```

EXECUTABLE ASSET 26-K - STABLE DESCRIPTOR CONSTRUCTION

```
public static class AlarmFloodDetectedV1
{
    public static PublicFactDescriptor Describe(
        AlarmFloodConfirmed transition,
        CorrelationContext correlation)
    {
        var factKey = PublicFactIdentity.Create(
            "alarm-management",
            transition.OperationId,
            "AlarmFloodDetected.v1",
            ordinal: 0);

        return new(
            transition.MessageId,
            factKey,
            "alarm-management",
            "AlarmFloodDetected.v1",
            1,
            "alarm-management.alarm-flood-detected.v1",
            transition.OccurredAtUtc,
            new(
                correlation.CorrelationId,
                correlation.CausationId,
                correlation.TraceParent,
                correlation.TraceState));
    }
}
```

IDENTITY RULE

FactKey prevents duplicate semantic intent. MessageId identifies the public fact on the wire. Both remain stable after a successful or ambiguously observed commit; delivery attempts never allocate a new identity.

AMBIGUOUS DATABASE OUTCOME

Unknown is a first-class result, not permission to write again

A connection can fail after SQL Server commits but before the client receives confirmation. The next action is a fresh read by FactKey.

EXECUTABLE ASSET 26-L - UNKNOWN-COMMIT RESOLVER

```
public sealed class UnknownProducerCommitResolver<TDbContext>(  
    IDbContextFactory<TDbContext> contexts)  
    where TDbContext : DbContext, IProducerDbContext  
{  
    public async Task<ProducerCommitOutcome> ResolveAsync(  
        string factKey,  
        CancellationToken cancellationToken)  
    {  
        await using var db =  
            await contexts.CreateDbContextAsync(  
                cancellationToken);  
  
        var durable = await db.OutboxMessages  
            .AsNoTracking()  
            .Where(x => x.FactKey == factKey)  
            .Select(x => new  
            {  
                x.MessageId,  
                x.FactKey  
            })  
            .SingleOrDefaultAsync(cancellationToken);  
  
        return durable is null  
            ? new ProducerCommitOutcome.Unknown(factKey)  
            : new ProducerCommitOutcome.AlreadyCommitted(  
                durable.MessageId, durable.FactKey);  
    }  
}
```

Resolution policy

Fresh read	Meaning	Allowed next action
FactKey and owner outcome exist	The original transaction committed	Return/replay the stored owner result; do not create a new fact
Neither record exists after bounded reconciliation	No durable commit was found	Re-run the semantic command under the same operationId
Owner state exists but outbox is absent	Atomicity invariant violated	Stop dispatch, alert and preserve evidence
Outbox exists but owner state is absent	Atomicity invariant violated	Quarantine; never publish

CANCELLATION BOUNDARY

Cancellation before SaveChanges begins is cancellation. Cancellation or network loss during commit may be unknown. The API and logs must not label an unknown outcome as rolled back.

CONCURRENCY AND PROVIDER SEMANTICS

Retry the meaning with a new unit of work

A rowversion conflict means the aggregate changed after it was read. The handler must reload and decide whether the command is still valid, already satisfied or rejected.

EXECUTABLE ASSET 26-M - BOUNDED SEMANTIC RETRY

```
public sealed class ProducerOperationRunner(
    IProducerOperationAttemptFactory attempts,
    IUnknownCommitResolver unknowns)
{
    public async Task<ProducerCommitOutcome> ExecuteAsync(
        ProducerOperation operation,
        CancellationToken cancellationToken)
    {
        for (var attempt = 1; attempt <= 3; attempt++)
        {
            await using var unit =
                await attempts.StartAsync(cancellationToken);
            var outcome = await unit.TryAsync(
                operation, cancellationToken);

            switch (outcome)
            {
                case ProducerCommitOutcome.Committed:
                case ProducerCommitOutcome.AlreadyCommitted:
                case ProducerCommitOutcome.Rejected:
                    return outcome;

                case ProducerCommitOutcome.Unknown unknown:
                    var resolved = await unknowns.ResolveAsync(
                        unknown.FactKey, cancellationToken);
                    if (resolved is not
                        ProducerCommitOutcome.Unknown)
                        return resolved;
                    break;
            }
        }

        return new ProducerCommitOutcome.Unknown(
            operation.FactKey);
    }
}
```

Failure	Classifier	Response
Optimistic concurrency	DbUpdateConcurrencyException	New DbContext; reload aggregate; re-evaluate invariant
Duplicate FactKey	Provider unique constraint	Read durable fact and return AlreadyCommitted
Transient before commit	Pinned provider classifier	Bounded new attempt under same operationId
Connection loss during commit	CommitMayBeUnknown	Fresh read and reconciliation; never blind retry
Serialization or domain rejection	Application/domain type	No infrastructure retry

NO PROVIDER GUESSING

SQL error numbers and retry categories are pinned to the selected provider/version and verified by component tests. String matching on exception messages is rejected.

PROVIDER-BACKED ATOMICITY PROOF

Success and rollback are counted in the real database

The acceptance lane applies the production migration to a disposable SQL Server database. SQLite and EF's in-memory provider cannot prove the selected constraints.

EXECUTABLE ASSET 26-N - ATOMIC SUCCESS

```
public sealed class OutboxAtomicityTests(
    SqlServerProducerFixture fixture)
{
    [Fact]
    public async Task OUTBOX_ATOMIC_001_state_and_fact_commit_once()
    {
        var operation = fixture.ConfirmFlood(
            operationId:
                Guid.Parse("3fb96970-85a1-4d73-91e2-4f39cd848b46"));

        var outcome = await fixture.ExecuteAsync(operation);

        outcome.Should().BeOfType<
            ProducerCommitOutcome.Committed>();
        await fixture.Database.AssertFloodStateAsync(
            operation.FloodId, AlarmFloodState.Confirmed);
        await fixture.Database.AssertOutboxCountAsync(
            operation.FactKey, expected: 1);
        await fixture.Database.AssertEnvelopeDigestAsync(
            operation.FactKey);
        fixture.BrokerConnections.Should().Be(0);
    }
}
```

EXECUTABLE ASSET 26-O - ATOMIC ROLLBACK

```
public sealed partial class OutboxAtomicityTests
{
    [Fact]
    public async Task OUTBOX_ROLLBACK_001_both_records_roll_back()
    {
        fixture.Faults.FailCommand(
            "INSERT INTO [messaging].[OutboxMessage]");
        var operation = fixture.ConfirmFlood();

        await Assert.ThrowsAsync<InjectedDatabaseFailure>(
            () => fixture.ExecuteAsync(operation));

        await fixture.Database.AssertFloodStateAsync(
            operation.FloodId, AlarmFloodState.Candidate);
        await fixture.Database.AssertOutboxCountAsync(
            operation.FactKey, expected: 0);
    }
}
```

ACCEPTANCE ORACLE

The test passes only when the owner row and outbox row agree. An HTTP 202, a log line or a mocked repository call is not atomicity evidence.

CRASH AND RACE PROOF

Dangerous timing windows are forced deterministically

The fixture uses provider commands and barriers, not sleeps. It can commit and drop the client response, or hold two operations immediately before the unique write.

EXECUTABLE ASSET 26-P - LOST COMMIT RESPONSE

```
[Fact]
public async Task OUTBOX_UNKNOWN_001_commit_is_reconciled()
{
    var operation = fixture.ConfirmFlood();
    fixture.Faults.DropCommitResponseAfterServerCommit();

    var first = await fixture.ExecuteAsync(operation);
    first.Should().BeOfType<
        ProducerCommitOutcome.Unknown>();

    var resolved = await fixture.ResolveAsync(
        operation.FactKey);
    resolved.Should().BeOfType<
        ProducerCommitOutcome.AlreadyCommitted>();

    await fixture.Database.AssertFloodStateAsync(
        operation.FloodId, AlarmFloodState.Confirmed);
    await fixture.Database.AssertOutboxCountAsync(
        operation.FactKey, expected: 1);
}
```

EXECUTABLE ASSET 26-Q - SAME SEMANTIC FACT UNDER RACE

```
[Fact]
public async Task OUTBOX_RACE_001_one_fact_survives()
{
    var operation = fixture.ConfirmFlood();
    await using var race = fixture.Faults
        .BarrierBeforeFactKeyInsert(participants: 2);

    var outcomes = await Task.WhenAll(
        fixture.ExecuteFreshAsync(operation),
        fixture.ExecuteFreshAsync(operation));

    outcomes.Count(x => x is
        ProducerCommitOutcome.Committed).Should().Be(1);
    outcomes.Count(x => x is
        ProducerCommitOutcome.AlreadyCommitted).Should().Be(1);
    await fixture.Database.AssertOutboxCountAsync(
        operation.FactKey, expected: 1);
    await fixture.Database.AssertSingleTransitionAsync(
        operation.FloodId, operation.OperationId);
}
```

PROCESS DEATH CHECKPOINT

A separate deployable test kills the producer after database commit and before any relay starts. Restart must expose one Pending row with the original bytes. Chapter 27 then proves its recovery to the broker.

HANDOFF TO CHAPTER 27

The committed row is the relay's complete input

The producer transaction ends at Pending. A later worker may claim, publish and record a result, but it may never edit the public identity or bytes.

Field group	Mutable after commit?	Owner
MessageId, FactKey, producer, event type/version	No	Producer application contract
Routing key, content type, envelope bytes, digest	No	Producer serializer
OccurredAtUtc, correlation and causation	No	Accepted domain transition
State, attempt count, next attempt, lease	Yes, under concurrency control	Chapter 27 relay
DispatchedAtUtc and confirm evidence	Yes, append/transition only	Chapter 27 relay

EXECUTABLE ASSET 26-R - NARROW DISPATCHER PROJECTION

```
public sealed record DispatchableOutboxMessage(
    Guid MessageId,
    string FactKey,
    string EventType,
    string RoutingKey,
    string ContentType,
    DateTimeOffset OccurredAtUtc,
    ReadOnlyMemory<byte> EnvelopeBytes,
    ReadOnlyMemory<byte> EnvelopeSha256,
    byte[] Version);

public interface IOutboxDispatchStore
{
    // Chapter 27 supplies claim, confirm and recovery semantics.
    Task<IReadOnlyList<DispatchableOutboxMessage>>
        ClaimBatchAsync(
            DispatchLease lease,
            int batchSize,
            CancellationToken cancellationToken);
}
```

EXPLICIT NON-GOALS

This chapter does not claim publish ordering, exactly-once delivery, broker acceptance, subscriber completion, dead-letter safety or global consistency. The outbox is durable intent, not proof of delivery.

OPERATIONAL AND SECURITY CONTRACT

The table is durable evidence and a bounded liability

Outbox rows need retention, access control, payload limits and safe telemetry. Durability is not permission to retain sensitive data forever.

Concern	Required control	Evidence
Database authority	Runtime principal inserts/reads only its producer outbox; migration principal owns DDL	Permission test and denied cross-service access
Payload content	Producer-approved contract, 256 KiB cap, no secrets or internal entity graph	Schema corpus and sensitive-token scan
Telemetry	Log opaque IDs, state and age; never payload or headers	Captured log allowlist
Retention	Delete only terminal rows after configured evidence window	Clock-controlled retention test
Backlog	Measure pending count/age/bytes per producer	OUTBOX-OPS-001 query snapshot
Restore	Backup owner state and outbox in one database restore unit	Restore count and digest reconciliation

EXECUTABLE ASSET 26-S - VALUE-FREE HEALTH SNAPSHOT

```
public sealed record OutboxHealthSnapshot(  
    string Producer,  
    long PendingCount,  
    long PendingBytes,  
    TimeSpan OldestPendingAge,  
    long RetryableCount,  
    long QuarantinedCount,  
    DateTimeOffset ObservedAtUtc);  
  
// Safe logs include producer, counts, ages and opaque IDs.  
// They exclude EnvelopeBytes, HeadersJson, tokens and domain values.
```

READINESS IS NOT BACKLOG ZERO

A temporary broker outage should grow durable Pending work without making the producer unable to commit owner truth. Readiness fails only when the producer cannot safely accept more work under its declared storage/backpressure policy.

EVIDENCE AND DECISION RECORD

The chapter closes only on identified provider runs

EXECUTABLE ASSET 26-T - FAIL-CLOSED EVIDENCE MANIFEST

```
{
  "evidenceId": "OUTBOX-REVIEW-001",
  "chapter": 26,
  "targetFramework": "net8.0",
  "provider": null,
  "databaseImageDigest": null,
  "migrationSha256": null,
  "serializerProfileSha256": null,
  "tests": {
    "OUTBOX-SCHEMA-001": "PENDING",
    "OUTBOX-ATOMIC-001": "PENDING",
    "OUTBOX-ROLLBACK-001": "PENDING",
    "OUTBOX-UNKNOWN-001": "PENDING",
    "OUTBOX-RACE-001": "PENDING",
    "OUTBOX-NOBROKER-001": "PENDING",
    "OUTBOX-RESTORE-001": "PENDING",
    "OUTBOX-SEC-001": "PENDING",
    "OUTBOX-OPS-001": "PENDING"
  },
  "discoveredTests": null,
  "resultsSha256": null,
  "accepted": false
}
```

ADR-024 - Local truth and public intent

ADR field	ADR-024 record
Decision	Each producer commits owned state and frozen public fact in one local database transaction.
Consequences	Broker outage is decoupled from owner commit; relay and consumer remain at-least-once and idempotent.
Rejected	Publish-before-commit, commit-then-best-effort-publish, broker inside DB transaction, central outbox DB, mutable replay serialization.
Review trigger	Provider, envelope, payload cap, identity, dispatch state, retention, restore or ownership change.

CURRENT EVIDENCE STATUS

The code and tests are compile-oriented reference assets. No .NET SDK, selected EF Core provider, SQL Server image, migration, fault interceptor or test run was available in this manuscript environment. Every OUTBOX result remains PENDING.

IMPLEMENTATION SEQUENCE AND CHECKPOINT

Durable intent is specified; dispatch remains the next proof

Implementation sequence

- Accept ADR-024 and name the owner for every producer's public fact catalogue, migration and retention policy.
- Implement the immutable serializer output, FactKey grammar and exact envelope digest fixtures.
- Apply the messaging.OutboxMessage migration independently to every approved producer database.
- Stage owner state and one or more finite public facts before one SaveChanges call.
- Implement provider-specific unique, concurrency, transient and unknown-commit classification.
- Run atomic success, rollback, no-broker, lost-response, race, security, restore and backlog tests against the production database engine.
- Publish OUTBOX-SCHEMA-001 through OUTBOX-REVIEW-001 with image, migration, serializer and result digests.

Official-source validation

The transaction boundary was checked against Microsoft EF Core transaction guidance: one SaveChanges call is transactional for providers that support transactions, and an existing transaction receives a savepoint before SaveChanges. Microsoft's microservice guidance distinguishes domain events from integration events and places public event propagation outside the local domain transaction. SQL Server documentation confirms that transaction isolation and locking must be used efficiently under concurrency. NEXUS-1's schema, identities, evidence IDs, payload cap and failure policy remain project decisions.

Sources: <https://learn.microsoft.com/ef/core/saving/transactions> ;

<https://learn.microsoft.com/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/domain-events-design-implementation> ;

<https://learn.microsoft.com/dotnet/architecture/microservices/multi-container-microservice-net-applications/integration-event-based-microservice-communications> ;

<https://learn.microsoft.com/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide>

What this chapter has proved

Chapter 26 has specified one producer-side atomic boundary: owner state and an immutable serialized public fact share one local commit; semantic identity survives retry and unknown outcomes; each producer owns its own outbox; and acceptance requires provider-backed counts rather than prose or mocks. It has not proved dispatch, broker confirmation, ordering, consumer completion or exactly-once delivery.

NEXT CHAPTER

Chapter 27, Dispatch, Publisher Confirms, and Recovery, will claim Pending rows safely, publish their frozen bytes, correlate mandatory returns and confirms, and recover from every ambiguous relay interruption without allocating a new messageId.

CHAPTER CHECKPOINT - Continue only when OUTBOX-ATOMIC-001 and OUTBOX-ROLLBACK-001 pass on the selected provider, OUTBOX-UNKNOWN-001 resolves a real ambiguous commit, OUTBOX-NOBROKER-001 proves zero broker dependency, and all remaining evidence stays explicitly PENDING rather than implied.

PART V - THE MESSAGE BACKBONE

27. Dispatch, Publisher Confirms, and Recovery

A committed outbox row is only publication intent. The relay must claim it without double work, publish its frozen bytes, interpret routing and broker confirmation, and survive every ambiguous interruption without inventing a new public fact.

CHAPTER DECISION

Each producer runs an at-least-once outbox relay against its own database. Only a positively confirmed, routable publication may become Dispatched. Every unknown outcome remains replayable with the same MessageId and bytes.

27.1 The guarantee ledger

The ★ acceptance surface is executable: SQL Server claim/lease scripts, a bounded dispatcher, RabbitMQ publisher-confirm handling, deterministic recovery probes and a fail-closed evidence record. It does not claim exactly-once delivery or consumer-side atomicity.

GUARANTEE LEDGER 27-A - CLAIMS ALLOWED BY EVIDENCE

Boundary	Proven mechanism
Pending row -> one active relay	Atomic UPDATE...OUTPUT claim + lease
Relay -> RabbitMQ	mandatory publish + publisher confirms
Confirm -> Dispatched row	owner/version-guarded database update
Relay crash -> retry	expired lease + same stored MessageId
Frozen fact -> replay	stored bytes + SHA-256 verification
Allowed claim	Explicitly not claimed
At-least-once broker publication	Exactly-once broker delivery
One active lease per row	One global dispatcher across services
Positive broker acceptance	Consumer processing or business success
Duplicate-safe message identity	Duplicate-free delivery

BOUNDARY TO CHAPTER 28

A crash after broker acceptance but before the database mark can publish a duplicate. Chapter 28 must make the consumer's inbox and local side effect idempotent. Chapter 27 preserves identity; it cannot remove that ambiguity.

RELAY PROTOCOL

The state machine records knowledge, not optimism

Pending and Retryable are dispatchable. InFlight means one unexpired lease owns an attempt. Dispatched means a routable positive confirm was observed. Quarantined means automation stopped because replay would repeat a deterministic fault.

EXECUTABLE ASSET 27-B - RELAY STATE AND IMMUTABLE CLAIM

```
// OutboxState is the persisted contract defined by asset 26-A.
public sealed record OutboxClaim(
    Guid MessageId,
    string FactKey,
    string Producer,
    string EventType,
    int SchemaVersion,
    string RoutingKey,
    string ContentType,
    DateTimeOffset OccurredAtUtc,
    DateTimeOffset StoredAtUtc,
    byte[] EnvelopeBytes,
    byte[] EnvelopeSha256,
    string HeadersJson,
    int AttemptCount,
    string LeaseOwner,
    DateTimeOffset LeaseExpiresAtUtc,
    byte[] Version);

public abstract record PublishObservation
{
    public sealed record Confirmed(DateTimeOffset AtUtc)
        : PublishObservation;
    public sealed record Rejected(string Code)
        : PublishObservation;
    public sealed record Unknown(string Code)
        : PublishObservation;
}
```

EXECUTABLE ASSET 27-C - ALLOWED TRANSITIONS

```
public static class DispatchTransition
{
    public static bool CanClaim(
        OutboxState state,
        DateTimeOffset? nextAttemptAtUtc,
        DateTimeOffset now) =>
        state is OutboxState.Pending ||
        state is OutboxState.Retryable &&
        (nextAttemptAtUtc is null ||
         nextAttemptAtUtc <= now);

    public static OutboxState Complete(
        PublishObservation observation) => observation switch
    {
        PublishObservation.Confirmed =>
            OutboxState.Dispatched,
        PublishObservation.Rejected =>
            OutboxState.Quarantined,
        PublishObservation.Unknown =>
            OutboxState.Retryable,
        _ => throw new UnreachableException()
    };
}
```

INVARIANT

Dispatched is irreversible. A lease is not proof of publication, a socket write is not a confirm, and a positive confirm is not proof of consumer work.

PROVIDER MIGRATION

Lease recovery needs one additional access path

Chapter 26 created the dispatchable index. Chapter 27 adds an index for expired InFlight leases and constraints that reject partially populated lease state.

EXECUTABLE ASSET 27-D - SQL SERVER LEASE MIGRATION

```
ALTER TABLE messaging.OutboxMessage
ADD CONSTRAINT CK_Outbox_LeaseShape CHECK
(
    (State = 1
     AND LockedBy IS NOT NULL
     AND LockExpiresAtUtc IS NOT NULL)
    OR
    (State <> 1
     AND LockedBy IS NULL
     AND LockExpiresAtUtc IS NULL)
);

ALTER TABLE messaging.OutboxMessage
ADD CONSTRAINT CK_Outbox_DispatchedShape CHECK
(
    (State = 3 AND DispatchedAtUtc IS NOT NULL)
    OR
    (State <> 3 AND DispatchedAtUtc IS NULL)
);

CREATE INDEX IX_Outbox_ExpiredLease
ON messaging.OutboxMessage
(LockExpiresAtUtc, StoredAtUtc, MessageId)
WHERE State = 1;
```

EXECUTABLE ASSET 27-E - RECOVERY SWEEP

```
DECLARE @Now datetimeoffset(7) = SYSUTCDATETIME();

;WITH expired AS
(
    SELECT TOP (@RecoveryBatchSize) *
    FROM messaging.OutboxMessage
        WITH (UPDLOCK, READPAST, ROWLOCK)
    WHERE State = 1
        AND LockExpiresAtUtc < @Now
    ORDER BY LockExpiresAtUtc, StoredAtUtc, MessageId
)
UPDATE expired
SET State = 2,
    NextAttemptAtUtc = @Now,
    LockedBy = NULL,
    LockExpiresAtUtc = NULL
OUTPUT
    inserted.MessageId,
    deleted.LockedBy,
    deleted.LockExpiresAtUtc,
    inserted.Version
;
```

SERVER TIME

Eligibility and lease expiry use SQL Server time in the claim transaction. Host clocks remain useful for telemetry, but cannot decide ownership.

ATOMIC CLAIM

Claim a bounded ordered batch in one short transaction

UPDLOCK makes candidates update-owned, **READPAST** avoids waiting behind another claimer, and **UPDATE...OUTPUT** returns the exact rows whose lease was acquired. The transaction ends before any network publication begins.

EXECUTABLE ASSET 27-F - SQL SERVER CLAIM BATCH

```
DECLARE @Now datetimeoffset(7) = SYSUTCDATETIME();
DECLARE @LeaseUntil datetimeoffset(7) =
    DATEADD(second, @LeaseSeconds, @Now);

;WITH candidate AS
(
    SELECT TOP (@BatchSize) *
    FROM messaging.OutboxMessage
        WITH (UPDLOCK, READPAST, ROWLOCK)
    WHERE State IN (0, 2)
        AND (NextAttemptAtUtc IS NULL
            OR NextAttemptAtUtc <= @Now)
    ORDER BY StoredAtUtc, MessageId
)
UPDATE candidate
SET State = 1,
    AttemptCount = AttemptCount + 1,
    NextAttemptAtUtc = NULL,
    LockedBy = @DispatcherId,
    LockExpiresAtUtc = @LeaseUntil
OUTPUT
    inserted.MessageId,
    inserted.FactKey,
    inserted.Producer,
    inserted.EventType,
    inserted.SchemaVersion,
    inserted.RoutingKey,
    inserted.ContentType,
    inserted.OccurredAtUtc,
    inserted.StoredAtUtc,
    inserted.EnvelopeBytes,
    inserted.EnvelopeSha256,
    inserted.HeadersJson,
    inserted.AttemptCount,
    inserted.LockedBy,
    inserted.LockExpiresAtUtc,
    inserted.Version;
```

LOCK HINT BUDGET

ROWLOCK is a preference, not a promise. Batch size, transaction duration and the selected SQL Server isolation mode are measured under production-like load; the relay never holds database locks while waiting for RabbitMQ.

INFRASTRUCTURE PORT

The store exposes claims, not tracked EF entities

Raw provider SQL owns the claim because it needs SQL Server lock semantics. The Application worker receives immutable values and later applies a guarded result.

EXECUTABLE ASSET 27-G - DISPATCH STORE CONTRACT

```
public interface IOutboxDispatchStore
{
    Task<IReadOnlyList<OutboxClaim>> ClaimAsync(
        DispatcherIdentity dispatcher,
        int batchSize,
        TimeSpan leaseDuration,
        CancellationToken cancellationToken);

    Task<ApplyResult> MarkDispatchedAsync(
        OutboxClaim claim,
        DateTimeOffset confirmedAtUtc,
        CancellationToken cancellationToken);

    Task<ApplyResult> ScheduleRetryAsync(
        OutboxClaim claim,
        DateTimeOffset nextAttemptAtUtc,
        string reasonCode,
        CancellationToken cancellationToken);

    Task<ApplyResult> QuarantineAsync(
        OutboxClaim claim,
        string reasonCode,
        CancellationToken cancellationToken);

    Task<int> RecoverExpiredLeasesAsync(
        int batchSize,
        CancellationToken cancellationToken);
}

public readonly record struct DispatcherIdentity(string Value);
public enum ApplyResult { Applied, LeaseLost, AlreadyFinal }
```

EXECUTABLE ASSET 27-H - PARAMETERIZED CLAIM EXECUTION

```
public sealed class SqlServerOutboxDispatchStore(
    IDbConnectionFactory connections,
    IClaimMapper mapper) : IOutboxDispatchStore
{
    public async Task<IReadOnlyList<OutboxClaim>> ClaimAsync(
        DispatcherIdentity dispatcher,
        int batchSize,
        TimeSpan leaseDuration,
        CancellationToken cancellationToken)
    {
        await using var connection =
            await connections.OpenAsync(cancellationToken);
        await using var transaction =
            await connection.BeginTransactionAsync(
                IsolationLevel.ReadCommitted,
                cancellationToken);

        var command = connection.CreateCommand();
        command.Transaction = transaction;
        command.CommandText = ClaimSql.Text;
        command.Add("@DispatcherId", dispatcher.Value);
        command.Add("@BatchSize", batchSize);
        command.Add("@LeaseSeconds",
            checked((int)leaseDuration.TotalSeconds));

        var claims = await mapper.ReadAsync(
            command, cancellationToken);
        await transaction.CommitAsync(cancellationToken);
        return claims;
    }
}
```

GUARDED COMPLETION

Only the active lease may write a publication result

Every final or retry update compares MessageId, state, lease owner and rowversion. A late publisher cannot overwrite work reclaimed by another process.

EXECUTABLE ASSET 27-I - MARK CONFIRMED PUBLICATION

```
UPDATE messaging.OutboxMessage
SET State = 3,
    DispatchedAtUtc = @ConfirmedAtUtc,
    LockedBy = NULL,
    LockExpiresAtUtc = NULL,
    NextAttemptAtUtc = NULL
WHERE MessageId = @MessageId
    AND State = 1
    AND LockedBy = @DispatcherId
    AND Version = @ClaimedVersion;

SELECT @@ROWCOUNT;
```

EXECUTABLE ASSET 27-J - RELEASE UNKNOWN OUTCOME

```
UPDATE messaging.OutboxMessage
SET State = 2,
    NextAttemptAtUtc = @NextAttemptAtUtc,
    LockedBy = NULL,
    LockExpiresAtUtc = NULL
WHERE MessageId = @MessageId
    AND State = 1
    AND LockedBy = @DispatcherId
    AND Version = @ClaimedVersion;

SELECT @@ROWCOUNT;
```

EXECUTABLE ASSET 27-K - INTERPRET ZERO AFFECTED ROWS

```
public static async Task<ApplyResult> ResolveZeroAsync(
    Guid messageId,
    IOutboxStateReader reader,
    CancellationToken cancellationToken)
{
    var current = await reader.ReadAsync(
        messageId, cancellationToken);

    return current.State switch
    {
        {
            OutboxState.Dispatched or
            OutboxState.Quarantined =>
                ApplyResult.AlreadyFinal,
        } => ApplyResult.LeaseLost
    };
}
```

NO BLIND UPDATE

Updating by MessageId alone can let a timed-out publisher mark a row after another relay already reclaimed it. Lease identity and rowversion close that race.

BOUNDED WORKER

One batch owns one scope and one finite budget

The hosted service creates a fresh scope for every batch, performs a bounded recovery sweep, claims rows, dispatches them, then yields. Cancellation stops new claims and allows only the configured drain budget.

EXECUTABLE ASSET 27-L - VALIDATED OPTIONS

```
public sealed class OutboxDispatcherOptions
{
    public const string Section = "Messaging:OutboxDispatcher";
    public int BatchSize { get; init; } = 64;
    public int RecoveryBatchSize { get; init; } = 128;
    public TimeSpan LeaseDuration { get; init; } =
        TimeSpan.FromSeconds(45);
    public TimeSpan ConfirmTimeout { get; init; } =
        TimeSpan.FromSeconds(10);
    public TimeSpan IdleDelay { get; init; } =
        TimeSpan.FromMilliseconds(250);
    public TimeSpan ShutdownDrainBudget { get; init; } =
        TimeSpan.FromSeconds(20);
}

public sealed class OutboxDispatcherOptionsValidator
    : IValidateOptions<OutboxDispatcherOptions>
{
    public ValidateOptionsResult Validate(
        string? name, OutboxDispatcherOptions value)
    {
        if (value.BatchSize is < 1 or > 500)
            return ValidateOptionsResult.Fail("BatchSize: 1..500");
        if (value.ConfirmTimeout >= value.LeaseDuration)
            return ValidateOptionsResult.Fail(
                "ConfirmTimeout must be shorter than the lease.");
        if (value.ShutdownDrainBudget >= value.LeaseDuration)
            return ValidateOptionsResult.Fail(
                "Drain budget must be shorter than the lease.");
        return ValidateOptionsResult.Success;
    }
}
```

EXECUTABLE ASSET 27-M - HOSTED RELAY LOOP

```
public sealed partial class OutboxDispatcherService(
    IServiceScopeFactory scopes,
    IOptions<OutboxDispatcherOptions> options)
    : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using var scope =
                scopes.CreateAsyncScope();
            var batch = scope.ServiceProvider
                .GetRequiredService<IOutboxBatchRunner>();
            var count = await batch.RunOnceAsync(stoppingToken);
            if (count == 0)
                await Task.Delay(
                    options.Value.IdleDelay, stoppingToken);
        }
    }
}
```

BATCH ORCHESTRATION

Verify bytes before the network boundary

The runner treats the stored envelope as the public fact. A digest mismatch is deterministic corruption and is quarantined without publishing. Other claims are sent through one confirming publisher session.

EXECUTABLE ASSET 27-N - ONE DISPATCHER ITERATION

```
public sealed class OutboxBatchRunner(
    IOutboxDispatchStore store,
    IConfirmingPublisher publisher,
    IDispatchOutcomeApplier outcomes,
    DispatcherIdentity identity,
    IOptions<OutboxDispatcherOptions> options)
    : IOutboxBatchRunner
{
    public async Task<int> RunOnceAsync(
        CancellationToken cancellationToken)
    {
        var policy = options.Value;
        await store.RecoverExpiredLeasesAsync(
            policy.RecoveryBatchSize, cancellationToken);

        var claims = await store.ClaimAsync(
            identity,
            policy.BatchSize,
            policy.LeaseDuration,
            cancellationToken);

        foreach (var claim in claims)
        {
            if (!EnvelopeIntegrity.Matches(claim))
            {
                await store.QuarantineAsync(
                    claim, "envelope-digest-mismatch",
                    cancellationToken);
                continue;
            }

            var observation = await publisher.PublishAsync(
                claim, policy.ConfirmTimeout,
                cancellationToken);
            await outcomes.ApplyAsync(
                claim, observation, cancellationToken);
        }

        return claims.Count;
    }
}
```

EXECUTABLE ASSET 27-O - CONSTANT-TIME DIGEST CHECK

```
public static class EnvelopeIntegrity
{
    public static bool Matches(OutboxClaim claim)
    {
        var actual = SHA256.HashData(claim.EnvelopeBytes);
        return actual.Length == claim.EnvelopeSha256.Length &&
            CryptographicOperations.FixedTimeEquals(
                actual, claim.EnvelopeSha256);
    }
}
```

ORDERING

StoredAtUtc and MessageId make claiming deterministic, not globally ordered. Multiple producers and partitions progress independently; consumers must not infer a total business order from broker arrival.

RABBITMQ SESSION

Mandatory routing and confirms are one acceptance test

The **RabbitMQ.Client 7.x** channel enables publisher confirms and confirmation tracking. **BasicPublishAsync** with **mandatory=true** completes only after the client has correlated the broker response; a **nack** or **basic.return** raises **PublishException**.

EXECUTABLE ASSET 27-P - CONFIRMING PUBLISHER ADAPTER

```
public sealed class RabbitMqConfirmingPublisher(
    IConnection connection,
    RabbitMqTopology topology,
    IOutboxHeaderReader headers)
    : IConfirmingPublisher
{
    public async Task<PublishObservation> PublishAsync(
        OutboxClaim claim,
        TimeSpan timeout,
        CancellationToken cancellationToken)
    {
        using var confirmBudget =
            CancellationTokenSource.CreateLinkedTokenSource(
                cancellationToken);
        confirmBudget.CancelAfter(timeout);

        try
        {
            var options = new CreateChannelOptions(
                publisherConfirmationsEnabled: true,
                publisherConfirmationTrackingEnabled: true);
            await using var channel =
                await connection.CreateChannelAsync(
                    options, confirmBudget.Token);

            var properties = CreateProperties(claim);
            await channel.BasicPublishAsync(
                exchange: topology.PublicFactsExchange,
                routingKey: claim.RoutingKey,
                mandatory: true,
                basicProperties: properties,
                body: claim.EnvelopeBytes,
                cancellationToken: confirmBudget.Token);

            return new PublishObservation.Confirmed(
                DateTimeOffset.UtcNow);
        }
        catch (PublishException)
        {
            return new PublishObservation.Rejected(
                "broker-nack-or-unroutable");
        }
        catch (OperationCanceledException)
            when (!cancellationToken.IsCancellationRequested)
        {
            return new PublishObservation.Unknown(
                "confirm-timeout");
        }
        catch (AlreadyClosedException)
        {
            return new PublishObservation.Unknown(
                "channel-closed");
        }
    }
}
```

WHAT A CONFIRM MEANS

For durable queues and persistent messages, RabbitMQ confirms after the configured durability condition; quorum queues confirm after acceptance by a quorum. A confirm still says nothing about consumer execution.

WIRE IDENTITY

Transport attempts may change; public identity may not

The adapter reconstructs AMQP properties from persisted metadata. MessageId, type, content type, occurrence time, correlation and frozen body are stable across retries.

EXECUTABLE ASSET 27-Q - AMQP PROPERTIES FROM STORED INTENT

```
private BasicProperties CreateProperties(OutboxClaim claim)
{
    var stored = headers.Read(claim.HeadersJson);
    return new BasicProperties
    {
        AppId = claim.Producer,
        MessageId = claim.MessageId.ToString("D"),
        Type = claim.EventType,
        ContentType = claim.ContentType,
        ContentEncoding = "utf-8",
        CorrelationId = stored.CorrelationId,
        DeliveryMode = DeliveryModes.Persistent,
        Timestamp = new AmqpTimestamp(
            claim.OccurredAtUtc.ToUnixTimeSeconds()),
        Headers = new Dictionary<string, object?>
        {
            ["schema-version"] = claim.SchemaVersion,
            ["causation-id"] = stored.CausationId,
            ["traceparent"] = stored.TraceParent,
            ["tracestate"] = stored.TraceState,
            ["envelope-sha256"] =
                Convert.ToHexString(claim.EnvelopeSha256)
        }
    };
}
```

EXECUTABLE ASSET 27-R - TRANSPORT ATTEMPT TELEMETRY

```
public sealed record DispatchAttemptTelemetry(
    Guid MessageId,
    string Producer,
    string EventType,
    int AttemptCount,
    string DispatcherId,
    DateTimeOffset StartedAtUtc,
    TimeSpan ConfirmLatency,
    string OutcomeCode);

// AttemptCount and DispatcherId are telemetry, not public headers.
// They must not alter EnvelopeBytes or MessageId.
```

HEADER SAFETY

The relay never logs the envelope or arbitrary headers. Correlation values are validated and length-bounded when Chapter 26 writes HeadersJson; secrets and credentials never enter the outbox.

CONFIRM THROUGHPUT

Bound the in-flight window; do not wait synchronously per row

Publisher confirms are asynchronous. The production adapter may keep a channel-scoped window of outstanding publications, but every entry has a MessageId, deadline and completion. Channel access remains serialized.

EXECUTABLE ASSET 27-S - BOUNDED CONFIRM WINDOW

```
public sealed class ConfirmWindow(
    int capacity,
    IChannelWriter writer)
{
    private readonly SemaphoreSlim _slots =
        new(capacity, capacity);
    private readonly ConcurrentDictionary<Guid, TaskCompletionSource<
        PublishObservation>> _pending = new();

    public async Task<PublishObservation> PublishAsync(
        OutboxClaim claim,
        CancellationToken cancellationToken)
    {
        await _slots.WaitAsync(cancellationToken);
        var completion = new TaskCompletionSource<
            PublishObservation>(
                TaskCreationOptions.RunContinuationsAsynchronously);

        if (!_pending.TryAdd(claim.MessageId, completion))
            throw new DuplicateInFlightMessageId(
                claim.MessageId);

        try
        {
            await writer.WriteMandatoryAsync(
                claim, cancellationToken);
            return await completion.Task.WaitAsync(
                cancellationToken);
        }
        finally
        {
            _pending.TryRemove(claim.MessageId, out _);
            _slots.Release();
        }
    }
}
```

27.2 Two acceptable adapters

The simple adapter on the previous page uses `RabbitMQ.Client` confirmation tracking and awaits one publish. It is the easiest correctness baseline. A measured production adapter may pipeline a bounded window and correlate `basic.ack/basic.nack/basic.return` by publish sequence number. Both must make mandatory returns non-success, fail all pending entries on channel shutdown, and create a fresh channel after a channel-level exception.

THROUGHPUT RULE

Never remove confirmation to gain throughput. Increase a measured bounded window, partition relay ownership, or add dispatcher instances while keeping database and broker backpressure visible.

OUTCOME APPLICATION

Confirmed, rejected and unknown take different paths

The result applier is the only policy point between transport observation and durable state. It never interprets cancellation or a closed connection as a negative confirm.

EXECUTABLE ASSET 27-T - APPLY TRANSPORT OBSERVATION

```
public sealed class DispatchOutcomeApplier(
    IOutboxDispatchStore store,
    IDispatchBackoff backoff,
    IClock clock) : IDispatchOutcomeApplier
{
    public Task<ApplyResult> ApplyAsync(
        OutboxClaim claim,
        PublishObservation observation,
        CancellationToken cancellationToken) =>
        observation switch
        {
            PublishObservation.Confirmed confirmed =>
                store.MarkDispatchedAsync(
                    claim,
                    confirmed.AtUtc,
                    cancellationToken),

            PublishObservation.Rejected rejected =>
                store.QuarantineAsync(
                    claim,
                    rejected.Code,
                    cancellationToken),

            PublishObservation.Unknown unknown =>
                store.ScheduleRetryAsync(
                    claim,
                    backoff.Next(
                        claim.MessageId,
                        claim.AttemptCount,
                        clock.UtcNow),
                    unknown.Code,
                    cancellationToken),

            _ => throw new UnreachableException()
        };
}
```

DECISION TABLE 27-U - DURABLE RESPONSE

Observation	Durable state	Reason
positive confirm, routable	Dispatched	broker accepted responsibility
basic.nack	Quarantined	explicit negative broker result
mandatory basic.return	Quarantined	topology/routing defect
confirm timeout	Retryable	publication may have succeeded
connection/channel loss	Retryable	publication may have succeeded
digest mismatch	Quarantined	stored intent is corrupt
active lease lost	unchanged	another owner now decides

POLICY BOUNDARY

Chapter 27 stops deterministic routing/configuration faults and retries ambiguous transport outcomes. Chapter 29 generalizes retry, DLQ and poison handling across the wider messaging backbone.

DETERMINISTIC BACKOFF

Retries spread load without changing identity

Backoff is bounded exponential delay plus stable per-message jitter. Restarting a relay does not reset attempts or synchronize every pending row onto the same instant.

EXECUTABLE ASSET 27-V - STABLE BOUNDED BACKOFF

```
public sealed class DispatchBackoff : IDispatchBackoff
{
    private static readonly TimeSpan Base =
        TimeSpan.FromMilliseconds(250);
    private static readonly TimeSpan Maximum =
        TimeSpan.FromMinutes(2);

    public DateTimeOffset Next(
        Guid messageId,
        int attempt,
        DateTimeOffset now)
    {
        var exponent = Math.Min(Math.Max(attempt - 1, 0), 9);
        var exponentialMs = Math.Min(
            Base.TotalMilliseconds * Math.Pow(2, exponent),
            Maximum.TotalMilliseconds);

        Span<byte> hash = stackalloc byte[32];
        SHA256.HashData(messageId.ToArray(), hash);
        var unit = BinaryPrimitives.ReadUInt32LittleEndian(hash)
            / (double)uint.MaxValue;
        var jitter = 0.75 + unit * 0.50;

        return now.AddMilliseconds(
            exponentialMs * jitter);
    }
}
```

EXECUTABLE ASSET 27-W - BACKOFF INVARIANTS

```
[Theory]
[InlineData(1)]
[InlineData(2)]
[InlineData(8)]
[InlineData(100)]
public void BACKOFF_001_is_bounded_and_stable(int attempt)
{
    var id = Guid.Parse(
        "bd06b230-e183-46f1-8f18-4370227988a4");
    var now = DateTimeOffset.Parse(
        "2026-01-01T00:00:00Z");
    var sut = new DispatchBackoff();

    var first = sut.Next(id, attempt, now);
    var second = sut.Next(id, attempt, now);

    first.Should().Be(second);
    (first - now).Should().BePositive();
    (first - now).Should().BeLessThanOrEqualTo(
        TimeSpan.FromMinutes(2.5));
}
```

ATTEMPT BUDGET

AttemptCount supports evidence and operational policy. It must not allocate a new MessageId. A later Chapter 29 policy may escalate persistent transient failure, but it cannot rewrite the original public fact.

CRASH SEMANTICS

Every interruption maps to one permitted recovery action

There is no atomic transaction between SQL Server and RabbitMQ. The design therefore makes each ambiguity explicit and chooses replay over loss.

CRASH MATRIX 27-X - RELAY INTERRUPTION POINTS

Crash point	Database	Broker
before claim	Pending	absent
after claim, before publish	InFlight	absent
after socket write, before confirm	InFlight	unknown
after positive confirm, before mark	InFlight	accepted
during mark response	unknown to relay	accepted
after Dispatched commit	Dispatched	accepted
Recovery		
Pending: claim normally.		
InFlight before lease expiry: do not steal.		
InFlight after lease expiry: recovery sweep -> Retryable.		
Retryable: publish stored bytes with the same MessageId.		
Dispatched: never publish again through the normal relay.		

EXECUTABLE ASSET 27-Y - CRASH CHECKPOINTS

```
public enum RelayCheckpoint
{
    Claimed,
    IntegrityVerified,
    PublishWriteStarted,
    ConfirmObserved,
    ResultWriteStarted,
    ResultCommitted
}

public interface IRelayFaultPoint
{
    ValueTask HitAsync(
        RelayCheckpoint checkpoint,
        Guid messageId,
        CancellationToken cancellationToken);
}

public sealed class NoRelayFaultPoint : IRelayFaultPoint
{
    public ValueTask HitAsync(
        RelayCheckpoint checkpoint,
        Guid messageId,
        CancellationToken cancellationToken) =>
        ValueTask.CompletedTask;
}
```

THE UNAVOIDABLE DUPLICATE WINDOW

ConfirmObserved → ResultCommitted cannot be made atomic without a distributed transaction. If the relay dies there, the message is republished. Stable MessageId makes Chapter 28's inbox able to collapse the duplicate effect.

RESTART RECOVERY

The database is the relay's durable memory

Client automatic recovery can rebuild connections and topology, but it does not buffer every application publication. The relay must close its failed channel, rely on the lease, and reconstruct work from the outbox.

EXECUTABLE ASSET 27-Z - RECOVERY COORDINATOR

```
public sealed class RelayRecoveryCoordinator(
    IOutboxDispatchStore store,
    IRelaySessionFactory sessions,
    IClock clock)
{
    public async Task<RecoveryReport> RecoverAsync(
        int batchSize,
        CancellationToken cancellationToken)
    {
        await sessions.DiscardClosedChannelsAsync(
            cancellationToken);

        var recovered = 0;
        while (true)
        {
            var count = await store.RecoverExpiredLeasesAsync(
                batchSize, cancellationToken);
            recovered += count;
            if (count < batchSize)
                break;
        }

        return new RecoveryReport(
            recovered,
            clock.UtcNow);
    }
}

public sealed record RecoveryReport(
    int ExpiredLeasesReleased,
    DateTimeOffset CompletedAtUtc);
```

RESTART CHECKLIST 27-AA - FAIL CLOSED

1. Validate database and broker configuration.
2. Verify expected exchange passively; never create surprise topology.
3. Open a new confirm-enabled channel.
4. Release only expired leases in bounded batches.
5. Claim eligible Pending/Retryable rows.
6. Verify stored digest before every publish.
7. Treat connection recovery time as no-publication time.
8. Resume only when readiness and topology gates pass.

NO IN-MEMORY RECOVERY ORACLE

A dictionary of pending confirms accelerates one live channel; it is not durable evidence. After process loss, the outbox state and lease expiry are the only authoritative restart inputs.

UNIT PROOF

State and identity rules run without infrastructure

Fast tests pin the relay's knowledge model. They do not pretend to prove SQL locking or RabbitMQ confirms; those require the selected provider and broker.

EXECUTABLE ASSET 27-AB - UNKNOWN KEEPS THE SAME FACT

```
[Fact]
public async Task RELAY_UNIT_001_unknown_retries_same_identity()
{
    var claim = RelayFixture.Claim(
        messageId: Guid.Parse(
            "990777ce-7c71-4321-b46a-11ef01f506fc"),
        bytes: "frozen-envelope-v1"u8.ToArray(),
        attempt: 3);
    var store = new RecordingDispatchStore();
    var sut = RelayFixture.OutcomeApplier(store);

    await sut.ApplyAsync(
        claim,
        new PublishObservation.Unknown("confirm-timeout"),
        CancellationToken.None);

    store.Retry!.MessageId.Should().Be(claim.MessageId);
    store.Retry.EnvelopeBytes.Should().Equal(
        claim.EnvelopeBytes);
    store.Dispatched.Should().BeNull();
}
```

EXECUTABLE ASSET 27-AC - RETURN IS NEVER SUCCESS

```
[Fact]
public async Task RELAY_UNIT_002_unroutable_is_quarantined()
{
    var claim = RelayFixture.Claim();
    var store = new RecordingDispatchStore();
    var sut = RelayFixture.OutcomeApplier(store);

    await sut.ApplyAsync(
        claim,
        new PublishObservation.Rejected(
            "broker-nack-or-unroutable"),
        CancellationToken.None);

    store.Quarantined!.MessageId.Should().Be(
        claim.MessageId);
    store.Dispatched.Should().BeNull();
    store.Retry.Should().BeNull();
}
```

EXECUTABLE ASSET 27-AD - CORRUPTION BLOCKS PUBLICATION

```
[Fact]
public async Task RELAY_UNIT_003_digest_mismatch_never_publishes()
{
    var fixture = RelayFixture.WithCorruptClaim();
    await fixture.Runner.RunOnceAsync(
        CancellationToken.None);

    fixture.Publisher.Calls.Should().Be(0);
    fixture.Store.QuarantineReason.Should().Be(
        "envelope-digest-mismatch");
}
```

SQL SERVER COMPONENT PROOF

Competing dispatchers must not share an active lease

The component lane applies the real migration to disposable SQL Server. Barriers align two claim transactions without sleeps, then inspect the returned claims and durable rows.

EXECUTABLE ASSET 27-AE - CONCURRENT CLAIM EXCLUSION

```
[Fact]
public async Task RELAY_DB_001_two_claimers_do_not_overlap()
{
    await fixture.SeedPendingAsync(count: 100);
    var gate = new Barrier(participantCount: 2);

    var left = fixture.ClaimAfterBarrierAsync(
        "relay-a", batchSize: 64, gate);
    var right = fixture.ClaimAfterBarrierAsync(
        "relay-b", batchSize: 64, gate);

    var batches = await Task.WhenAll(left, right);
    var leftIds = batches[0].Select(x => x.MessageId);
    var rightIds = batches[1].Select(x => x.MessageId);

    leftIds.Should().NotIntersectWith(rightIds);
    batches.Sum(x => x.Count).Should().Be(100);
    await fixture.AssertAllInFlightOwnedOnceAsync();
}
```

EXECUTABLE ASSET 27-AF - EXPIRED LEASE RECOVERY

```
[Fact]
public async Task RELAY_DB_002_only_expired_lease_is_released()
{
    var expired = await fixture.SeedInFlightAsync(
        "dead-relay", expiresIn: TimeSpan.FromSeconds(-1));
    var active = await fixture.SeedInFlightAsync(
        "live-relay", expiresIn: TimeSpan.FromMinutes(1));

    var recovered =
        await fixture.Store.RecoverExpiredLeasesAsync(
            batchSize: 10, Cancellation.Token.None);

    recovered.Should().Be(1);
    await fixture.AssertStateAsync(
        expired, OutboxState.Retryable);
    await fixture.AssertStateAsync(
        active, OutboxState.InFlight);
}
```

REQUIRED PROVIDER

EF InMemory and SQLite cannot validate SQL Server UPDLOCK, READPAST, rowversion or filtered-index behavior. A passing fake-store test is not claim/lease evidence.

RABBITMQ COMPONENT PROOF

The acceptance lane observes ack, return and outage

A disposable RabbitMQ deployment declares the production exchange and queue shape. The tests publish persistent messages through the real adapter and inspect both the outbox result and broker-visible delivery.

EXECUTABLE ASSET 27-AG - ROUTABLE CONFIRM

```
[Fact]
public async Task RELAY_RMQ_001_routable_confirm_marks_dispatched()
{
    var claim = await fixture.SeedPendingClaimAsync();
    await fixture.Topology.BindExpectedQueueAsync(
        claim.RoutingKey);

    await fixture.DispatchAsync(claim);

    await fixture.Database.AssertStateAsync(
        claim.MessageId,
        OutboxState.Dispatched);
    var delivered = await fixture.Broker.GetAsync();
    delivered.Properties.MessageId.Should().Be(
        claim.MessageId.ToString("D"));
    delivered.Body.ToArray().Should().Equal(
        claim.EnvelopeBytes);
}
```

EXECUTABLE ASSET 27-AH - MANDATORY RETURN

```
[Fact]
public async Task RELAY_RMQ_002_unroutable_never_marks_success()
{
    var claim = await fixture.SeedPendingClaimAsync(
        routingKey: "not.bound.anywhere");

    await fixture.DispatchAsync(claim);

    await fixture.Database.AssertStateAsync(
        claim.MessageId,
        OutboxState.Quarantined);
    await fixture.Database.AssertReasonAsync(
        claim.MessageId,
        "broker-nack-or-unroutable");
}
```

EXECUTABLE ASSET 27-AI - BROKER OUTAGE

```
[Fact]
public async Task RELAY_RMQ_003_outage_preserves_replayable_row()
{
    var claim = await fixture.SeedPendingClaimAsync();
    await fixture.Broker.StopAsync();

    await fixture.DispatchAsync(claim);

    await fixture.Database.AssertStateAsync(
        claim.MessageId,
        OutboxState.Retryable);
    await fixture.Database.AssertFrozenFactAsync(claim);
}
```

DETERMINISTIC CRASH LABORATORY

Kill the relay before and after publication

The crash harness stops the process at named checkpoints, advances provider time past the lease, restarts with a new dispatcher identity and records database plus broker observations. It never depends on arbitrary sleeps.

EXECUTABLE ASSET 27-AJ - CRASH AFTER CLAIM

```
[Fact]
public async Task RELAY_CRASH_001_claimed_row_is_recovered()
{
    var scenario = await lab.StartAsync(
        crashAt: RelayCheckpoint.Claimed);
    await scenario.WaitForCrashAsync();

    await scenario.AdvanceDatabaseTimePastLeaseAsync();
    await scenario.RestartAsync("relay-replacement");

    await scenario.AssertEventuallyDispatchedAsync();
    scenario.PublishedBodies.Should().ContainSingle()
        .Which.Should().Equal(scenario.FrozenBody);
    scenario.MessageIds.Should().OnlyContain(
        id => id == scenario.MessageId);
}
```

EXECUTABLE ASSET 27-AK - CRASH AFTER CONFIRM

```
[Fact]
public async Task RELAY_CRASH_002_confirm_then_crash_may_duplicate()
{
    var scenario = await lab.StartAsync(
        crashAt: RelayCheckpoint.ConfirmObserved);
    await scenario.WaitForCrashAsync();

    await scenario.AdvanceDatabaseTimePastLeaseAsync();
    await scenario.RestartAsync("relay-replacement");
    await scenario.AssertEventuallyDispatchedAsync();

    scenario.PublishedBodies.Should().OnlyContain(
        body => body.SequenceEqual(scenario.FrozenBody));
    scenario.MessageIds.Distinct().Should().Equal(
        scenario.MessageId);
    scenario.PublishedBodies.Count.Should().BeInRange(1, 2);
}
```

HONEST ORACLE

The second test accepts one or two broker deliveries, requires one MessageId and identical bytes, and requires the durable row to converge to Dispatched. A test that insists on exactly one delivery would encode a false guarantee.

BACKPRESSURE AND SHUTDOWN

Stop new claims before the lease budget is exhausted

Broker flow control and confirm latency can stall publishers. The dispatcher caps claimed rows, outstanding confirms and shutdown drain time so it does not own more work than it can safely finish.

EXECUTABLE ASSET 27-AL - BOUNDED SHUTDOWN

```
public sealed partial class OutboxDispatcherService
{
    public override async Task StopAsync(
        CancellationToken cancellationToken)
    {
        _acceptNewClaims.Cancel();
        using var drain = CancellationTokenSource
            .CreateLinkedTokenSource(cancellationToken);
        drain.CancelAfter(
            _options.Value.ShutdownDrainBudget);

        try
        {
            await _activeBatch.WaitAsync(drain.Token);
        }
        catch (OperationCanceledException)
            when (!cancellationToken.IsCancellationRequested)
        {
            // Unfinished claims remain InFlight until lease expiry.
        }

        await base.StopAsync(cancellationToken);
    }
}
```

OPERATIONAL LIMITS 27-AM - ONE DEPLOYABLE POLICY

Limit	Initial value	Evidence signal
claim batch	64	batch duration
confirm window	32	confirm latency p95/p99
lease duration	45 s	lease expiry count
confirm timeout	10 s	unknown outcomes
shutdown drain	20 s	unfinished claims
recovery sweep	128	recovery convergence
envelope maximum	256 KiB	quarantine count

CAPACITY RELATION

LeaseDuration must exceed the measured worst-case claim-to-result interval with margin. Increasing it hides slow recovery; decreasing it risks two active publishers. The value is an evidence-backed operating decision.

OPERATIONAL EVIDENCE

Metrics describe backlog, ambiguity and recovery

The relay emits bounded-cardinality metrics and structured logs without payloads. Readiness fails only when the service's declared policy requires an available relay; liveness remains process-local.

EXECUTABLE ASSET 27-AN - RELAY MEASUREMENTS

```
public static class OutboxRelayTelemetry
{
    public static readonly Meter Meter =
        new("Nexus1.Messaging.OutboxRelay");

    public static readonly Counter<long> Claims =
        Meter.CreateCounter<long>("outbox.claims");
    public static readonly Counter<long> Outcomes =
        Meter.CreateCounter<long>("outbox.outcomes");
    public static readonly Histogram<double> ConfirmMs =
        Meter.CreateHistogram<double>(
            "outbox.confirm.duration", "ms");
    public static readonly Histogram<double> OldestPendingSec =
        Meter.CreateHistogram<double>(
            "outbox.oldest_pending.age", "s");
    public static readonly Counter<long> ExpiredLeases =
        Meter.CreateCounter<long>(
            "outbox.expired_leases");
}

// Allowed tags: producer, event_type, outcome_code.
// Forbidden tags: message_id, correlation_id, payload, exception text.
```

EVIDENCE THRESHOLDS 27-AO - ALERTING SURFACE

Signal	Warning	Critical
oldest Pending/Retryable age	> 60 s	> 300 s
confirm latency p99	> 2 s	> 8 s
Unknown outcome ratio	> 0.5% / 10 min	> 2% / 10 min
expired leases	> 0 / 15 min	sustained growth
quarantined rows	any new row	repeated event type
claim batch duration	> 5 s	> lease / 2

SECURITY BOUNDARY

The dispatcher principal reads and updates only messaging.OutboxMessage. The broker principal publishes only to the declared exchange. Neither has consumer permissions or cross-service database access.

ACCEPTANCE EVIDENCE

The ★ gate fails closed when runtime proof is absent

A manifest links each claim to an executable asset, selected provider, broker version, migration digest, test count and timestamp. Proposed code is not promoted to verified behavior merely because the PDF rendered successfully.

EXECUTABLE ASSET 27-AP - EVIDENCE MANIFEST

```
{
  "chapter": 27,
  "decision": "ADR-025",
  "claimSet": "outbox-relay-v1",
  "database": {
    "provider": "sqlserver",
    "migrationDigest": "<required>",
    "tests": {
      "claimExclusion": "<required>",
      "expiredLeaseRecovery": "<required>",
      "ownerVersionGuard": "<required>"
    }
  },
  "broker": {
    "product": "rabbitmq",
    "version": "<required>",
    "queueType": "<required>",
    "tests": {
      "routableConfirm": "<required>",
      "mandatoryReturn": "<required>",
      "outageRecovery": "<required>"
    }
  },
  "crashTests": {
    "afterClaim": "<required>",
    "afterConfirm": "<required>"
  },
  "zeroDiscoveryRejected": true,
  "generatedAtUtc": "<required>"
}
```

EXECUTABLE ASSET 27-AQ - FAIL-CLOSED VALIDATOR

```
public static void DemandChapter27Evidence(
    Chapter27Evidence evidence)
{
    Evidence.RequireSha256(evidence.MigrationDigest);
    Evidence.RequireVersion(evidence.BrokerVersion);
    Evidence.RequireNonZero(evidence.ClaimTests);
    Evidence.RequireNonZero(evidence.ConfirmTests);
    Evidence.RequireNonZero(evidence.CrashTests);
    Evidence.Require(evidence.MandatoryReturnPassed);
    Evidence.Require(evidence.SameMessageIdReplayPassed);
    Evidence.RequireFresh(
        evidence.GeneratedAtUtc,
        maximumAge: TimeSpan.FromDays(14));
}
```

CURRENT EVIDENCE STATUS

PENDING. This manuscript environment produced and inspected the reference assets, but did not execute a .NET solution, SQL Server migration, RabbitMQ deployment or crash laboratory. ADR-025 remains proposed until those gates pass.

ADR-025

Choose a leased at-least-once relay with publisher confirms

The decision keeps producer commits local, makes relay ownership observable and treats ambiguous publication as replayable work. It prefers loss avoidance over the false appearance of duplicate-free delivery.

DECISION RECORD 27-AR - ADR-025

Status	Proposed until Chapter 27 evidence manifest passes.
Context	Producer truth and frozen outbox intent already commit atomically. SQL Server and RabbitMQ cannot share one local transaction.
Decision	Claim with short SQL Server UPDATE...OUTPUT leases. Publish persistent messages with mandatory routing. Require publisher confirms with correlation tracking. Mark Dispatched only after routable positive confirmation. Replay Unknown outcomes with identical bytes and MessageId.
Consequences	Duplicate deliveries remain possible after ambiguous failure. Chapter 28 must make consumer effects idempotent. Expired leases and quarantines become operational obligations. Throughput is bounded by claim, confirm and lease budgets.
Rejected alternatives	Publish inside the owner transaction. Mark Dispatched before broker confirmation. Fire-and-forget publication. Allocate a new MessageId for every retry. Treat automatic connection recovery as an outgoing buffer.

REVIEW TRIGGER

Revisit ADR-025 when the database provider, broker client major version, queue durability model, topology ownership or confirmation strategy changes.

IMPLEMENTATION PATH

Build correctness first, then widen the confirm window

The sequence below preserves a working proof at every step. No optimization is accepted until claim exclusion, mandatory routing, confirm handling and crash recovery pass.

IMPLEMENTATION SEQUENCE 27-AS

1. Apply lease-shape constraints and expired-lease index per producer.
2. Implement parameterized recovery and claim SQL.
3. Prove concurrent claim exclusion on disposable SQL Server.
4. Implement rowversion/lease-owner guarded result updates.
5. Add digest verification and quarantine path.
6. Implement the simple RabbitMQ.Client confirming adapter.
7. Prove routable confirm, mandatory return and broker outage.
8. Add deterministic crash checkpoints before/after publication.
9. Prove replay uses the same MessageId and exact stored bytes.
10. Add bounded worker, drain budget, metrics and alert thresholds.
11. Run load evidence; only then introduce a pipelined confirm window.
12. Sign ADR-025 and publish the fail-closed evidence manifest.

27.3 Official technical anchors

RabbitMQ Publisher Confirms: <https://www.rabbitmq.com/docs/confirms>

RabbitMQ .NET Client API Guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>

RabbitMQ Publishers: <https://www.rabbitmq.com/docs/publishers>

RabbitMQ Reliability: <https://www.rabbitmq.com/docs/reliability>

RabbitMQ Quorum Queues: <https://www.rabbitmq.com/docs/quorum-queues>

SQL Server locking and row versioning:

<https://learn.microsoft.com/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide>

WHAT IS PROVED HERE

The design names every state, database guard, broker observation, crash window and evidence gate required by the project description. Runtime proof remains PENDING until the pinned solution executes these assets.

NEXT - CHAPTER 28

Inbox and Idempotent Consumers consumes the same stable MessageId, inserts an inbox receipt and the local business effect atomically, and proves that broker redelivery changes delivery count without repeating business meaning.

CHAPTER 27 - CLOSURE

Checkpoint: can the relay prove every state transition?

The relay is complete only when claim, publish, confirmation, uncertainty, shutdown and evidence rules answer the same way under replay and crash.

CHECKPOINT 27-A - MANDATORY ANSWERS

Question	Required answer	Executable evidence
Can two workers claim the same row?	No; the provider claim is atomic and lease-bounded.	concurrent claimant component test
When is a row Dispatched?	After mandatory route, positive broker confirm and guarded update.	confirm + state-transition trace
What follows an unknown confirm?	Replay the same Messageld and the exact frozen bytes.	connection-loss recovery test
What follows broker outage?	The row stays Pending/Retryable; owner truth remains committed.	fault injection + backlog metric
How does shutdown behave?	Stop new claims, drain bounded in-flight work, preserve leases.	bounded shutdown probe
What may the manuscript claim now?	The design is specified; runtime proof remains PENDING.	provider + broker + crash evidence manifest

EXIT CONDITION

Chapter 27 exits only when no status change depends on broker optimism, exception text or process memory. Unknown delivery remains replayable, not guessed.

Handoff to Chapter 28

The producer may now redeliver safely. The next boundary proves that a consumer can absorb that redelivery without repeating its local effect.

PART V - THE MESSAGE BACKBONE

28. Inbox and Idempotent Consumers

RabbitMQ may deliver the same public fact again after a consumer crash, connection loss or lost acknowledgement. The consumer must make repeated delivery ordinary while preventing repeated owned business meaning.

CHAPTER DECISION

Each subscription stores a durable inbox receipt in the consumer-owned database transaction that applies the local state change and stages any new outbox fact. The broker delivery is acknowledged only after that commit.

28.1 The exact guarantee

The inbox does not promise exactly-once broker delivery. It proves effect-once inside one consumer database for one ConsumerName and MessageId, for as long as the receipt identity remains retained. Unknown commit or acknowledgement outcomes intentionally produce redelivery.

GUARANTEE LEDGER 28-A - EVIDENCE AND LIMITS

Boundary	Mechanism
-----	-----
delivery -> admitted message	envelope and metadata validation
message -> active subscription	ConsumerName + MessageId identity
receipt -> owned effect	one local database transaction
owned effect -> new public fact	same transaction + producer outbox
local commit -> broker removal	manual ack on receiving channel
lost ack -> repeated delivery	existing receipt -> no repeated effect
-----	-----
Allowed claim	Explicitly not claimed
-----	-----
one committed local effect	exactly-once broker delivery
duplicate delivery is harmless	duplicate delivery cannot occur
receipt-scoped deduplication	global deduplication across consumers
stable business convergence	total ordering across messages

BOUNDARY TO CHAPTER 29

Chapter 28 decides duplicate versus first processing and establishes safe ack timing. Retry schedules, dead-letter routing, poison-message budgets and operator replay policy belong to Chapter 29.

CONSUMER SEMANTICS

Delivery count may increase; business meaning may not

The broker owns delivery attempts. The consumer owns whether a delivery creates a new local consequence. RabbitMQ's redelivered flag is evidence about transport history, not an idempotency key.

TRUTH TABLE 28-B - REPEATED DELIVERY

Delivery	Inbox	Local effect
first, transaction commits	inserted	committed once
duplicate after commit	already exists	skipped
crash before database commit	absent	rolled back
crash after commit, before ack	exists	skipped on redelivery
database commit response lost	unknown	resolved by receipt
acknowledgement lost	exists	skipped on redelivery
same MessageId, different digest	conflict	never treated duplicate
new MessageId, same business fact	new receipt	business key decides

EXECUTABLE ASSET 28-C - CONSUMER OUTCOMES

```
public abstract record ConsumeOutcome
{
    public sealed record Committed(
        Guid MessageId, string EffectCode) : ConsumeOutcome;
    public sealed record Duplicate(
        Guid MessageId, string PriorEffectCode) : ConsumeOutcome;
    public sealed record IdentityConflict(
        Guid MessageId, string Code) : ConsumeOutcome;
    public sealed record Invalid(string Code) : ConsumeOutcome;
    public sealed record Transient(string Code) : ConsumeOutcome;
    public sealed record Unknown(Guid MessageId) : ConsumeOutcome;
}

public enum DeliveryDisposition
{
    Ack,
    Retry,
    Reject
}
```

REDELIVERED IS NOT IDENTITY

A duplicate can arrive with redelivered=false after an application replay or a producer defect. A first delivery can arrive with redelivered=true after another consumer lost its channel. Always use durable message identity.

CONSUMER-OWNED IDENTITY

Deduplicate per subscription, then verify the frozen fact

The same public fact is legitimately consumed once by RootCause, Compliance, Audit and Reporting. ConsumerName scopes the receipt; MessageId identifies the public fact; the stored digest detects identity reuse with different bytes.

EXECUTABLE ASSET 28-D - ADMITTED DELIVERY VALUE

```
public readonly record struct ConsumerName
{
    public string Value { get; }

    public ConsumerName(string value)
    {
        if (string.IsNullOrEmpty(value) ||
            value.Length > 120)
            throw new InvalidConsumerName(value);
        Value = value;
    }
}

public sealed record AdmittedDelivery(
    ConsumerName Consumer,
    Guid MessageId,
    string Producer,
    string EventType,
    int SchemaVersion,
    DateTimeOffset OccurredAtUtc,
    string? CorrelationId,
    byte[] EnvelopeBytes,
    byte[] EnvelopeSha256,
    bool BrokerRedelivered);
```

EXECUTABLE ASSET 28-E - IDENTITY AND INTEGRITY RULES

```
public static class InboxIdentity
{
    public static void DemandValid(AdmittedDelivery delivery)
    {
        if (delivery.MessageId == Guid.Empty)
            throw new MissingMessageId();
        if (delivery.SchemaVersion < 1)
            throw new InvalidSchemaVersion();
        if (delivery.EnvelopeBytes is { Length: > 0 and <= 262_144 })
        {
            var actual = SHA256.HashData(
                delivery.EnvelopeBytes);
            if (CryptographicOperations.FixedTimeEquals(
                actual, delivery.EnvelopeSha256))
                return;
        }
        throw new InvalidEnvelopeDigest();
    }
}
```

TWO DUPLICATE CLASSES

Transport duplicate means the same consumer and MessageId. Semantic duplicate means a different MessageId asks for an already satisfied business outcome. The inbox handles the first; domain uniqueness handles the second.

PROVIDER SCHEMA

A receipt exists only for a completed local decision

The reference schema has no durable Processing state. The receipt, owned state and optional outbox rows either commit together or disappear together. In-progress work remains represented by an unacknowledged broker delivery.

EXECUTABLE ASSET 28-F - SQL SERVER INBOX TABLE

```
CREATE SCHEMA messaging AUTHORIZATION dbo;
GO

CREATE TABLE messaging.InboxReceipt
(
    ConsumerName    varchar(120)    NOT NULL,
    MessageId       uniqueidentifier NOT NULL,
    Producer        varchar(80)     NOT NULL,
    EventType       varchar(160)    NOT NULL,
    SchemaVersion   int             NOT NULL,
    EnvelopeSha256  binary(32)      NOT NULL,
    OccurredAtUtc   datetimeoffset(7) NOT NULL,
    ReceivedAtUtc   datetimeoffset(7) NOT NULL,
    CompletedAtUtc  datetimeoffset(7) NOT NULL,
    OutcomeCode     varchar(80)     NOT NULL,
    CorrelationId   varchar(128)    NULL,

    CONSTRAINT PK_InboxReceipt
        PRIMARY KEY (ConsumerName, MessageId),
    CONSTRAINT CK_Inbox_SchemaVersion
        CHECK (SchemaVersion >= 1),
    CONSTRAINT CK_Inbox_CompletionOrder
        CHECK (CompletedAtUtc >= ReceivedAtUtc)
);

CREATE INDEX IX_InboxReceipt_Completed
ON messaging.InboxReceipt
(CompletedAtUtc, ConsumerName, MessageId);
```

ONE TABLE PER CONSUMER DATABASE

ComplianceDb, AuditDb, ReportingDb and RootCauseDb apply their own migration. There is no central NEXUS-1 inbox database and no cross-service receipt lookup.

EF CORE MAPPING

The unique key is the race arbiter

A preliminary read avoids ordinary duplicate work, but it cannot close a concurrent race. The composite primary key decides which transaction may commit.

EXECUTABLE ASSET 28-G - RECEIPT ROW AND MAPPING

```
internal sealed class InboxReceiptRow
{
    public string ConsumerName { get; init; } = null!;
    public Guid MessageId { get; init; }
    public string Producer { get; init; } = null!;
    public string EventType { get; init; } = null!;
    public int SchemaVersion { get; init; }
    public byte[] EnvelopeSha256 { get; init; } = null!;
    public DateTimeOffset OccurredAtUtc { get; init; }
    public DateTimeOffset ReceivedAtUtc { get; init; }
    public DateTimeOffset CompletedAtUtc { get; init; }
    public string OutcomeCode { get; init; } = null!;
    public string? CorrelationId { get; init; }
}

internal sealed class InboxReceiptConfiguration
    : IEntityTypeConfiguration<InboxReceiptRow>
{
    public void Configure(
        EntityTypeBuilder<InboxReceiptRow> builder)
    {
        builder.ToTable("InboxReceipt", "messaging",
            table =>
            {
                table.HasCheckConstraint(
                    "CK_Inbox_SchemaVersion",
                    "[SchemaVersion] >= 1");
                table.HasCheckConstraint(
                    "CK_Inbox_CompletionOrder",
                    "[CompletedAtUtc] >= [ReceivedAtUtc]");
            });
        builder.HasKey(x => new
            { x.ConsumerName, x.MessageId });
        builder.Property(x => x.ConsumerName)
            .HasMaxLength(120).IsUnicode(false);
        builder.Property(x => x.EnvelopeSha256)
            .HasColumnType("binary(32)");
        builder.Property(x => x.OutcomeCode)
            .HasMaxLength(80).IsUnicode(false);
        builder.HasIndex(x => new
            { x.CompletedAtUtc, x.ConsumerName, x.MessageId });
    }
}
```

PROVIDER CLASSIFICATION

The duplicate branch is entered only when the selected SQL Server classifier identifies PK_InboxReceipt. String matching on exception text is rejected.

APPLICATION BOUNDARY

The handler returns owned changes, never broker operations

Application code interprets the admitted contract and changes only consumer-owned state. It may stage a finite public fact through the existing outbox contract, but it cannot acknowledge, reject, republish or call another service database.

EXECUTABLE ASSET 28-H - CONSUMER APPLICATION PORT

```
public interface IIntegrationEventHandler<TContract>
{
    Task<ConsumerDecision> HandleAsync(
        TContract contract,
        ConsumerContext context,
        CancellationToken cancellationToken);
}

public sealed record ConsumerContext(
    Guid MessageId,
    string Producer,
    DateTimeOffset OccurredAtUtc,
    string? CorrelationId);

public sealed record ConsumerDecision(
    string OutcomeCode,
    IReadOnlyList<OwnedStateChange> StateChanges,
    IReadOnlyList<SerializedPublicFact> PublicFacts)
{
    public static ConsumerDecision NoChange(string code) =>
        new(code, [], []);
}
```

ARCHITECTURE RULES 28-I - FORBIDDEN DEPENDENCIES

```
[Fact]
public void INBOX_ARCH_001_handler_has_no_broker_dependency()
{
    Types.InAssembly(Compliance.Application)
        .That().ImplementInterface(
            typeof(IIntegrationEventHandler<>))
        .ShouldNot().HaveDependencyOn("RabbitMQ.Client")
        .GetResult().IsSuccessful.Should().BeTrue();
}

[Fact]
public void INBOX_ARCH_002_consumer_does_not_reference_producer_db()
{
    Types.InAssembly(Compliance.Infrastructure)
        .ShouldNot().HaveDependencyOn(
            "Nexus1.RootCause.Infrastructure")
        .GetResult().IsSuccessful.Should().BeTrue();
}
```

ATOMIC LOCAL UNIT

Receipt, business state and outgoing intent share SaveChanges

The transaction contains only the consumer's database. Contract parsing and canonical serialization happen before SaveChanges. No HTTP or broker call is made while the local transaction is open.

EXECUTABLE ASSET 28-J - ONE CONSUMER DATABASE COMMIT

```
public sealed class EfInboxConsumer<TDbContext, TContract>(
    TDbContext db,
    IIntegrationEventHandler<TContract> handler,
    IOutboxHeaderWriter headers,
    IClock clock)
    where TDbContext : DbContext, IConsumerDbContext
{
    public async Task<ConsumeOutcome> ConsumeAsync(
        AdmittedDelivery delivery,
        TContract contract,
        CancellationToken cancellationToken)
    {
        var receivedAt = clock.UtcNow;
        var context = new ConsumerContext(
            delivery.MessageId,
            delivery.Producer,
            delivery.OccurredAtUtc,
            delivery.CorrelationId);

        var decision = await handler.HandleAsync(
            contract, context, cancellationToken);

        foreach (var change in decision.StateChanges)
            change.Apply(db);
        foreach (var fact in decision.PublicFacts)
            db.OutboxMessages.Add(
                OutboxMessageRow.Pending(
                    fact,
                    headers.ForConsumed(delivery),
                    clock.UtcNow));

        db.InboxReceipts.Add(new InboxReceiptRow
        {
            ConsumerName = delivery.Consumer.Value,
            MessageId = delivery.MessageId,
            Producer = delivery.Producer,
            EventType = delivery.EventType,
            SchemaVersion = delivery.SchemaVersion,
            EnvelopeSha256 =
                delivery.EnvelopeSha256.ToArray(),
            OccurredAtUtc = delivery.OccurredAtUtc,
            ReceivedAtUtc = receivedAt,
            CompletedAtUtc = clock.UtcNow,
            OutcomeCode = decision.OutcomeCode,
            CorrelationId = delivery.CorrelationId
        });

        await db.SaveChangesAsync(cancellationToken);
        return new ConsumeOutcome.Committed(
            delivery.MessageId, decision.OutcomeCode);
    }
}
```

ATOMICITY SCOPE

EF Core wraps one SaveChanges call in a transaction when the provider supports transactions. If any receipt, owned row or outbox insert fails, none may remain.

REFERENCE CONSUMER

One verdict opens at most one Compliance review

The example demonstrates consumer mechanics without claiming that the full Part VI slice already exists. Chapter 34 will later assemble the real RootCause-to-Compliance flow and its deployment evidence.

EXECUTABLE ASSET 28-K - VERSIONED CONTRACT HANDLER

```
public sealed class RootCauseVerdictIssuedV1Handler(
    IComplianceReviewRepository reviews,
    IPublicFactSerializer serializer,
    IClock clock)
    : IIntegrationEventHandler<RootCauseVerdictIssuedV1>
{
    public async Task<ConsumerDecision> HandleAsync(
        RootCauseVerdictIssuedV1 contract,
        ConsumerContext context,
        CancellationToken cancellationToken)
    {
        var existing = await reviews.FindBySourceVerdictAsync(
            contract.VerdictId, cancellationToken);
        if (existing is not null)
            return ConsumerDecision.NoChange(
                "review-already-exists");

        var review = ComplianceReview.Open(
            ComplianceReviewId.New(),
            contract.VerdictId,
            contract.RootCauseCaseId,
            contract.UnitId,
            context.MessageId,
            clock.UtcNow);

        var fact = serializer.Serialize(
            ComplianceReviewOpenedV1.Describe(
                review, context.CorrelationId),
            ComplianceReviewOpenedV1.From(review));

        return new ConsumerDecision(
            "review-opened",
            [OwnedStateChange.For(review)],
            [fact]);
    }
}
```

EXECUTABLE ASSET 28-L - BUSINESS UNIQUENESS

```
builder.HasIndex(x => x.SourceVerdictId)
    .HasDatabaseName(
        "UX_ComplianceReview_SourceVerdictId")
    .IsUnique();

// Inbox key: same public message for this subscription.
// Business key: same verdict meaning under any message identity.
```

NO SIDE EFFECT OUTSIDE THE DATABASE

Email, HTTP callbacks, audit writes and broker publication are not invoked inside HandleAsync. If they are required, the consumer stages owned intent or a public fact and lets another durable boundary execute it.

DUPLICATE RESOLUTION

Read fast, but let the constraint decide the race

An existing matching receipt can return Duplicate before domain work. Two simultaneous first deliveries can both miss that read, so the PK conflict must be classified and resolved with a fresh DbContext.

EXECUTABLE ASSET 28-M - FAST DUPLICATE CHECK

```
public static async Task<ConsumeOutcome?> TryReadReceiptAsync(
    IConsumerDbContext db,
    AdmittedDelivery delivery,
    CancellationToken cancellationToken)
{
    var existing = await db.InboxReceipts
        .AsNoTracking()
        .SingleOrDefaultAsync(
            x => x.ConsumerName == delivery.Consumer.Value &&
                x.MessageId == delivery.MessageId,
            cancellationToken);

    if (existing is null)
        return null;

    if (!CryptographicOperations.FixedTimeEquals(
        existing.EnvelopeSha256,
        delivery.EnvelopeSha256))
        return new ConsumeOutcome.IdentityConflict(
            delivery.MessageId,
            "message-id-digest-conflict");

    return new ConsumeOutcome.Duplicate(
        delivery.MessageId,
        existing.OutcomeCode);
}
```

EXECUTABLE ASSET 28-N - UNIQUE-RACE RESOLVER

```
public async Task<ConsumeOutcome> ExecuteAsync(
    AdmittedDelivery delivery,
    TContract contract,
    CancellationToken cancellationToken)
{
    var known = await TryReadReceiptAsync(
        db, delivery, cancellationToken);
    if (known is not null)
        return known;

    try
    {
        return await consumer.ConsumeAsync(
            delivery, contract, cancellationToken);
    }
    catch (DbUpdateException exception)
        when (errors.IsInboxPrimarykey(exception) ||
            errors.IsConsumerBusinessKey(exception))
    {
        return await resolver.ResolveAsync(
            delivery, cancellationToken);
    }
}
```

FRESH UNIT OF WORK

After a failed SaveChanges, the resolver opens a new DbContext. It never continues with domain objects or tracked rows from the losing transaction.

AMBIGUOUS COMMIT

Unknown means do not acknowledge

SQL Server may commit before the client loses the response. The consumer cannot call that failure a rollback. It resolves by durable receipt identity or allows broker redelivery to repeat the same decision path.

EXECUTABLE ASSET 28-O - UNKNOWN COMMIT RESOLVER

```
public sealed class InboxCommitResolver<TDbContext>(
    IDbContextFactory<TDbContext> contexts)
    where TDbContext : DbContext, IConsumerDbContext
{
    public async Task<ConsumeOutcome> ResolveAsync(
        AdmittedDelivery delivery,
        CancellationToken cancellationToken)
    {
        await using var db =
            await contexts.CreateDbContextAsync(
                cancellationToken);
        var receipt = await db.InboxReceipts
            .AsNoTracking()
            .SingleOrDefaultAsync(
                x => x.ConsumerName ==
                    delivery.Consumer.Value &&
                    x.MessageId == delivery.MessageId,
                cancellationToken);

        if (receipt is null)
            return new ConsumeOutcome.Unknown(
                delivery.MessageId);

        return CryptographicOperations.FixedTimeEquals(
            receipt.EnvelopeSha256,
            delivery.EnvelopeSha256)
            ? new ConsumeOutcome.Duplicate(
                delivery.MessageId,
                receipt.OutcomeCode)
            : new ConsumeOutcome.IdentityConflict(
                delivery.MessageId,
                "message-id-digest-conflict");
    }
}
```

DECISION TABLE 28-P - COMMIT OBSERVATION

Observation	ConsumeOutcome	Ack?
SaveChanges success	Committed	yes
matching receipt found	Duplicate	yes
PK race, matching receipt	Duplicate	yes
response lost, receipt found	Duplicate	yes
response lost, receipt absent	Unknown	no
same key, different digest	IdentityConflict	no

RABBITMQ DELIVERY ADAPTER

Copy first, process, then acknowledge on the same channel

RabbitMQ.Client 7.x exposes the body as `ReadOnlyMemory<byte>`. The handler copies it before returning, uses manual acknowledgement, and keeps the receiving channel long-lived.

EXECUTABLE ASSET 28-Q - MANUAL-ACK SUBSCRIPTION

```
public sealed class RabbitMqInboxSubscription(
    IChannel channel,
    IRabbitDeliveryMapper mapper,
    IConsumerEndpoint endpoint,
    RabbitMqConsumerOptions options)
{
    public async Task<string> StartAsync(
        CancellationToken cancellationToken)
    {
        await channel.BasicQosAsync(
            prefetchSize: 0,
            prefetchCount: options.PrefetchCount,
            global: false,
            cancellationToken);

        var consumer = new AsyncEventingBasicConsumer(channel);
        consumer.ReceivedAsync += async (_, args) =>
        {
            DeliveryDisposition disposition;
            try
            {
                var body = args.Body.ToArray();
                var delivery = mapper.Admit(
                    args.BasicProperties,
                    body,
                    args.Redelivered);
                disposition = await endpoint.HandleAsync(
                    delivery, cancellationToken);
            }
            catch (Exception exception)
            {
                when (exception is MissingMessageId or
                    EnvelopePropertyMismatch or
                    InvalidSchemaVersion or
                    InvalidEnvelopeDigest)
                {
                    disposition = DeliveryDisposition.Reject;
                }
            }

            switch (disposition)
            {
                case DeliveryDisposition.Ack:
                    await channel.BasicAckAsync(
                        args.DeliveryTag,
                        multiple: false,
                        cancellationToken);
                    break;
                case DeliveryDisposition.Retry:
                    await channel.BasicNackAsync(
                        args.DeliveryTag,
                        multiple: false,
                        requeue: true,
                        cancellationToken);
                    break;
                case DeliveryDisposition.Reject:
                    await channel.BasicRejectAsync(
                        args.DeliveryTag,
                        requeue: false,
                        cancellationToken);
                    break;
            }
        };

        return await channel.BasicConsumeAsync(
            options.QueueName,
            autoAck: false,
            consumer,
            cancellationToken);
    }
}
```

CHANNEL RULE

Delivery tags are scoped to a channel. Ack, nack or reject must use the same channel that received the delivery; using a different channel is a protocol error.

DISPOSITION POLICY

Only durable success and matching duplicate are acknowledged

The endpoint maps typed processing outcomes to a broker action. It never acknowledges an unknown database commit, and it never treats an identity conflict as an ordinary duplicate.

EXECUTABLE ASSET 28-R - OUTCOME TO BROKER ACTION

```
public static class ConsumerDisposition
{
    public static DeliveryDisposition For(
        ConsumeOutcome outcome) => outcome switch
    {
        ConsumeOutcome.Committed =>
            DeliveryDisposition.Ack,
        ConsumeOutcome.Duplicate =>
            DeliveryDisposition.Ack,
        ConsumeOutcome.Transient =>
            DeliveryDisposition.Retry,
        ConsumeOutcome.Unknown =>
            DeliveryDisposition.Retry,
        ConsumeOutcome.Invalid =>
            DeliveryDisposition.Reject,
        ConsumeOutcome.IdentityConflict =>
            DeliveryDisposition.Reject,
        _ => throw new UnreachableException()
    };
}
```

FAILURE CLASSIFICATION 28-S - CHAPTER BOUNDARY

Failure	Chapter 28 action
invalid envelope/version	reject; preserve evidence
digest/MessageId conflict	reject; security alert
domain no-op/already satisfied	commit receipt; ack
database transient	retry; no ack
database commit unknown	retry; no ack
process/channel loss	no action; broker requeues
handler defect	reject/retry policy deferred to Chapter 29

DEAD-LETTER CAVEAT

Reject with `requeue=false` is useful only when the declared queue policy routes the message to owned evidence. Chapter 29 must prove the complete dead-letter and poison-message path before production use.

CONCURRENCY AND PREFETCH

Scale with bounded channel ownership

The correctness baseline uses sequential dispatch on each channel and multiple consumer instances for parallelism. This keeps acknowledgements ordered per channel and leaves the database unique key as the final duplicate arbiter.

EXECUTABLE ASSET 28-T - BOUNDED CONSUMER OPTIONS

```
public sealed class RabbitMqConsumerOptions
{
    public required string QueueName { get; init; }
    public ushort PrefetchCount { get; init; } = 32;
    public ushort ConsumerDispatchConcurrency { get; init; } = 1;
    public int MaxEnvelopeBytes { get; init; } = 262_144;
    public TimeSpan ShutdownDrainBudget { get; init; } =
        TimeSpan.FromSeconds(20);
}

public sealed class RabbitMqConsumerOptionsValidator
    : IValidateOptions<RabbitMqConsumerOptions>
{
    public ValidateOptionsResult Validate(
        string? name, RabbitMqConsumerOptions value)
    {
        if (string.IsNullOrEmpty(value.QueueName))
            return ValidateOptionsResult.Fail("Queue required.");
        if (value.PrefetchCount is < 1 or > 500)
            return ValidateOptionsResult.Fail(
                "PrefetchCount must be 1..500.");
        if (value.ConsumerDispatchConcurrency != 1)
            return ValidateOptionsResult.Fail(
                "Reference channel requires sequential dispatch.");
        return ValidateOptionsResult.Success;
    }
}
```

CAPACITY RELATION 28-U - INITIAL OPERATING POINT

```
consumer instances x prefetch <= database concurrency budget

RootCause:    4 instances x 32 = 128 unacknowledged maximum
Compliance:   2 instances x 16 = 32 unacknowledged maximum
Audit:        4 instances x 64 = 256 unacknowledged maximum
Reporting:    4 instances x 32 = 128 unacknowledged maximum

Values are starting hypotheses, not universal defaults.
```

CONCURRENT CALLBACKS

If dispatch concurrency later exceeds one, acknowledgements on a shared channel require explicit serialization and `multiple=false`. Load evidence must precede that optimization.

CONTRACT ADMISSION

Reject ambiguity before domain work

Transport properties and the Chapter 20 envelope must agree. MessageId, event type, schema version, producer, occurrence time and body digest are validated before any owned aggregate is loaded.

EXECUTABLE ASSET 28-V - STRICT MAPPER

```
public sealed class RabbitDeliveryMapper(
    IVersionedEnvelopeReader envelopes,
    ConsumerName consumer)
    : IRabbitDeliveryMapper
{
    public AdmittedDelivery Admit(
        IReadOnlyBasicProperties properties,
        byte[] body,
        bool redelivered)
    {
        if (!Guid.TryParse(
            properties.MessageId, out var wireMessageId))
            throw new MissingMessageId();

        var envelope = envelopes.ReadV1(body);
        if (envelope.MessageId != wireMessageId)
            throw new EnvelopePropertyMismatch(
                "message-id");
        if (!StringComparer.Ordinal.Equals(
            envelope.EventType, properties.Type))
            throw new EnvelopePropertyMismatch(
                "event-type");

        var delivery = new AdmittedDelivery(
            consumer,
            envelope.MessageId,
            envelope.Producer,
            envelope.EventType,
            envelope.SchemaVersion,
            envelope.OccurredAtUtc,
            properties.CorrelationId,
            body,
            SHA256.HashData(body),
            redelivered);
        InboxIdentity.DemandValid(delivery);
        return delivery;
    }
}
```

IDENTITY COLLISION IS NOT RETRY NOISE

If the same ConsumerName and MessageId arrive with different bytes, processing stops and evidence is preserved. Silently accepting the later body would let one public identity describe two histories.

SEMANTIC IDEMPOTENCY

Inbox uniqueness is necessary but not sufficient

A defective producer can publish the same business fact under two MessageIds. Consumer aggregates therefore enforce their own semantic keys in addition to the transport inbox.

EXECUTABLE ASSET 28-W - DOMAIN-LEVEL DUPLICATE DECISION

```
public sealed class ComplianceReview
{
    public ComplianceReviewId Id { get; private set; }
    public RootCauseVerdictId SourceVerdictId { get; private set; }
    public Guid SourceMessageId { get; private set; }
    public ComplianceReviewState State { get; private set; }

    public static ComplianceReview Open(
        ComplianceReviewId id,
        RootCauseVerdictId verdictId,
        RootCauseCaseId caseId,
        UnitId unitId,
        Guid sourceMessageId,
        DateTimeOffset openedAtUtc)
    {
        if (sourceMessageId == Guid.Empty)
            throw new MissingSourceMessageIdentity();
        return new ComplianceReview(
            id, verdictId, caseId, unitId,
            sourceMessageId, openedAtUtc);
    }
}
```

TWO-KEY ORACLE 28-X - REQUIRED COUNTS

Scenario	Inbox rows	Review rows	New outbox rows
first MessageId / verdict	1	1	1
same MessageId / same bytes	1	1	1
new MessageId / same verdict	2	1	1
same MessageId / new bytes	1	1	1 + conflict
rollback during first attempt	0	0	0

BUSINESS NO-OP STILL RECORDS RECEIPT

When a new MessageId describes an already satisfied verdict, the transaction stores an inbox receipt with outcome review-already-exists. Future delivery does not re-run the lookup or emit another fact.

CRASH SEMANTICS

Every interruption has one safe broker consequence

The broker and consumer database do not share a transaction. Safety comes from local atomicity plus delayed acknowledgement, not from pretending the final ack is atomic with SQL Server.

CRASH MATRIX 28-Y - CONSUMER INTERRUPTION POINTS

Crash point	Database	Broker
before validation	unchanged	unacked
after validation, before handler	unchanged	unacked
after handler, before SaveChanges	unchanged	unacked
during database commit	unknown	unacked
after commit, before ack	receipt + effect	unacked
during ack write	receipt + effect	unknown
after broker accepts ack	receipt + effect	removed
Recovery		
No receipt: process again under the same MessageId.		
Matching receipt: skip owned work and ack the delivery.		
Conflicting digest: reject and preserve evidence.		
Unknown ack: tolerate redelivery; never delete the receipt.		

EXECUTABLE ASSET 28-Z - DETERMINISTIC CHECKPOINTS

```
public enum ConsumerCheckpoint
{
    BodyCopied,
    ContractAdmitted,
    HandlerCompleted,
    SaveStarted,
    SaveCommitted,
    AckStarted,
    AckCompleted
}

public interface IConsumerFaultPoint
{
    ValueTask HitAsync(
        ConsumerCheckpoint checkpoint,
        Guid messageId,
        CancellationToken cancellationToken);
}
```

THE UNAVOIDABLE REDELIVERY WINDOW

SaveCommitted -> AckCompleted cannot be closed by a local transaction. The inbox makes this window safe by converting the next delivery into Duplicate.

FAST PROOF

Unit tests pin identity and disposition

These tests prove deterministic rules without claiming provider or broker behavior. They use exact bytes and typed outcomes rather than mock call counts alone.

EXECUTABLE ASSET 28-AA - MATCHING DUPLICATE

```
[Fact]
public async Task INBOX_UNIT_001_matching_receipt_is_duplicate()
{
    var delivery = InboxFixture.Delivery(
        consumer: "compliance.verdict-issued.v1",
        messageId: Guid.Parse(
            "4974f50f-855e-4538-8dda-d0f42accff20"));
    var db = InboxFixture.WithReceipt(
        delivery, outcome: "review-opened");

    var outcome = await TryReadReceiptAsync(
        db, delivery, CancellationToken.None);

    outcome.Should().Be(
        new ConsumeOutcome.Duplicate(
            delivery.MessageId, "review-opened"));
}
```

EXECUTABLE ASSET 28-AB - DIGEST CONFLICT

```
[Fact]
public async Task INBOX_UNIT_002_same_id_new_bytes_is_conflict()
{
    var original = InboxFixture.Delivery(
        bytes: "original"u8.ToArray());
    var replay = original with
    {
        EnvelopeBytes = "changed"u8.ToArray(),
        EnvelopeSha256 =
            SHA256.HashData("changed"u8.ToArray())
    };
    var db = InboxFixture.WithReceipt(original);

    var outcome = await TryReadReceiptAsync(
        db, replay, CancellationToken.None);

    outcome.Should().BeOfType<
        ConsumeOutcome.IdentityConflict>();
}
```

EXECUTABLE ASSET 28-AC - ACK POLICY

```
[Theory]
[MemberData(nameof(DispositionCases))]
public void INBOX_UNIT_003_outcomes_map_safely(
    ConsumeOutcome outcome,
    DeliveryDisposition expected)
{
    ConsumerDisposition.For(outcome).Should().Be(expected);
}
```

PROVIDER-BACKED ATOMICITY

Success commits three records; failure commits none

The SQL Server component lane applies production migrations and counts the inbox receipt, ComplianceReview and new outbox fact. EF InMemory and SQLite cannot substitute for the selected constraints and provider error numbers.

EXECUTABLE ASSET 28-AD - ATOMIC SUCCESS

```
[Fact]
public async Task INBOX_DB_001_receipt_effect_and_fact_commit_once()
{
    var delivery = await fixture.VerdictDeliveryAsync();

    var outcome = await fixture.ConsumeAsync(delivery);

    outcome.Should().BeOfType<
        ConsumeOutcome.Committed>();
    await fixture.Database.AssertInboxCountAsync(
        delivery.Consumer, delivery.MessageId, 1);
    await fixture.Database.AssertReviewCountAsync(
        delivery.Contract.VerdictId, 1);
    await fixture.Database.AssertOutboxCountAsync(
        delivery.ExpectedFactKey, 1);
    fixture.Acknowledgements.Should().BeEmpty();
}
```

EXECUTABLE ASSET 28-AE - ATOMIC ROLLBACK

```
[Fact]
public async Task INBOX_DB_002_outbox_failure_rolls_back_all()
{
    var delivery = await fixture.VerdictDeliveryAsync();
    fixture.Faults.FailCommand(
        "INSERT INTO [messaging].[OutboxMessage]");

    await Assert.ThrowsAsync<InjectedDatabaseFailure>(
        () => fixture.ConsumeAsync(delivery));

    await fixture.Database.AssertInboxCountAsync(
        delivery.Consumer, delivery.MessageId, 0);
    await fixture.Database.AssertReviewCountAsync(
        delivery.Contract.VerdictId, 0);
    await fixture.Database.AssertOutboxCountAsync(
        delivery.ExpectedFactKey, 0);
}
```

NO PREMATURE ACK

The component test stops at the database boundary. The broker adapter may ack only after it receives Committed or matching Duplicate.

CONCURRENT DUPLICATE PROOF

Two transactions race; one business effect wins

A deterministic barrier holds two consumers after their fast-path reads and before SaveChanges. Both execute domain code in isolated memory; the composite key permits only one database commit.

EXECUTABLE ASSET 28-AF - SIMULTANEOUS SAME-MESSAGE DELIVERY

```
[Fact]
public async Task INBOX_DB_003_concurrent_duplicate_commits_once()
{
    var delivery = await fixture.VerdictDeliveryAsync();
    var barrier = fixture.Faults.Barrier(
        ConsumerCheckpoint.SaveStarted,
        participants: 2);

    var left = fixture.ConsumeInNewContextAsync(
        delivery, barrier);
    var right = fixture.ConsumeInNewContextAsync(
        delivery, barrier);
    var outcomes = await Task.WhenAll(left, right);

    outcomes.Should().ContainSingle(
        x => x is ConsumeOutcome.Committed);
    outcomes.Should().ContainSingle(
        x => x is ConsumeOutcome.Duplicate);
    await fixture.Database.AssertInboxCountAsync(
        delivery.Consumer, delivery.MessageId, 1);
    await fixture.Database.AssertReviewCountAsync(
        delivery.Contract.VerdictId, 1);
    await fixture.Database.AssertOutboxCountAsync(
        delivery.ExpectedFactKey, 1);
}
```

EXECUTABLE ASSET 28-AG - NEW IDENTITIES, SAME VERDICT

```
[Fact]
public async Task INBOX_DB_004_semantic_duplicate_is_no_op()
{
    var first = await fixture.VerdictDeliveryAsync();
    var second = first.WithNewMessageId();

    await fixture.ConsumeAsync(first);
    await fixture.ConsumeAsync(second);

    await fixture.Database.AssertInboxTotalAsync(2);
    await fixture.Database.AssertReviewCountAsync(
        first.Contract.VerdictId, 1);
    await fixture.Database.AssertOutboxCountAsync(
        first.ExpectedFactKey, 1);
}
```

BROKER-BACKED REDELIVERY PROOF

A lost acknowledgement changes delivery count, not owned state

A disposable RabbitMQ deployment uses the production queue type, manual ack and real consumer channel. The fixture closes the channel after SQL commit and before ack so the broker redelivers the message.

EXECUTABLE ASSET 28-AH - LOST ACK REDELIVERY

```
[Fact]
public async Task INBOX_RM0_001_lost_ack_repeats_delivery_not_effect()
{
    var message = await fixture.PublishVerdictAsync();
    fixture.Faults.CloseChannelAt(
        ConsumerCheckpoint.AckStarted);

    await fixture.StartConsumerAsync();
    await fixture.WaitForChannelClosureAsync();
    await fixture.StartConsumerAsync();
    await fixture.WaitUntilQueueEmptyAsync();

    fixture.ObservedDeliveries.Should().Be(2);
    fixture.ObservedRedelivered.Should().Contain(true);
    await fixture.Database.AssertInboxCountAsync(
        fixture.ConsumerName, message.MessageId, 1);
    await fixture.Database.AssertReviewCountAsync(
        message.VerdictId, 1);
    await fixture.Database.AssertOutboxCountAsync(
        message.ExpectedFactKey, 1);
    fixture.SuccessfulAcks.Should().Be(1);
}
```

EXECUTABLE ASSET 28-AI - REDELIVERED FLAG IS NOT REQUIRED

```
[Fact]
public async Task INBOX_RM0_002_republished_fact_is_deduplicated()
{
    var message = await fixture.PublishVerdictAsync();
    await fixture.PublishExactBytesAgainAsync(message);

    await fixture.StartConsumerAsync();
    await fixture.WaitUntilQueueEmptyAsync();

    fixture.ObservedDeliveries.Should().Be(2);
    await fixture.Database.AssertInboxCountAsync(
        fixture.ConsumerName, message.MessageId, 1);
    await fixture.Database.AssertReviewCountAsync(
        message.VerdictId, 1);
}
```

BROKER ORACLE

The test expects two deliveries and one effect. Requiring one delivery would test a guarantee RabbitMQ and the consumer transaction do not make.

UNKNOWN DATABASE OUTCOME

Lose the commit response, then let redelivery reconcile

The fault harness allows SQL Server to commit and drops the client response. The consumer returns Unknown, sends no ack, and the next delivery finds the durable receipt.

EXECUTABLE ASSET 28-AJ - COMMITTED RESPONSE LOSS

```
[Fact]
public async Task INBOX_CRASH_001_unknown_commit_converges()
{
    var message = await fixture.PublishVerdictAsync();
    fixture.DatabaseFaults
        .DropCommitResponseAfterServerCommit();

    await fixture.StartConsumerAsync();
    await fixture.WaitForUnknownOutcomeAsync();
    fixture.SuccessfulAcks.Should().Be(0);

    await fixture.RestartConsumerAsync();
    await fixture.WaitUntilQueueEmptyAsync();

    await fixture.Database.AssertInboxCountAsync(
        fixture.ConsumerName, message.MessageId, 1);
    await fixture.Database.AssertReviewCountAsync(
        message.VerdictId, 1);
    await fixture.Database.AssertOutboxCountAsync(
        message.ExpectedFactKey, 1);
    fixture.SuccessfulAcks.Should().Be(1);
}
```

EXECUTABLE ASSET 28-AK - CRASH BEFORE COMMIT

```
[Fact]
public async Task INBOX_CRASH_002_precommit_crash_reprocesses()
{
    var message = await fixture.PublishVerdictAsync();
    fixture.Faults.KillProcessAt(
        ConsumerCheckpoint.SaveStarted);

    await fixture.StartConsumerAsync();
    await fixture.WaitForProcessExitAsync();
    await fixture.RestartConsumerAsync();
    await fixture.WaitUntilQueueEmptyAsync();

    await fixture.Database.AssertInboxCountAsync(
        fixture.ConsumerName, message.MessageId, 1);
    await fixture.Database.AssertReviewCountAsync(
        message.VerdictId, 1);
}
```

CANCELLATION BOUNDARY

Cancellation before SaveChanges is cancellation. Cancellation or connection loss during commit may be Unknown. The broker action must follow the observed outcome, not an optimistic rollback assumption.

RETENTION AND RECOVERY

Deleting a receipt reopens the duplicate window

Inbox retention is part of the guarantee. A receipt can be removed only after every broker redelivery, replay tool, backup restore and incident hold that may reintroduce the MessageId is outside its declared horizon.

RETENTION RULE 28-AL - SAFE DELETION FENCE

```
retention_floor =
  min(
    broker_replay_floor,
    operator_replay_floor,
    backup_restore_floor,
    incident_hold_floor,
    contract_support_floor)

delete receipt only when:
  CompletedAtUtc < retention_floor
  AND no legal/evidence hold exists
  AND the owning contract version is outside replay support

If any floor is unknown, deletion is forbidden.
```

EXECUTABLE ASSET 28-AM - RESTORE/REPLAY ACCEPTANCE

```
[Fact]
public async Task INBOX_RESTORE_001_receipts_survive_restore()
{
  var snapshot = await fixture.ProcessAndBackupAsync();
  await fixture.RestoreIntoCleanDatabaseAsync(snapshot);
  var replay = fixture.OriginalDelivery;

  var outcome = await fixture.ConsumeAsync(replay);

  outcome.Should().BeOfType<
    ConsumeOutcome.Duplicate>();
  await fixture.Database.AssertReviewCountAsync(
    replay.Contract.VerdictId, 1);
  await fixture.Database.AssertOutboxCountAsync(
    replay.ExpectedFactKey, 1);
}
```

ARCHIVE IS STILL QUERIED

Moving old receipts to cheaper storage is safe only if duplicate admission checks that archive before business work. An archive that is invisible to the consumer is evidence storage, not idempotency storage.

OPERATIONAL EVIDENCE

Observe duplicate pressure without exposing payloads

Metrics distinguish first processing, matching duplicates, semantic no-ops, identity conflicts, unknown commits and broker dispositions. Messageld remains a structured log field, never a metric label.

EXECUTABLE ASSET 28-AN - INBOX TELEMETRY

```
public static class InboxTelemetry
{
    public static readonly Meter Meter =
        new("Nexus1.Messaging.Inbox");
    public static readonly Counter<long> Deliveries =
        Meter.CreateCounter<long>("inbox.deliveries");
    public static readonly Counter<long> Outcomes =
        Meter.CreateCounter<long>("inbox.outcomes");
    public static readonly Counter<long> Duplicates =
        Meter.CreateCounter<long>("inbox.duplicates");
    public static readonly Counter<long> IdentityConflicts =
        Meter.CreateCounter<long>("inbox.identity_conflicts");
    public static readonly Histogram<double> ProcessingMs =
        Meter.CreateHistogram<double>("inbox.processing.duration", "ms");
}

// Allowed tags: consumer, producer, event_type, outcome.
// Forbidden tags: message_id, correlation_id, payload, raw exception.
```

EVIDENCE THRESHOLDS 28-AO - INITIAL ALERT SURFACE

Signal	Warning	Critical
identity conflict	any	immediate security page
Unknown commit ratio	> 0.5% / 10 min	> 2% / 10 min
matching duplicate ratio	baseline + 3 sigma	sustained growth
processing latency p99	> 2 s	> ack budget / 2
unacknowledged deliveries	capacity 70%	capacity 90%
inbox insert failures	any burst	sustained
receipt cleanup lag	> 1 day	storage budget breach

SECURITY BOUNDARY

The consumer principal reads and updates only its owned database and consumes only its declared queue. It cannot administer topology, publish arbitrary exchanges or read producer tables.

EXECUTABLE EVIDENCE

The acceptance record fails closed

The manifest binds schema, provider, broker, contract corpus, concurrency tests, crash checkpoints and retention proof to exact versions and result digests. Rendering these pages does not promote proposed code to executed evidence.

EXECUTABLE ASSET 28-AP - EVIDENCE MANIFEST

```
{
  "chapter": 28,
  "decision": "ADR-026",
  "claimSet": "consumer-inbox-v1",
  "database": {
    "provider": "sqlserver",
    "migrationDigest": "<required>",
    "tests": {
      "atomicSuccess": "<required>",
      "atomicRollback": "<required>",
      "concurrentDuplicate": "<required>",
      "unknownCommit": "<required>"
    }
  },
  "broker": {
    "product": "rabbitmq",
    "version": "<required>",
    "queueType": "<required>",
    "tests": {
      "lostAckRedelivery": "<required>",
      "sameBytesReplay": "<required>",
      "channelRecovery": "<required>"
    }
  },
  "retention": {
    "policyDigest": "<required>",
    "restoreReplay": "<required>"
  },
  "zeroDiscoveryRejected": true,
  "generatedAtUtc": "<required>"
}
```

EXECUTABLE ASSET 28-AQ - EVIDENCE VALIDATOR

```
public static void DemandChapter28Evidence(
    Chapter28Evidence evidence)
{
    Evidence.RequireSha256(evidence.MigrationDigest);
    Evidence.RequireVersion(evidence.BrokerVersion);
    Evidence.RequireNonZero(evidence.ProviderTests);
    Evidence.RequireNonZero(evidence.BrokerTests);
    Evidence.Require(evidence.ConcurrentDuplicatePassed);
    Evidence.Require(evidence.LostAckPassed);
    Evidence.Require(evidence.UnknownCommitPassed);
    Evidence.Require(evidence.RestoreReplayPassed);
    Evidence.RequireFresh(
        evidence.GeneratedAtUtc,
        maximumAge: TimeSpan.FromDays(14));
}
```

CURRENT EVIDENCE STATUS

PENDING. The manuscript specifies compile-oriented assets and exact oracles, but this environment has not compiled the .NET solution, applied SQL Server migrations or executed RabbitMQ redelivery and process-crash tests.

ADR-026

Choose atomic inbox plus owned effect and post-commit ack

The decision makes repeated delivery safe without hiding transport reality. It uses one consumer database as the authority and relies on broker redelivery whenever local commit or acknowledgement is uncertain.

DECISION RECORD 28-AR - ADR-026**Status**

Proposed until the Chapter 28 evidence manifest passes.

Context

RabbitMQ can redeliver unacknowledged messages.
SQL Server and RabbitMQ do not share one local transaction.

Decision

Scope receipts by ConsumerName and MessageId.
Store the immutable envelope digest with the receipt.
Commit receipt, owned effect and new outbox intent together.
Ack only Committed or matching Duplicate.
Retry Unknown; reject identity conflict.
Preserve receipts through every supported replay horizon.

Consequences

Broker delivery remains at least once.
Local owned effect is once per retained receipt identity.
Domain keys still guard semantic duplicates.
Receipt storage and retention become operational obligations.

Rejected alternatives

autoAck before processing
inbox insert before a separate business transaction
in-memory duplicate cache
redelivered flag as the deduplication key
distributed transaction with RabbitMQ
deleting receipts on a fixed age without replay evidence

REVIEW TRIGGER

Revisit ADR-026 when consumer identity, provider, broker client major version, queue type, replay horizon, contract identity or transaction boundary changes.

IMPLEMENTATION PATH

Prove duplicate delivery before adding poison policy

The sequence establishes local atomicity, then transport redelivery, then operations. Chapter 29 may build retry and dead-letter policy only on top of these verified outcomes.

IMPLEMENTATION SEQUENCE 28-AS

1. Publish ConsumerName ownership for every queue subscription.
 2. Apply InboxReceipt migration independently per consumer database.
 3. Implement strict property/envelope admission and digest verification.
 4. Implement one-SaveChanges receipt/effect/outbox transaction.
 5. Classify only PK_InboxReceipt with provider-specific evidence.
 6. Add fresh-context duplicate and unknown-commit resolution.
 7. Configure manual ack, bounded prefetch and sequential channel dispatch.
 8. Prove atomic success, rollback and simultaneous duplicate races.
 9. Force commit-response loss and lost-ack broker redelivery.
 10. Prove semantic duplicate convergence through business uniqueness.
 11. Publish retention, backup, restore and replay horizons.
 12. Add bounded telemetry, sign ADR-026 and publish evidence.

28.2 Official technical anchors

RabbitMQ Consumer Acknowledgements: <https://www.rabbitmq.com/docs/confirms>
RabbitMQ .NET Client API Guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>
RabbitMQ Queues and Redelivery: <https://www.rabbitmq.com/docs/queues>
EF Core Transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
EF Core Unique Indexes: <https://learn.microsoft.com/ef/core/modeling/indexes>

WHAT THIS CHAPTER HAS PROVED

The design specifies durable consumer identity, exact duplicate and conflict rules, one local atomic commit, safe acknowledgement timing, crash/redelivery oracles, retention obligations and executable evidence. Runtime proof is PENDING.

NEXT - CHAPTER 29

Retry, Backoff, Dead Letters, and Poison Messages will classify repeatable failure, assign retry ownership and budgets, route exhausted work to durable evidence, and prove operator replay without bypassing the inbox.

CHAPTER 28 - CLOSURE

Checkpoint: does the consumer turn redelivery into a no-op?

Consumer idempotency is not an in-memory cache. It is a durable identity, digest and transaction rule aligned with the broker acknowledgement.

CHECKPOINT 28-A - CONSUME BOUNDARY

Invariant	Required behavior	Failure if absent
Stable consumer identity	Inbox key includes consumer and MessageId.	one handler masks another
Same ID, same digest	Return the stored completion; apply no effect.	duplicate local mutation
Same ID, different digest	Terminal identity conflict and operator evidence.	silent contract corruption
Atomic local work	Inbox, effect and local outbox commit together.	processed marker disagrees with state
Broker acknowledgement	Ack only after a known successful local commit.	message loss after rollback
Unknown commit outcome	Dispose; resolve by identity in a fresh context.	reuse of an invalid tracker
Retention	Inbox horizon exceeds the supported replay horizon.	old redelivery becomes a new effect

EXIT CONDITION

A broker duplicate may cost another read, but it cannot create a second domain effect. Evidence remains PENDING until the selected provider and broker execute the crash-window suite.

PART V - THE MESSAGE BACKBONE

29. Retry, Backoff, Dead Letters, and Poison Messages

A failed delivery needs a finite, durable decision. NEXUS-1 retries only classified transient work, preserves the exact admitted bytes, and turns terminal failure into inspectable evidence before acknowledging the broker.

CHAPTER DECISION

The consumer records either a due `RetryTicket` or a `PoisonMessage` in its own database before acknowledging the failed delivery. Unknown outcomes remain unacknowledged. Replay publishes through the normal exchange and therefore cannot bypass the Chapter 28 inbox.

29.1 The exact guarantee

The chapter does not promise that every dependency will recover or that the broker delivers once. It proves that retries are bounded by attempt and elapsed-time budgets, that every delayed attempt has one durable owner, and that exhausted or permanent work becomes visible.

GUARANTEE LEDGER 29-A - FAILURE TO VISIBLE OUTCOME

Observed outcome	Durable decision before broker action
transient + budget remains	<code>RetryTicket(DueAtUtc, exact envelope)</code>
transient + budget exhausted	<code>PoisonMessage(reason=budget-exhausted)</code>
permanent contract failure	<code>PoisonMessage(reason=contract-invalid)</code>
explicit operator hold	<code>PoisonMessage(reason=operator-hold)</code>
local commit outcome unknown	none; stop without acknowledgement
evidence store unavailable	none; stop without acknowledgement
Later action	Required path
scheduled retry	confirm-tracked normal exchange publish
operator replay	frozen bytes -> normal exchange -> inbox
broker dead-letter	safety-net intake -> durable evidence

BOUNDARY TO CHAPTER 30

This chapter owns failure classification, retry timing, terminal evidence and replay. Per-aggregate ordering, sequence gaps, duplicate contract meaning and compatible schema evolution belong to Chapter 30.

ORDINARY DEVELOPER PROBLEM

A retry is a new scheduled delivery, not a hot loop

Immediate requeue can return the same failing delivery before the dependency has recovered. It consumes broker, network and consumer capacity while hiding the absence of a real failure decision.

REJECTED ASSET 29-B - THE LOOP THAT MUST NOT SHIP

```
consumer.ReceivedAsync += async (_, delivery) =>
{
    try
    {
        await handler.HandleAsync(delivery, stoppingToken);
        await channel.BasicAckAsync(
            delivery.DeliveryTag, multiple: false);
    }
    catch
    {
        await channel.BasicNackAsync(
            delivery.DeliveryTag,
            multiple: false,
            requeue: true); // immediate, unclassified, unbounded
    }
};
```

DECISION TABLE 29-C - WHY THE NAIVE LOOP FAILS

Missing decision	Consequence
no failure classification	invalid contracts retry forever
no attempt budget	dependency outage becomes retry storm
no elapsed-time budget	stale work can live indefinitely
no durable due time	restart resets all waiting
no jitter	instances retry in synchrony
no terminal record	operator sees only a quiet or hot queue
no replay authority	manual publish bypasses audit and inbox

PLAIN-LANGUAGE PRINCIPLE

A transient failure is not permission to try again immediately. It is a request to create one bounded future attempt whose ownership survives process and broker restarts.

FAILURE VOCABULARY

Classify the failure before choosing a broker action

The same exception type can mean different things at different phases. A timeout before a remote request was sent differs from an unknown response after a non-idempotent dependency may have accepted the request.

EXECUTABLE ASSET 29-D - CLOSED FAILURE VOCABULARY

```
public enum FailureKind
{
    TransientDependency,
    Throttled,
    PermanentContract,
    PermanentConfiguration,
    OperatorHold,
    ConsumerStopping,
    UnknownOutcome
}

public enum FailurePhase
{
    Admission,
    DomainDecision,
    LocalCommit,
    RemoteRead,
    RemoteCommand,
    RetryPersistence,
    QuarantinePersistence
}

public sealed record FailureObservation(
    Exception Error,
    FailurePhase Phase,
    string Dependency,
    bool RequestMayHaveCompleted,
    TimeSpan? RetryAfter);
```

DECISION CONTRACT 29-E - BOUNDED DISPOSITIONS

```
public abstract record FailureDecision(string Code)
{
    public sealed record ScheduleRetry(
        string Code, TimeSpan? MinimumDelay)
        : FailureDecision(Code);

    public sealed record Quarantine(
        string Code, bool ReplayMayBeRequested)
        : FailureDecision(Code);

    public sealed record StopWithoutAck(string Code)
        : FailureDecision(Code);
}
```

UNKNOWN IS A FIRST-CLASS ANSWER

If the classifier cannot prove transient or permanent meaning, it returns `StopWithoutAck`. Guessing transient can repeat a remote command; guessing permanent can discard recoverable work.

EXPLICIT CLASSIFICATION

Exception types become evidence only after context is checked

The reference classifier is an allowlist. It uses phase, dependency and provider evidence; it never retries every `TimeoutException`, `HttpRequestException` or database exception merely because the type sounds temporary.

EXECUTABLE ASSET 29-F - ALLOWLISTED CLASSIFIER

```
public sealed class FailureClassifier : IFailureClassifier
{
    public FailureDecision Classify(FailureObservation item) =>
    {
        item switch
        {
            { Error: ConsumerStoppingException } =>
                new FailureDecision.StopWithoutAck(
                    "consumer-stopping"),

            { Phase: FailurePhase.LocalCommit,
              RequestMayHaveCompleted: true } =>
                new FailureDecision.StopWithoutAck(
                    "local-commit-unknown"),

            { Phase: FailurePhase.RemoteCommand,
              RequestMayHaveCompleted: true } =>
                new FailureDecision.StopWithoutAck(
                    "remote-command-unknown"),

            { Error: ContractAdmissionException error } =>
                new FailureDecision.Quarantine(
                    error.Code, ReplayMayBeRequested: true),

            { Error: OperatorHoldException hold } =>
                new FailureDecision.Quarantine(
                    hold.Code, ReplayMayBeRequested: true),

            { Error: DependencyThrottledException throttled,
              Phase: FailurePhase.RemoteRead } =>
                new FailureDecision.ScheduleRetry(
                    "dependency-throttled",
                    throttled.RetryAfter),

            { Error: DependencyUnavailableException,
              Phase: FailurePhase.RemoteRead } =>
                new FailureDecision.ScheduleRetry(
                    "dependency-unavailable", null),

            { Error: ConfigurationException error } =>
                new FailureDecision.Quarantine(
                    error.Code, ReplayMayBeRequested: true),

            _ => new FailureDecision.StopWithoutAck(
                "unclassified-failure")
        };
    }
}
```

DOMAIN REJECTION IS NOT AUTOMATICALLY POISON

An already-satisfied domain outcome may be an acknowledged semantic no-op. A sequence gap may require Chapter 30 policy. Only an explicitly classified terminal delivery becomes poison evidence.

POLICY OWNERSHIP

Every subscription publishes an explicit retry policy

A global retry count hides different latency and safety needs. The RootCause alarm subscription owns a named policy whose digest is captured with every ticket, poison record and acceptance run.

EXECUTABLE ASSET 29-G - POLICY VALUE

```
public sealed record RetryPolicy(
    string PolicyId,
    int MaxRetryAttempts,
    TimeSpan MaxElapsed,
    TimeSpan InitialDelay,
    TimeSpan MaxDelay,
    int EqualJitterPercent)
{
    public void Validate()
    {
        if (String.IsNullOrEmpty(PolicyId))
            throw new InvalidRetryPolicy("policy-id");
        if (MaxRetryAttempts is < 0 or > 20)
            throw new InvalidRetryPolicy("attempt-budget");
        if (MaxElapsed <= TimeSpan.Zero)
            throw new InvalidRetryPolicy("elapsed-budget");
        if (InitialDelay <= TimeSpan.Zero ||
            InitialDelay > MaxDelay)
            throw new InvalidRetryPolicy("delay-range");
        if (MaxDelay > MaxElapsed)
            throw new InvalidRetryPolicy("delay-exceeds-horizon");
        if (EqualJitterPercent is < 0 or > 50)
            throw new InvalidRetryPolicy("jitter-range");
    }
}
```

POLICY CATALOGUE 29-H - CONSUMER-OWNED BASELINE

ConsumerName	PolicyId	Budget
rootcause.alarm-events.v1	rc-alarm-read-v1	5 / 15 min
compliance.verdict-events.v1	cmp-verdict-read-v1	4 / 30 min
audit.public-facts.v1	audit-append-v1	8 / 60 min
reporting.public-facts.v1	report-project-v1	6 / 45 min

Policy identity includes:
classifier version, attempt count, elapsed horizon,
delay algorithm, delay bounds and replay rules.

POLICY CHANGES DO NOT REWRITE HISTORY

A ticket keeps the policy identity and due time chosen when it was created. A later configuration change governs new decisions only unless an explicit migration records the operator's authority.

ATTEMPT AND AGE BUDGETS

Backoff is bounded by attempt count and elapsed age

MaxRetryAttempts counts retries after the initial delivery. A policy with five retries allows at most six handler deliveries, and **MaxElapsed** can terminate the sequence earlier.

EXECUTABLE ASSET 29-I - RETRY BUDGET EVALUATOR

```
public sealed record RetryBudgetDecision(
    bool CanRetry,
    int NextAttempt,
    string Reason);

public static class RetryBudget
{
    public static RetryBudgetDecision Evaluate(
        RetryPolicy policy,
        int currentAttempt,
        DateTimeOffset firstFailedAtUtc,
        DateTimeOffset nowUtc)
    {
        policy.Validate();
        if (currentAttempt < 0)
            throw new InvalidAttempt(currentAttempt);

        var next = checked(currentAttempt + 1);
        if (next > policy.MaxRetryAttempts)
            return new(false, next, "attempt-budget-exhausted");

        if (nowUtc - firstFailedAtUtc >= policy.MaxElapsed)
            return new(false, next, "elapsed-budget-exhausted");

        return new(true, next, "retry-permitted");
    }
}
```

BUDGET EXAMPLES 29-J - FINITE DELIVERY

Policy: MaxRetryAttempts=3, MaxElapsed=00:10:00

Delivery	currentAttempt	Next durable outcome
original	0	RetryTicket attempt 1
retry 1	1	RetryTicket attempt 2
retry 2	2	RetryTicket attempt 3
retry 3	3	PoisonMessage exhausted

If elapsed age reaches ten minutes before retry 3 is due, the message becomes poison without creating another ticket.

TWO BOUNDS, ONE SERVICE OBJECTIVE

Attempt count protects capacity. Elapsed age protects meaning. Both must fit the end-to-end latency objective; exponential growth without an elapsed horizon is still unbounded.

DETERMINISTIC JITTER

Stable equal-jitter spreads work without moving the due date

The scheduler computes delay once from `MessageId`, policy identity and attempt, then persists `DueAtUtc`. Equal jitter keeps half the exponential cap and spreads the remainder without relying on process-local random state.

EXECUTABLE ASSET 29-K - EXPONENTIAL CAP

```
public static TimeSpan ExponentialCap(
    RetryPolicy policy, int nextAttempt)
{
    if (nextAttempt < 1)
        throw new InvalidAttempt(nextAttempt);

    var ticks = policy.InitialDelay.Ticks;
    for (var step = 1; step < nextAttempt; step++)
    {
        if (ticks >= policy.MaxDelay.Ticks / 2)
            return policy.MaxDelay;
        ticks *= 2;
    }

    return TimeSpan.FromTicks(
        Math.Min(ticks, policy.MaxDelay.Ticks));
}
```

EXECUTABLE ASSET 29-L - STABLE EQUAL JITTER

```
public static TimeSpan EqualJitter(
    RetryPolicy policy,
    Guid messageId,
    int nextAttempt)
{
    var cap = ExponentialCap(policy, nextAttempt);
    var floorPercent = 100 - policy.EqualJitterPercent;
    var floorTicks = cap.Ticks * floorPercent / 100;
    var spreadTicks = cap.Ticks - floorTicks;

    var input = Encoding.UTF8.GetBytes(
        $"{policy.PolicyId}\n{messageId:D}\n{nextAttempt}");
    var hash = SHA256.HashData(input);
    var sample = BinaryPrimitives.ReadUInt64BigEndian(hash);
    var offset = spreadTicks == 0
        ? 0
        : (long)(sample % (ulong)(spreadTicks + 1));

    return TimeSpan.FromTicks(floorTicks + offset);
}
```

RETRY-AFTER IS A MINIMUM, NOT PERMISSION TO EXCEED THE BUDGET

A trusted dependency delay may raise the computed due time, but it is clamped by `MaxDelay` and `MaxElapsed`. If the resulting due time crosses the elapsed horizon, the budget is exhausted.

DURABLE WAITING

Sleep never owns a retry

Task.Delay retains no recovery evidence and consumes a live worker lifetime. The durable state machine releases the consumer after recording the future attempt and lets an independent dispatcher recover due work.

STATE MODEL 29-M - RETRYTICKET LIFECYCLE

State	Allowed transition	Durable condition
Due	-> Claimed	DueAtUtc <= database UTC
Claimed	-> Published	broker confirm received
Claimed	-> Due	lease expired or publish failed
Published	-> Consumed	next delivery observed
Published	-> Superseded	matching later terminal record
any	-> Quarantined	budget/policy terminal decision
Forbidden:		
Due	-> Published without a claim	
Claimed	-> Published without a confirm	
Published	-> Due because handler failed	
Quarantined	-> Published without approved replay	

TIMING ORACLE 29-N - DATABASE TIME OWNS READINESS

```
due(candidate, database_now) :=
  candidate.State = Due
  AND candidate.DueAtUtc <= database_now
  AND (
    candidate.LeaseUntilUtc IS NULL
    OR candidate.LeaseUntilUtc < database_now)

process time is used for cancellation only.
persisted database UTC decides due order and lease expiry.
```

NO LONG CONSUMER HOLD

The original delivery is acknowledged only after the RetryTicket commit. The dispatcher, not the handler callback, waits for DueAtUtc.

CONSUMER-OWNED RETRY STORE

RetryTicket preserves the exact admitted bytes

A scheduled retry is not a reconstructed event. It carries the original MessageId, envelope digest and protected bytes so later dispatch cannot change the producer-owned fact.

EXECUTABLE ASSET 29-O - SQL SERVER RETRY TABLE

```
CREATE TABLE messaging.RetryTicket
(
    RetryTicketId uniqueidentifier NOT NULL,
    ConsumerName nvarchar(120) NOT NULL,
    MessageId uniqueidentifier NOT NULL,
    ReplayGeneration int NOT NULL,
    Attempt int NOT NULL,
    PolicyId nvarchar(100) NOT NULL,
    FailureCode nvarchar(100) NOT NULL,
    FirstFailedAtUtc datetimeoffset(7) NOT NULL,
    DueAtUtc datetimeoffset(7) NOT NULL,
    State tinyint NOT NULL,
    EnvelopeSha256 binary(32) NOT NULL,
    ProtectedEnvelope varbinary(max) NOT NULL,
    ProtectionKeyId nvarchar(100) NOT NULL,
    OriginalExchange nvarchar(200) NOT NULL,
    OriginalRoutingKey nvarchar(240) NOT NULL,
    LeaseOwner nvarchar(160) NULL,
    LeaseUntilUtc datetimeoffset(7) NULL,
    PublishedAtUtc datetimeoffset(7) NULL,
    CreatedAtUtc datetimeoffset(7) NOT NULL,
    Version rowversion NOT NULL,
    CONSTRAINT PK_RetryTicket
        PRIMARY KEY (RetryTicketId),
    CONSTRAINT UQ_RetryTicket_Attempt
        UNIQUE (
            ConsumerName, MessageId,
            ReplayGeneration, Attempt),
    CONSTRAINT CK_RetryTicket_ReplayGeneration
        CHECK (ReplayGeneration >= 0),
    CONSTRAINT CK_RetryTicket_Attempt
        CHECK (Attempt >= 1)
);

CREATE INDEX IX_RetryTicket_Due
ON messaging.RetryTicket(State, DueAtUtc)
INCLUDE (LeaseUntilUtc, RetryTicketId);
```

PAYLOAD CUSTODY IS EXPLICIT

ProtectedEnvelope is encrypted under a recorded key identity. Logs, metrics and ordinary operator lists expose its hash and metadata, never the body.

ATTEMPT EVIDENCE

Redelivery is not automatically a new retry attempt

One FailureAttempt row represents one application handler attempt under one policy. A broker redelivery caused by lost acknowledgement resolves the existing ticket or poison identity and does not consume another application budget slot.

EXECUTABLE ASSET 29-01 - SQL SERVER ATTEMPT HISTORY

```
CREATE TABLE messaging.FailureAttempt
(
    FailureAttemptId uniqueidentifier NOT NULL,
    ConsumerName nvarchar(120) NOT NULL,
    MessageId uniqueidentifier NOT NULL,
    EnvelopeSha256 binary(32) NOT NULL,
    ReplayGeneration int NOT NULL,
    Attempt int NOT NULL,
    PolicyId nvarchar(100) NOT NULL,
    FailureCode nvarchar(100) NOT NULL,
    FailureFingerprint char(64) NOT NULL,
    Phase nvarchar(60) NOT NULL,
    Dependency nvarchar(120) NOT NULL,
    ObservedAtUtc datetimeoffset(7) NOT NULL,
    Disposition nvarchar(60) NOT NULL,
    RetryTicketId uniqueidentifier NULL,
    PoisonMessageId uniqueidentifier NULL,
    CONSTRAINT PK_FailureAttempt
        PRIMARY KEY (FailureAttemptId),
    CONSTRAINT UQ_FailureAttempt_Identity
        UNIQUE (
            ConsumerName, MessageId,
            EnvelopeSha256,
            ReplayGeneration, Attempt)
);

CREATE INDEX IX_FailureAttempt_Message
ON messaging.FailureAttempt(
    ConsumerName, MessageId,
    ReplayGeneration, Attempt);
```

ATTEMPT IDENTITY 29-02 - WHAT INCREMENTS THE BUDGET

Increment application attempt when:

an original delivery enters the handler	-> generation 0, attempt 0
a RetryTicket delivery enters the handler	-> ticket generation/attempt
an approved replay enters the handler	-> generation N, attempt 0

Do not increment when:

- the original ack is lost
- a retry publish is duplicated
- a delivery fails admission before ticket authority
- unknown scheduling commit is being resolved
- dead-letter evidence is imported

The unique key makes repeated observation of the same replay generation and attempt idempotent.

HISTORY IS APPEND-ONLY

A later retry or replay adds evidence; it does not rewrite the failure code, fingerprint, policy identity or time of an earlier attempt.

EF CORE PERSISTENCE

Uniqueness arbitrates duplicate retry scheduling

The same failed broker delivery can return after the scheduling commit but before its acknowledgement. The unique ConsumerName, MessageId and Attempt key turns that race into one durable ticket.

EXECUTABLE ASSET 29-P - RETRY RECORD AND MAPPING

```
internal sealed class RetryTicketRecord
{
    public Guid RetryTicketId { get; set; }
    public string ConsumerName { get; set; } = null!;
    public Guid MessageId { get; set; }
    public int ReplayGeneration { get; set; }
    public int Attempt { get; set; }
    public string PolicyId { get; set; } = null!;
    public string FailureCode { get; set; } = null!;
    public DateTimeOffset FirstFailedAtUtc { get; set; }
    public DateTimeOffset DueAtUtc { get; set; }
    public RetryTicketState State { get; set; }
    public byte[] EnvelopeSha256 { get; set; } = null!;
    public byte[] ProtectedEnvelope { get; set; } = null!;
    public string ProtectionKeyId { get; set; } = null!;
    public string OriginalExchange { get; set; } = null!;
    public string OriginalRoutingKey { get; set; } = null!;
    public string? LeaseOwner { get; set; }
    public DateTimeOffset? LeaseUntilUtc { get; set; }
    public DateTimeOffset? PublishedAtUtc { get; set; }
    public DateTimeOffset CreatedAtUtc { get; set; }
    public byte[] Version { get; set; } = null!;
}

internal sealed class RetryTicketMap
    : IEntityTypeConfiguration<RetryTicketRecord>
{
    public void Configure(
        EntityTypeBuilder<RetryTicketRecord> entity)
    {
        entity.ToTable("RetryTicket", "messaging");
        entity.HasKey(x => x.RetryTicketId);
        entity.HasIndex(x => new
        {
            x.ConsumerName, x.MessageId,
            x.ReplayGeneration, x.Attempt
        }).IsUnique();
        entity.HasIndex(x => new { x.State, x.DueAtUtc });
        entity.Property(x => x.Version).IsRowVersion();
    }
}
```

THE CONSTRAINT DECIDES THE RACE

A pre-read may avoid work, but only the provider unique constraint proves that simultaneous callbacks cannot create two tickets for the same attempt.

ATOMIC FAILURE DECISION

Failure and retry scheduling form a local durable decision

The failed business transaction is already rolled back. A fresh consumer DbContext records the attempt and ticket in one short transaction, after which the adapter may acknowledge the original delivery.

EXECUTABLE ASSET 29-Q - SCHEDULING OUTCOME

```
public abstract record FailureRecordingOutcome
{
    public sealed record RetryRecorded(
        Guid RetryTicketId, DateTimeOffset DueAtUtc
        ) : FailureRecordingOutcome;

    public sealed record PoisonRecorded(Guid PoisonMessageId)
        : FailureRecordingOutcome;

    public sealed record AlreadyRecorded(Guid RecordId)
        : FailureRecordingOutcome;

    public sealed record Unknown
        : FailureRecordingOutcome;
}
```

EXECUTABLE ASSET 29-R - RECORD A RETRY BEFORE ACK

```
public async Task<FailureRecordingOutcome> ScheduleAsync(
    FailedDelivery failed,
    RetryPlan plan,
    CancellationToken cancellationToken)
{
    await using var db =
        await factory.CreateDbContextAsync(cancellationToken);

    var protectedBody = await protector.ProtectAsync(
        failed.EnvelopeBytes, cancellationToken);
    if (failed.RetryTicketId is Guid priorId)
    {
        var prior = await db.RetryTickets.SingleAsync(
            x => x.RetryTicketId == priorId,
            cancellationToken);
        prior.MarkSuperseded(plan.RetryTicketId);
    }
    db.RetryTickets.Add(RetryTicketRecord.Create(
        failed, plan, protectedBody, clock.UtcNow));
    db.FailureAttempts.Add(FailureAttemptRecord.Create(
        failed, plan.FailureCode, clock.UtcNow));

    try
    {
        await db.SaveChangesAsync(cancellationToken);
        return new FailureRecordingOutcome.RetryRecorded(
            plan.RetryTicketId, plan.DueAtUtc);
    }
    catch (DbUpdateException error)
    {
        when (errors.IsRetryAttemptUnique(error))
        {
            return await resolver.ResolveExistingAsync(
                failed,
                plan.ReplayGeneration,
                plan.Attempt,
                cancellationToken);
        }
    }
}
```

BROKER ACTION FOLLOWS DURABLE OUTCOME

RetryRecorded and matching **AlreadyRecorded** map to ack. **Unknown** maps to stop-without-ack. No path acknowledges merely because **SaveChanges** was attempted.

AMBIGUOUS COMMITTS

Unknown commit is not a transient failure

There are two distinct unknowns. The Chapter 28 business commit may have applied the effect; the Chapter 29 scheduling commit may have created the ticket. Neither case may be converted into a second independent decision.

TRUTH TABLE 29-S - UNKNOWN OUTCOME HANDLING

Unknown phase	Safe immediate action
business SaveChanges	no retry ticket; no ack; redeliver
retry-ticket SaveChanges	no second ticket; no ack; redeliver
poison-record SaveChanges	no second poison; no ack; redeliver
retry publish confirm	keep claim; recover through lease
replay publish confirm	keep replay pending; recover through lease
Redelivery resolver:	
inbox exists	acknowledge as prior business success
matching retry exists	acknowledge original failed delivery
matching poison exists	acknowledge original terminal delivery
none exists	repeat the classified local decision
digest differs	quarantine identity conflict

EXECUTABLE ASSET 29-T - RESOLUTION ORDER

```
public async Task<RecoveredFailureState> ResolveAsync(
    FailedDelivery failed,
    int nextAttempt,
    CancellationToken cancellationToken)
{
    await using var db =
        await factory.CreateDbContextAsync(cancellationToken);

    if (await db.InboxReceipts.MatchesAsync(
        failed.Identity, failed.EnvelopeSha256,
        cancellationToken))
        return RecoveredFailureState.BusinessCommitted;

    if (await db.RetryTickets.MatchesAsync(
        failed.Identity,
        failed.ReplayGeneration,
        nextAttempt,
        failed.EnvelopeSha256, cancellationToken))
        return RecoveredFailureState.RetryRecorded;

    if (await db.PoisonMessages.MatchesAsync(
        failed.Identity, failed.EnvelopeSha256,
        cancellationToken))
        return RecoveredFailureState.PoisonRecorded;

    return RecoveredFailureState.NotFound;
}
```

RESOLUTION USES A FRESH CONTEXT

The context that observed a connection failure cannot prove server state. Unknown recovery always opens a new connection and reads durable identity before deciding.

RETRY DISPATCHER

The dispatcher republishes the same identity with confirms

The scheduled worker borrows the Chapter 27 claim, lease and confirm discipline. Its publication changes transport metadata only; the producer envelope bytes and MessageId remain frozen.

EXECUTABLE ASSET 29-U - PUBLISH CANDIDATE

```
public sealed record RetryPublishCandidate(
    Guid RetryTicketId,
    string ConsumerName,
    Guid MessageId,
    int ReplayGeneration,
    int Attempt,
    string PolicyId,
    DateTimeOffset FirstFailedAtUtc,
    string OriginalExchange,
    string OriginalRoutingKey,
    ReadOnlyMemory<byte> EnvelopeBytes,
    byte[] EnvelopeSha256,
    byte[] Version);

public interface IRetryTicketStore
{
    Task<IReadOnlyList<RetryPublishCandidate>> ClaimDueAsync(
        string owner, int batchSize, TimeSpan lease,
        CancellationToken cancellationToken);

    Task MarkPublishedAsync(
        RetryPublishCandidate item,
        DateTimeOffset confirmedAtUtc,
        CancellationToken cancellationToken);

    Task ReleaseAsync(
        RetryPublishCandidate item,
        DateTimeOffset dueAtUtc,
        string failureCode,
        CancellationToken cancellationToken);
}
```

EXECUTABLE ASSET 29-V - BOUNDED DISPATCH LOOP

```
public async Task DispatchBatchAsync(
    CancellationToken cancellationToken)
{
    var batch = await store.ClaimDueAsync(
        instance.Value,
        options.BatchSize,
        options.LeaseDuration,
        cancellationToken);

    foreach (var item in batch)
    {
        var result = await publisher.PublishConfirmedAsync(
            RetryPublication.From(item),
            cancellationToken);

        if (result is PublishResult.Confirmed confirmed)
            await store.MarkPublishedAsync(
                item, confirmed.AtUtc, cancellationToken);
        else
            await store.ReleaseAsync(
                item,
                clock.UtcNow + options.PublishFailureDelay,
                result.Code,
                cancellationToken);
    }
}
```


ATTEMPT PROVENANCE

Attempt headers are hints; the ledger is authority

Transport headers help tracing and diagnosis, but a publisher can forge or reset them. A retry is admitted only when `RetryTicketId` resolves to the same consumer, `MessageId`, digest, policy and attempt in the consumer database.

HEADER CONTRACT 29-W - RETRY TRANSPORT METADATA

Header	Value
x-nexus-retry-ticket	RetryTicketId as lowercase GUID
x-nexus-replay-generation	non-negative Int32 generation
x-nexus-attempt	positive Int32 retry number
x-nexus-retry-policy	immutable PolicyId
x-nexus-first-failed-at	RFC 3339 UTC timestamp
x-nexus-envelope-sha256	uppercase hexadecimal digest
Frozen producer data:	
basic.properties.message_id	
envelope.messageId	
envelope.eventType	
envelope.schemaVersion	
envelope.occurredAtUtc	
complete envelope bytes	

EXECUTABLE ASSET 29-X - AUTHORITATIVE RETRY ADMISSION

```
public async Task<RetryAdmission> AdmitRetryAsync(
    BrokerRetryHeaders headers,
    AdmittedDelivery delivery,
    CancellationToken cancellationToken)
{
    var ticket = await store.FindAsync(
        headers.RetryTicketId, cancellationToken);
    if (ticket is null)
        return RetryAdmission.Reject("ticket-not-found");

    if (ticket.ConsumerName != delivery.Consumer.Value ||
        ticket.MessageId != delivery.MessageId ||
        ticket.ReplayGeneration !=
            headers.ReplayGeneration ||
        ticket.Attempt != headers.Attempt ||
        ticket.PolicyId != headers.PolicyId ||
        !CryptographicOperations.FixedTimeEquals(
            ticket.EnvelopeSha256,
            delivery.EnvelopeSha256))
        return RetryAdmission.Reject("ticket-mismatch");

    return RetryAdmission.Accept(
        ticket.ReplayGeneration, ticket.Attempt);
}
```

BROKER DELIVERY COUNT IS NOT THE APPLICATION BUDGET

RabbitMQ delivery-count and x-death describe broker history. The consumer-owned RetryTicket ledger decides the NEXUS-1 attempt number. Committed or matching Duplicate handling marks the published ticket Consumed in the consumer's local transaction; a later failure marks it Superseded when the next ticket is recorded.

CONFIRM RECOVERY

A retry publish can duplicate, never disappear silently

A confirmed publish followed by process death leaves the ticket Claimed. After lease expiry the dispatcher may publish the same bytes again. Chapter 28 makes that duplicate harmless.

CRASH MATRIX 29-Y - RETRY PUBLICATION

Crash point	Durable/broker consequence
before claim commit	ticket remains Due
after claim commit	lease expiry returns candidate
before broker write	lease expiry returns candidate
after write, before confirm	outcome unknown; later republish
after confirm, before mark	later republish can duplicate
after Published mark	no automatic republish
after retry delivery, before ack	broker redelivers; inbox/ticket resolve

Required invariant:

- no path marks Published before a broker confirm
- no path deletes the ticket as publication proof

EXECUTABLE ASSET 29-Z - CONFIRM-LOSS ACCEPTANCE

```
[Fact]
public async Task RETRY_DISPATCH_001_confirm_loss_may_duplicate()
{
    var ticket = await fixture.ScheduleDueRetryAsync();
    fixture.BrokerFaults.DropConfirmAfterAccept(ticket);

    await fixture.DispatchOnceAsync();
    await fixture.AdvanceBeyondLeaseAsync();
    await fixture.DispatchOnceAsync();
    await fixture.StartConsumerAsync();

    fixture.Broker.Publications(ticket.MessageId)
        .Should().HaveCount(2);
    await fixture.Database.AssertInboxCountAsync(
        ticket.ConsumerName, ticket.MessageId, 1);
    await fixture.Database.AssertOwnedEffectCountAsync(
        ticket.MessageId, 1);
}
```

AT-LEAST-ONCE PUBLICATION IS INTENTIONAL

The dispatcher chooses possible duplicate publication over silent loss. Its safety depends on the inbox and semantic uniqueness already proved in Chapter 28.

TERMINAL SEMANTICS

Poison is a decision, not a message personality

A public fact is poison only for a particular subscription, policy version and time. Audit may append it successfully while Reporting quarantines the same MessageId because its current projection contract is unsupported.

POISON TAXONOMY 29-AA - BOUNDED TERMINAL REASONS

Reason code	Meaning
contract-invalid	envelope/schema admission failed
identity-conflict	MessageId reused with different digest
configuration-permanent	deployed mapping/config cannot process
attempt-budget-exhausted	retry count reached policy bound
elapsed-budget-exhausted	retry age reached policy horizon
operator-hold	explicit operational containment
broker-terminal	DLX safety-net delivery was imported
Not terminal by default:	
unknown local commit	
consumer shutdown	
evidence database unavailable	
unclassified exception	

EXECUTABLE ASSET 29-AB - TERMINAL IDENTITY

```
public sealed record PoisonIdentity(
    string ConsumerName,
    Guid MessageId,
    string EnvelopeSha256Hex);

public enum PoisonState
{
    Active,
    ReplayRequested,
    ReplayApproved,
    ReplayScheduled,
    Resolved,
    Retained
}
```

A QUIET QUEUE PROVES NOTHING

Acknowledging a failed delivery without a durable terminal record can make the queue empty while the business fact has vanished from operator view. Poison evidence exists before ack.

DURABLE QUARANTINE

Quarantine is durable evidence before acknowledgement

The record retains frozen bytes under encryption, bounded failure metadata, policy identity and attempt count. Full exception text is not the durable contract because it is unstable and may contain sensitive data.

EXECUTABLE ASSET 29-AC - SQL SERVER POISON TABLE

```
CREATE TABLE messaging.PoisonMessage
(
    PoisonMessageId uniqueidentifier NOT NULL,
    ConsumerName nvarchar(120) NOT NULL,
    MessageId uniqueidentifier NOT NULL,
    EnvelopeSha256 binary(32) NOT NULL,
    ProtectedEnvelope varbinary(max) NOT NULL,
    ProtectionKeyId nvarchar(100) NOT NULL,
    EventType nvarchar(200) NULL,
    SchemaVersion int NULL,
    TerminalReason nvarchar(100) NOT NULL,
    FailureFingerprint char(64) NOT NULL,
    ClassifierVersion nvarchar(100) NOT NULL,
    PolicyId nvarchar(100) NOT NULL,
    RetryAttempts int NOT NULL,
    FirstFailedAtUtc datetimeoffset(7) NOT NULL,
    QuarantinedAtUtc datetimeoffset(7) NOT NULL,
    State tinyint NOT NULL,
    LegalHold bit NOT NULL,
    ResolvedAtUtc datetimeoffset(7) NULL,
    Version rowversion NOT NULL,
    CONSTRAINT PK_PoisonMessage
        PRIMARY KEY (PoisonMessageId),
    CONSTRAINT UQ_PoisonMessage_Identity
        UNIQUE (
            ConsumerName, MessageId, EnvelopeSha256)
);

CREATE INDEX IX_PoisonMessage_Operations
ON messaging.PoisonMessage(State, QuarantinedAtUtc)
INCLUDE (ConsumerName, TerminalReason, RetryAttempts);
```

FINGERPRINT, NOT STACK DUMP

FailureFingerprint hashes a canonical exception type, bounded provider code, phase and dependency. Sensitive messages and stack frames remain in protected diagnostic storage with separate retention.

TERMINAL RECORDING

One poison record closes the failed delivery

Quarantine uses a fresh transaction just like retry scheduling. A redelivery after lost acknowledgement resolves the same poison identity and does not create a second operator case.

EXECUTABLE ASSET 29-AD - RECORD TERMINAL EVIDENCE

```
public async Task<FailureRecordingOutcome> QuarantineAsync(
    FailedDelivery failed,
    TerminalPlan terminal,
    CancellationToken cancellationToken)
{
    await using var db =
        await factory.CreateDbContextAsync(cancellationToken);
    var protectedBody = await protector.ProtectAsync(
        failed.EnvelopeBytes, cancellationToken);

    var poison = PoisonMessageRecord.Create(
        failed, terminal, protectedBody, clock.UtcNow);
    db.PoisonMessages.Add(poison);
    db.FailureAttempts.Add(FailureAttemptRecord.Terminal(
        failed, terminal, clock.UtcNow));
    db.OutboxMessages.Add(PendingPublicFact.Create(
        PoisonMessageObservedV1.From(poison)));

    try
    {
        await db.SaveChangesAsync(cancellationToken);
        return new FailureRecordingOutcome.PoisonRecorded(
            poison.PoisonMessageId);
    }
    catch (DbUpdateException error)
        when (errors.IsPoisonIdentityUnique(error))
    {
        return await resolver.ResolveExistingPoisonAsync(
            failed, cancellationToken);
    }
}
```

DISPOSITION MAP 29-AE - TERMINAL HANDLING

Recording outcome	Broker action
PoisonRecorded	Ack
matching AlreadyRecorded	Ack
identity conflict recorded	Ack
Unknown	Stop consumer; no ack
evidence store unavailable	Stop consumer; no ack

PoisonMessageObserved.v1 contains:
poison id, consumer, event type, terminal reason,
attempt count, occurred time and correlation id.
It never contains the quarantined payload.

OPERATOR NOTIFICATION USES THE OUTBOX

The poison row and notification intent share one local transaction. Notification publication may lag, but the operator API can already discover the durable record.

BROKER SAFETY NET

Broker dead-lettering has explicit limits

RabbitMQ can dead-letter rejection, expiration, queue overflow and quorum delivery-limit outcomes. That mechanism is useful, but default re-publication does not itself prove lossless terminal evidence.

CAPABILITY LEDGER 29-AF - BROKER FACTS

Broker behavior	Engineering consequence
nack/reject requeue=false	eligible for DLX routing
message TTL expires	eligible for DLX routing
queue max length exceeded	eligible for DLX routing
quorum delivery limit exceeded	eligible for DLX routing
default DLX republish	internal confirms are not guaranteed
quorum at-least-once DLX	source retained until target confirms
at-least-once DLX may duplicate	dead queue consumer must be idempotent
x-death history compressed	not a complete application attempt log

EFFECTIVE POLICY INVENTORY 29-AG - REQUIRED, NOT ASSUMED

Source queue: rootcause.alarm-events.v1	
queue type	quorum
dead-letter-exchange	nexus.dead
dead-letter-route	rootcause.alarm-events.v1.poison
dead-letter-strategy	at-least-once
overflow	reject-publish
delivery-limit	bounded safety limit
max-length-bytes	bounded and environment-recorded
required feature flag	stream_queue enabled and recorded
Target queue: rootcause.alarm-events.v1.dead	
durable	true
queue type	quorum
consumer	dead-letter evidence importer
Configure mutable DLX values through policies, not hardcoded queue x-arguments. Capture the effective policy manifest.	

THE DATABASE REMAINS THE EVIDENCE AUTHORITY

The DLX catches topology-level terminal outcomes and feeds an idempotent importer. Application-classified poison is recorded before ack and does not depend on a second broker hop.

BROKER PROVENANCE

x-death explains broker history, not application budget

The most recent x-death entry identifies queue, reason and count for a dead-letter event. It does not replace RetryTicket history, and compressed entries cannot prove every application attempt.

EXECUTABLE ASSET 29-AH - NORMALIZED BROKER DEATH

```
public sealed record BrokerDeathObservation(  
    string SourceQueue,  
    string Reason,  
    long Count,  
    DateTimeOffset? FirstObservedAtUtc,  
    string SourceExchange,  
    IReadOnlyList<string> RoutingKeys);  
  
public sealed record DeadLetterDelivery(  
    string DeadQueue,  
    Guid? MessageId,  
    ReadOnlyMemory<byte> EnvelopeBytes,  
    byte[] EnvelopeSha256,  
    BrokerDeathObservation MostRecentDeath);
```

EXECUTABLE ASSET 29-AI - IDEMPOTENT DEAD-LETTER IMPORT

```
public async Task<DeadLetterImportOutcome> ImportAsync(  
    DeadLetterDelivery delivery,  
    CancellationToken cancellationToken)  
{  
    var identity = BrokerPoisonIdentity.Create(  
        delivery.DeadQueue,  
        delivery.MessageId,  
        delivery.EnvelopeSha256,  
        delivery.MostRecentDeath);  
  
    var outcome = await quarantine.UpsertBrokerTerminalAsync(  
        identity,  
        delivery.EnvelopeBytes,  
        delivery.MostRecentDeath,  
        cancellationToken);  
  
    return outcome switch  
    {  
        PoisonRecorded => DeadLetterImportOutcome.Ack,  
        MatchingPoison => DeadLetterImportOutcome.Ack,  
        _ => DeadLetterImportOutcome.StopWithoutAck  
    };  
}
```

NEVER COUNT BY PARSING A HEADER ALONE

Broker headers are retained as provenance. Retry budgets and terminal decisions use consumer-owned rows with explicit policy identity.

OPERATOR INSPECTION

Inspection is read-only, redacted and owner-scoped

Operators need cause, age, attempt history, contract metadata and current state without routine access to raw payloads. Every list query is bounded and authorized by the owning consumer.

EXECUTABLE ASSET 29-AJ - REDACTED PROJECTION

```
public sealed record PoisonListItemV1(
    Guid PoisonMessageId,
    string ConsumerName,
    Guid MessageId,
    string EnvelopeSha256,
    string? EventType,
    int? SchemaVersion,
    string TerminalReason,
    int RetryAttempts,
    DateTimeOffset FirstFailedAtUtc,
    DateTimeOffset QuarantinedAtUtc,
    string State,
    bool LegalHold,
    string ETag);

public sealed record PoisonAttemptItemV1(
    int Attempt,
    DateTimeOffset ObservedAtUtc,
    string FailureCode,
    string FailureFingerprint,
    string Dependency,
    string Disposition);
```

EDGE CONTRACT 29-AK - OPERATOR ENDPOINTS

```
GET /operations/messaging/poison
policy: MessagingEvidence.Read
filters: consumer, state, reason, from, cursor
returns: PoisonListItemV1 only

GET /operations/messaging/poison/{id}
policy: MessagingEvidence.Read
returns: PoisonListItemV1 only

GET /operations/messaging/poison/{id}/attempts
policy: MessagingEvidence.Read
returns: bounded PoisonAttemptItemV1 page

POST /operations/messaging/poison/{id}/replay-requests
policy: MessagingReplay.Request
body: reason, fixReference, expectedVersion

POST /operations/messaging/replay-requests/{id}/approve
policy: MessagingReplay.Approve
requestor and approver must differ
```

PAYLOAD ACCESS IS EXCEPTIONAL

A separate audited permission may reveal protected payload bytes for incident analysis. Listing, metrics and replay approval do not require the browser to receive them.

REPLAY AUTHORITY

Replay is a new authorized delivery of frozen evidence

A replay request does not edit the dead letter. It records who requested it, which remediation was deployed, who approved it and which exact digest will be delivered again.

EXECUTABLE ASSET 29-AL - REPLAY COMMAND AND STATE

```
public sealed record RequestPoisonReplay(
    Guid PoisonMessageId,
    string Reason,
    string FixReference,
    string ExpectedETag);

public sealed record ApprovePoisonReplay(
    Guid ReplayRequestId,
    string ApprovalReason,
    string ExpectedETag);

public sealed record ReplayRequestRecord(
    Guid ReplayRequestId,
    Guid PoisonMessageId,
    int ReplayGeneration,
    string RequestedBy,
    DateTimeOffset RequestedAtUtc,
    string Reason,
    string FixReference,
    string? ApprovedBy,
    DateTimeOffset? ApprovedAtUtc,
    ReplayState State,
    byte[] Version);
```

REPLAY PRECONDITIONS 29-AM - FAIL CLOSED

```
Require all:
  poison state is Active or Retained
  expected row version matches
  payload digest verifies after decryption
  contract version is currently supported
  fixReference resolves to deployed evidence
  requestor is not approver
  no active replay exists for the same generation
  target consumer and normal route still exist

If bytes must change:
  do not call it replay;
  create a new producer-owned fact with a new MessageId
  and an explicit causal link to the poison record.
```

REPLAY DOES NOT ERASE FAILURE

The poison record and attempt history remain immutable evidence. Resolution adds a state transition and timestamps; it does not delete the reason the original delivery became terminal.

REPLAY EXECUTION

Replay never calls domain code directly

Approval creates one durable ReplayDispatch record. The dispatcher publishes the frozen bytes to the original normal exchange with confirms; the production consumer performs admission, inbox and domain work.

EXECUTABLE ASSET 29-AN - APPROVE AND SCHEDULE REPLAY

```
public async Task<Guid> HandleAsync(
    ApprovePoisonReplay command,
    OperatorIdentity approver,
    CancellationToken cancellationToken)
{
    await using var db =
        await factory.CreateDbContextAsync(cancellationToken);
    var request = await db.ReplayRequests.SingleAsync(
        x => x.ReplayRequestId == command.ReplayRequestId,
        cancellationToken);
    var poison = await db.PoisonMessages.SingleAsync(
        x => x.PoisonMessageId == request.PoisonMessageId,
        cancellationToken);

    request.Approve(
        approver,
        command.ApprovalReason,
        command.ExpectedETag,
        clock.UtcNow);
    var dispatch = ReplayDispatchRecord.Create(
        request, poison, clock.UtcNow);
    db.ReplayDispatches.Add(dispatch);
    poison.MarkReplayScheduled(request.ReplayRequestId);

    await db.SaveChangesAsync(cancellationToken);
    return dispatch.ReplayDispatchId;
}
```

TRANSPORT CONTRACT 29-AO - REPLAY METADATA

Frozen:
MessageId, body, event type, schema version,
occurred time, producer, correlation and causation.

Added transport headers:
x-nexus-replay-request
x-nexus-replay-generation
x-nexus-poison-id
x-nexus-envelope-sha256

Route:
original normal exchange + original routing key
-> production queue -> admission -> inbox -> handler

THE INBOX IS THE REPLAY BRAKE

If the business effect already committed before a broker-level terminal outcome, replay finds the existing receipt and acknowledges without repeating meaning.

FAILURE CONVERGENCE

Every interruption has one visible recovery path

The crash oracle names the row, broker state and next action expected at each boundary. A test passes only when the system converges without an unbounded loop, silent loss or direct handler invocation.

CRASH MATRIX 29-AP - SCHEDULE, POISON AND REPLAY

Interruption	Recovery oracle
before retry commit	original remains unacked
after retry commit, before ack	redelivery finds same ticket
before poison commit	original remains unacked
after poison commit, before ack	redelivery finds same poison
dead queue import before commit	dead delivery remains unacked
dead import commit, before ack	importer resolves same poison
replay approval before commit	no dispatch exists
replay approval after commit	dispatcher finds pending replay
replay publish after confirm	possible duplicate normal delivery
replay consumed before ack	inbox makes redelivery harmless
Terminal acceptance: finite handler deliveries one active poison identity no due retry remains operator state is queryable	

BROKER EMPTY IS NOT THE PRIMARY ORACLE

The broker may be empty because work was acknowledged, expired, dropped or dead-lettered. Acceptance asserts consumer rows, attempt history, publisher confirms, queue counts and operator projection together.

DETERMINISTIC UNIT PROOF

Unit tests pin classification, budget and jitter

Pure tests use fixed time and policy identities. They reject broad transient classification, prove stable due calculations and show that both retry bounds terminate the sequence.

EXECUTABLE ASSET 29-AQ - CLASSIFICATION CASES

```
[Theory]
[MemberData(nameof(Cases))]
public void RETRY_UNIT_001_classifies_only_allowlisted_cases(
    FailureObservation observation,
    Type expectedDecision,
    string expectedCode)
{
    var actual = classifier.Classify(observation);

    actual.Should().BeOfType(expectedDecision);
    actual.Code.Should().Be(expectedCode);
}

public static TheoryData<
    FailureObservation, Type, string> Cases() => new()
{
    {
        Observations.ContractInvalid(),
        typeof(FailureDecision.Quarantine),
        "schema-unsupported"
    },
    {
        Observations.ReadDependencyUnavailable(),
        typeof(FailureDecision.ScheduleRetry),
        "dependency-unavailable"
    },
    {
        Observations.UnknownLocalCommit(),
        typeof(FailureDecision.StopWithoutAck),
        "local-commit-unknown"
    }
};
```

EXECUTABLE ASSET 29-AR - BOUNDED STABLE DELAY

```
[Fact]
public void RETRY_UNIT_002_equal_jitter_is_stable_and_bounded()
{
    var first = Backoff.EqualJitter(
        policy, messageId, nextAttempt: 4);
    var second = Backoff.EqualJitter(
        policy, messageId, nextAttempt: 4);
    var cap = Backoff.ExponentialCap(policy, 4);
    var floor = TimeSpan.FromTicks(
        cap.Ticks *
        (100 - policy.EqualJitterPercent) / 100);

    first.Should().Be(second);
    first.Should().BeGreaterThanOrEqualTo(floor);
    first.Should().BeLessThanOrEqualTo(cap);
}
```

PROVIDER-BACKED PROOF

SQL tests prove schedule and quarantine atomicity

The component lane applies production SQL Server migrations. It does not replace provider error codes, unique constraints, rowversion or transaction behavior with EF InMemory.

EXECUTABLE ASSET 29-AS - RETRY RECORD COMMITS ONCE

```
[Fact]
public async Task RETRY_DB_001_duplicate_failure_records_one_ticket()
{
    var failed = await fixture.FailedDeliveryAsync();
    var plan = fixture.RetryPlan(failed, attempt: 1);

    var left = fixture.ScheduleInNewContextAsync(failed, plan);
    var right = fixture.ScheduleInNewContextAsync(failed, plan);
    var outcomes = await Task.WhenAll(left, right);

    outcomes.Should().ContainSingle(
        x => x is FailureRecordingOutcome.RetryRecorded);
    outcomes.Should().ContainSingle(
        x => x is FailureRecordingOutcome.AlreadyRecorded);
    await fixture.Database.AssertRetryTicketCountAsync(
        failed.Consumer, failed.MessageId, attempt: 1, count: 1);
    await fixture.Database.AssertAttemptCountAsync(
        failed.Consumer, failed.MessageId, count: 1);
}
```

EXECUTABLE ASSET 29-AT - POISON AND NOTIFICATION ARE ATOMIC

```
[Fact]
public async Task RETRY_DB_002_poison_and_outbox_commit_together()
{
    var failed = await fixture.InvalidContractDeliveryAsync();

    await fixture.QuarantineAsync(failed);

    await fixture.Database.AssertPoisonCountAsync(
        failed.Consumer, failed.MessageId, 1);
    await fixture.Database.AssertFailureAttemptCountAsync(
        failed.Consumer, failed.MessageId, 1);
    await fixture.Database.AssertOutboxFactCountAsync(
        "PoisonMessageObserved.v1",
        failed.MessageId, 1);
}
```

ROLLBACK MUST ALSO BE PROVED

The companion case injects failure into the outbox insert and expects zero poison, zero attempt and zero notification rows.

BROKER-BACKED PROOF

Finite failure ends in one poison record

A disposable RabbitMQ vhost, production queue type and SQL Server database run the real consumer and retry dispatcher. The dependency fails every attempt so the policy must terminate exactly.

EXECUTABLE ASSET 29-AU - EXHAUSTION ACCEPTANCE

```
[Fact]
public async Task RETRY_RM0_001_transient_failure_is_finite()
{
    fixture.Dependency.AlwaysUnavailable();
    fixture.Policy = Policies.With(
        maxRetryAttempts: 3,
        initialDelay: TimeSpan.FromMilliseconds(50),
        maxDelay: TimeSpan.FromMilliseconds(200));
    var message = await fixture.PublishAlarmAsync();

    await fixture.RunUntilPoisonAsync(message.MessageId);

    fixture.HandlerDeliveries(message.MessageId)
        .Should().Be(4); // initial + three retries
    await fixture.Database.AssertRetryAttemptsAsync(
        message.MessageId, 3);
    await fixture.Database.AssertActiveRetryCountAsync(
        message.MessageId, 0);
    await fixture.Database.AssertPoisonCountAsync(
        fixture.ConsumerName, message.MessageId, 1);
    await fixture.Broker.AssertReadyCountAsync(
        fixture.LiveQueue, 0);
}
```

EXECUTABLE ASSET 29-AV - DLX SAFETY-NET ACCEPTANCE

```
[Fact]
public async Task RETRY_RM0_002_reject_is_imported_as_evidence()
{
    var message = await fixture.PublishMalformedAsync();
    await fixture.ForceBrokerRejectAsync(message);
    await fixture.StartDeadLetterImporterAsync();

    var poison = await fixture.Eventually.PoisonAsync(
        fixture.ConsumerName, message.MessageId);

    poison.TerminalReason.Should().Be("broker-terminal");
    poison.BrokerReason.Should().Be("rejected");
    poison.BrokerSourceQueue.Should().Be(fixture.LiveQueue);
    await fixture.Broker.AssertReadyCountAsync(
        fixture.DeadQueue, 0);
}
```

OPERATOR REPLAY PROOF

End-to-end replay proves the inbox still wins

The test quarantines an unsupported contract, deploys the compatible consumer, records separate request and approval identities, and waits for the normal subscription to consume the confirmed replay.

EXECUTABLE ASSET 29-AW - SAFE REPLAY ACCEPTANCE

```
[Fact]
public async Task RETRY_E2E_001_replay_uses_normal_inbox_path()
{
    var message = await fixture.PublishUnsupportedV2Async();
    var poison = await fixture.WaitForPoisonAsync(
        message.MessageId);
    await fixture.DeployConsumerSupportingV2Async();

    var request = await fixture.RequestReplayAsync(
        poison.PoisonMessageId,
        requestedBy: "operator-a",
        fixReference: fixture.DeploymentEvidenceId);
    await fixture.ApproveReplayAsync(
        request.ReplayRequestId,
        approvedBy: "operator-b");
    await fixture.WaitForReplayConsumptionAsync(request);

    fixture.ReplayPublication.MessageId
        .Should().Be(message.MessageId);
    fixture.ReplayPublication.Body
        .Should().Equal(message.Body);
    await fixture.Database.AssertInboxCountAsync(
        fixture.ConsumerName, message.MessageId, 1);
    await fixture.Database.AssertOwnedEffectCountAsync(
        message.MessageId, 1);
    await fixture.Database.AssertPoisonStateAsync(
        poison.PoisonMessageId, PoisonState.Resolved);
}
```

EXECUTABLE ASSET 29-AX - REPLAYING PRIOR SUCCESS IS A NO-OP

```
[Fact]
public async Task RETRY_E2E_002_inbox_blocks_repeated_effect()
{
    var poison = await fixture.BrokerTerminalAfterCommitAsync();

    await fixture.RequestApproveAndReplayAsync(poison);
    await fixture.WaitForReplayConsumptionAsync();

    await fixture.Database.AssertInboxCountAsync(
        poison.ConsumerName, poison.MessageId, 1);
    await fixture.Database.AssertOwnedEffectCountAsync(
        poison.MessageId, 1);
}
```

A DIRECT HANDLER TEST IS INSUFFICIENT

The acceptance path includes authorization, replay persistence, publisher confirm, exchange routing, consumer admission and inbox resolution.

OPERATIONAL VISIBILITY

Operations can see pressure without cardinality explosions

Metrics answer whether retries are accumulating, aging, exhausting and replaying. MessageId, poison id, exception text and payload digest remain structured trace or log fields, never metric labels.

TELEMETRY CONTRACT 29-AY - BOUNDED DIMENSIONS

Metric	Allowed labels
nexus_retry_decisions_total	consumer, policy, disposition, code
nexus_retry_due	consumer, policy
nexus_retry_oldest_due_seconds	consumer, policy
nexus_retry_dispatch_total	consumer, outcome
nexus_poison_active	consumer, terminal_reason
nexus_replay_requests_total	consumer, outcome
nexus_dead_queue_ready	queue

Structured log fields:
MessageId, RetryTicketId, PoisonMessageId, ReplayRequestId, Attempt, DueAtUtc, FailureFingerprint, CorrelationId.

Trace rule:
each delayed attempt starts a new consumer span
and links to prior trace context; it does not pretend that
a multi-hour retry is one continuously active span.

ALERT INVENTORY 29-AZ - ACTIONABLE CONDITIONS

Condition	Required operator action
oldest due age > policy objective	inspect dispatcher/dependency
due count grows while publish=0	inspect claim and broker confirms
active poison > 0	triage terminal evidence
unclassified failure > 0	stop and extend classifier safely
dead queue ready > 0	inspect importer and broker policy
replay approved but not published	inspect replay dispatcher lease
protection key unavailable	block replay; restore key access

PARTIAL PROGRESS IS VISIBLE

The operator can distinguish waiting, due, claimed, published, quarantined, replay-requested and replay-resolved work. A single queue depth cannot express those states.

SECURITY AND RETENTION

Failure evidence has stricter custody than ordinary telemetry

Quarantined envelopes may contain sensitive operational context. Encryption, key identity, retention, legal hold and replay authorization are part of the message guarantee rather than later administrative decoration.

CONTROL LEDGER 29-BA - EVIDENCE CUSTODY

Control	Enforcement
payload at rest	authenticated encryption
encryption identity	ProtectionKeyId on every record
list/read access	MessagingEvidence.Read
payload reveal	separate audited permission
replay request	MessagingReplay.Request
replay approval	MessagingReplay.Approve
separation of duties	requestor != approver
immutable digest	verify before reveal or replay
legal hold	blocks purge and key retirement
retention purge	signed manifest + row counts
backup/restore	includes tickets, poison and replay

RETENTION RULE 29-BB - FAIL CLOSED

RetryTicket:	retain through terminal resolution and evidence horizon.
PoisonMessage:	retain while Active, replayable, legally held or referenced by an unresolved incident.
ReplayRequest and FailureAttempt:	retain with the poison evidence and audit policy.
Purge only when:	
state is terminal	
AND no legal/evidence hold exists	
AND backup restore horizon cannot reintroduce the delivery	
AND a signed purge manifest records count and digest.	

KEY DELETION IS DATA DELETION

Retiring a protection key while evidence remains legally or operationally required destroys replay and inspection capability. Key retention must cover the longest evidence horizon.

SECURITY ACCEPTANCE

Authorization is exercised through the production edge

Security evidence calls the same HTTP pipeline used by the Operator Console. It proves redacted reads, denied payload access, separation of duties and an audit record for every replay transition.

EXECUTABLE ASSET 29-BB1 - REDACTION AND ROLE GATES

```
[Fact]
public async Task RETRY_SEC_001_operator_surface_is_fail_closed()
{
    var poison = await fixture.CreatePoisonAsync();

    var reader = fixture.ClientAs("evidence-reader");
    var item = await reader.GetFromJsonAsync<PoisonListItemV1>(
        $"/operations/messaging/poison/" +
        $"{poison.PoisonMessageId}");
    item.Should().NotBeNull();
    item!.EnvelopeSha256.Should().Be(poison.DigestHex);
    fixture.LastResponseBody.Should().NotContain(
        poison.PlaintextMarker);

    var reveal = await reader.GetAsync(
        $"/operations/messaging/poison/" +
        $"{poison.PoisonMessageId}/payload");
    reveal.StatusCode.Should().Be(HttpStatusCode.Forbidden);

    var request = await fixture.RequestReplayAsync(
        poison, requestedBy: "operator-a");
    var sameActorApproval = await fixture.ApproveReplayAsync(
        request, approvedBy: "operator-a");
    sameActorApproval.StatusCode
        .Should().Be(HttpStatusCode.Conflict);
}
```

EXECUTABLE ASSET 29-BB2 - IMMUTABLE AUDIT TRAIL

```
[Fact]
public async Task RETRY_SEC_002_replay_transitions_are_audited()
{
    var poison = await fixture.CreatePoisonAsync();
    var request = await fixture.RequestReplayAsync(
        poison, requestedBy: "operator-a");
    await fixture.ApproveReplayAsync(
        request, approvedBy: "operator-b");

    var audit = await fixture.Audit.ReadAsync(
        request.ReplayRequestId);
    audit.Select(x => x.Action).Should().Equal(
        "ReplayRequested", "ReplayApproved");
    audit.Should().OnlyContain(
        x => x.ActorId is "operator-a" or "operator-b");
    audit.Should().OnlyContain(
        x => x.PayloadDigest == poison.DigestHex);
}
```

NO SHARED OPERATOR CREDENTIAL

The acceptance fixture uses distinct authenticated principals. A test that changes role claims on one identity cannot prove separation of duties.

FAIL-CLOSED ACCEPTANCE

The chapter closes only on identified evidence

Code shape is not runtime proof. The evidence gate requires discovered tests, pinned provider and broker identities, effective policy digests, database migration identity and reproducible result hashes.

EXECUTABLE ASSET 29-BC - EVIDENCE GATE

```
public enum RetryEvidenceStatus { Pending, Passed, Failed }

public sealed record RetryEvidence(
    string EvidenceId,
    RetryEvidenceStatus Status,
    int DiscoveredTests,
    string BuildIdentity,
    string ServerImageDigest,
    string ServerVersion,
    string ClientPackageVersion,
    string DatabaseMigrationId,
    string RetryPolicySha256,
    string BrokerPolicySha256,
    string ResultSha256);

public static class RetryEvidenceGate
{
    private static readonly string[] Required =
    [
        "RETRY-UNIT-001", "RETRY-UNIT-002",
        "RETRY-DB-001", "RETRY-DB-002",
        "RETRY-DISPATCH-001", "RETRY-RMQ-001",
        "RETRY-RMQ-002", "RETRY-E2E-001",
        "RETRY-E2E-002", "RETRY-SEC-001"
    ];

    public static void Accept(
        IReadOnlyCollection<RetryEvidence> evidence)
    {
        var byId = evidence.ToDictionary(
            x => x.EvidenceId, StringComparer.Ordinal);
        foreach (var id in Required)
        {
            if (!byId.TryGetValue(id, out var item))
                throw new MissingRetryEvidence(id);
            if (item.Status != RetryEvidenceStatus.Passed ||
                item.DiscoveredTests <= 0)
                throw new RetryEvidenceNotAccepted(id);
            if (new[]
                {
                    item.BuildIdentity,
                    item.ServerImageDigest,
                    item.ServerVersion,
                    item.ClientPackageVersion,
                    item.DatabaseMigrationId,
                    item.RetryPolicySha256,
                    item.BrokerPolicySha256,
                    item.ResultSha256
                }.Any(String.IsNullOrEmpty))
                throw new UnidentifiedRetryEvidence(id);
        }
    }
}
```

ZERO DISCOVERY IS FAILURE

An empty test selection, missing environment identity or stale policy digest cannot be recorded as a successful retry experiment.

EVIDENCE RECORD AND DECISION

The manuscript separates specified behavior from executed proof

The acceptance manifest is intentionally incomplete until the repository pins versions and a disposable SQL Server plus RabbitMQ cluster execute the required failure scenarios.

EVIDENCE MANIFEST 29-BD - CURRENT HONEST STATE

```
{
  "evidenceId": "RETRY-REVIEW-001",
  "chapter": 29,
  "status": "PENDING",
  "buildIdentity": null,
  "serverImageDigest": null,
  "serverVersion": null,
  "clientPackageVersion": null,
  "databaseMigrationId": null,
  "retryPolicySha256": null,
  "brokerPolicySha256": null,
  "tests": {
    "RETRY-UNIT-001": "PENDING",
    "RETRY-UNIT-002": "PENDING",
    "RETRY-DB-001": "PENDING",
    "RETRY-DB-002": "PENDING",
    "RETRY-DISPATCH-001": "PENDING",
    "RETRY-RMQ-001": "PENDING",
    "RETRY-RMQ-002": "PENDING",
    "RETRY-E2E-001": "PENDING",
    "RETRY-E2E-002": "PENDING",
    "RETRY-SEC-001": "PENDING"
  },
  "discoveredTests": null,
  "resultSha256": null,
  "accepted": false
}
```

ADR-027 - DURABLE BOUNDED RETRY AND QUARANTINE

Status: Proposed

Decision

Consumer-owned SQL stores RetryTicket, FailureAttempt, PoisonMessage and ReplayRequest truth.
Retry uses persisted exponential delay with equal jitter.
Application poison is recorded before broker acknowledgement.
Broker DLX is an imported safety net.
Replay republishes frozen bytes through the normal route.

Rejected alternatives

unbounded nack/queue
Task.Delay inside the consumer callback
exception-type catch-all retry
per-message TTL as the only retry authority
x-death as the application attempt counter
manual exchange publish without approval or inbox
deleting poison evidence after successful replay

REVIEW TRIGGER

Revisit ADR-027 when the broker major version, queue type, consumer identity, classifier, retry algorithm, evidence provider, replay authority or retention horizon changes.

IMPLEMENTATION PATH

Prove finite failure before reasoning about ordering

The sequence first establishes classified failure, durable retry and terminal evidence. Chapter 30 may then add ordering and contract-evolution policy on top of a backbone that no longer hides failure in endless redelivery.

IMPLEMENTATION SEQUENCE 29-BE

1. Publish one RetryPolicy identity per subscription.
2. Implement the allowlisted phase-aware failure classifier.
3. Prove attempt and elapsed-time budget termination.
4. Persist stable equal-jitter DueAtUtc once per attempt.
5. Apply RetryTicket, FailureAttempt and PoisonMessage migrations.
6. Record retry or poison before acknowledging failed delivery.
7. Add fresh-context unknown-commit resolution.
8. Reuse claim, lease, mandatory routing and publisher confirms.
9. Import broker DLX outcomes into durable evidence.
10. Add redacted inspection and separated replay approval.
11. Publish replay through the normal exchange and inbox.
12. Execute crash, exhaustion, DLX and replay acceptance lanes.
13. Capture versions, policy digests, commands and result hashes.
14. Sign ADR-027 only after the evidence gate passes.

29.2 Official technical anchors

RabbitMQ Dead Letter Exchanges: <https://www.rabbitmq.com/docs/dlx>

RabbitMQ Quorum Queues: <https://www.rabbitmq.com/docs/quorum-queues>

RabbitMQ .NET Client API Guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>

RabbitMQ TTL and Expiration: <https://www.rabbitmq.com/docs/ttl>

RabbitMQ Policies: <https://www.rabbitmq.com/docs/policies>

Microsoft transient-fault guidance: <https://learn.microsoft.com/azure/well-architected/design-guides/handle-transient-faults>

CHAPTER CHECKPOINT

Can the team name every retryable failure, calculate the last possible attempt time, find the durable owner of a waiting delivery, prove one poison record before ack, and replay exact bytes without bypassing the inbox? If any answer is no, the chapter remains PENDING.

NEXT - CHAPTER 30

Ordering, Duplication, and Contract Evolution will demonstrate out-of-order and duplicate events, per-aggregate sequence handling and compatible schema evolution while stating what RabbitMQ does not guarantee.

CHAPTER 29 - CLOSURE

What this retry and poison policy proves—and what it does not

A retry policy is truthful when every attempt is finite, classified, durable and inspectable. It is not a substitute for ordering, idempotency or operator judgment.

PROOF BOUNDARY 29-A

Specified and testable	Not proved by this chapter
Transient failures enter a finite, classified retry path.	Exactly-once broker delivery.
Equal-jitter delay is persisted with attempt identity.	Global or per-key ordering across all failures.
Terminal poison state records reason, payload identity and policy.	That every poison message is automatically safe to replay.
Operator replay is a separate authorized command with audit evidence.	That a retry is harmless without consumer idempotency.
Budget exhaustion is visible in metrics, traces and durable state.	That recovery completed in a production environment.

EVIDENCE STATUS

The policy, state machine and test surfaces are executable reference material. Provider-, broker- and deployment-backed results remain PENDING until the named fault suite produces versioned artifacts.

Reader checkpoint

- Can every exception category be traced to retry, poison or fail-fast behavior?
- Can an operator explain why a message stopped and who authorized replay?
- Can replay preserve the original identity while recording a new operation?
- Can the system prove that retry budgets do not reset after process restart?

PART V - THE MESSAGE BACKBONE

30. Ordering, Duplication, and Contract Evolution

A broker can deliver durable facts without delivering business meaning in the order a consumer needs. NEXUS-1 therefore treats arrival order as transport evidence, publication sequence as producer-owned contract data, and compatibility as a tested relationship between exact versions.

CHAPTER DECISION

Each ordered public stream carries a stable producer stream identity and a monotonic publication sequence assigned in the producer transaction. Each consumer advances its own checkpoint atomically with its local effect, buffers bounded gaps durably, and treats identity reuse or sequence disagreement as terminal evidence. Contract versions are exact; rollout is dual-read and single-write until compatibility evidence permits retirement.

30.1 The exact guarantee

This chapter does not promise global order, duplicate-free delivery, or automatic schema compatibility. It specifies per-consumer, per-producer-stream convergence when the producer supplies a contiguous sequence and the configured gap and retention horizons are not exceeded.

GUARANTEE LEDGER 30-A

Boundary	Mechanism	Allowed claim	Not claimed
producer state -> sequence	same local transaction	position belongs to committed fact	broker order
delivery -> local effect	inbox + stream checkpoint	contiguous effect-once	exactly-once delivery
gap -> later progress	bounded durable deferral	restart-safe waiting	infinite buffering
V1 -> V2	exact readers + adapters	tested compatibility window	broker schema validation

ORDINARY DEVELOPER PROBLEM

Queue position is not a business clock

The naive design sorts by OccurredAtUtc or trusts the first callback to be the earliest business transition. Clock skew, multiple publishing channels, consumer concurrency, redelivery and priority can all make that inference false.

REJECTED ASSET 30-B - ARRIVAL ORDER MASQUERADING AS CAUSALITY

```
// Wrong: timestamp and callback order are observations, not authority.
public async Task HandleAsync(AlarmFloodDetectedV1 message)
{
    if (message.OccurredAtUtc <= _lastObservedAt)
        return;

    await projection.ApplyAsync(message);
    _lastObservedAt = message.OccurredAtUtc;
}
```

RABBITMQ GUARANTEE LEDGER 30-C

Observation	What RabbitMQ says	NEXUS-1 consequence
one channel -> one queue	enqueue follows publish order	useful transport fact, not aggregate authority
multiple channels/connections	publications interleave	no global producer order
multiple consumers	dequeue is FIFO; processing races	completion order can differ
requeue/redelivery	effective order can change	idempotency and sequence gate remain required
priority queues	higher priority may pass lower	never encode causality in queue position

PLAIN-LANGUAGE PRINCIPLE

The broker moves messages. The producer names business order. The consumer proves which contiguous position it has applied.

THREE INDEPENDENT IDENTITIES

Delivery, meaning, and order must not share one key

EXECUTABLE ASSET 30-D - PRODUCER-OWNED STREAM POSITION

```
public readonly record struct ProducerStreamId
{
    public string Value { get; }

    public ProducerStreamId(string value)
    {
        if (string.IsNullOrEmpty(value) ||
            value.Length > 160)
            throw new InvalidProducerStreamId(value);
        Value = value;
    }
}

public readonly record struct PublicationSequence
{
    public long Value { get; }

    public PublicationSequence(long value)
    {
        if (value < 1)
            throw new InvalidPublicationSequence(value);
        Value = value;
    }
}

public sealed record ProducerStreamPosition(
    ProducerStreamId StreamId,
    PublicationSequence Sequence);
```

IDENTITY LEDGER 30-E

Identity	Scope	Purpose	Never substitutes for
MessageId	public fact on wire	transport duplicate and digest conflict	business sequence
FactKey / domain key	producer semantic intent	producer retry and business convergence	delivery identity
StreamId + Sequence	one producer public stream	contiguous consumer progress	global total order
SchemaVersion	one EventType	exact decoder and compatibility policy	stream reset

SEQUENCE SCOPE

The sequence belongs to one producer-owned public aggregate stream. It continues across compatible contract versions and is never inferred from time, queue offset, delivery tag or publisher-confirm sequence number.

PRODUCER ALLOCATION

Sequence reservation shares the outbox transaction

A committed fact cannot borrow a number from an external counter and then fail before storing its outbox row. The producer locks its local stream row, reserves the required finite range, and stages owner state plus exact envelopes before the same transaction commits.

EXECUTABLE ASSET 30-F - SQL SERVER PUBLICATION STREAM

```
CREATE TABLE messaging.PublicationStream
(
    StreamId varchar(160) NOT NULL,
    LastAssignedSequence bigint NOT NULL,
    Version rowversion NOT NULL,

    CONSTRAINT PK_PublicationStream PRIMARY KEY (StreamId),
    CONSTRAINT CK_PublicationStream_Last
        CHECK (LastAssignedSequence >= 0)
);

-- Executed inside the producer's existing local transaction.
UPDATE messaging.PublicationStream WITH (UPDLOCK, HOLDLOCK)
SET LastAssignedSequence = LastAssignedSequence + @count
OUTPUT
    deleted.LastAssignedSequence + 1 AS FirstReserved,
    inserted.LastAssignedSequence AS LastReserved
WHERE StreamId = @streamId;
```

EXECUTABLE ASSET 30-G - ONE TRANSACTION, FINITE RESERVATION

```
await using var tx = await db.Database.BeginTransactionAsync(ct);

var events = aggregate.DequeuePublicFacts()
    .OrderBy(x => x.Ordinal)
    .ToArray();
var range = await sequences.ReserveAsync(
    aggregate.PublicStreamId, events.Length, ct);

foreach (var pair in events.Zip(range.Positions))
{
    var frozen = serializer.Serialize(
        pair.First, pair.Second);
    db.OutboxMessages.Add(
        OutboxMessageRow.Pending(frozen, clock.UtcNow));
}

await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

ROLLBACK RULE

A rolled-back transaction consumes no durable position. A committed range contains one outbox row for every reserved number. A provider-backed test must reject any committed hole or duplicate assignment.

CONTRACT POSITION

Order data belongs to the public payload contract

Chapter 20's strict `EnvelopeDocumentV1` remains unchanged. Ordered contracts carry a reviewed stream position inside their producer-owned payload. New contracts such as `AlarmFloodDetected.v1` include it from their first published version.

EXECUTABLE ASSET 30-H - ORDERED ALARMMANAGEMENT FACT

```
[JsonUnmappedMemberHandling(
    JsonUnmappedMemberHandling.Disallow)]
public sealed record AlarmFloodDetectedV1(
    [property: JsonRequired] string AlarmScopeId,
    [property: JsonRequired] long WindowStartUnixMs,
    [property: JsonRequired] long WindowEndUnixMs,
    [property: JsonRequired] int AlarmCount,
    [property: JsonRequired] string DetectionPolicyId,
    [property: JsonRequired] ProducerStreamPositionV1 Stream);

[JsonUnmappedMemberHandling(
    JsonUnmappedMemberHandling.Disallow)]
public sealed record ProducerStreamPositionV1(
    [property: JsonRequired] string StreamId,
    [property: JsonRequired] long Sequence);

public sealed record OrderedContractDescriptor(
    string Producer,
    string EventType,
    int SchemaVersion,
    bool RequiresContiguousSequence);
```

EXECUTABLE ASSET 30-I - ADMISSION BINDS PAYLOAD TO REGISTRY POLICY

```
var descriptor = registry.GetRequired(
    envelope.Producer,
    envelope.EventType,
    envelope.SchemaVersion,
    envelope.ContentType);

var contract = descriptor.DecodeStrict(envelope.Payload);
var position = descriptor.ReadPosition(contract);

if (descriptor.RequiresContiguousSequence &&
    position is null)
    throw new OrderedContractMissingPosition();

return new OrderedDelivery(
    admitted, descriptor, contract, position);
```

LEGACY POLICY IS EXPLICIT

A historical contract without a sequence is not assigned one from arrival time. Its subscription is declared unordered, migrated through an owner-approved bridge, or rejected. No consumer silently fabricates authority.

CONSUMER DECISION ALGEBRA

Every arrival is classified against durable progress

DECISION FLOW 30-H - ONE STREAM, FOUR EXPLICIT OUTCOMES

NEXT

seq = last + 1

apply + checkpoint**STALE**

seq <= last

digest match -> no-op**CONFLICT**

seq <= last

digest differs -> poison**GAP**

seq > last + 1

defer exact bytes

EXECUTABLE ASSET 30-J - SEQUENCE OUTCOMES

```
public abstract record SequenceDecision
{
    public sealed record ApplyNext(
        long Sequence) : SequenceDecision;
    public sealed record StaleDuplicate(
        long Sequence) : SequenceDecision;
    public sealed record SequenceConflict(
        long Sequence, string Code) : SequenceDecision;
    public sealed record DeferGap(
        long Expected, long Observed) : SequenceDecision;
}

public static SequenceDecision Decide(
    long lastApplied,
    long observed,
    bool knownDigestMatches)
{
    if (observed == lastApplied + 1)
        return new SequenceDecision.ApplyNext(observed);
    if (observed <= lastApplied)
        return knownDigestMatches
            ? new SequenceDecision.StaleDuplicate(observed)
            : new SequenceDecision.SequenceConflict(
                observed, "sequence-digest-conflict");
    return new SequenceDecision.DeferGap(
        lastApplied + 1, observed);
}
```

NO TIMESTAMP BRANCH**OccurredAtUtc remains audit evidence. It never decides ApplyNext, StaleDuplicate, SequenceConflict or DeferGap.**

CONSUMER-OWNED PERSISTENCE

Checkpoint, applied digest, and deferred bytes have distinct jobs

EXECUTABLE ASSET 30-K - SQL SERVER SEQUENCE SCHEMA

```
CREATE TABLE messaging.ConsumerStreamCheckpoint
(
    ConsumerName varchar(120) NOT NULL,
    Producer varchar(80) NOT NULL,
    StreamId varchar(160) NOT NULL,
    LastAppliedSequence bigint NOT NULL,
    UpdatedAtUtc datetimeoffset(7) NOT NULL,
    Version rowversion NOT NULL,
    CONSTRAINT PK_ConsumerStreamCheckpoint PRIMARY KEY
        (ConsumerName, Producer, StreamId),
    CONSTRAINT CK_StreamCheckpoint_Last
        CHECK (LastAppliedSequence >= 0)
);

CREATE TABLE messaging.AppliedPublication
(
    ConsumerName varchar(120) NOT NULL,
    Producer varchar(80) NOT NULL,
    StreamId varchar(160) NOT NULL,
    PublicationSequence bigint NOT NULL,
    SemanticSha256 binary(32) NOT NULL,
    MessageId uniqueidentifier NOT NULL,
    AppliedAtUtc datetimeoffset(7) NOT NULL,
    CONSTRAINT PK_AppliedPublication PRIMARY KEY
        (ConsumerName, Producer, StreamId,
        PublicationSequence),
    CONSTRAINT UQ_AppliedPublication_Message UNIQUE
        (ConsumerName, MessageId)
);
```

EXECUTABLE ASSET 30-L - DEFERRED DELIVERY IDENTITY

```
CREATE TABLE messaging.DeferredDelivery
(
    ConsumerName varchar(120) NOT NULL,
    Producer varchar(80) NOT NULL,
    StreamId varchar(160) NOT NULL,
    PublicationSequence bigint NOT NULL,
    MessageId uniqueidentifier NOT NULL,
    SemanticSha256 binary(32) NOT NULL,
    ProtectedEnvelope varbinary(max) NOT NULL,
    ProtectionKeyId varchar(100) NOT NULL,
    FirstObservedAtUtc datetimeoffset(7) NOT NULL,
    GapDeadlineUtc datetimeoffset(7) NOT NULL,
    CONSTRAINT PK_DeferredDelivery PRIMARY KEY
        (ConsumerName, Producer, StreamId,
        PublicationSequence),
    CONSTRAINT UQ_DeferredDelivery_Message UNIQUE
        (ConsumerName, MessageId)
);
```

CONCURRENCY BOUNDARY

The stream row serializes one stream, not the whole consumer

Parallel streams should progress independently. The provider adapter acquires or creates exactly one checkpoint row under a short transaction and protects updates with the production provider's locking and rowversion behavior.

EXECUTABLE ASSET 30-M - STREAM-SCOPED LEASE

```
public sealed record LockedStream(
    ConsumerName Consumer,
    string Producer,
    ProducerStreamId StreamId,
    long LastAppliedSequence,
    byte[] Version);

public interface IConsumerStreamStore
{
    Task<LockedStream> LockAsync(
        ConsumerName consumer,
        string producer,
        ProducerStreamId streamId,
        CancellationToken cancellationToken);

    Task AdvanceAsync(
        LockedStream locked,
        long next,
        byte[] semanticSha256,
        Guid messageId,
        DateTimeOffset now,
        CancellationToken cancellationToken);
}
```

EXECUTABLE ASSET 30-N - PROVIDER LOCK SHAPE

```
SELECT ConsumerName, Producer, StreamId,
       LastAppliedSequence, Version
FROM messaging.ConsumerStreamCheckpoint
WITH (UPDLOCK, HOLDLOCK, ROWLOCK)
WHERE ConsumerName = @consumer
AND Producer = @producer
AND StreamId = @streamId;

-- Create-on-miss is protected by the same serializable key range.
-- The transaction remains short and never performs network I/O.
```

PROVIDER EVIDENCE

EF InMemory and SQLite cannot prove SQL Server key-range locks, rowversion conflicts or concurrent unique-key arbitration. The acceptance suite uses the selected SQL Server image and deterministic barriers.

SEQUENCE GATE

Admission, inbox, and sequence classification share one path

EXECUTABLE ASSET 30-O - ORDERED CONSUMER ENTRY POINT

```
public async Task<ConsumeOutcome> ConsumeAsync(
    OrderedDelivery delivery,
    CancellationToken cancellationToken)
{
    InboxIdentity.DemandValid(delivery.Admitted);
    var knownReceipt = await inbox.TryReadAsync(
        delivery.Admitted, cancellationToken);
    if (knownReceipt is not null)
        return knownReceipt;

    await using var tx = await db.Database
        .BeginTransactionAsync(cancellationToken);
    var stream = await streams.LockAsync(
        delivery.Admitted.Consumer,
        delivery.Admitted.Producer,
        delivery.Position.StreamId,
        cancellationToken);

    var knownDigest = await applied.TryReadDigestAsync(
        stream, delivery.Position.Sequence,
        cancellationToken);
    var decision = SequenceGate.Decide(
        stream.LastAppliedSequence,
        delivery.Position.Sequence.Value,
        knownDigest is not null &&
        Fixed.Equals(knownDigest, delivery.SemanticSha256));

    return await ExecuteAsync(
        stream, delivery, decision, tx,
        cancellationToken);
}
```

INBOX REMAINS FIRST

The same **ConsumerName + MessageId** still collapses transport redelivery exactly as Chapter 28 specified. Sequence logic handles a different question: whether new message identity represents the next, stale, conflicting or future publication.

DECISION AND ACKNOWLEDGEMENT 30-P

Decision	Durable transaction	Broker action
ApplyNext	effect + checkpoint + receipt	ack after commit
StaleDuplicate	matching receipt/no-op evidence	ack after commit
SequenceConflict	Chapter 29 poison evidence	ack after poison commit
DeferGap	protected exact bytes + receipt policy	ack after defer commit
unknown commit	resolve from fresh context	no ack until known

CONTIGUOUS APPLY

The effect and checkpoint advance atomically

EXECUTABLE ASSET 30-Q - APPLYNEXT TRANSACTION

```
private async Task<ConsumeOutcome> ApplyNextAsync(
    LockedStream stream,
    OrderedDelivery delivery,
    CancellationToken cancellationToken)
{
    var effect = await localHandler.ApplyAsync(
        delivery.Contract, cancellationToken);

    await streams.AdvanceAsync(
        stream,
        delivery.Position.Sequence.Value,
        delivery.SemanticSha256,
        delivery.Admitted.MessageId,
        clock.UtcNow,
        cancellationToken);

    db.InboxReceipts.Add(
        InboxReceipt.Completed(
            delivery.Admitted,
            effect.Code,
            clock.UtcNow));

    foreach (var fact in effect.PendingPublicFacts)
        db.OutboxMessages.Add(fact);

    await db.SaveChangesAsync(cancellationToken);
    await db.Database.CommitTransactionAsync(
        cancellationToken);
    return new ConsumeOutcome.Committed(
        delivery.Admitted.MessageId, effect.Code);
}
```

ATOMIC BOUNDARY 30-R

Commits together	May happen later	Forbidden inside transaction
owned aggregate/projection	broker acknowledgement	HTTP or broker call
checkpoint + applied digest	deferred contiguous drain	another service database
inbox receipt	outbox dispatch	email or external callback
new outbox facts	metrics export	schema registry lookup over network

CRASH ORACLE

A crash before commit leaves checkpoint, receipt and effect absent. A crash after commit but before ack causes redelivery; the inbox returns Duplicate and the checkpoint does not advance twice.

STALE AND CONFLICTING PUBLICATIONS

Old position is harmless only when meaning matches

EXECUTABLE ASSET 30-S - STALE DECISION RESOLVER

```
public async Task<SequenceDecision> ResolveStaleAsync(
    LockedStream stream,
    OrderedDelivery delivery,
    CancellationToken cancellationToken)
{
    var prior = await db.AppliedPublications
        .AsNoTracking()
        .SingleOrDefaultAsync(
            x => x.ConsumerName == stream.Consumer.Value &&
                x.Producer == stream.Producer &&
                x.StreamId == stream.StreamId.Value &&
                x.PublicationSequence ==
                    delivery.Position.Sequence.Value,
            cancellationToken);

    if (prior is null)
        return new SequenceDecision.SequenceConflict(
            delivery.Position.Sequence.Value,
            "checkpoint-without-position-evidence");

    return Fixed.Equals(
        prior.SemanticSha256,
        delivery.SemanticSha256)
        ? new SequenceDecision.StaleDuplicate(
            delivery.Position.Sequence.Value)
        : new SequenceDecision.SequenceConflict(
            delivery.Position.Sequence.Value,
            "sequence-digest-conflict");
}
```

DUPLICATE MEANING MATRIX 30-T

Arrival	Identity evidence	Outcome
same MessageId, same envelope digest	Chapter 28 receipt	transport Duplicate
new MessageId, same sequence + semantic digest	AppliedPublication	StaleDuplicate no-op
same MessageId, different envelope digest	receipt digest mismatch	identity-conflict poison
new MessageId, same sequence, different meaning	semantic digest mismatch	sequence-digest-conflict poison
new MessageId, next sequence	checkpoint + 1	ApplyNext

DIGEST SCOPE

SemanticSha256 is computed from the version adapter's canonical internal meaning, not from transport headers. Equivalent V1 and V2 facts can therefore agree without pretending that their wire bytes are identical.

DURABLE GAPS

Future position is stored before the original delivery is acknowledged

EXECUTABLE ASSET 30-U - GAP DEFERRAL

```
private async Task<ConsumeOutcome> DeferAsync(
    LockedStream stream,
    OrderedDelivery delivery,
    GapPolicy policy,
    CancellationToken cancellationToken)
{
    policy.DemandWithinBounds(
        stream.LastAppliedSequence,
        delivery.Position.Sequence.Value,
        await deferred.ReadUsageAsync(stream, cancellationToken),
        delivery.Admitted.EnvelopeBytes.Length);

    db.DeferredDeliveries.Add(
        DeferredDelivery.Create(
            stream,
            delivery.Position.Sequence.Value,
            delivery.Admitted.MessageId,
            delivery.SemanticSha256,
            protector.Protect(
                delivery.Admitted.EnvelopeBytes),
            clock.UtcNow,
            clock.UtcNow + policy.MaxGapAge));

    await db.SaveChangesAsync(cancellationToken);
    await db.Database.CommitTransactionAsync(
        cancellationToken);

    return new ConsumeOutcome.Committed(
        delivery.Admitted.MessageId,
        "sequence-gap-deferred");
}
```

RECEIPT POLICY

The original broker delivery may be acknowledged after the deferred row commits. It does not receive a normal completed inbox receipt yet; otherwise the later drain would be mistaken for a duplicate before applying the missing effect.

EXECUTABLE ASSET 30-V - DEFERRED DUPLICATE RESOLVER

```
public async Task<ConsumeOutcome> ResolveDeferredRaceAsync(
    OrderedDelivery delivery,
    CancellationToken cancellationToken)
{
    await using var fresh =
        await contexts.CreateDbContextAsync(cancellationToken);
    var existing = await fresh.DeferredDeliveries
        .AsNoTracking()
        .SingleOrDefaultAsync(
            x => x.ConsumerName ==
                delivery.Admitted.Consumer.Value &&
                x.MessageId == delivery.Admitted.MessageId,
            cancellationToken);

    if (existing is null)
        return new ConsumeOutcome.Unknown(
            delivery.Admitted.MessageId);
    if (existing.PublicationSequence !=
        delivery.Position.Sequence.Value ||
        !Fixed.Equals(
            existing.SemanticSha256,
            delivery.SemanticSha256))
        return new ConsumeOutcome.IdentityConflict(
            delivery.Admitted.MessageId,
            "deferred-identity-conflict");

    return new ConsumeOutcome.Committed(
        delivery.Admitted.MessageId,
        "sequence-gap-already-deferred");
}
```

CONTIGUOUS DRAIN

The missing fact unlocks only the next durable position

EXECUTABLE ASSET 30-W - BOUNDED CONTIGUOUS DRAINER

```
public async Task<int> DrainAsync(
    StreamKey key,
    int maximum,
    CancellationToken cancellationToken)
{
    var applied = 0;
    while (applied < maximum)
    {
        var checkpoint = await store.ReadAsync(
            key, cancellationToken);
        var next = await deferred.TryClaimAsync(
            key,
            checkpoint.LastAppliedSequence + 1,
            cancellationToken);
        if (next is null)
            break;

        var bytes = protector.Unprotect(
            next.ProtectedEnvelope,
            next.ProtectionKeyId);
        var outcome = await normalConsumer.ConsumeDeferredAsync(
            next, bytes, cancellationToken);

        if (outcome is not ConsumeOutcome.Committed and
            not ConsumeOutcome.Duplicate)
            break;

        await deferred.RemoveClaimedAsync(
            next, cancellationToken);
        applied++;
    }
    return applied;
}
```

DRAIN INVARIANTS 30-X

Invariant	Enforcement	Failure action
only last + 1 is selected	PK lookup under stream lock	stop drain
same exact bytes	protected envelope + digest	quarantine corruption
normal handler path	admission + inbox + gate	no direct domain call
bounded work	maximum per invocation	reschedule remaining
crash-safe	claim lease + idempotent path	reclaim after expiry

FAIRNESS

A hot stream cannot monopolize the worker. The drainer enforces per-stream and per-batch limits, then yields so other streams can progress.

BOUNDED WAITING

A gap buffer is a budget, not a second unbounded queue

EXECUTABLE ASSET 30-Y - GAP POLICY

```
public sealed record GapPolicy(
    string PolicyId,
    long MaximumDistance,
    int MaximumDeferredPerStream,
    long MaximumDeferredBytesPerStream,
    TimeSpan MaxGapAge)
{
    public void DemandValid()
    {
        if (MaximumDistance is < 1 or > 10_000 ||
            MaximumDeferredPerStream is < 1 or > 10_000 ||
            MaximumDeferredBytesPerStream is < 1 ||
            MaxGapAge <= TimeSpan.Zero)
            throw new InvalidGapPolicy(PolicyId);
    }
}

public sealed record DeferredUsage(
    int Count,
    long ProtectedBytes,
    DateTimeOffset? OldestObservedAtUtc);
```

BUDGET BREACH 30-Z

Condition	Terminal reason	Evidence retained
observed - expected > max distance	sequence-gap-distance	expected/observed + exact bytes
rows exceed per-stream cap	sequence-gap-overflow	count + policy digest
protected bytes exceed cap	sequence-gap-byte-overflow	byte count + key id
oldest gap passes deadline	sequence-gap-timeout	first seen + deadline
decrypt/digest mismatch	deferred-evidence-corrupt	fingerprint only; no apply

POLICY VALUES ARE HYPOTHESES

The manuscript names enforceable dimensions but does not invent production thresholds. Accepted values come from load, outage, retention and repair evidence and are captured by policy digest.

BOUNDARY TO CHAPTER 29

A terminal gap becomes durable poison evidence

Chapter 30 detects ordering failure. Chapter 29 already owns terminal evidence, approval and replay. The integration uses the same poison table and normal-exchange replay path rather than creating a silent side channel.

EXECUTABLE ASSET 30-AA - GAP ESCALATION

```
public async Task EscalateExpiredAsync(
    DeferredDelivery expired,
    CancellationToken cancellationToken)
{
    await using var tx = await db.Database
        .BeginTransactionAsync(cancellationToken);

    db.PoisonMessages.Add(
        PoisonMessage.FromDeferred(
            expired,
            terminalReason: "sequence-gap-timeout",
            classifierVersion: classifier.Version,
            policyId: gapPolicy.PolicyId,
            replayMayBeRequested: true));

    db.DeferredDeliveries.Remove(expired);
    await db.SaveChangesAsync(cancellationToken);
    await tx.CommitAsync(cancellationToken);
}
```

OPERATOR REPLAY MATRIX 30-AB

Situation	Approval question	Safe path
missing position later recovered	does checkpoint still expect it?	normal exchange -> inbox -> gate
gap timeout, source repaired	is original sequence now valid?	replay frozen bytes in order
sequence digest conflict	which producer fact is authoritative?	hold; producer investigation
unsupported schema	is compatible reader deployed?	replay only after capability evidence
checkpoint corruption	can applied ledger reconstruct truth?	audited repair, never blind replay

REPLAY DOES NOT RESET SEQUENCE

A replay preserves MessageId, wire bytes, StreamId and PublicationSequence. Allocating a new sequence would rewrite history and conceal the original gap.

FAILURE LABORATORY

Deliver 2 before 1 and prove convergence

EXECUTABLE ASSET 30-AC - REVERSE-DELIVERY COMPONENT TEST

```
[Fact]
public async Task ORDER_DB_001_two_then_one_converges()
{
    var one = fixture.AlarmDelivery(sequence: 1);
    var two = fixture.AlarmDelivery(sequence: 2);

    var first = await fixture.ConsumeAsync(two);
    first.EffectCode.Should().Be(
        "sequence-gap-deferred");
    await fixture.AssertCheckpointAsync(
        one.StreamId, expected: 0);
    await fixture.AssertDeferredAsync(two, present: true);

    await fixture.ConsumeAsync(one);
    await fixture.DrainUntilIdleAsync(one.StreamId);

    await fixture.AssertCheckpointAsync(
        one.StreamId, expected: 2);
    await fixture.AssertAppliedOrderAsync(
        one.StreamId, 1, 2);
    await fixture.AssertOwnedEffectCountAsync(2);
    await fixture.AssertDeferredCountAsync(
        one.StreamId, expected: 0);
}
```

EXECUTABLE ASSET 30-AD - CRASH AFTER DEFERRED COMMIT

```
[Fact]
public async Task ORDER_CRASH_001_deferred_row_survives_restart()
{
    var two = fixture.AlarmDelivery(sequence: 2);
    await fixture.ConsumeWithCrashAsync(
        two, after: GapCheckpoint.DeferredCommit);

    await fixture.RestartConsumerAsync();
    await fixture.AssertDeferredAsync(two, present: true);
    await fixture.AssertCheckpointAsync(
        two.StreamId, expected: 0);

    await fixture.DeliverAsync(
        fixture.AlarmDelivery(sequence: 1));
    await fixture.DrainUntilIdleAsync(two.StreamId);
    await fixture.AssertCheckpointAsync(
        two.StreamId, expected: 2);
}
```

REQUIRED ORACLE

The test asserts durable intermediate state, final position, exact apply order, effect count and empty gap buffer. A final value alone could hide duplicate work.

DUPLICATE LABORATORY

Wire duplicate and semantic duplicate are tested separately

EXECUTABLE ASSET 30-AE - DUPLICATE AND CONFLICT CORPUS

```
[Fact]
public async Task ORDER_DB_002_same_message_id_effect_once()
{
    var delivery = fixture.AlarmDelivery(sequence: 1);
    await fixture.ConsumeAsync(delivery);
    await fixture.ConsumeAsync(delivery);
    await fixture.AssertOwnedEffectCountAsync(1);
    await fixture.AssertInboxCountAsync(
        delivery.MessageId, 1);
}

[Fact]
public async Task ORDER_DB_003_new_id_same_meaning_is_stale()
{
    var first = fixture.AlarmDelivery(sequence: 1);
    var duplicate = first.WithNewMessageId();
    await fixture.ConsumeAsync(first);
    var outcome = await fixture.ConsumeAsync(duplicate);
    outcome.EffectCode.Should().Be(
        "stale-sequence-duplicate");
    await fixture.AssertOwnedEffectCountAsync(1);
}

[Fact]
public async Task ORDER_DB_004_same_position_new_meaning_is_poison()
{
    var first = fixture.AlarmDelivery(sequence: 1);
    var conflict = first.WithNewMessageId()
        .WithAlarmCount(first.AlarmCount + 1);
    await fixture.ConsumeAsync(first);
    await fixture.ConsumeAsync(conflict);
    await fixture.AssertPoisonAsync(
        conflict, "sequence-digest-conflict");
}
```

CONFLICT IS NOT LAST-WRITE-WINS

The consumer cannot choose the later arrival, newest timestamp or highest MessageId. Two meanings at one producer position are a producer integrity incident.

EVIDENCE COUNTS 30-AE recorded as completed delivery evidence without creating another AppliedPublication position.

Test	Inbox	Applied position	Effect	Poison
same MessageId twice	1	1	1	0
new ID, same meaning	2*	1	1	0
new ID, conflicting meaning	1 + terminal	1	1	1

CONCURRENCY LABORATORY

Parallel callbacks preserve per-stream order and cross-stream progress

EXECUTABLE ASSET 30-AG - SAME-STREAM RACE

```
[Fact]
public async Task ORDER_DB_005_parallel_one_and_two_converge()
{
    var one = fixture.AlarmDelivery(sequence: 1);
    var two = fixture.AlarmDelivery(sequence: 2);
    var barrier = fixture.Barrier(
        StreamCheckpoint.ReadyToLock, participants: 2);

    await Task.WhenAll(
        fixture.ConsumeInNewContextAsync(one, barrier),
        fixture.ConsumeInNewContextAsync(two, barrier));
    await fixture.DrainUntilIdleAsync(one.StreamId);

    await fixture.AssertCheckpointAsync(
        one.StreamId, expected: 2);
    await fixture.AssertAppliedOrderAsync(
        one.StreamId, 1, 2);
    await fixture.AssertOwnedEffectCountAsync(2);
}
```

EXECUTABLE ASSET 30-AH - INDEPENDENT STREAMS

```
[Fact]
public async Task ORDER_DB_006_gap_does_not_block_other_stream()
{
    var a2 = fixture.AlarmDelivery("scope-a", sequence: 2);
    var b1 = fixture.AlarmDelivery("scope-b", sequence: 1);

    await Task.WhenAll(
        fixture.ConsumeAsync(a2),
        fixture.ConsumeAsync(b1));

    await fixture.AssertCheckpointAsync(
        a2.StreamId, expected: 0);
    await fixture.AssertDeferredAsync(a2, present: true);
    await fixture.AssertCheckpointAsync(
        b1.StreamId, expected: 1);
    await fixture.AssertEffectAsync(b1, present: true);
}
```

LOCK GRANULARITY

The test fails if a global consumer mutex or table lock makes scope-b wait behind scope-a's missing position. Ordering is local to the producer stream.

RECOVERY AND EXHAUSTION

Restart, timeout, and overflow all have durable oracles

EXECUTABLE ASSET 30-AI - EXPIRED GAP BECOMES VISIBLE

```
[Fact]
public async Task ORDER_DB_007_gap_timeout_is_terminal_evidence()
{
    var future = fixture.AlarmDelivery(sequence: 3);
    await fixture.ConsumeAsync(future);
    fixture.Clock.Advance(
        fixture.GapPolicy.MaxGapAge + TimeSpan.FromSeconds(1));

    await fixture.RunGapSweepAsync();

    await fixture.AssertDeferredAsync(future, present: false);
    await fixture.AssertPoisonAsync(
        future, "sequence-gap-timeout");
    await fixture.AssertCheckpointAsync(
        future.StreamId, expected: 0);
    await fixture.AssertOwnedEffectCountAsync(0);
}

[Fact]
public async Task ORDER_DB_008_overflow_never_acks_without_evidence()
{
    await fixture.FillGapBufferToLimitAsync("scope-a");
    var overflow = fixture.NextFutureDelivery("scope-a");

    await fixture.ConsumeAsync(overflow);

    await fixture.AssertPoisonAsync(
        overflow, "sequence-gap-overflow");
    await fixture.AssertNoOwnedEffectAsync(overflow);
}
```

RECOVERY MATRIX 30-AJ

Crash point	Durable state	Recovery
before deferred commit	no deferred row	broker redelivery
after deferred commit, before ack	deferred row	redelivery resolves duplicate defer
after ApplyNext commit, before drain	checkpoint advanced	worker selects next
during deferred claim	lease expires	replacement reclaims
after poison commit, before ack	terminal evidence	redelivery sees terminal identity

RABBITMQ ACCEPTANCE LABORATORY

Demonstrate out-of-order completion instead of arguing about FIFO

The broker test uses the production queue type and client package. Two callbacks are released with controlled handler delays so observed completion becomes 2,1 while the sequence gate still commits 1,2.

EXECUTABLE ASSET 30-AK - BROKER-BACKED ORDERING PROBE

```
[Fact]
public async Task ORDER_RMQ_001_completion_can_be_out_of_order()
{
    await fixture.DeclareQueueAsync(
        consumerDispatchConcurrency: 2,
        singleActiveConsumer: false);
    var one = fixture.AlarmDelivery(sequence: 1);
    var two = fixture.AlarmDelivery(sequence: 2);
    fixture.Delays.Set(one.MessageId, TimeSpan.FromMilliseconds(250));
    fixture.Delays.Set(two.MessageId, TimeSpan.Zero);

    await fixture.PublishConfirmedAsync(one, two);
    await fixture.WaitForTwoCallbacksAsync();

    fixture.CallbackCompletionOrder.Should().Equal(2, 1);
    await fixture.DrainUntilIdleAsync(one.StreamId);
    await fixture.AssertAppliedOrderAsync(
        one.StreamId, 1, 2);
    await fixture.AssertCheckpointAsync(
        one.StreamId, expected: 2);
}
```

EXECUTABLE ASSET 30-AL - REDELIVERY STILL NEEDS THE GATE

```
[Fact]
public async Task ORDER_RMQ_002_redelivery_preserves_identity()
{
    var delivery = fixture.AlarmDelivery(sequence: 1);
    await fixture.PublishConfirmedAsync(delivery);
    await fixture.CrashConsumerAfterCommitBeforeAckAsync();
    await fixture.RestartConsumerAsync();

    await fixture.AssertEventuallyAkedAsync(delivery);
    await fixture.AssertOwnedEffectCountAsync(1);
    await fixture.AssertInboxCountAsync(
        delivery.MessageId, expected: 1);
}
```

PINNED ENVIRONMENT

Evidence records broker image digest, server version, queue type, client package, consumer concurrency, prefetch, Single Active Consumer setting and test result hash.

BROKER TOPOLOGY CHOICES

Single Active Consumer reduces concurrency; it does not create causality

ORDERING OPTION MATRIX 30-AM

Option	Helps with	Does not solve	NEXUS-1 position
one active consumer	sequential queue dispatch	multi-publisher order, duplicates	optional measured topology
prefetch = 1	limits unacked work	producer gaps or redelivery	capacity setting, not contract
priority queue	urgent transport preference	causal order	avoid on ordered facts
hash partitioned queues	parallel per-key routing	wrong/missing producer sequence	possible after evidence
RabbitMQ stream	stable log offset	business version compatibility	different design, separate ADR

What the broker does not guarantee

- No global order across multiple publishing connections, channels, queues, exchanges or producers.
- No effect-once guarantee after publish confirmation, consumer acknowledgement loss or redelivery.
- No business meaning for timestamps, delivery tags, queue positions or confirm sequence numbers.
- No schema registry, producer-consumer compatibility check, semantic digest or version retirement gate.
- No automatic repair of a missing producer publication or a conflicting position.

TOPOLOGY IS AN OPTIMIZATION BOUNDARY

The application sequence gate remains correct when consumer count, prefetch or Single Active Consumer changes. If correctness disappears with a tuning change, the guarantee was never in the contract.

CONTRACT EVOLUTION PROBLEM

Compatibility is directional and version-specific

An additive field is not universally safe. A strict old reader may reject it, an absent value may have no deterministic meaning, and an enum addition may break an exhaustive consumer. NEXUS-1 classifies the exact producer/consumer pair.

CHANGE CLASSIFICATION 30-AN

Change	Same version?	Reason
new optional member with specified absence meaning	only if archived readers accept it	directional evidence required
new required member	no	old payload cannot satisfy new reader
rename or remove member	no	wire name is contractual
numeric/string type change	no	decoder and meaning differ
unit, scale or semantic reinterpretation	no	same bytes acquire new meaning
new enum value	only with explicit unknown policy	exhaustive readers may fail
stream sequence reset	never	breaks consumer checkpoint identity

VERSION RULE

A breaking change receives a new EventType/SchemaVersion pair and an explicit adapter. The broker routes bytes; it does not declare them compatible.

EXACT V1 AND V2 CONTRACTS

Dual readers converge on one canonical meaning

EXECUTABLE ASSET 30-AO - FROZEN CONTRACT VERSIONS

```
[JsonUnmappedMemberHandling(
    JsonUnmappedMemberHandling.Disallow)]
public sealed record AlarmFloodDetectedV1(
    [property: JsonRequired] string AlarmScopeId,
    [property: JsonRequired] int AlarmCount,
    [property: JsonRequired] string DetectionPolicyId,
    [property: JsonRequired] ProducerStreamPositionV1 Stream);

[JsonUnmappedMemberHandling(
    JsonUnmappedMemberHandling.Disallow)]
public sealed record AlarmFloodDetectedV2(
    [property: JsonRequired] string AlarmScopeId,
    [property: JsonRequired] int AlarmCount,
    [property: JsonRequired] string DetectionPolicyId,
    [property: JsonRequired] string DetectionProfile,
    AlarmFloodDiagnosticsV2? Diagnostics,
    [property: JsonRequired] ProducerStreamPositionV1 Stream);

public sealed record CanonicalAlarmFlood(
    string AlarmScopeId,
    int AlarmCount,
    string DetectionPolicyId,
    string DetectionProfile,
    ProducerStreamPosition Stream);
```

EXECUTABLE ASSET 30-AP - VERSION-OWNED ADAPTERS

```
public static CanonicalAlarmFlood Adapt(AlarmFloodDetectedV1 v1) =>
    new(
        v1.AlarmScopeId,
        v1.AlarmCount,
        v1.DetectionPolicyId,
        DetectionProfile: "Legacy-policy-default",
        StreamPosition.Parse(v1.Stream));

public static CanonicalAlarmFlood Adapt(AlarmFloodDetectedV2 v2) =>
    new(
        v2.AlarmScopeId,
        v2.AlarmCount,
        v2.DetectionPolicyId,
        v2.DetectionProfile,
        StreamPosition.Parse(v2.Stream));
```

NO SEQUENCE RESET

A stream can publish V1 at positions 1-84 and V2 at position 85. The consumer checkpoint key excludes SchemaVersion; evolution changes representation, not history.

REGISTRY AND ADMISSION

Unknown versions fail before stream state is touched

EXECUTABLE ASSET 30-AQ - EXACT CONTRACT REGISTRY

```
public sealed record ContractKey(
    string Producer,
    string EventType,
    int SchemaVersion,
    string ContentType);

public sealed class AlarmContractRegistry
{
    private readonly IReadOnlyDictionary<
        ContractKey, IContractAdapter> _adapters;

    public CanonicalAlarmFlood Decode(
        EnvelopeDocumentV1 envelope)
    {
        var key = new ContractKey(
            envelope.Producer,
            envelope.EventType,
            envelope.SchemaVersion,
            envelope.ContentType);
        if (!_adapters.TryGetValue(key, out var adapter))
            throw new UnsupportedContractVersion(key);

        return adapter.DecodeStrict(envelope.Payload);
    }
}
```

EXECUTABLE ASSET 30-AR - STRICT V2 DECODER

```
public CanonicalAlarmFlood DecodeStrict(JsonElement payload)
{
    var wire = payload.Deserialize(
        AlarmContractJsonContext.Default
            .AlarmFloodDetectedV2);
    ?? throw new ContractPayloadMissing();
    validators.DemandValid(wire);
    return AlarmAdapters.Adapt(wire);
}

[Fact]
public void CONTRACT_ORD_001_unknown_v3_is_rejected()
{
    var envelope = fixture.Envelope(
        eventType: "AlarmFloodDetected.v3",
        schemaVersion: 3);
    var act = () => fixture.Registry.Decode(envelope);
    act.Should().Throw<UnsupportedContractVersion>();
    fixture.StreamStoreCalls.Should().Be(0);
}
```

STRICT DOES NOT MEAN INCOMPATIBLE FOREVER

Strict version readers reject accidental drift. Planned evolution is handled by a new exact reader plus a tested adapter, not by weakening every decoder.

COMPATIBILITY CORPUS

Golden bytes test producers and consumers as separate builds

EXECUTABLE ASSET 30-AS - CONTRACT CORPUS LAYOUT

```
contracts/
  alarm-flood-detected/
    v1/
      accepted/minimal.json
      accepted/boundary-values.json
      rejected/missing-stream.json
      rejected/unknown-member.json
      manifest.sha256
    v2/
      accepted/minimal.json
      accepted/diagnostics-present.json
      rejected/missing-profile.json
      rejected/unknown-enum.json
      manifest.sha256
```

EXECUTABLE ASSET 30-AT - DUAL-READER COMPATIBILITY TESTS

```
[Theory]
[InlineData("v1/accepted/minimal.json")]
[InlineData("v1/accepted/boundary-values.json")]
[InlineData("v2/accepted/minimal.json")]
[InlineData("v2/accepted/diagnostics-present.json")]
public void CONTRACT_ORD_002_supported_corpus_adapts(
    string fixturePath)
{
    var envelope = corpus.ReadEnvelope(fixturePath);
    var canonical = registry.Decode(envelope);
    canonical.Stream.Sequence.Value.Should().BePositive();
    canonical.AlarmScopeId.Should().NotNullOrWhiteSpace();
}

[Theory]
[InlineData("v1/rejected/missing-stream.json")]
[InlineData("v1/rejected/unknown-member.json")]
[InlineData("v2/rejected/missing-profile.json")]
[InlineData("v2/rejected/unknown-enum.json")]
public void CONTRACT_ORD_003_rejected_corpus_stays_rejected(
    string fixturePath)
{
    var act = () => registry.Decode(
        corpus.ReadEnvelope(fixturePath));
    act.Should().Throw<ContractRejected>();
}
```

SEPARATE ARTIFACTS

Archived producer and consumer assemblies compile from their own frozen contracts. A test that shares the candidate DTO can turn a breaking change green by accident.

SAFE ROLLOUT

Dual-read, single-write, then evidence-based retirement

CONTRACT ROLLOUT STATE MACHINE 30-AU

State	Producer writes	Consumer reads	Exit evidence
V1 stable	V1	V1	V2 design and corpus approved
reader expansion	V1	V1 + V2	all live consumers report capability
writer cutover	V2	V1 + V2	V2 traffic and rollback window observed
V1 drain	V2	V1 + V2	oldest replay/poison/deferred V1 expired
V1 retired	V2	V2	retirement manifest accepted

EXECUTABLE ASSET 30-AV - WRITER SELECTION IS ONE VALUE

```
public enum AlarmFloodWriteVersion
{
    V1 = 1,
    V2 = 2
}

public sealed record ContractRolloutOptions(
    AlarmFloodWriteVersion WriteVersion,
    bool ReadV1,
    bool ReadV2)
{
    public void Validate()
    {
        if (WriteVersion == AlarmFloodWriteVersion.V1 && !ReadV1 ||
            WriteVersion == AlarmFloodWriteVersion.V2 && !ReadV2)
            throw new WriterVersionNotReadable();
    }
}
```

WHY NOT DUAL-WRITE?

Writing V1 and V2 as two public facts can create two sequence positions and two business effects. If a bridge is unavoidable, it owns explicit identity and deduplication semantics; it is not the default rollout.

CAPABILITY AND RETIREMENT EVIDENCE

A version is retired only after every replay surface is checked

EXECUTABLE ASSET 30-AW - CONSUMER CAPABILITY MANIFEST

```
public sealed record ConsumerContractCapability(
    string ConsumerName,
    string BuildIdentity,
    IReadOnlySet<ContractKey> AcceptedContracts,
    string ContractCorpusSha256,
    DateTimeOffset GeneratedAtUtc);

public sealed record RetirementEvidence(
    ContractKey Retiring,
    IReadOnlyList<ConsumerContractCapability> Consumers,
    long LiveQueueCount,
    long RetryTicketCount,
    long PoisonReplayCandidateCount,
    long DeferredDeliveryCount,
    DateTimeOffset ReplayHorizonEndsAtUtc,
    string ResultSha256);
```

EXECUTABLE ASSET 30-AX - FAIL-CLOSED RETIREMENT GATE

```
public static void DemandRetirable(
    RetirementEvidence evidence,
    DateTimeOffset now)
{
    Evidence.RequireNonZero(evidence.Consumers.Count);
    Evidence.Require(evidence.LiveQueueCount == 0);
    Evidence.Require(evidence.RetryTicketCount == 0);
    Evidence.Require(
        evidence.PoisonReplayCandidateCount == 0);
    Evidence.Require(evidence.DeferredDeliveryCount == 0);
    Evidence.Require(now >= evidence.ReplayHorizonEndsAtUtc);
    Evidence.RequireSha256(evidence.ResultSha256);

    foreach (var consumer in evidence.Consumers)
    {
        Evidence.RequireVersion(consumer.BuildIdentity);
        Evidence.RequireSha256(
            consumer.ContractCorpusSha256);
    }
}
```

QUEUE DEPTH ALONE IS INSUFFICIENT

Old-version bytes may wait in durable retry tickets, deferred gaps, poison replay candidates, backups or an operator export. Retirement inventory covers every supported replay surface.

OBSERVABILITY

Measure sequence health without turning identities into labels

EXECUTABLE ASSET 30-AY - BOUNDED METRICS

```
public static class OrderingTelemetry
{
    private static readonly Meter Meter =
        new("Nexus1.Messaging.Ordering");

    public static readonly Counter<long> Decisions =
        Meter.CreateCounter<long>(
            "messaging.sequence.decisions");
    public static readonly UpDownCounter<long> Deferred =
        Meter.CreateUpDownCounter<long>(
            "messaging.sequence.deferred");
    public static readonly Histogram<double> OldestGapSeconds =
        Meter.CreateHistogram<double>(
            "messaging.sequence.oldest_gap.seconds");
    public static readonly Counter<long> ContractRejected =
        Meter.CreateCounter<long>(
            "messaging.contract.rejected");
}

// Allowed tags: consumer, producer, event_type,
// schema_version, outcome, policy_id.
// Forbidden tags: MessageId, StreamId, CorrelationId.
```

OPERATIONAL SIGNALS 30-AZ

Signal	Meaning	Suggested investigation
gap rate	future positions observed	publisher concurrency, retries, routing
oldest gap age	stream progress at risk	missing outbox fact or poison
deferred bytes	consumer storage pressure	capacity and source repair
sequence conflicts	producer integrity failure	freeze replay; inspect source
schema rejection	rollout/capability mismatch	registry and deployment versions
stale duplicate ratio	replay or duplicate pressure	relay and operator replay history

TRACE RELATIONSHIP

A deferred delivery creates a span linked to the original message context. The later drain creates a new processing span with an explicit link; it does not pretend that one trace remained open for hours.

SECURITY AND OPERATOR HANDLING

Deferred bytes are evidence with the same sensitivity as poison

EVIDENCE PROTECTION 30-BA

Asset	Protection	Operator view
DeferredDelivery	application encryption + key id	metadata and bounded redaction
AppliedPublication	database access control	sequence + digest, no payload
ConsumerStreamCheckpoint	owner-only write	progress and age
PoisonMessage	Chapter 29 protected envelope	approval-gated decrypt/replay
capability manifest	signed build/evidence identity	versions and digests

EXECUTABLE ASSET 30-BB - REPAIR COMMAND BOUNDARY

```
public sealed record RepairStreamCommand(  
    string ConsumerName,  
    string Producer,  
    string StreamId,  
    long ExpectedLastApplied,  
    string Reason,  
    string EvidenceBundleSha256,  
    string ApprovedBy);  
  
public interface IOrderingRepairService  
{  
    Task<RepairResult> ExecuteAsync(  
        RepairStreamCommand command,  
        CancellationToken cancellationToken);  
}  
  
// No generic "set checkpoint" endpoint exists.  
// Every repair compares expected state and appends audit evidence.
```

- Payload bytes are never placed in logs, metric labels, exception messages or ordinary support exports.
- Decrypt and replay permissions are separate from read-only sequence diagnostics.
- A repair cannot skip a missing position unless an owner-approved reconciliation proves the resulting state.
- Retention must keep applied digests for at least the supported replay horizon.

ADR-028

Choose producer sequence and consumer-owned contiguous checkpoints

DECISION RECORD 30-BC

ADR field	ADR-028 record
Status	Proposed until ORDER and CONTRACT evidence manifests pass.
Context	RabbitMQ delivery is durable and at-least-once but cannot be treated as a global business order.
Decision	Assign StreamId + PublicationSequence in the producer transaction; advance a consumer checkpoint atomically with each local effect; persist bounded gaps; use exact version readers and dual-read/single-write rollout.
Consequences	Consumers own checkpoints, applied digests, deferred storage, retention, terminal gaps and version capability evidence.
Rejected	timestamp sorting; queue position as causality; global sequence service; silent last-write-wins; unbounded buffering; permissive dynamic JSON; dual-write by default.
Review trigger	broker/queue type, envelope, sequence scope, provider, gap policy, replay horizon, contract registry, partitioning or repair authority changes.

Why this fits the previous chapters

- Chapter 20 keeps the strict versioned envelope; order data lives in producer-owned payload contracts.
- Chapters 26-27 preserve immutable bytes and stable MessageId through at-least-once dispatch.
- Chapter 28 remains the transport-deduplication and local-effect atomicity boundary.
- Chapter 29 remains the owner of bounded retry, terminal evidence and approved replay.
- Chapter 30 adds only the missing business-order and compatibility responsibilities.

REJECTED CENTRAL SEQUENCER

A global sequence service couples unrelated streams, creates a new availability dependency and still cannot define domain causality. Sequence authority stays with the producer that owns the aggregate transition.

EVIDENCE MANIFEST

The chapter remains **PENDING** until real providers execute

EXECUTABLE ASSET 30-BD - FAIL-CLOSED EVIDENCE RECORD

```
{
  "chapter": 30,
  "status": "PENDING",
  "buildIdentity": null,
  "databaseImageDigest": null,
  "rabbitmqImageDigest": null,
  "rabbitmqVersion": null,
  "clientPackageVersion": null,
  "gapPolicySha256": null,
  "contractCorpusSha256": null,
  "tests": {
    "ORDER-DB-001": "PENDING",
    "ORDER-DB-002": "PENDING",
    "ORDER-DB-003": "PENDING",
    "ORDER-DB-004": "PENDING",
    "ORDER-DB-005": "PENDING",
    "ORDER-DB-006": "PENDING",
    "ORDER-DB-007": "PENDING",
    "ORDER-RMQ-001": "PENDING",
    "ORDER-RMQ-002": "PENDING",
    "CONTRACT-ORD-001": "PENDING",
    "CONTRACT-ORD-002": "PENDING",
    "CONTRACT-ORD-003": "PENDING"
  },
  "discoveredTests": null,
  "resultSha256": null,
  "accepted": false
}
```

EXECUTABLE ASSET 30-BE - ACCEPTANCE VALIDATOR

```
public static void DemandAccepted(Chapter30Evidence evidence)
{
    Evidence.Require(evidence.Status == EvidenceStatus.Passed);
    Evidence.RequireVersion(evidence.BuildIdentity);
    Evidence.RequireSha256(evidence.DatabaseImageDigest);
    Evidence.RequireSha256(evidence.RabbitMqImageDigest);
    Evidence.RequireVersion(evidence.RabbitMqVersion);
    Evidence.RequireVersion(evidence.ClientPackageVersion);
    Evidence.RequireSha256(evidence.GapPolicySha256);
    Evidence.RequireSha256(evidence.ContractCorpusSha256);
    Evidence.Require(evidence.DiscoveredTests > 0);
    Evidence.RequireAllPassed(evidence.Tests);
    Evidence.RequireSha256(evidence.ResultSha256);
}
```

CURRENT VERDICT

The manuscript now contains an implementable ordering and evolution design, but no repository build, SQL Server race, RabbitMQ delivery probe, contract corpus or restart laboratory was executed in this production run. All runtime evidence is honestly PENDING.

IMPLEMENTATION SEQUENCE AND CHECKPOINT

The message backbone is specified; the first runtime service comes next

Implementation sequence

- Accept ADR-028 and catalogue every ordered producer stream and unordered legacy exception.
- Add producer-local PublicationStream storage and prove atomic range allocation with outbox rows.
- Freeze ordered contract payloads, exact version readers, canonical adapters and golden corpora.
- Apply consumer checkpoint, applied-position and protected deferred-delivery migrations.
- Implement the sequence gate and keep the Chapter 28 inbox as the first duplicate boundary.
- Prove reverse delivery, transport duplicate, semantic duplicate, conflict, concurrency and independent streams on SQL Server.
- Prove restart recovery, gap timeout, overflow and Chapter 29 poison integration.
- Run RabbitMQ completion-order and redelivery probes with pinned topology and client versions.
- Exercise dual-read/single-write rollout and fail-closed retirement evidence.
- Publish the Chapter 30 manifest only with non-zero discovered tests and exact result digests.

Official technical anchors

RabbitMQ Queues / Message Ordering: <https://www.rabbitmq.com/docs/queues>

RabbitMQ Consumers / Single Active Consumer and concurrency: <https://www.rabbitmq.com/docs/consumers>

RabbitMQ Reliability Guide: <https://www.rabbitmq.com/docs/reliability>

System.Text.Json unmapped members: <https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/missing-members>

System.Text.Json required properties: <https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/required-properties>

What this chapter has proved

Chapter 30 has specified producer-owned per-stream sequence, consumer-owned contiguous progress, exact duplicate/conflict semantics, bounded durable gaps, restart and exhaustion oracles, RabbitMQ non-guarantees, exact contract readers, canonical adapters, safe rollout and retirement evidence. It has not executed the runtime assets and does not claim global order, duplicate-free delivery or automatic compatibility.

NEXT - PART VI / CHAPTER 31

AlarmManagement Enters the Runtime will use this completed backbone to create the first extracted service, its input edge, service-owned database, deterministic alarm-flood rules, outbox, telemetry and tests. It will not reopen the ordering or contract-evolution decisions made here.

CHAPTER CHECKPOINT

Can the team identify sequence authority, prove 2-before-1 convergence, distinguish wire duplicate from semantic conflict, bound every gap, name every replay surface and show exact old/new contract pairs? If any answer is no, Chapter 30 remains PENDING.

PART VI

The First End-to-End Distributed Slice

Parts I-V now meet in one service-owned runtime, one public fact and one consumer path whose uncertainty is explicit.

CHAPTER 31**Owner runtime**

input -> decision -> local commit

CHAPTER 32**Public fact**

V1 bytes -> relay -> confirm

CHAPTERS 33-34**Consumer truth**

inbox -> case -> audit

CHAPTERS 35-36**Delayed truth + slice**

workflow -> insight -> recovery

ENTRY RULE

Nothing claims to exist that does not. Chapter 31 proves only the owner runtime; public delivery begins when Chapter 32 proves it.

BEGIN WITH CHAPTER 31 / OWNER RUNTIME / INPUT -> DECISION -> LOCAL COMMIT

PART VI - THE FIRST END-TO-END DISTRIBUTED SLICE

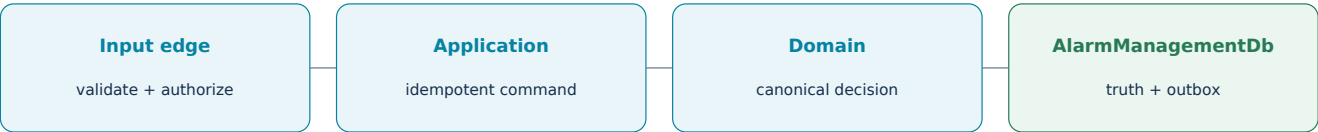
31. AlarmManagement Enters the Runtime

The first extracted service is not a controller wrapped around the shared database. It is a separately hosted owner that accepts one explicit input, replays deterministic alarm-flood policy, commits its own truth and frozen publication intent, and exposes evidence about what it can safely do.

CHAPTER DECISION

AlarmManagement becomes the first independently composed NEXUS-1 runtime. The request path ends at one AlarmManagementDb commit containing accepted observation truth, the deterministic flood decision and any pending outbox fact. The relay is disabled here. Chapter 32 owns the exact AlarmFloodDetected.v1 mapping, broker dispatch, trace and recovery proof.

RUNTIME SLICE 31-A - ONE OWNER, ONE SYNCHRONOUS DURABILITY BOUNDARY



GUARANTEE LEDGER 31-B

Boundary	Mechanism	Allowed claim	Not yet allowed
request -> command	versioned edge + policy	validated, authorized intent	downstream completion
command -> decision	canonical inputs + pinned policy	replayable owner decision	arrival-order semantics
decision -> durability	one provider transaction	truth and intent agree	broker acceptance
runtime -> evidence	health, metrics, tests	named verification surface	executed production proof

ORDINARY DEVELOPER PROBLEM

A process that starts is not an extracted service

The tempting first slice is an endpoint that writes the old tables and then publishes a message. It is visible in a demo, yet still shares authority, uses arrival order as policy and contains a dual-write crash window.

REJECTED ASSET 31-A - ATTRACTIVE BUT FALSE EXTRACTION

```
app.MapPost("/alarms", async (
    AlarmDto dto,
    LegacyDbContext db,
    IModel rabbit) =>
{
    db.Alarms.Add(new AlarmRow(dto.Id, dto.Severity));
    await db.SaveChangesAsync();

    rabbit.BasicPublish(
        exchange: "nexus.events",
        routingKey: "alarm.flood",
        body: JsonSerializer.SerializeToUtf8Bytes(dto));

    return Results.Accepted();
});
```

Why the demo lies

- LegacyDbContext keeps the old ownership boundary alive; the new host is only another writer.
- The DTO becomes domain input and public event without an owner-controlled translation.
- No stable source-event identity distinguishes retry from conflicting reuse.
- The database commit and broker publish can disagree after any process or network failure.
- No pinned policy snapshot or canonical membership makes the flood decision replayable.
- An empty 202 response hides accepted, replayed, rejected and unknown outcomes.

PLAIN-LANGUAGE PRINCIPLE

Extraction begins when one runtime can defend its language, data, decision and failure semantics without synchronously borrowing another owner's database or the broker.

RUNTIME PASSPORT

The host owns a narrow, reviewable promise

The service passport turns a deployment name into an operational contract. Everything not named remains outside this boundary.

SERVICE PASSPORT 31-C

Field	AlarmManagement runtime decision	Evidence
owned language	observation, managed alarm, detector scope, flood detection	domain tests
write authority	AlarmManagementDb only; runtime DML principal	permission suite
input	POST /api/v1/alarm-observations	edge contract suite
durable output	owner truth + pending public fact	provider atomicity
synchronous dependencies	identity validation and AlarmManagementDb	host inventory
delayed dependencies	RabbitMQ and all consumers	zero-broker test
health	process liveness; config/schema/database readiness	probe tests
evidence state	reference assets complete; runtime results PENDING	manifest 31-Z

Explicit non-ownership

- RootCause owns investigation and verdicts; AlarmManagement does not write RootCauseDb.
- Audit owns evidence acceptance; an application log is not an accepted audit record.
- Compliance owns obligations and deadlines; detector thresholds do not become compliance rules.
- Reporting owns projections; it may lag and cannot be a command-side source of truth.

BOUNDARY CHECK

No request path reads a foreign service database, waits for a consumer or reports broker delivery as part of accepting an observation.

BUILDABLE TOPOLOGY

Projects make authority visible before runtime

The composition root may reference every outer adapter. Domain and Application cannot reference ASP.NET Core, EF Core, RabbitMQ or another service's internals.

EXECUTABLE ASSET 31-B - SOLUTION SLICE

```
services/AlarmManagement/  
  src/  
    Nexus1.AlarmManagement.Domain/  
    Nexus1.AlarmManagement.Application/  
    Nexus1.AlarmManagement.Infrastructure/  
    Nexus1.AlarmManagement.Edge.Contracts/  
    Nexus1.AlarmManagement.Host/  
  tests/  
    Nexus1.AlarmManagement.Domain.Tests/  
    Nexus1.AlarmManagement.Application.Tests/  
    Nexus1.AlarmManagement.ComponentTests/  
    Nexus1.AlarmManagement.Edge.ContractTests/  
    Nexus1.AlarmManagement.Host.Tests/  
    Nexus1.AlarmManagement.ArchitectureTests/
```

REFERENCE RULE SET 31-D

Project	May reference	Rejected reference
Domain	BCL + approved primitives	EF, ASP.NET, contracts
Application	Domain + application abstractions	Host, provider, broker
Infrastructure	Application + provider + producer contract	foreign service internals
Edge.Contracts	dependency-poor DTO primitives	Domain entities, DbContext
Host	all local projects + ServiceDefaults	foreign DbContext
Tests	surface under test	production-only shortcuts

ARCHITECTURE ACCEPTANCE

A graph test fails if Domain or Application gains an outer reference, if Host loads a foreign persistence assembly, or if the edge contract exposes a domain or EF type.

COMPOSITION ROOT

Program.cs is the executable runtime inventory

All production registrations and endpoints pass through one composition path. Tests boot this same path and replace only explicitly approved outer resources.

EXECUTABLE ASSET 31-C - .NET 8 HOST COMPOSITION

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddAlarmManagementConfiguration(builder.Configuration)
    .AddAlarmManagementApplication()
    .AddAlarmManagementInfrastructure(builder.Configuration)
    .AddAlarmManagementEdge()
    .AddAlarmManagementObservability(builder.Configuration);

builder.Services.AddProblemDetails();
builder.Services.AddAuthentication("Bearer").AddJwtBearer();
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy(
        AlarmPolicies.SubmitObservation,
        policy => policy.RequireClaim(
            "capability", "alarm.observation.submit"));
});

var app = builder.Build();
app.UseExceptionHandler();
app.UseAuthentication();
app.UseAuthorization();
app.MapAlarmObservationEndpoints();
app.MapAlarmManagementHealth();
app.Run();

public partial class Program { }
```

Validation at build and startup

EXECUTABLE ASSET 31-D - SERVICE GRAPH GATE

```
await using var provider = services.BuildServiceProvider(
    new ServiceProviderOptions
    {
        ValidateOnBuild = true,
        ValidateScopes = true
    });

await using var scope = provider.CreateAsyncScope();
_ = scope.ServiceProvider.GetRequiredService<
    IAcceptAlarmObservationHandler>();
_ = scope.ServiceProvider.GetRequiredService<
    AlarmManagementDbContext>();
```

INVENTORY RULE

There is one registered command handler, one database context factory, one provider classifier and no outbox relay worker in Chapter 31.

CONFIGURATION IDENTITY

Policy is validated input, not ambient state

The runtime binds infrastructure settings separately from the immutable policy snapshot used to explain a flood decision. Reload may prepare a later policy; it cannot reinterpret a committed decision.

EXECUTABLE ASSET 31-E - STRICT RUNTIME OPTIONS

```
public sealed class AlarmManagementOptions
{
    public const string Section = "AlarmManagement";

    [Required, RegularExpression(
        "[a-z0-9]{2,79}$")]
    public string ServiceIdentity { get; init; } = "";

    [Range(1, 3)]
    public int MaxSemanticAttempts { get; init; } = 3;

    [Range(1, 262_144)]
    public int MaxObservationBytes { get; init; } = 32_768;

    [Required]
    public string RequiredMigration { get; init; } = "";

    public bool OutboxRelayEnabled { get; init; }
}

services.AddOptions<AlarmManagementOptions>()
    .BindConfiguration(AlarmManagementOptions.Section)
    .ValidateDataAnnotations()
    .Validate(
        x => !x.OutboxRelayEnabled,
        "Chapter 31 requires OutboxRelayEnabled=false.")
    .ValidateOnStart();
```

EXECUTABLE ASSET 31-F - IMMUTABLE POLICY IDENTITY

```
public sealed record AlarmFloodPolicySnapshot(
    string PolicyId,
    int PolicyVersion,
    int Threshold,
    TimeSpan Window,
    TimeSpan AllowedLatency,
    ImmutableArray<AlarmSeverity> IncludedSeverities,
    string CanonicalSha256);
```

REFERENCE VALUES, NOT PLANT TRUTH

Examples later use policy alarm-flood-lab-v7 with five activations in sixty seconds. Those values are an acceptance-lab fixture. Production thresholds require an owner-approved policy artifact and digest.

INPUT EDGE

The HTTP contract carries source identity and accepted time

The edge DTO is versioned and dependency-poor. It carries the producer's observation identity and timestamp, while the server assigns ReceivedAtUtc and never accepts a caller-supplied decision or flood identifier.

EXECUTABLE ASSET 31-G - INPUT AND RECEIPT CONTRACTS

```
public sealed record SubmitAlarmObservationV1(
    Guid OperationId,
    string SourceSystem,
    string SourceEventId,
    string AlarmKey,
    string DetectorScope,
    string Severity,
    DateTimeOffset SourceOccurredAtUtc,
    string PayloadSha256);

public sealed record AlarmObservationReceiptV1(
    Guid OperationId,
    Guid ObservationId,
    string Outcome,
    Guid? FloodDetectionId,
    string DecisionIdentity,
    DateTimeOffset AcceptedAtUtc);
```

VALIDATION MATRIX 31-E

Field	Accepted rule	Failure code
OperationId	non-empty; retry-stable	operation_id.invalid
SourceSystem	allowlisted ASCII identifier, <= 80	source_system.invalid
SourceEventId	non-empty, <= 120; unique with source	source_event_id.invalid
AlarmKey / Scope	normalized, bounded, non-secret	alarm_key.invalid
Severity	exact reviewed enum token	severity.unsupported
SourceOccurredAtUtc	UTC offset zero; within ingest horizon	occurred_at.invalid
PayloadSha256	64 lowercase hex characters	payload_digest.invalid

IDENTITY CONFLICT

The same SourceSystem + SourceEventId + digest is a replay. Reusing that identity with different canonical meaning is a 409 conflict and security signal.

EDGE MAPPING

HTTP status reports the owner outcome, not the future

The endpoint performs boundary work and delegates once. It never opens a DbContext, runs policy, serializes a broker envelope or waits for a consumer.

EXECUTABLE ASSET 31-H - MINIMAL API ENDPOINT

```
group.MapPost("/", async (
    SubmitAlarmObservationV1 request,
    ClaimsPrincipal principal,
    IAcceptAlarmObservationHandler handler,
    TimeProvider time,
    CancellationToken cancellationToken) =>
{
    var command = AlarmObservationMapper.Map(
        request,
        principal.GetRequiredSubject(),
        time.GetUtcNow());

    var outcome = await handler.HandleAsync(
        command, cancellationToken);

    return AlarmObservationHttp.Map(outcome);
})
.RequireAuthorization(AlarmPolicies.SubmitObservation)
.Accepts<SubmitAlarmObservationV1>("application/json")
.Produces<AlarmObservationReceiptV1>("application/json")
.ProducesProblem(StatusCodes.Status202Accepted)
.ProducesProblem(StatusCodes.Status409Conflict)
.ProducesProblem(StatusCodes.Status503ServiceUnavailable);
```

OUTCOME CONTRACT 31-F

Application outcome	HTTP	Stable external meaning	Retry
Accepted	202	new owner state committed	same operation is safe
Replay	200	same durable receipt returned	stop
IdentityConflict	409	same source ID, different meaning	do not retry
Rejected	400/422	input or domain rule failed	change request
NotReady	503	host cannot accept safely	bounded caller retry
Unknown	503	commit outcome requires lookup	same operation ID

SECURITY AND ERRORS

Authorization precedes domain work; errors reveal no private state

Authentication answers who called. The named capability answers whether that identity may submit observations. Domain acceptance still validates the event; authorization never converts untrusted payload into truth.

EXECUTABLE ASSET 31-I - STABLE PROBLEM DETAILS

```
public static IResult IdentityConflict(
    SourceEventIdentity identity,
    HttpContext http) =>
    TypedResults.Problem(
        statusCode: StatusCodes.Status409Conflict,
        title: "Observation identity conflict",
        type: "urn:nexusl:alarm-management:identity-conflict",
        extensions: new Dictionary<string, object?>
        {
            ["code"] = "observation.identity_conflict",
            ["traceId"] = Activity.Current?.TraceId.ToString()
                ?? http.TraceIdentifier
        }
    );
```

SECURITY BOUNDARY 31-G

Control	Runtime rule	Negative test
authentication	validated issuer, audience, lifetime	missing/expired token -> 401
authorization	alarm.observation.submit capability	authenticated without claim -> 403
tenant/site scope	claim must authorize detector scope	cross-scope request -> 403
body limits	content type and byte cap	oversize -> 413
error body	opaque IDs and stable code only	no SQL/payload/token
logs	subject hash, outcome, trace; no claims dump	sensitive-token scan

FAIL CLOSED

Unknown severity, unknown policy identity, missing scope authority or invalid digest rejects before the command can mutate owned state.

APPLICATION CONTRACT

The use case has one semantic command and finite outcomes

Application coordinates identity resolution, state loading, domain policy and one commit. It does not know HTTP, SQL error numbers or RabbitMQ channels.

EXECUTABLE ASSET 31-J - COMMAND AND OUTCOMES

```
public sealed record AcceptAlarmObservationCommand(
    Guid OperationId,
    SourceEventIdentity Source,
    AlarmKey Alarm,
    DetectorScopeKey Scope,
    AlarmSeverity Severity,
    DateTimeOffset SourceOccurredAtUtc,
    DateTimeOffset ReceivedAtUtc,
    CanonicalPayloadDigest PayloadDigest,
    SubjectIdentity AcceptedBy);

public abstract record AcceptAlarmObservationOutcome
{
    public sealed record Accepted(
        AlarmObservationReceipt Receipt) : AcceptAlarmObservationOutcome;
    public sealed record Replay(
        AlarmObservationReceipt Receipt) : AcceptAlarmObservationOutcome;
    public sealed record IdentityConflict(
        SourceEventIdentity Source) : AcceptAlarmObservationOutcome;
    public sealed record Rejected(
        string Code) : AcceptAlarmObservationOutcome;
    public sealed record Unknown(
        Guid OperationId) : AcceptAlarmObservationOutcome;
}
```

EXECUTABLE ASSET 31-K - PURPOSE-BUILT ATTEMPT BOUNDARY

```
public interface IAlarmObservationAttempt : IAsyncDisposable
{
    Task<ObservationIdentityState> ResolveIdentityAsync(
        SourceEventIdentity source,
        CanonicalPayloadDigest digest,
        Cancellation token cancellationToken);

    Task<AlarmDecisionState> LoadDecisionStateAsync(
        DetectorScopeKey scope,
        Cancellation token cancellationToken);

    Task<AcceptAlarmObservationOutcome> CommitAsync(
        AlarmObservationPlan plan,
        Cancellation token cancellationToken);
}
```

NO GENERIC REPOSITORY

The port names the exact state and atomic outcome the use case needs. It cannot be reused to query arbitrary tables or cross an owner boundary.

DETERMINISTIC POLICY

A flood decision is a function of accepted evidence

The policy consumes canonical observations and an immutable snapshot. It does not inspect wall-clock now, database insertion order, thread schedule or broker delivery order.

EXECUTABLE ASSET 31-L - CANONICAL OBSERVATION

```
public sealed record AcceptedAlarmObservation(
    AlarmObservationId ObservationId,
    SourceEventIdentity Source,
    AlarmKey Alarm,
    DetectorScopeKey Scope,
    AlarmSeverity Severity,
    DateTimeOffset SourceOccurredAtUtc,
    CanonicalPayloadDigest PayloadDigest)
{
    public AlarmObservationOrderKey OrderKey => new(
        SourceOccurredAtUtc,
        Source.SourceSystem,
        Source.SourceEventId,
        ObservationId);
}

public readonly record struct AlarmObservationOrderKey(
    DateTimeOffset SourceOccurredAtUtc,
    string SourceSystem,
    string SourceEventId,
    AlarmObservationId ObservationId)
    : IComparable<AlarmObservationOrderKey>
{
    public int CompareTo(AlarmObservationOrderKey other)
    {
        var time = SourceOccurredAtUtc.CompareTo(
            other.SourceOccurredAtUtc);
        if (time != 0) return time;
        var source = StringComparer.Ordinal.Compare(
            SourceSystem, other.SourceSystem);
        if (source != 0) return source;
        var eventId = StringComparer.Ordinal.Compare(
            SourceEventId, other.SourceEventId);
        return eventId != 0
            ? eventId
            : ObservationId.Value.CompareTo(
                other.ObservationId.Value);
    }
}
```

POLICY INPUTS 31-H

Input	Canonical rule	Recorded evidence
membership	scope + severity + inclusive window	ordered observation IDs
order	occurred time, source, event ID, observation ID	order-key digest
threshold	pinned integer from policy artifact	policy ID/version
window	pinned duration; UTC arithmetic	start/end UTC
lateness	pinned acceptance horizon	accepted/rejected code
decision ID	hash of scope + policy + membership	SHA-256 identity

PRESERVATION RULE ALM-DET-001

The same ordered accepted inputs and configuration identity produce the same decision, membership and decision digest.

PURE DECISION ENGINE

Qualification is deterministic and abstention is explicit

The reference engine uses an acceptance-lab policy. A production policy must supply the same immutable fields and approval evidence; the algorithm does not silently fall back to defaults.

EXECUTABLE ASSET 31-M - ISSUE-OR-ABSTAIN POLICY

```
public static class AlarmFloodPolicy
{
    public static AlarmFloodDecision Decide(
        AlarmFloodPolicySnapshot policy,
        DetectorScopeKey scope,
        IEnumerable<AcceptedAlarmObservation> accepted)
    {
        var ordered = accepted
            .Where(x => x.Scope == scope)
            .Where(x => policy.IncludedSeverities.Contains(x.Severity))
            .OrderBy(x => x.OrderKey)
            .ToImmutableArray();

        if (ordered.Length < policy.Threshold)
            return AlarmFloodDecision.Abstain(
                scope, policy.PolicyId, "below_threshold");

        for (var end = policy.Threshold - 1;
             end < ordered.Length;
             end++)
        {
            var start = end - policy.Threshold + 1;
            var members = ordered.Slice(
                start, policy.Threshold);
            var span = members[^1].SourceOccurredAtUtc
                - members[0].SourceOccurredAtUtc;

            if (span <= policy.Window)
                return AlarmFloodDecision.Qualified(
                    scope, policy, members);
        }

        return AlarmFloodDecision.Abstain(
            scope, policy.PolicyId, "outside_window");
    }
}
```

BOUNDARY SEMANTICS

The reference fixture is inclusive: exactly Threshold observations and exactly Window duration qualify. That boundary is a reviewed rule and appears in the golden test matrix.

DECISION IDENTITY

Exact membership makes the result explainable

A detection records why it exists: scope, policy identity, ordered membership and a canonical digest. It is not reconstructed later from whichever rows happen to remain in a rolling window.

EXECUTABLE ASSET 31-N - QUALIFIED DECISION

```
public sealed record AlarmFloodQualified(
    AlarmFloodDetectionId DetectionId,
    DetectorScopeKey Scope,
    string PolicyId,
    int PolicyVersion,
    ImmutableArray<AlarmObservationId> OrderedMembers,
    DateTimeOffset WindowStartedAtUtc,
    DateTimeOffset WindowEndedAtUtc,
    string DecisionSha256);

public static AlarmFloodQualified Create(
    DetectorScopeKey scope,
    AlarmFloodPolicySnapshot policy,
    ImmutableArray<AcceptedAlarmObservation> members)
{
    var canonical = AlarmFloodCanonicalBytes.Write(
        scope, policy, members.Select(x => x.ObservationId));
    var digest = Convert.ToHexString(
        SHA256.HashData(canonical)).ToLowerInvariant();
    var id = AlarmFloodDetectionId.FromSha256(digest);

    return new(
        id,
        scope,
        policy.PolicyId,
        policy.PolicyVersion,
        members.Select(x => x.ObservationId).ToImmutableArray(),
        members[0].SourceOccurredAtUtc,
        members[^1].SourceOccurredAtUtc,
        digest);
}
```

IDENTITY INVARIANTS 31-I

Invariant	Reason	Failure meaning
same meaning -> same ID	semantic retry converges	canonicalization drift
same ID -> same digest	identity cannot be reused	terminal conflict
membership immutable	audit/replay interpret history	data corruption
policy digest pinned	configuration is evidence	unexplained decision
ordered IDs persisted	arrival order cannot rewrite truth	nondeterminism

APPLICATION ORCHESTRATION

The handler re-runs meaning in a fresh unit of work

Optimistic concurrency is not solved by saving stale tracked entities again. Each bounded attempt reloads current state, recomputes the same semantic command and resolves stable duplicate or conflict outcomes.

EXECUTABLE ASSET 31-O - BOUNDED SEMANTIC RUNNER

```
public sealed class AcceptAlarmObservationHandler(
    IAlarmObservationAttemptFactory attempts,
    IAlarmFloodPolicyProvider policies,
    IUnknownObservationResolver unknowns,
    IOptions<AlarmManagementOptions> options)
    : IAcceptAlarmObservationHandler
{
    public async Task<AcceptAlarmObservationOutcome> HandleAsync(
        AcceptAlarmObservationCommand command,
        CancellationToken cancellationToken)
    {
        for (var attempt = 1;
            attempt <= options.Value.MaxSemanticAttempts;
            attempt++)
        {
            await using var unit =
                await attempts.StartAsync(cancellationToken);
            var identity = await unit.ResolveIdentityAsync(
                command.Source,
                command.PayloadDigest,
                cancellationToken);

            if (identity is ObservationIdentityState.Replay replay)
                return new AcceptAlarmObservationOutcome.Replay(replay.Receipt);
            if (identity is ObservationIdentityState.Conflict)
                return new AcceptAlarmObservationOutcome.IdentityConflict(command.Source);

            var state = await unit.LoadDecisionStateAsync(
                command.Scope, cancellationToken);
            var policy = policies.GetRequired(state.PolicyIdentity);
            var plan = AlarmObservationPlanner.Plan(command, state, policy);
            var outcome = await unit.CommitAsync(plan, cancellationToken);

            if (outcome is not RetryableConcurrency)
                return await unknowns.ResolveIfRequiredAsync(
                    outcome, command.OperationId, cancellationToken);
        }

        return new AcceptAlarmObservationOutcome.Unknown(command.OperationId);
    }
}
```

RETRY BUDGET

Three attempts are a reference maximum, not an infinite loop. Exhaustion returns an opaque unknown outcome, emits a metric and preserves the operation identity for later reconciliation.

ATOMIC PERSISTENCE

Observation, decision evidence and outbox share one commit

The Chapter 14 schema and Chapter 26 outbox become one actual `AlarmManagementDb` unit of work. The handler never calls the broker.

EXECUTABLE ASSET 31-P - PROVIDER REALIZATION

```
public sealed class EfAlarmObservationAttempt(
    AlarmManagementDbContext db,
    IAlarmFloodPublicFactFactory publicFacts,
    IProviderErrorClassifier errors,
    TimeProvider time)
    : IAlarmObservationAttempt
{
    public async Task<AcceptAlarmObservationOutcome> CommitAsync(
        AlarmObservationPlan plan,
        CancellationToken cancellationToken)
    {
        db.AlarmObservations.Add(AlarmObservationRow.From(plan.Observation));
        plan.ApplyOwnedState(db);

        if (plan.Qualified is { } qualified)
        {
            db.FloodDetections.Add(FloodDetectionRow.From(qualified));
            db.FloodDetectionInputs.AddRange(
                qualified.OrderedMembers.Select(
                    (id, index) => FloodDetectionInputRow.From(
                        qualified.DetectionId, id, index)));

            var fact = publicFacts.Create(qualified, plan.Correlation);
            db.OutboxMessages.Add(OutboxMessageRow.Pending(
                fact, plan.HeadersJson, time.GetUtcNow()));
        }

        try
        {
            await db.SaveChangesAsync(cancellationToken);
            return plan.AcceptedOutcome();
        }
        catch (DbUpdateConcurrencyException)
        {
            return RetryableConcurrency.Instance;
        }
        catch (DbUpdateException exception)
            when (errors.IsDuplicateObservation(exception))
        {
            return await ResolveDuplicateAsync(plan, cancellationToken);
        }
    }
}
```

CHAPTER 32 BOUNDARY

`IAlarmFloodPublicFactFactory` is the owner-controlled seam. Chapter 31 verifies that its frozen result is staged atomically and remains Pending. Chapter 32 freezes the exact `AlarmFloodDetected.v1` mapping, envelope bytes and dispatch path.

PROVIDER SCHEMA

Constraints make duplicate and concurrency semantics durable

Application outcomes depend on named provider constraints. An in-memory fake cannot prove their error categories, rowversion behavior or transaction rollback.

EXECUTABLE ASSET 31-Q - CRITICAL SQL SERVER CONSTRAINTS

```
ALTER TABLE alarm.AlarmObservation ADD CONSTRAINT
UX_AlarmObservation_Source
UNIQUE (SourceSystem, SourceEventId);

ALTER TABLE alarm.AlarmObservation ADD CONSTRAINT
UX_AlarmObservation_Operation
UNIQUE (OperationId);

ALTER TABLE alarm.FloodDetection ADD CONSTRAINT
UX_FloodDetection_Decision
UNIQUE (DecisionSha256);

ALTER TABLE alarm.FloodDetectionInput ADD CONSTRAINT
PK_FloodDetectionInput
PRIMARY KEY (FloodDetectionId, MemberOrdinal);

ALTER TABLE alarm.FloodDetectionInput ADD CONSTRAINT
UX_FloodDetectionInput_Member
UNIQUE (FloodDetectionId, ObservationId);

-- DetectorScope.Version is SQL Server rowversion.
-- messaging.OutboxMessage follows Chapter 26 exactly.
```

PROVIDER CLASSIFIER 31-J

Provider evidence	Application meaning	Response
UX source + same digest	semantic replay	stored receipt
UX source + different digest	identity conflict	409 + alert
UX operation	operation replay	stored receipt
rowversion mismatch	state changed after load	fresh attempt
connection loss at commit	outcome unknown	fresh identity lookup
other provider failure	unclassified infrastructure fault	fail + evidence

NO EXCEPTION-MESSAGE PARSING

SQL Server error numbers and constraint names are pinned to the selected provider version and covered by component tests.

MIGRATION AUTHORITY

The runtime verifies schema identity; it does not migrate itself

Startup uses the DML principal. A separate migration job applies reviewed DDL. Readiness stays false when the required migration identity is absent or ahead of the compatible range.

EXECUTABLE ASSET 31-R - SCHEMA COMPATIBILITY CHECK

```
public sealed class AlarmSchemaHealthCheck(
    IDbContextFactory<AlarmManagementDbContext> contexts,
    IOptions<AlarmManagementOptions> options)
    : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        await using var db =
            await contexts.CreateDbContextAsync(cancellationToken);
        var applied = await db.Database
            .GetAppliedMigrationsAsync(cancellationToken);

        return applied.Contains(options.Value.RequiredMigration)
            ? HealthCheckResult.Healthy("schema-compatible")
            : HealthCheckResult.Unhealthy("schema-mismatch");
    }
}
```

CREDENTIAL SPLIT 31-K

Principal	Allowed	Denied
alarm_runtime	SELECT/INSERT/UPDATE owned DML	DDL, foreign DB, login admin
alarm_migrator	reviewed AlarmManagementDb DDL	runtime requests, foreign DB
alarm_observer	safe metrics/health projections	payload and mutation
backup operator	approved backup/restore operation	application command access

STARTUP POLICY

Database creation, Migrate() and broad administrative credentials are prohibited inside the host process.

HEALTH CONTRACT

Liveness is process-local; readiness protects acceptance

A broker outage does not kill or unready an outbox producer. Readiness evaluates only dependencies required to accept and durably record the next observation.

EXECUTABLE ASSET 31-S - SEPARATE PROBES

```
services.AddHealthChecks()
    .AddCheck(
        "self",
        () => HealthCheckResult.Healthy(),
        tags: ["live"])
    .AddCheck<ValidatedConfigurationHealthCheck>(
        "configuration",
        tags: ["ready"])
    .AddCheck<AlarmSchemaHealthCheck>(
        "alarm-schema",
        tags: ["ready"])
    .AddDbContextCheck<AlarmManagementDbContext>(
        "alarm-database",
        tags: ["ready"]);

app.MapHealthChecks("/health/live", new()
{
    Predicate = check => check.Tags.Contains("live")
});

app.MapHealthChecks("/health/ready", new()
{
    Predicate = check => check.Tags.Contains("ready"),
    ResponseWriter = SafeHealthResponse.WriteAsync
});
```

PROBE SEMANTICS 31-L

Condition	Liveness	Readiness	Reason
process running	healthy	depends	restart only on process failure
invalid config	healthy	unhealthy	cannot accept safely
schema missing	healthy	unhealthy	deployment action required
database unavailable	healthy	unhealthy	owner commit impossible
RabbitMQ unavailable	healthy	healthy	outbox decouples accept path
outbox pressure above cap	healthy	unhealthy	bounded storage policy

TELEMETRY CONTRACT

Signals explain the owner path without exposing the payload

Trace, metrics and structured logs share stable operation, observation and decision identities. Domain values, tokens, raw claims and outbox bytes remain outside telemetry.

EXECUTABLE ASSET 31-T - BOUNDED INSTRUMENTATION

```
internal static class AlarmTelemetry
{
    public const string Name = "Nexus1.AlarmManagement";
    public static readonly ActivitySource Activities = new(Name);
    public static readonly Meter Meter = new(Name);

    public static readonly Counter<long> Accepted =
        Meter.CreateCounter<long>("alarm_observations_accepted");
    public static readonly Counter<long> Replays =
        Meter.CreateCounter<long>("alarm_observation_replays");
    public static readonly Counter<long> Conflicts =
        Meter.CreateCounter<long>("alarm_identity_conflicts");
    public static readonly Counter<long> Floods =
        Meter.CreateCounter<long>("alarm_floods_qualified");
    public static readonly Histogram<double> CommitMilliseconds =
        Meter.CreateHistogram<double>("alarm_commit_duration", unit: "ms");
}

using var activity = AlarmTelemetry.Activities.StartActivity(
    "alarm_observation_accept", ActivityKind.Internal);
activity?.SetTag("alarm_outcome", outcome.Code);
activity?.SetTag("alarm_policy_id", policy.PolicyId);
```

ATTRIBUTE POLICY 31-M

Allowed bounded label	Example	Forbidden
outcome	accepted / replay / conflict	source event ID
severity class	critical / major / minor	alarm description
policy ID	alarm-flood-lab-v7	policy document contents
attempt bucket	1 / 2 / 3 / exhausted	exception text
dependency	alarm-db	connection string
trace correlation	trace/span IDs	access token or claims

OPERATIONAL VIEW

A service is observable when operators can choose an action

Dashboards distinguish inability to accept, deterministic conflicts, local contention and delayed publication. One generic error counter cannot support safe operation.

OPERATIONAL QUESTIONS 31-N

Question	Signal	Action
Can the owner accept?	readiness + commit latency/error	protect input; inspect DB/schema
Are senders replaying?	replay ratio by approved source class	inspect caller retry policy
Is identity being reused?	identity conflict count	security/integration investigation
Is policy qualifying floods?	qualified count + policy ID	review input and policy evidence
Is publication delayed?	pending count/oldest age/bytes	Chapter 32 relay runbook
Is concurrency exhausting?	semantic attempt histogram	inspect scope hot spots

EXECUTABLE ASSET 31-U - SAFE STRUCTURED LOG

```
logger.LogInformation(
    "Alarm observation {Outcome}; operation={OperationId}; " +
    "observation={ObservationId}; policy={PolicyId}; attempt={Attempt}",
    outcome.Code,
    command.OperationId,
    outcome.ObservationId,
    policy.PolicyId,
    attempt);
```

NO SUCCESS-BY-LOG

The log line is diagnostic evidence, not proof of commit. Provider state and the durable receipt remain authoritative.

DOMAIN ACCEPTANCE

Golden examples pin every threshold boundary

The lab fixture uses five included observations in sixty seconds. Tests name boundary meaning; they do not merely increase line coverage.

EXECUTABLE ASSET 31-V - DETERMINISTIC MATRIX

```
public static TheoryData<AlarmFixture, string> Cases => new()
{
    { AlarmFixture.FourIn60Seconds(), "below_threshold" },
    { AlarmFixture.FiveIn60Seconds(), "qualified" },
    { AlarmFixture.FiveInExactly60Seconds(), "qualified" },
    { AlarmFixture.FiveIn60SecondsOneMinorExcluded(), "below_threshold" },
    { AlarmFixture.FiveAcrossTwoScopes(), "below_threshold" },
    { AlarmFixture.SixWithEarliestQualifyingWindow(), "qualified" }
};

[Theory, MemberData(nameof(Cases))]
public void ALM_DET_001_golden_boundaries(
    AlarmFixture fixture,
    string expected)
{
    {
        var first = AlarmFloodPolicy.Decide(
            fixture.Policy, fixture.Scope, fixture.Observations);
        var second = AlarmFloodPolicy.Decide(
            fixture.Policy, fixture.Scope, fixture.Observations);

        first.Code.Should().Be(expected);
        second.Should().BeEquivalentTo(first);
    }
}
```

ADDITIONAL DOMAIN ORACLES 31-O

Case	Expected invariant
same timestamp tie	source system/event ID breaks tie deterministically
late accepted fact	canonical replay may select same earliest window
unsupported severity	excluded or rejected by explicit policy
same member set	same DetectionId and DecisionSha256
policy version changes	different decision identity
malformed policy	startup/selection fails; no default

DETERMINISM ORACLE

Every delivery permutation converges to one decision

Arrival order is deliberately varied while accepted evidence is held constant. The oracle compares decision ID, ordered membership, window and digest.

EXECUTABLE ASSET 31-W - PERMUTATION PROPERTY

```
[Fact]
public void ALM_DET_002_all_delivery_orders_converge()
{
    var fixture = AlarmFixture.SixWithEarliestQualifyingWindow();
    var expected = AlarmFloodPolicy.Decide(
        fixture.Policy,
        fixture.Scope,
        fixture.Observations);

    foreach (var permutation in fixture.Observations.Permutations())
    {
        var actual = AlarmFloodPolicy.Decide(
            fixture.Policy,
            fixture.Scope,
            permutation);

        actual.DetectionId.Should().Be(expected.DetectionId);
        actual.OrderedMembers.Should().Equal(expected.OrderedMembers);
        actual.WindowStartedAtUtc.Should().Be(expected.WindowStartedAtUtc);
        actual.WindowEndedAtUtc.Should().Be(expected.WindowEndedAtUtc);
        actual.DecisionSha256.Should().Be(expected.DecisionSha256);
    }
}
```

What this does and does not prove

- It proves the pure decision result ignores collection enumeration and delivery order for the finite fixture.
- A generator-based suite expands timestamps, scopes, severities and duplicate placement within bounded domains.
- It does not prove database concurrency, serialization bytes or production policy approval.
- Provider tests must still show that committed membership preserves the exact domain result after restart.

NO RANDOM-ONLY EVIDENCE

Every generated failure records its seed and minimized counterexample so the decision can be replayed as an immutable fixture.

APPLICATION TESTS

Orchestration tests prove call order and absent effects

A duplicate must not reload policy or stage another fact. A conflict must not mutate anything. A qualified first acceptance must commit once and never call a broker.

EXECUTABLE ASSET 31-X - REPLAY SHORT-CIRCUIT

```
[Fact]
public async Task ALM_APP_001_replay_returns_stored_receipt()
{
    var attempts = new ScriptedAttempts()
        .ResolveAsReplay(Receipts.AcceptedFlood);
    var handler = Fixture.Handler(attempts);

    var outcome = await handler.HandleAsync(
        Commands.ValidObservation, CancellationToken.None);

    outcome.Should().BeOfType<
        AcceptAlarmObservationOutcome.Replay>();
    attempts.Calls.Should().Equal(
        "Start",
        "ResolveIdentity");
    attempts.CommitCount.Should().Be(0);
    Fixture.BrokerPublishCount.Should().Be(0);
}

[Fact]
public async Task ALM_APP_002_qualified_plan_commits_once()
{
    var attempts = new ScriptedAttempts()
        .ResolveAsNew()
        .Load(States.OneBeforeThreshold)
        .CommitAsAccepted();

    var outcome = await Fixture.Handler(attempts)
        .HandleAsync(Commands.ValidObservation, default);

    outcome.Should().BeOfType<
        AcceptAlarmObservationOutcome.Accepted>();
    attempts.Calls.Should().Equal(
        "Start", "ResolveIdentity", "LoadDecisionState", "Commit");
    attempts.QualifiedPlanCount.Should().Be(1);
}
```

TEST SURFACE

These are fast Application tests with strict fakes. Transactional claims remain reserved for the selected provider suite.

PROVIDER COMPONENT TESTS

The real database counts truth and publication together

The acceptance lane applies the production migration to a disposable SQL Server instance and invokes the real EF adapter. SQLite and EF InMemory do not substitute for these constraints.

EXECUTABLE ASSET 31-Y - ATOMIC SUCCESS AND ROLLBACK

```
[Fact]
public async Task ALM_DB_001_threshold_commit_is_atomic()
{
    await fixture.SeedAsync(AlarmFixtures.FourAccepted());

    var outcome = await fixture.AcceptAsync(
        AlarmFixtures.FifthObservation());

    outcome.Should().BeAcceptedWithFlood();
    await fixture.Database.AssertCountsAsync(
        observations: 5,
        detections: 1,
        members: 5,
        pendingOutbox: 1);
    fixture.BrokerConnections.Should().Be(0);
}

[Fact]
public async Task ALM_DB_002_outbox_failure_rolls_back_all()
{
    await fixture.SeedAsync(AlarmFixtures.FourAccepted());
    fixture.Faults.FailInsert("[messaging].[OutboxMessage]");

    await Assert.ThrowsAsync<InjectedDatabaseFailure>(
        () => fixture.AcceptAsync(
            AlarmFixtures.FifthObservation()));

    await fixture.Database.AssertCountsAsync(
        observations: 4,
        detections: 0,
        members: 0,
        pendingOutbox: 0);
}
```

ACCEPTANCE ORACLE

A 202 response or mocked SaveChanges call is insufficient. The committed owner rows, membership and pending outbox row must agree in the selected engine.

CONCURRENCY, REPLAY AND RESTART

Dangerous windows are forced with barriers, never sleeps

Two writers are held after loading the same detector-scope version. One commits; the other must discard its context, reload and converge to replay or a later valid decision.

EXECUTABLE ASSET 31-Z - CONCURRENT THRESHOLD RACE

```
[Fact]
public async Task ALM_DB_003_race_converges_after_fresh_load()
{
    await fixture.SeedAsync(AlarmFixtures.FourAccepted());
    await using var barrier =
        fixture.Faults.BarrierAfterScopeLoad(participants: 2);

    var results = await Task.WhenAll(
        fixture.AcceptFreshAsync(AlarmFixtures.FifthObservation()),
        fixture.AcceptFreshAsync(AlarmFixtures.FifthObservation()));

    results.Count(x => x.IsAccepted).Should().Be(1);
    results.Count(x => x.IsReplay).Should().Be(1);
    await fixture.Database.AssertCountsAsync(
        observations: 5,
        detections: 1,
        members: 5,
        pendingOutbox: 1);
}

[Fact]
public async Task ALM_DB_004_restart_preserves_receipt_and_members()
{
    var first = await fixture.AcceptThresholdAsync();
    await fixture.RestartHostAsync();
    var replay = await fixture.AcceptAsync(first.OriginalRequest);

    replay.Receipt.Should().BeEquivalentTo(first.Receipt);
    await fixture.Database.AssertDecisionDigestAsync(
        first.Receipt.FloodDetectionId!.Value);
}
```

UNKNOWN COMMIT

A separate fault drops the client connection after server commit. Resolution queries by operation/source identity before any retry and must return the stored receipt when durability is found.

PRODUCTION-PATH HOST TESTS

WebApplicationFactory boots the same Program.cs

The test host replaces identity and database resources deliberately but preserves routing, middleware, authorization, validation, problem details, DI and endpoint metadata.

EXECUTABLE ASSET 31-AA - SECURED API CONTRACT TEST

```
public sealed class AlarmObservationApiTests(
    AlarmManagementFactory factory)
{
    [Fact]
    public async Task ALM_EDGE_001_authorized_request_returns_receipt()
    {
        using var client = factory.CreateAuthorizedClient(
            capability: "alarm.observation.submit",
            detectorScope: "site-a/line-7");

        using var response = await client.PostAsJsonAsync(
            "/api/v1/alarm-observations",
            Requests.ValidCritical());

        response.StatusCode.Should().Be(HttpStatusCode.Accepted);
        response.Content.Headers.ContentType.MediaType
            .Should().Be("application/json");
        var receipt = await response.Content
            .ReadFromJsonAsync<AlarmObservationReceiptV1>();
        receipt!.OperationId.Should()
            .Be(Requests.ValidCritical.OperationId);
    }

    [Fact]
    public async Task ALM_EDGE_002_conflict_has_stable_problem_code()
    {
        await factory.SeedAcceptedAsync(Requests.ValidCritical);
        var changed = Requests.ValidCritical with
        { PayloadSha256 = Digests.Different };

        using var response = await factory.Authorized
            .PostAsJsonAsync("/api/v1/alarm-observations", changed);

        response.StatusCode.Should().Be(HttpStatusCode.Conflict);
        await response.ShouldHaveProblemCodeAsync(
            "observation.identity_conflict");
    }
}
```

TEST CONFIGURATION

The factory pins the environment explicitly; an unset test host would default to Development and could hide production-path configuration differences.

NEGATIVE HOST TESTS

Unauthorized, oversized and unready paths mutate nothing

A secure failure is not only the correct HTTP status. The database, outbox and telemetry capture must prove that the rejected request created no business effect and leaked no sensitive value.

EXECUTABLE ASSET 31-AB - FAIL-CLOSED MATRIX

```
[Theory]
[InlineData("anonymous", HttpStatusCode.Unauthorized)]
[InlineData("wrong-capability", HttpStatusCode.Forbidden)]
[InlineData("wrong-scope", HttpStatusCode.Forbidden)]
public async Task ALM_SEC_001_denial_has_no_effect(
    string caller,
    HttpStatusCode expected)
{
    var before = await fixture.Database.CountOwnedRowsAsync();
    using var response = await fixture.Client(caller)
        .PostAsJsonAsync(
            "/api/v1/alarm-observations",
            Requests.ValidCritical);

    response.StatusCode.Should().Be(expected);
    (await fixture.Database.CountOwnedRowsAsync()).Should().Be(before);
    fixture.BrokerPublishCount.Should().Be(0);
    fixture.Telemetry.ShouldNotContain(
        Requests.ValidCritical.PayloadSha256);
}

[Fact]
public async Task ALM_HEALTH_001_schema_mismatch_is_not_ready()
{
    await fixture.Database.RemoveRequiredMigrationMarkerAsync();
    using var response = await fixture.Anonymous
        .GetAsync("/health/ready");

    response.StatusCode.Should()
        .Be(HttpStatusCode.ServiceUnavailable);
    (await response.Content.ReadAsStringAsync())
        .Should().NotContain(fixture.ConnectionString);
}
```

ADDITIONAL EDGE CASES 31-P

Case	Expected evidence
body above cap	413; handler not resolved
unsupported content type	415; zero rows
invalid UTC offset	stable validation problem
missing required member	400; exact code
unhandled exception	opaque 500; trace ID only

NO-BROKER ACCEPTANCE

Chapter 31 ends with durable Pending, never Published

The broker endpoint is intentionally unreachable and no relay worker is registered. AlarmManagement still accepts the threshold observation because its only synchronous durability boundary is AlarmManagementDb.

EXECUTABLE ASSET 31-AC - BROKER ABSENCE PROOF

```
[Fact]
public async Task ALM_RUNTIME_001_accepts_with_broker_absent()
{
    await fixture.SeedAsync(AlarmFixtures.FourAccepted());
    fixture.Broker.MakeUnreachable();

    using var response = await fixture.Authorized.PostAsJsonAsync(
        "/api/v1/alarm-observations",
        Requests.FifthCritical);

    response.StatusCode.Should().Be(HttpStatusCode.Accepted);
    await fixture.Database.AssertPendingOutboxAsync(
        expectedCount: 1,
        expectedState: OutboxState.Pending);
    fixture.Broker.ConnectionAttempts.Should().Be(0);
    fixture.Workers.ShouldNotContain("OutboxRelayWorker");
}
```

CHAPTER BOUNDARY 31-Q

Present now	Deliberately absent until Chapter 32
service-owned HTTP input and policy	published broker delivery
AlarmManagementDb migration and DML	mandatory return/confirm correlation
frozen pending public fact row	exact V1 fixture and compatibility window
safe outbox backlog metrics	relay lease/retry/recovery trace
zero broker connection attempts	consumer inbox or downstream completion

ALLOWED CLAIM

The owner has durable publication intent. No text, metric or API field says that AlarmFloodDetected.v1 reached RabbitMQ.

DEPLOYMENT ARTIFACT

The manifest keeps the first runtime narrow

The deployment supplies one host identity, one owned database connection, one telemetry export boundary and no active broker relay. Migration runs separately.

EXECUTABLE ASSET 31-AD - REFERENCE CONTAINER CONTRACT

```
services:
  alarm-management:
    image: nexus1/alarm-management@sha256:<PENDING>
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      AlarmManagement__ServiceIdentity: alarm-management
      AlarmManagement__RequiredMigration: 2026xxxx_RuntimeSlice
      AlarmManagement__OutboxRelayEnabled: "false"
      OTEL_SERVICE_NAME: nexus1-alarm-management
    secrets:
      - alarm_management_runtime_connection
    healthcheck:
      test: ["CMD", "probe", "http://localhost:8080/health/ready"]

  alarm-management-migrator:
    image: nexus1/alarm-management-migrations@sha256:<PENDING>
    profiles: ["migration"]
    secrets:
      - alarm_management_migration_connection
```

DEPLOYMENT ACCEPTANCE 31-R

Artifact	Required Identity	Evidence
host image	immutable image digest + SBOM	scan/provenance PENDING
migration	script digest + expected schema	provider apply PENDING
config	value-free manifest + policy digest	equivalence PENDING
runtime principal	secret reference, never value	permission PENDING
health	live/ready routes and timeout	fault suite PENDING
telemetry	service/resource identity	capture PENDING

ADR-029

Activate AlarmManagement as the first owner runtime

The decision binds language, data, input, policy, health and evidence into one deployable unit while keeping broker dispatch and consumers outside the synchronous acceptance path.

ADR RECORD 31-S

ADR field	ADR-029 record
decision	Run AlarmManagement as a separately composed net8.0 host with service-owned AlarmManagementDb and one versioned observation edge.
atomic boundary	Accepted observation, deterministic owner state, exact membership and pending public fact share one provider transaction.
policy	Canonical accepted inputs plus immutable policy identity; no arrival-order or wall-clock decision.
reliability	Stable source/operation identities, bounded semantic retries, unknown commit reconciliation and no synchronous broker dependency.
security	Capability + scope authorization, DML/migration credential split, safe problems/logs and payload limits.
rejected	Shared legacy writes, controller-owned policy, commit-then-publish, runtime auto-migration, broker-driven readiness.
review trigger	Input contract, policy, database provider, migration, principal, outbox, health, telemetry or ownership change.

CONSEQUENCES

The service can accept owner work during broker outage and replay decisions after restart. It also acquires explicit operational ownership of schema compatibility, idempotency conflicts, outbox pressure and policy evidence.

EVIDENCE RECORD

The runtime remains **PENDING** until exact artifacts execute

Reference code and test names define the acceptance surface. They do not create a build, database image, migration, identity provider, deployment or trace.

EXECUTABLE ASSET 31-AE - FAIL-CLOSED MANIFEST

```
{
  "evidenceId": "ALM-RUNTIME-REVIEW-001",
  "chapter": 31,
  "targetFramework": "net8.0",
  "sourceCommit": null,
  "imageDigest": null,
  "databaseProvider": null,
  "databaseImageDigest": null,
  "migrationSha256": null,
  "policySha256": null,
  "tests": {
    "ALM-ARCH-001": "PENDING",
    "ALM-DET-001": "PENDING",
    "ALM-DET-002": "PENDING",
    "ALM-APP-001": "PENDING",
    "ALM-DB-001": "PENDING",
    "ALM-DB-002": "PENDING",
    "ALM-DB-003": "PENDING",
    "ALM-EDGE-001": "PENDING",
    "ALM-SEC-001": "PENDING",
    "ALM-HEALTH-001": "PENDING",
    "ALM-RUNTIME-001": "PENDING"
  },
  "discoveredTests": null,
  "resultSha256": null,
  "accepted": false
}
```

FAIL-CLOSED VALIDATOR 31-T

Condition	Required result
discoveredTests is null or zero	reject
any required test missing/skipped/failed	reject
provider or migration digest absent	reject provider claims
policy digest differs from test fixture	reject determinism claim
broker publish claimed in Chapter 31	reject scope
all artifacts match and all tests pass	eligible for review

CHAPTER 31 - EXECUTABLE CLARIFICATION

Unknown commits and duplicate races require a fresh unit of work

A failed SaveChanges attempt does not authorize continued use of the same change tracker. Resolution starts only after disposal and performs an identity lookup in a newly created context.

EXECUTABLE ASSET 31-AA - FRESH DUPLICATE RESOLVER

```
public sealed class DuplicateObservationResolver(
    IDbContextFactory<AlarmManagementDbContext> contexts)
{
    public async Task<IngestOutcome> ResolveAsync(
        IngestPlan plan,
        CancellationToken cancellationToken)
    {
        await using var verification =
            await contexts.CreateDbContextAsync(cancellationToken);

        var stored = await verification.AlarmObservations
            .AsNoTracking()
            .SingleOrDefaultAsync(
                x => x.SourceSystem == plan.SourceSystem &&
                    x.SourceEventId == plan.SourceEventId,
                    cancellationToken);

        if (stored is null)
            return RetryableUnknownCommit.Instance;

        return FixedTimeEquals(stored.PayloadSha256, plan.PayloadSha256)
            ? StoredReceipt.From(stored)
            : ObservationIdentityConflict.From(stored, plan);
    }
}

// The catch boundary returns or exits the failed unit of work.
catch (DbUpdateConcurrencyException)
{
    return RetryableConcurrency.Instance;
}
catch (DbUpdateException exception)
{
    when (errors.IsDuplicateObservation(exception))
    {
        return await duplicates.ResolveAsync(plan, cancellationToken);
    }
}
```

EXECUTABLE ASSET 31-AB - COMPATIBLE SCHEMA HEAD AND FINGERPRINT

```
public sealed record AlarmSchemaContract(
    string RequiredBase,
    IReadOnlySet<string> SupportedHeads,
    IReadOnlyDictionary<string, string> FingerprintsByHead);

var applied = (await db.Database
    .GetAppliedMigrationsAsync(cancellationToken)).ToArray();
var head = applied.LastOrDefault();
var actual = await fingerprints.ReadAsync(db, cancellationToken);

var compatible =
    applied.Contains(contract.RequiredBase, StringComparer.Ordinal) &&
    head is not null &&
    contract.SupportedHeads.Contains(head) &&
    contract.FingerprintsByHead.TryGetValue(head, out var expected) &&
    CryptographicOperations.FixedTimeEquals(
        Convert.FromHexString(actual),
        Convert.FromHexString(expected));

return compatible
    ? HealthCheckResult.Healthy("schema-compatible")
    : HealthCheckResult.Unhealthy(
        $"schema-incompatible; head={head ?? "<none>"}");
```

SUPERSEDING CLARIFICATION

This page narrows the earlier illustrative excerpts: a required migration being present is necessary but not sufficient, and a failed tracker is never the duplicate-resolution surface.

CHAPTER 31 - EXECUTABLE CLARIFICATION

AllowedLateness is a decision input, not unused configuration

Event-time qualification must apply the configured policy before canonical ordering and flood detection. Rejection is deterministic and observable.

EXECUTABLE ASSET 31-AC - EXPLICIT EVENT-TIME GATE

```
public static EventTimeDecision QualifyEventTime(
    DateTimeOffset occurredAtUtc,
    DateTimeOffset receivedAtUtc,
    AlarmManagementPolicy policy)
{
    if (occurredAtUtc > receivedAtUtc + policy.MaximumClockSkew)
        return EventTimeDecision.Reject("future-source-time");

    var lateness = receivedAtUtc - occurredAtUtc;
    if (lateness > policy.AllowedLateness)
        return EventTimeDecision.Reject("allowed-lateness-exceeded");

    return EventTimeDecision.Accept(
        occurredAtUtc,
        lateness,
        policy.Identity);
}

var timeDecision = QualifyEventTime(
    input.OccurredAtUtc,
    clock.GetUtcNow(),
    policy);

if (timeDecision is EventTimeDecision.Rejected rejected)
    return IngestOutcome.Rejected(rejected.Code);

// Only accepted observations enter canonical ordering and detection.
```

COMPONENT EVIDENCE 31-AD

Scenario	Expected outcome	Durable assertion
within lateness budget	accepted	observation enters canonical decision input
exactly at budget	accepted	boundary behavior pinned by policy identity
one tick beyond budget	rejected	no observation, decision or outbox row
future within clock skew	accepted	skew recorded for evidence
future beyond clock skew	rejected	stable future-source-time code
policy identity changes	separate replay meaning	decision digest and evidence record change

PROOF BOUNDARY

These rules prove deterministic qualification only when the provider-backed tests execute with a fixed clock. They do not claim globally ordered arrival, perfect source clocks or broker delivery.

IMPLEMENTATION SEQUENCE AND CHECKPOINT

The owner runtime is specified; public delivery is next

Implementation should preserve the dependency order of the proof. A team must not activate the relay to make an incomplete owner path look end-to-end.

Implementation sequence

- Accept ADR-029 and freeze the service passport, input V1 contract and authorization policy.
- Implement Domain and Application projects; run the golden and permutation determinism suites.
- Apply the Chapter 14 AlarmManagementDb model and Chapter 26 outbox migration with separate credentials.
- Implement provider classification, atomicity, rollback, race, unknown-commit and restart tests on the selected SQL Server image.
- Compose Program.cs, health, telemetry and the production-path WebApplicationFactory suite.
- Deploy with the relay disabled; capture value-free configuration, image, migration, policy and test digests.
- Publish ALM-RUNTIME-REVIEW-001 only when every required result is explicit; otherwise retain PENDING.

Official technical anchors

Microsoft ASP.NET Core integration tests: <https://learn.microsoft.com/aspnet/core/test/integration-tests>

Microsoft ASP.NET Core health checks: <https://learn.microsoft.com/aspnet/core/host-and-deploy/health-checks>

Microsoft EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>

Microsoft EF Core concurrency: <https://learn.microsoft.com/ef/core/saving/concurrency>

Microsoft TimeProvider: <https://learn.microsoft.com/dotnet/api/system.timeprovider>

Microsoft policy authorization and API problem details: <https://learn.microsoft.com/aspnet/core/security/authorization/policies> ;

<https://learn.microsoft.com/aspnet/core/fundamentals/error-handling-api>

What this chapter has proved

Chapter 31 has specified the first service-owned runtime path: a versioned and authorized observation edge, finite application outcomes, canonical alarm-flood policy, exact membership, one AlarmManagementDb commit, pending outbox intent, separate health semantics, bounded telemetry and layered acceptance tests. It has not executed those artifacts and has not claimed broker publication or downstream completion.

NEXT - CHAPTER 32

Publishing AlarmFloodDetected.v1 will freeze the internal-to-public mapper and exact envelope bytes, activate the producer relay, capture mandatory routing and publisher-confirm evidence, propagate trace context and force every ambiguous crash/recovery window without allocating a new MessageId.

CHAPTER CHECKPOINT

Can the team boot the production composition path, accept the same evidence in every order, prove owner rows and Pending outbox agree, survive concurrency and restart, deny unauthorized effects and show zero broker attempts? If not, Chapter 31 remains PENDING.

PART VI - THE FIRST END-TO-END DISTRIBUTED SLICE

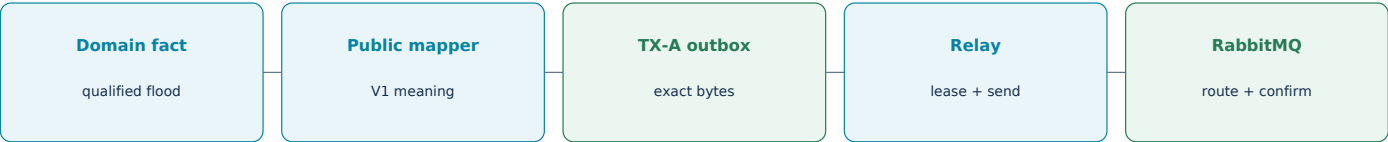
32. Publishing AlarmFloodDetected.v1

AlarmManagement already owns a committed flood decision and a Pending outbox row. This chapter freezes the public promise, publishes the exact stored bytes and records what RabbitMQ accepted without claiming consumer completion.

CHAPTER DECISION

AlarmFloodDetected.v1 is a producer-owned historical fact. Its payload, envelope identity, canonical bytes and route are fixed before the owner commit. The relay uses mandatory routing and publisher confirms; only a positive observed result may move the row to Dispatched. Unknown outcomes are replayed with the same MessageId and bytes.

PROOF PATH 32-A - FROM OWNER COMMIT TO BROKER ACCEPTANCE



GUARANTEE LEDGER 32-A

Boundary	Mechanism	Allowed claim	Still not allowed
domain -> public	reviewed deterministic mapper	stable producer meaning	consumer action
public -> outbox	one owner transaction	durable exact intent	broker acceptance
outbox -> exchange	lease + mandatory confirm	routed broker acceptance	consumer processing
attempt -> trace	bounded metadata + spans	inspectable delivery attempt	business identity from traces

ORDINARY DEVELOPER PROBLEM

The row exists, but no public delivery exists yet

A Pending outbox row proves that AlarmManagement committed publication intent. It does not prove that any queue accepted the event.

The tempting implementation loads the flood aggregate after commit, serializes its current shape and calls BasicPublishAsync. That code has four independent failure windows: current state may differ from historical state; serialization may change after deployment; an unroutable mandatory message may be mistaken for success; and a lost confirmation may lead to a new identity on retry.

NAIVE PATH 32-A - EVERY LINE WEAKENS THE HISTORICAL FACT

```
var flood = await db.FloodDetections.FindAsync(id);
var body = JsonSerializer.SerializeToUtf8Bytes(flood);
await channel.BasicPublishAsync(
    "nexus.events",
    "alarm-management.alarm-flood-detected.v1",
    mandatory: false,
    basicProperties: new BasicProperties(),
    body);

flood.MarkPublished();           // no confirm, no route evidence
await db.SaveChangesAsync();    // separate truth after broker call
```

FAILURE REVIEW 32-B

Shortcut	Why it fails	Required correction
serialize current aggregate	rewrites historical meaning	publish committed envelope bytes
mandatory=false	unroutable publish can appear accepted	mandatory route + explicit failure
no confirm budget	socket write is treated as broker durability	bounded publisher confirmation
new ID on retry	consumer dedup cannot recognize replay	same MessageId, fingerprint and bytes
mark inside publisher	mixes broker lifetime with DbContext state	fresh result transaction

PLAIN-LANGUAGE PRINCIPLE

Publish stored history. Observe the route and confirmation. Persist that observation separately. Never infer success from the absence of an exception.

EXACT PROOF PATH

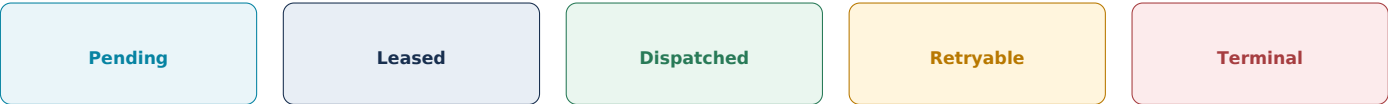
Five owners, five different truths

The public fact crosses several mechanisms, but no mechanism inherits another owner's semantics.

OWNERSHIP PATH 32-C

Stage	Owner	Durable artifact	Failure meaning
qualification	AlarmManagement Domain	qualified flood decision	no public fact if rejected
public mapping	AlarmManagement Application	V1 payload + descriptor	commit rejected if unrepresentable
owner commit	AlarmManagementDb	decision + exact outbox bytes	broker can be absent
dispatch attempt	relay worker	lease + attempt evidence	retry or terminal classification
route and persistence	RabbitMQ	confirm / return / nack	no consumer claim
consumption	RootCause	Chapter 33 inbox + case	outside this chapter

STATE MACHINE 32-B - OBSERVED OUTCOMES, NOT OPTIMISM



Confirmed + mandatory route -> Dispatched

Timeout / channel loss -> Retryable

Contract/topology defect -> Terminal

ATOMICITY REMAINS LOCAL

TX-A is the AlarmManagementDb commit. TX-B records the observed dispatch result in AlarmManagementDb after the broker operation. RabbitMQ is never enlisted in the owner transaction, and the result transaction never manufactures the fact.

The acceptance boundary

- Dispatched means mandatory routing succeeded and the confirm-aware client observed broker acceptance.
- Retryable means the broker outcome is absent or ambiguous; replay preserves identity and bytes.
- Terminal means a classified contract, authorization or topology defect needs operator action.
- None of these states says RootCause processed the message.

NORMATIVE CONTRACT FREEZE

Chapter 32 resolves the earlier illustrative V1 shapes

Chapters 19, 20, 26 and 30 introduced complementary mapper, envelope, outbox and evolution examples. This chapter now fixes the runtime contract that all later chapters must consume.

RESOLUTION LEDGER 32-D

Earlier material	What remains authoritative	What Chapter 32 fixes
Chapter 19 mapper	decision, site, line, window, ordered alarm refs, policy, detected time	normative V1 payload
Chapter 20 envelope	outer identity, producer, occurrence, correlation, causation, JCS digest	exact names and golden bytes
Chapter 26 outbox	FactKey, Messageld, route, exact bytes and local transaction	actual AlarmManagement staging path
Chapter 30 ordering lab	producer stream identity and position survive contract evolution	stream member in V1

NORMATIVE RULE

The contract below supersedes reduced or representative AlarmFloodDetectedV1 record shapes used by earlier isolated labs. Those labs remain valid for their mechanism; this exact payload, event type, route and byte corpus govern the end-to-end runtime from Chapter 32 onward.

Frozen public names

CONTRACT COORDINATES 32-E

Coordinate	Exact value
producer	alarm-management
event type	nexus1.alarm-management.alarm-flood-detected.v1
schema version	1
routing key	alarm-management.alarm-flood-detected.v1
content type	application/json; UTF-8 by envelope v1
fingerprint profile	sha-256+jcs
consumer contract	RootCause accepts only after envelope and producer validation

PRODUCER-OWNED PUBLIC CONTRACT

V1 carries sufficient immutable evidence, not the aggregate

RootCause must be able to open a case without reading AlarmManagementDb, while AlarmManagement must not publish private policy internals or EF shapes.

EXECUTABLE ASSET 32-A - EXACT PAYLOAD CONTRACT

```
namespace AlarmManagement.Contracts.Integration.V1;

[JsonUnmappedMemberHandling(
    JsonUnmappedMemberHandling.Disallow)]
public sealed record AlarmFloodDetectedV1(
    [property: JsonRequired] string DecisionId,
    [property: JsonRequired] string SiteId,
    [property: JsonRequired] string LineId,
    [property: JsonRequired] DateTimeOffset WindowStartUtc,
    [property: JsonRequired] DateTimeOffset WindowEndUtc,
    [property: JsonRequired] IReadOnlyList<AlarmReferenceV1> Alarms,
    [property: JsonRequired] string PolicyId,
    [property: JsonRequired] DateTimeOffset DetectedAtUtc,
    [property: JsonRequired] ProducerStreamPositionV1 ProducerStream);

public sealed record AlarmReferenceV1(
    [property: JsonRequired] string AlarmId,
    [property: JsonRequired] int Position);

public sealed record ProducerStreamPositionV1(
    [property: JsonRequired] string StreamId,
    [property: JsonRequired] long Position);
```

PAYLOAD MEANING 32-F

Field	Public meaning	Excluded inference
DecisionId	producer flood-decision identity	RootCause case identity
SiteId + LineId	public correlation scope	database foreign key
Window	qualified event-time interval	arrival order
Alarms	ordered immutable public references	private alarm rows
PolicyId	qualification policy identity	policy implementation
ProducerStream	producer-local sequence	global order

MINIMIZATION

The event contains no RootCause command, hypothesis, verdict, operator token, database key, stack trace, telemetry payload or mutable aggregate snapshot.

INTERNAL-TO-PUBLIC MAPPING

The mapper is deterministic and fails closed

The private domain fact may be richer. The mapper selects only reviewed public values and preserves the deterministic order already proved in Chapter 31.

EXECUTABLE ASSET 32-B - OWNER-CONTROLLED MAPPER

```
internal sealed class AlarmFloodDetectedV1Mapper
{
    public AlarmFloodDetectedV1 Map(
        AlarmFloodQualified source,
        ProducerStreamPosition stream)
    {
        var alarms = source.AcceptedAlarms
            .OrderBy(x => x.Position)
            .Select(x => new AlarmReferenceV1(
                PublicId.Demand(x.AlarmId),
                x.Position))
            .ToArray();

        if (alarms.Length == 0)
            throw new PublicFactMappingRejected("empty-alarm-set");
        if (alarms.Select(x => x.Position)
            .Distinct().Count() != alarms.Length)
            throw new PublicFactMappingRejected("duplicate-position");

        return new(
            PublicId.Demand(source.DecisionId),
            PublicId.Demand(source.Scope.SiteId),
            PublicId.Demand(source.Scope.LineId),
           Utc.Demand(source.Window.StartUtc),
            Utc.Demand(source.Window.EndUtc),
            alarms,
            PolicyId.Demand(source.PolicyId),
            Utc.Demand(source.DecidedAtUtc),
            new(stream.StreamId, stream.Position));
    }
}
```

MAPPER PROOF 32-G

Property	Required assertion	Negative case
completeness	every target field has an accepted source	missing line identity
determinism	same source and policy -> same payload	input enumeration order
no leakage	approved public IDs only	private row/version field
time clarity	UTC and start <= end <= detected	offset/local time
order	positions unique and contiguous	duplicate/gap
failure	unrepresentable value rejects commit	silent coercion

MAPPER TESTS

Permutation and leakage tests protect public meaning

A mapping test is more than a snapshot. It proves ordering, field lineage and absence of private material.

EXECUTABLE ASSET 32-C - MAPPING AND REJECTION CORPUS

```
[Theory]
[MemberData(nameof(AlarmPermutations))]
public void MAP_PUB_AM_001_is_order_independent(
    AlarmFloodQualified qualified)
{
    var mapped = mapper.Map(qualified, Streams.Position84);

    mapped.Should().BeEquivalentTo(Expected.V1,
        options => options.StrictOrdering());
    mapped.Alarms.Select(x => x.Position)
        .Should().Equal(0, 1, 2, 3, 4);
}

[Fact]
public void MAP_PUB_AM_002_rejects_private_or_invalid_identity()
{
    var source = Fixtures.Qualified with
    {
        Scope = Fixtures.ScopeWithPrivateDatabaseKey()
    };

    var act = () => mapper.Map(source, Streams.Position84);
    act.Should().Throw<PublicFactMappingRejected>()
        .Which.Code.Should().Be("non-public-identity");
}

[Fact]
public void MAP_PUB_AM_003_has_no_private_members()
{
    ContractSurface.For<AlarmFloodDetectedV1>()
        .ShouldNotContain(
            "RowVersion", "OperatorToken", "DbId",
            "InternalSeverityScore", "TraceParent");
}
```

COMMIT POLICY

When the accepted operation contract requires publication, mapping or encoding failure rejects the owner commit. The implementation does not commit a flood decision and silently omit its required public fact.

What the mapper still does not prove

- That the consumer accepts or uses the fact.
- That the public vocabulary is safe for every future producer change.
- That serialized bytes match the envelope profile.
- That the broker route exists.

STABLE FACT IDENTITY

One transition allocates one MessageId and one FactKey

Execution retries may repeat the semantic operation. They may not mint another public history for the same accepted transition.

EXECUTABLE ASSET 32-D - EXACT DESCRIPTOR

```
internal static PublicFactDescriptor Describe(
    AlarmFloodQualified source,
    CorrelationContext correlation,
    ProducerStreamPosition stream)
{
    const string producer = "alarm-management";
    const string eventType = "nexus1.alarm-management.alarm-flood-detected.v1";
    const string route = "alarm-management.alarm-flood-detected.v1";

    var factKey = PublicFactIdentity.Create(
        producer,
        source.OperationId,
        eventType,
        ordinal: 0);

    return new(
        MessageId: source.PublicFactId,
        FactKey: factKey,
        Producer: producer,
        EventType: eventType,
        SchemaVersion: 1,
        RoutingKey: route,
        OccurredAtUtc: source.DecidedAtUtc,
        Headers: new(
            correlation.CorrelationId,
            correlation.CausationId,
            correlation.TraceParent,
            correlation.TraceState),
        ProducerStream: stream);
}
```

IDENTITY SEPARATION 32-H

Identity	Purpose	Retry behavior
OperationId	idempotent accepted command	stable
DecisionId	AlarmManagement owner decision	stable
FactKey	semantic publication uniqueness	recomputed identically
MessageId	wire/inbox/replay identity	never reallocated
Stream position	producer-local ordering fact	never reset
delivery tag	channel-local transport attempt	changes
span id	diagnostic attempt	changes

UNKNOWN OWNER COMMIT

Resolve by FactKey in a fresh DbContext. If the row exists, reuse its MessageId and bytes. If no row exists, rerun the semantic operation under the original OperationId.

ENVELOPE CONSTRUCTION

The producer freezes final bytes before TX-A commits

Dispatch is a transport operation. It never invokes the mapper, JSON serializer or current domain model.

EXECUTABLE ASSET 32-E - FACTORY AND CODEC SEAM

```
public sealed class AlarmFloodPublicFactFactory(
    AlarmFloodDetectedV1Mapper mapper,
    IPublicFactSerializer serializer
) : IAlarmFloodPublicFactFactory
{
    public SerializedPublicFact Create(
        AlarmFloodQualified source,
        CorrelationContext correlation,
        ProducerStreamPosition stream)
    {
        var descriptor = Describe(source, correlation, stream);
        var payload = mapper.Map(source, stream);
        var fact = serializer.Serialize(descriptor, payload);

        if (fact.EventType != descriptor.EventType ||
            fact.RoutingKey != descriptor.RoutingKey ||
            fact.MessageId != descriptor.MessageId)
            throw new PublicFactSelfVerificationFailed();

        return fact;
    }
}

public sealed record SerializedPublicFact(
    Guid MessageId,
    string FactKey,
    string Producer,
    string EventType,
    int SchemaVersion,
    string RoutingKey,
    string ContentType,
    DateTimeOffset OccurredAtUtc,
    ReadOnlyMemory<byte> EnvelopeBytes,
    ReadOnlyMemory<byte> EnvelopeSha256);
```

SELF-VERIFICATION

The factory compares descriptor, envelope document and stored row coordinates before the transaction can commit. A route/header mismatch is a producer defect, not a retryable broker condition.

No ambient reconstruction

- OccurredAtUtc comes from the accepted decision, not dispatch time.
- Correlation and causation are persisted at creation, not read from Activity.Current later.
- Trace context can describe the creation path, but it is not hashed into business identity.
- The final byte array is copied into the outbox row and treated as immutable.

GOLDEN CONTRACT CORPUS

The fixture names one exact historical fact

The full one-line body is stored as an artifact. The page shows the semantic fixture in readable form; tests compare the exact canonical UTF-8 bytes.

EXECUTABLE ASSET 32-F - READABLE GOLDEN INSTANCE

```
{
  "envelopeVersion": 1,
  "messageId": "019bc4d4-5dc1-7b8d-9d48-542d25315356",
  "eventType": "nexus1.alarm-management.alarm-flood-detected.v1",
  "schemaVersion": 1,
  "occurredAtUtc": "2026-07-19T07:43:11.482Z",
  "producer": "alarm-management",
  "correlationId": "request-am-20482",
  "causationId": "decision-am-9184",
  "contentType": "application/json",
  "fingerprintAlgorithm": "sha-256+jcs",
  "contentFingerprint": "sha256:ef70dfaafe0e08599eb82877a77e60fe3d94b2ae0aa1456d2a175af9e2712b97",
  "payload": {
    "decisionId": "flood-am-6031",
    "siteId": "site-ath-17",
    "lineId": "line-02",
    "windowStartUtc": "2026-07-19T07:38:00.000Z",
    "windowEndUtc": "2026-07-19T07:43:00.000Z",
    "alarms": [
      { "alarmId": "alarm-am-9001", "position": 0 },
      { "alarmId": "alarm-am-9002", "position": 1 },
      { "alarmId": "alarm-am-9003", "position": 2 },
      { "alarmId": "alarm-am-9004", "position": 3 },
      { "alarmId": "alarm-am-9005", "position": 4 }
    ],
    "policyId": "alarm-flood-policy/3",
    "detectedAtUtc": "2026-07-19T07:43:11.482Z",
    "producerStream": {
      "streamId": "alarm-management/site-ath-17/line-02",
      "position": 84
    }
  }
}
```

READABLE IS NOT CANONICAL

Whitespace and display order above aid review. The artifact used by tests is the RFC 8785 canonical one-line representation, exactly 1003 UTF-8 bytes.

EXACT BYTE EVIDENCE

Two digests answer two different questions

The content fingerprint protects canonical envelope meaning with its own field omitted. EnvelopeSha256 identifies the final stored byte sequence including that fingerprint.

GOLDEN VECTOR 32-I

Artifact	Exact result	Purpose
fingerprint input	canonical envelope without contentFingerprint	semantic integrity profile
contentFingerprint	sha256:ef70dfaafe0e08599eb82877a77e60fe3d94b2ae0aa1456d2a175af9e2712b97	body self-verification
final canonical body	1003 UTF-8 bytes	published and replayed bytes
EnvelopeSha256	7cf98319c20a5915d5710bbccb5878fe51cdae89a5b68081c135fd0e962c276a	outbox/transport byte identity

EXECUTABLE ASSET 32-G - BYTE AND DIGEST GATE

```
[Fact]
public void CONTRACT_AM_FLOOD_V1_matches_golden_corpus()
{
    var encoded = fixture.Encode(Expected.QualifiedFlood);

    encoded.FinalBytes.Should().Equal(
        File.ReadAllBytes(
            "contracts/alarm-flood-detected.v1.json"));
    encoded.Document.ContentFingerprint.Should().Be(
        "sha256:ef70dfaafe0e08599eb82877a77e60fe3d94b2ae0aa1456d2a175af9e2712b97");
    Convert.ToHexString(
        SHA256.HashData(encoded.FinalBytes))
        .Should().Be(
            "7cf98319c20a5915d5710bbccb5878fe51cdae89a5b68081c135fd0e962c276a".ToUpperInvariant());
    encoded.FinalBytes.Length.Should().Be(1003);
}

[Fact]
public void one_payload_change_changes_both_digests()
{
    var changed = fixture.Encode(
        Expected.QualifiedFlood.WithPolicy("alarm-flood-policy/4"));
    changed.Fingerprint.Should().NotBe(Expected.Fingerprint);
    changed.FinalBytes.Should().NotEqual(Expected.FinalBytes);
}
```

EVIDENCE HONESTY

The vector above is deterministic manuscript evidence. The repository still needs the selected RFC 8785 implementation, official vectors and the executed .NET test report before runtime status may become VERIFIED.

TX-A REALIZATION

Decision, stream position and exact bytes commit once

Chapter 31's owner path now receives the normative factory and the producer stream reservation from Chapter 30.

EXECUTABLE ASSET 32-H - ONE ALARMMANAGEMENTDB UNIT

```
public async Task<AcceptAlarmObservationOutcome> CommitAsync(
    AlarmObservationPlan plan,
    CancellationToken cancellationToken)
{
    db.AlarmObservations.Add(
        AlarmObservationRow.From(plan.Observation));
    plan.ApplyOwnedState(db);

    if (plan.Qualified is { } qualified)
    {
        var stream = await sequences.ReserveOneAsync(
            qualified.PublicStreamId,
            cancellationToken);
        var fact = publicFacts.Create(
            qualified, plan.Correlation, stream);

        db.FloodDetections.Add(
            FloodDetectionRow.From(qualified));
        db.FloodDetectionInputs.AddRange(
            qualified.OrderedMembers.Select(
                (id, index) => FloodDetectionInputRow.From(
                    qualified.DetectionId, id, index)));
        db.OutboxMessages.Add(
            OutboxMessageRow.Pending(
                fact, plan.HeadersJson, time.GetUtcNow()));
    }

    await db.SaveChangesAsync(cancellationToken);
    return plan.AcceptedOutcome();
}
```

TX-A ROLLBACK ASSERTIONS 32-J

Injected failure	Owner rows	stream position	outbox row
payload mapping	none	none	none
canonical encoding	none	none	none
stream reservation SQL	none	none	none
outbox constraint	none	none	none
SaveChanges commit loss	resolve fresh	resolve fresh	resolve fresh
successful commit	present	84	Pending exact bytes

RELAY ACTIVATION

The relay is a separately owned process role

Chapter 31 deliberately omitted the worker. Chapter 32 enables it only when configuration, schema, topology and broker credentials pass their own gates.

EXECUTABLE ASSET 32-I - EXPLICIT ROLE REGISTRATION

```
public static IServiceCollection AddAlarmFloodPublication(
    this IServiceCollection services,
    IConfiguration configuration)
{
    services.AddOptions<AlarmFloodPublicationOptions>()
        .BindConfiguration("AlarmManagement:Publication")
        .ValidateDataAnnotations()
        .ValidateOnStart();

    services.AddSingleton<IRabbitConnectionFactory,
        RabbitConnectionFactory>();
    services.AddScoped<IOutboxClaimer, SqlOutboxClaimer>();
    services.AddScoped<IDispatchResultWriter,
        SqlDispatchResultWriter>();
    services.AddSingleton<IConfirmPublisher,
        RabbitMqConfirmPublisher>();
    services.AddSingleton<DispatchTelemetry>();

    if (configuration.GetValue<bool>("AlarmManagement:Publication:RelayEnabled"))
        services.AddHostedService<AlarmFloodRelayWorker>();

    return services;
}
```

ACTIVATION GATE 32-K

Gate	Required evidence	Failure behavior
schema	supported outbox migration/fingerprint	readiness false
topology	passive exchange + approved bindings	no relay claims
credentials	publish only to nexus.events	startup/readiness failure
contract	golden corpus digest	fail closed
configuration	confirm budget, batch, lease, retry bounds	ValidateOnStart
role	RelayEnabled and worker inventory	owner API may still run

DEPLOYMENT CHOICE

The owner API and relay may share one process in Phase-0 or run as separate roles. Their code path, credentials, health and evidence remain distinguishable.

BOUNDED WORKER LIFECYCLE

Each batch uses fresh scopes and shutdown stops new claims

A singleton hosted worker must not capture scoped DbContexts or keep claiming rows after shutdown begins.

EXECUTABLE ASSET 32-J - RELAY LOOP

```
public sealed class AlarmFloodRelayWorker(
    IServiceScopeFactory scopes,
    IConfirmingPublisher publisher,
    IOptions<AlarmFloodPublicationOptions> options)
    : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            OutboxClaim[] claims;
            await using (var claimScope =
                scopes.CreateAsyncScope())
            {
                claims = await claimScope.ServiceProvider
                    .GetRequiredService<IOutboxClaimer>()
                    .ClaimAsync(
                        options.Value.BatchSize,
                        options.Value.LeaseDuration,
                        stoppingToken);
            }

            foreach (var claim in claims)
            {
                if (stoppingToken.IsCancellationRequested)
                    break;
                var observed = await publisher.PublishAsync(
                    claim,
                    options.Value.ConfirmTimeout,
                    stoppingToken);
                await PersistResultInFreshScopeAsync(
                    claim, observed, stoppingToken);
            }

            await DelayIfEmptyAsync(claims, stoppingToken);
        }
    }
}
```

SHUTDOWN RULE

Cancellation stops new claims. In-flight publish operations receive the bounded shutdown budget. Unresolved claims expire and become eligible for replay; the worker never marks them Dispatched merely because the process is stopping.

Lifecycle tests

- No DbContext instance survives a claim or result scope.
- StopAsync completes within the declared host budget.
- A canceled unknown publish leaves a recoverable lease.
- Restart reclaims expired rows without changing identity or bytes.

TOPOLOGY PREFLIGHT

The producer proves its route before calling itself ready

A declared exchange is insufficient. AlarmFloodDetected.v1 needs at least the approved RootCause binding and any intentionally broad Audit/Reporting bindings.

EXECUTABLE ASSET 32-K - VALUE-FREE ROUTE MANIFEST

```
public static readonly PublicRouteManifest AlarmFloodRoute = new(
    Exchange: "nexus.events",
    ExchangeType: "topic",
    Durable: true,
    RoutingKey: "alarm-management.alarm-flood-detected.v1",
    RequiredBindings:
    [
        new("rootcause.alarm-events.v1",
            "alarm-management.alarm-flood-detected.v1"),
        new("audit.integration-events.v1",
            "alarm-management.#"),
        new("reporting.integration-events.v1",
            "alarm-management.#")
    ]);

public async Task<RouteReadiness> CheckAsync(
    CancellationToken cancellationToken)
{
    var observed = await brokerInventory.ReadAsync(
        AlarmFloodRoute, cancellationToken);
    return observed.SemanticallyEquals(AlarmFloodRoute)
        ? RouteReadiness.Ready(observed.Digest)
        : RouteReadiness.NotReady(
            "alarm-flood-route-drift",
            observed.ValueFreeDiff);
}
```

DRIFT MATRIX 32-L

Observed defect	Publish observation	Operational result
exchange absent	channel error / unknown	readiness false
no matching binding	mandatory return	terminal topology defect
queue absent	mandatory return	terminal topology defect
queue has no quorum	confirm delayed/unknown	retryable + alert
credential denied	channel closes	terminal authorization defect
approved route	positive confirm	eligible for Dispatched

RABBITMQ .NET 7.X ADAPTER

Mandatory publish and confirm tracking form one observation

The current asynchronous client enables confirmations when creating the channel. **BasicPublishAsync** is bounded by the confirmation budget and mandatory routing.

EXECUTABLE ASSET 32-L - CONFIRMING PUBLISHER

```
public sealed class RabbitMqConfirmingPublisher(
    IConnection connection,
    RabbitMqTopology topology,
    IAmqpPropertyFactory properties)
    : IConfirmingPublisher
{
    public async Task<PublishObservation> PublishAsync(
        OutboxClaim claim,
        TimeSpan timeout,
        CancellationToken cancellationToken)
    {
        using var budget =
            CancellationTokenSource.CreateLinkedTokenSource(
                cancellationToken);
        budget.CancelAfter(timeout);

        var channelOptions = new CreateChannelOptions(
            publisherConfirmationsEnabled: true,
            publisherConfirmationTrackingEnabled: true);

        try
        {
            await using var channel =
                await connection.CreateChannelAsync(
                    channelOptions, budget.Token);
            await channel.BasicPublishAsync(
                exchange: topology.PublicFactsExchange,
                routingKey: claim.RoutingKey,
                mandatory: true,
                basicProperties: properties.Create(claim),
                body: claim.EnvelopeBytes,
                cancellationToken: budget.Token);
            return PublishObservation.Confirmed(clock.UtcNow());
        }
        catch (PublishException ex)
        {
            return PublishObservation.Rejected(
                ClassifyPublishException(ex));
        }
        catch (OperationCanceledException)
        {
            when (!cancellationToken.IsCancellationRequested)
            {
                return PublishObservation.Unknown("confirm-timeout");
            }
        }
        catch (AlreadyClosedException)
        {
            return PublishObservation.Unknown("channel-closed");
        }
    }
}
```

CURRENT API NOTE

RabbitMQ.Client 7.x BasicPublishAsync throws **PublishException** when confirmation tracking observes a nack or mandatory return. A confirmation proves broker-side acceptance for the route; it remains orthogonal to consumer acknowledgement.

AMQP BINDING

Broker properties duplicate selected facts; the body stays authoritative

Properties support routing, diagnostics and broker durability. Any duplicated identity must agree with the stored envelope before publishing.

EXECUTABLE ASSET 32-M - PROPERTIES FROM PERSISTED INTENT

```
public BasicProperties Create(OutboxClaim claim)
{
    headers.DemandBodyAgreement(claim);
    var trace = headers.ReadTraceContext(claim.HeadersJson);

    return new BasicProperties
    {
        AppId = claim.Producer,
        MessageId = claim.MessageId.ToString("D").ToLowerInvariant(),
        Type = claim.EventType,
        ContentType = claim.ContentType,
        ContentEncoding = "utf-8",
        CorrelationId = claim.CorrelationId,
        DeliveryMode = DeliveryModes.Persistent,
        Timestamp = new AmqpTimestamp(
            claim.OccurredAtUtc.ToUnixTimeSeconds()),
        Headers = new Dictionary<string, object?>
        {
            ["schema-version"] = claim.SchemaVersion,
            ["causation-id"] = claim.CausationId,
            ["traceparent"] = trace.TraceParent,
            ["tracestate"] = trace.TraceState,
            ["envelope-sha256"] =
                Convert.ToHexString(
                    claim.EnvelopeSha256).ToLowerInvariant()
        }
    };
}
```

HEADER POLICY 32-M

Property/header	Source	Retry behavior
MessageId	stored envelope/outbox	stable
Type	stored event type	stable
CorrelationId	stored creation context	stable
traceparent	stored creation diagnostic context	stable carrier
delivery tag	RabbitMQ channel	changes
attempt count	outbox telemetry	never published as business data

HEADER SAFETY

Headers are length-bounded and allow-listed. Credentials, user tokens, raw payload fragments and arbitrary baggage never enter the outbox or logs.

TRACE CONTINUITY

Business causation survives even when tracing does not

CorrelationId and CausationId are public contract values. Trace context is diagnostic metadata whose availability and sampling may vary by attempt.

IDENTITY AND TRACE LEDGER 32-N

Value	Question answered	Authority	Retry behavior
MessageId	which public fact?	producer contract	stable
CorrelationId	which larger workflow?	producer contract	stable
CausationId	which accepted cause?	producer contract	stable
traceparent	which creation context?	W3C diagnostic carrier	stored carrier stable
publish span	which send attempt?	telemetry runtime	new per attempt
consumer process span	which handler execution?	consumer telemetry	Chapter 33

EXECUTABLE ASSET 32-N - PRODUCER SEND SPAN

```
using var activity = Telemetry.ActivitySource.StartActivity(
    "publish nexus.events",
    ActivityKind.Producer,
    parentContext: traceCreationContext,
    tags:
    [
        new("messaging.system", "rabbitmq"),
        new("messaging.destination.name", "nexus.events"),
        new("messaging.operation.name", "publish"),
        new("messaging.operation.type", "send"),
        new("messaging.message.id", claim.MessageId.ToString("D"))
    ]
);

var observed = await publisher.SendCoreAsync(claim, cancellationToken);
activity?.SetTag("nexus1.publish.outcome", observed.Code);
if (observed.IsFailure)
    activity?.SetTag("error.type", observed.ErrorType);
```

NO TRACE-BASED CORRECTNESS

Sampling, exporter outage or an invalid incoming traceparent may remove a trace. Outbox state, MessageId, correlation, causation and broker observation still determine correctness.

DISPATCH STATE MACHINE

The persisted state records what was observed

The relay does not store a philosophical truth about network reality. It records the strongest result available from mandatory routing and publisher confirmation.

EXECUTABLE ASSET 32-O - RESULT ALGEBRA

```
public abstract record PublishObservation(string Code)
{
    public sealed record Confirmed(DateTimeOffset AtUtc)
        : PublishObservation("confirmed");
    public sealed record Rejected(string Reason)
        : PublishObservation(Reason);
    public sealed record Unknown(string Reason)
        : PublishObservation(Reason);
}

public static DispatchTransition Decide(
    OutboxClaim claim,
    PublishObservation observed,
    DispatchPolicy policy) => observed switch
{
    PublishObservation.Confirmed confirmed =>
        DispatchTransition.MarkDispatched(
            claim, confirmed.AtUtc),

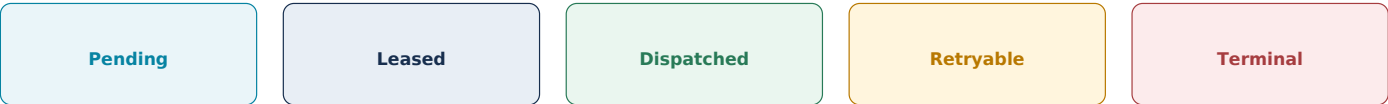
    PublishObservation.Rejected rejected
        when policy.IsTerminal(rejected.Reason) =>
        DispatchTransition.MarkTerminal(
            claim, rejected.Reason),

    PublishObservation.Rejected rejected =>
        DispatchTransition.Retry(
            claim, policy.NextDelay(claim.AttemptCount),
            rejected.Reason),

    PublishObservation.Unknown unknown =>
        DispatchTransition.RetrySameIdentity(
            claim, policy.NextDelay(claim.AttemptCount),
            unknown.Reason),

    _ => throw new UnreachableException()
};
```

STATE MACHINE 32-B - OBSERVED OUTCOMES, NOT OPTIMISM



Confirmed + mandatory route -> Dispatched Timeout / channel loss -> Retryable Contract/topology defect -> Terminal

ALLOWED TRANSITIONS 32-O

From	Observation	To	Invariant
Pending/Retryable	claim succeeds	Leased	bounded owner lease
Leased	positive confirm	Dispatched	same row/version
Leased	return/nack classified	Retryable/Terminal	reason recorded
Leased	timeout/channel loss	Retryable	same MessageId/bytes
Leased	worker dies	lease expires	no optimistic success

ATOMIC CLAIMING

SQL Server grants a finite lease, not ownership forever

Multiple relay instances may run. One provider statement selects and updates the eligible rows so two workers cannot legitimately hold the same active lease.

EXECUTABLE ASSET 32-P - ALARMMANAGEMENT OUTBOX CLAIM

```
;WITH candidates AS
(
    SELECT TOP (@batchSize) *
    FROM messaging.OutboxMessage WITH
        (UPDLOCK, READPAST, ROWLOCK)
    WHERE State IN (0, 2)
        AND (NextAttemptAtUtc IS NULL
            OR NextAttemptAtUtc <= @now)
        AND (LockExpiresAtUtc IS NULL
            OR LockExpiresAtUtc <= @now)
    ORDER BY StoredAtUtc, MessageId
)
UPDATE candidates
SET State = 1,
    LockedBy = @dispatcherId,
    LockExpiresAtUtc = @leaseUntil,
    AttemptCount = AttemptCount + 1
OUTPUT
    inserted.MessageId,
    inserted.FactKey,
    inserted.Producer,
    inserted.EventType,
    inserted.SchemaVersion,
    inserted.RoutingKey,
    inserted.ContentType,
    inserted.OccurredAtUtc,
    inserted.EnvelopeBytes,
    inserted.EnvelopeSha256,
    inserted.HeadersJson,
    inserted.AttemptCount,
    inserted.Version;
```

CLAIM PROOF 32-P

Scenario	Required observation	Forbidden result
two workers	each row returned once	dual active lease
expired lease	new worker reclaims	identity/bytes changed
future retry	row skipped	hot-loop claim
terminal row	never selected	automatic poison replay
canceled batch	unclaimed rows untouched	partial hidden claim

PROVIDER PROOF REQUIRED

READPAST, UPDLOCK and rowversion behavior are SQL Server claims. They require concurrent component tests on the selected provider image; an in-memory fake cannot validate them.

FRESH RESULT TRANSACTION

Broker observation is persisted in a new DbContext

The context used to claim the row is already disposed. The result writer checks the lease owner and rowversion before applying the transition.

EXECUTABLE ASSET 32-Q - GUARDED RESULT WRITE

```
public sealed class SqlDispatchResultWriter(
    IDbContextFactory<AlarmManagementDbContext> contexts,
    TimeProvider time)
    : IDispatchResultWriter
{
    public async Task<ResultWriteOutcome> ApplyAsync(
        OutboxClaim claim,
        DispatchTransition transition,
        CancellationToken cancellationToken)
    {
        await using var db =
            await contexts.CreateDbContextAsync(cancellationToken);
        var row = await db.OutboxMessages.SingleOrDefaultAsync(
            x => x.MessageId == claim.MessageId,
            cancellationToken);

        if (row is null)
            return ResultWriteOutcome.Missing;
        if (!row.IsLeaseHeldBy(
            claim.DispatcherId, claim.Version, time.GetUtcNow()))
            return ResultWriteOutcome.StaleLease;

        row.Apply(transition, time.GetUtcNow());
        try
        {
            await db.SaveChangesAsync(cancellationToken);
            return ResultWriteOutcome.Committed;
        }
        catch (DbUpdateConcurrencyException)
        {
            return ResultWriteOutcome.ConcurrentChange;
        }
    }
}
```

RESULT RACES 32-Q

Race	Safe outcome	Next action
lease expired before result	StaleLease	reload; never overwrite
another worker changed row	ConcurrentChange	fresh reconciliation
row already Dispatched	idempotent no-op	record duplicate observation
row missing	Missing + alert	repair investigation
cancel before SaveChanges	no transition	lease/retry recovery

UNKNOWN RESULT COMMIT

A lost SQL response after the result write is resolved in yet another fresh context by MessageId and state. It never triggers a new public fact.

AMBIGUOUS WINDOWS

Every crash point has one explicit recovery rule

At-least-once publication is produced by honest handling of uncertainty, not by pretending the boundary is atomic.

CRASH MATRIX 32-R

Crash or loss point	Durable state	What may be true	Recovery
before TX-A	no owner fact/outbox	nothing published	retry original operation
after TX-A, before claim	Pending exact bytes	not sent	normal claim
after claim, before socket write	Leased	not sent	lease expiry + replay
after write, before route result	Leased	unknown	same bytes/MessageId
after route, before confirm received	Leased	broker may hold it	same bytes/MessageId
after confirm, before TX-B	Leased	broker holds it	duplicate-safe replay
after TX-B	Dispatched	broker accepted route	no normal republish
after consumer, before its ack	Dispatched	consumer may commit	Chapter 33 inbox

WHY DUPLICATION IS CORRECT

If the publisher cannot distinguish 'lost before broker' from 'accepted but confirmation lost', republishing is the only safe producer action. Stable MessageId and the Chapter 33 inbox turn that uncertainty into a duplicate no-op.

Forbidden recovery shortcuts

- Do not query RootCause to discover whether publication happened.
- Do not inspect queue contents as business truth.
- Do not mark Dispatched after timeout to avoid duplicates.
- Do not create a replacement event with a new MessageId.

FAILURE LABORATORY

The crash harness controls the exact boundary

A useful failure test pauses at named checkpoints, kills the worker and restarts the real production composition path.

EXECUTABLE ASSET 32-R - DETERMINISTIC CRASH PROBE

```
public enum PublishCheckpoint
{
    Claimed,
    BeforeSocketWrite,
    AfterSocketWrite,
    MandatoryRouteObserved,
    ConfirmObserved,
    BeforeResultCommit,
    ResultCommitted
}

[Theory]
[InlineData(PublishCheckpoint.Claimed)]
[InlineData(PublishCheckpoint.AfterSocketWrite)]
[InlineData(PublishCheckpoint.ConfirmObserved)]
[InlineData(PublishCheckpoint.BeforeResultCommit)]
public async Task PUB_REC_001_restart_preserves_identity(
    PublishCheckpoint checkpoint)
{
    var expected = await fixture.StageGoldenFactAsync();
    fixture.Probe.KillWorkerAt(checkpoint);

    await fixture.StartRelayAsync();
    await fixture.Probe.WaitForKillAsync();
    await fixture.RestartRelayAsync();
    await fixture.WaitForOutboxDispatchedAsync(expected.MessageId);

    fixture.Broker.BodiesFor(expected.MessageId)
        .Should().OnlyContain(bytes =>
            bytes.SequenceEqual(expected.EnvelopeBytes));
    fixture.Broker.MessageIds.Should()
        .OnlyContain(id => id == expected.MessageId);
}
```

HARNESS EVIDENCE 32-S

Artifact	Required content	Reject when
probe log	checkpoint + worker instance + timestamp	checkpoint not reached
database trace	state/version/lease transitions	hidden state rewrite
broker capture	MessageId + body digest + route	body unavailable
restart record	image/config/build identity	different untracked build
test result	discovered and executed cases	zero discovery

ROUTING AND CONFIRM TESTS

Unroutable is failure even when a broker ack follows

RabbitMQ can return a mandatory unroutable message before confirming that it processed the publish. The client observation must classify the return as failure.

EXECUTABLE ASSET 32-S - BROKER-BACKED ROUTE MATRIX

```
[Fact]
public async Task PUB_ROUTE_001_required_binding_confirms()
{
    await fixture.Topology.DeclareApprovedAsync();
    var claim = await fixture.StageAndClaimGoldenAsync();

    var observed = await fixture.Publisher.PublishAsync(
        claim, TimeSpan.FromSeconds(5), CancellationToken.None);

    observed.Should().BeOfType<PublishObservation.Confirmed>();
    await fixture.AssertQueueContainsAsync(
        "rootcause.alarm-events.v1", claim.MessageId);
}

[Fact]
public async Task PUB_ROUTE_002_missing_binding_is_not_success()
{
    await fixture.Topology.DeclareWithoutRootCauseBindingAsync();
    var claim = await fixture.StageAndClaimGoldenAsync();

    var observed = await fixture.Publisher.PublishAsync(
        claim, TimeSpan.FromSeconds(5), CancellationToken.None);

    observed.Should().BeEquivalentTo(
        new PublishObservation.Rejected(
            "mandatory-return-unroutable"));
    await fixture.AssertNotDispatchedAsync(claim.MessageId);
}
```

NEGATIVE TOPOLOGY PROOF

The test intentionally removes only the binding, not the exchange. It therefore proves that mandatory routing is observed instead of merely proving that a publish call can reach RabbitMQ.

Additional broker cases

- Broker unavailable before connection: Retryable and no row loss.
- Channel closed by permission error: Terminal authorization defect.
- Confirm delayed beyond budget: Unknown and same-identity replay.
- Broker restart after confirmed persistent publish: queue evidence retained under the selected durability profile.

PRODUCER CONTRACT TESTS

The producer owns the contract and the route together

A payload test that ignores envelope coordinates and routing can pass while the deployed consumer receives nothing.

EXECUTABLE ASSET 32-T - PRODUCER CONTRACT SURFACE

```
[Fact]
public void CONTRACT_AM_001_freezes_coordinates()
{
    ContractRegistry.AlarmFloodDetectedV1.Should().BeEquivalentTo(
        new ContractDescriptor(
            Producer: "alarm-management",
            EventType: "nexus1.alarm-management.alarm-flood-detected.v1",
            SchemaVersion: 1,
            RoutingKey: "alarm-management.alarm-flood-detected.v1",
            ContentType: "application/json",
            FingerprintProfile: "sha-256+jcs"));
}

[Fact]
public void CONTRACT_AM_002_schema_and_route_agree()
{
    var descriptor = Describe(
        Fixtures.Qualified, Correlations.Valid, Streams.Position84);
    var document = fixture.DecodeGoldenEnvelope();

    descriptor.EventType.Should().Be(document.EventType);
    descriptor.SchemaVersion.Should().Be(document.SchemaVersion);
    descriptor.Producer.Should().Be(document.Producer);
    TopologyManifest.RoutesFor(descriptor.RoutingKey)
        .Should().Contain("rootcause.alarm-events.v1");
}

[Fact]
public void CONTRACT_AM_003_contract_project_has_no_owner_dependencies()
{
    Architecture.ForAssembly(typeof(AlarmFloodDetectedV1).Assembly)
        .ShouldNotReference(
            "AlarmManagement.Domain",
            "AlarmManagement.Infrastructure",
            "Microsoft.EntityFrameworkCore");
}
```

CHANGE CLASSIFICATION 32-T

Proposed change	V1 action
rename public field	breaking; new version
change units or meaning	breaking; new version
add required field	breaking; new version
add optional field	only after supported-consumer proof
change route binding	topology migration; bytes unchanged
change serializer output	reject unless golden bytes remain exact

COMPATIBILITY CORPUS

The frozen V1 is tested against every supported consumer

Chapter 33 will own RootCause behavior, but Chapter 32 already proves that the published bytes match the consumer's accepted V1 grammar and semantic mapper.

EXECUTABLE ASSET 32-U - PRODUCER/CONSUMER CORPUS GATE

```
[Fact]
public void CONTRACT_AM_RC_001_golden_v1_is_accepted()
{
    var bytes = Corpus.AlarmFloodDetectedV1.GoldenBytes;
    var accepted = rootCauseContractReader.Accept(
        bytes,
        AuthenticatedProducer.AlarmManagement,
        BrokerProperties.ForGoldenV1);

    accepted.Document.MessageId.Should().Be(
        "019bc4d4-5dc1-7b8d-9d48-542d25315356");
    accepted.TypedPayload.Should()
        .BeOfType<AlarmFloodDetectedV1>();
}

[Theory]
[InlineData(nameof(RejectedV1Corpus))]
public void CONTRACT_AM_RC_002_invalid_v1_fails_before_domain(
    byte[] bytes,
    string expectedCode)
{
    var act = () => rootCauseContractReader.Accept(
        bytes,
        AuthenticatedProducer.AlarmManagement,
        BrokerProperties.ForGoldenV1);

    act.Should().Throw<EnvelopeRejected>()
        .Which.Code.Should().Be(expectedCode);
    rootCauseDomain.Invocations.Should().Be(0);
}
```

REJECTED CORPUS 32-U

Mutation	Stable rejection
unknown top-level member	envelope-unknown-member
changed MessageId with old body digest	broker-body-identity-mismatch
same ID with changed alarm reference	fingerprint-mismatch
unsupported schemaVersion	unsupported-contract
alarm position duplicated	invalid-public-order
producer credential not AlarmManagement	producer-not-authorized

BOUNDARY

This proves contract admission only. Chapter 33 must still prove inbox atomicity, case creation, initial evidence and consumer acknowledgement.

OPERATIONAL TELEMETRY

Metrics describe backlog, attempts and uncertainty

Telemetry supports diagnosis but does not upgrade a state transition. Every metric uses bounded labels and every log omits payloads and credentials.

TELEMETRY CONTRACT 32-V

Signal	Meaning	Allowed labels
outbox.pending	eligible pending/retryable rows	producer,event_type
outbox.oldest_age	age of oldest eligible row	producer
publish.attempts	transport attempts	event_type,outcome
publish.confirm_latency	observed confirm duration	event_type
publish.unknown	ambiguous outcomes	reason
publish.terminal	operator-required defects	reason
outbox.lease_expired	reclaimed work	producer

EXECUTABLE ASSET 32-V - SAFE DISPATCH EVIDENCE

```
logger.LogInformation(
    "Public fact dispatch observed " +
    "message_id={MessageId} event_type={EventType} " +
    "attempt={Attempt} outcome={Outcome} " +
    "confirm_ms={ConfirmMs}",
    claim.MessageId,
    claim.EventType,
    claim.AttemptCount,
    observed.Code,
    observed.ConfirmLatency.TotalMilliseconds);

// Never log:
// claim.EnvelopeBytes, claim.HeadersJson, broker credentials,
// user tokens, alarm descriptions or arbitrary exception bodies.
```

CARDINALITY GATE

MessageId belongs in trace/log evidence, not metric labels. SiteId, LineId, correlation, causation and dispatcher instance also stay out of metric dimensions.

HEALTH AND BACKLOG

Readiness protects the role; backlog health protects the workflow

A temporary broker outage should not kill the owner API. The relay role exposes its own readiness and a separate operational backlog assessment.

HEALTH MODEL 32-W

Probe/signal	Healthy when	Must not do
API liveness	process loop responsive	call broker/database
API readiness	owner schema + DML ready	depend on consumer
relay liveness	worker scheduler responsive	publish test message
relay readiness	schema, credential, topology checks pass	mutate topology
backlog status	age/count within declared SLO	mark process dead
consumer progress	Chapter 33 evidence	block producer commit

EXECUTABLE ASSET 32-W - BACKLOG ASSESSMENT

```
public BacklogStatus Evaluate(
    OutboxBacklogSnapshot snapshot,
    PublicationPolicy policy)
{
    if (snapshot.TerminalCount > 0)
        return BacklogStatus.Unhealthy(
            "terminal-publication-defect");
    if (snapshot.OldestEligibleAge > policy.CriticalAge)
        return BacklogStatus.Unhealthy(
            "publication-backlog-critical");
    if (snapshot.OldestEligibleAge > policy.WarningAge)
        return BacklogStatus.Degraded(
            "publication-backlog-delayed");
    return BacklogStatus.Healthy();
}
```

NO PROBE SIDE EFFECTS

The route readiness check is passive and value-free. It never declares objects, publishes a synthetic business event or drains a queue.

SECURITY BOUNDARY

The relay credential can publish, but cannot administer

Phase-0 still separates runtime publication from topology provisioning and consumer administration.

CREDENTIAL MATRIX 32-X

Principal	Allowed	Denied
topology provisioner	declare approved exchanges/queues/bindings	consume business queues
AlarmManagement relay	connect + write nexus.events	configure, consume, write nexus.dead
RootCause consumer	read/ack its live queue	publish as AlarmManagement
operator repair	authorized replay path	unrecorded arbitrary publish
telemetry exporter	export bounded signals	broker/database credentials

EXECUTABLE ASSET 32-X - VALUE-FREE SECRET CONFIGURATION

```
"AlarmManagement": {
  "Publication": {
    "RelayEnabled": true,
    "BrokerEndpoint": "amqps://rabbitmq:5671",
    "VirtualHost": "/nexus1",
    "CredentialReference": "secret://alarm-relay",
    "PublicFactsExchange": "nexus.events",
    "ConfirmTimeout": "00:00:05",
    "LeaseDuration": "00:00:30",
    "BatchSize": 32
  }
}
```

DEMONSTRATOR BOUNDARY

This credential split is a Phase-0 trust mechanism, not a claim of zero-trust maturity, production secrets management or penetration hardening. Part VIII will own the wider distributed security model.

RETENTION AND REPLAY

Dispatched history remains long enough to support evidence

Outbox cleanup is an audited lifecycle, not a nightly delete that erases the ability to explain publication.

LIFECYCLE POLICY 32-Y

State	Retention rule	Replay authority
Pending	never delete; alert by age	automatic relay
Retryable	never delete before terminal decision	bounded automatic relay
Leased	recover by lease expiry	automatic after expiry
Dispatched	retain through audit/replay horizon	operator-only republish
Terminal	retain with reason and incident	authorized repair only

EXECUTABLE ASSET 32-Y - REPLAY PRESERVES THE PUBLIC FACT

```
public AuthorizedReplay CreateReplay(
    OutboxHistoryRow original,
    OperatorAuthorization authorization,
    string reason)
{
    authorization.Demand("public-fact:replay");
    if (!original.IsReplayEligible)
        throw new ReplayRejected("state-not-replayable");

    return new AuthorizedReplay(
        OriginalMessageId: original.MessageId,
        OriginalEnvelopeBytes: original.EnvelopeBytes.ToArray(),
        OriginalEnvelopeSha256:
            original.EnvelopeSha256.ToArray(),
        NewReplayOperationId: replayIds.New(),
        RequestedBy: authorization.SubjectDigest,
        ReasonCode: ReasonCode.Demand(reason),
        RequestedAtUtc: clock.UtcNowNow());
}
```

REPLAY IS NEW WORK, NOT A NEW FACT

The replay operation gets its own authorization, audit identity and send span. The public MessageId, envelope bytes, fingerprint and producer stream position remain unchanged.

REPRODUCIBLE BROKER LAB

The test environment pins the durability assumptions

The repository records the RabbitMQ image digest, queue policy, topology manifest and client package line used by the evidence run.

EXECUTABLE ASSET 32-Z - COMPOSE TEST SLICE

```
services:
  rabbitmq:
    image: rabbitmq:4-management@sha256:<pinned-digest>
    hostname: rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 5s
      timeout: 5s
      retries: 20

  alarm-relay:
    image: nexus1/alarm-management@sha256:<pinned-digest>
    environment:
      AlarmManagement__Publication__RelayEnabled: "true"
      AlarmManagement__Publication__BrokerEndpoint:
        "amqp://rabbitmq:5672"
    depends_on:
      rabbitmq:
        condition: service_healthy

  sql:
    image: mcr.microsoft.com/mssql/server:2025-latest@sha256:<pinned-digest>
```

LAB SEQUENCE 32-Z

Step	Evidence
apply AlarmManagement migration	schema head + fingerprint
provision topology/policy	value-free manifest digest
stage golden fact with broker stopped	Pending owner commit
start broker and relay	mandatory confirm + TX-B
interrupt confirm channel	same-identity replay
read RootCause queue without ack	exact body digest
restart broker	durability observation

NO DECORATIVE EXECUTION

Placeholders above must be replaced by actual image digests in the repository manifest. Until the lab executes, the chapter remains PENDING.

FAIL-CLOSED EVIDENCE

The review record distinguishes specified from executed

A successful PDF build is not broker evidence. The manifest accepts only named, fresh and version-correlated artifacts.

EXECUTABLE ASSET 32-AA - PUBLICATION EVIDENCE RECORD

```
{
  "claimId": "ALM-PUBLISH-V1-REVIEW-001",
  "status": "PENDING",
  "bookChapter": 32,
  "contract": {
    "eventType":
      "nexus1.alarm-management.alarm-flood-detected.v1",
    "schemaVersion": 1,
    "routingKey":
      "alarm-management.alarm-flood-detected.v1",
    "goldenBodySha256":
      "7cf98319c20a5915d5710bbccb5878fe51cdae89a5b68081c135fd0e962c276a"
  },
  "requiredArtifacts": [
    "mapper-and-golden-corpus.trx",
    "sqlserver-owner-atomicity.trx",
    "rabbitmq-route-confirm.trx",
    "crash-window-recovery.trx",
    "trace-and-metrics-snapshot.json",
    "topology-manifest.json",
    "runtime-identity.json"
  ],
  "executedArtifacts": [],
  "missingReason":
    "reference repository and provider lab not executed here"
}
```

FAIL-CLOSED VALIDATOR 32-AA

Reject when	Why
test discovery is zero	absence is not success
artifact build differs	results do not describe this runtime
broker/topology digest absent	route assumptions are unbound
golden body digest differs	contract bytes drifted
crash case skipped	recovery claim unproved
timestamp outside review window	stale evidence

HONEST BOUNDARY

This chapter specifies executable .NET, SQL Server and RabbitMQ evidence. It does not claim that the NEXUS-1 repository currently builds or that any broker, provider or failure lab has produced the required artifacts.

ADR AND IMPLEMENTATION SEQUENCE

Freeze public history before optimizing throughput

Batch size, channel pooling and concurrency are secondary. The first accepted implementation proves one fact through every uncertainty window.

ADR-030 32-AB

Field	Decision
Context	AlarmManagement has durable Pending V1 intent but no public delivery proof.
Decision	Freeze one producer-owned V1; dispatch stored bytes with mandatory confirms.
Consequences	at-least-once publication, duplicate-safe replay, explicit topology dependency
Rejected	direct aggregate serialization, fire-and-forget, new ID on retry, broker transaction
Evidence	golden corpus, provider atomicity, route/confirm and crash-window suites

Implementation sequence

- Accept ADR-030 and the normative contract-resolution ledger.
- Generate producer contract types, mapper, serializer registry and golden corpus.
- Bind the Chapter 31 commit path to stream reservation and exact V1 staging.
- Enable the relay role with validated options, separate credentials and passive readiness.
- Run SQL Server claim/result concurrency tests before RabbitMQ tests.
- Run mandatory-route, confirm, timeout, restart and broker durability cases.
- Capture safe traces, bounded metrics and value-free topology/runtime manifests.
- Publish ALM-PUBLISH-V1-REVIEW-001 only when every required artifact is present.

OPTIMIZATION GATE

Only after the one-message proof passes may the team reuse channels, pipeline confirms or widen batches. Every optimization must preserve per-message result correlation, shutdown bounds and exact-byte replay.

IMPLEMENTATION SEQUENCE AND CHECKPOINT

The public fact is specified; consumer-owned truth is next

Chapter 32 ends at broker acceptance and a durable producer dispatch record. RootCause has not yet created a case.

Official technical anchors

RabbitMQ publisher confirms and mandatory routing: <https://www.rabbitmq.com/docs/confirm>

RabbitMQ .NET client API guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>

RabbitMQ.Client IChannel and CreateChannelOptions API: <https://rabbitmq.github.io/rabbitmq-dotnet-client/api/>

W3C Trace Context: <https://www.w3.org/TR/trace-context/>

OpenTelemetry messaging span conventions: <https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/>

RFC 8785 JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785>

What this chapter has proved

Chapter 32 has fixed one normative AlarmFloodDetected.v1 payload, event type, route, golden body and digest; joined that representation to the owner commit; activated a bounded relay; classified route, confirm and crash outcomes; preserved identity and bytes on replay; and named the exact provider, broker, trace and evidence gates. It has not executed those artifacts and has not claimed RootCause receipt, case creation, consumer acknowledgement or end-to-end completion.

NEXT - CHAPTER 33

RootCause Consumes and Opens a Case will authenticate AlarmManagement, validate the exact V1 envelope, atomically record the inbox and RootCauseCase with initial evidence, publish RootCauseCaseOpened.v1 and acknowledge only after the local commit.

Chapter checkpoint

- Which exact payload and envelope bytes define AlarmFloodDetected.v1?
- Why does mandatory routing matter in addition to publisher confirms?
- What may Dispatched claim, and what can it never claim?
- Why does timeout force replay with the same MessageId and bytes?
- How are business causation and trace context kept separate?
- Which crash point creates a possible duplicate after a positive broker outcome?
- What evidence must exist before the chapter status can change from PENDING?

PART VI - THE FIRST END-TO-END DISTRIBUTED SLICE

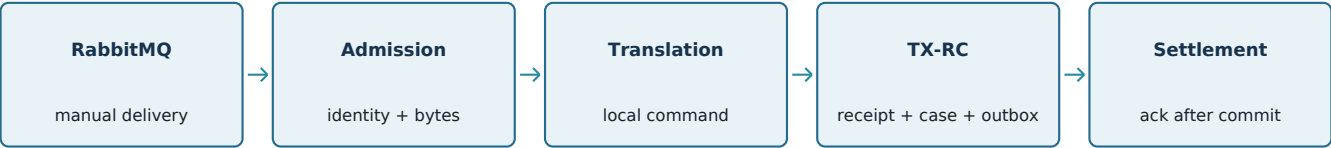
33. RootCause Consumes and Opens a Case

RootCause consumes the exact AlarmFloodDetected.v1 bytes fixed in Chapter 32, creates one owner-local case with initial evidence, and acknowledges only after its database records both the effect and the next public intent.

CHAPTER DECISION

The RootCause consumer authenticates the producer path, validates the envelope and payload before domain work, then commits InboxReceipt, inbound lineage, RootCauseCase, five initial evidence items and RootCauseCaseOpened.v1 in one RootCauseDb transaction. Only a known commit or matching duplicate permits acknowledgement.

PROOF PATH 33-A - FROM BROKER DELIVERY TO OWNER TRUTH



GUARANTEE LEDGER 33-A

Boundary	Mechanism	Allowed claim	Still not allowed
queue -> contract	strict admission	accepted V1 meaning	producer private state
contract -> case	RootCause policy	one owned case effect	verdict exists
case -> outbox	one local transaction	durable CaseOpened intent	broker delivery
commit -> ack	manual settlement	safe queue removal	exactly-once transport

ORDINARY DEVELOPER PROBLEM

The callback is not the transaction

A broker callback can deliver valid bytes more than once, disappear after a local commit, or be interrupted before acknowledgement. The tempting handler deserializes, inserts a case, calls another service for missing data and acknowledges in a finally block. Every step silently expands the failure window.

NAIVE PATH 33-A - TRANSPORT OPTIMISM BECOMES BUSINESS LOSS

```
consumer.ReceivedAsync += async (_, ea) =>
{
    var message = JsonSerializer.Deserialize<AlarmFloodDetectedV1>(ea.Body.Span!);
    var alarm = await alarmManagementDb.Alarms.FindAsync(message.DecisionId);

    db.RootCauseCases.Add(new RootCauseCaseRow
    {
        CaseId = Guid.NewGuid(),
        SourceDecisionId = message.DecisionId
    });

    await db.SaveChangesAsync();
    await channel.BasicAckAsync(ea.DeliveryTag, multiple: false);
};
```

FAILURE REVIEW 33-B

Shortcut	Failure	Required correction
deserialize DTO only	properties, fingerprint and exact grammar are ignored	admit envelope before Domain
read AlarmManagementDb	shared truth and availability coupling	consume sufficient public fact
Guid.NewGuid per attempt	duplicate delivery creates another case	RootCause semantic correlation
Save case only	no receipt or public intent	one RootCauseDb commit
ack after any return	rollback or unknown commit can lose work	ack only durable success/duplicate

PLAIN-LANGUAGE PRINCIPLE

Copy the delivery, prove what it is, translate it into RootCause language, commit all owner-local truth once, and only then settle the broker delivery.

EXACT CONSUMER BOUNDARY

Six owners, no borrowed authority

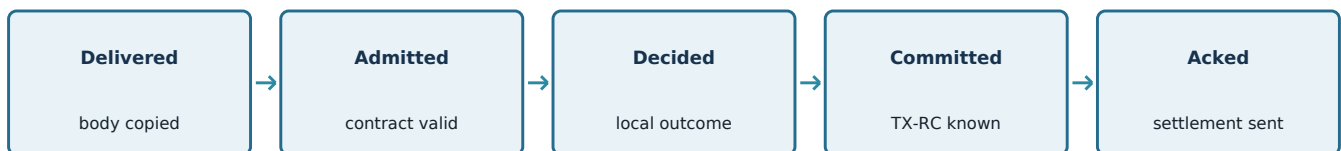
The same delivery crosses transport, contract, application, domain, persistence and settlement surfaces. Each surface may assert only the fact it owns.

OWNERSHIP PATH 33-C

Stage	Owner	Durable artifact	Explicit limit
delivery	RabbitMQ	queued body and properties	not RootCause processing
admission	RootCause Edge	accepted typed delivery in memory	not a database effect
translation	RootCause Application	owner-local command	not aggregate mutation yet
decision	RootCause Domain	case and initial evidence transition	not public delivery
TX-RC	RootCauseDb	receipt, lineage, case, evidence, outbox	not broker acknowledgement
settlement	consumer adapter	ack/retry/reject observation	not exactly-once transport

DELIVERY STATE MACHINE

STATE MACHINE 33-D - OBSERVED OUTCOMES



OUTCOME BOUNDARY 33-E

Observation	Local meaning	Broker action
matching receipt	effect already durable	ack
new commit succeeds	one effect and public intent durable	ack
commit unknown	receipt may or may not exist	do not ack; resolve/redeliver
contract/auth invalid	no domain invocation	reject to terminal evidence
transient dependency	no known commit	bounded retry policy

PHYSICAL DELIVERY ADMISSION

Copy first; references do not outlive the callback

RabbitMQ.Client 7.x exposes the body as ReadOnlyMemory<byte>. The consumer copies or fully deserializes it before the delivery handler returns. RootCause also freezes the properties needed for admission and settlement; it never retains EventingBasicConsumerEventArgs.

EXECUTABLE ASSET 33-A - IMMUTABLE RAW DELIVERY

```
public sealed record RawAlarmFloodDelivery(
    ulong DeliveryTag,
    bool Redelivered,
    string Exchange,
    string RoutingKey,
    string? MessageId,
    string? Type,
    string? AppId,
    string? ContentType,
    string? ContentEncoding,
    string? CorrelationId,
    IReadOnlyDictionary<string, object?> Headers,
    byte[] Body);

internal static RawAlarmFloodDelivery Copy(
    BasicDeliverEventArgs ea) => new(
    ea.DeliveryTag,
    ea.Redelivered,
    ea.Exchange,
    ea.RoutingKey,
    ea.BasicProperties.MessageId,
    ea.BasicProperties.Type,
    ea.BasicProperties.AppId,
    ea.BasicProperties.ContentType,
    ea.BasicProperties.ContentEncoding,
    ea.BasicProperties.CorrelationId,
    CopyHeaders(ea.BasicProperties.Headers),
    ea.Body.ToArray());
```

ADMISSION LIMITS 33-F

Input	Exact rule	Failure code
body	1..262144 bytes; copied before callback exits	body-size-invalid
exchange	nexus.events	unexpected-exchange
routing key	alarm-management.alarm-flood-detected.v1	unexpected-route
content type	application/json + utf-8	unsupported-content
message type	exact normative event type	unexpected-event-type
message identity	UUID property equals envelope messageId	message-id-mismatch

MEMORY SAFETY IS NOT IDEMPOTENCY

Copying preserves bytes after callback return. Only the durable RootCause inbox and semantic key prevent repeated business meaning.

STRICT V1 READER

Admission proves coordinates and meaning together

EXECUTABLE ASSET 33-B - ROOTCAUSE CONTRACT READER

```
public sealed class AlarmFloodV1Admission(
    ICanonicalEnvelopeReader envelopes,
    IProducerPathAuthorizer producerPath)
{
    public AcceptedAlarmFloodDelivery Accept(
        RawAlarmFloodDelivery raw,
        BrokerRouteProvenance provenance)
    {
        DemandPhysicalCoordinates(raw);
        producerPath.DemandAlarmManagement(provenance, raw);

        var document = envelopes.ReadV1Strict(raw.Body);
        DemandEqual(raw.MessageId, document.MessageId, "message-id");
        DemandEqual(raw.Type, document.EventType, "event-type");
        DemandEqual(raw.CorrelationId, document.CorrelationId, "correlation-id");
        DemandEqual("alarm-management", document.Producer, "producer");
        DemandEqual(Expected.EventType, document.EventType, "event-type");
        DemandEqual(1, document.SchemaVersion, "schema-version");
        DemandEqual("sha-256+jcs", document.FingerprintAlgorithm, "fingerprint-profile");

        var bodySha256 = SHA256.HashData(raw.Body);
        envelopes.VerifyContentFingerprint(document);
        var payload = document.Payload.Deserialize(
            RootCauseInboundJsonContext.Default.AlarmFloodDetectedV1)
            ?? throw EnvelopeRejected.Required("payload");
        AlarmFloodSemantics.DemandValid(payload);

        return AcceptedAlarmFloodDelivery.Create(
            raw, document, payload, bodySha256);
    }
}
```

AUTHORITATIVE CONSTANTS 33-G

Coordinate	Required value
consumer	root-cause/alarm-flood/v1
event type	nexus1.alarm-management.alarm-flood-detected.v1
routing key	alarm-management.alarm-flood-detected.v1
schema version	1
body digest for golden input	7cf98319c20a5915d5710bbcbb5878fe51cdae89a5b68081c135fd0e962c276a

NO PARTIAL ACCEPTANCE

A valid JSON payload inside mismatched AMQP properties is rejected. Matching properties around a mutated body are also rejected. Transport and envelope identities describe one history or no history.

REJECTED CONTRACT CORPUS

Invalid input stops before RootCause Domain

EXECUTABLE ASSET 33-C - FAIL-CLOSED CORPUS GATE

```
[Theory]
[MemberData(nameof(RejectedAlarmFloodV1Corpus))]
public void RC_CONTRACT_001_rejects_before_domain(
    RawAlarmFloodDelivery delivery,
    string expectedCode)
{
    var act = () => admission.Accept(
        delivery, RouteProvenance.AlarmManagementOnly);

    act.Should().Throw<EnvelopeRejected>()
        .Which.Code.Should().Be(expectedCode);
    rootCauseDomain.Invocations.Should().Be(0);
    rootCauseDb.Commands.Should().BeEmpty();
}

[Fact]
public void RC_CONTRACT_002_accepts_chapter32_golden_bytes()
{
    var accepted = admission.Accept(
        Deliveries.Chapter32Golden,
        RouteProvenance.AlarmManagementOnly);

    accepted.MessageId.Should().Be(
        Guid.Parse("019bc4d4-5dc1-7b8d-9d48-542d25315356"));
    accepted.EnvelopeSha256.Should().Equal(
        Convert.FromHexString(
            "7cf98319c20a5915d5710bbcb5878fe51cdae89a5b68081c135fd0e962c276a"));
}
```

REJECTED CORPUS 33-H

Mutation	Stable rejection	Domain calls
unknown top-level member	envelope-unknown-member	0
AMQP MessageId differs	message-id-mismatch	0
old fingerprint after payload mutation	fingerprint-mismatch	0
schemaVersion 2	unsupported-contract	0
duplicate alarm position	invalid-public-order	0
window end before start	invalid-window	0
wrong producer route authority	producer-path-not-authorized	0

EVIDENCE RULE

Rejected bytes are identified by digest, code, route and observation time. Payload values are not copied into ordinary logs or metric labels.

CONSUMER-OWNED TRANSLATION

A public fact becomes a RootCause command

RootCause does not import the AlarmManagement aggregate and does not treat DecisionId as its own Caseld. It creates an explicit source key, a local evidence seed and a causality context. Delivery attempts, redelivery flags and trace identifiers are excluded from Domain meaning.

EXECUTABLE ASSET 33-D - TRANSLATION BOUNDARY

```
public sealed record OpenCaseFromAlarmFlood(
    SourceFloodKey Source,
    RootCauseCorrelationKey Correlation,
    AlarmFloodEvidenceSeed Evidence,
    RootCauseCasePolicyId CasePolicy,
    PublicFactCausality Causality);

internal static OpenCaseFromAlarmFlood Translate(
    AcceptedAlarmFloodDelivery accepted,
    RootCauseCasePolicyId casePolicy)
{
    var p = accepted.Payload;
    return new(
        SourceFloodKey.Create("alarm-management", p.DecisionId),
        RootCauseCorrelationKey.ForFlood(p.SiteId, p.LineId, p.DecisionId),
        AlarmFloodEvidenceSeed.Create(
            accepted.MessageId, p.DecisionId, p.SiteId, p.LineId,
            p.WindowStartUtc, p.WindowEndUtc, p.Alarms,
            p.PolicyId, p.DetectedAtUtc, p.ProducerStream),
        casePolicy,
        new PublicFactCausality(
            accepted.CorrelationId,
            accepted.MessageId.ToString("D")));
}
```

TRANSLATION EXCLUSIONS 33-I

Public input	RootCause use	Never becomes
DecisionId	source semantic key	Caseld
alarm references	candidate initial evidence	AlarmManagement entity
PolicyId	producer evidence lineage	RootCause case policy
producer stream	ordering/provenance fact	database FK
MessageId	inbox + causation identity	verdict identity
traceparent	telemetry parent when valid	business causation

INITIAL EVIDENCE

Five alarm references become five admitted items

The Chapter 32 golden fact contains five ordered alarm references. RootCause admits them as evidence only after validating the complete source window and order. A stable EvidenceKey is derived from producer, decision, alarm and position; the local EvidenceId remains RootCause-owned.

EXECUTABLE ASSET 33-E - EVIDENCE IDENTITY AND PROVENANCE

```
public sealed record InitialAlarmEvidence(
    RootCauseEvidenceId Id,
    EvidenceKey Key,
    int CanonicalPosition,
    string AlarmId,
    AlarmFloodProvenance Provenance);

public static InitialAlarmEvidence From(
    SourceFloodKey source,
    AlarmReferenceV1 alarm,
    AlarmFloodProvenance provenance,
    IRootCauseIdentitySource identities)
{
    var material = String.Join('|',
        "alarm-flood-v1",
        source.Producer,
        source.DecisionId,
        alarm.AlarmId,
        alarm.Position.ToString(CultureInfo.InvariantCulture));

    return new(
        identities.NewEvidenceId(),
        EvidenceKey.Sha256(material),
        alarm.Position,
        alarm.AlarmId,
        provenance);
}
```

EVIDENCE ADMISSION 33-J

Rule	Required behavior	Rejection
non-empty	at least one alarm reference	initial-evidence-empty
order	positions exactly 0..N-1	invalid-public-order
window	start < end <= detected time	invalid-window
identity	unique EvidenceKey within case	evidence-key-conflict
lineage	source message, decision, stream and policy retained	lineage-incomplete

RECORDED TIME IS NOT OBSERVED TIME

DetectedAtUtc and the flood window come from the producer fact. ReceivedAtUtc and RecordedAtUtc are operational observations and never reorder evidence.

ROOTCAUSE AGGREGATE TRANSITION

Opening a case is a domain decision

EXECUTABLE ASSET 33-F - ONE OPENING TRANSITION

```
public sealed class RootCauseCase
{
    private readonly List<InitialAlarmEvidence> _evidence = [];
    private readonly List<object> _facts = [];

    private RootCauseCase() { }

    public RootCauseCaseId Id { get; private set; }
    public SourceFloodKey Source { get; private set; }
    public RootCauseCorrelationKey Correlation { get; private set; }
    public RootCauseCaseStatus Status { get; private set; }
    public RootCauseCasePolicyId PolicyId { get; private set; }
    public DateTimeOffset OpenedAtUtc { get; private set; }
    public IReadOnlyList<InitialAlarmEvidence> Evidence => _evidence;

    public static RootCauseCase OpenFromAlarmFlood(
        RootCauseCaseId id,
        OpenCaseFromAlarmFlood command,
        DateTimeOffset openedAtUtc,
        IRootCauseIdentitySource identities)
    {
        var items = command.Evidence.Alarms
            .Select(x => InitialAlarmEvidence.From(
                command.Source, x, command.Evidence.Provenance, identities))
            .OrderBy(x => x.CanonicalPosition)
            .ToArray();
        DemandInitialEvidence(items);

        var result = new RootCauseCase
        {
            Id = id,
            Source = command.Source,
            Correlation = command.Correlation,
            PolicyId = command.CasePolicy,
            Status = RootCauseCaseStatus.Open,
            OpenedAtUtc = openedAtUtc
        };
        result._evidence.AddRange(items);
        result._facts.Add(new RootCauseCaseOpened(
            id, command.Source, openedAtUtc,
            items.Count, command.Causality));
        return result;
    }
}
```

OPENING POSTCONDITIONS 33-K

Postcondition	Exact result
status	Open
verdicts/hypotheses	none
initial evidence	five ordered items for the golden fact
source key	alarm-management + flood-am-6031
domain fact	one RootCauseCaseOpened internal fact
public promise	not yet serialized or dispatched

AGGREGATE PROOF

Opening is deterministic over accepted meaning

EXECUTABLE ASSET 33-G - DOMAIN TESTS

```
[Fact]
public void RC_OPEN_001_golden_flood_opens_with_five_items()
{
    var sut = RootCauseCase.OpenFromAlarmFlood(
        Ids.Case, Commands.Chapter32Golden,
        Time.OpenedAt, Ids.Fixed);

    sut.Status.Should().Be(RootCauseCaseStatus.Open);
    sut.Evidence.Should().HaveCount(5);
    sut.Evidence.Select(x => x.CanonicalPosition)
        .Should().Equal(0, 1, 2, 3, 4);
    sut.Facts.Should().ContainSingle()
        .Which.Should().BeOfType<RootCauseCaseOpened>();
}

[Fact]
public void RC_OPEN_002_permuted_input_has_same_evidence_keys()
{
    var left = Open(Commands.Chapter32Golden);
    var right = Open(Commands.Chapter32Golden.WithReorderedTransportArray());

    right.Evidence.OrderBy(x => x.CanonicalPosition)
        .Select(x => x.Key)
        .Should().Equal(left.Evidence.Select(x => x.Key));
}

[Theory]
[InlineData("empty")]
[InlineData("gap")]
[InlineData("duplicate-position")]
[InlineData("window-reversed")]
public void RC_OPEN_003_invalid_seed_has_no_fact(string mutation)
{
    var act = () => Open(Commands.Invalid(mutation));
    act.Should().Throw<DomainRuleViolation>();
    Ids.Fixed.PublishedIdentities.Should().BeEmpty();
}
```

DETERMINISM BOUNDARY
The same accepted public meaning produces the same evidence keys and order. Locally allocated UUIDs and OpenedAtUtc are supplied once by Application and persisted; a retry reloads committed values instead of regenerating history.

WHAT OPENING DOES NOT DECIDE 33-L

Question	Owner / later chapter
root cause verdict	RootCause analysis policy; Chapter 34 starts only after a verdict exists
Audit evidence	Audit subscriber
Compliance review	Compliance subscriber
console freshness	BFF/Angular projection in Chapter 35

NORMATIVE OUTGOING CONTRACT

RootCauseCaseOpened.v1 states creation, not conclusion

The outgoing fact allows approved observers to know that RootCause created an owner-local case from one AlarmFloodDetected.v1. It contains no hypothesis, verdict, mutable aggregate graph or private evidence payload.

EXECUTABLE ASSET 33-H - EXACT V1 PAYLOAD

```
namespace RootCause.Contracts.Integration.V1;

[JsonUnmappedMemberHandling(JsonUnmappedMemberHandling.Disallow)]
public sealed record RootCauseCaseOpenedV1(
    [property: JsonRequired] string CaseId,
    [property: JsonRequired] string SourceAlarmFloodMessageId,
    [property: JsonRequired] string SourceDecisionId,
    [property: JsonRequired] string SiteId,
    [property: JsonRequired] string LineId,
    [property: JsonRequired] DateTimeOffset OpenedAtUtc,
    [property: JsonRequired] int InitialEvidenceCount,
    [property: JsonRequired] string CasePolicyId,
    [property: JsonRequired] ProducerStreamPositionV1 ProducerStream);
```

CONTRACT COORDINATES 33-M

Coordinate	Exact value
producer	root-cause
event type	nexus1.root-cause.root-cause-case-opened.v1
schema version	1
routing key	root-cause.root-cause-case-opened.v1
content type	application/json; UTF-8 by envelope v1
fingerprint profile	sha-256+jcs

MINIMIZATION

InitialEvidenceCount proves that opening did not create an empty case. The individual evidence items remain RootCause-owned and are exposed only through a separately authorized contract if a future use case requires them.

STABLE OUTGOING IDENTITY

One case-open transition creates one public fact

EXECUTABLE ASSET 33-I - DESCRIPTOR AND EXPLICIT MAPPER

```
internal static PublicFactDescriptor Describe(
    RootCauseCaseOpened source,
    RootCauseCaseOpenedIdentity identity)
{
    const string type =
        "nexus1.root-cause.root-cause-case-opened.v1";
    const string route = "root-cause.root-cause-case-opened.v1";

    return new(
        MessageId: identity.MessageId,
        FactKey: $"root-cause-case-opened-v1|{source.CaseId.Value:D}",
        Producer: "root-cause",
        EventType: type,
        SchemaVersion: 1,
        RoutingKey: route,
        OccurredAtUtc: source.OpenedAtUtc,
        CorrelationId: source.Causality.CorrelationId,
        CausationId: source.Causality.SourceMessageId);
}

internal static RootCauseCaseOpenedV1 Map(
    RootCauseCase aggregate,
    RootCauseCaseOpened source) => new(
    aggregate.Id.Value.ToString("D"),
    source.Causality.SourceMessageId,
    aggregate.Source.DecisionId,
    aggregate.Correlation.SiteId,
    aggregate.Correlation.LineId,
    source.OpenedAtUtc,
    aggregate.Evidence.Count,
    aggregate.PolicyId.Value,
    new ProducerStreamPositionV1(
        $"root-cause/case/{aggregate.Id.Value:D}", 1));
```

IDENTITY RULES 33-N

Identity	Authority	Retry rule
CaseId	RootCause first successful opening	reload committed value
FactKey	case-open transition	recompute identically
MessageId	RootCause public fact	never reallocate after commit
CausationId	AlarmFloodDetected MessageId	preserve exactly
producer stream position	RootCause case stream	first fact is position 1

GOLDEN OUTGOING CORPUS

One readable case-opened instance

EXECUTABLE ASSET 33-J - READABLE GOLDEN INSTANCE

```
{
  "envelopeVersion": 1,
  "messageId": "019bc4d4-6ea0-7d14-a57a-3e8643c55f13",
  "eventType": "nexus1.root-cause.root-cause-case-opened.v1",
  "schemaVersion": 1,
  "occurredAtUtc": "2026-07-19T07:43:12.017Z",
  "producer": "root-cause",
  "correlationId": "request-am-20482",
  "causationId": "019bc4d4-5dc1-7b8d-9d48-542d25315356",
  "contentType": "application/json",
  "fingerprintAlgorithm": "sha-256+jcs",
  "payload": {
    "caseId": "019bc4d4-6dfe-7bcb-9de8-3af71979285a",
    "sourceAlarmFloodMessageId": "019bc4d4-5dc1-7b8d-9d48-542d25315356",
    "sourceDecisionId": "flood-am-6031",
    "siteId": "site-ath-17",
    "lineId": "line-02",
    "openedAtUtc": "2026-07-19T07:43:12.017Z",
    "initialEvidenceCount": 5,
    "casePolicyId": "root-cause/open-case/1",
    "producerStream": {
      "streamId": "root-cause/case/019bc4d4-6dfe-7bcb-9de8-3af71979285a",
      "position": 1
    }
  },
  "contentFingerprint": "sha256:4a04853244adcdbc8f35834949ea0532fdd43d3cc24fe225e71104c8a581ce27"
}
```

READABLE IS NOT CANONICAL

Tests use the RFC 8785 canonical one-line representation: exactly 876 UTF-8 bytes. Whitespace and display order on this page exist only for human review.

EXACT OUTGOING BYTE EVIDENCE

Meaning digest and final-byte digest stay separate

GOLDEN VECTOR 33-O

Artifact	Exact result	Purpose
fingerprint input	canonical envelope without contentFingerprint	semantic integrity profile
contentFingerprint	sha256:4a04853244adcdbc8f35834949ea0532fdd43d3cc24fe225e71104c8a581ce27	body self-verification
final canonical body	876 UTF-8 bytes	stored and replayed bytes
EnvelopeSha256	454f51cf4b0250b9c855346a6648a7d73530f19687ee3b52cda2edf9d0339c5b	outbox/transport byte identity

EXECUTABLE ASSET 33-K - OUTGOING GOLDEN GATE

```
[Fact]
public void CONTRACT_RC_CASE_OPENED_V1_matches_golden_corpus()
{
    var encoded = fixture.Encode(Expected.Case0pened);

    encoded.FinalBytes.Should().Equal(
        File.ReadAllBytes(
            "contracts/root-cause-case-opened.v1.json"));
    encoded.Document.ContentFingerprint.Should().Be(
        "sha256:4a04853244adcdbc8f35834949ea0532fdd43d3cc24fe225e71104c8a581ce27");
    Convert.ToHexString(SHA256.HashData(encoded.FinalBytes))
        .Should().Be(
            "454f51cf4b0250b9c855346a6648a7d73530f19687ee3b52cda2edf9d0339c5b".ToUpperInvariant());
    encoded.FinalBytes.Length.Should().Be(876);
}

[Fact]
public void causation_is_inbound_message_not_trace_id()
{
    var document = fixture.DecodeGolden();
    document.CausationId.Should().Be(
        "019bc4d4-5dc1-7b8d-9d48-542d25315356");
    document.CausationId.Should().NotBe(TraceIds.Sample);
}
```

EVIDENCE HONESTY

These vectors are internally calculated for the manuscript fixture. Repository compilation, serializer conformance and cross-service execution remain PENDING until their versioned artifacts exist.

ROOTCAUSEDB SPECIALIZATION

Transport idempotency and business lineage are different rows

Chapter 28 introduced `messaging.InboxReceipt`. Chapter 15 introduced `root_cause.inbound_public_facts`. Chapter 33 resolves their roles: `InboxReceipt` absorbs repeated transport identity, while `InboundPublicFact` preserves the `RootCause` interpretation of one producer decision. Both are local to `RootCauseDb` and commit together.

RESOLUTION LEDGER 33-P

Artifact	Unique key	Meaning
<code>messaging.InboxReceipt</code>	<code>ConsumerName + MessageId</code>	this subscription completed this body
<code>root_cause.InboundPublicFact</code>	<code>Producer + DecisionId + ContractVersion</code>	this producer decision already entered <code>RootCause</code>
<code>root_cause.RootCauseCase</code>	<code>Caseld; unique SourceFloodKey</code>	one owner-local case
<code>root_cause.CaseEvidence</code>	<code>Caseld + EvidenceKey</code>	one admitted evidence meaning
<code>messaging.OutboxMessage</code>	<code>MessageId; unique FactKey</code>	one outgoing public intent

EXECUTABLE ASSET 33-L - ROOTCAUSE SEMANTIC INTAKE

```
CREATE TABLE root_cause.InboundPublicFact
(
  InboundFactId      uniqueidentifier NOT NULL,
  Producer            varchar(80)      NOT NULL,
  DecisionId         varchar(120)     NOT NULL,
  ContractVersion     varchar(32)      NOT NULL,
  SourceMessageId     uniqueidentifier NOT NULL,
  EnvelopeSha256      binary(32)      NOT NULL,
  RootCauseCaseId     uniqueidentifier NOT NULL,
  OutcomeCode         varchar(80)      NOT NULL,
  RecordedAtUtc       datetimeoffset(7) NOT NULL,

  CONSTRAINT PK_RootCause_InboundPublicFact
    PRIMARY KEY (InboundFactId),
  CONSTRAINT UX_RootCause_InboundDecision
    UNIQUE (Producer, DecisionId, ContractVersion)
);
```

WHY TWO IDENTITIES

A replay with the same `MessageId` is a transport duplicate. A defective or repaired producer may send the same `DecisionId` under a new `MessageId`; that is a semantic duplicate and still must not open another case.

PERSISTENCE ROWS AND MAPPINGS

Constraints arbitrate races; adapters do not guess

EXECUTABLE ASSET 33-M - SPECIALIZED ROWS

```

internal sealed class RootCauseCaseRow
{
    public Guid CaseId { get; init; }
    public string SourceFloodKey { get; init; } = null!;
    public string SiteId { get; init; } = null!;
    public string LineId { get; init; } = null!;
    public string State { get; set; } = "Open";
    public string PolicyIdentity { get; init; } = null!;
    public DateTimeOffset OpenedAtUtc { get; init; }
    public byte[] Version { get; set; } = null!;
    public List<CaseEvidenceRow> Evidence { get; } = [];
}

internal sealed class CaseEvidenceRow
{
    public Guid EvidenceId { get; init; }
    public Guid CaseId { get; init; }
    public byte[] EvidenceKey { get; init; } = null!;
    public int CanonicalPosition { get; init; }
    public string AlarmId { get; init; } = null!;
    public Guid SourceMessageId { get; init; }
    public string SourceDecisionId { get; init; } = null!;
}

```

EXECUTABLE ASSET 33-N - EXACT EF CORE CONSTRAINTS

```

cases.ToTable("RootCauseCase", "root_cause");
cases.HasKey(x => x.CaseId);
cases.HasIndex(x => x.SourceFloodKey)
    .HasDatabaseName("UX_RootCauseCase_SourceFlood")
    .IsUnique();
cases.Property(x => x.Version).IsRowVersion();

evidence.ToTable("CaseEvidence", "root_cause");
evidence.HasKey(x => x.EvidenceId);
evidence.Property(x => x.EvidenceKey).HasColumnType("binary(32)");
evidence.HasIndex(x => new { x.CaseId, x.EvidenceKey })
    .HasDatabaseName("UX_CaseEvidence_Meaning").IsUnique();
evidence.HasIndex(x => new { x.CaseId, x.CanonicalPosition })
    .HasDatabaseName("UX_CaseEvidence_Order").IsUnique();
evidence.HasOne<RootCauseCaseRow>().WithMany(x => x.Evidence)
    .HasForeignKey(x => x.CaseId).OnDelete(DeleteBehavior.Restrict);

```

NAMED CONFLICTS 33-Q

Constraint	Meaning	Resolution
PK_InboxReceipt	same transport fact raced	fresh receipt read
UX_RootCause_InboundDecision	same source decision raced	fresh semantic outcome read
UX_RootCauseCase_SourceFlood	same case correlation raced	load winner
UX_Outbox_FactKey	same case-open public fact raced	reuse winner bytes

APPLICATION ORCHESTRATION

One handler creates one complete local decision

EXECUTABLE ASSET 33-O - OWNER-LOCAL HANDLER

```
public sealed class HandleAlarmFloodDetectedV1(
    IRootCauseUnitOfWorkFactory units,
    IRootCauseCaseOpenedFactFactory publicFacts,
    IRootCauseDuplicateResolver duplicates,
    TimeProvider time,
    IRootCauseIdentitySource identities)
{
    public async Task<RootCauseConsumeOutcome> HandleAsync(
        AcceptedAlarmFloodDelivery accepted,
        CancellationToken cancellationToken)
    {
        var command = AlarmFloodTranslation.Translate(
            accepted, RootCauseCasePolicyId.OpenCaseV1);

        await using var unit = await units.CreateAsync(cancellationToken);
        var prior = await unit.TryReadOutcomeAsync(
            accepted.Consumer, accepted.MessageId,
            accepted.EnvelopeSha256, cancellationToken);
        if (prior is not null)
            return prior;

        var existing = await unit.FindBySourceAsync(
            command.Source, cancellationToken);
        if (existing is not null)
            return await unit.CommitSemanticDuplicateAsync(
                accepted, existing, cancellationToken);

        var openedAt = time.GetUtcNow();
        var aggregate = RootCauseCase.OpenFromAlarmFlood(
            identities.NewCaseId(), command, openedAt, identities);
        var outgoing = publicFacts.Create(aggregate);

        unit.Stage(accepted, aggregate, outgoing, openedAt);
        return await CommitFirstAttemptAsync(
            unit, accepted, aggregate, outgoing, cancellationToken);
    }
}
```

STAGE SET 33-R

Tracked addition	Count for golden delivery	Atomic with
InboxReceipt	1	all rows below
InboundPublicFact	1	receipt + case
RootCauseCase	1	evidence + outbox
CaseEvidence	5	case + lineage
OutboxMessage	1	exact RootCauseCaseOpened.v1 bytes

THE ROOTCAUSE TRANSACTION

TX-RC is the only atomicity claim**EXECUTABLE ASSET 33-P - ONE SAVECHANGES BOUNDARY**

```

internal void Stage(
    AcceptedAlarmFloodDelivery accepted,
    RootCauseCase aggregate,
    SerializedPublicFact outgoing,
    DateTimeOffset committedAtUtc)
{
    db.InboxReceipts.Add(Rows.InboxCommitted(
        accepted, "case-opened", committedAtUtc));
    db.InboundPublicFacts.Add(Rows.Inbound(
        accepted, aggregate, "case-opened", committedAtUtc));
    db.RootCauseCases.Add(Rows.Case(aggregate));
    db.CaseEvidence.AddRange(
        aggregate.Evidence.Select(Rows.Evidence));
    db.OutboxMessages.Add(
        OutboxMessageRow.Pending(outgoing, committedAtUtc));
}

internal async Task CommitAsync(CancellationTokentoken cancellationTokentoken)
{
    // One relational SaveChanges: provider transaction or full rollback.
    await db.SaveChangesAsync(cancellationTokentoken);
}

```

TX-RC 33-S - ONE DATABASE, ONE COMMIT**ROLLBACK ORACLE 33-T**

Injected failure	Inbox	Case	Evidence	Outbox	Ack
inbound insert	0	0	0	0	no
third evidence insert	0	0	0	0	no
outbox insert	0	0	0	0	no
successful SaveChanges	1	1	5	1	allowed

NO DISTRIBUTED TRANSACTION

RabbitMQ and AlarmManagementDb never enlist in TX-RC. The incoming bytes are already sufficient; the outgoing fact remains Pending until RootCause's own relay publishes it.

DUPLICATE AND RACE RESOLUTION

The loser reloads the winner in a fresh context

EXECUTABLE ASSET 33-Q - CLASSIFIED CONFLICT RESOLVER

```
private async Task<RootCauseConsumeOutcome> CommitFirstAttemptAsync(
    IRootCauseUnitOfWork unit,
    AcceptedAlarmFloodDelivery accepted,
    RootCauseCase aggregate,
    SerializedPublicFact outgoing,
    CancellationToken cancellationToken)
{
    try
    {
        await unit.CommitAsync(cancellationToken);
        return RootCauseConsumeOutcome.Committed(
            accepted.MessageId, aggregate.Id, outgoing.MessageId);
    }
    catch (DbUpdateException error)
    {
        when (constraints.TryClassify(error, out var conflict))
        {
            await unit.DisposeAsync();
            return await (conflict switch
            {
                RootCauseUniqueConflict.Inbox =>
                    duplicates.ResolveTransportAsync(accepted, cancellationToken),
                RootCauseUniqueConflict.SourceDecision or
                RootCauseUniqueConflict.SourceCase =>
                    duplicates.ResolveSemanticAsync(accepted, cancellationToken),
                RootCauseUniqueConflict.OutboxFactKey =>
                    duplicates.ResolveOutgoingAsync(accepted, cancellationToken),
                _ => throw new UnreachableException()
            });
        }
    }
}
```

TWO-KEY ORACLE 33-U

Delivery pair	Inbox	Inbound	Case	Evidence	Outbox
first message	1	1	1	5	1
same MessageId / same bytes	1	1	1	5	1
new MessageId / same DecisionId	2	1	1	5	1
same MessageId / changed bytes	1 + conflict evidence	1	1	5	1

PROVIDER HONESTY

Only named unique constraints enter the duplicate path. Exception text, SQL error-message language and any generic `DbUpdateException` are not semantic evidence.

AMBIGUOUS DATABASE COMMIT

Unknown means do not acknowledge

A connection can fail after SQL Server commits but before the client observes success. RootCause must resolve by the durable receipt and digest in a new DbContext. It never repeats the effect inside the failed context and never calls the absence of a response a rollback.

EXECUTABLE ASSET 33-R - FRESH-CONTEXT COMMIT RESOLVER

```
public sealed class RootCauseCommitResolver(
    IDbContextFactory<RootCauseDbContext> contexts)
{
    public async Task<RootCauseConsumeOutcome> ResolveAsync(
        AcceptedAlarmFloodDelivery accepted,
        CancellationToken cancellationToken)
    {
        await using var db = await contexts.CreateDbContextAsync(
            cancellationToken);
        var receipt = await db.InboxReceipts.AsNoTracking()
            .SingleOrDefaultAsync(x =>
                x.ConsumerName == accepted.Consumer.Value &&
                x.MessageId == accepted.MessageId,
                cancellationToken);

        if (receipt is null)
            return RootCauseConsumeOutcome.Unknown(accepted.MessageId);
        if (!CryptographicOperations.FixedTimeEquals(
            receipt.EnvelopeSha256, accepted.EnvelopeSha256))
            return RootCauseConsumeOutcome.IdentityConflict(
                accepted.MessageId, "message-id-digest-conflict");

        return RootCauseConsumeOutcome.Duplicate(
            accepted.MessageId,
            new RootCauseCaseId(receipt.RootCauseCaseId),
            receipt.OutcomeCode);
    }
}
```

COMMIT OBSERVATION 33-V

Observation	Outcome	Ack?
SaveChanges returned success	Committed	yes
matching receipt found	Duplicate	yes
receipt absent after response loss	Unknown	no
receipt digest differs	IdentityConflict	no; terminal evidence
resolver database unavailable	Transient	no

FRESH CONTEXT RULE

The failed DbContext is disposed. A resolver never trusts its change tracker, pending entities or exception message to infer what the database committed.

RABBITMQ .NET CONSUMER

Manual acknowledgement, bounded prefetch, sequential first slice

EXECUTABLE ASSET 33-S - RABBITMQ.CLIENT 7.X RUNTIME

```
protected override async Task ExecuteAsync(
    CancellationToken stoppingToken)
{
    _channel = await connection.CreateChannelAsync(
        cancellationTokens: stoppingToken);
    await _channel.BasicQosAsync(
        prefetchSize: 0,
        prefetchCount: options.PrefetchCount,
        global: false,
        stoppingToken);

    var consumer = new AsyncEventingBasicConsumer(_channel);
    consumer.ReceivedAsync += async (_, ea) =>
    {
        var raw = RabbitDeliveryCopy.Copy(ea); // Body.ToArray()
        await using var scope = scopes.CreateAsyncScope();
        var endpoint = scope.ServiceProvider
            .GetRequiredService<IRootCauseAlarmFloodEndpoint>();

        var disposition = await endpoint.HandleAsync(
            raw, stoppingToken);
        await SettleAsync(_channel, ea.DeliveryTag,
            disposition, stoppingToken);
    };

    _consumerTag = await _channel.BasicConsumeAsync(
        queue: "rootcause.alarm-events.v1",
        autoAck: false,
        consumer: consumer,
        cancellationTokens: stoppingToken);
}
```

RUNTIME BOUNDS 33-W

Setting	First-slice value	Reason
ConsumerDispatchConcurrency	1	preserve simple callback/channel ownership
prefetchCount	8	bound unacknowledged memory and DB pressure
autoAck	false	settlement follows local outcome
multiple ack	false	one outcome settles one tag
scope	one DI scope per delivery	one fresh RootCauseDbContext

CURRENT API NOTE

RabbitMQ.Client 7.x requires copying or deserializing `ReadOnlyMemory<byte>` before the callback returns. Consumer callbacks are sequential by default; any later concurrency increase requires provider, ordering and acknowledgement evidence.

DISPOSITION AND SETTLEMENT

Only durable success and matching duplicate are acked

EXECUTABLE ASSET 33-T - OUTCOME-TO-BROKER ACTION

```
internal static DeliveryDisposition For(
    RootCauseConsumeOutcome outcome) => outcome switch
{
    RootCauseConsumeOutcome.Committed => DeliveryDisposition.Ack,
    RootCauseConsumeOutcome.Duplicate => DeliveryDisposition.Ack,
    RootCauseConsumeOutcome.SemanticDuplicate => DeliveryDisposition.Ack,
    RootCauseConsumeOutcome.Transient => DeliveryDisposition.Retry,
    RootCauseConsumeOutcome.Unknown => DeliveryDisposition.Retry,
    RootCauseConsumeOutcome.Invalid => DeliveryDisposition.Reject,
    RootCauseConsumeOutcome.IdentityConflict => DeliveryDisposition.Reject,
    _ => throw new UnreachableException()
};

private static async Task SettleAsync(
    IChannel channel, ulong tag, DeliveryDisposition disposition,
    CancellationToken cancellationToken)
{
    switch (disposition)
    {
        case DeliveryDisposition.Ack:
            await channel.BasicAckAsync(tag, false, cancellationToken);
            break;
        case DeliveryDisposition.Retry:
            await BoundedBackoff.BeforeRequeueAsync(cancellationToken);
            await channel.BasicNackAsync(tag, false, true, cancellationToken);
            break;
        case DeliveryDisposition.Reject:
            await channel.BasicRejectAsync(tag, false, cancellationToken);
            break;
    }
}
```

FAILURE OWNERSHIP 33-X

Failure	Disposition	Bound
invalid contract/auth path	reject; no requeue	dead-letter evidence
identity/digest conflict	reject; security signal	operator-only replay
SQL transient	requeue after local backoff	quorum delivery-limit / Chapter 29 policy
commit unknown	requeue	receipt resolves next attempt
handler defect	classified terminal/retry	never infinite in-memory loop

NO UNBOUNDED REQUEUE

The live queue is a quorum queue with the Chapter 29 delivery-limit and dead-letter evidence policy. Immediate endless requeue is forbidden; a production implementation may substitute the already specified delayed-retry route without changing the inbox rule.

LIFECYCLE AND READINESS

The consumer starts and stops as a bounded runtime role

EXECUTABLE ASSET 33-U - BOUNDED SHUTDOWN

```
public override async Task StopAsync(CancellationTokentoken cancellationToken)
{
    if (_channel is not null && _consumerTag is not null)
        await _channel.BasicCancelAsync(
            _consumerTag, cancellationToken: cancellationToken);

    using var drain = CancellationTokenSource.CreateLinkedTokenSource(
        cancellationToken);
    drain.CancelAfter(options.ShutdownDrainBudget);
    await inFlight.WaitForZeroAsync(drain.Token);

    if (_channel is not null)
        await _channel.CloseAsync(cancellationToken: cancellationToken);
    await base.StopAsync(cancellationToken);
}
```

READINESS MANIFEST 33-Y

Gate	Required evidence	Failure behavior
schema	RootCause migration + model fingerprint	readiness false
contract	Chapter 32 golden body accepted	fail closed
topology	queue/exchange/binding passive checks	consumer not registered
authorization	consumer can read/ack only its queue	startup/readiness failure
policy	prefetch, shutdown, retry limits validated	ValidateOnStart
runtime	consumer tag active + channel open	role not ready

PROBE SEMANTICS 33-Z

Probe	May inspect	Must not do
liveness	process loop and scheduler	query database or broker
readiness	bounded schema/topology/role checks	consume a test business fact
backlog status	queue depth/age and inbox throughput	declare process dead

SHUTDOWN UNCERTAINTY

If the drain budget expires, in-flight unacknowledged deliveries return through broker recovery. The inbox makes their later redelivery safe; shutdown never writes optimistic completion.

TRACE AND CAUSATION

A process span describes the attempt; MessageId explains the business chain

EXECUTABLE ASSET 33-AA - CONSUMER PROCESS SPAN

```
var parent = TraceContext.TryExtract(raw.Headers, out var remote)
? remote.ActivityContext
: default(ActivityContext);

using var activity = Telemetry.ActivitySource.StartActivity(
    "process rootcause.alarm-flood-detected.v1",
    ActivityKind.Consumer,
    parent,
    tags:
    [
        new("messaging.system", "rabbitmq"),
        new("messaging.destination.name", "rootcause.alarm-events.v1"),
        new("messaging.operation.name", "process"),
        new("messaging.operation.type", "process"),
        new("messaging.message.id", accepted.MessageId.ToString("D")),
        new("nexus1.contract", "alarm-flood-detected.v1")
    ]
);

var outcome = await handler.HandleAsync(accepted, cancellationTokens);
activity?.SetTag("nexus1.consume.outcome", outcome.Code);
```

IDENTITY SEPARATION 33-AB

Value	Question	Authority	Retry behavior
MessageId	which inbound fact?	AlarmManagement contract	stable
CorrelationId	which wider workflow?	incoming envelope	preserved
CausationId on CaseOpened	which fact caused opening?	RootCause mapper	inbound MessageId
traceparent	which telemetry attempt lineage?	W3C context	may change/be absent
process span	which handler execution?	consumer runtime	new per attempt

NO TRACE-BASED CORRECTNESS

Sampling, exporter outage or an invalid traceparent may remove telemetry. Inbox identity, source decision, case key, causation and stored bytes still determine correctness.

PRODUCER-PATH AUTHORIZATION**Broker authorization is provenance, not message signing**

RabbitMQ authenticates the publishing connection and enforces write permissions. A normal delivery does not carry a broker-attested publisher principal for the consumer. Phase-0 therefore combines a restricted exchange/vhost route with strict payload and property checks. Appld and the producer field are corroborating claims, not cryptographic proof by themselves.

TRUST MODEL 33-AC

Control	What it proves	What it does not prove
publisher credential	broker authenticated one connection	identity inside stored body
write ACL	only approved principal may write nexus.events	per-message non-repudiation
route/binding	RootCause queue receives approved coordinate	payload integrity alone
JCS fingerprint + body digest	body meaning/bytes unchanged	who produced bytes
Appld/producer equality	contract claims agree	broker-attested publisher identity

EXECUTABLE ASSET 33-AD - AUTHORIZATION EVIDENCE TESTS

```
[Fact]
public async Task RC_AUTH_001_alarm_principal_can_publish_route()
{
    await topology.PublishGoldenAsAsync("alarm-management-relay");
    await topology.AssertQueueContainsAsync(
        "rootcause.alarm-events.v1", Golden.MessageId);
}

[Theory]
[InlineData("root-cause-consumer")]
[InlineData("reporting-reader")]
[InlineData("anonymous")]
public async Task RC_AUTH_002_other_principals_cannot_write_route(
    string principal)
{
    var act = () => topology.PublishGoldenAsAsync(principal);
    await act.Should().ThrowAsync<BrokerAuthorizationFailure>();
    await topology.AssertQueueDoesNotContainAsync(Golden.MessageId);
}
```

UPGRADE PATH

If later threat analysis requires end-to-end producer authentication independent of broker ACLs, introduce a versioned signature profile and key-rotation evidence. Do not retrospectively claim that Appld already provides it.

NO ALARMMANAGEMENT DATABASE DEPENDENCY

The public fact is sufficient or the contract is wrong

EXECUTABLE ASSET 33-AE - ARCHITECTURE AND CONFIGURATION GATES

```
[Fact]
public void RC_ARCH_001_has_no_alarm_management_data_reference()
{
    var rootCauseAssemblies = Assemblies.RootCauseRuntime;
    rootCauseAssemblies.Should().NotReference(
        "Nexus1.AlarmManagement.Infrastructure");
    rootCauseAssemblies.Should().NotContainTypeNamed(
        "AlarmManagementDbContext");
}

[Fact]
public void RC_CFG_001_runtime_has_one_owner_connection()
{
    var manifest = RootCauseRuntimeManifest.From(configuration);
    manifest.DatabaseAuthorities.Should().Equal("RootCauseDb");
    manifest.ConnectionSettingNames.Should().Equal("RootCauseDb");
}

[Fact]
public async Task RC_NET_001_alarm_database_is_unreachable()
{
    await networkPolicy.AssertDeniedAsync(
        principal: "root-cause-runtime",
        destination: "alarm-management-db:1433");
}
```

DATA BOUNDARY 33-AF

Need	Source	Forbidden substitute
decision identity	AlarmFloodDetected.v1 DecisionId	query producer table
alarm references	ordered public alarm list	shared Alarm entity
window and line	public payload	cross-database join
producer policy lineage	public PolicyId	read detector configuration
case state/evidence	RootCauseDb	write producer status

CONTRACT INSUFFICIENCY

If RootCause cannot make the opening decision from the accepted V1 plus its own policy, the remedy is a reviewed public contract or explicit synchronous API. Direct database access is never the fallback.

CRASH WINDOWS

Every interruption has one observable recovery

CRASH MATRIX 33-AG

Checkpoint	RootCauseDb	Broker state	Recovery
before body copy	no rows	unacked delivery	redelivery
after admission	no rows	unacked delivery	repeat admission
after Domain decision	no rows	unacked delivery	recreate deterministic decision
during SaveChanges	all or none	unacked delivery	receipt resolver / redelivery
after commit, before ack	receipt + case + evidence + outbox	unacked	duplicate no-op then ack
during ack write	all rows durable	ack outcome unknown	redelivery is harmless
after ack accepted	all rows durable	delivery removed	normal continuation

EXECUTABLE ASSET 33-AH - DETERMINISTIC FAULT POINTS

```
public enum RootCauseConsumerCheckpoint
{
    BodyCopied,
    ContractAdmitted,
    CommandTranslated,
    DomainOpened,
    SaveStarted,
    SaveCommitted,
    AckStarted,
    AckCompleted
}

public interface IRootCauseConsumerFaultPoint
{
    ValueTask ReachedAsync(
        RootCauseConsumerCheckpoint checkpoint,
        Guid messageId,
        CancellationToken cancellationToken);
}
```

THE UNAVOIDABLE WINDOW

SaveCommitted -> AckCompleted cannot be closed by a local transaction. The inbox turns that uncertainty into a matching Duplicate outcome; it does not remove broker redelivery.

CRASH AND RESTART HARNESS

Kill the process, then compare durable truth

EXECUTABLE ASSET 33-AI - COMMIT-BEFORE-ACK RECOVERY

```
[Fact]
public async Task RC_CRASH_001_commit_then_process_death_is_safe()
{
    var published = await fixture.PublishChapter32GoldenAsync();
    fixture.Faults.KillAt(
        RootCauseConsumerCheckpoint.SaveCommitted);

    await fixture.StartRootCauseAsync();
    await fixture.WaitForProcessExitAsync();
    fixture.SuccessfulAcks.Should().Be(0);

    await fixture.RestartRootCauseAsync();
    await fixture.WaitUntilQueueEmptyAsync();

    var state = await fixture.RootCauseDb.ReadGoldenSnapshotAsync();
    state.InboxReceipts.Should().Be(1);
    state.InboundFacts.Should().Be(1);
    state.Cases.Should().Be(1);
    state.Evidence.Should().Be(5);
    state.CaseOpenedOutboxRows.Should().Be(1);
    state.CaseOpenedBodies.Should().OnlyContain(
        bytes => bytes.SequenceEqual(published.ExpectedOpenedBytes));
    fixture.ObservedDeliveries.Should().Be(2);
    fixture.SuccessfulAcks.Should().Be(1);
}

[Theory]
[InlineData(RootCauseConsumerCheckpoint.BodyCopied)]
[InlineData(RootCauseConsumerCheckpoint.DomainOpened)]
[InlineData(RootCauseConsumerCheckpoint.SaveStarted)]
public async Task RC_CRASH_002_precommit_death_replays_once(
    RootCauseConsumerCheckpoint checkpoint)
{
    await fixture.AssertRestartConvergesAsync(checkpoint,
        expectedCases: 1, expectedEvidence: 5, expectedOutbox: 1);
}
```

HARNESS EVIDENCE 33-AJ

Capture	Required content	Failure
process record	image/build/config identity + kill checkpoint	different untracked build
database snapshot	row counts, keys, digests, constraints	mutable inspection only
broker capture	delivery count, redelivered flag, ack outcome	queue state inferred
contract artifact	input and output body SHA-256	re-serialized comparison

PROVIDER-BACKED COMPONENT TESTS

The real provider must prove the atomic row set

EXECUTABLE ASSET 33-AK - FIRST DELIVERY AND ROLLBACK

```
[Fact]
public async Task RC_DB_001_first_delivery_commits_complete_set()
{
    var delivery = fixture.AcceptChapter32Golden();
    var outcome = await fixture.ConsumeInNewScopeAsync(delivery);

    outcome.Should().BeOfType<RootCauseConsumeOutcome.Committed>();
    var counts = await fixture.ReadCountsAsync();
    counts.Should().Be(new(
        Inbox: 1, Inbound: 1, Cases: 1,
        Evidence: 5, CaseOpenedOutbox: 1));
}

[Fact]
public async Task RC_DB_002_outbox_failure_rolls_back_everything()
{
    fixture.SqlFaults.FailCommand(
        "INSERT INTO [messaging].[OutboxMessage]");

    var act = () => fixture.ConsumeInNewScopeAsync(
        fixture.AcceptChapter32Golden());
    await act.Should().ThrowAsync<InjectedDatabaseFailure>();

    (await fixture.ReadCountsAsync()).Should().Be(
        new(Inbox: 0, Inbound: 0, Cases: 0,
            Evidence: 0, CaseOpenedOutbox: 0));
    fixture.Acknowledgements.Should().BeEmpty();
}
```

EXECUTABLE ASSET 33-AL - CONCURRENT DUPLICATE ORACLE

```
[Fact]
public async Task RC_DB_003_concurrent_delivery_has_one_effect()
{
    var delivery = fixture.AcceptChapter32Golden();
    var barrier = new CommitBarrier(2);

    var outcomes = await Task.WhenAll(
        fixture.ConsumeInNewScopeAsync(delivery, barrier),
        fixture.ConsumeInNewScopeAsync(delivery, barrier));

    outcomes.Should().ContainSingle(x => x is RootCauseConsumeOutcome.Committed);
    outcomes.Should().ContainSingle(x => x is RootCauseConsumeOutcome.Duplicate);
    (await fixture.ReadCountsAsync()).Should().Be(
        new(1, 1, 1, 5, 1));
}
```

PROVIDER SELECTION

The suite runs against the exact supported production provider and migration artifact. SQLite or EF InMemory cannot prove SQL Server constraint names, rowversion behavior or transaction failure semantics.

SEMANTIC DUPLICATES

A new MessageId cannot create a second case for one DecisionId

EXECUTABLE ASSET 33-AM - TRANSPORT AND SEMANTIC IDENTITIES

```
[Fact]
public async Task RC_DB_004_new_message_same_decision_is_no_op()
{
    var first = fixture.AcceptChapter32Golden();
    var second = first.WithNewMessageIdAndValidFingerprint();

    await fixture.ConsumeInNewScopeAsync(first);
    var outcome = await fixture.ConsumeInNewScopeAsync(second);

    outcome.Should().BeOfType<
        RootCauseConsumeOutcome.SemanticDuplicate>();
    var counts = await fixture.ReadCountsAsync();
    counts.Inbox.Should().Be(2);
    counts.Inbound.Should().Be(1);
    counts.Cases.Should().Be(1);
    counts.Evidence.Should().Be(5);
    counts.CaseOpenedOutbox.Should().Be(1);
}

[Fact]
public async Task RC_DB_005_same_id_changed_body_is_terminal()
{
    await fixture.ConsumeInNewScopeAsync(
        fixture.AcceptChapter32Golden());

    var conflict = fixture.GoldenWithChangedAlarmAndOldMessageId();
    var outcome = await fixture.ConsumeRawInNewScopeAsync(conflict);

    outcome.Should().BeOfType<RootCauseConsumeOutcome.IdentityConflict>();
    (await fixture.ReadCountsAsync()).Should().Be(new(1, 1, 1, 5, 1));
    fixture.SecurityEvidence.Should().ContainSingle(x =>
        x.Code == "message-id-digest-conflict");
}
```

SEMANTIC NO-OP RECEIPT 33-AN

Stored field	Value
ConsumerName + second MessageId	new InboxReceipt
OutcomeCode	case-already-open
RootCauseCaseId	existing winner CaseId
EnvelopeSha256	second accepted body digest
new Domain fact / outbox	none

BUSINESS NO-OP IS STILL DURABLE

The second transport identity is acknowledged only after its no-op receipt commits. A later redelivery then avoids even the semantic lookup.

BROKER-BACKED TESTS

RabbitMQ redelivery must converge with RootCauseDb

EXECUTABLE ASSET 33-AO - LIVE BROKER DUPLICATE

```
[Fact]
public async Task RC_RM0_001_duplicate_delivery_opens_one_case()
{
    await fixture.PublishChapter32GoldenAsync(times: 2);
    await fixture.StartRootCauseAsync();
    await fixture.WaitUntilQueueEmptyAsync();

    fixture.ObservedDeliveries.Should().Be(2);
    fixture.SuccessfulAcks.Should().Be(2);
    (await fixture.RootCauseDb.ReadCountsAsync())
        .Should().Be(new(1, 1, 1, 5, 1));
}

[Fact]
public async Task RC_RM0_002_lost_ack_redelivers_without_new_effect()
{
    fixture.ChannelFaults.DropAfterDatabaseCommitBeforeAck();
    await fixture.PublishChapter32GoldenAsync();

    await fixture.StartRootCauseAsync();
    await fixture.WaitForChannelLossAsync();
    await fixture.RestartRootCauseAsync();
    await fixture.WaitUntilQueueEmptyAsync();

    fixture.ObservedRedelivered.Should().Contain(true);
    (await fixture.RootCauseDb.ReadCountsAsync())
        .Should().Be(new(1, 1, 1, 5, 1));
}

[Fact]
public async Task RC_RM0_003_invalid_body_never_reaches_domain()
{
    await fixture.PublishRawAsync(Corpus.FingerprintMismatch);
    await fixture.WaitForDeadLetterAsync("fingerprint-mismatch");
    fixture.DomainInvocations.Should().Be(0);
    (await fixture.RootCauseDb.ReadBusinessCountsAsync()).Should().BeZero();
}
```

LIVE-LAB IDENTITIES 33-AP

Component	Pinned evidence
RabbitMQ	image digest, version, vhost, queue type, delivery-limit
SQL Server	image/version, collation, migration id, model hash
RootCause	build SHA, packages, configuration digest
corpus	input/output filenames, byte length, SHA-256

OUTGOING CONTRACT AND DISPATCH HANDOFF

The local transaction ends at Pending

RootCauseCaseOpened.v1 enters RootCause's outbox in TX-RC. The RootCause relay later applies the same mandatory-route, publisher-confirm and stable-byte discipline proved by Chapter 32. The consumer callback never publishes directly.

EXECUTABLE ASSET 33-AQ - OUTGOING PRODUCER GATE

```
[Fact]
public async Task RC_OUTBOX_001_committed_case_has_exact_pending_fact()
{
    await fixture.ConsumeInNewScopeAsync(
        fixture.AcceptChapter32Golden());

    var row = await fixture.RootCauseDb.SingleOutboxAsync(
        "root-cause-case-opened-v1|" + Expected.CaseId);

    row.State.Should().Be(OutboxState.Pending);
    row.EventType.Should().Be(
        "nexus1.root-cause.root-cause-case-opened.v1");
    row.RoutingKey.Should().Be(
        "root-cause.root-cause-case-opened.v1");
    row.CausationId.Should().Be(
        "019bc4d4-5dc1-7b8d-9d48-542d25315356");
    row.EnvelopeBytes.Should().Equal(Corpus.CaseOpenedV1.GoldenBytes);
}

[Fact]
public async Task RC_OUTBOX_002_consumer_does_not_publish_inline()
{
    await fixture.ConsumeInNewScopeAsync(
        fixture.AcceptChapter32Golden());
    fixture.Broker.PublishesByConsumerRole.Should().BeEmpty();
}
```

PRODUCER/CONSUMER COMPATIBILITY 33-AR

Gate	Required result
AlarmManagement -> RootCause	Chapter 32 golden bytes accepted by Chapter 33 reader
RootCause internal -> V1	explicit mapper and golden byte corpus
RootCause -> approved consumers	every supported V1 reader accepts exact golden body
route manifest	at least one intentional binding or explicit no-subscriber decision
breaking change	new event/schema version; never mutate stored V1 bytes

NO VERDICT CLAIM

CaseOpened says that RootCause owns an open case with initial evidence. RootCauseVerdictIssued.v1 remains a different future fact with different authorization, policy and evidence membership.

OPERATIONAL EVIDENCE

Observe processing without leaking evidence

EXECUTABLE ASSET 33-AS - SAFE STRUCTURED LOG

```
logger.LogInformation(  
    "RootCause delivery completed " +  
    "message_id={MessageId} event_type={EventType} " +  
    "outcome={Outcome} case_id={CaseId} " +  
    "redelivered={Redelivered} duration_ms={DurationMs}",  
    accepted.MessageId,  
    accepted.EventType,  
    outcome.Code,  
    outcome.CaseId,  
    raw.Redelivered,  
    elapsed.TotalMilliseconds);
```

SIGNAL INVENTORY 33-AT

Signal	Safe dimensions	Forbidden labels/values
consume_total	contract, outcome	MessageId, CaseId, SiteId
consume_duration	contract, outcome	correlation, decision, alarm
duplicate_total	duplicate class	source identifiers
contract_rejected_total	stable rejection code	body or payload fields
inflight	consumer role	delivery tag
case_opened_total	policy version	case/evidence identity

BACKLOG AND SLO EVIDENCE 33-AU

Observation	Meaning	Not inferred
queue ready/unacked	transport work waiting/in progress	cases missing
oldest message age	consumer lag pressure	service dead
receipt/case rate	local commit throughput	broker settlement success
outbox Pending age	next public intent delayed	case did not open

CARDINALITY GATE

MessageId and CaseId may appear in bounded logs and traces for investigation, never as metric labels. Alarm IDs and evidence values stay out of ordinary telemetry.

RETENTION AND REPLAY

Deduplication lasts only as long as its evidence

RETENTION CONTRACT 33-AV

Artifact	Minimum horizon	Deletion prerequisite
InboxReceipt	max producer replay + broker retention + operator margin	case/outcome lineage retained
InboundPublicFact	RootCause case/audit lifetime	explicit archival policy
EnvelopeSha256	same as inbox identity	collision/conflict evidence preserved
Case and evidence	domain retention policy	authorized archival, never silent purge
CaseOpened outbox	dispatch + replay policy	durable dispatch/retirement evidence

EXECUTABLE ASSET 33-AW - AUTHORIZED REPLAY INVARIANT

```
public AuthorizedConsumerReplay RequestReplay(  
    DeadLetterEvidence original,  
    ReplayAuthorization authorization,  
    DateTimeOffset requestedAtUtc)  
{  
    authorization.Demand("root-cause:alarm-flood:replay");  
    if (!original.HasExactBody || !original.IsReplayEligible)  
        throw new ReplayRejected("evidence-incomplete");  
  
    return new(  
        OriginalMessageId: original.MessageId,  
        OriginalEnvelopeBytes: original.EnvelopeBytes.ToArray(),  
        OriginalEnvelopeSha256: original.EnvelopeSha256.ToArray(),  
        ReplayOperationId: replayIds.New(),  
        RequestedByDigest: authorization.SubjectDigest,  
        RequestedAtUtc: requestedAtUtc);  
}
```

REPLAY OUTCOMES 33-AX

Existing state	Expected result
matching inbox receipt	Duplicate; no case/evidence/outbox change; ack
semantic case exists under new MessageId	commit no-op receipt; ack
no receipt/case	normal first processing
same MessageId, different digest	terminal conflict; never overwrite
evidence expired	replay rejected; no guessed reconstruction

REPLAY IS NEW WORK, NOT NEW HISTORY

Authorization, audit identity and process span are new. The original inbound MessageId and exact body remain unchanged; any existing RootCause business effect remains singular.

REPRODUCIBLE EVIDENCE PACKAGE

The chapter exits only through a fail-closed manifest

EXECUTION SEQUENCE 33-AY

Step	Required evidence
apply RootCause migration	schema head + model fingerprint
provision queue and ACLs	topology and permission manifest
run contract corpus	Chapter 32 input accepted; rejected mutations stable
consume with broker stopped/restarted	durable local commit independent of relay
kill after RootCause commit	redelivery + matching duplicate + one effect
inject outbox insert fault	complete rollback and no ack
dispatch CaseOpened fact	exact stored output bytes and route/confirm evidence

EXECUTABLE ASSET 33-AZ - EVIDENCE MANIFEST

```
{
  "chapter": 33,
  "status": "PENDING",
  "input": {
    "eventType": "nexus1.alarm-management.alarm-flood-detected.v1",
    "bodySha256": "7cf98319c20a5915d5710bbcbb5878fe51cdae89a5b68081c135fd0e962c276a"
  },
  "output": {
    "eventType": "nexus1.root-cause.root-cause-case-opened.v1",
    "bodySha256": "454f51cf4b0250b9c855346a6648a7d73530f19687ee3b52cda2edf9d0339c5b"
  },
  "requiredArtifacts": [
    "rootcause-contract-corpus.trx",
    "sqlserver-atomicity-and-races.trx",
    "rabbitmq-redelivery-and-ack.trx",
    "rootcause-crash-window.trx",
    "topology-and-acl-manifest.json",
    "trace-metrics-log-snapshot.json"
  ]
}
```

FAIL-CLOSED VALIDATION 33-BA

Missing/drifted evidence	Result
zero discovered tests	reject manifest
input/output digest differs	contract drift
provider/broker identity absent	environment unbound
crash checkpoint skipped	recovery claim unproved
artifact older than build	stale evidence

ADR, PROOF BOUNDARY, AND CHECKPOINT

The consumer owns truth only after TX-RC

ADR-031 33-BB

Field	Decision
Status	Accepted for implementation; runtime evidence PENDING
Decision	strict admission -> local translation -> one RootCauseDb commit -> ack
Idempotency	InboxReceipt for transport + SourceFloodKey for semantic effect
Outgoing fact	exact RootCauseCaseOpened.v1 bytes staged in TX-RC
Rejected	cross-db reads, inline publish, auto-ack, in-memory dedup, trace identity
Evidence	contract, provider, broker, crash, authorization and telemetry suites

OFFICIAL TECHNICAL ANCHORS

- RabbitMQ consumer acknowledgements and publisher confirms: <https://www.rabbitmq.com/docs/confirmations>
- RabbitMQ consumer prefetch: <https://www.rabbitmq.com/docs/consumer-prefetch>
- RabbitMQ .NET client API guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>
- RabbitMQ.Client IChannel API:
<https://rabbitmq.github.io/rabbitmq-dotnet-client/api/RabbitMQ.Client.IChannel.html>
- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- EF Core optimistic concurrency: <https://learn.microsoft.com/ef/core/saving/concurrency>
- OpenTelemetry messaging spans:
<https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/>
- RFC 8785 JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785>

WHAT THIS CHAPTER HAS PROVED

Chapter 33 has specified exact input admission, producer-path trust, consumer-owned translation, one case with five initial evidence items, two-layer idempotency, atomic Inbox/lineage/case/evidence/outbox persistence, commit ambiguity recovery, manual acknowledgement, crash tests, exact RootCauseCaseOpened.v1 bytes and fail-closed evidence gates. It has not executed those artifacts, delivered CaseOpened, issued a verdict or updated Audit, Compliance, Reporting or the console.

NEXT - CHAPTER 34

A Verdict Fans Out to Audit and Compliance will begin after RootCause policy issues RootCauseVerdictIssued.v1. Audit will append evidence and Compliance will open its own review independently; either subscriber may be offline without rolling back RootCause truth.

CHAPTER CHECKPOINT

- Why are InboxReceipt and SourceFloodKey both required?
- Which exact rows enter TX-RC for the golden delivery?
- Why may a matching duplicate be acknowledged?
- What happens after commit succeeds but acknowledgement is lost?
- Why is broker ACL provenance weaker than per-message signing?
- Which fields define RootCauseCaseOpened.v1 and what does it deliberately omit?
- What evidence must exist before status can change from PENDING?

PART VI - THE FIRST END-TO-END DISTRIBUTED SLICE

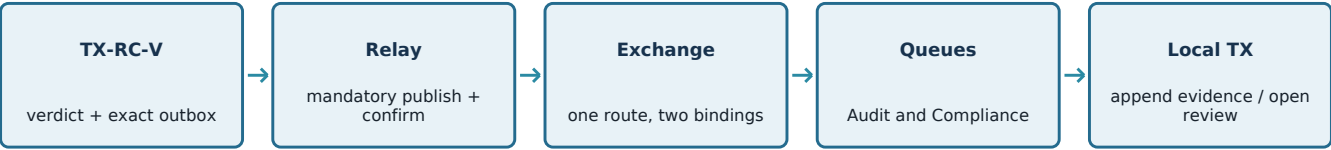
34. A Verdict Fans Out to Audit and Compliance

RootCause commits one verdict and one exact public intent. Audit and Compliance then advance independently, each under its own inbox, database and authority.

CHAPTER DECISION

RootCause closes the Chapter 33 case with a deterministic, evidence-bound verdict and stages RootCauseVerdictIssued.v1 in RootCauseDb. The relay publishes one immutable body to two durable queues. Audit appends evidence in AuditDb; Compliance opens one review in ComplianceDb. No subscriber participates in the RootCause transaction, and neither subscriber can roll it back.

PROOF PATH 34-A - ONE FACT, TWO INDEPENDENT COMMITMENTS



GUARANTEE LEDGER 34-A

Boundary	Allowed claim	Still forbidden
RootCause commit	verdict truth and public intent durable	subscriber completion
broker confirm	broker accepted the routed publication	all bindings or consumer commit
Audit receipt	immutable evidence appended once	Compliance progress
Compliance receipt	one local review opened	Audit progress

THE FAILURE IN THE NAIVE DESIGN

A distributed transaction would erase the boundary

The tempting implementation closes the RootCause case, calls Audit and Compliance, and commits only if every call succeeds. That turns temporary subscriber availability into a precondition for the producer's own truth. It also mixes four authorities inside one failure domain.

REJECTED SHAPE 34-B - SYNCHRONOUS FAN-OUT DISGUISED AS SAFETY

```
await rootCause.CloseCaseAsync(caseId, verdict, ct);
await audit.AppendVerdictAsync(caseId, verdict, ct);
await compliance.OpenReviewAsync(caseId, verdict, ct);
await rootCause.CommitAsync(ct); // impossible global promise
```

WHY THE SHAPE IS REJECTED 34-C

Defect	Observed consequence	Correct boundary
subscriber outage	RootCause cannot preserve its own decision	owner-local TX-RC-V
timeout	caller cannot know which remote writes committed	durable facts + inboxes
retry	remote effects may repeat	transport and semantic keys
authorization	producer gains downstream mutation power	consumer-owned handlers
schema coupling	one deployment must understand private models	producer-owned V1

CONSISTENCY BOUNDARY

Strong consistency applies inside each owner-local database transaction. Eventual consistency begins when RootCause truth is durable and the other owners have not yet observed it. This interval is expected state, not corruption.

INHERITED STATE

Chapter 34 starts from one exact open case

The scenario does not invent another case or another alarm flood. It reuses the exact identities fixed by Chapters 32 and 33, then adds one authorized RootCause decision.

GOLDEN CONTINUITY LEDGER 34-D

Coordinate	Exact value	Authority
AlarmFlood Messageld	019bc4d4-5dc1-7b8d-9d48-542d25315356	AlarmManagement
RootCauseCaseld	019bc4d4-6dfe-7bcb-9de8-3af71979285a	RootCause
CaseOpened Messageld	019bc4d4-6ea0-7d14-a57a-3e8643c55f13	RootCause
VerdictId	019bc4d4-81c0-7ea1-82d5-4f9b8719d0a1	RootCause
Verdict public Messageld	019bc4d4-82a4-77f2-ae06-09b172aa59db	RootCause
correlation	request-am-20482	original public flow
decision causation	decision-rc-7184	RootCause Application

PRECONDITION 34-E

RootCause state	Required observation
case	Open; Caseld matches Chapter 33
evidence	five admitted items in canonical position 0..4
policy	root-cause/verdict/3 is enabled for this case scope
prior verdict	none; producer stream position 1 is CaseOpened
consumer state	Audit and Compliance may both be absent or offline

CAUSATION IS NOT CHRONOLOGY

CaseOpened Messageld is the previous public position, but the direct cause of VerdictIssued is the authorized RootCause decision identified by decision-rc-7184. The stream position preserves order; causation preserves the initiating action.

ROOTCAUSE AUTHORITY

Only RootCause may issue the verdict

EXECUTABLE ASSET 34-F - ISSUE COMMAND AND AUTHORIZATION

```
public sealed record IssueRootCauseVerdict(
    RootCauseCaseId CaseId,
    RootCauseVerdictId VerdictId,
    Guid PublicFactMessageId,
    RootCauseCode Cause,
    ConfidenceScore Confidence,
    PolicyIdentity Policy,
    DecisionIdentity DecisionId,
    ActorIdentity Actor);

public sealed class IssueVerdictHandler(
    IRootCauseCaseRepository cases,
    IRootCauseDecisionStore decisions,
    IRootCauseAuthorization authorization,
    TimeProvider clock)
{
    public async Task<IssueVerdictOutcome> HandleAsync(
        IssueRootCauseVerdict command, CancellationToken ct)
    {
        authorization.Demand(command.Actor, "root-cause:verdict:issue");
        var aggregate = await cases.GetRequiredAsync(command.CaseId, ct);
        var fact = aggregate.IssueVerdict(command, clock.GetUtcNow());
        var result = IssueVerdictResult.From(aggregate, fact, command);
        decisions.Stage(result);
        await decisions.CommitAsync(ct);
        return new IssueVerdictOutcome.Issued(result.Verdict.Id);
    }
}
```

AUTHORITY EXCLUSIONS 34-G

Principal	May	Must not
RootCause operator/policy	issue under approved capability	write AuditDb or ComplianceDb
Audit consumer	record accepted public evidence	approve/correct the verdict
Compliance consumer	open its own review	recalculate RootCause meaning
broker relay	publish stored bytes	create a verdict

DETERMINISTIC DECISION

Verdict meaning excludes delivery and wall-clock noise

EXECUTABLE ASSET 34-H - CANONICAL DECISION BASIS

```
public sealed record VerdictBasis(
    RootCauseCaseId CaseId,
    RootCauseCode Cause,
    decimal Confidence,
    PolicyIdentity PolicyIdentity,
    int EvidenceRevision,
    IReadOnlyList<EvidenceKey> OrderedEvidence);

public static VerdictIdentity Identify(VerdictBasis basis)
{
    var canonical = CanonicalJson.Serialize(new {
        rootCauseCaseId = basis.CaseId.Value,
        verdictCode = basis.Cause.Value,
        confidence = basis.Confidence,
        policyIdentity = basis.PolicyIdentity.Value,
        evidenceRevision = basis.EvidenceRevision,
        evidenceKeys = basis.OrderedEvidence.Select(x => x.Hex)
    });
    return VerdictIdentity.Sha256(canonical);
}
```

DECISION INPUT BOUNDARY 34-I

Included	Excluded
case identity + approved policy	broker attempt/redelivered flag
cause code + normalized confidence	trace/span identifiers
ordered evidence membership + revision	consumer availability
stable policy meaning	database row version and processing time

REPLAY CLAIM

The same case, approved policy and ordered evidence must produce the same VerdictIdentity. VerdictId and IssuedAtUtc identify the committed occurrence; they are allocated once and reloaded on retry.

AGGREGATE TRANSITION

Evidence membership is frozen with the verdict

EXECUTABLE ASSET 34-J - OWNER-LOCAL INVARIANT

```
public VerdictIssued IssueVerdict(
    IssueRootCauseVerdict command,
    DateTimeOffset issuedAtUtc)
{
    if (Status != RootCauseCaseStatus.Open)
        throw DomainRuleViolation.Create("RC-INV-001");
    if (_evidence.Count == 0)
        throw DomainRuleViolation.Create("RC-INV-002");
    if (!PolicyIdentity.Equals(command.Policy))
        throw DomainRuleViolation.Create("RC-POL-001");

    var ordered = _evidence
        .OrderBy(x => x.CanonicalPosition)
        .Select(x => x.Key).ToArray();
    var basis = VerdictBasis.Create(
        Id, command.Cause, command.Confidence,
        command.Policy, EvidenceRevision, ordered);
    var verdict = RootCauseVerdict.Issue(
        command.VerdictId, basis, issuedAtUtc);

    _verdicts.Add(verdict);
    Status = RootCauseCaseStatus.Closed;
    return new VerdictIssued(Id, verdict, command.DecisionId);
}
```

GOLDEN DECISION 34-K

Field	Value
cause	COMMON_POWER_SUPPLY_INSTABILITY
confidence	0.94; dimensionless public value
policy	root-cause/verdict/3
evidence revision/count	1 / 5
evidence membership SHA-256	af3f3ad8da7edfaa50d14ebf422d424e2dd4e49767a2f4a056ce7682a801b5b7
verdict identity SHA-256	a2451a72990d5933e175396c86fd96854ff1db0e6571e0ae85afc37fb1734c17

TX-RC-V

Verdict and public intent commit together

EXECUTABLE ASSET 34-L - ONE ROOTCAUSEDB SAVECHANGES

```
internal sealed class EfRootCauseVerdictStore(
    RootCauseDbContext db,
    IRootCauseVerdictFactFactory facts) : IRootCauseDecisionStore
{
    public void Stage(IssueVerdictResult result)
    {
        db.Cases.Update(Rows.CaseState(result.Case));
        db.Verdicts.Add(Rows.Verdict(result.Verdict));
        db.VerdictEvidence.AddRange(
            result.Verdict.OrderedEvidence.Select(
                (key, position) => Rows.Membership(
                    result.Verdict.Id, key, position)));
        db.OutboxMessages.Add(
            facts.RootCauseVerdictIssuedV1(result).ToPendingRow());
    }

    public Task CommitAsync(CancellationTokens ct) =>
        db.SaveChangesAsync(ct);
}
```

TX-RC-V ROW ORACLE 34-M

Owner table	Golden delta	Failure rule
root_cause.RootCauseCase	state Open -> Closed	roll back
root_cause.CaseVerdict	+1 immutable verdict	roll back
root_cause.VerdictEvidence	+5 ordered memberships	roll back
messaging.OutboxMessage	+1 exact V1 body	roll back
AuditDb / ComplianceDb	no connection	outside transaction

PROVIDER BOUNDARY

One SaveChanges is atomic only inside RootCauseDb when the selected relational provider supports transactions. No TransactionScope, linked DbContext or broker operation joins TX-RC-V.

NORMATIVE PUBLIC CONTRACT

RootCauseVerdictIssued.v1 carries decision evidence, not private state

EXECUTABLE ASSET 34-N - EXACT V1 PAYLOAD

```
namespace RootCause.Contracts.Integration.V1;

[JsonUnmappedMemberHandling(JsonUnmappedMemberHandling.Disallow)]
public sealed record RootCauseVerdictIssuedV1(
    [property: JsonRequired] string VerdictId,
    [property: JsonRequired] string RootCauseCaseId,
    [property: JsonRequired] string SourceAlarmFloodMessageId,
    [property: JsonRequired] string SiteId,
    [property: JsonRequired] string LineId,
    [property: JsonRequired] string VerdictCode,
    [property: JsonRequired] decimal Confidence,
    [property: JsonRequired] string PolicyIdentity,
    [property: JsonRequired] string VerdictIdentity,
    [property: JsonRequired] int EvidenceRevision,
    [property: JsonRequired] int EvidenceCount,
    [property: JsonRequired] string EvidenceMembershipSha256,
    [property: JsonRequired] DateTimeOffset IssuedAtUtc,
    [property: JsonRequired] ProducerStreamPositionV1 ProducerStream);
```

RESOLUTION OF EARLIER REFERENCE SHAPES 34-O

Earlier illustration	Chapter 34 binding decision
Caseld / EventId	RootCauseCaseld / VerdictId plus envelope Messageld
UnitId placeholder	Siteld + LineId from the frozen Chapter 32 flow
EvidenceRevision only	revision, count and membership digest
internal verdict object	never serialized; explicit V1 mapper

CONTRACT COORDINATES

One transition owns one identity and one route

EXECUTABLE ASSET 34-P - DESCRIPTOR AND MAPPER

```
internal static PublicFactDescriptor Describe(IssueVerdictResult source) => new(
    MessageId: source.PublicFactMessageId,
    FactKey: $"root-cause-verdict-issued-v1|{source.Verdict.Id:D}",
    Producer: "root-cause",
    EventType: "nexus1.root-cause.root-cause-verdict-issued.v1",
    SchemaVersion: 1,
    RoutingKey: "root-cause.root-cause-verdict-issued.v1",
    OccurredAtUtc: source.Verdict.IssuedAtUtc,
    CorrelationId: source.CorrelationId,
    CausationId: source.DecisionId.Value);

internal static RootCauseVerdictIssuedV1 Map(IssueVerdictResult source) =>
    new(source.Verdict.Id.Value.ToString("D"),
        source.CaseId.Value.ToString("D"), source.SourceFloodMessageId,
        source.SiteId.Value, source.LineId.Value,
        source.Verdict.Cause.Value, source.Verdict.Confidence.Value,
        source.Verdict.PolicyIdentity.Value,
        source.Verdict.Identity.ToString(), source.EvidenceRevision,
        source.Verdict.OrderedEvidence.Count,
        source.Verdict.EvidenceMembershipSha256,
        source.Verdict.IssuedAtUtc,
        new($"root-cause/case/{source.CaseId.Value:D}", 2));
```

FROZEN COORDINATES 34-Q

Coordinate	Exact value
producer	root-cause
event type	nexus1.root-cause.root-cause-verdict-issued.v1
routing key	root-cause.root-cause-verdict-issued.v1
schema / envelope	1 / 1
fingerprint	sha-256+jcs
producer stream	root-cause/case/019bc4d4-6dfe-7bcb-9de8-3af71979285a @ 2

GOLDEN PUBLIC FACT

The same exact body reaches both subscribers

EXECUTABLE ASSET 34-R - READABLE GOLDEN INSTANCE

```
{
  "envelopeVersion": 1,
  "messageId": "019bc4d4-82a4-77f2-ae06-09b172aa59db",
  "eventType": "nexus1.root-cause.root-cause-verdict-issued.v1",
  "schemaVersion": 1,
  "occurredAtUtc": "2026-07-19T07:47:38.442Z",
  "producer": "root-cause",
  "correlationId": "request-am-20482",
  "causationId": "decision-rc-7184",
  "contentType": "application/json",
  "fingerprintAlgorithm": "sha-256+jcs",
  "payload": {
    "verdictId": "019bc4d4-81c0-7eal-82d5-4f9b8719d0a1",
    "rootCauseCaseId": "019bc4d4-6dfe-7bcb-9de8-3af71979285a",
    "sourceAlarmFloodMessageId": "019bc4d4-5dc1-7b8d-9d48-542d25315356",
    "siteId": "site-ath-17",
    "lineId": "line-02",
    "verdictCode": "COMMON_POWER_SUPPLY_INSTABILITY",
    "confidence": 0.94,
    "policyIdentity": "root-cause/verdict/3",
    "verdictIdentity": "sha256:a2451a72990d5933e175396c86fd96854ff1db0e6571e0ae85afc37fb1734c17",
    "evidenceRevision": 1,
    "evidenceCount": 5,
    "evidenceMembershipSha256": "sha256:af3f3ad8da7edfaa50d14ebf422d424e2dd4e49767a2f4a056ce7682a801b5b7",
    "issuedAtUtc": "2026-07-19T07:47:38.442Z",
    "producerStream": {
      "streamId": "root-cause/case/019bc4d4-6dfe-7bcb-9de8-3af71979285a",
      "position": 2
    }
  },
  "contentFingerprint": "sha256:07d530ef636be3d2af8dcc3b20cd91b212a076a4e70c1a9c55730930fbfe1e2e"
}
```

READABLE IS NOT CANONICAL
The corpus stores the RFC 8785 canonical one-line representation: exactly 1157 UTF-8 bytes. Audit and Compliance must admit the same body SHA-256; subscriber-specific envelopes are forbidden.

EXACT BYTE EVIDENCE

Meaning, fingerprint and transport bytes have separate identities

GOLDEN DIGEST LEDGER 34-S

Artifact	Exact result	Protects
evidence membership	af3f3ad8da7edfaa50d14ebf422d424e2dd4e49767a2f4a056ce7682a801b5b7	ordered local basis
verdict identity	a2451a72990d5933e175396c86fd96854ff1db0e6571e0ae85afc37fb1734c17	decision meaning
content fingerprint	07d530ef636be3d2af8dcc3b20cd91b212a076a4e70c1a9c55730930fbfe1e2e	envelope meaning without fingerprint
final body SHA-256	192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00	stored/published bytes
canonical body length	1157 bytes	corpus drift

EXECUTABLE ASSET 34-T - PRODUCER AND CONSUMER CORPUS

```
[Fact]
public void RC_CONTRACT_001_golden_bytes_are_frozen()
{
    Corpus.VerdictIssuedV1.Bytes.Length.Should().Be(1157);
    SHA256.HashData(Corpus.VerdictIssuedV1.Bytes)
        .Should().Equal(Convert.FromHexString(
            "192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00"));
}

[Fact]
public void RC_AUD_COMP_001_both_consumers_accept_same_body()
{
    var bytes = Corpus.VerdictIssuedV1.Bytes;
    audit.Admit(bytes).EnvelopeSha256
        .Should().Equal(compliance.Admit(bytes).EnvelopeSha256);
    audit.Admit(bytes).Contract.Should().BeEquivalentTo(
        compliance.Admit(bytes).Contract);
}
```

BROKER TOPOLOGY

Independent subscribers require independent queues

A single shared queue would create competing consumers: one service could remove the delivery intended for the other. Fan-out is expressed as two bindings from the public-facts exchange to two durable queues.

EXECUTABLE ASSET 34-U - VALUE-FREE TOPOLOGY MANIFEST

exchange:
 name: nexus.events
 type: topic
 durable: true

bindings:
 - routing_key: root-cause.root-cause-verdict-issued.v1
 queue: audit.root-cause-verdicts.v1
 - routing_key: root-cause.root-cause-verdict-issued.v1
 queue: compliance.root-cause-verdicts.v1

queues:
 - name: audit.root-cause-verdicts.v1
 durable: true
 type: quorum
 - name: compliance.root-cause-verdicts.v1
 durable: true
 type: quorum

TOPOLOGY INVARIANTS 34-V

Invariant	Failure if violated
two distinct queue names	one capability steals the other's fact
same exact binding key	one subscriber observes different semantics
durable owner queues	offline interval loses work
provisioner owns declarations	runtime credential gains administration
mandatory publisher	missing binding can masquerade as success

TOPOLOGY PROOF

The manifest is validated before traffic

EXECUTABLE ASSET 34-W - DECLARED TOPOLOGY GATE

```
[Fact]
public async Task FANOUT_TOP0_001_exact_bindings_exist()
{
    var actual = await broker.ReadTopologyAsync();
    actual.Exchange("nexus.events").Type.Should().Be("topic");
    actual.BindingsFor(
        "root-cause.root-cause-verdict-issued.v1")
        .Select(x => x.Queue).Should().BeEquivalentTo([
            "audit.root-cause-verdicts.v1",
            "compliance.root-cause-verdicts.v1"
        ]);
}

[Fact]
public async Task FANOUT_TOP0_002_subscribers_are_not_competitors()
{
    var queues = await broker.QueuesForContractAsync(
        "nexus1.root-cause.root-cause-verdict-issued.v1");
    queues.Should().OnlyHaveUniqueItems().And.HaveCount(2);
}
```

NEGATIVE TOPOLOGY CASES 34-X

Mutation	Expected result
remove Audit binding	topology gate fails; mandatory alone cannot see partial binding loss
bind both capabilities to one queue	topology gate fails
change queue type/durability	property-equivalence/provisioning failure
add wildcard consumer silently	manifest digest mismatch
runtime declares missing object	authorization failure; no self-healing claim

PUBLISH/SUBSCRIBE MEANING

RabbitMQ routes a published message from an exchange to every matching bound queue. The broker does not create two business facts: both queues receive copies of the same immutable public identity and body.

ROOTCAUSE RELAY

Publication reuses the proven outbox discipline

TX-RC-V ends before the network. The RootCause relay later claims the stored body, publishes with mandatory routing and publisher confirms, then records the result in RootCauseDb. It never rebuilds the verdict from current tables.

EXECUTABLE ASSET 34-Y - STABLE RELAY HANDOFF

```
var claim = await claims.TryClaimNextAsync(ct);
if (claim is null) return RelayCycle.Empty;

var observed = await publisher.PublishAsync(
    exchange: "nexus.events",
    routingKey: claim.RoutingKey,
    mandatory: true,
    properties: BrokerProperties.From(claim),
    body: claim.EnvelopeBytes,
    cancellation_token: ct);

await results.RecordAsync(
    claim.MessageId, claim.LeaseToken, observed, ct);
```

RELAY OUTCOMES 34-Z

Observation	Outbox action	Subscriber claim
mandatory return	retry/terminal by policy	none
publisher nack	retryable	none
confirm timeout	unknown; same bytes/ID replay	unknown
positive confirm	mark dispatched	routed publication accepted; processing unknown
result commit lost	resolve/replay same identity	may duplicate

NO GLOBAL COMPLETION

RootCause does not wait for subscriber receipts

Publisher confirms and consumer acknowledgements are orthogonal. A positive RootCause publish result cannot say that Audit or Compliance committed. Likewise, an Audit acknowledgement contains no information about Compliance.

DISTRIBUTED STATE VECTOR 34-AA

Coordinate	Possible value	Owner
RootCause verdict	Committed	RootCauseDb
RootCause outbox	Pending / Leased / Dispatched	RootCauseDb
Audit queue	Absent / Ready / Unacked	broker topology
Audit evidence	NotRecorded / Recorded	AuditDb
Compliance queue	Absent / Ready / Unacked	broker topology
Compliance review	NotStarted / Pending / later states	ComplianceDb

ARCHITECTURE TEST 34-AB - NO COMPLETION CALLBACK

```
[Fact]
public void FANOUT_ARCH_001_rootcause_has_no_subscriber_client()
{
    Types.InAssembly(typeof(IssueVerdictHandler).Assembly)
        .ShouldNot().HaveDependencyOnAny(
            "Audit.Contracts.Commands",
            "Compliance.Contracts.Commands",
            "System.Transactions")
        .GetResult().IsSuccessful.Should().BeTrue();

    typeof(IssueVerdictHandler).Assembly.GetExportedTypes()
        .SelectMany(x => x.GetMethods())
        .Should().NotContain(x => x.Name == "MarkAllSubscribersComplete");
}
```

ABSENCE IS OBSERVABLE

Operational views may calculate queue age and consumer progress from evidence. They must not write a synthetic global Succeeded flag back into RootCause truth.

CONSUMER ADMISSION

Both consumers validate the same public boundary

EXECUTABLE ASSET 34-AC - STRICT SHARED GRAMMAR, OWNED POLICY

```
public AdmittedVerdictDelivery Admit(
    BrokerRoute route,
    BrokerProperties properties,
    byte[] copiedBody,
    AuthenticatedBrokerPrincipal principal)
{
    authorization.DemandProducer(principal, "root-cause-relay");
    route.Demand("root-cause.root-cause-verdict-issued.v1");
    var document = envelopes.ReadV1(copiedBody);
    document.DemandCoordinates(
        producer: "root-cause",
        eventType: "nexus1.root-cause.root-cause-verdict-issued.v1",
        schemaVersion: 1,
        messageId: properties.MessageId);
    fingerprints.Verify(document, copiedBody);
    var contract = strictJson.Deserialize<RootCauseVerdictIssuedV1>(
        document.PayloadUtf8);
    VerdictSemantics.DemandValid(contract);
    return new(document, contract, SHA256.HashData(copiedBody));
}
```

ADMISSION REJECTS BEFORE OWNED STATE 34-AD

Defect	Stable rejection
wrong producer credential	producer-not-authorized
route/type/version mismatch	contract-coordinate-mismatch
unknown member or malformed UUID	invalid-v1-payload
fingerprint/body mismatch	public-fact-integrity-failed
evidence count/digest invalid	verdict-evidence-summary-invalid
unsupported verdict code	verdict-code-unsupported

AUDIT OWNERSHIP

Audit appends evidence; it never directs workflow

EXECUTABLE ASSET 34-AE - IMMUTABLE AUDIT EVIDENCE

```
public sealed class AuditEvidenceRecord
{
    public AuditEvidenceId Id { get; private init; }
    public Guid SourceMessageId { get; private init; }
    public Guid SourceVerdictId { get; private init; }
    public Guid RootCauseCaseId { get; private init; }
    public string EventType { get; private init; } = null!;
    public int SchemaVersion { get; private init; }
    public byte[] EnvelopeBytes { get; private init; } = null!;
    public byte[] EnvelopeSha256 { get; private init; } = null!;
    public string CorrelationId { get; private init; } = null!;
    public string CausationId { get; private init; } = null!;
    public DateTimeOffset OccurredAtUtc { get; private init; }
    public DateTimeOffset RecordedAtUtc { get; private init; }

    public static AuditEvidenceRecord Append(
        AdmittedVerdictDelivery delivery,
        AuditEvidenceId id, DateTimeOffset recordedAtUtc) =>
        new()
        {
            Id = id,
            SourceMessageId = delivery.MessageId,
            SourceVerdictId = Guid.ParseExact(
                delivery.Contract.VerdictId, "D"),
            RootCauseCaseId = Guid.ParseExact(
                delivery.Contract.RootCauseCaseId, "D"),
            EventType = delivery.EventType,
            SchemaVersion = delivery.SchemaVersion,
            EnvelopeBytes = delivery.EnvelopeBytes.ToArray(),
            EnvelopeSha256 = delivery.EnvelopeSha256.ToArray(),
            CorrelationId = delivery.CorrelationId,
            CausationId = delivery.CausationId,
            OccurredAtUtc = delivery.OccurredAtUtc,
            RecordedAtUtc = recordedAtUtc
        };
}
```

AUDIT SEMANTIC BOUNDARY 34-AF

Audit stores	Audit deliberately does not do
accepted identity, bytes digest and causality	recompute the verdict
indexed VerdictId and Caseld	open/close Compliance review
append timestamp and source occurrence	update the source evidence record
future correction as another linked record	erase or overwrite history

TX-AUD

Inbox receipt and evidence append are one AuditDb commit

EXECUTABLE ASSET 34-AG - AUDIT CONSUMER TRANSACTION

```
public sealed class AuditVerdictHandler(
    AuditDbContext db,
    IInboxReceiptResolver receipts,
    IAuditIdentitySource identities,
    TimeProvider clock)
{
    public async Task<ConsumeOutcome> HandleAsync(
        AdmittedVerdictDelivery delivery, CancellationToken ct)
    {
        var known = await receipts.TryResolveAsync(db, delivery, ct);
        if (known is not null) return known;

        var verdictId = Guid.ParseExact(delivery.Contract.VerdictId, "D");
        var existing = await db.Evidence.SingleOrDefaultAsync(
            x => x.SourceVerdictId == verdictId, ct);
        var outcome = "audit-evidence-already-recorded";
        if (existing is null)
        {
            db.Evidence.Add(AuditEvidenceRecord.Append(
                delivery, identities.NewEvidenceId(), clock.GetUtcNow()));
            outcome = "audit-evidence-recorded";
        }
        db.InboxReceipts.Add(InboxReceiptRow.Completed(
            "audit/root-cause-verdict/v1", delivery,
            outcome, clock.GetUtcNow()));

        await db.SaveChangesAsync(ct);
        return ConsumeOutcome.Committed(
            delivery.MessageId, outcome);
    }
}
```

AUDITDB CONSTRAINTS 34-AH

Constraint	Purpose
PK InboxReceipt(ConsumerName, MessageId)	transport duplicate arbitration
UX AuditEvidence(SourceProducer, VerdictId, Version)	semantic duplicate arbitration
binary(32) EnvelopeSha256	same identity cannot describe new bytes
update/delete denied on evidence principal	append-only database defence
no FK to RootCauseDb	consumer owns copied public evidence only

AUDIT IDEMPOTENCY

Transport replay and semantic repetition are different

AUDIT TWO-KEY ORACLE 34-AI

Delivery	Inbox	Evidence	Result
first ID / verdict	1	1	commit + ack
same ID / same bytes	1	1	duplicate + ack
new ID / same VerdictId	2	1	semantic no-op receipt + ack
same ID / changed bytes	1 + conflict evidence	1	reject; no mutation
new correction event	new receipt	new linked record	append; never update

EXECUTABLE ASSET 34-AJ - APPEND-ONLY MUTATION GATE

```
public override InterceptionResult<int> SavingChanges(
    DbContextEventData data, InterceptionResult<int> result)
{
    var forbidden = data.Context!.ChangeTracker
        .Entries<AuditEvidenceRecord>()
        .Where(x => x.State is EntityState.Modified or EntityState.Deleted)
        .Select(x => x.Entity.Id).ToArray();
    if (forbidden.Length != 0)
        throw new AuditMutationRejected(forbidden);
    return result;
}

[Fact]
public async Task AUD_INV_001_existing_evidence_cannot_be_changed()
{
    await fixture.SeedGoldenEvidenceAsync();
    await Assert.ThrowsAsync<AuditMutationRejected>(
        () => fixture.TryOverwriteVerdictCodeAsync("DIFFERENT"));
}
```

CORRECTION RULE

A future RootCauseVerdictCorrected.v1 is a new accepted public fact and a new Audit record linked to the original. Chapter 34 does not invent correction authorization or mutate the V1 verdict history.

COMPLIANCE OWNERSHIP

Compliance opens a review; it does not inherit the verdict

EXECUTABLE ASSET 34-AK - CONSUMER-OWNED REVIEW

```
public sealed class ComplianceReview
{
    public ComplianceReviewId Id { get; private init; }
    public Guid SourceVerdictId { get; private init; }
    public Guid SourceCaseId { get; private init; }
    public string SiteId { get; private init; } = null!;
    public string LineId { get; private init; } = null!;
    public string ObservedVerdictIdentity { get; private init; } = null!;
    public ComplianceReviewState State { get; private set; }
    public Guid SourceMessageId { get; private init; }
    public DateTimeOffset OpenedAtUtc { get; private init; }

    public static ComplianceReview Open(
        ComplianceReviewId id,
        RootCauseVerdictIssuedV1 source,
        Guid sourceMessageId,
        DateTimeOffset openedAtUtc) => new()
    {
        Id = id,
        SourceVerdictId = Guid.ParseExact(source.VerdictId, "D"),
        SourceCaseId = Guid.ParseExact(source.RootCauseCaseId, "D"),
        SiteId = SiteIdValue.Demand(source.SiteId),
        LineId = LineIdValue.Demand(source.LineId),
        ObservedVerdictIdentity =
            VerdictIdentityValue.Demand(source.VerdictIdentity),
        State = ComplianceReviewState.Pending,
        SourceMessageId = MessageIdentity.Demand(sourceMessageId),
        OpenedAtUtc = openedAtUtc
    };
}
```

COMPLIANCE AUTHORITY 34-AL

May own	Must never infer or mutate
review identity and Pending state	RootCause verdict correctness
review assignment, findings and decision	RootCause evidence membership
human-review authorization and deadlines	Audit evidence history
its own later public review facts	source event or producer stream

CONTRACT MINIMIZATION
Compliance receives only the public verdict summary required to open and correlate a review. Detailed RootCause evidence remains owner-private; any later access requires a separately authorized query contract.

TX-COMP

Inbox receipt and review creation share ComplianceDb SaveChanges

EXECUTABLE ASSET 34-AM - COMPLIANCE CONSUMER TRANSACTION

```
public sealed class ComplianceVerdictHandler(
    ComplianceDbContext db,
    IInboxReceiptResolver receipts,
    IComplianceIdentitySource identities,
    TimeProvider clock)
{
    public async Task<ConsumeOutcome> HandleAsync(
        AdmittedVerdictDelivery delivery, CancellationToken ct)
    {
        var known = await receipts.TryResolveAsync(db, delivery, ct);
        if (known is not null) return known;

        var verdictId = Guid.ParseExact(delivery.Contract.VerdictId, "D");
        var existing = await db.Reviews.SingleOrDefaultAsync(
            x => x.SourceVerdictId == verdictId, ct);
        var outcome = "review-already-exists";
        if (existing is null)
        {
            db.Reviews.Add(ComplianceReview.Open(
                identities.NewReviewId(), delivery.Contract,
                delivery.MessageId, clock.GetUtcNow()));
            outcome = "review-opened";
        }
        db.InboxReceipts.Add(InboxReceiptRow.Completed(
            "compliance/root-cause-verdict/v1",
            delivery, outcome, clock.GetUtcNow()));
        await db.SaveChangesAsync(ct);
        return ConsumeOutcome.Committed(delivery.MessageId, outcome);
    }
}
```

COMPLIANCEDB CONSTRAINTS 34-AN

Constraint	Purpose
PK InboxReceipt(ConsumerName, MessageId)	transport replay
UX ComplianceReview(SourceVerdictId)	one business review per verdict
SourceMessageId retained	causal and replay evidence
State initialized Pending	consumer does not auto-approve
no FK to RootCauseDb/AuditDb	local ownership

COMPLIANCE IDEMPOTENCY

A new MessageId cannot open a second review

COMPLIANCE TWO-KEY ORACLE 34-AO

Delivery	Inbox	Review	Outcome
first ID / verdict	1	1	review-opened
same ID / same bytes	1	1	prior outcome
new ID / same VerdictId	2	1	review-already-exists
same ID / changed bytes	1 + conflict	1	identity-conflict
different VerdictId	+1	+1	independent review

EXECUTABLE ASSET 34-AP - BUSINESS UNIQUENESS MAPPING

```
internal sealed class ComplianceReviewConfiguration
    : IEntityConfiguration<ComplianceReviewRow>
{
    public void Configure(EntityTypeBuilder<ComplianceReviewRow> b)
    {
        b.ToTable("ComplianceReview", "compliance");
        b.HasKey(x => x.ReviewId);
        b.HasIndex(x => x.SourceVerdictId)
            .HasDatabaseName("UX_ComplianceReview_SourceVerdictId")
            .IsUnique();
        b.Property(x => x.State).HasMaxLength(24).IsUnicode(false);
        b.Property(x => x.ObservedVerdictIdentity)
            .HasMaxLength(71).IsUnicode(false);
    }
}
```

REFINEMENT OF CHAPTER 28

The earlier generic handler staged ComplianceReviewOpened.v1 to demonstrate a consumer outbox. This binding slice creates only the review because no approved downstream contract requires that fact yet. Adding one later needs its own consumer, schema and evidence decision.

INDEPENDENT SETTLEMENT

Each queue acknowledges only its own database outcome

EXECUTABLE ASSET 34-AQ - TYPED DISPOSITION

```
public static DeliveryDisposition For(ConsumeOutcome outcome) =>
    outcome switch
    {
        ConsumeOutcome.Committed => DeliveryDisposition.Ack,
        ConsumeOutcome.Duplicate => DeliveryDisposition.Ack,
        ConsumeOutcome.Transient => DeliveryDisposition.Retry,
        ConsumeOutcome.Unknown => DeliveryDisposition.Retry,
        ConsumeOutcome.Invalid => DeliveryDisposition.Reject,
        ConsumeOutcome.IdentityConflict => DeliveryDisposition.Reject,
        _ => throw new UnreachableException()
    };

// Audit uses the delivery tag from its Audit channel.
// Compliance uses the delivery tag from its Compliance channel.
// No shared acknowledgement coordinator exists.
```

SETTLEMENT INDEPENDENCE 34-AR

Audit outcome	Compliance outcome	Allowed broker actions
Committed	Transient	Audit ack; Compliance retry
Transient	Committed	Audit retry; Compliance ack
Duplicate	Committed	both ack locally
IdentityConflict	Committed	Audit reject/evidence; Compliance ack
Unknown	Unknown	neither ack; independent redelivery

CHANNEL RULE

RabbitMQ delivery tags are channel-scoped. Ack, nack or reject is sent on the same long-lived channel that received the delivery, with `multiple=false` under concurrent dispatch.

AUDIT OFFLINE

Compliance progresses while Audit retains backlog

The first required outage proof stops Audit before publication. RootCause commits and dispatches normally; the broker holds Audit's copy while Compliance consumes its own copy.

EXECUTABLE ASSET 34-AS - AUDIT OUTAGE SCENARIO

```
[Fact]
public async Task FANOUT_OUTAGE_001_audit_offline_does_not_block_truth()
{
    await fixture.StopAuditAsync();
    var verdict = await fixture.RootCause.IssueGoldenVerdictAsync();
    await fixture.RootCause.WaitUntilDispatchedAsync(verdict.MessageId);
    await fixture.Compliance.WaitForReviewAsync(verdict.VerdictId);

    (await fixture.RootCause.ReadVerdictAsync(verdict.VerdictId))
        .Should().NotBeNull();
    await fixture.Compliance.AssertReviewCountAsync(verdict.VerdictId, 1);
    await fixture.Audit.AssertEvidenceCountAsync(verdict.VerdictId, 0);
    await fixture.Broker.AssertReadyAsync(
        "audit.root-cause-verdicts.v1", verdict.MessageId);

    await fixture.StartAuditAsync();
    await fixture.Audit.WaitForEvidenceAsync(verdict.VerdictId);
}
```

OBSERVED STATE 34-AT

Moment	RootCause	Audit	Compliance
before restart	Verdict committed	queue ready; no DB row	review Pending
after Audit restart	unchanged	evidence recorded	unchanged
reconciliation	same IDs/bytes	one inbox + one evidence	one inbox + one review

COMPLIANCE OFFLINE

Audit progresses while Compliance retains backlog

EXECUTABLE ASSET 34-AU - COMPLIANCE OUTAGE SCENARIO

```
[Fact]
public async Task FANOUT_OUTAGE_002_compliance_offline_does_not_block_truth()
{
    await fixture.StopComplianceAsync();
    var verdict = await fixture.RootCause.IssueGoldenVerdictAsync();
    await fixture.RootCause.WaitUntilDispatchedAsync(verdict.MessageId);
    await fixture.Audit.WaitForEvidenceAsync(verdict.VerdictId);

    (await fixture.RootCause.ReadVerdictAsync(verdict.VerdictId))
        .Should().NotBeNull();
    await fixture.Audit.AssertEvidenceCountAsync(verdict.VerdictId, 1);
    await fixture.Compliance.AssertReviewCountAsync(verdict.VerdictId, 0);
    await fixture.Broker.AssertReadyAsync(
        "compliance.root-cause-verdicts.v1", verdict.MessageId);

    await fixture.StartComplianceAsync();
    await fixture.Compliance.WaitForReviewAsync(verdict.VerdictId);
}
```

PROOF REQUIREMENTS 34-AV

Artifact	Must show
RootCauseDb trace	verdict/outbox commit precedes downstream work
broker capture	same MessageId/body digest appears in both queues
AuditDb trace	one local commit while Compliance is stopped
ComplianceDb trace	later local commit after restart
negative connection evidence	no cross-database session/transaction

SYMMETRY MATTERS

Testing only one offline subscriber can hide a central coordinator or dependency in the opposite direction. Both permutations are required and must reuse the same production composition path.

PARTIAL PROGRESS

There is no single instantaneous global status

VALID DISTRIBUTED STATES 34-AW

RootCause	Audit	Compliance	Interpretation
Committed	NotRecorded	NotStarted	publication/backlog pending
Committed	Recorded	NotStarted	Compliance lag only
Committed	NotRecorded	Pending	Audit lag only
Committed	Recorded	Pending	both observed independently
Committed	Rejected conflict	Pending	Audit incident; Compliance truth intact

EXECUTABLE ASSET 34-AX - EVIDENCE-BASED PROGRESS VIEW

```
public sealed record FanOutObservation(  
    Guid VerdictId,  
    OwnerTruthStatus RootCause,  
    SubscriberObservation Audit,  
    SubscriberObservation Compliance,  
    DateTimeOffset ObservedAtUtc)  
{  
    public bool IsFullyObserved =>  
        Audit.State == "recorded" &&  
        Compliance.State == "review-opened";  
}  
  
// IsFullyObserved is a read-side observation.  
// It is not stored as RootCause verdict state and cannot trigger rollback.
```

UI HANDOFF

Chapter 35 will build delayed Reporting truth and expose projection age. Chapter 34 records the underlying owner/subscriber positions but does not yet claim a console projection.

CRASH SEMANTICS

Every local interruption has an independent recovery rule

CROSS-BOUNDARY CRASH MATRIX 34-AY

Crash point	Durable state	Recovery
before TX-RC-V	no verdict/outbox	retry decision command
after TX-RC-V, before publish	verdict + pending body	relay claims later
after broker confirm	both queues may hold body	subscriber-independent consumption
Audit before SaveChanges	Audit unmodified; delivery unacked	Audit redelivery
Audit after commit, before ack	receipt + evidence	Audit duplicate then ack
Compliance before SaveChanges	Compliance unmodified; delivery unacked	Compliance redelivery
Compliance after commit, before ack	receipt + review	Compliance duplicate then ack
one DB commit response lost	owner-local result unknown	fresh-context receipt resolution

EXECUTABLE ASSET 34-AZ - NAMED SUBSCRIBER CHECKPOINTS

```
public enum FanOutCheckpoint
{
    RootCauseCommitted,
    BrokerConfirmed,
    AuditBodyCopied,
    AuditSaveStarted,
    AuditSaveCommitted,
    AuditAckCompleted,
    ComplianceBodyCopied,
    ComplianceSaveStarted,
    ComplianceSaveCommitted,
    ComplianceAckCompleted
}
```

UNAVOIDABLE WINDOWS

Neither subscriber can atomically join its database commit to its broker acknowledgement. Each inbox converts its own post-commit crash into an idempotent duplicate; it does not deduplicate the other subscriber.

BROKER-BACKED FAILURE LABORATORY

The harness stops real roles, not mock callbacks

EXECUTABLE ASSET 34-BA - OUTAGE AND RECOVERY MATRIX

```
[Theory]
[InlineData("audit", FanOutCheckpoint.AuditSaveStarted)]
[InlineData("audit", FanOutCheckpoint.AuditSaveCommitted)]
[InlineData("compliance", FanOutCheckpoint.ComplianceSaveStarted)]
[InlineData("compliance", FanOutCheckpoint.ComplianceSaveCommitted)]
public async Task FANOUT_REC_001_restart_converges_once(
    string subscriber, FanOutCheckpoint checkpoint)
{
    var verdict = await fixture.RootCause.IssueGoldenVerdictAsync();
    fixture.Probe.KillAt(subscriber, checkpoint, verdict.MessageId);
    await fixture.StartAllAsync();
    await fixture.Probe.WaitForKillAsync();
    await fixture.RestartAsync(subscriber);
    await fixture.WaitForConvergenceAsync(verdict.VerdictId);

    await fixture.AssertOneAuditEvidenceAsync(verdict.VerdictId);
    await fixture.AssertOneComplianceReviewAsync(verdict.VerdictId);
    await fixture.AssertSameBodyOnBothRoutesAsync(
        verdict.MessageId, verdict.BodySha256);
}
```

HARNESS EVIDENCE 34-BB

Evidence	Reject when
process instance + kill checkpoint	target checkpoint not reached
queue ready/unacked counts	only application memory observed
three database traces	cross-owner write/connection hidden
exact MessageId and body SHA	replacement identity/body appears
restart build/config identity	different untracked artifact
test discovery/execution	case skipped or zero discovered

AUDIT PROVIDER PROOF

Audit success commits two rows; failure commits none

EXECUTABLE ASSET 34-BC - SQL SERVER ATOMICITY

```
[Fact]
public async Task AUD_DB_001_receipt_and_evidence_commit_once()
{
    var delivery = Corpus.VerdictIssuedV1.AdmittedForAudit();
    var outcome = await fixture.Audit.ConsumeAsync(delivery);

    outcome.Should().BeOfType<ConsumeOutcome.Committed>();
    await fixture.AuditDb.AssertInboxAsync(delivery.MessageId, 1);
    await fixture.AuditDb.AssertEvidenceAsync(delivery.VerdictId, 1);
}

[Fact]
public async Task AUD_DB_002_evidence_insert_failure_rolls_back_receipt()
{
    var delivery = Corpus.VerdictIssuedV1.AdmittedForAudit();
    fixture.AuditDb.FailInsert("audit.AuditEvidenceRecord");

    await Assert.ThrowsAsync<InjectedDatabaseFailure>(
        () => fixture.Audit.ConsumeAsync(delivery));
    await fixture.AuditDb.AssertInboxAsync(delivery.MessageId, 0);
    await fixture.AuditDb.AssertEvidenceAsync(delivery.VerdictId, 0);
}
```

REQUIRED PROVIDER CASES 34-BD

Case	Oracle
simultaneous same MessageId	one commit, one matching duplicate
new MessageId / same VerdictId	two receipts, one evidence row
same MessageId / different digest	conflict; original row unchanged
commit response lost	fresh context resolves receipt or returns Unknown
attempted update/delete	interceptor and database permission reject

COMPLIANCE PROVIDER PROOF

Compliance success commits receipt and review once

EXECUTABLE ASSET 34-BE - SQL SERVER ATOMICITY

```
[Fact]
public async Task COMP_DB_001_receipt_and_review_commit_once()
{
    var delivery = Corpus.VerdictIssuedV1.AdmittedForCompliance();
    var outcome = await fixture.Compliance.ConsumeAsync(delivery);

    outcome.Should().BeEquivalentTo(
        ConsumeOutcome.Committed(delivery.MessageId, "review-opened"));
    await fixture.ComplianceDb.AssertInboxAsync(delivery.MessageId, 1);
    await fixture.ComplianceDb.AssertReviewAsync(delivery.VerdictId, 1);
}

[Fact]
public async Task COMP_DB_002_review_insert_failure_rolls_back_receipt()
{
    var delivery = Corpus.VerdictIssuedV1.AdmittedForCompliance();
    fixture.ComplianceDb.FailInsert("compliance.ComplianceReview");

    await Assert.ThrowsAsync<InjectedDatabaseFailure>(
        () => fixture.Compliance.ConsumeAsync(delivery));
    await fixture.ComplianceDb.AssertInboxAsync(delivery.MessageId, 0);
    await fixture.ComplianceDb.AssertReviewAsync(delivery.VerdictId, 0);
}
```

NO IN-MEMORY SUBSTITUTION 34-BF

Production behavior	Why the selected provider is required
named unique constraints	classify transport vs business conflict
transaction rollback	receipt and effect share exact commit
concurrent insert race	database is final arbiter
permission boundary	Compliance principal cannot write Audit/RootCause
query/index evidence	actual plan and lock behavior

CONCURRENT REDELIVERY

Every owner resolves its own race with a fresh unit of work

EXECUTABLE ASSET 34-BG - TWO SUBSCRIBERS, TWO LOCAL RACES

```
[Fact]
public async Task FANOUT_DB_001_concurrent_replays_converge_per_owner()
{
    var golden = Corpus.VerdictIssuedV1;
    var audit = await fixture.RaceAuditAsync(golden, participants: 2);
    var compliance = await fixture.RaceComplianceAsync(
        golden, participants: 2);

    audit.Should().ContainSingle(x => x is ConsumeOutcome.Committed);
    audit.Should().ContainSingle(x => x is ConsumeOutcome.Duplicate);
    compliance.Should().ContainSingle(
        x => x is ConsumeOutcome.Committed);
    compliance.Should().ContainSingle(
        x => x is ConsumeOutcome.Duplicate);

    await fixture.AssertOneAuditEvidenceAsync(golden.VerdictId);
    await fixture.AssertOneComplianceReviewAsync(golden.VerdictId);
}
```

RACE RESOLVER RULES 34-BH

Rule	Reason
classify only named provider constraints	generic DbUpdateException is not duplicate evidence
discard failed DbContext	tracked losing rows cannot be reused
reload receipt and business row	resolve actual committed state
compare stored digest in fixed time	identity conflict is not a retry
return Unknown when no durable row exists	never acknowledge an assumption

DISTRIBUTED AUTHORIZATION

Each runtime credential has one narrow capability

CREDENTIAL MATRIX 34-BI

Principal	Allowed	Denied
topology provisioner	declare approved exchange/queues/bindings	consume or publish business facts
RootCause relay	write nexus.events on approved route	configure/read subscriber queues
Audit consumer	read/ack Audit queue; write AuditDb	Compliance queue/DB; RootCauseDb
Compliance consumer	read/ack Compliance queue; write ComplianceDb	Audit queue/DB; RootCauseDb
evidence reviewer	read manifests/test artifacts	runtime secrets and payload dumps

EXECUTABLE ASSET 34-BJ - NEGATIVE AUTHORIZATION PROOF

```
[Fact]
public async Task FANOUT_AUTH_001_consumers_cannot_cross_queues()
{
    await broker.As("audit-consumer")
        .ConsumeDeniedAsync("compliance.root-cause-verdicts.v1");
    await broker.As("compliance-consumer")
        .ConsumeDeniedAsync("audit.root-cause-verdicts.v1");
}

[Fact]
public async Task FANOUT_AUTH_002_runtime_cannot_mutate_topology()
{
    await broker.As("root-cause-relay")
        .DeclareQueueDeniedAsync("shadow-verdicts");
}
```

TRUST STATEMENT

Broker ACL provenance authenticates the configured connection path; it is not per-message cryptographic signing. The envelope producer field cannot elevate an unauthorized principal.

ARCHITECTURE ENFORCEMENT

No service imports another service implementation or database

EXECUTABLE ASSET 34-BK - REFERENCE AND CONNECTION GATES

```
[Fact]
public void FANOUT_ARCH_002_consumers_reference_contract_only()
{
    Types.InAssemblies([
        typeof(AuditVerdictHandler).Assembly,
        typeof(ComplianceVerdictHandler).Assembly])
        .ShouldNot().HaveDependencyOnAny(
            "RootCause.Domain",
            "RootCause.Application",
            "RootCause.Infrastructure")
        .GetResult().IsSuccessful.Should().BeTrue();
}

[Fact]
public void FANOUT_ARCH_003_each_host_registers_one_owned_db()
{
    HostManifest.Audit.ConnectionPurposes
        .Should().Equal("AuditDb");
    HostManifest.Compliance.ConnectionPurposes
        .Should().Equal("ComplianceDb");
    HostManifest.RootCause.ConnectionPurposes
        .Should().Equal("RootCauseDb");
}

[Fact]
public void FANOUT_ARCH_004_no_ambient_transaction()
{
    Types.InAssemblies([
        typeof(IssueVerdictHandler).Assembly,
        typeof(AuditVerdictHandler).Assembly,
        typeof(ComplianceVerdictHandler).Assembly])
        .ShouldNot().HaveDependencyOn("System.Transactions")
        .GetResult().IsSuccessful.Should().BeTrue();
}
```

RUNTIME NEGATIVE EVIDENCE 34-BL

Observation	Required result
SQL session inventory	each service principal reaches only its database
distributed transaction inventory	zero enlisted cross-resource transactions
HTTP client registrations	no RootCause -> Audit/Compliance mutation clients
broker permissions	each consumer reads only its own queue
assembly graph	producer contracts remain dependency-poor

CAUSALITY AND TRACES

One business chain produces independent processing spans

EXECUTABLE ASSET 34-BM - BOUNDED MESSAGING SPANS

```
using var activity = Sources.FanOut.StartActivity(  
    $"process {delivery.Destination}",  
    ActivityKind.Consumer,  
    parentContext: remoteTraceContext,  
    tags: new ActivityTagsCollection  
    {  
        ["messaging.system"] = "rabbitmq",  
        ["messaging.operation.type"] = "process",  
        ["messaging.destination.name"] = delivery.Destination,  
        ["messaging.message.id"] = delivery.MessageId,  
        ["nexus1.contract"] = "root-cause-verdict-issued.v1",  
        ["nexus1.outcome"] = outcome.Code  
    }  
);  
  
// Business causation remains decision-rc-7184 in the envelope.  
// A trace can be absent, sampled differently or restarted without changing it.
```

TRACE RELATIONSHIP 34-BN

Span	Parent/link	Must not imply
RootCause verdict command	local operation	subscriber completion
RootCause relay send	outbox creation/send relation	Audit/Compliance commit
Audit process	remote publish context	Compliance order
Compliance process	remote publish context	Audit order
restart/replay process	new processing attempt/link	new business fact

TELEMETRY STABILITY

OpenTelemetry messaging semantic conventions are versioned and currently evolving. The evidence bundle records the emitted convention profile and instrumentation package; business correctness never depends on a span name.

BACKLOG AND READINESS

Subscriber lag is visible without blocking RootCause

BOUNDED TELEMETRY 34-BO

Signal	Meaning	Allowed labels
fanout.delivery	per-consumer outcomes	consumer,outcome
fanout.processing.duration	local processing latency	consumer
fanout.queue.ready	broker ready count	consumer
fanout.queue.unacked	in-flight deliveries	consumer
fanout.oldest.age	oldest not locally observed	consumer
fanout.identity_conflict	security/data-integrity incident	consumer

EXECUTABLE ASSET 34-BP - ROLE-SPECIFIC HEALTH

```
public FanOutHealth Assess(SubscriberBacklog backlog, FanOutPolicy p)
{
    if (backlog.IdentityConflicts > 0)
        return FanOutHealth.Unhealthy("identity-conflict");
    if (backlog.OldestReadyAge > p.CriticalAge)
        return FanOutHealth.Unhealthy("backlog-critical");
    if (backlog.OldestReadyAge > p.WarningAge)
        return FanOutHealth.Degraded("backlog-delayed");
    return FanOutHealth.Healthy();
}

// RootCause API readiness does not call this subscriber assessment.
// Audit and Compliance expose their own database/consumer readiness.
```

CARDINALITY RULE

VerdictId, MessageId, CaseId, SiteId, correlation and causation remain in bounded logs/traces, never metric labels. Payloads, evidence digests and credentials are not logged.

RETENTION, REPLAY AND REPAIR

Recovery preserves the original fact and local histories

LIFECYCLE POLICY 34-BQ

Asset	Retention / replay rule
RootCause verdict/outbox	retain immutable verdict and exact body through audit/replay horizon
Audit inbox/evidence	retain identity/digest with append-only correction lineage
Compliance inbox/review	retain source verdict identity through review lifecycle
broker dead-letter evidence	operator-authorized repair; never silent republish
expired inbox	block replay unless another complete semantic oracle is approved

EXECUTABLE ASSET 34-BR - AUTHORIZED SUBSCRIBER REPLAY

```
public async Task ReplayAsync(
    DeadLetterEvidence evidence,
    ReplayAuthorization authorization,
    CancellationToken ct)
{
    authorization.Demand("fanout:subscriber:replay");
    evidence.DemandCompleteOriginal(
        messageId: "019bc4d4-82a4-77f2-ae06-09b172aa59db",
        bodySha256: "192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00");
    await replay.PublishToOriginalQueueAsync(
        evidence.OriginalQueue,
        evidence.OriginalMessageId,
        evidence.OriginalEnvelopeBytes,
        authorization.OperationId,
        ct);
}
```

REPAIR EXCLUSIONS 34-BS

Forbidden shortcut	Why
new MessageId for replay	creates a new transport identity
rebuild body from current RootCauseDb	historical bytes may drift
copy Audit row into ComplianceDb	steals consumer authority
mark review open without inbox evidence	causality and deduplication lost

FAIL-CLOSED EVIDENCE

A PDF specification is not a successful fan-out run

EXECUTABLE ASSET 34-BT - EVIDENCE MANIFEST

```
{
  "claimId": "RC-VERDICT-FANOUT-V1-REVIEW-001",
  "status": "PENDING",
  "bookChapter": 34,
  "contract": {
    "eventType": "nexus1.root-cause.root-cause-verdict-issued.v1",
    "schemaVersion": 1,
    "routingKey": "root-cause.root-cause-verdict-issued.v1",
    "goldenBodySha256": "192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00"
  },
  "requiredArtifacts": [
    "rootcause-verdict-atomicity.trx",
    "producer-consumer-golden-corpus.trx",
    "two-queue-topology-and-route.trx",
    "audit-provider-and-recovery.trx",
    "compliance-provider-and-recovery.trx",
    "subscriber-outage-permutations.trx",
    "authorization-and-no-cross-db.trx",
    "fanout-observability-snapshot.json",
    "runtime-identity.json"
  ],
  "executedArtifacts": [],
  "missingReason": "reference repository and provider/broker lab not executed here"
}
```

FAIL-CLOSED VALIDATOR 34-BU

Reject when	Meaning
golden digest or topology manifest drifts	different contract/runtime
only one outage permutation executes	independence incompletely proved
test discovery is zero/skipped	absence is not success
provider/broker version or image digest absent	behavior unbound
artifact build identity differs or is stale	evidence describes another build

ADR, PROOF BOUNDARY AND CHECKPOINT**Independent progress is the intended consistency model****ADR-032 34-BV**

Field	Decision
status	Accepted for implementation; runtime evidence PENDING
decision	TX-RC-V -> one exact V1 -> two queues -> TX-AUD and TX-COMP
idempotency	consumer-scoped inbox plus owner-local semantic key
rejected	distributed transaction, shared queue, callback completion, cross-db read/write
evidence	contract, provider, broker, outage, crash, authorization, telemetry

OFFICIAL TECHNICAL ANCHORS

- RabbitMQ publish/subscribe and exchanges: <https://www.rabbitmq.com/tutorials/tutorial-three-dotnet>
- RabbitMQ acknowledgements and confirms: <https://www.rabbitmq.com/docs/confirmations>
- RabbitMQ .NET client 7.x API guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>
- RabbitMQ queues and ordering: <https://www.rabbitmq.com/docs/queues>
- RabbitMQ consumer prefetch: <https://www.rabbitmq.com/docs/consumer-prefetch>
- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- OpenTelemetry messaging spans:
<https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/>
- RFC 8785 ISON Canonicalization: <https://www.rfc-editor.org/rfc/rfc8785>

WHAT THIS CHAPTER HAS PROVED

Chapter 34 has specified one evidence-bound RootCause verdict and exact RootCauseVerdictIssued.v1 body; owner-local RootCause, Audit and Compliance commits; two independent queues; two-layer idempotency; manual settlement; partial-progress, outage, crash, authorization, provider and broker proof paths; and fail-closed evidence gates. It has not executed those artifacts, completed a Compliance review, built Reporting truth or claimed that every subscriber is current.

NEXT - CHAPTER 35

Reporting Builds Delayed Truth will consume RootCauseVerdictIssued.v1 into its own projection, expose watermark, age and update status through a BFF query, and prevent the console from presenting eventual consistency as instant certainty.

CHAPTER CHECKPOINT

- Why do Audit and Compliance require different queues and different inbox keys?
- Which exact rows enter TX-RC-V, TX-AUD and TX-COMP?
- What does a positive publisher confirm prove, and what does it not prove?
- How does each subscriber recover after commit succeeds but acknowledgement is lost?
- Why can Audit be current while Compliance is not, without corrupting RootCause truth?
- Which golden digests freeze RootCauseVerdictIssued.v1?
- What evidence must exist before status may change from PENDING?

PART VI - THE FIRST END-TO-END DISTRIBUTED SLICE

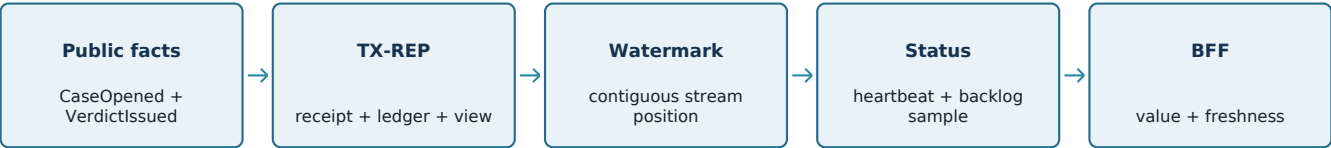
35. Reporting Builds Delayed Truth

Reporting consumes the exact RootCause case stream fixed by Chapters 33 and 34, builds one owner-local read model, and exposes what it knows together with how recently and how completely it knows it.

CHAPTER DECISION

ReportingDb stores an inbox receipt, immutable event ledger, gap buffer, root-cause case summary, contiguous producer-stream watermark and projection generation. A separate status sampler records consumer heartbeat and broker backlog observations. The BFF returns the projection and this evidence together; it never labels delayed, unknown or failed data as live truth.

PROOF PATH 35-A - PUBLIC FACT TO HONEST CONSOLE OBSERVATION



GUARANTEE LEDGER 35-A

Boundary	Allowed claim	Still forbidden
Reporting commit	one observed projection generation	RootCause is unchanged
watermark @ 2	positions 1 and 2 applied contiguously	global broker order
fresh status sample	no known local gap/backlog/failure	all source facts exist
BFF 200	observation returned with evidence	instant or authoritative truth

THE FAILURE IN THE NAIVE DESIGN

A fast answer can still be an old answer

The naive console calls RootCause, Audit and Compliance on every refresh, merges whichever response arrives, then renders a green Current badge. That is a distributed read transaction without a transaction, a stable observation point or an owner for partial failure.

REJECTED SHAPE 35-B - SYNCHRONOUS COMPOSITION MASQUERADES AS TRUTH

```
var verdict = await rootCause.GetVerdictAsync(caseId, ct);
var audit = await audit.GetEvidenceAsync(caseId, ct);
var review = await compliance.GetReviewAsync(caseId, ct);

return Results.Ok(new {
    verdict,
    audit,
    review,
    status = "current" // no common snapshot or freshness proof
});
```

FAILURE REVIEW 35-B

Shortcut	Observed defect	Required boundary
fan-in query	one slow owner blocks the console	Reporting-owned projection
merge timestamps	unrelated clocks become a fake snapshot	watermark + observed-at
404 means absent	lag is rendered as business nonexistence	not-observed-yet state
green by low latency	fast stale data looks current	known-gap/backlog predicate
browser cache	old freshness travels without new status	no-store in Phase 0

PLAIN-LANGUAGE RULE

A read model is useful because it is deliberately delayed. Its API must return the delay evidence with the value, not hide it behind a confident adjective.

SUBSCRIPTION OWNERSHIP

Reporting receives a third independent copy

Chapter 25 already gave Reporting its own durable integration queue with a `root-cause.#` binding. Chapter 35 activates that existing boundary for two approved V1 facts. Chapter 34's two-subscriber route snapshot is therefore extended, not contradicted: the exact `VerdictIssued` body now reaches Audit, Compliance and Reporting independently.

TOPOLOGY RESOLUTION 35-C

Queue	Binding	Capability	Offline effect
audit.root-cause-verdicts.v1	exact VerdictIssued V1	append evidence	Audit backlog only
compliance.root-cause-verdicts.v1	exact VerdictIssued V1	open review	Compliance backlog only
reporting.integration-events.v1	root-cause.#	project approved contracts	Reporting view delays

EXECUTABLE ASSET 35-A - ROUTE GATE AFTER REPORTING ACTIVATION

```
var routed = await topology.QueuesMatchingAsync(
    "nexus.events",
    "root-cause.root-cause-verdict-issued.v1",
    ct);

routed.Should().BeEquivalentTo([
    "audit.root-cause-verdicts.v1",
    "compliance.root-cause-verdicts.v1",
    "reporting.integration-events.v1"
]);

await topology.AssertBindingAsync(
    "reporting.integration-events.v1", "root-cause.#", ct);
```

ROUTING LIMIT

The wildcard selects public `RootCause` facts, not private models. The Reporting dispatcher still rejects every event type/schema that is absent from its explicit projection contract set.

INHERITED STATE

One case stream, two frozen public facts

The projection reuses the exact identities and bytes already frozen. It does not allocate a new case, verdict, correlation or occurrence time. Reporting adds only consumer-owned processing identities and observation times.

GOLDEN CONTINUITY LEDGER 35-D

Coordinate	Exact value	Authority
RootCauseCaseld	019bc4d4-6dfe-7bcb-9de8-3af71979285a	RootCause
CaseOpened Messageld	019bc4d4-6ea0-7d14-a57a-3e8643c55f13	RootCause
VerdictId	019bc4d4-81c0-7ea1-82d5-4f9b8719d0a1	RootCause
VerdictIssued Messageld	019bc4d4-82a4-77f2-ae06-09b172aa59db	RootCause
producer stream	root-cause/case/019bc4d4-6dfe-7bcb-9de8-3af71979285a	RootCause
positions	CaseOpened @ 1; VerdictIssued @ 2	RootCause
correlation / causation	request-am-20482 / decision-rc-7184	public envelopes

EXACT BYTE ANCHORS 35-E

Artifact	Frozen result
CaseOpened body SHA-256	454f51cf4b0250b9c855346a6648a7d73530f19687ee3b52cda2edf9d0339c5b
VerdictIssued body SHA-256	192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00
VerdictIssued canonical length	1157 UTF-8 bytes
projection contract set	reporting/root-cause-case-summary/1

CONSUMER-OWNED ADDITIONS

AppliedAtUtc, ProjectionGenerationId, RowVersion and status observations describe Reporting processing. They never replace OccurredAtUtc, Messageld, VerdictIdentity or producer stream position.

PROJECTION CONTRACT

A wildcard binding is not a wildcard reducer

A broad queue needs a narrow executable allowlist. The contract set fixes the accepted event types, schema versions, mapper identities and one projection reducer. Adding a fact changes the set digest and requires a migration/rebuild decision.

EXECUTABLE ASSET 35-B - EXPLICIT PROJECTION CONTRACT SET

```
public static class RootCauseCaseSummaryContractSet
{
    public const string Identity =
        "reporting/root-cause-case-summary/1";

    public static readonly ProjectionContract[] Contracts =
    [
        new("nexus1.root-cause.root-cause-case-opened.v1", 1,
            "RootCauseCaseOpenedV1Mapper/1"),
        new("nexus1.root-cause.root-cause-verdict-issued.v1", 1,
            "RootCauseVerdictIssuedV1Mapper/1")
    ];

    public static string Digest =>
        ContractSetDigest.Compute(Identity, Contracts);
}
```

ADMISSION RESULT 35-F

Input	Projection action
known type + schema + valid bytes	translate to immutable ProjectionEvent
known type + unsupported schema	terminal unsupported-contract evidence
unknown public RootCause type	quarantine for this subscription; no reducer
valid duplicate	resolve stored receipt/ledger
same identity, changed bytes	identity conflict; never overwrite

CONSUMER ADMISSION

Validate public meaning before projection work

Reporting copies the delivery body, authenticates the approved RootCause relay path and applies the same envelope/fingerprint grammar used by Audit and Compliance. Translation happens only after the exact public boundary passes.

EXECUTABLE ASSET 35-C - STRICT REPORTING ADMISSION

```
public AdmittedProjectionDelivery Admit(
    RawBrokerDelivery raw,
    AuthenticatedBrokerPrincipal principal)
{
    authorization.DemandProducer(principal, "root-cause-relay");
    raw.Route.DemandExchange("nexus.events");

    var document = envelopes.ReadV1(raw.CopiedBody);
    document.DemandProducer("root-cause");
    document.DemandMessageId(raw.Properties.MessageId);
    fingerprints.Verify(document, raw.CopiedBody);

    var descriptor = contractSet.GetRequired(
        document.EventType, document.SchemaVersion);
    var projectionEvent = descriptor.Map(document.PayloadUtf8);
    projectionEvent.Stream.DemandPrefix("root-cause/case/");

    return new(document, projectionEvent,
        SHA256.HashData(raw.CopiedBody));
}
```

ADMISSION ORDER 35-G

Order	Proof	No mutation before
1	credential + exchange/route	producer path accepted
2	envelope coordinates	identity grammar accepted
3	fingerprint + body digest	bytes accepted
4	contract set lookup	type/schema supported
5	payload invariants	stream event constructed

REPORTING-OWNED MODEL

A useful copy stays an explicitly delayed copy

The read row is a Reporting-owned observation. RootCause identifiers remain source coordinates; RowVersion and ProjectionGenerationId belong to Reporting. Nullable verdict fields are valid while only CaseOpened has been applied.

EXECUTABLE ASSET 35-D - ROOT-CAUSE CASE SUMMARY ROW

```
public sealed class RootCauseCaseSummaryRow
{
    public Guid ProjectionGenerationId { get; set; }
    public Guid RootCauseCaseId { get; set; }
    public string SiteId { get; set; } = null!;
    public string LineId { get; set; } = null!;
    public string Status { get; set; } = null!; // Open | VerdictIssued
    public Guid? VerdictId { get; set; }
    public string? VerdictCode { get; set; }
    public decimal? Confidence { get; set; }
    public string? PolicyIdentity { get; set; }
    public string? VerdictIdentity { get; set; }
    public int EvidenceCount { get; set; }
    public DateTimeOffset SourceOccurredAtUtc { get; set; }
    public DateTimeOffset LastAppliedAtUtc { get; set; }
    public long LastAppliedStreamPosition { get; set; }
    public Guid LastAppliedMessageId { get; set; }
    public byte[] RowVersion { get; set; } = null!;
}
```

READ-MODEL LIMIT

This row is safe for operator display and filtering. It is forbidden as a command-side existence check, RootCause invariant source, Compliance decision input or Audit history replacement.

PROGRESS EVIDENCE

Watermark and health are different coordinates

A single LastUpdated timestamp cannot distinguish a quiet but healthy projection from a dead consumer, a broker backlog or a stream gap. Reporting therefore stores structural progress separately from operational observations.

EXECUTABLE ASSET 35-E - CHECKPOINT AND STATUS SNAPSHOT

```
public sealed class ProjectionCheckpointRow
{
    public string ProjectionName { get; set; } = null!;
    public Guid ProjectionGenerationId { get; set; }
    public string StreamId { get; set; } = null!;
    public long LastContiguousPosition { get; set; }
    public Guid LastAppliedMessageId { get; set; }
    public DateTimeOffset LastSourceOccurredAtUtc { get; set; }
    public DateTimeOffset LastAppliedAtUtc { get; set; }
    public long? GapStartsAtPosition { get; set; }
}

public sealed class ProjectionStatusRow
{
    public string ProjectionName { get; set; } = null!;
    public string State { get; set; } = null!;
    public DateTimeOffset ObservedAtUtc { get; set; }
    public DateTimeOffset? ConsumerHeartbeatAtUtc { get; set; }
    public long ReadyCount { get; set; }
    public long UnackedCount { get; set; }
    public TimeSpan? OldestReadyAge { get; set; }
    public long GapCount { get; set; }
    public string? FailureCode { get; set; }
}
```

TWO EVIDENCE PLANES 35-H

Plane	Meaning	Update owner
checkpoint	contiguous semantic progress	projector TX-REP
status	sampled consumer/path health	Reporting status monitor

REPORTINGDB MAPPING

The database is the final duplicate arbiter

Database constraints arbitrate duplicates and conflicting stream positions. Reporting keeps an immutable ledger even after events have entered the read row, so a new MessageId cannot silently replace a previously applied position.

EXECUTABLE ASSET 35-F - EF CORE UNIQUENESS AND CONCURRENCY

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<InboxReceiptRow>().HasKey(x =>
        new { x.ConsumerName, x.MessageId });

    modelBuilder.Entity<ProjectionEventLedgerRow>().HasKey(x =>
        new { x.ProjectionName, x.GenerationId,
              x.StreamId, x.StreamPosition });

    modelBuilder.Entity<PendingProjectionEventRow>().HasKey(x =>
        new { x.ProjectionName, x.GenerationId,
              x.StreamId, x.StreamPosition });

    modelBuilder.Entity<ProjectionCheckpointRow>().HasKey(x =>
        new { x.ProjectionName, x.ProjectionGenerationId,
              x.StreamId });

    modelBuilder.Entity<RootCauseCaseSummaryRow>().HasKey(x =>
        new { x.ProjectionGenerationId, x.RootCauseCaseId });
    modelBuilder.Entity<RootCauseCaseSummaryRow>()
        .Property(x => x.RowVersion).IsRowVersion();
}
```

DATABASE ORACLE 35-I

Constraint	Conflict classified
Inbox PK	same transport MessageId for this consumer
ledger position PK	two meanings claim one stream position
pending position PK	two buffered bodies claim one gap slot
projection PK	one row per generation/case
rowversion	concurrent reducer update; retry from fresh context

FIRST REDUCER

CaseOpened establishes the projection baseline

CaseOpened at producer stream position 1 creates the projection. The reducer verifies that the source identity in the stream suffix matches the payload case and stores the public occurrence time separately from the local apply time.

EXECUTABLE ASSET 35-G - APPLY ROOTCAUSECASEOPENED.V1

```
public static RootCauseCaseSummaryRow Apply(
    RootCauseCaseOpenedProjectionEvent e,
    Guid generationId,
    DateTimeOffset appliedAtUtc)
{
    e.Stream.DemandPosition(1);
    e.Stream.DemandEntity(e.CaseId);

    return new RootCauseCaseSummaryRow
    {
        ProjectionGenerationId = generationId,
        RootCauseCaseId = e.CaseId,
        SiteId = e.SiteId,
        LineId = e.LineId,
        Status = "Open",
        EvidenceCount = e.InitialEvidenceCount,
        SourceOccurredAtUtc = e.OccurredAtUtc,
        LastAppliedAtUtc = appliedAtUtc,
        LastAppliedStreamPosition = 1,
        LastAppliedMessageId = e.MessageId
    };
}
```

GOLDEN ROW AFTER POSITION 1 35-J

Field	Value
case / site / line	019bc4d4-6dfe-7bcb-9de8-3af71979285a / site-ath-17 / line-02
status	Open
evidence count	5
source occurred	2026-07-19T07:43:12.017Z
watermark	root-cause/case/019bc4d4-6dfe-7bcb-9de8-3af71979285a @ 1

SECOND REDUCER

VerdictIssued advances one observed case

VerdictIssued at position 2 updates the existing row only when all inherited coordinates agree. The reducer does not recompute a verdict or fetch RootCauseDb; it copies the approved public summary and advances the same stream.

EXECUTABLE ASSET 35-H - APPLY ROOTCAUSEVERDICTISSUED.V1

```
public static void Apply(
    RootCauseCaseSummaryRow row,
    RootCauseVerdictIssuedProjectionEvent e,
    DateTimeOffset appliedAtUtc)
{
    e.Stream.DemandPosition(row.LastAppliedStreamPosition + 1);
    e.Stream.DemandEntity(row.RootCauseCaseId);
    if (row.SiteId != e.SiteId || row.LineId != e.LineId)
        throw ProjectionConflict.Create("source-coordinate-drift");

    row.Status = "VerdictIssued";
    row.VerdictId = e.VerdictId;
    row.VerdictCode = e.VerdictCode;
    row.Confidence = e.Confidence;
    row.PolicyIdentity = e.PolicyIdentity;
    row.VerdictIdentity = e.VerdictIdentity;
    row.EvidenceCount = e.EvidenceCount;
    row.SourceOccurredAtUtc = e.OccurredAtUtc;
    row.LastAppliedAtUtc = appliedAtUtc;
    row.LastAppliedStreamPosition = e.Stream.Position;
    row.LastAppliedMessageId = e.MessageId;
}
```

GOLDEN RESULT

The row reaches VerdictIssued with VerdictId 019bc4d4-81c0-7ea1-82d5-4f9b8719d0a1, code COMMON_POWER_SUPPLY_INSTABILITY, confidence 0.94 and watermark position 2. This is Reporting observation, not a new verdict occurrence.

TX-REP

Projection and contiguous watermark commit together

For a contiguous delivery, receipt, event ledger, projection row and checkpoint must commit together. The broker acknowledgement follows the known ReportingDb outcome and cannot enter the transaction.

EXECUTABLE ASSET 35-I - ONE OWNER-LOCAL TX-REP

```
await using var tx = await db.Database.BeginTransactionAsync(ct);

db.InboxReceipts.Add(receipts.Completed(delivery));
db.ProjectionEvents.Add(ledger.Applied(delivery, generation));

var row = await rows.GetForUpdateAsync(
    generation, delivery.CaseId, ct);
reducers.Apply(row, delivery.Event, clock.GetUtcNow());

var checkpoint = await checkpoints.GetForUpdateAsync(
    Projection.Name, generation, delivery.StreamId, ct);
checkpoint.AdvanceTo(delivery.StreamPosition,
    delivery.MessageId, delivery.OccurredAtUtc, clock.GetUtcNow());

await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
return ConsumeOutcome.Committed(delivery.MessageId);
```

TX-REP ROW ORACLE 35-K

Table	Contiguous delta	Failure rule
messaging.InboxReceipt	+1	roll back
reporting.ProjectionEventLedger	+1	roll back
reporting.RootCauseCaseSummary	insert/update	roll back
reporting.ProjectionCheckpoint	advance exactly one	roll back
broker delivery	unacked during TX	ack only after known commit

ORDERING BOUNDARY

The watermark is per producer stream, never global

The only ordering coordinate trusted by the reducer is the producer-owned (StreamId, Position). Queue order is useful transport behavior, but redelivery, multiple consumers and replay can change processing order. Reporting never creates a global sequence across cases.

EXECUTABLE ASSET 35-J - THREE-WAY POSITION CLASSIFIER

```
public static PositionDisposition Classify(
    long lastContiguous, long incoming)
{
    if (incoming <= 0)
        return PositionDisposition.Invalid;
    if (incoming <= lastContiguous)
        return PositionDisposition.AlreadyApplied;
    if (incoming == lastContiguous + 1)
        return PositionDisposition.Contiguous;
    return PositionDisposition.Gap(
        expected: lastContiguous + 1,
        observed: incoming);
}
```

ORDERING CLAIMS 35-L

Coordinate	Allowed claim	Forbidden inference
case stream @ 2	positions 1..2 applied	all other case streams current
queue delivery tag	settlement on one channel	business sequence
OccurredAtUtc	producer occurrence time	arrival or apply order
AppliedAtUtc	Reporting commit time	source truth time
global wall clock	observation calculation	cross-service transaction

OUT-OF-ORDER DELIVERY

A gap is stored, not guessed away

If VerdictIssued position 2 arrives before CaseOpened position 1, applying it would manufacture a row without a baseline. Rejecting it permanently would also lose a valid fact. Reporting stores the exact admitted event as pending and records the gap in the same transaction.

EXECUTABLE ASSET 35-K - DURABLE GAP BUFFERING

```
private async Task<ConsumeOutcome> BufferGapAsync(
    AdmittedProjectionDelivery d,
    ProjectionCheckpointRow checkpoint,
    CancellationToken ct)
{
    db.InboxReceipts.Add(receipts.Completed(
        d, outcome: "buffered-gap"));
    db.PendingProjectionEvents.Add(
        PendingProjectionEventRow.FromExactDelivery(d, generation));

    checkpoint.ObserveGap(
        expectedPosition: checkpoint.LastContiguousPosition + 1,
        observedPosition: d.StreamPosition,
        observedAtUtc: clock.GetUtcNow());

    await db.SaveChangesAsync(ct);
    return ConsumeOutcome.BufferedGap(d.MessageId);
}
```

GAP STATE 35-M

Fact	Durable result
position 2 admitted	inbox + exact pending bytes/digest
position 1 missing	watermark remains 0; GapStartsAtPosition=1
console query	no fabricated verdict; updateStatus=gap-detected
broker delivery	may ack because exact event is durably owned

GAP RECOVERY

Apply only the contiguous pending prefix

When the missing position arrives, the projector applies it and then drains only the contiguous pending prefix. Every drained event enters the immutable ledger. A later gap stops the loop without skipping a position.

EXECUTABLE ASSET 35-L - CONTIGUOUS PENDING DRAIN

```
while (true)
{
    var next = checkpoint.LastContiguousPosition + 1;
    var pending = await db.PendingProjectionEvents
        .SingleOrDefaultAsync(x =>
            x.ProjectionName == Projection.Name &&
            x.GenerationId == generation &&
            x.StreamId == checkpoint.StreamId &&
            x.StreamPosition == next, ct);

    if (pending is null) break;

    var admitted = pending.RehydrateAndVerify();
    reducers.Apply(row, admitted.Event, clock.GetUtcNow());
    db.ProjectionEvents.Add(ledger.Applied(admitted, generation));
    db.PendingProjectionEvents.Remove(pending);
    checkpoint.AdvanceTo(next, admitted.MessageId,
        admitted.OccurredAtUtc, clock.GetUtcNow());
}

checkpoint.RecalculateGapFromPending();
await db.SaveChangesAsync(ct);
```

ATOMIC CATCH-UP

For the golden reversal, the transaction that admits CaseOpened @ 1 may also drain the already-owned VerdictIssued @ 2. The visible row and watermark jump from absent/@0 to VerdictIssued/@2 together; no query can observe @2 with an Open row.

IDEMPOTENCY AND CONFLICT

A duplicate is proved, never inferred from age

Transport identity and stream semantics answer different questions. A matching MessageId proves a replay of one delivery only after its digest matches; a new MessageId at an already applied stream position is resolved against the immutable event ledger.

TWO-KEY PLUS POSITION ORACLE 35-N

Incoming observation	Receipt	Ledger/view	Disposition
same ID + same digest	match	unchanged	duplicate; ack
same ID + changed digest	conflict	unchanged	terminal identity conflict
new ID + same position + same meaning	+1 evidence	unchanged	semantic duplicate
new ID + same position + changed meaning	+1 conflict	unchanged	position conflict
new ID + next position	+1	advance atomically	committed

EXECUTABLE ASSET 35-M - ALREADY-APPLIED RESOLUTION

```
var applied = await db.ProjectionEvents.SingleAsync(x =>
    x.ProjectionName == Projection.Name &&
    x.GenerationId == generation &&
    x.StreamId == delivery.StreamId &&
    x.StreamPosition == delivery.StreamPosition, ct);

if (!CryptographicOperations.FixedTimeEquals(
    applied.SemanticDigest, delivery.SemanticDigest))
    return ConsumeOutcome.IdentityConflict(
        delivery.MessageId, "stream-position-conflict");

db.InboxReceipts.Add(receipts.Completed(
    delivery, "semantic-duplicate"));
await db.SaveChangesAsync(ct);
return ConsumeOutcome.Committed(
    delivery.MessageId, "semantic-duplicate");
```

CONCURRENT CONSUMPTION

Provider constraints arbitrate competing projectors

Multiple Reporting instances may consume the broad queue. Correctness comes from database constraints and per-stream concurrency, not from assuming a single callback. An optional stable partition key can reduce collisions but never replaces the provider oracle.

EXECUTABLE ASSET 35-N - FRESH-CONTEXT RACE RESOLUTION

```
try
{
    return await projector.ApplyAsync(delivery, ct);
}
catch (DbUpdateConcurrencyException)
{
    await db.DisposeAsync();
    await using var fresh = factory.CreateDbContext();
    return await resolver.ResolveFromDurableStateAsync(
        fresh, delivery, ct);
}
catch (DbUpdateException ex)
    when (constraints.IsKnownProjectionConflict(ex))
{
    await db.DisposeAsync();
    await using var fresh = factory.CreateDbContext();
    return await resolver.ResolveFromDurableStateAsync(
        fresh, delivery, ct);
}
```

CONCURRENCY RULES 35-O

Rule	Reason
classify named constraints only	generic provider error is not duplicate proof
discard losing DbContext	tracked speculative state must not leak
reload receipt + ledger + checkpoint	durable winner decides outcome
return Unknown if no oracle row exists	never acknowledge an assumption
partition by StreamId only as optimization	correctness survives partition drift

BROKER SETTLEMENT**Acknowledgement follows a known ReportingDb outcome**

A buffered gap is a successful local ownership transfer: Reporting has durably accepted exact bytes and can acknowledge them. Invalid or conflicting input is rejected to terminal evidence. Unknown database outcomes remain unacknowledged until resolved.

EXECUTABLE ASSET 35-O - TYPED SETTLEMENT POLICY

```
public static DeliveryDisposition For(ConsumeOutcome outcome) =>
    outcome switch
    {
        ConsumeOutcome.Committed => DeliveryDisposition.Ack,
        ConsumeOutcome.Duplicate => DeliveryDisposition.Ack,
        ConsumeOutcome.BufferedGap => DeliveryDisposition.Ack,
        ConsumeOutcome.Transient => DeliveryDisposition.Retry,
        ConsumeOutcome.Unknown => DeliveryDisposition.Retry,
        ConsumeOutcome.Invalid => DeliveryDisposition.Reject,
        ConsumeOutcome.IdentityConflict => DeliveryDisposition.Reject,
        _ => throw new UnreachableException()
    };
```

SETTLEMENT MEANING 35-P

Local state	Broker action	Console effect
contiguous committed	ack	row/watermark advances
gap buffered	ack	status gap-detected; row does not jump
matching duplicate	ack	no visible change
commit unknown	retry/redelivery	status may degrade/unknown
identity conflict	reject + incident evidence	status failed

CHANNEL RULE

Delivery tags are channel-scoped. The Reporting adapter settles on the same long-lived channel that delivered the body and uses `multiple=false` under concurrent dispatch.

BFF QUERY BOUNDARY

Return an observation, not a distributed snapshot

The BFF endpoint answers one operator question: what has the Reporting projection observed for this case, and what evidence qualifies that observation now? It does not join service databases or synchronously validate the row against RootCause.

EXECUTABLE ASSET 35-P - VERSIONED OPERATOR QUERY

```
GET /api/operator/v1/root-cause-cases/{caseId}
```

```
Accept: application/vnd.nexus1.operator-case.v1+json
```

```
200 OK
```

```
Cache-Control: no-store
```

```
Content-Type: application/vnd.nexus1.operator-case.v1+json
```

BFF OWNERSHIP 35-Q

Concern	Owner	Rule
authentication + operator capability	BFF Edge	deny before query
case summary	Reporting projection	read one active generation
watermark	Reporting projector	return exact stream/position
update status	Reporting status policy	include reason and sample time
UI rendering	console	exhaustive state mapping

NO SYNCHRONOUS AUTHORITY CHECK

Calling RootCause to validate every projection query would reintroduce availability coupling and still would not create a common transaction. Repairs and reconciliation use explicit operational paths, not the interactive BFF request.

BFF CONTRACT

Value, watermark and freshness travel together

The V1 response makes three layers visible: the projected business summary, the contiguous source watermark and the operational freshness assessment. Every time value carries a distinct name and meaning.

EXECUTABLE ASSET 35-Q - BFF RESPONSE CONTRACTS

```
public sealed record OperatorCaseV1(  
    string ObservationState,  
    RootCauseCaseSummaryV1? Data,  
    ProjectionFreshnessV1 Freshness,  
    DateTimeOffset ObservedAtUtc);  
  
public sealed record ProjectionFreshnessV1(  
    string ProjectionName,  
    string ContractSetIdentity,  
    string ProjectionGenerationId,  
    string UpdateStatus,  
    string ReasonCode,  
    long? ProjectionAgeMs,  
    long? RepresentedEventAgeMs,  
    long StatusObservationAgeMs,  
    StreamWatermarkV1? Watermark,  
    DateTimeOffset? LastAppliedAtUtc,  
    DateTimeOffset StatusObservedAtUtc);  
  
public sealed record StreamWatermarkV1(  
    string StreamId, long LastContiguousPosition);
```

STABLE VOCABULARY 35-R

Field	Meaning
ObservationState	observed not-observed-yet
UpdateStatus	current-as-observed delayed catching-up gap-detected failed unknown
ProjectionAgeMs	now minus last Reporting apply for this row
RepresentedEventAgeMs	now minus newest represented source occurrence
StatusObservationAgeMs	now minus operational status sample

FRESHNESS SEMANTICS

Age measures elapsed time, not completeness

Projection age is not source lag. A verdict issued yesterday can be projected one second ago; its represented event age is one day while projection age is one second. Conversely, a quiet healthy case can have a large projection age without a known backlog. Status needs the other evidence planes.

FOUR-CLOCK LEDGER 35-S

Coordinate	Owner	Calculation / claim
OccurredAtUtc	RootCause	when represented fact occurred
LastAppliedAtUtc	Reporting TX-REP	when row/watermark committed
StatusObservedAtUtc	status sampler	when backlog/heartbeat were observed
ObservedAtUtc	BFF	single clock used for response ages

EXECUTABLE ASSET 35-R - SATURATING AGE CALCULATION

```
public static long AgeMs(
    DateTimeOffset observedAtUtc,
    DateTimeOffset earlierUtc)
{
    var delta = observedAtUtc - earlierUtc;
    if (delta <= TimeSpan.Zero) return 0;
    return checked((long)Math.Min(
        delta.TotalMilliseconds, long.MaxValue));
}

var projectionAge = row is null
    ? (long?)null
    : AgeMs(now, row.LastAppliedAtUtc);
var representedEventAge = row is null
    ? (long?)null
    : AgeMs(now, row.SourceOccurredAtUtc);
```

CLOCK-SKEW HANDLING

Negative age is clamped for display and emits a bounded clock-skew signal. It never upgrades status to current; a stale status sample still forces Unknown.

UPDATE STATUS

Current is the last state earned, not the default

Current is deliberately named current-as-observed. It means the sampled Reporting input path has no known gap, backlog or failure inside the policy horizon. It does not prove that RootCause has published every fact it will ever publish.

EXECUTABLE ASSET 35-S - FAIL-CLOSED STATUS CLASSIFIER

```
public ProjectionUpdateStatus Assess(
    ProjectionStatusSnapshot s,
    DateTimeOffset now,
    ProjectionFreshnessPolicy p)
{
    if (now - s.ObservedAtUtc > p.MaxStatusAge)
        return Unknown("status-sample-stale");
    if (s.FailureCode is not null)
        return Failed(s.FailureCode);
    if (s.GapCount > 0)
        return GapDetected("stream-gap");
    if (s.ConsumerHeartbeatAtUtc is null ||
        now - s.ConsumerHeartbeatAtUtc > p.MaxHeartbeatAge)
        return Unknown("consumer-heartbeat-stale");
    if (s.OldestReadyAge is { } oldest && oldest > p.WarningAge)
        return Delayed("oldest-ready-over-slo");
    if (s.ReadyCount > 0 || s.UnackedCount > 0)
        return CatchingUp("reporting-backlog");
    return CurrentAsObserved("no-known-local-lag");
}
```

STATUS PRECEDENCE 35-T

Precedence	State	Why it wins
1	unknown	sample too old to classify
2	failed	terminal/local incident known
3	gap-detected	semantic completeness broken
4	unknown	consumer liveness unproved
5	catching-up/delayed	work known outstanding
6	current-as-observed	only after all negative gates pass

OPERATIONAL OBSERVATION

Status is sampled evidence, not a request-time fan-in

The BFF does not query the broker on every user request. A bounded Reporting monitor samples the dedicated queue, projector heartbeat and durable gap count, then writes one operational status row. The query returns both sample time and its age.

EXECUTABLE ASSET 35-T - BOUNDED STATUS SAMPLER

```
public async Task SampleAsync(CancellationTokentoken ct)
{
    var observedAt = clock.UtcNow();
    var queue = await broker.ReadQueueAsync(
        "reporting.integration-events.v1", ct);
    var heartbeat = await heartbeats.ReadAsync(Projection.Name, ct);
    var gaps = await db.PendingProjectionEvents
        .Where(x => x.ProjectionName == Projection.Name &&
            x.GenerationId == generations.ActiveId)
        .LongCountAsync(ct);

    await statuses.UpsertAsync(new ProjectionStatusSnapshot(
        Projection.Name, observedAt, heartbeat.LastSeenAtUtc,
        queue.MessagesReady, queue.MessagesUnacknowledged,
        queue.OldestReadyAge, gaps, heartbeat.FailureCode), ct);
}
```

SAMPLING LIMITS 35-U

Design	Limit
bounded cadence	no management API call per BFF query
dedicated status credential	read queue statistics; cannot consume/publish
sample timestamp returned	caller can detect stale monitoring evidence
database write failure	prior sample ages into Unknown
broker API unavailable	status becomes Unknown, not Current

ABSENCE IS NOT NONEXISTENCE

Not observed yet is a first-class answer

No projection row is ambiguous in an eventually consistent system. It may mean the case never existed, the event has not arrived, the projection is rebuilding, a gap blocks application or the query uses the wrong scope. The BFF therefore does not turn row absence into a confident business 404.

EXECUTABLE ASSET 35-U - EXPLICIT NOT-OBSERVED RESPONSE

```
{
  "observationState": "not-observed-yet",
  "data": null,
  "freshness": {
    "projectionName": "root-cause-case-summary",
    "contractSetIdentity": "reporting/root-cause-case-summary/1",
    "updateStatus": "catching-up",
    "reasonCode": "reporting-backlog",
    "projectionAgeMs": null,
    "representedEventAgeMs": null,
    "statusObservationAgeMs": 184,
    "watermark": null,
    "lastAppliedAtUtc": null,
    "statusObservedAtUtc": "2026-07-19T07:48:00.000Z"
  },
  "observedAtUtc": "2026-07-19T07:48:00.184Z"
}
```

ABSENCE SEMANTICS 35-V

Condition	HTTP / body
row absent; path healthy	200 + not-observed-yet/current-as-observed
row absent; backlog/gap	200 + not-observed-yet/catching-up or gap-detected
Reporting unavailable	503 Problem Details; no fabricated stale body
invalid UUID	400 versioned Problem Details
unauthorized scope	403 without projection disclosure

BFF APPLICATION

One query composes only Reporting-owned evidence

The application query captures one BFF observation time, reads the active generation, row, checkpoint and status snapshot, then derives ages and status without side effects. Authorization scope is supplied by Edge and enforced in the query.

EXECUTABLE ASSET 35-V - QUERY HANDLER

```
public async Task<OperatorCaseV1> HandleAsync(
    GetOperatorCase query, CancellationToken ct)
{
    query.Authorization.Demand(
        "reporting:root-cause-case:read", query.SiteScope);
    var observedAt = clock.UtcNow();
    var active = await generations.ReadActiveAsync(ct);
    var row = await cases.TryReadAsync(
        active.Id, query.CaseId, query.SiteScope, ct);
    var checkpoint = row is null ? null :
        await checkpoints.ReadAsync(active.Id,
            row.ProducerStreamId, ct);
    var sample = await statuses.TryReadAsync(Projection.Name, ct)
        ?? ProjectionStatusSnapshot.Unknown(observedAt);

    var update = freshness.Assess(sample, observedAt, policy);
    return responses.Map(row, checkpoint, sample,
        active, update, observedAt);
}
```

QUERY INVARIANTS 35-W

Invariant	Enforcement
one active generation	generation pointer read first
scope before row disclosure	site predicate in repository query
one ObservedAtUtc	all ages share BFF clock sample
no write	read-only credential/context and no SaveChanges
status absent/stale	fail closed to Unknown

HTTP EDGE

No-store prevents freshness metadata from going stale in cache

The HTTP adapter freezes media type, Problem Details and cache behavior. Phase 0 sends Cache-Control: no-store because a cached response could preserve old freshness metadata while appearing to be a new console refresh.

EXECUTABLE ASSET 35-W - MINIMAL API ENDPOINT

```
app.MapGet("/api/operator/v1/root-cause-cases/{caseId:guid}",
    async (Guid caseId, ClaimsPrincipal user,
        IOperatorCaseQueries queries, HttpContext http,
        CancellationToken ct) =>
    {
        var scope = SiteScope.FromClaims(user);
        var result = await queries.GetAsync(caseId, scope, ct);
        http.Response.Headers.CacheControl = "no-store";
        return Results.Json(result,
            statusCode: StatusCodes.Status200OK,
            contentType:
                "application/vnd.nexus1.operator-case.v1+json");
    })
    .RequireAuthorization("operator-case-read-v1")
    .Produces<OperatorCaseV1>(200,
        "application/vnd.nexus1.operator-case.v1+json")
    .ProducesProblem(400).ProducesProblem(403).ProducesProblem(503);
```

HTTP CONTRACT 35-X

Observation	Response
projected or not observed	200 typed observation
invalid route value/media	400/406 versioned problem
scope denied	403 problem; no row hint
Reporting query unavailable	503 problem + Retry-After by policy
authenticated response	Cache-Control: no-store

QUERY SECURITY

Freshness does not bypass least privilege

Reporting contains operationally useful identifiers and a verdict summary, but the BFF still applies least privilege. It returns no raw envelope, evidence membership digest, internal failure detail, queue name, database key or credential material.

AUTHORIZATION AND MINIMIZATION 35-Y

Layer	Allowed	Denied
Reporting consumer	read Reporting queue; write ReportingDb	RootCause/Audit/Compliance DBs
status monitor	read queue stats; write status row	consume/publish payloads
BFF query	read active Reporting projection by site	write projection or call owner mutations
operator	approved site/case summary	other site, raw bytes, secrets
evidence reviewer	manifests and bounded diagnostics	production payload dump

EXECUTABLE ASSET 35-X - SITE SCOPE IS A QUERY PREDICATE

```
public Task<RootCauseCaseSummaryRow?> TryReadAsync(
    Guid generationId,
    Guid caseId,
    SiteScope scope,
    CancellationToken ct) =>
    db.RootCauseCaseSummaries
        .AsNoTracking()
        .SingleOrDefaultAsync(x =>
            x.ProjectionGenerationId == generationId &&
            x.RootCauseCaseId == caseId &&
            scope.AllowedSiteIds.Contains(x.SiteId), ct);
```

SIDE-CHANNEL LIMIT

A denied case and a nonexistent case share the same external authorization failure policy. Logs retain a bounded reason code under protected access; the response does not disclose cross-site existence.

CONSOLE CONTRACT

The UI has no optimistic default state

The console consumes a discriminated update status. There is no fallback branch that paints an unknown value green. Labels describe Reporting observation explicitly: Current as observed, Catching up, Delayed, Gap detected, Failed or Status unknown.

EXECUTABLE ASSET 35-Y - EXHAUSTIVE CONSOLE STATE MAPPING

```
type UpdateStatus =  
  | "current-as-observed"  
  | "catching-up"  
  | "delayed"  
  | "gap-detected"  
  | "failed"  
  | "unknown";  
  
function badge(status: UpdateStatus): BadgeModel {  
  switch (status) {  
    case "current-as-observed": return observedCurrent();  
    case "catching-up": return catchingUp();  
    case "delayed": return delayed();  
    case "gap-detected": return gapDetected();  
    case "failed": return failed();  
    case "unknown": return unknown();  
    default: return assertNever(status);  
  }  
}
```

CONSOLE COPY 35-Z

State	Visible wording	Forbidden wording
current-as-observed	Current as observed; watermark @ N	Live / authoritative
catching-up	Updating; N queued	Current
delayed	Delayed; last update X ago	No issues
gap-detected	Update gap detected	Complete
failed/unknown	Update failed / status unknown	Fresh

CONSOLE TESTS

Visual confidence must be contract-tested

Contract tests prove that each BFF state survives serialization and is rendered with the correct accessibility text. The most important negative test is that delayed, failed and unknown responses never reuse the success color or label.

EXECUTABLE ASSET 35-Z - UI TRUTHFULNESS TESTS

```
describe.each([
  ["catching-up", "Updating", "Current"],
  ["delayed", "Delayed", "Current"],
  ["gap-detected", "Update gap detected", "Complete"],
  ["failed", "Update failed", "Fresh"],
  ["unknown", "Status unknown", "Current"]
])("projection status %s", (status, visible, forbidden) => {
  const view = renderCase(goldenResponse({ updateStatus: status }));
  expect(view.getByRole("status")).toHaveTextContent(visible);
  expect(view.getByRole("status")).not.toHaveTextContent(forbidden);
  expect(view.getByTestId("projection-watermark"))
    .toHaveTextContent("position 2");
});

it("renders absence as not observed, never not found", () => {
  const view = renderCase(goldenNotObservedResponse());
  expect(view.getByText("Not observed yet")).toBeVisible();
});
```

ACCESSIBILITY PROOF 35-AA

Surface	Required evidence
color	never sole carrier of freshness state
status role	announces transition after refresh
age	human label plus exact observed timestamp
watermark	text available outside tooltip
auto-refresh	does not erase failed/unknown state

REPORTING OFFLINE

Delayed observation cannot roll back owner truth

Stopping Reporting before RootCause publishes must not roll back the verdict, Audit evidence or Compliance review. The Reporting queue retains its copy. During the outage the prior projection is served only with a delayed/unknown status that records the observation horizon.

EXECUTABLE ASSET 35-AA - REPORTING OUTAGE SCENARIO

```
[Fact]
public async Task REPORT_OUTAGE_001_truth_survives_reporting_offline()
{
    await fixture.StopReportingAsync();
    var verdict = await fixture.RootCause.IssueGoldenVerdictAsync();
    await fixture.RootCause.WaitUntilDispatchedAsync(verdict.MessageId);
    await fixture.Audit.WaitForEvidenceAsync(verdict.VerdictId);
    await fixture.Compliance.WaitForReviewAsync(verdict.VerdictId);

    await fixture.Broker.AssertReadyAsync(
        "reporting.integration-events.v1", verdict.MessageId);
    await fixture.RootCause.AssertVerdictAsync(verdict.VerdictId);
    await fixture.Reporting.AssertWatermarkBelowAsync(STREAM_ID, 2);

    var view = await fixture.Bff.GetCaseAsync(CASE_ID);
    view.Freshness.UpdateStatus.Should().NotBe("current-as-observed");
}
```

OFFLINE STATE VECTOR 35-AB

Owner	Expected state
RootCause	verdict committed; unchanged
Audit	evidence may commit
Compliance	review may open
broker	Reporting copy ready
Reporting/BFF	old/absent row plus delayed or unknown status

CATCH-UP

Currentness is earned after convergence

After restart, status moves through CatchingUp and reaches CurrentAsObserved only after the queue sample is empty, the heartbeat is fresh, no durable gap remains and the golden case watermark is 2. A timer alone cannot authorize the transition.

EXECUTABLE ASSET 35-AC - CATCH-UP CONVERGENCE

```
[Fact]
public async Task REPORT_REC_001_restart_converges_with_evidence()
{
    await fixture.StartReportingAsync();
    await fixture.Reporting.WaitForStatusAsync("catching-up");
    await fixture.Reporting.WaitForWatermarkAsync(STREAM_ID, 2);
    await fixture.Broker.WaitUntilEmptyAsync(
        "reporting.integration-events.v1");
    await fixture.Reporting.WaitForFreshStatusSampleAsync();

    var view = await fixture.Bff.GetCaseAsync(CASE_ID);
    view.Data!.VerdictId.Should().Be(VERDICT_ID);
    view.Freshness.Watermark!.LastContiguousPosition.Should().Be(2);
    view.Freshness.UpdateStatus.Should().Be("current-as-observed");
    view.Freshness.ReasonCode.Should().Be("no-known-local-lag");
}
```

TRANSITION GATES 35-AC

From -> to	Required observation
unknown -> catching-up	fresh heartbeat + backlog above zero
catching-up -> gap-detected	durable pending gap exists
gap-detected -> catching-up	gap drained; backlog remains
catching-up -> current-as-observed	fresh sample; zero ready/unacked/gaps
any -> failed	terminal conflict or projector failure code

CRASH SEMANTICS

Row and watermark fail or commit together

Every crash point has a local recovery rule. The invariant is that the visible row and contiguous watermark never disagree, while exact pending bytes survive a gap acknowledgement.

CRASH MATRIX 35-AD

Crash point	Durable state	Recovery
before admission	none	broker redelivery
after copy, before TX	none	redelivery; revalidate bytes
during contiguous TX	all or none	redelivery; receipt/ledger resolution
after commit, before ack	receipt + ledger + row + checkpoint	duplicate then ack
during gap buffer TX	all or none	redelivery or durable pending event
after gap commit, before ack	receipt + pending + gap	duplicate then ack
during status sample	prior sample	ages to Unknown; next sample replaces
during BFF query	no write	request fails/retries safely

EXECUTABLE ASSET 35-AD - NAMED PROJECTION CHECKPOINTS

```
public enum ReportingCheckpoint
{
    BodyCopied,
    AdmissionCompleted,
    TxRepStarted,
    InboxInserted,
    ProjectionReduced,
    WatermarkAdvanced,
    TxRepCommitted,
    BrokerAckCompleted,
    StatusSampleCommitted,
    BffResponseWritten
}
```

PROVIDER PROOF

Atomic projection claims require the real provider

Provider-backed tests prove atomicity on the selected relational provider. An in-memory store cannot reproduce named constraints, rowversion races, rollback, execution plans or permission boundaries.

EXECUTABLE ASSET 35-AE - SQL SERVER ATOMICITY

```
[Fact]
public async Task REPORT_DB_001_verdict_and_watermark_commit_once()
{
    await fixture.SeedCaseOpenedAtPositionOneAsync();
    var delivery = Corpus.VerdictIssuedV1.ForReporting();

    await fixture.Reporting.ConsumeAsync(delivery);

    await fixture.ReportingDb.AssertInboxAsync(delivery.MessageId, 1);
    await fixture.ReportingDb.AssertLedgerAsync(STREAM_ID, 2, 1);
    await fixture.ReportingDb.AssertCaseVerdictAsync(CASE_ID, VERDICT_ID);
    await fixture.ReportingDb.AssertWatermarkAsync(STREAM_ID, 2);
}

[Fact]
public async Task REPORT_DB_002_reducer_failure_rolls_back_all_rows()
{
    await fixture.SeedCaseOpenedAtPositionOneAsync();
    fixture.ReportingDb.FailUpdate("reporting.RootCauseCaseSummary");
    await Assert.ThrowsAsync<InjectedDatabaseFailure>(() =>
        fixture.Reporting.ConsumeGoldenVerdictAsync());
    await fixture.ReportingDb.AssertNoPositionTwoDeltaAsync(STREAM_ID);
}
```

REQUIRED PROVIDER CASES 35-AF

Case	Oracle
same MessageId race	one commit + one matching duplicate
same position/different digest	conflict; original row unchanged
rowversion collision	fresh-context durable resolution
commit response lost	receipt/ledger/checkpoint resolve or Unknown
permission probe	BFF read principal cannot mutate ReportingDb

BROKER-BACKED ORDERING PROOF**Out-of-order arrival cannot create out-of-order truth**

The broker laboratory publishes the exact Chapter 33 and 34 bodies to the real topology in both orders. It proves that Reporting never advances over a missing stream position and that the same VerdictIssued bytes reach all three subscriber queues.

EXECUTABLE ASSET 35-AG - REORDERED DELIVERY LABORATORY

```
[Fact]
public async Task REPORT_BROKER_001_position_two_waits_for_one()
{
    await broker.PublishExactAsync(Corpus.VerdictIssuedV1);
    await reporting.WaitForBufferedAsync(STREAM_ID, position: 2);
    await reportingDb.AssertWatermarkAsync(STREAM_ID, 0);
    (await bff.GetCaseAsync(CASE_ID)).Freshness.UpdateStatus
        .Should().Be("gap-detected");

    await broker.PublishExactAsync(Corpus.CaseOpenedV1);
    await reporting.WaitForWatermarkAsync(STREAM_ID, 2);

    await reportingDb.AssertNoPendingAsync(STREAM_ID);
    await reportingDb.AssertCaseVerdictAsync(CASE_ID, VERDICT_ID);
}

[Fact]
public async Task REPORT_BROKER_002_verdict_body_is_shared()
{
    await broker.AssertSameBodyOnRoutesAsync(
        VERDICT_MESSAGE_ID, EXPECTED_VERDICT_BODY_SHA256,
        "audit.root-cause-verdicts.v1",
        "compliance.root-cause-verdicts.v1",
        "reporting.integration-events.v1");
}
```

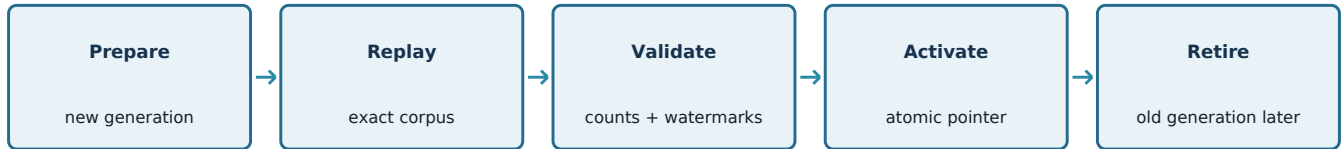
BROKER CLAIM

Queue routing and ordering evidence does not prove ReportingDb commit. The harness records broker observations and provider observations separately, then correlates them by the frozen MessageId and body digest.

PROJECTION REBUILD

Rebuild beside the active generation

A projection rebuild never truncates the generation currently served. Reporting creates a new generation, replays an authorized immutable corpus into shadow rows, validates completeness, then switches one active-generation pointer atomically.

REBUILD STATE MACHINE 35-AH**EXECUTABLE ASSET 35-AH - GENERATION CREATION**

```
var generation = ProjectionGeneration.Start(  
  projectionName: Projection.Name,  
  contractSetIdentity:  
    RootCauseCaseSummaryContractSet.Identity,  
  contractSetDigest:  
    RootCauseCaseSummaryContractSet.Digest,  
  reducerIdentity: "RootCauseCaseSummaryReducer/1",  
  sourceCorpusIdentity: replay.CorporusIdentity,  
  startedAtUtc: clock.GetUtcNow());  
  
await generations.InsertPreparingAsync(generation, ct);  
await replay.ProjectIntoAsync(generation.Id, ct);  
await validations.RecordAsync(  
  generation.Id, await validator.ValidateAsync(generation.Id, ct), ct);
```

REBUILD EXCLUSIONS 35-AI

Forbidden shortcut	Failure
truncate active rows	console loses the last known observation
rebuild from mutable owner tables	historical public meaning can drift
activate by elapsed time	incomplete generation becomes visible
reuse generation ID	old/new evidence cannot be separated

GENERATION ACTIVATION

Only validated shadow truth becomes visible

Activation is a database decision guarded by a validation manifest. Every stream watermark in the new generation must match the approved replay horizon, no gap/conflict may remain and golden queries must match expected semantic digests.

EXECUTABLE ASSET 35-AI - FAIL-CLOSED GENERATION SWITCH

```
await using var tx = await db.Database.BeginTransactionAsync(ct);
var candidate = await generations.GetForUpdateAsync(candidateId, ct);
var active = await generations.GetActiveForUpdateAsync(ct);
var proof = await validations.GetRequiredAsync(candidateId, ct);

proof.DemandPassed(
    zeroGaps: true,
    zeroIdentityConflicts: true,
    expectedContractSetDigest:
        RootCauseCaseSummaryContractSet.Digest,
    expectedReplayHorizon: replay.ApprovedHorizon);

active?.Retire(clock.GetUtcNow());
candidate.Activate(clock.GetUtcNow());
await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

GENERATION VALIDATION 35-AJ

Evidence	Reject when
contract/reducer identities	digest differs from deployed reader
per-stream watermarks	below approved replay horizon
pending gaps/conflicts	nonzero
row count and semantic digest	unexpected delta against oracle
golden BFF corpus	field/status mapping drifts
source corpus	bytes, retention or provenance incomplete

QUERY PERFORMANCE

Projection shape follows an explicit read contract

The projection is designed for the operator query and its bounded filters. Indexes are owned by Reporting and may evolve without changing RootCause. Performance evidence binds a query shape, provider version, data volume and execution plan.

EXECUTABLE ASSET 35-AJ - QUERY-ORIENTED INDEXES

```
model.Entity<RootCauseCaseSummaryRow>()
    .HasIndex(x => new {
        x.ProjectionGenerationId,
        x.SiteId,
        x.Status,
        x.LastAppliedAtUtc
    })
    .HasDatabaseName("IX_RC_Summary_Site_Status_Updated");

model.Entity<RootCauseCaseSummaryRow>()
    .HasIndex(x => new {
        x.ProjectionGenerationId,
        x.VerdictId
    })
    .HasDatabaseName("UX_RC_Summary_Generation_Verdict")
    .IsUnique()
    .HasFilter("[VerdictId] IS NOT NULL");
```

PERFORMANCE PROOF 35-AK

Case	Required observation
case-by-ID BFF query	bounded seek under active generation + site
site/status list	stable ordering + cursor, no unbounded scan
status join	single-row lookup; no broker call
rebuild write load	active query latency remains inside SLO
plan regression	captured plan/hash and representative cardinality

PHASE 0 SCOPE

This chapter specifies the case-by-ID BFF query. A sortable cross-site reporting catalogue needs its own pagination, authorization, export and load contract; it is not silently inferred from this index.

OBSERVABILITY**Measure delay without creating cardinality or data leaks**

Telemetry distinguishes delivery, reduction, buffering, query and freshness assessment. Caseld, VerdictId, Messageld, StreamId and correlation remain in bounded logs/traces, never metric labels.

BOUNDED SIGNALS 35-AL

Signal	Meaning	Allowed labels
reporting.delivery	consumer disposition	contract,outcome
reporting.apply.duration	TX-REP reducer latency	projection,event_type
reporting.pending.gaps	durable missing positions	projection
reporting.watermark.advance	contiguous progress count	projection,event_type
reporting.projection.age	BFF row age distribution	projection,status
reporting.query.duration	BFF query latency	route,outcome
reporting.status.sample_age	monitor evidence age	projection

EXECUTABLE ASSET 35-AK - BOUNDED QUERY ACTIVITY

```
using var activity = Sources.Reporting.StartActivity(
    "query operator case", ActivityKind.Server);
activity?.SetTag("nexus1.projection",
    "root-cause-case-summary");
activity?.SetTag("nexus1.projection.generation", generation.Id);
activity?.SetTag("nexus1.update_status", result.UpdateStatus);
activity?.SetTag("nexus1.observation_state", result.ObservationState);

logger.LogInformation(
    "Reporting query {Outcome} for case hash {CaseHash}; " +
    "watermark {Position}; status {UpdateStatus}",
    outcome, hashes.BoundedCase(caseId), position, updateStatus);
```

PAYLOAD SAFETY

Raw envelope bytes, evidence membership digests, verdict payloads and authorization claims are not logged. Protected incident evidence retains only the minimum exact bytes needed under a separate access/retention policy.

HEALTH AND READINESS

Projection freshness is not process liveness

Reporting has three different health questions: can the process run, can it consume and commit, and can the projection satisfy its freshness objective? RootCause readiness never depends on any of them.

HEALTH SURFACES 35-AM

Probe	Checks	Does not check
/health/live	process-local event loop	broker/DB/backlog
/health/ready	ReportingDb + consumer channel + contract set	RootCause/Audit/Compliance
projection status	heartbeat, backlog, gaps, failure	process restart policy
BFF dependency	Reporting query availability	source owner currentness

EXECUTABLE ASSET 35-AL - ROLE-SPECIFIC READINESS

```
public ReportingReadiness Assess(ReportingRuntime r)
{
    if (!r.ContractSetDigestMatches)
        return NotReady("projection-contract-drift");
    if (!r.ReportingDbReachable)
        return NotReady("reporting-db-unavailable");
    if (!r.ConsumerChannelOpen)
        return NotReady("reporting-consumer-unavailable");
    if (r.IdentityConflicts > 0)
        return NotReady("projection-identity-conflict");
    return Ready();
}

// A large but processable backlog degrades projection freshness.
// It does not make RootCause or the Reporting process non-live.
```

INDEPENDENT DEPLOYMENT

A Reporting readiness failure removes only Reporting replicas from their service path. It cannot block RootCause verdict issuance, relay dispatch, Audit append or Compliance review creation.

RETENTION AND REPAIR

A projection can be rebuilt only from trustworthy history

Replay safety depends on retaining enough evidence to distinguish transport duplicate, semantic repetition and identity conflict. Pruning the inbox or ledger without a complete replacement oracle turns an old fact into an apparently new mutation.

LIFECYCLE POLICY 35-AN

Asset	Retention / repair rule
inbox receipt	through maximum broker/DLQ replay horizon
event ledger	through projection generation audit/repair horizon
pending event bytes	until gap resolved or authorized terminal disposition
active projection	serve until validated successor activates
retired generation	retain by evidence/rollback policy, then purge explicitly
status samples	bounded operational history; not business evidence

EXECUTABLE ASSET 35-AM - AUTHORIZED EXACT REPLAY

```
public async Task ReplayAsync(
    ProjectionReplayEvidence evidence,
    ReplayAuthorization authorization,
    CancellationToken ct)
{
    authorization.Demand("reporting:projection:replay");
    evidence.DemandOriginalIdentityAndBytes();
    evidence.DemandContractSet(
        RootCauseCaseSummaryContractSet.Identity);

    await replay.PublishToOriginalQueueAsync(
        "reporting.integration-events.v1",
        evidence.MessageId,
        evidence.EnvelopeBytes,
        authorization.OperationId,
        ct);
}
```

REPAIR EXCLUSIONS

Never allocate a new MessageId for replay, rebuild an old body from current RootCauseDb, edit a watermark by hand, copy an Audit/Compliance row into ReportingDb or mark a generation Current without its validation manifest.

ARCHITECTURE ENFORCEMENT

The delayed copy cannot become a command authority

Architecture tests make the read-side boundary executable. Reporting references public contracts and its own layers; the BFF references the Reporting query contract. No command handler may depend on the projection.

EXECUTABLE ASSET 35-AN - DEPENDENCY AND CONNECTION GATES

```
[Fact]
public void REPORT_ARCH_001_projection_has_no_owner_implementation_ref()
{
    Types.InAssembly(typeof(RootCauseCaseProjector).Assembly)
        .ShouldNot().HaveDependencyOnAny(
            "RootCause.Domain", "RootCause.Application",
            "RootCause.Infrastructure", "Audit.Infrastructure",
            "Compliance.Infrastructure")
        .GetResult().IsSuccessful.Should().BeTrue();
}

[Fact]
public void REPORT_ARCH_002_commands_do_not_read_projection()
{
    Types.InAssemblies(CommandAssemblies.All)
        .ShouldNot().HaveDependencyOnAny(
            "Reporting.Projections", "ReportingDbContext")
        .GetResult().IsSuccessful.Should().BeTrue();
}

[Fact]
public void REPORT_ARCH_003_host_owns_one_database()
{
    HostManifest.Reporting.ConnectionPurposes
        .Should().Equal("ReportingDb");
}
```

RUNTIME NEGATIVE EVIDENCE 35-AO

Observation	Required result
SQL session inventory	Reporting principals reach only ReportingDb
HTTP client inventory	projector has no owner mutation/query client
assembly graph	contracts only; no service implementation references
broker ACL	consumer reads Reporting queue only
command trace	zero Reporting projection database reads

PRODUCTION-PATH VERIFICATION

The HTTP result is tied back to exact public bytes

The production-path test starts the real Reporting host and BFF, applies real migrations, declares the approved topology, publishes the two frozen bodies and queries the HTTP endpoint. Test doubles control time and failure checkpoints, not business behavior.

EXECUTABLE ASSET 35-AO - FULL PROJECTION/BFF CONTRACT TEST

```
[Fact]
public async Task REPORT_E2E_001_golden_case_is_honestly_observed()
{
    await fixture.StartProductionPathAsync();
    await fixture.PublishExactAsync(Corpus.CaseOpenedV1);
    await fixture.PublishExactAsync(Corpus.VerdictIssuedV1);
    await fixture.WaitForProjectionAsync(STREAM_ID, 2);
    await fixture.SampleStatusAsync();

    var response = await fixture.Bff.GetAsync(
        $"/api/operator/v1/root-cause-cases/{CASE_ID}");
    response.StatusCode.Should().Be(HttpStatusCode.OK);
    response.Headers.CacheControl!.NoStore.Should().BeTrue();

    var body = await response.ReadStrictV1Async<OperatorCaseV1>();
    body.Data!.VerdictId.Should().Be(VERDICT_ID);
    body.Freshness.Watermark!.LastContiguousPosition.Should().Be(2);
    body.Freshness.UpdateStatus.Should().Be("current-as-observed");
}
```

HARNESS EVIDENCE 35-AP

Artifact	Must bind
runtime identity	build/config/package/provider/broker identities
golden corpus	both exact bytes and digests
database trace	TX-REP rows and watermark
broker trace	three routes and Reporting settlement
HTTP corpus	media, status, no-store, body schema
console test	every update state and no optimistic fallback

FAIL-CLOSED EVIDENCE

A designed projection is not an executed projection

This chapter defines executable assets and required evidence; it does not claim they ran in the NEXUS-1 repository or a provider/broker environment. The manifest remains PENDING until every required artifact is executed against one recorded build.

EXECUTABLE ASSET 35-AP - FAIL-CLOSED EVIDENCE MANIFEST

```
{
  "claimId": "REPORTING-DELAYED-TRUTH-V1-REVIEW-001",
  "status": "PENDING",
  "bookChapter": 35,
  "projection": "reporting/root-cause-case-summary/1",
  "goldenInputs": {
    "caseOpenedBodySha256":
      "454f51cf4b0250b9c855346a6648a7d73530f19687ee3b52cda2edf9d0339c5b",
    "verdictIssuedBodySha256":
      "192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00"
  },
  "requiredArtifacts": [
    "reporting-contract-and-topology.trx",
    "reporting-provider-atomicity-and-races.trx",
    "reporting-gap-reorder-and-recovery.trx",
    "reporting-outage-and-catchup.trx",
    "reporting-rebuild-generation-switch.trx",
    "reporting-bff-http-contract.trx",
    "operator-console-freshness-contract.xml",
    "reporting-security-architecture-telemetry.trx",
    "runtime-identity.json"
  ],
  "executedArtifacts": [],
  "missingReason": "reference repository and runtime lab not executed here"
}
```

VALIDATOR REJECTS 35-AQ

Condition	Reason
zero/skipped tests	absence is not success
one input digest differs	different public fact corpus
provider/broker/build identity absent	behavior is unbound
gap/rebuild/outage case missing	freshness claim incomplete
console states not exhaustive	UI may present false currentness

ADR, PROOF BOUNDARY AND CHECKPOINT

Delayed truth stays useful only when delay is visible

ADR-033 35-AR

Field	Decision
status	Accepted for implementation; runtime evidence PENDING
decision	two approved RootCause V1 facts -> TX-REP -> watermark/status -> BFF
ordering	per producer stream; gaps buffered durably; no global order
freshness	ages + status sample; current-as-observed only after negative gates
absence	not-observed-yet; no confident business 404
rebuild	shadow generation + validation + atomic active pointer
rejected	fan-in query, projection commands, optimistic UI, cached freshness

OFFICIAL TECHNICAL ANCHORS

- RabbitMQ exchanges and routing: <https://www.rabbitmq.com/docs/4.1/exchanges>
- RabbitMQ queues and ordering: <https://www.rabbitmq.com/docs/queues>
- RabbitMQ acknowledgements and confirms: <https://www.rabbitmq.com/docs/confirmations>
- RabbitMQ .NET client guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>
- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- EF Core optimistic concurrency: <https://learn.microsoft.com/ef/core/saving/concurrency>
- ASP.NET Core response caching: <https://learn.microsoft.com/aspnet/core/performance/caching/response>
- RFC 9111 HTTP Caching and RFC 9205 HTTP protocol guidance: <https://www.rfc-editor.org/rfc/rfc9111> and <https://www.rfc-editor.org/rfc/rfc9205>
- OpenTelemetry messaging semantic conventions: <https://opentelemetry.io/docs/specs/semconv/messaging/>

WHAT THIS CHAPTER HAS PROVED

Chapter 35 has specified a Reporting-owned projection over the exact CaseOpened and VerdictIssued facts; strict admission; receipt, event ledger, gap buffer, summary row and contiguous watermark transactions; honest age/status semantics; a no-store BFF contract; exhaustive console states; outage, catch-up, crash, provider, broker, rebuild, security, performance and observability proof paths; and a fail-closed evidence manifest. It has not executed those artifacts or claimed source-authoritative currentness.

NEXT - CHAPTER 36

The Complete Slice and the Optional RL Advisory Branch will execute the full scenario with correlation and causation preserved. RL Advisory will appear only as an optional subscriber/adaptor, and the chapter will state the evidence required before it could become an independent service.

CHAPTER CHECKPOINT

- Why are watermark, projection age and update status separate coordinates?
- What happens when producer stream position 2 arrives before position 1?
- Why may a durably buffered gap delivery be acknowledged?
- Why does an absent projection row not prove an absent RootCause case?
- What does current-as-observed prove, and what does it deliberately not prove?
- How does a shadow generation become active without hiding a failed rebuild?
- Which evidence must exist before the manifest may leave PENDING?

PART VI - THE FIRST END-TO-END DISTRIBUTED SLICE

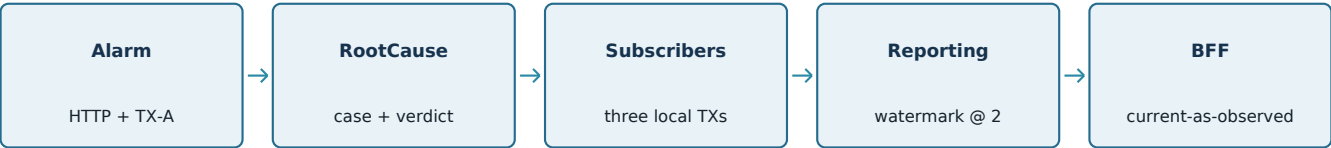
36. The Complete Slice and the Optional RL Advisory Branch

The exact public facts and owner-local commits from Chapters 32-35 now become one production-path scenario with preserved correlation, direct causation and reviewable recovery evidence. RL Advisory remains an optional non-command observer.

CHAPTER DECISION

The core scenario enters through the secured Alarm HTTP edge, proves TX-A, the Alarm relay, TX-RC-OPEN, CaseOpened publication, TX-RC-VERDICT, verdict fan-out, TX-AUD, TX-COMP, TX-REP and the freshness-bearing BFF response. Completion is an evidence vector, never a global transaction. The manuscript defines the runnable proof but keeps runtime status PENDING until real artifacts exist.

PROOF PATH 36-A - COMPLETE CORE SLICE



GUARANTEE LEDGER 36-A

Question	Answer
What is preserved?	three frozen bodies, one correlation and direct-hop causation
What is independent?	every service database, retry path and failure outcome
What is optional?	RL Advisory queue, subscriber and module store
What is still pending?	repository execution and attached runtime evidence

PROOF BOUNDARY

A complete slice is not one distributed transaction

The scenario is complete when every owner can prove its own durable effect and the public facts connect those effects. It is not complete because one coordinator held locks across five databases or because one test method returned without throwing.

DECISION LEDGER 36-2

Owner boundary	Durable claim	Cross-owner claim forbidden
AlarmManagementDb	threshold decision + pending Flood fact	RootCause case exists
RootCauseDb	case + verdict + two pending facts	Audit/Compliance effects exist
AuditDb / ComplianceDb	independent observed consequences	Reporting is current
ReportingDb	delayed projection + watermark	source-authoritative truth

COMPLETION RULE

The proof is a conjunction of local commits, exact message identities, broker settlement evidence and an honest final observation. No row participates in a global ACID boundary.

GOLDEN CORPUS

Three public bodies remain byte-for-byte fixed

Chapter 36 consumes the artifacts frozen by Chapters 32-35. The runner does not regenerate convenient equivalents; it compares the captured bodies with the canonical corpus and fails if one coordinate, byte length or digest drifts.

EXECUTABLE ASSET 36-0 - SHARED GOLDEN IDENTIFIERS

```
internal static class Golden36
{
    public const string CORRELATION_ID = "request-am-20482";
    public const string FLOOD_MESSAGE_ID =
        "019bc4d4-5dc1-7b8d-9d48-542d25315356";
    public const string CASE_ID =
        "019bc4d4-6dfe-7bcb-9de8-3af71979285a";
    public const string CASE_OPENED_MESSAGE_ID =
        "019bc4d4-6ea0-7d14-a57a-3e8643c55f13";
    public const string VERDICT_ID =
        "019bc4d4-81c0-7eal-82d5-4f9b8719d0a1";
    public const string VERDICT_MESSAGE_ID =
        "019bc4d4-82a4-77f2-ae06-09b172aa59db";
}
```

DECISION LEDGER 36-3

Fact	MessageId / length	Final body SHA-256
AlarmFloodDetected.v1	019bc4d4-5dc1-7b8d-9d48-542d25315356 / 1003	7cf98319c20a5915d5710bbcb5878fe51cdae89a5b68081c135fd0e962c276a
RootCauseCaseOpened.v1	019bc4d4-6ea0-7d14-a57a-3e8643c55f13 / 876	454f51cf4b0250b9c855346a6648a7d73530f19687ee3b52cda2edf9d0339c5b
RootCauseVerdictIssued.v1	019bc4d4-82a4-77f2-ae06-09b172aa59db / 1157	192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00

BYTE AUTHORITY

Pretty-printed JSON is documentation. The one-line RFC 8785-compatible canonical UTF-8 bodies and their stored digests are the executable contract corpus.

LINEAGE MODEL

Correlation stays constant; causation moves one hop

Correlation groups the business journey. Causation identifies the immediate business stimulus. Reusing the first command as every event's cause erases the real path; copying a trace span into CausationId confuses observability with business meaning.

DECISION LEDGER 36-4

Artifact	CorrelationId	Immediate CausationId
AlarmFloodDetected	request-am-20482	decision-am-9184
RootCauseCaseOpened	request-am-20482	019bc4d4-5dc1-7b8d-9d48-542d25315356
RootCauseVerdictIssued	request-am-20482	decision-rc-7184
subscriber lineage	request-am-20482	019bc4d4-82a4-77f2-ae06-09b172aa59db

DIRECT-HOP RULE

The scenario validator accepts one correlation value across the journey and demands a causation value appropriate to each direct edge.

RUNTIME TOPOLOGY

Core delivery and optional advice never share availability

The core profile boots AlarmManagement, RootCause, Audit, Compliance, Reporting and the BFF with owner-specific credentials and stores. The advisory profile may add one independent queue and subscriber. RootCause publication success never depends on that optional queue.

DECISION LEDGER 36-5

Route	Required recipients	Optional recipient
alarm-management.alarm-flood-detected.v1	root-cause	none
root-cause.root-cause-case-opened.v1	reporting	none
root-cause.root-cause-verdict-issued.v1	audit, compliance, reporting	rl-advisory when enabled

TOPOLOGY INVARIANT

The required route gate is evaluated against the core recipients only. Optional queue absence cannot block a producer outbox row from reaching Published.

DEPLOYMENT PROFILES

The optional branch is explicit configuration, not ambient discovery

Two manifests make the difference reviewable. Core is the default. CoreWithAdvisory adds a queue, binding, module store and worker registration; it does not replace any core registration or broaden any database principal.

EXECUTABLE ASSET 36-A - EXPLICIT SLICE PROFILE

```
public enum SliceProfile { Core, CoreWithAdvisory }

public static void AddFullSlice(
    this IServiceCollection services, SliceProfile profile)
{
    services.AddAlarmManagementPath();
    services.AddRootCausePath();
    services.AddAuditComplianceReporting();

    if (profile is SliceProfile.CoreWithAdvisory)
        services.AddRlAdvisorySubscriber();
}
```

FAIL CLOSED

An unknown profile, missing required queue or unexpected advisory registration fails startup evidence. The system never guesses from a hostname or environment label.

SCENARIO HARNESS

One runner observes production paths without owning them

The runner sequences test actions and bounded observations. It never writes business tables, publishes owner facts, acknowledges deliveries or retries production commands on behalf of a service. Those actions remain inside the production composition roots.

EXECUTABLE ASSET 36-B - EVIDENCE RUNNER BOUNDARY

```
public sealed class FullSliceScenario(
    SliceEnvironment environment,
    AlarmApiDriver alarmApi,
    RootCauseApiDriver rootCauseApi,
    SliceObserver observer,
    EvidenceSink evidence)
{
    public async Task<ScenarioResult> RunAsync(
        SliceProfile profile, CancellationToken ct)
    {
        await environment.StartAsync(profile, ct);
        await alarmApi.SubmitGoldenObservationsAsync(ct);
        await observer.AwaitCaseOpenedAsync(ct);
        await rootCauseApi.IssueGoldenVerdictAsync(ct);
        return await observer.AwaitConvergenceAsync(profile, ct);
    }
}
```

NOT A PROCESS MANAGER

This type exists in the test/evidence assembly. It has no production endpoint, database, queue, retry policy or deployment. Chapter 37 introduces the durable production workflow owner.

ENVIRONMENT ISOLATION

Every run leases names, databases and credentials

Parallel or failed runs must not share queues, schemas, users or evidence paths. A lease resolves concrete resource names, records them before startup and removes only its own isolated resources during teardown.

EXECUTABLE ASSET 36-C - ISOLATED RESOURCE LEASE

```
public sealed record SliceLease(  
    string RunId,  
    string BrokerVHost,  
    IReadOnlyDictionary<string, string> Databases,  
    IReadOnlyDictionary<string, string> Principals,  
    string EvidenceDirectory);  
  
var lease = await allocator.CreateAsync(new LeaseRequest(  
    Prefix: "slice36",  
    Databases: ["alarm", "rootcause", "audit",  
        "compliance", "reporting", "advisory"]), ct);
```

DECISION LEDGER 36-8

Isolation gate	Required proof
names	run id is embedded in every resource
credentials	one principal per runtime/migration owner
cleanup	explicit resource inventory; no broad wildcard delete

PREFLIGHT

No scenario starts on an ambiguous environment

The harness records build identity, schema markers, topology digest and service configuration identities before the first request. A dirty migration state, reused queue, unknown image or missing evidence writer stops the run.

EXECUTABLE ASSET 36-D - PREFLIGHT GATE

```
await preflight.RequireAsync(  
    BuildIdentity.FromRepositoryAndArtifacts(),  
    DatabaseState.AllEmptyAtRequiredMigration(),  
    TopologyState.Matches(profile),  
    ConfigurationState.HasExpectedIdentities(),  
    BrokerState.NoReadyOrUnackedMessages(),  
    EvidenceState.WritableAndEmpty(),  
    ct);
```

WHY BEFORE ACT

A green scenario over stale rows or residual messages proves nothing. Preflight failure is evidence, not a reason to clean silently and continue.

DETERMINISM

Fixed time and identities are test-only substitutions

The production hosts are booted through their real composition paths. The harness replaces only explicitly approved nondeterministic boundaries: TimeProvider, UUID factory, authenticated test principal and external resource locations. Policy, handlers, mappings, transactions and workers remain production registrations.

EXECUTABLE ASSET 36-E - DELIBERATE SUBSTITUTIONS

```
builder.ConfigureTestServices(services =>
{
    services.RemoveAll<TimeProvider>();
    services.AddSingleton<TimeProvider>(
        new FakeTimeProvider(Golden.StartUtc));

    services.RemoveAll<IIdentityFactory>();
    services.AddSingleton<IIdentityFactory>(
        Golden.IdentityFactory());

    services.RemoveAll<IPrincipalIssuer>();
    services.AddSingleton<IPrincipalIssuer>(
        Golden.PrincipalIssuer());
});

services.AssertOnlyApprovedReplacements(
    typeof(TimeProvider), typeof(IIdentityFactory),
    typeof(IPrincipalIssuer));
```

REPRODUCIBILITY LIMIT

Deterministic time makes assertions stable. It does not prove production clock synchronization; clock skew remains a separate operational test.

PRODUCTION ENTRY

The scenario enters through the secured Alarm HTTP edge

Five observations are submitted to the exact Chapter 31 endpoint under the alarm.observation.submit capability. The fifth crosses the flood policy. Repeating the request with the same idempotency identity must return the stored outcome, while changed bytes must conflict without a second decision.

EXECUTABLE ASSET 36-F - AUTHENTICATED GOLDEN INPUT

```
using var client = alarmFactory.CreateAuthorizedClient(
    capability: "alarm.observation.submit",
    detectorScope: "site-ath-17/line-02");

foreach (var request in Golden.AlarmObservations)
{
    request.CorrelationId.Should().Be("request-am-20482");
    using var response = await client.PostAsJsonAsync(
        "/api/v1/alarm-observations", request, ct);
    response.StatusCode.Should().Be(HttpStatusCode.Accepted);
}
```

DECISION LEDGER 36-11

Input coordinate	Golden value
idempotency key	idem-am-7f9c2a
threshold decision	decision-am-9184 / flood-am-6031
scope	site-ath-17 / line-02

TX-A

Alarm acceptance finishes before any broker success

The fifth request commits the accepted observation, deterministic flood decision and one Pending outbox body in AlarmManagementDb. The HTTP 202 is justified by local durability. The broker can be unavailable at this instant without changing that response.

EXECUTABLE ASSET 36-G - OWNER-LOCAL ALARM ORACLE

```
await alarmDb.AssertTransactionAsync("TX-A", tx => tx
    .HasAcceptedObservation("alarm-am-9005")
    .HasFloodDecision("flood-am-6031", "decision-am-9184")
    .HasPendingOutbox(FLOOD_MESSAGE_ID)
    .HasCanonicalBodySha256(FLOOD_BODY_SHA256)
    .HasNoPublishedTimestamp());

await foreignStores.AssertNoRowsAsync(
    "RootCauseDb", "AuditDb", "ComplianceDb", "ReportingDb");
```

ALLOWED CLAIM

At this checkpoint AlarmManagement has durably decided and intends to publish. Nothing yet proves broker acceptance or a downstream effect.

FIRST PUBLIC FACT

AlarmFloodDetected.v1 is compared as captured bytes

The relay capture stores body and broker properties before any decoder formats them. The test validates MessageId, route, content type, canonical length, content fingerprint and final body digest against the Chapter 32 corpus.

EXECUTABLE ASSET 36-H - BYTE CORPUS ASSERTION

```
var delivery = await brokerCapture.RequiredAsync(
    "019bc4d4-5dc1-7b8d-9d48-542d25315356", ct);

delivery.Route.Should().Be(
    "alarm-management.alarm-flood-detected.v1");
delivery.Body.Length.Should().Be(1003);
Sha256.Hex(delivery.Body).Should().Be(
    "7cf98319c20a5915d5710bbcb5878fe51cdae89a5b68081c135fd0e962c276a");
Envelope.Read(delivery.Body).ContentFingerprint.Should().Be(
    "sha256:ef70dfaaf0e08599eb82877a77e60fe3d94b2ae0aa1456d2a175af9e2712b97");
```

NO CONVENIENT RESERIALIZATION

The assertion hashes the exact captured body. A semantically similar object serialized with different bytes is a contract failure, not a pass.

OUTBOX RELAY

Published means correlated broker acceptance

The Alarm relay claims one outbox lease, publishes with mandatory routing and correlates the returned confirm to that exact row. Only then does it mark the intent Published. A timeout leaves the outcome Unknown and relies on safe replay.

EXECUTABLE ASSET 36-I - RELAY CHECKPOINT

```
var publication = await relayEvidence.RequiredAsync(
    messageId: FLOOD_MESSAGE_ID, ct);

publication.Mandatory.Should().BeTrue();
publication.Confirm.Should().Be(PublishConfirm.Ack);
publication.Return.Should().BeNull();
publication.BodySha256.Should().Be(FLOOD_BODY_SHA256);

await alarmDb.AssertOutboxAsync(
    FLOOD_MESSAGE_ID, OutboxState.Published);
```

DECISION LEDGER 36-14

Crash point	Recoverable state
before publish	Pending row remains
after broker accept / before local mark	redelivery; consumer dedupes
after Published	no second relay claim

ROOTCAUSE ADMISSION

A delivery becomes a command only after strict validation

RootCause authenticates the relay principal, verifies exchange/route, MessageId, producer, fingerprint and payload invariants, then translates the public fact into a local application command. It never queries AlarmManagementDb to fill a missing field.

EXECUTABLE ASSET 36-J - ADMISSION ASSERTION

```
var admitted = rootCauseAdmission.Admit(
    capturedFloodDelivery, authenticatedRelay);

admitted.MessageId.Should().Be(FLOOD_MESSAGE_ID);
admitted.CorrelationId.Should().Be("request-am-20482");
admitted.CausationId.Should().Be("decision-am-9184");
admitted.Command.SourceDecisionId.Should().Be("flood-am-6031");
admitted.Command.Evidence.Should().HaveCount(5);
alarmManagementProbe.DatabaseReads.Should().Be(0);
```

SUFFICIENT PUBLIC FACT

RootCause can open the case from the event alone. Any required private database read would be evidence that the public contract is insufficient.

TX-RC-OPEN

Case creation and CaseOpened intent commit together

RootCauseDb records the inbox receipt, inbound lineage, one case, five ordered evidence items and one Pending RootCauseCaseOpened.v1 body in the same local transaction. The broker delivery remains unacknowledged until the commit outcome is known.

EXECUTABLE ASSET 36-K - ROOTCAUSE OPEN ORACLE

```
await rootCauseDb.AssertTransactionAsync("TX-RC-OPEN", tx => tx
    .HasInboxReceipt(FLOOD_MESSAGE_ID)
    .HasInboundLineage(FLOOD_MESSAGE_ID, FLOOD_BODY_SHA256)
    .HasCase(CASE_ID, sourceDecisionId: "flood-am-6031")
    .HasOrderedEvidence(count: 5)
    .HasPendingOutbox(CASE_OPENED_MESSAGE_ID));

await broker.AssertSettledAsync(
    FLOOD_MESSAGE_ID, Settlement.AckAfterCommit);
```

DUPLICATE RESULT

A later delivery with the same MessageId and body resolves the stored receipt. A changed body under that identity is quarantined as an identity conflict.

SECOND PUBLIC FACT

RootCauseCaseOpened.v1 preserves the first causal edge

The public case identity is allocated once inside RootCause. Its envelope retains the journey correlation and names the AlarmFloodDetected MessageId as its immediate cause. Reporting may observe it before the verdict and legitimately expose an Open case.

DECISION LEDGER 36-17

Coordinate	Exact value
CaseId	019bc4d4-6dfe-7bcb-9de8-3af71979285a
MessageId	019bc4d4-6ea0-7d14-a57a-3e8643c55f13
correlation / causation	request-am-20482 / 019bc4d4-5dc1-7b8d-9d48-542d25315356
stream	root-cause/case/019bc4d4-6dfe-7bcb-9de8-3af71979285a @ 1
body	876 bytes / 454f51cf4b0250b9c855346a6648a7d73530f19687ee3b52cda2edf9d0339c5b

INTERMEDIATE TRUTH

A Reporting row at stream position 1 is not stale corruption. It is an honest intermediate observation until position 2 arrives.

VERDICT COMMAND

The verdict enters through RootCause's authorized application edge

The scenario waits for the stored case, then submits the deterministic verdict command under the capability that RootCause owns. The command references evidence revision 1 and does not write Audit, Compliance or Reporting state.

EXECUTABLE ASSET 36-L - DETERMINISTIC VERDICT INPUT

```
var command = new IssueRootCauseVerdictV1(
    CommandId: "decision-rc-7184",
    CaseId: Guid.Parse(CASE_ID),
    VerdictCode: "COMMON_POWER_SUPPLY_INSTABILITY",
    Confidence: 0.94m,
    EvidenceRevision: 1,
    PolicyIdentity: "root-cause/verdict/3",
    CorrelationId: "request-am-20482");

var response = await rootCauseApi.IssueVerdictAsync(command, ct);
response.StatusCode.Should().Be(HttpStatusCode.Accepted);
```

AUTHORIZATION BOUNDARY

The test principal may invoke the public/application edge. It receives no RootCauseDb write credential and cannot manufacture a verdict outbox row.

TX-RC-VERDICT

Verdict state and VerdictIssued intent are one commit

RootCause verifies the case is open at evidence revision 1, applies policy root-cause/verdict/3, stores the verdict identity and adds one Pending RootCauseVerdictIssued.v1 body. No subscriber availability check participates in this transaction.

EXECUTABLE ASSET 36-M - VERDICT COMMIT ORACLE

```
await rootCauseDb.AssertTransactionAsync("TX-RC-VERDICT", tx => tx
    .HasVerdict(VERDICT_ID,
        code: "COMMON_POWER_SUPPLY_INSTABILITY",
        confidence: 0.94m,
        policy: "root-cause/verdict/3")
    .HasEvidenceRevision(1, count: 5)
    .HasPendingOutbox(VERDICT_MESSAGE_ID)
    .HasStreamPosition(CASE_ID, expected: 2));

await subscribers.AssertNoSynchronousCallsAsync();
```

COMMIT BOUNDARY

The Accepted response proves a RootCause-owned verdict and durable publication intent. It does not prove Audit evidence, Compliance review, Reporting projection or advice.

THIRD PUBLIC FACT

RootCauseVerdictIssued.v1 is the fan-out anchor

The exact Chapter 34 body is published once and independently delivered to Audit, Compliance and Reporting. Its causation is the owner-local verdict decision, while each subscriber records the public MessageId as its input lineage.

DECISION LEDGER 36-20

Coordinate	Frozen value
VerdictId / MessageId	019bc4d4-81c0-7ea1-82d5-4f9b8719d0a1 / 019bc4d4-82a4-77f2-ae06-09b172aa59db
correlation / causation	request-am-20482 / decision-rc-7184
stream	root-cause/case/019bc4d4-6dfe-7bcb-9de8-3af71979285a @ 2
content fingerprint	07d530ef636be3d2af8dcc3b20cd91b212a076a4e70c1a9c55730930fbfe1e2e
body	1157 bytes / 192e5e73c5a35d438e2ee9edb0a9e7044293c29bf321ba1abe34f191d8b44c00

FAN-OUT RULE

One public fact produces independent local consequences. There is no broker transaction that atomically commits all subscriber databases.

REQUIRED ROUTING

Three core queues must receive the exact verdict body

The topology probe resolves the verdict route to the required core queues. Every captured delivery must have the same MessageId and body digest. Consumer-specific delivery tags and observation timestamps are allowed to differ.

EXECUTABLE ASSET 36-N - CORE FAN-OUT GATE

```
var routed = await topology.QueuesMatchingAsync(
    "nexus.events",
    "root-cause.root-cause-verdict-issued.v1", ct);

routed.Core.Should().BeEquivalentTo([
    "audit.root-cause-verdicts.v1",
    "compliance.root-cause-verdicts.v1",
    "reporting.integration-events.v1"
]);

await captures.AssertSameBodyAsync(
    VERDICT_MESSAGE_ID, VERDICT_BODY_SHA256, routed.Core, ct);
```

OPTIONAL ROUTE EXCLUSION

An advisory queue may appear only in the optional set. It is never added to the core route requirement evaluated by the RootCause relay.

TX-AUD

Audit appends one immutable observed consequence

Audit validates the verdict delivery and commits its inbox receipt, inbound lineage and immutable evidence record in AuditDb. A duplicate returns the stored receipt; no update path edits the original evidence payload.

EXECUTABLE ASSET 36-O - AUDIT ORACLE

```
await auditDb.AssertTransactionAsync("TX-AUD", tx => tx
    .HasInboxReceipt(VERDICT_MESSAGE_ID)
    .HasInboundLineage(
        sourceMessageId: VERDICT_MESSAGE_ID,
        correlationId: "request-am-20482")
    .HasEvidenceRecord(
        caseId: CASE_ID,
        verdictId: VERDICT_ID,
        code: "COMMON_POWER_SUPPLY_INSTABILITY")
    .HasNoMutableReplacement());
```

DECISION LEDGER 36-22

Audit fact	Claim
record exists	Audit observed the approved VerdictIssued body
record absent	Audit has not recorded it yet; verdict may still exist

TX-COMP

Compliance opens one Pending review, not a verdict copy

Compliance consumes the same public fact and commits one Pending review with its own lifecycle. It records the source verdict identity and policy but does not reinterpret confidence or promote the verdict into compliance approval.

EXECUTABLE ASSET 36-P - COMPLIANCE ORACLE

```
await complianceDb.AssertTransactionAsync("TX-COMP", tx => tx
    .HasInboxReceipt(VERDICT_MESSAGE_ID)
    .HasReview(
        rootCauseCaseId: CASE_ID,
        sourceVerdictId: VERDICT_ID,
        state: ComplianceReviewState.Pending)
    .HasSourcePolicy("root-cause/verdict/3")
    .HasNoAutomaticApproval());
```

SEMANTIC BOUNDARY

A root-cause verdict is evidence for compliance work. It is not a compliance decision and cannot skip human or policy authorization.

TX-REP

Reporting reaches stream position 2 contiguously

Reporting may receive CaseOpened and VerdictIssued in either delivery order. It durably buffers a gap, then commits inbox, event ledger, summary and contiguous checkpoint. The final oracle requires the case stream watermark to reach exactly 2.

EXECUTABLE ASSET 36-Q - REPORTING CONVERGENCE ORACLE

```
await reportingDb.AssertProjectionAsync(  
    projection: "root-cause-case-summary",  
    generation: activeGeneration,  
    streamId: $"root-cause/case/{CASE_ID}",  
    expectedPosition: 2,  
    expectedMessageId: VERDICT_MESSAGE_ID,  
    assertRow: row => row  
        .HasStatus("VerdictIssued")  
        .HasVerdict(VERDICT_ID)  
        .HasCode("COMMON_POWER_SUPPLY_INSTABILITY")  
        .HasConfidence(0.94m));
```

DELAYED TRUTH

The projection is accepted only after the gap count is zero and the checkpoint advances contiguously. Delivery order is never treated as producer order.

BFF OBSERVATION

The final HTTP answer carries freshness evidence

The BFF reads only Reporting-owned state and returns the verdict summary, watermark, source age, projection age and sampled update status. The scenario requires current-as-observed; it does not relabel the projection as globally current truth.

EXECUTABLE ASSET 36-R - FINAL READ ASSERTION

```
var observation = await bff.GetRootCauseCaseAsync(CASE_ID, ct);  
  
observation.Value.Status.Should().Be("VerdictIssued");  
observation.Value.VerdictId.Should().Be(VERDICT_ID);  
observation.Evidence.LastContiguousPosition.Should().Be(2);  
observation.Evidence.GapCount.Should().Be(0);  
observation.Evidence.UpdateStatus.Should().Be("current-as-observed");  
observation.Evidence.ObservedAtUtc.Should().NotBe(default);
```

UI WORDING

The operator may see Current as observed with the observation timestamp. The UI never displays source-authoritative now or hides an unknown/failed status behind the last value.

DISTRIBUTED OBSERVATION

Completion is an evidence vector, not one boolean row

A scenario result records each checkpoint separately. The aggregate status becomes Passed only if all required coordinates agree. A partial result names the missing or conflicting boundary instead of returning a generic timeout.

EXECUTABLE ASSET 36-S - CONVERGENCE VECTOR

```
public sealed record SliceConvergence(  
    Checkpoint AlarmCommit,  
    Checkpoint FloodPublication,  
    Checkpoint RootCauseOpenCommit,  
    Checkpoint CaseOpenedPublication,  
    Checkpoint VerdictCommit,  
    Checkpoint VerdictPublication,  
    Checkpoint AuditCommit,  
    Checkpoint ComplianceCommit,  
    Checkpoint ReportingPositionTwo,  
    Checkpoint BffObservation);  
  
public bool IsCoreComplete => AllRequired.All(x => x.IsProved);
```

DECISION LEDGER 36-26

Status	Meaning
Proved	artifact exists and coordinates match
Pending	valid earlier state; bounded wait not yet satisfied
Failed	explicit invariant or identity mismatch
Unknown	observation path could not establish outcome

CAUSATION VALIDATOR

Business lineage is checked as an edge set

The validator builds an expected directed graph from frozen identities. It rejects a missing edge, a cycle, an event that names itself as cause, and a subscriber record that points to the verdict decision instead of the delivered public MessageId.

EXECUTABLE ASSET 36-T - EXACT CAUSAL EDGES

```
var expected = new CausalEdge[]  
{  
    new("decision-am-9184", FLOOD_MESSAGE_ID),  
    new(FLOOD_MESSAGE_ID, CASE_OPENED_MESSAGE_ID),  
    new("decision-rc-7184", VERDICT_MESSAGE_ID),  
    new(VERDICT_MESSAGE_ID, "audit-evidence"),  
    new(VERDICT_MESSAGE_ID, "compliance-review"),  
    new(VERDICT_MESSAGE_ID, "reporting-position-2")  
};  
  
lineageGraph.AssertExactBusinessEdges(expected);  
lineageGraph.AssertCorrelation("request-am-20482");
```

OWNER-LOCAL DECISIONS

decision-am-9184 and decision-rc-7184 are valid causal nodes even though they are not public broker messages. Evidence must label their owning boundary.

TRACE CONTINUITY

traceparent supports diagnosis; it does not define causation

Each HTTP and messaging hop propagates W3C Trace Context when valid, records the producer and consumer spans, and links redelivery attempts without changing business identifiers. A missing trace is an observability defect, not permission to rewrite CausationId.

EXECUTABLE ASSET 36-U - TWO INDEPENDENT ASSERTIONS

```
businessLineage.CorrelationId.Should().Be("request-am-20482");
businessLineage.CausationEdges.Should().Equal(expectedEdges);

traceEvidence.RootTraceId.Should().NotBeEmpty();
traceEvidence.Spans.Should().ContainRequired([
    "alarm.http.accept", "alarm.outbox.publish",
    "rootcause.consume", "rootcause.verdict",
    "audit.consume", "compliance.consume", "reporting.consume"
]);

traceEvidence.Values.ShouldNotBeUsedAsBusinessIds();
```

TWO PLANES

Business lineage survives sampling and tracing-tool changes. Trace context improves timing diagnosis but cannot become the domain's durable identity scheme.

CONSISTENCY MODEL

No checkpoint pretends to be globally simultaneous

Owner timestamps, source occurrence times, local commit sequence numbers and broker observations are recorded as different coordinates. The evidence bundle can reconstruct a partial order; it cannot claim a single globally serializable instant.

DECISION LEDGER 36-29

Coordinate	Use	Misuse rejected
OccurredAtUtc	producer business occurrence	cross-host transaction order
committedAt / local sequence	owner-local durability	global database sequence
broker observedAt	transport observation	business cause
trace span time	diagnostic latency	contract identity

ALLOWED CONCLUSION

The graph proves that required durable effects eventually converged with preserved identities. It does not prove they were atomically visible together.

FAULT LABORATORY

Outages are injected at owner boundaries

The scenario has a happy-path profile and focused recovery profiles. Each profile makes exactly one dependency unavailable, observes the durable intermediate state, restores the dependency and waits for convergence without rewriting or manually advancing business data.

DECISION LEDGER 36-30

Fault	Intermediate truth	Recovery proof
broker offline after TX-A	Flood outbox Pending	same body later Published
RootCause offline	queue retains Flood	one case after restart
Audit offline	core verdict and other consumers progress	one evidence record later
Reporting offline	owner truth complete; view delayed	watermark reaches 2
RL offline	core complete	optional backlog only

FOCUSED EVIDENCE

One giant chaos test is hard to diagnose. The matrix preserves the same corpus while varying one failure boundary per run.

CRASH SCHEDULE

Every commit/settlement gap has a named replay outcome

Crash injection is placed before local commit, after local commit but before acknowledgement, after broker acceptance but before outbox completion, and during Reporting gap repair. The oracle asserts row counts and identities, not merely eventual HTTP success.

EXECUTABLE ASSET 36-V - CRASH POINT THEORY

```
[Theory]
[MemberData(nameof(CrashSchedule.AllOwnerBoundaries))]
public async Task SLICE36_CRASH_001_recovers_without_new_meaning(
    CrashPoint point)
{
    await scenario.RunUntilAsync(point);
    await environment.RestartCrashedOwnerAsync();
    var result = await scenario.ResumeAndAwaitConvergenceAsync();

    result.Core.IsCoreComplete.Should().BeTrue();
    await corpus.AssertNoNewPublicIdentityAsync();
    await stores.AssertNoDuplicateSemanticEffectAsync();
}
```

UNKNOWN COMMIT

A client timeout is not treated as rollback. Recovery queries the owner-local idempotency record or retries the same identity and body.

RETRY DISCIPLINE

Retries repeat identity and re-open scoped resources

Transport retries reuse the same public MessageId and exact bytes. Application semantic retries reuse the command/idempotency identity. Provider retries create a fresh DbContext and transaction; they do not reuse a poisoned unit of work.

DECISION LEDGER 36-32

Retry	Must remain stable	Must be fresh
HTTP	idempotency key + request digest	request attempt / trace span
outbox	MessageId + canonical body	channel / confirm sequence
consumer	MessageId + received body	delivery tag / DbContext
provider	semantic operation identity	transaction / DbContext

FORBIDDEN RETRY

Allocating a new public MessageId or verdict identity because an acknowledgement was lost creates new meaning and fails the corpus gate.

CORE CONVERGENCE

The happy path finishes only after negative gates pass

Presence checks are insufficient. The final gate also requires no identity conflicts, no quarantined golden messages, no unacknowledged required deliveries, no stream gaps and no unexpected synchronous cross-owner database access.

EXECUTABLE ASSET 36-W - FINAL CORE GATE

```
result.Core.IsCoreComplete.Should().BeTrue();

await evidence.AssertZeroAsync(
    Metric.IdentityConflicts,
    Metric.QuarantinedGoldenMessages,
    Metric.RequiredUnackedDeliveries,
    Metric.ReportingStreamGaps,
    Metric.CrossOwnerDatabaseReads,
    Metric.UnexpectedPublicIdentities);

await topology.AssertNoRequiredBacklogAsync(ct);
```

BOUNDED WAIT

AwaitConvergence uses an explicit deadline and polling/backoff policy. Deadline expiry returns the unresolved checkpoint vector and captured diagnostics; it never spins indefinitely.

EVIDENCE BUNDLE**Artifacts are addressable, hashed and reproducible**

A passing local assertion is not release evidence until its outputs are retained. The bundle records the repository revision, build identity, package lock digest, environment manifest, exact commands, test results, database oracles, broker captures, trace export and redacted logs.

DECISION LEDGER 36-34

Artifact	Required coordinate
build.json	repository URI, commit, dirty flag, artifact digests
environment.json	profile, resource lease, configuration identities
scenario.trx	discovered/executed/failed/skipped counts
broker/	properties, routes, bodies, confirm/settlement chronology
stores/	provider-backed row oracles and migration identities
traces.otlp.json	trace ids/spans with redaction report

REPRODUCTION COMMAND

The manifest must contain the exact command and configuration digest needed to reproduce the profile from the named build. A prose statement that tests passed is rejected.

FAIL-CLOSED EVIDENCE**The manuscript specifies a run; it does not counterfeit one**

No repository execution artifacts are attached to this chapter. Therefore the implementation and runtime status remain PENDING. A validator rejects missing files, zero discovery, stale timestamps, build mismatch, skipped required cases and evidence recorded under a different profile.

EXECUTABLE ASSET 36-X - MANIFEST VALIDATOR

```
validator.RequireFile("scenario.trx");
validator.RequireFile("build.json");
validator.RequireFile("environment.json");
validator.RequireDirectory("broker");
validator.RequireDirectory("stores");

validator.RequireTests(discovered: GreaterThan.Zero,
    failed: 0, skippedRequired: 0);
validator.RequireSameBuildAcrossArtifacts();
validator.RequireProfile(requestedProfile);
validator.RequireFreshness(maxAge: TimeSpan.FromHours(24));
```

CURRENT STATUS: PENDING

The chapter provides compile-oriented assets, exact fixtures, oracles and commands. It does not assert that <https://github.com/gregory82gr/Nexus-1-phase-0> currently contains or has executed them.

OPTIONAL RL BRANCH**Advice is permitted only as a non-blocking observation**

RL Advisory may observe RootCauseVerdictIssued.v1 and compute Recommend or Abstain. It cannot prevent the core scenario from completing, cannot change the RootCause verdict, and cannot execute an operational action. The branch is absent by default.

DECISION LEDGER 36-36

Question	Chapter 36 decision
Required for core?	No
Input	approved public VerdictIssued.v1 only
Output effect now	module-owned advisory snapshot or abstention
Operational command?	Never
Independent service?	Not yet justified

PRESERVATION RULE RL-ADV-001

A recommendation remains advice. Neither a handler, UI, adapter nor future consumer may reinterpret it as an authorized operational command.

OPTIONAL REGISTRATION**Disabled means no host, queue, binding or database activity**

The optional profile adds a distinct queue bound to the exact verdict route, a subscriber registration and module-owned storage. When disabled, topology and process inventories must prove complete absence rather than a dormant worker polling shared infrastructure.

EXECUTABLE ASSET 36-Y - OPTIONAL TOPOLOGY GATE

```
if (profile is SliceProfile.CoreWithAdvisory)
{
    await topology.AssertBindingAsync(
        "rl-advisory.root-cause-verdicts.v1",
        "root-cause.root-cause-verdict-issued.v1", ct);
    hosts.Should().Contain("RLAdvisorySubscriber");
}
else
{
    topology.ShouldNotContainQueue("rl-advisory.root-cause-verdicts.v1");
    hosts.ShouldNotContain("RLAdvisorySubscriber");
    advisoryDb.ConnectionAttempts.Should().Be(0);
}
```

PRODUCER INDEPENDENCE

RootCause does not know whether the optional queue exists. Its required publication gate is unchanged between profiles.

RL ADMISSION

The adapter authenticates and validates before model work

The subscriber applies the same envelope and byte checks as other consumers, restricts the producer to root-cause and accepts only VerdictIssued schema 1. It translates public fields into an internal advisory state; it never reads RootCauseDb or ReportingDb.

EXECUTABLE ASSET 36-Z - STRICT ADVISORY ADMISSION

```
var delivery = advisoryAdmission.Admit(
    rawDelivery, authenticatedBrokerPrincipal);

delivery.Envelope.DemandProducer("root-cause");
delivery.Envelope.DemandEvent(
    "nexus1.root-cause.root-cause-verdict-issued.v1", schema: 1);
delivery.Envelope.DemandMessageId(rawDelivery.Properties.MessageId);
fingerprints.Verify(delivery.Envelope, rawDelivery.CopiedBody);

privateDatabaseProbe.TotalReads.Should().Be(0);
```

CONTRACT-ONLY INPUT

If the public verdict lacks a value required by the advisor, the correct outcomes are contract evolution or abstention, never a private cross-database read.

ADVISORY STATE

Public verdict fields map to a bounded feature vector

The adapter records source identities, policy/model identities and the feature mapping version. It allows only reviewed categorical mappings and bounded numeric values. Unknown verdict codes or unsupported mapping versions produce Abstain with a stable reason.

EXECUTABLE ASSET 36-AA - EXPLICIT FEATURE MAPPING

```
public sealed record AdvisoryState(
    Guid SourceVerdictId,
    Guid SourceMessageId,
    string SiteClass,
    string LineClass,
    string VerdictCode,
    decimal VerdictConfidence,
    int EvidenceCount,
    string FeatureMapIdentity);

var state = featureMapper.Map(
    verdict, identity: "rl-advisory/features/1");
state.DemandFiniteAndBounded();
```

DECISION LEDGER 36-39

Rejected input	Outcome
unknown verdict code	Abstain(unsupported-verdict-code)
confidence outside 0..1	terminal invalid-contract
unknown feature map	Abstain(unsupported-feature-map)

POLICY IDENTITY

A model file without provenance is not a policy

The advisor identifies the algorithm, model artifact digest, training-data snapshot, feature map, action catalogue and decision thresholds. The combined PolicyIdentity is stored with every outcome. Floating latest aliases are forbidden in evidence-bearing profiles.

EXECUTABLE ASSET 36-AB - IMMUTABLE POLICY DESCRIPTOR

```
public sealed record AdvisoryPolicyIdentity(  
    string Algorithm,  
    string ModelSha256,  
    string TrainingSnapshot,  
    string FeatureMap,  
    string ActionCatalogue,  
    string Thresholds)  
{  
    public string Digest => CanonicalSha256.Of(this);  
}  
  
var policy = registry.GetRequired(  
    "rl-advisory/power-instability/1");  
policy.AssertArtifactDigest();
```

DETERMINISM CLAIM

Given the same admitted public fact and identical policy identity, the adapter must produce the same outcome bytes. This is a testable adapter guarantee, not a claim that the learned policy is optimal.

DECISION SEMANTICS

Recommend and Abstain are the only valid outcomes

The policy may return a ranked advisory action when confidence, coverage and safety gates pass; otherwise it abstains. A Q-value or discounted return is labeled as such and is never displayed as a probability. The action catalogue contains advice codes, not actuator targets.

EXECUTABLE ASSET 36-AC - TYPED RESULT

```
public abstract record AdvisoryOutcome  
{  
    public sealed record Recommend(  
        string AdviceCode,  
        decimal Score,  
        string ScoreMeaning,  
        AdvisoryPolicyIdentity Policy) : AdvisoryOutcome;  
  
    public sealed record Abstain(  
        string ReasonCode,  
        AdvisoryPolicyIdentity Policy) : AdvisoryOutcome;  
}  
  
const string ScoreMeaning = "discounted-return-not-probability";
```

NO FORCED ANSWER

Abstention is a successful, auditable outcome. The module must not choose a low-confidence action merely to keep the UI populated.

TX-ADV

Receipt, lineage and advisory outcome commit locally

When enabled, the module commits one inbox receipt, exact input lineage, policy identity and either recommendation or abstention in its own store. It does not write an outbox because Chapter 36 activates no public advisory event.

EXECUTABLE ASSET 36-AD - ADVISORY LOCAL TRANSACTION

```
await using var tx = await advisoryDb.Database.BeginTransactionAsync(ct);

advisoryDb.Inbox.Add(receipt.Completed(delivery));
advisoryDb.InputLineage.Add(lineage.From(delivery));
advisoryDb.Decisions.Add(decisionRow.From(
    sourceVerdictId: VERDICT_ID,
    sourceMessageId: VERDICT_MESSAGE_ID,
    outcome, policyIdentity));

await advisoryDb.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
return ConsumeOutcome.Committed(VERDICT_MESSAGE_ID);
```

NO PUBLIC FACT YET

A future outbox would require a separately approved RecommendationGenerated.v1 contract, routing policy and real consumer. This chapter deliberately stops before that boundary.

ADVISORY IDEMPOTENCY

Transport and semantic duplicates use different keys

The inbox key proves receipt of one MessageId. A semantic unique key prevents a replay or contract migration from producing two advisory decisions for the same source verdict under the same policy identity. A different policy identity creates a new, explicitly versioned evaluation.

EXECUTABLE ASSET 36-AE - TWO-KEY DATABASE ORACLE

```
model.Entity<AdvisoryInboxRow>().HasKey(x =>
    new { x.ConsumerName, x.MessageId });

model.Entity<AdvisoryDecisionRow>().HasIndex(x => new
{
    x.SourceVerdictId,
    x.PolicyIdentityDigest
}).IsUnique();

model.Entity<AdvisoryDecisionRow>().HasIndex(x =>
    x.SourceMessageId);
```

DECISION LEDGER 36-43

Collision	Classification
same MessageId + same bytes	transport duplicate
same verdict + same policy	semantic duplicate
same MessageId + changed bytes	identity conflict
same verdict + new policy	new evaluation, separately identified

NON-COMMAND ENFORCEMENT

Architecture and contracts prevent advice from becoming control

Naming is not enough. The advisory assembly cannot reference actuator, command-dispatch or AlarmManagement application packages; the advice contract contains no target endpoint, execution deadline or authorization token; the UI requires an explicit human/policy action outside this module.

EXECUTABLE ASSET 36-AF - PRESERVATION GATES

```
Types.InAssembly(RLAdvisoryAssembly)
    .ShouldNot().HaveDependencyOnAny(
        "Nexus1.Actuation",
        "Nexus1.AlarmManagement.Application",
        "Nexus1.CommandDispatch");

typeof(AdvisoryOutcome.Recommend)
    .ShouldNotExposeMembers([
        "Execute", "TargetEndpoint", "AuthorizationToken",
        "CommandPayload", "MustCompleteBy"
    ]);
```

RL-ADV-001

Any adapter that turns AdviceCode into a command belongs to an explicit authorized decision boundary with its own contract and evidence. It cannot be hidden inside RL Advisory.

OPTIONAL FAILURE

Advisory outage cannot reduce core convergence

With the optional queue enabled and the advisor offline, the queue may retain the verdict while Audit, Compliance and Reporting complete. With the queue disabled, there is no backlog to interpret. Restoring the subscriber produces one stored advisory outcome without replaying core effects.

EXECUTABLE ASSET 36-AG - OFFLINE ADVISORY PROOF

```
await environment.StopAsync("RLAdvisorySubscriber", ct);
await scenario.RunCoreThroughVerdictAsync(ct);

(await observer.AwaitCoreConvergenceAsync(ct))
    .IsCoreComplete.Should().BeTrue();
await topology.AssertReadyCountAsync(
    "rl-advisory.root-cause-verdicts.v1", expected: 1, ct);

await environment.StartAsync("RLAdvisorySubscriber", ct);
await advisoryDb.AwaitOneDecisionAsync(VERDICT_ID, ct);
await coreStores.AssertUnchangedAsync();
```

HISTORY WHEN ENABLING LATER

Adding the optional queue after facts have already passed does not magically recover history. An explicit replay source, start coordinate and retention proof are required.

FUTURE PUBLIC CONTRACT

RecommendationGenerated.v1 remains a candidate, not an activated fact

A future extraction may justify a public advisory fact. Chapter 36 records its semantic outline but does not claim a schema file, outbox row, exchange binding or consumer. This prevents an aspirational name from appearing as implemented architecture.

DECISION LEDGER 36-46

Candidate field	Required meaning before activation
recommendationId	stable producer-owned identity
sourceVerdictMessageId	direct public cause
adviceCode	non-command catalogue entry
score / scoreMeaning	bounded value with honest semantics
policyIdentity	model, data, mapping and threshold provenance
expiresAt / abstention	explicit usefulness and refusal semantics

STATUS: NOT ACTIVATED

RecommendationGenerated.v1 is a design candidate only. No architecture inventory, route gate or evidence manifest may list it as a present public contract.

EXTRACTION DECISION

An adapter becomes a service only after ownership evidence exists

Process isolation is not an architectural reward for using machine learning. Extraction is justified only when the capability has an independently valuable contract, deployment and failure boundary, sustained scaling need, data/store ownership, security policy and an accountable operating team.

DECISION LEDGER 36-47

Evidence dimension	Required before independent service
contract	stable input/output, compatibility policy, real consumer need
operations	SLO, error budget, on-call owner, runbooks, dashboards
deployment	independent release/rollback and failure isolation evidence
data	owned schema, migrations, retention, backup/restore, replay
security	separate principals, ACLs, threat model, audit trail
economics	measured load/cost or cadence benefit exceeding complexity

CURRENT DECISION

Keep RL Advisory as an optional adapter/module. The chapter records how to prove a future extraction, but present evidence does not justify a service boundary.

EVIDENCE MATURITY

Core and optional profiles have separate release claims

A core release may pass without the advisory profile. An advisory-enabled release must first inherit a passing core result, then prove its extra topology, store, contract admission, deterministic policy and non-command gates. The optional result can fail without rewriting the core result.

DECISION LEDGER 36-48

Profile	Required suites	Present manuscript status
Core	golden path + provider + broker + crash + BFF	PENDING runtime evidence
CoreWithAdvisory	Core plus RL admission, TX-ADV, offline, RL-ADV-001	PENDING runtime evidence
future RL service	all above plus extraction dimensions	NOT JUSTIFIED

NO STATUS INHERITANCE UPWARD

Passing the core profile does not silently approve RL. Passing the optional module does not justify service extraction. Each claim has its own gate.

SECURITY AND PRIVACY

The proof path uses least privilege and redacted evidence

Each service receives credentials only for its database and declared broker operations. The scenario principal is scoped to the two public input capabilities. Captures retain identifiers and digests needed for proof while redacting tokens, connection strings, raw credentials and unapproved payload fields.

DECISION LEDGER 36-49

Principal	Allowed	Denied
alarm-runtime	AlarmManagementDb DML + Alarm outbox publish	foreign databases
rootcause-runtime	RootCauseDb DML + RootCause publish/consume	AlarmManagementDb reads
subscribers	own DB + own queue consume	RootCauseDb reads
scenario	approved HTTP calls + read-only evidence probes	business-table writes
rl-advisory	own DB + optional queue consume	command/actuator paths

EVIDENCE HYGIENE

A redaction report lists removed fields and scans retained artifacts for configured secret patterns. Evidence that leaks a credential is a failed run.

OBSERVABILITY

Metrics explain the path without inventing high-cardinality dimensions

Service, operation, event type, schema version, outcome and bounded failure code are metric labels. MessageId, CaselId and CorrelationId belong in structured logs/traces under access controls, not metric labels. Each owner emits commit and settlement evidence at its boundary.

EXECUTABLE ASSET 36-AH - BOUNDED TELEMETRY

```
meter.Counter<long>("nexus.slice.effects")
    .Add(1,
        KeyValuePair.Create<string, object?>("owner", owner),
        KeyValuePair.Create<string, object?>("event_type", eventType),
        KeyValuePair.Create<string, object?>("outcome", outcome));

logger.LogInformation(
    "Slice effect {Owner} {MessageId} {CorrelationId} {Outcome}",
    owner, messageId, correlationId, outcome);
```

CARDINALITY RULE

Golden identifiers make one run easy to find, but they still remain log/trace attributes. Production metric dimensions must stay bounded.

PERFORMANCE ENVELOPE

Latency budgets are per boundary and advisory work is outside core

The capstone records time to local acceptance, outbox publication, downstream commit, Reporting convergence and BFF observation separately. One end-to-end percentile without stage evidence cannot identify queueing, provider or projection delay.

DECISION LEDGER 36-51

Measure	Start -> stop	Release interpretation
accept latency	HTTP receive -> TX-A commit	synchronous budget
publication lag	TX commit -> broker confirm	relay capacity
consumer lag	broker enqueue -> local commit	subscriber capacity
observation lag	source occurrence -> BFF evidence	user-visible delay
advisory lag	verdict enqueue -> TX-ADV	optional SLO only

NO BORROWED BUDGET

A slow or unavailable advisor cannot consume the core convergence deadline. Its result is evaluated in the optional profile after core completion.

CI EXECUTION

One command produces a reviewable, fail-closed bundle

The repository should expose a single documented entry point that restores locked dependencies, builds the named revision, starts isolated dependencies, runs the selected profile, validates evidence and archives the bundle. Individual hidden developer steps are not release procedure.

EXECUTABLE ASSET 36-AI - PROPOSED REPRODUCTION COMMAND

```
dotnet test tests/Nexus1.FullSlice.EvidenceTests \
  --configuration Release \
  --no-restore \
  --filter "Category=Slice36&Profile=Core" \
  --logger "trx;LogFileName=scenario.trx" \
  --results-directory artifacts/evidence/slice36/core

dotnet run --project tools/EvidenceValidator -- \
  artifacts/evidence/slice36/core/manifest.json
```

PROPOSED PATH

The command and paths are the chapter's target contract. They become factual only when committed and executed in the named repository/build; until then the manifest status is PENDING.

SCENARIO RECORD

The final JSON cannot say Passed without attached proof

The release record binds every checkpoint to an artifact path and digest. It lists missing requirements explicitly. A generated timestamp, author name or manual approval cannot replace machine-readable evidence.

EXECUTABLE ASSET 36-AJ - FAIL-CLOSED RECORD SHAPE

```
{
  "claimId": "NEXUS1-SLICE36-CORE-001",
  "profile": "Core",
  "status": "PENDING",
  "buildIdentity": null,
  "scenarioTrx": null,
  "brokerCaptureManifest": null,
  "storeOracleManifest": null,
  "traceExport": null,
  "missing": [
    "repository implementation",
    "provider-backed execution",
    "broker-backed execution"
  ]
}
```

NOTHING CLAIMS TO EXIST THAT DOES NOT

The honest PENDING record is stronger than an unsupported green badge. It names exactly what must be produced before the claim can change.

ADR AND HANDOFF

The first distributed slice is specified completely and honestly

Chapter 36 closes Part VI with exact owner-local transactions, three frozen public bodies, preserved correlation/direct causation, broker and provider proof paths, delayed-truth observation and an explicitly optional advisory adapter. It deliberately leaves durable multi-step workflow ownership to the next Part.

DECISION LEDGER 36-54

ADR-036 field	Decision
core slice	Alarm -> RootCause -> Audit/Compliance/Reporting -> BFF
completion	evidence vector; no global transaction
lineage	one correlation; direct-hop business causation; trace separate
RL Advisory	optional non-command adapter; no public output activated
service extraction	not justified without contract/ops/data/security/economic evidence
runtime status	PENDING until real build, .trx, broker/provider/trace artifacts exist

NEXT - PART VII, CHAPTER 37

The NEXUS-1 Distributed Process Manager introduces durable workflow state, expected events, timeouts, completion and correlation for the alarm-to-compliance path. Chapter 36's runner is a test/evidence harness only; it is not that production process manager.

OFFICIAL TECHNICAL ANCHORS

The implementation path ends in evidence, not assertion

- ASP.NET Core integration tests and WebApplicationFactory: <https://learn.microsoft.com/aspnet/core/test/integration-tests>
- Microsoft.Extensions.Time.Testing FakeTimeProvider: <https://learn.microsoft.com/dotnet/api/microsoft.extensions.time.testing.faketimeprovider>
- EF Core transactions and optimistic concurrency: <https://learn.microsoft.com/ef/core/saving/transactions> and <https://learn.microsoft.com/ef/core/saving/concurrency>
- RabbitMQ acknowledgements and publisher confirms: <https://www.rabbitmq.com/docs/confirms>
- RabbitMQ queues and ordering: <https://www.rabbitmq.com/docs/queues>
- RabbitMQ .NET client guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>
- W3C Trace Context: <https://www.w3.org/TR/trace-context/>
- OpenTelemetry messaging conventions: <https://opentelemetry.io/docs/specs/semconv/messaging/>
- RFC 8785 JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785>

WHAT THIS CHAPTER HAS PROVED

The chapter specifies the exact full-slice path, identities, bytes, owner-local transactions, direct causal edges, topology, bounded waits, provider/broker/crash evidence, final delayed-truth observation, optional RL admission and persistence, non-command preservation rule, and service-extraction criteria. It has not represented unexecuted repository artifacts as present.

CHAPTER CHECKPOINT

- Why is the scenario runner not a production coordinator or process manager?
- Which three canonical public bodies must remain byte-identical?
- Why does correlation remain constant while causation changes by hop?
- Which local commits may exist while the next service is offline?
- What negative gates must pass before core convergence is accepted?
- Why may RL Advisory fail without changing the core result?
- Which rule prevents a recommendation from becoming an operational command?
- What evidence is still required before runtime status leaves PENDING?
- What evidence would justify extracting the advisory adapter as a service?

HANDOFF TO PART VII

Chapter 37 begins Workflow, Consistency, and Repair by implementing the NEXUS-1 Distributed Process Manager. That durable production component will own expected events, timeouts, resumability and completion; the Chapter 36 runner remains confined to tests and evidence.

PART VII

Workflow, Consistency and Repair

Partial progress is normal distributed state. It must remain explainable, resumable and repairable without a global transaction.

CHAPTER 37

Distributed Process Manager

facts -> state -> expectation -> completion

CHAPTER 38

Coordination ADR

choreography -> orchestration -> decision

CHAPTER 39

Timeout and escalation

deadline -> overdue -> human review

CHAPTER 40

Compensation

new fact -> retained history -> idempotency

CHAPTER 41

Operator consistency

pending -> delayed -> completed -> failed

CHAPTER 42

Reconciliation and repair

compare -> replay -> audit -> converge

ENTRY RULE

A workflow may coordinate progress. It may not steal the decisions owned by AlarmManagement, RootCause, Compliance, Audit or Reporting.

BEGIN WITH CHAPTER 37 / DURABLE PROGRESS WITHOUT A GLOBAL TRANSACTION

PART VII - WORKFLOW, CONSISTENCY, AND REPAIR

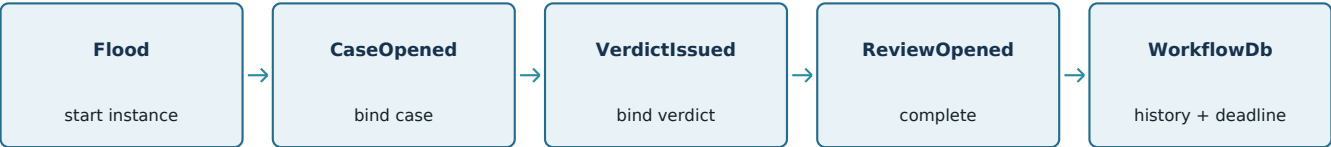
37. The NEXUS-1 Distributed Process Manager

The alarm-to-compliance journey becomes a durable, explainable state machine. It remembers what public fact started the work, what fact is expected next, when that expectation becomes overdue and exactly what completed the path.

CHAPTER DECISION

WorkflowDb owns one AlarmToComplianceProcess per AlarmFloodDetected.v1 start fact. The process expects CaseOpened, VerdictIssued and a newly justified ComplianceReviewOpened.v1, records each transition with exact lineage, survives duplicates, early delivery, crashes and restarts, and marks lateness without pretending it is business failure. It never opens a case, issues a verdict or approves compliance.

PROOF PATH 37-A - DURABLE WORKFLOW PROGRESSION



GUARANTEE LEDGER 37-A

Owned by Process Manager	Still owned elsewhere
workflow identity and current phase	Alarm flood decision
expected fact and deadline	RootCause case/verdict policy
observed fact lineage and transition history	Compliance review policy/lifecycle
overdue/completion disposition	Audit, Reporting and RL side paths

PART VII PREMISE**Partial progress is a valid distributed state**

The workflow can have a durable Alarm decision while RootCause is offline, a verdict while Compliance is delayed, or a Compliance review while Reporting is catching up. The process manager names that progress explicitly instead of forcing every intermediate condition into Success or Failure.

DECISION LEDGER 37-2

Observed boundary	Workflow interpretation	Not implied
Flood published	AwaitingCaseOpened	RootCause has consumed
CaseOpened observed	AwaitingVerdict	a verdict exists
VerdictIssued observed	AwaitingComplianceReview	Compliance has committed
ReviewOpened observed	Completed	review was approved/closed

PART RULE

The workflow must remain explainable, resumable and repairable without a global transaction. Every phase is derived from public facts and WorkflowDb history.

PROCESS AUTHORITY**The manager owns progress, not local business decisions**

RootCause still decides whether and what verdict to issue. Compliance still decides how a review is opened and later resolved. The process manager owns only workflow identity, expected public facts, deadlines, observed transitions and completion semantics.

DECISION LEDGER 37-3

Decision	Owner	Process manager role
flood detection	AlarmManagement	observe start fact
case creation / verdict	RootCause	expect and correlate facts
review creation	Compliance	expect terminal opening fact
deadline policy	Process Manager	mark overdue; retain expectation
human escalation	Chapter 39 boundary	not activated here

MINIMUM DURABLE MANAGER

A process manager is a durable state machine even before it issues commands. Chapter 38 will compare choreography and command-issuing orchestration against the actual runtime before granting more authority.

COMPLETION BOUNDARY

Alarm-to-compliance ends when a Pending review is publicly observed

For this workflow, completion means that Compliance has durably opened one Pending review for the exact verdict and published that owner fact. It does not mean the human review is resolved, Audit has no backlog, Reporting is current or RL Advisory produced advice.

DECISION LEDGER 37-4

Required path	Completion role
AlarmFloodDetected.v1	starts one workflow
RootCauseCaseOpened.v1	binds the owner case
RootCauseVerdictIssued.v1	binds the verdict
ComplianceReviewOpened.v1	completes the alarm-to-compliance workflow
Audit / Reporting / RL	independent side paths; not completion gates

NARROW MEANING

Completed is a workflow-progress claim. It is not a compliance approval, incident closure or statement that every subscriber is current.

REJECTED DESIGN

Polling foreign databases would make WorkflowDb a shadow authority

A naive coordinator repeatedly queries RootCauseDb and ComplianceDb, infers progress from private rows and updates one status record. That design couples credentials, schemas, availability and timing while leaving no immutable public evidence for what was actually observed.

REJECTED SHAPE 37-A - CROSS-DATABASE PROGRESS INFERENCE

```
var verdict = await rootCauseDb.Verdicts
    .SingleOrDefaultAsync(x => x.CaseId == caseId, ct);
var review = await complianceDb.Reviews
    .SingleOrDefaultAsync(x => x.SourceVerdictId == verdict!.Id, ct);

workflow.Status = review is null ? "waiting" : "complete";
await workflowDb.SaveChangesAsync(ct);
```

REQUIRED CORRECTION

Consume sufficient owner facts. WorkflowDb may store their identities and exact digests, but it receives no credential for RootCauseDb or ComplianceDb.

EVENT-DRIVEN TOPOLOGY

One durable queue receives the four approved facts

The manager has one service-owned queue with four exact bindings. It authenticates each producer independently and uses an explicit contract set. A broad wildcard would make any new event an accidental workflow transition candidate.

DECISION LEDGER 37-6

Binding	Producer	Workflow action
alarm-management.alarm-flood-detected.v1	alarm-management	Start
root-cause.root-cause-case-opened.v1	root-cause	BindCase
root-cause.root-cause-verdict-issued.v1	root-cause	BindVerdict
compliance.compliance-review-opened.v1	compliance	Complete

QUEUE IDENTITY

workflow.alarm-to-compliance.v1 is consumed only by the process-manager principal. No other service shares its delivery tags, acknowledgements or inbox key.

NEW PUBLIC CONTRACT

ComplianceReviewOpened.v1 now has a real consumer

Chapter 34 deliberately withheld a review-opened fact because no approved downstream contract required it. The process manager supplies that missing consumer. Compliance therefore activates one minimal V1 fact without exposing private review notes, assignees or authorization data.

DECISION LEDGER 37-7

Public field	Why sufficient
ReviewId	stable Compliance-owned review identity
RootCauseCaseId	binds the workflow case
SourceVerdictId / MessageId	proves exact upstream meaning
ReviewState = Pending	states opening semantics honestly
PolicyIdentity	records which opening policy acted
producer stream @ 1	owner ordering coordinate

DATA MINIMIZATION

The fact announces durable review creation. It does not carry reviewer identity, findings, attachments, deadlines or later compliance decisions.

CONTRACT TYPE

The V1 payload encodes only stable owner meaning

The contract is an immutable transport type. Strong local types remain inside Compliance; the mapper converts them only after the review commit has produced a complete fact snapshot.

EXECUTABLE ASSET 37-A - PUBLIC REVIEW CONTRACT

```
public sealed record ComplianceReviewOpenedV1(
    Guid ReviewId,
    Guid RootCauseCaseId,
    Guid SourceVerdictId,
    Guid SourceVerdictMessageId,
    string ReviewState,
    string ReviewPolicyIdentity,
    DateTimeOffset OpenedAtUtc,
    ProducerStreamV1 ProducerStream);

public static class ComplianceReviewStatesV1
{
    public const string Pending = "Pending";
}
```

COMPATIBILITY RULE

Adding or changing meaning requires a new schema version. Unknown fields, states or policy grammar are not silently guessed by the process manager.

CANONICAL CONTRACT CORPUS

ComplianceReviewOpened.v1 is frozen as 925 UTF-8 bytes

The golden body extends the Chapter 36 lineage without rewriting any prior fact. Correlation remains request-am-20482 and direct causation is the exact VerdictIssued MessageId.

EXECUTABLE ASSET 37-0 - SHARED PART VII FIXTURE

```
global using static Golden36;
global using static Golden37;

internal static class Golden37
{
    public const string WORKFLOW_ID =
        "019bc4d4-61a8-76f1-a935-b953af1dd9c4";
    public const string REVIEW_ID =
        "019bc4d4-8380-7b4a-9d80-12950da72d31";
    public const string REVIEW_OPENED_MESSAGE_ID =
        "019bc4d4-8420-77c8-a355-523cd43b5a18";
    public const string REVIEW_OPENED_BODY_SHA256 =
        "86f2ab796f6c7c84f1ad82c5042210c641f55cac1ed96d71a3f85b06c4e2287c";
}
```

DECISION LEDGER 37-9

Coordinate	Frozen value
ReviewId	019bc4d4-8380-7b4a-9d80-12950da72d31
MessageId	019bc4d4-8420-77c8-a355-523cd43b5a18
correlation / causation	request-am-20482 / 019bc4d4-82a4-77f2-ae06-09b172aa59db
content fingerprint	7e84ce5372fdb75e50c0ee9b7e8d6aaff6771b053d747248b4f889c07dae908
canonical body	925 bytes / 86f2ab796f6c7c84f1ad82c5042210c641f55cac1ed96d71a3f85b06c4e2287c

GOLDEN EXTENSION

The original three Chapter 36 bodies remain unchanged. This fourth body is the first contract added by Part VII.

COMPLIANCE TRANSACTION

Review and public intent now commit together

The Compliance consumer still admits the exact VerdictIssued delivery and creates one Pending review. The only extension is a Pending outbox row for ComplianceReviewOpened.v1 in the same ComplianceDb transaction. The broker remains outside that commit.

EXECUTABLE ASSET 37-B - EXTENDED TX-COMP

```
await using var tx = await db.Database.BeginTransactionAsync(ct);

var review = ComplianceReview.Open(
    identities.NewReviewId(), delivery.Contract, clock.UtcNow());
db.Reviews.Add(review);
db.InboxReceipts.Add(receipts.Completed(delivery));
db.Outbox.Add(reviewFacts.OpenedV1(
    review, delivery.Envelope).ToPendingRow());

await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
return ConsumeOutcome.Committed(delivery.MessageId, "review-opened");
```

DUPLICATE RULE

The semantic unique constraint on SourceVerdictId resolves a replay to the existing review and outbox identity. A duplicate delivery never allocates a second public fact.

COMPLIANCE RELAY

The new fact follows the proven outbox discipline

A Compliance-owned relay publishes the frozen body with mandatory routing, correlates publisher confirmation to the outbox lease and marks the row Published only after a known acknowledgement and no return.

EXECUTABLE ASSET 37-C - PUBLICATION GATE

```
var publication = await complianceRelay.RequiredAsync(
    REVIEW_OPENED_MESSAGE_ID, ct);

publication.Route.Should().Be(
    "compliance.compliance-review-opened.v1");
publication.BodyLength.Should().Be(925);
publication.BodySha256.Should().Be(
    REVIEW_OPENED_BODY_SHA256);
publication.Mandatory.Should().BeTrue();
publication.Confirm.Should().Be(PublishConfirm.Ack);
publication.Return.Should().BeNull();
```

UNKNOWN PUBLICATION

A lost confirm leaves the outbox result Unknown/Pending for safe replay. Process Manager inbox idempotency absorbs a second delivery of the same body.

TOPOLOGY GATE

The review fact is routed only after its consumer exists

The topology manifest now adds the Process Manager binding for the Compliance route. The producer gate proves that the exact workflow queue is reachable; it does not inspect or depend on Process Manager health.

EXECUTABLE ASSET 37-D - EXACT BINDING INVENTORY

```
await topology.AssertBindingAsync(  
    exchange: "nexus.events",  
    queue: "workflow.alarm-to-compliance.v1",  
    routingKey: "compliance.compliance-review-opened.v1",  
    ct);  
  
var recipients = await topology.QueuesMatchingAsync(  
    "nexus.events",  
    "compliance.compliance-review-opened.v1", ct);  
recipients.Should().ContainSingle()  
    .Which.Should().Be("workflow.alarm-to-compliance.v1");
```

NO ACCIDENTAL ACTIVATION

The contract, outbox mapping, relay route, binding, consumer admission and evidence fixture are reviewed as one change set.

WORKFLOW IDENTITY

Correlation locates a journey; it does not uniquely identify one instance

The process manager allocates WorkflowId once when the Flood fact starts the instance and enforces semantic uniqueness on AlarmFloodMessageId. CorrelationId is indexed for search but is not unique because one request or incident may legitimately relate multiple workflows.

DECISION LEDGER 37-13

Identity	Owner / use	Uniqueness
WorkflowId	Process Manager aggregate	primary key
AlarmFloodMessageId	start fact semantic key	unique per workflow type
CorrelationId	journey lookup	non-unique index
Caseld / VerdictId / ReviewId	observed owner coordinates	unique when bound

GOLDEN INSTANCE

The fixture allocates WorkflowId 019bc4d4-61a8-76f1-a935-b953af1dd9c4 for Flood MessageId 019bc4d4-5dc1-7b8d-9d48-542d25315356; replay resolves that same instance.

STATE DIMENSIONS

Phase and timeliness remain separate

Combining progress and lateness into one enum creates a growing state explosion. The aggregate stores the current Phase and an OverdueSinceUtc marker. Completion records whether the terminal fact arrived on time or late.

EXECUTABLE ASSET 37-E - STATE VOCABULARY

```
public enum WorkflowPhase
{
    AwaitingCaseOpened,
    AwaitingVerdict,
    AwaitingComplianceReview,
    Completed
}

public enum CompletionDisposition { OnTime, Late }

public sealed record WorkflowProgress(
    WorkflowPhase Phase,
    DateTimeOffset? OverdueSinceUtc,
    CompletionDisposition? Completion);
```

TIMEOUT MEANING

Overdue is an observation about time, not a business failure and not a terminal phase. The expected fact remains open so a late valid delivery can complete the workflow.

AGGREGATE ROOT

The process instance changes only through admitted facts or deadlines

The aggregate holds workflow-owned coordinates and delegates each transition to a method that checks the current expectation. Setters are private; infrastructure cannot jump directly to Completed.

EXECUTABLE ASSET 37-F - DURABLE PROCESS AGGREGATE

```
public sealed class AlarmToComplianceProcess
{
    public WorkflowId Id { get; private init; }
    public string CorrelationId { get; private init; } = null!;
    public Guid AlarmFloodMessageId { get; private init; }
    public Guid? RootCauseCaseId { get; private set; }
    public Guid? VerdictId { get; private set; }
    public Guid? ComplianceReviewId { get; private set; }
    public WorkflowPhase Phase { get; private set; }
    public DateTimeOffset? OverdueSinceUtc { get; private set; }
    public DateTimeOffset NextDeadlineAtUtc { get; private set; }
    public byte[] RowVersion { get; private set; } = null!;
}
```

PRIVATE TRUTH REMAINS PRIVATE

The aggregate stores public identities needed for workflow progress. It does not copy verdict evidence, compliance notes or Reporting projection fields.

TRANSITION GRAMMAR

Every phase has one expected public fact

The transition table is executable policy. An accepted fact must match type, schema and owner coordinates; a different but correlated event is recorded as unrelated or buffered, never used to skip a phase.

DECISION LEDGER 37-16

Current phase	Expected event	Next phase
no instance	AlarmFloodDetected.v1	AwaitingCaseOpened
AwaitingCaseOpened	RootCauseCaseOpened.v1	AwaitingVerdict
AwaitingVerdict	RootCauseVerdictIssued.v1	AwaitingComplianceReview
AwaitingComplianceReview	ComplianceReviewOpened.v1	Completed
Completed	any duplicate	Completed / no transition

NO PHASE SKIPPING

An early VerdictIssued fact is durably buffered. It cannot bind a verdict until CaseOpened has established the expected case coordinates.

DEADLINE POLICY

Each expectation receives a policy-owned due time

Deadlines are relative to the time the preceding public fact was observed and stored, not to a long database transaction or caller wait. The policy identity and resolved duration are persisted so later configuration changes do not rewrite history.

EXECUTABLE ASSET 37-G - VERSIONED EXPECTATION POLICY

```
public sealed record WorkflowDeadlinePolicy(  
    string Identity,  
    TimeSpan CaseOpenedWithin,  
    TimeSpan VerdictWithin,  
    TimeSpan ReviewOpenedWithin);  
  
var policy = new WorkflowDeadlinePolicy(  
    "alarm-to-compliance/deadlines/1",  
    CaseOpenedWithin: TimeSpan.FromSeconds(30),  
    VerdictWithin: TimeSpan.FromMinutes(15),  
    ReviewOpenedWithin: TimeSpan.FromMinutes(2));
```

GOLDEN TIMING

The Chapter 36 corpus reaches CaseOpened in 0.535 s, VerdictIssued in 4 m 26.425 s and ReviewOpened in 0.589 s. The fixture is on time under policy version 1.

EXPECTED-EVENT RECORD

An expectation is durable evidence, not an in-memory timer

Each phase creates one ExpectedFact row with the event contract, match coordinates, deadline policy and status. Satisfaction records the exact MessageId and observation time. Superseded or repaired expectations remain in history.

EXECUTABLE ASSET 37-H - EXPECTATION ROW

```
public sealed class ExpectedFactRow
{
    public Guid ExpectationId { get; set; }
    public Guid WorkflowId { get; set; }
    public string EventType { get; set; } = null!;
    public int SchemaVersion { get; set; }
    public string MatchCoordinatesJson { get; set; } = null!;
    public string DeadlinePolicyIdentity { get; set; } = null!;
    public DateTimeOffset DueAtUtc { get; set; }
    public string State { get; set; } = null!; // Open | Satisfied
    public Guid? SatisfiedByMessageId { get; set; }
    public DateTimeOffset? SatisfiedAtUtc { get; set; }
    public byte? OpenSlot { get; private set; }
}
```

ONE OPEN EXPECTATION

A filtered unique index permits only one Open expectation per workflow. Transition and next expectation commit together.

WORKFLOWDB SCHEMA

State, inbox, observations, expectations and history have distinct roles

One table cannot safely represent transport deduplication, current phase, early facts and append-only transition evidence. WorkflowDb separates those concerns while keeping each handler commit owner-local.

DECISION LEDGER 37-19

Table	Purpose	Key
ProcessInstance	current workflow state	WorkflowId
InboxReceipt	transport idempotency	ConsumerName + MessageId
ObservedFact	immutable admitted lineage	WorkflowId + MessageId
ExpectedFact	current/historic expectations	ExpectationId
BufferedFact	early/unmatched valid facts	MessageId
Transition	append-only state history	WorkflowId + Sequence

NO FOREIGN KEYS ACROSS SERVICES

Caseld, VerdictId and ReviewId are public coordinates, not cross-database foreign keys. Referential meaning is proved by admitted facts and transition guards.

EF CORE MAPPING

Database constraints arbitrate duplicate starts and concurrent transitions

The provider mapping protects semantic uniqueness and optimistic concurrency. Insert collisions are classified from provider-specific constraint names; row updates retry from a new DbContext after a concurrency exception.

EXECUTABLE ASSET 37-I - CRITICAL MAPPING

```
model.Entity<ProcessInstanceRow>(b =>
{
    b.ToTable("ProcessInstance", "workflow");
    b.HasKey(x => x.WorkflowId);
    b.HasIndex(x => new { x.WorkflowType, x.AlarmFloodMessageId })
        .IsUnique().HasDatabaseName("UX_Workflow_StartFact");
    b.HasIndex(x => x.CorrelationId);
    b.Property(x => x.RowVersion).IsRowVersion();
});

model.Entity<ExpectedFactRow>(b =>
{
    b.Property(x => x.OpenSlot).HasComputedColumnSql(
        "CASE WHEN [State] = 'Open' THEN CONVERT(tinyint, 1) END",
        stored: true);
    b.HasIndex(x => new { x.WorkflowId, x.OpenSlot })
        .IsUnique().HasFilter("[OpenSlot] IS NOT NULL");
});
```

PROVIDER PROOF

The filtered index and rowversion behavior require SQL Server-backed component tests. An in-memory provider cannot substantiate these claims.

TX-PM-START

The Flood fact creates one durable process instance

Admission precedes mutation. The start transaction commits inbox receipt, immutable observation, process instance, first expectation and transition sequence 1. Broker acknowledgement follows the known WorkflowDb outcome.

EXECUTABLE ASSET 37-J - START TRANSACTION

```
await using var tx = await db.Database.BeginTransactionAsync(ct);

var process = AlarmToComplianceProcess.Start(
    ids.NewWorkflowId(), flood, deadlines, clock.GetUtcNow());
db.Processes.Add(process.ToRow());
db.Inbox.Add(receipts.Completed(flood));
db.ObservedFacts.Add(observations.From(flood, process.Id));
db.ExpectedFacts.Add(expectations.CaseOpened(process));
db.Transitions.Add(history.Started(process, flood));

await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

GOLDEN START

Workflow 019bc4d4-61a8-76f1-a935-b953af1dd9c4 starts from Flood 019bc4d4-5dc1-7b8d-9d48-542d25315356, correlation request-am-20482, and enters AwaitingCaseOpened.

TX-PM-CASE**CaseOpened satisfies one expectation and opens the next**

The matcher requires the same correlation, causation equal to the start Flood MessageId, source decision flood-am-6031 and the expected site/line. The transaction binds CaseId, closes the CaseOpened expectation and creates the VerdictIssued expectation.

EXECUTABLE ASSET 37-K - CASE TRANSITION

```
var process = await processes.GetForUpdateAsync(workflowId, ct);
var expected = await expectations.GetOpenAsync(workflowId, ct);

expected.DemandContract(CaseOpenedV1.Type, schema: 1);
matcher.DemandCaseOpened(process, delivery.Contract);

process.ObserveCaseOpened(delivery.Contract, deadlines, now);
expected.Satisfy(delivery.MessageId, now);
db.ExpectedFacts.Add(expectations.VerdictIssued(process));
db.Transitions.Add(history.CaseOpened(process, delivery));
db.Inbox.Add(receipts.Completed(delivery));
await db.SaveChangesAsync(ct);
```

ATOMIC PHASE ADVANCE

Inbox, observed fact, closed expectation, new expectation, current phase and history either commit together or remain unchanged.

TX-PM-VERDICT**VerdictIssued binds the owner verdict without recomputing it**

The transition verifies RootCauseCaseId, SourceAlarmFloodMessageId, site/line and producer stream position 2. It does not require the event causation to equal CaseOpened because the immediate cause is the RootCause verdict decision.

EXECUTABLE ASSET 37-L - VERDICT TRANSITION

```
matcher.DemandVerdictIssued(
    process,
    expectedCaseId: process.RootCauseCaseId!.Value,
    expectedFloodMessageId: process.AlarmFloodMessageId,
    delivery.Contract);

process.ObserveVerdictIssued(delivery.Contract, deadlines, now);
openExpectation.Satisfy(delivery.MessageId, now);
db.ExpectedFacts.Add(
    expectations.ComplianceReviewOpened(process));
db.Transitions.Add(history.VerdictIssued(process, delivery));
db.Inbox.Add(receipts.Completed(delivery));
await db.SaveChangesAsync(ct);
```

CAUSATION PRECISION

The workflow graph retains decision-rc-7184 -> VerdictIssued. Sequence membership comes from case identity and producer stream, not from inventing CaseOpened as the immediate cause.

TX-PM-COMPLETE

ReviewOpened closes the final expectation

The terminal transition requires the exact VerdictId and VerdictIssued MessageId, matching case, Pending semantics and direct causation. It records ReviewId and completion disposition, closes the expectation and appends the final transition.

EXECUTABLE ASSET 37-M - COMPLETION TRANSACTION

```
matcher.DemandReviewOpened(  
    process,  
    expectedVerdictId: process.VerdictId!.Value,  
    expectedVerdictMessageId: VERDICT_MESSAGE_ID,  
    delivery.Contract);  
  
process.ObserveComplianceReviewOpened(delivery.Contract, now);  
openExpectation.Satisfy(delivery.MessageId, now);  
db.Transitions.Add(history.Completed(process, delivery));  
db.Inbox.Add(receipts.Completed(delivery));  
db.ObservedFacts.Add(observations.From(delivery, process.Id));  
await db.SaveChangesAsync(ct);
```

TERMINAL CLAIM

Completion proves one Compliance Pending review was publicly observed for the exact verdict. It creates no Process Manager completion event because no approved consumer requires one yet.

TRANSITION HISTORY

Current state is explainable from an append-only ledger

ProcessInstance makes current reads efficient; Transition preserves how the workflow arrived there. Every entry records sequence, previous/next phase, trigger MessageId, event type, policy identity, observed time and build/config identity.

EXECUTABLE ASSET 37-N - TRANSITION EVIDENCE

```
public sealed record WorkflowTransitionRow(  
    Guid WorkflowId,  
    long Sequence,  
    string FromPhase,  
    string ToPhase,  
    Guid TriggerMessageId,  
    string TriggerEventType,  
    string TriggerBodySha256,  
    string PolicyIdentity,  
    DateTimeOffset ObservedAtUtc,  
    string RuntimeIdentity);
```

DECISION LEDGER 37-25

Golden sequence	Transition
1	none -> AwaitingCaseOpened
2	AwaitingCaseOpened -> AwaitingVerdict
3	AwaitingVerdict -> AwaitingComplianceReview
4	AwaitingComplianceReview -> Completed

ADMISSION BOUNDARY

Producer, route, envelope and bytes are verified before matching

The workflow consumer copies the delivery, authenticates the broker principal, demands the approved producer/route pair, verifies MessageId and canonical fingerprint, then selects a contract mapper. Correlation is considered only after the fact is trusted.

EXECUTABLE ASSET 37-O - STRICT WORKFLOW ADMISSION

```
public AdmittedWorkflowFact Admit(
    RawBrokerDelivery raw,
    AuthenticatedBrokerPrincipal principal)
{
    var route = routeCatalogue.GetRequired(raw.RoutingKey);
    authorization.DemandProducer(principal, route.Producer);

    var envelope = envelopes.ReadV1(raw.CopiedBody);
    envelope.DemandEvent(route.EventType, route.SchemaVersion);
    envelope.DemandMessageId(raw.Properties.MessageId);
    envelope.DemandProducer(route.Producer);
    fingerprints.Verify(envelope, raw.CopiedBody);
    return route.Mapper.Map(envelope, raw.CopiedBody);
}
```

NO CORRELATION TRUST SHORTCUT

A malicious or malformed body carrying a known CorrelationId never reaches the matcher or WorkflowDb transition code.

EXPECTED-EVENT MATCHER

Correlation narrows candidates; owner coordinates prove the match

A CorrelationId lookup can return multiple workflows. The matcher filters by expected contract and immutable coordinates from earlier facts, then requires exactly one candidate. Zero candidates may be buffered; multiple candidates are an ambiguity incident.

EXECUTABLE ASSET 37-P - EXACT CANDIDATE RESOLUTION

```
var candidates = await processes.ByCorrelationAsync(
    fact.CorrelationId, ct);

var matches = candidates
    .Where(x => x.OpenExpectation.MatchesContract(fact))
    .Where(x => coordinateMatcher.Matches(x, fact))
    .ToArray();

return matches.Length switch
{
    1 => MatchResult.Exact(matches[0].WorkflowId),
    0 => MatchResult.Unmatched,
    _ => MatchResult.Ambiguous(matches.Select(x => x.WorkflowId))
};
```

AMBIGUITY IS NOT A RACE TO FIRST

The consumer does not choose the oldest or newest correlated workflow. It quarantines ambiguity with the candidate identities for review.

CORRELATION COLLISION

Two workflows may legitimately share one request correlation

A component test starts two Flood facts under request-am-20482 and proves they receive different WorkflowIds. A later CaseOpened fact binds only the workflow whose AlarmFloodMessageId is named as causation and source identity.

EXECUTABLE ASSET 37-Q - COLLISION PROOF

```
var first = Golden.FloodDetected;
var second = Another.FloodDetected with
{
    CorrelationId = first.CorrelationId,
    MessageId = Another.FloodMessageId
};

await consumer.DeliverAsync(first);
await consumer.DeliverAsync(second);
await consumer.DeliverAsync(Golden.CaseOpened);

await db.AssertWorkflowCountAsync("request-am-20482", 2);
await db.AssertCaseBoundOnlyToAsync(
    first.MessageId, Golden.CaseId);
```

INDEXING RULE

CorrelationId is searchable and observable but never declared unique. Semantic start identity is the AlarmFloodMessageId.

EARLY DELIVERY

A valid later fact is buffered, acknowledged and revisited

Broker delivery order is not workflow order. If VerdictIssued arrives before CaseOpened, the manager stores the exact admitted body and match metadata in BufferedFact within the inbox transaction, then acknowledges. No phase transition occurs until the missing expectation is satisfied.

EXECUTABLE ASSET 37-R - DURABLE BUFFER OUTCOME

```
if (match is MatchResult.Unmatched &&
    contractSet.IsKnownLaterFact(fact.EventType))
{
    db.Inbox.Add(receipts.Buffered(fact));
    db.BufferedFacts.Add(BufferedFactRow.Pending(
        fact.MessageId,
        fact.CorrelationId,
        fact.EventType,
        fact.BodySha256,
        fact.CopiedBody,
        clock.GetUtcNow()));
    await db.SaveChangesAsync(ct);
    return ConsumeOutcome.Committed(fact.MessageId, "buffered");
}
```

SAFE ACKNOWLEDGEMENT

Acknowledgement is allowed because the exact body is durably owned by WorkflowDb. A memory-only buffer would require requeue instead.

BUFFER DRAIN

Each successful transition drains only the newly contiguous facts

After CaseOpened advances the phase, the handler queries Pending buffered facts for the same correlation and expected contract. It re-runs full coordinate matching and applies at most the bounded contiguous prefix inside new transactions.

EXECUTABLE ASSET 37-S - BOUNDED DRAIN LOOP

```
for (var applied = 0; applied < options.MaxDrainPerTurn; applied++)
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var worker = scope.ServiceProvider
        .GetRequiredService<BufferedFactApplier>();

    var outcome = await worker.TryApplyNextExpectedAsync(
        workflowId, ct);
    if (outcome is not ApplyOutcome.Applied)
        break;
}
```

FRESH SCOPE

Each buffered application uses a fresh DbContext and transaction. A concurrency conflict reloads current phase before another attempt.

DUPLICATE AND CONFLICT

Message identity and semantic transition are checked independently

Inbox deduplication resolves transport replay. Workflow guards and unique owner coordinates resolve semantic repetition. A reused MessageId with changed bytes is contract corruption; a new MessageId claiming an already-bound CaseId or VerdictId is a semantic conflict.

DECISION LEDGER 37-31

Input	Outcome
same MessageId + same body	stored duplicate outcome; ack
same MessageId + changed body	identity conflict; quarantine/reject
new MessageId + same satisfied fact	semantic duplicate or conflict by coordinates
valid unrelated correlation	new workflow or durable unmatched fact
known correlation + wrong case/verdict	coordinate conflict; no transition

NO LAST-WRITE-WINS

A later delivery cannot replace a previously observed owner identity. Corrections require a new approved public fact and explicit transition policy.

CONCURRENT CONSUMERS

Unique constraints and rowversion choose one winner

Two Process Manager instances may receive duplicate or adjacent deliveries. The database arbitrates start uniqueness and phase updates. A concurrency loser discards its tracked context, opens a new scope and reclassifies the delivery against committed state.

EXECUTABLE ASSET 37-T - SEMANTIC RETRY

```
for (var attempt = 1; attempt <= retry.MaxAttempts; attempt++)
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var handler = scope.ServiceProvider
        .GetRequiredService<WorkflowFactHandler>();
    try
    {
        return await handler.HandleOnceAsync(fact, ct);
    }
    catch (DbUpdateConcurrencyException) when (attempt < retry.MaxAttempts)
    {
        await retry.DelayAsync(attempt, ct);
    }
}
throw new WorkflowConcurrencyExhausted(fact.MessageId);
```

INSERT CONFLICTS DIFFER

A unique-index violation is classified by provider constraint name, not caught as `DbUpdateConcurrencyException`. The handler then resolves the existing semantic row.

CRASH SEMANTICS

Every transition has a named commit/ack recovery outcome

Crash probes cover admission, before `SaveChanges`, after commit before acknowledgement and after acknowledgement. The invariant is constant: either the transition is absent and the delivery remains available, or the complete `WorkflowDb` effect exists and replay resolves it.

DECISION LEDGER 37-33

Crash point	Database	Broker / retry
before <code>SaveChanges</code>	no receipt/transition	delivery unacked; redeliver
during <code>SaveChanges</code> failure	transaction rolled back	retry/queue
after commit before ack	complete transition exists	duplicate then ack
after ack	complete transition exists	no broker copy
during buffer drain	prior committed prefix only	next drain resumes

NO HALF TRANSITION

Current phase, expectation, observation, history and receipt are verified as one row oracle for every injected crash point.

BROKER SETTLEMENT

Acknowledgement follows a classified WorkflowDb outcome

Committed transitions, proved duplicates and durable buffers may be acknowledged. Transient provider failures are requeued with bounded policy. Identity conflicts and unsupported contracts are rejected into evidence-bearing quarantine without a hot retry loop.

DECISION LEDGER 37-34

Handler outcome	Settlement
Committed / Duplicate / Buffered	ack
Transient	nack requeue within retry budget
UnsupportedContract	reject, no requeue, quarantine evidence
IdentityConflict / Ambiguous	reject, no requeue, incident evidence
UnknownCommit	re-deliver same identity; resolve inbox

CHANNEL OWNERSHIP

The settlement operation uses the same channel that owns the delivery tag. Database completion never acknowledges a delivery on a different consumer channel.

TIMEOUT SCANNER

A background worker finds due open expectations in bounded batches

The worker creates one scope per batch, leases due expectations, marks workflow timeliness and appends timeout history. It does not hold a DbContext across delays and does not block consumer threads.

EXECUTABLE ASSET 37-U - SCOPED TIMEOUT WORKER

```
public sealed class WorkflowDeadlineWorker(
    IServiceScopeFactory scopes,
    TimeProvider clock) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        using var timer = new PeriodicTimer(
            TimeSpan.FromSeconds(5), clock);
        while (await timer.WaitForNextTickAsync(stoppingToken))
        {
            await using var scope = scopes.CreateAsyncScope();
            await scope.ServiceProvider
                .GetRequiredService<DeadlineBatch>()
                .RunAsync(clock.GetUtcNow(), stoppingToken);
        }
    }
}
```

BOUNDED SHUTDOWN

The host cancellation token reaches timer, lease query and SaveChanges. StopAsync evidence proves no new lease starts after shutdown begins.

DEADLINE LEASE

Multiple workers claim different due rows

The SQL Server implementation uses a short provider-specific claim transaction and an expiring lease. Business transition work happens after claim under a fresh unit of work; abandoned leases become eligible only after their recorded expiry.

EXECUTABLE ASSET 37-V - SQL SERVER CLAIM SHAPE

```
;WITH due AS (  
    SELECT TOP (@batchSize) *  
    FROM workflow.ExpectedFact WITH (UPDLOCK, READPAST, ROWLOCK)  
    WHERE State = 'Open'  
        AND DueAtUtc <= @now  
        AND (LeaseUntilUtc IS NULL OR LeaseUntilUtc < @now)  
    ORDER BY DueAtUtc, ExpectationId  
)  
UPDATE due  
SET LeaseOwner = @owner,  
    LeaseUntilUtc = @leaseUntil  
OUTPUT inserted.ExpectationId;
```

LEASE IS NOT COMPLETION

A lease only grants temporary processing authority. The expectation remains Open until a fact satisfies it; timeout processing marks timeliness separately.

OVERDUE TRANSITION

Deadline expiry records lateness without closing the expectation

The timeout transaction reloads the instance and expectation after the lease. If the expected fact already committed, the timeout becomes a no-op. Otherwise it sets OverdueSinceUtc once and appends an Overdue transition record.

EXECUTABLE ASSET 37-W - COMPARE-AND-TRANSITION

```
var expected = await db.ExpectedFacts  
    .SingleAsync(x => x.ExpectationId == leasedId, ct);  
var process = await db.Processes  
    .SingleAsync(x => x.WorkflowId == expected.WorkflowId, ct);  
  
if (expected.State == "Satisfied" || process.Phase == WorkflowPhase.Completed)  
    return DeadlineOutcome.AlreadySatisfied;  
  
if (process.MarkOverdue(clock.UtcNow()))  
    db.Transitions.Add(history.Overdue(process, expected));  
  
expected.ReleaseLease();  
await db.SaveChangesAsync(ct);
```

IDEMPOTENT TIMEOUT

Repeated scans do not append repeated Overdue transitions. The first OverdueSinceUtc is retained as evidence.

LATE FACT

A valid late event may still advance and complete the workflow

Overdue does not poison matching. The same strict expected-event guards apply. When the fact arrives after `DueAtUtc`, the transition records `CompletionDisposition.Late` and preserves both deadline and arrival times.

EXECUTABLE ASSET 37-X - LATE COMPLETION RULE

```
public void ObserveComplianceReviewOpened(
    ComplianceReviewOpenedV1 fact,
    DateTimeOffset observedAtUtc)
{
    DemandPhase(WorkflowPhase.AwaitingComplianceReview);
    DemandCoordinates(fact);

    ComplianceReviewId = new(fact.ReviewId);
    Completion = observedAtUtc <= NextDeadlineAtUtc
        ? CompletionDisposition.OnTime
        : CompletionDisposition.Late;
    Phase = WorkflowPhase.Completed;
}
```

HISTORY IS ADDITIVE

The earlier `Overdue` transition is not deleted when completion arrives. The workflow becomes `CompletedLate` with an explainable history.

TIMEOUT CLASSIFICATION

Late service and failed business decision are different facts

A deadline tells the manager that an expected public fact has not been observed by a policy time. It cannot tell whether the producer is down, a queue is backlogged, a decision abstained or an operator rejected work. Those require explicit owner facts or operational evidence.

DECISION LEDGER 37-39

Observation	Allowed label	Forbidden inference
due time passed	Overdue awaiting X	X failed
queue ready count > 0	known consumer lag	business rejection
owner publishes rejection fact	business outcome per contract	transport failure
late expected fact arrives	CompletedLate	rewrite deadline history

CHAPTER 39 SEAM

Chapter 39 will add durable operator-visible escalation and distinguish timeout actions. Chapter 37 provides the truthful overdue substrate only.

RESTART AND RESUME

No essential workflow state lives in worker memory

After process restart, the consumer resolves duplicates through InboxReceipt, the timeout worker reloads Open expectations and the buffer drainer resumes Pending facts. The host does not reconstruct progress by querying other service databases.

EXECUTABLE ASSET 37-Y - RESTART ORACLE

```
await scenario.RunUntilAsync(
    WorkflowCheckpoint.VerdictTransitionCommitted, ct);
await environment.KillProcessManagerAsync();
await environment.StartProcessManagerAsync();

var before = await workflowDb.RequiredAsync(WORKFLOW_ID, ct);
before.Phase.Should().Be(WorkflowPhase.AwaitingComplianceReview);

await broker.DeliverAsync(Golden.ComplianceReviewOpened, ct);
var after = await workflowDb.AwaitCompletedAsync(WORKFLOW_ID, ct);
after.TransitionCount.Should().Be(4);
```

NO STARTUP SWEEP MUTATION

Startup may inventory open work and expose metrics. It does not advance a phase unless an admitted fact or due expectation is processed through its normal transaction path.

REPLAY

Reprocessing the public history reconstructs the same process meaning

A shadow WorkflowDb generation can replay the retained four-fact history through the same admission, matching and transition code. It must produce the same semantic identities and phase, while consumer-owned observation timestamps and rowversions may differ.

DECISION LEDGER 37-41

Must match	May differ
Workflow semantic start key	replay run id
Caseld / VerdictId / ReviewId	AppliedAtUtc
phase and completion disposition	database rowversion
transition triggers and contract digests	lease owner / attempt count
policy identity	trace/span identities

IDENTITY SOURCE RULE

Production WorkflowId is allocated once and retained. A rebuild either imports that identity from trusted history or records an explicit generation-local mapping; it never presents a new identity as the original.

REBUILD ACTIVATION

A shadow generation becomes active only after comparison

The active WorkflowDb remains queryable while a shadow generation replays. A validator compares counts, semantic keys, final phases, open expectations, buffered facts and conflicts before an atomic active-generation pointer changes.

EXECUTABLE ASSET 37-Z - GENERATION GATE

```
var comparison = await validator.CompareAsync(
    activeGeneration, shadowGeneration, ct);

comparison.RequireEqual(
    "workflow-count",
    "semantic-start-key-set",
    "bound-owner-coordinate-set",
    "final-phase-set",
    "open-expectation-set");
comparison.RequireZero(
    "identity-conflicts",
    "ambiguous-matches",
    "unresolved-known-facts");

await generations.ActivateAsync(shadowGeneration, ct);
```

REPAIR BOUNDARY

Chapter 42 will build operational reconciliation and repair jobs. This chapter defines only the safe replay and comparison contract they must preserve.

RETENTION

Workflow evidence lives at least as long as its source lineage

Deleting inbox receipts before public facts can redeliver reopens duplicate risk. Deleting transition history before a Compliance review leaves retention destroys explainability. Retention is coordinated through policy, never broad age-only cleanup.

DECISION LEDGER 37-43

Artifact	Minimum retention dependency
InboxReceipt	broker replay/redelivery horizon
ObservedFact digest/body	contract audit and rebuild horizon
ExpectedFact / Transition	workflow + compliance evidence horizon
BufferedFact	resolved/quarantined plus investigation horizon
ProcessInstance	terminal retention and operator/audit policy

SAFE DELETION PROOF

A cleanup candidate is deleted only when every named dependency has expired and no legal/audit hold applies. Cleanup writes its own auditable batch record.

DIAGNOSTIC QUERY

One read model explains present progress without calling owners

The Process Manager exposes an internal/operator diagnostic query from WorkflowDb: current phase, expected fact, deadline, overdue state, bound identities and transition history. It is not a business command endpoint and does not replace owner APIs.

EXECUTABLE ASSET 37-AA - WORKFLOW OBSERVATION CONTRACT

```
public sealed record WorkflowObservationV1(  
    Guid WorkflowId,  
    string WorkflowType,  
    string CorrelationId,  
    string Phase,  
    ExpectedFactObservationV1? Expected,  
    DateTimeOffset? OverdueSinceUtc,  
    string? CompletionDisposition,  
    OwnerCoordinatesV1 Coordinates,  
    IReadOnlyList<TransitionObservationV1> History,  
    DateTimeOffset ObservedAtUtc);
```

FUTURE UI SEAM

Chapter 41 will define pending, delayed, completed and failed operator states. Chapter 37 supplies truthful workflow evidence without prescribing final UI wording.

HTTP EDGE

Workflow lookup uses stable authorization and absence semantics

An operator capability may query by WorkflowId or correlation. A missing row means not observed by Process Manager, not proof that AlarmManagement never detected a flood. Correlation lookup returns a collection because the value is non-unique.

EXECUTABLE ASSET 37-AB - DIAGNOSTIC ENDPOINTS

```
group.MapGet("/v1/workflows/{workflowId:guid}", GetWorkflow)  
    .RequireAuthorization("workflow.progress.read");  
  
group.MapGet("/v1/workflows", FindByCorrelation)  
    .RequireAuthorization("workflow.progress.read");  
  
static IResult Map(WorkflowLookup result) => result switch  
{  
    WorkflowLookup.Found x => Results.Ok(x.Value),  
    WorkflowLookup.NotObserved _ => Results.Problem(  
        statusCode: 404, extensions: new()  
        { ["code"] = "workflow.not-observed" }),  
    _ => Results.StatusCode(500)  
};
```

CACHE RULE

Progress responses are no-store in Phase 0 so an old deadline/status does not travel without its observation time.

COMPOSITION ROOT

The production host registers consumers, workers and owner-local persistence

The composition root exposes one Process Manager capability. Consumers and deadline worker receive scoped handlers through fresh scopes. The host has no HTTP clients or connection strings for AlarmManagement, RootCause, Compliance, Audit or Reporting databases.

EXECUTABLE ASSET 37-AC - PRODUCTION REGISTRATIONS

```
builder.Services.AddDbContext<WorkflowDbContext>(options =>
    options.UseSqlServer(configuration.GetConnectionString("WorkflowDb")));
builder.Services.AddScoped<WorkflowFactHandler>();
builder.Services.AddScoped<DeadlineBatch>();
builder.Services.AddScoped<BufferedFactApplier>();
builder.Services.AddSingleton<WorkflowContractSet>();
builder.Services.AddHostedService<WorkflowConsumerWorker>();
builder.Services.AddHostedService<WorkflowDeadlineWorker>();

builder.Services.AssertNoForeignOwnerConnections();
builder.Services.AssertScoped<WorkflowDbContext>();
```

TWO WORKERS, ONE OWNER

The event consumer and timeout scanner share WorkflowDb policy but not DbContext instances. Database concurrency and leases arbitrate races.

READINESS AND LIVENESS

Process health and workflow delay remain separate

Liveness proves the process can execute. Readiness proves required WorkflowDb schema, broker topology and contract-set identity are acceptable for new work. An overdue workflow is operational/business progress evidence, not automatic process unhealthiness.

DECISION LEDGER 37-47

Signal	Question answered
liveness	is the host event loop responsive?
readiness	can this build safely consume and persist new facts?
consumer heartbeat	has the delivery loop recently progressed?
open overdue count	how many workflows missed expectation deadlines?
oldest ready age	is broker backlog delaying observation?

NO BUSINESS QUERY IN LIVENESS

Liveness does not call WorkflowDb or RabbitMQ. Dependency checks belong to bounded readiness/diagnostics and cannot turn a remote outage into a process restart loop.

SECURITY BOUNDARY

The manager consumes public facts with least privilege

The runtime principal may consume/ack only workflow.alarm-to-compliance.v1 and perform DML only in WorkflowDb. A distinct migration principal owns DDL. The diagnostic edge requires workflow.progress.read and never returns captured raw bodies or credentials.

DECISION LEDGER 37-48

Principal	Allowed	Denied
workflow-runtime	WorkflowDb DML + queue consume	foreign DBs / publish business facts
workflow-migrator	WorkflowDb DDL	broker consume / runtime DML
workflow-reader	diagnostic query	transition mutation / raw body
topology-provisioner	queue/binding declaration	message consumption / DB access

CAPTURED BODY PROTECTION

Buffered fact bodies are encrypted/protected according to environment policy, access-audited and excluded from ordinary diagnostic responses.

ARCHITECTURE TESTS

The manager cannot become a shared decision engine

Static gates prohibit references to owner Domain/Application assemblies, command packages and foreign EF contexts. Only approved public contracts, Workflow Domain/Application, Workflow infrastructure and Edge are allowed.

EXECUTABLE ASSET 37-AD - DEPENDENCY GATES

```
Types.InAssembly(ProcessManagerApplication)
    .ShouldNot().HaveDependencyOnAny(
        "Nexus1.AlarmManagement.Domain",
        "Nexus1.RootCause.Domain",
        "Nexus1.Compliance.Domain",
        "Nexus1.Audit.Infrastructure",
        "Nexus1.Reporting.Infrastructure");

Types.InAssembly(ProcessManagerAssembly)
    .ShouldNot().ImplementInterface(typeof(IOwnerDecisionCommand));
```

PUBLIC-CONTRACT-ONLY COORDINATION

The manager may know stable public fact types and workflow-owned policies. It cannot instantiate owner aggregates or call their repositories.

OBSERVABILITY

Workflow transitions and transport attempts are linked but not conflated

Each consume attempt creates a messaging span. A successful transition records workflow phase, bounded outcome and expectation type. WorkflowId and CorrelationId remain trace/log attributes, not metric labels. Deadline scans record batch and lease outcomes.

EXECUTABLE ASSET 37-AE - BOUNDED TRANSITION TELEMETRY

```
transitionCounter.Add(1,
    new("workflow_type", "alarm-to-compliance/1"),
    new("from_phase", fromPhase),
    new("to_phase", toPhase),
    new("outcome", outcome));

activity?.SetTag("workflow.id", workflowId);
activity?.SetTag("workflow.correlation_id", correlationId);
activity?.SetTag("messaging.message.id", messageId);
activity?.SetTag("workflow.expected_event", expectedEventType);
```

LINEAGE PLANES

Business correlation and direct causation are stored in WorkflowDb. Trace context aids timing diagnosis and may be sampled without changing durable workflow meaning.

METRICS

Capacity signals remain bounded and actionable

The service reports open instances by phase, overdue count, transition outcome, buffered fact count, oldest buffer age, due-expectation lag, concurrency retries and settlement outcomes. Owner identities and correlation values never become metric dimensions.

DECISION LEDGER 37-51

Metric	Bounded labels
workflow_instances	workflow_type, phase, timeliness
workflow_transitions_total	from, to, outcome
workflow_buffered_facts	event_type, state
workflow_deadline_lag_seconds	expectation_type
workflow_consume_total	event_type, outcome, redelivered
workflow_concurrency_retries	operation, result

CARDINALITY BUDGET

WorkflowId, MessageId, ReviewId and CorrelationId belong in structured diagnostic evidence only. A metric-label test rejects them.

PERFORMANCE

Indexes follow matching, timeout and operator query shapes

The primary hot paths are inbox lookup, semantic start resolution, candidate correlation plus phase, one open expectation per workflow, due expectation scan and bounded buffer drain. Every index is justified by a captured provider plan and logical-read budget.

DECISION LEDGER 37-52

Query	Leading index coordinates
transport duplicate	ConsumerName, MessageId
start resolution	WorkflowType, AlarmFloodMessageId
candidate match	CorrelationId, Phase
current expectation	WorkflowId, OpenSlot
deadline scan	State, DueAtUtc, LeaseUntilUtc
buffer drain	CorrelationId, EventType, State, ObservedAtUtc

BATCH LIMIT

Consumer prefetch, deadline lease batch and buffer-drain limit are explicit configuration identities tested against memory, lock duration and shutdown budgets.

DOMAIN TESTS

The transition grammar is deterministic without infrastructure

Fast tests cover every allowed transition and every guard: wrong phase, wrong case, wrong verdict, wrong direct cause, deadline repetition, late completion and post-completion duplicate. The fixture uses the exact Chapter 36/37 identities.

EXECUTABLE ASSET 37-AF - AGGREGATE GOLDEN TEST

```
[Fact]
public void PM_DOMAIN_001_golden_facts_complete_once()
{
    var process = AlarmToComplianceProcess.Start(
        Golden.WorkflowId, Golden.FloodDetected,
        Golden.Deadlines, Golden.FloodObservedAt);

    process.ObserveCaseOpened(Golden.CaseOpened, Golden.CaseObservedAt);
    process.ObserveVerdictIssued(Golden.VerdictIssued, Golden.VerdictObservedAt);
    process.ObserveComplianceReviewOpened(
        Golden.ReviewOpened, Golden.ReviewObservedAt);

    process.Phase.Should().Be(WorkflowPhase.Completed);
    process.Completion.Should().Be(CompletionDisposition.OnTime);
}
```

NEGATIVE TESTS MATTER

A happy path alone cannot prove phase safety. Each mismatch asserts unchanged state and a stable failure code.

PROVIDER TESTS

SQL Server proves atomic transitions and concurrency

Component tests use the real provider to validate migration shape, filtered unique indexes, rowversion conflicts, transaction rollback, lease claims and provider error classification. Every test starts from an isolated database lease.

EXECUTABLE ASSET 37-AG - COMPETING TRANSITION TEST

```
[Fact]
public async Task PM_DB_003_two_workers_apply_one_transition()
{
    await fixture.SeedAwaitingVerdictAsync(Golden.Workflow);

    var outcomes = await fixture.RaceConsumersAsync(
        Golden.VerdictIssued, participants: 2);

    outcomes.Count(x => x.IsCommitted).Should().Be(1);
    outcomes.Count(x => x.IsDuplicate).Should().Be(1);
    await fixture.AssertPhaseAsync(
        Golden.WorkflowId, WorkflowPhase.AwaitingComplianceReview);
    await fixture.AssertTransitionCountAsync(Golden.WorkflowId, 3);
}
```

ROLLBACK ORACLE

Injected failure in history or next-expectation insert leaves inbox, observation, phase and previous expectation unchanged.

BROKER TESTS

Out-of-order and redelivery are proved on the real queue path

Broker-backed tests publish the exact golden bodies in deliberately reversed order, restart consumers around commit/ack gaps and verify durable buffer/drain behavior. Direct handler invocation cannot substantiate settlement claims.

EXECUTABLE ASSET 37-AH - REVERSED DELIVERY PROOF

```
await broker.PublishAsync(Golden.FloodDetectedBody, ct);
await broker.PublishAsync(Golden.VerdictIssuedBody, ct);
await broker.PublishAsync(Golden.CaseOpenedBody, ct);
await broker.PublishAsync(Golden.ReviewOpenedBody, ct);

var completed = await workflowDb.AwaitCompletedAsync(
    Golden.WorkflowId, ct);
completed.TransitionTriggers.Should().Equal(
    FLOOD_MESSAGE_ID,
    CASE_OPENED_MESSAGE_ID,
    VERDICT_MESSAGE_ID,
    REVIEW_OPENED_MESSAGE_ID);
await workflowDb.AssertNoPendingKnownBuffersAsync(ct);
```

TRANSPORT ORDER IS NOT SEMANTIC ORDER

The final transition order follows expectations even when broker processing order does not. No global broker order is claimed.

TIMEOUT TESTS

Synthetic time proves due, overdue and late behavior

The timeout suite injects FakeTimeProvider through the production composition root, advances exactly across each deadline and runs the real scoped DeadlineBatch. No test sleeps or depends on wall-clock scheduling.

EXECUTABLE ASSET 37-AI - DETERMINISTIC OVERDUE TEST

```
var time = fixture.FakeTime;
await fixture.SeedAwaitingComplianceReviewAsync(Golden.Workflow);

time.SetUtcNow(Golden.ReviewDeadline.AddTicks(-1));
await fixture.RunDeadlineBatchAsync();
await fixture.AssertNotOverdueAsync(Golden.WorkflowId);

time.Advance(TimeSpan.FromTicks(2));
await fixture.RunDeadlineBatchAsync();
await fixture.AssertOverdueOnceAsync(Golden.WorkflowId);

await fixture.DeliverAsync(Golden.ReviewOpened);
await fixture.AssertCompletedLateAsync(Golden.WorkflowId);
```

BOUNDARY PRECISION

The due comparison is specified and tested at one tick before, exactly due and one tick after. Time-zone conversion never participates.

PRODUCTION-PATH HOST TESTS

Program.cs, DI, migrations, worker scopes and diagnostic HTTP are exercised together

A host fixture boots the actual Process Manager composition root against isolated SQL Server and RabbitMQ resources. Only time, identities, credentials and resource locations are controlled. Contract set, handlers, mappings and workers are production registrations.

EXECUTABLE ASSET 37-AJ - COMPLETE HOST PATH

```
[Fact]
public async Task PM_RUNTIME_001_golden_workflow_completes()
{
    await fixture.PublishGoldenPathAsync();

    var observation = await fixture.AuthorizedClient
        .GetFromJsonAsync<WorkflowObservationV1>(
            $"/v1/workflows/{WORKFLOW_ID}");

    observation!.Phase.Should().Be("Completed");
    observation.CompletionDisposition.Should().Be("OnTime");
    observation.History.Select(x => x.Sequence)
        .Should().Equal(1, 2, 3, 4);
}
```

ZERO DISCOVERY GATE

The evidence validator requires discovered and executed test counts greater than zero and fails if the production-path category is skipped.

FAULT MATRIX

Each dependency outage preserves a resumable state

Focused scenarios stop one owner or Process Manager worker, capture the exact durable intermediate state, restore the component and require convergence without manual row edits or new business identities.

DECISION LEDGER 37-58

Fault	Durable state	Recovery
Process Manager offline	queue retains public facts	consume/buffer/advance after restart
Compliance offline	AwaitingComplianceReview / possibly overdue	ReviewOpened completes later
deadline worker offline	open expectation remains	scan marks overdue after restart
WorkflowDb unavailable	delivery unacked/requeued	same fact commits later
buffer drainer crash	pending buffered body retained	fresh scope resumes

NO REPAIR SHORTCUT

A test that updates Phase directly is rejected. Recovery must pass through admitted fact or deadline handlers.

EVIDENCE MANIFEST

The implementation claim remains PENDING until runtime artifacts exist

The chapter defines files, tests, migrations, topology and reproduction commands but does not assert that the repository currently contains or has executed them. The manifest names every missing artifact and rejects a partial or stale bundle.

EXECUTABLE ASSET 37-AK - FAIL-CLOSED CLAIM

```
{
  "claimId": "NEXUS1-PROCESS-MANAGER-V1-001",
  "workflowType": "alarm-to-compliance/1",
  "status": "PENDING",
  "buildIdentity": null,
  "providerTrx": null,
  "brokerTrx": null,
  "runtimeTrx": null,
  "topologyManifest": null,
  "executedArtifacts": [],
  "missing": [
    "repository implementation",
    "SQL Server evidence",
    "RabbitMQ evidence",
    "production-host evidence"
  ]
}
```

NOTHING CLAIMS TO EXIST THAT DOES NOT

The proposed paths and code are implementation targets. Runtime status changes only when exact build/config/schema identities and retained .trx/provider/broker artifacts pass validation.

CI EVIDENCE

One command should produce the process-manager dossier

The repository target restores locked dependencies, builds Release, provisions isolated WorkflowDb/broker resources, runs unit, provider, broker, timeout and host suites, validates artifacts and archives a value-free environment manifest.

EXECUTABLE ASSET 37-AL - PROPOSED REPRODUCTION PATH

```
dotnet test tests/Nexus1.ProcessManager.EvidenceTests \
  --configuration Release \
  --no-restore \
  --filter "Category=Chapter37" \
  --logger "trx;LogFileName=chapter37.trx" \
  --results-directory artifacts/evidence/chapter37
```

```
dotnet run --project tools/EvidenceValidator -- \
  artifacts/evidence/chapter37/manifest.json
```

REQUIRED BINDING

The dossier records repository URI, commit, dirty flag, package lock digest, migration identities, topology digest, command line and artifact SHA-256 values.

ADR BOUNDARY

Chapter 37 implements state ownership without pre-empting Chapter 38

The implemented minimum is fact-driven: it observes owner decisions, tracks expected facts and time, and exposes workflow evidence. It issues no domain commands. That gives Chapter 38 an actual runtime model against which to compare choreography, orchestration and a possible hybrid.

DECISION LEDGER 37-61

Question	Chapter 37 answer
Who owns progress?	Process Manager / WorkflowDb
Who owns local decisions?	AlarmManagement, RootCause, Compliance
How does it advance?	approved public facts
What happens on timeout?	durable Overdue marker; expectation stays open
Does it issue commands?	No, not before Chapter 38 ADR
Does it publish completion?	No approved consumer; state remains local

PRESERVED DECISION OWNERSHIP

The manager coordinates evidence of progress. It does not centralize RootCause or Compliance policy behind a workflow status enum.

OFFICIAL TECHNICAL ANCHORS

Durability is demonstrated at the boundaries that own it

- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- EF Core optimistic concurrency and SQL Server rowversion: <https://learn.microsoft.com/ef/core/saving/concurrency>
- Scoped services in BackgroundService: <https://learn.microsoft.com/dotnet/core/extensions/scoped-service>
- TimeProvider and deterministic testing: <https://learn.microsoft.com/dotnet/standard/datetime/timeprovider-overview>
- RabbitMQ queues and ordering: <https://www.rabbitmq.com/docs/queues>
- RabbitMQ acknowledgements and publisher confirms: <https://www.rabbitmq.com/docs/confirms>
- RabbitMQ .NET client guide: <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>
- W3C Trace Context: <https://www.w3.org/TR/trace-context/>
- RFC 8785 JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785>

WHAT THIS CHAPTER HAS PROVED

Chapter 37 specifies a fact-driven production Process Manager with one WorkflowDb; an activated ComplianceReviewOpened.v1 contract and atomic Compliance outbox; exact workflow identity and matching; phase plus timeliness state; inbox, observation, expectation, buffer and append-only transition evidence; owner-local transactions; out-of-order, duplicate, concurrency, crash, restart, replay, timeout and late-completion behavior; security, observability, performance and fail-closed verification paths. Runtime evidence remains PENDING.

CHAPTER CHECKPOINT

- Why is CorrelationId indexed but not unique?
- Which exact fact starts the workflow and which exact fact completes it?
- Why does VerdictIssued not name CaseOpened as its direct cause?
- When may an early delivery be acknowledged safely?
- Why are workflow phase and overdue state separate?
- How can a late fact complete without erasing timeout history?
- Which database constraints arbitrate duplicate starts and concurrent transitions?
- Why are Audit, Reporting and RL not completion gates?
- What artifacts must exist before the implementation claim leaves PENDING?

NEXT - CHAPTER 38

Choreography, Orchestration, and the Decision Record will compare the fact-driven manager, command-issuing orchestration and a possible hybrid against the actual runtime. It will record the chosen coordination model in an ADR while preserving local decision ownership.

PART VII - WORKFLOW, CONSISTENCY, AND REPAIR

38. Choreography, Orchestration, and the Decision Record

The actual Alarm-to-Compliance runtime chooses fact-driven choreography with a durable observational Process Manager. WorkflowDb owns progress and deadlines; AlarmManagement, RootCause and Compliance retain every business decision.

CHAPTER DECISION

ADR-038 accepts fact-driven choreography with an observational Process Manager. The normal path advances through the four approved public facts. The Process Manager correlates, records expectations, marks overdue state and supports query/rebuild, but its approved public command set is explicitly empty. Any future command requires a superseding ADR and must pass target ownership, necessity, refusal, idempotency, expiry, authorization and evidence gates.

DECISION PATH 38-A - COMPARE AGAINST THE REAL RUNTIME



GUARANTEE LEDGER 38-A

Chosen responsibility	Explicitly not granted
owner facts advance the normal path	central OpenCase / IssueVerdict / OpenReview commands
WorkflowDb explains partial progress	shared database or global transaction
manager records expectations and lateness	timeout interpreted as business failure
ADR and topology make authority testable	runtime proof without retained artifacts

DECISION QUESTION

The runtime must choose authority, not a fashionable label

Chapter 37 supplied a durable process state, but that state alone does not answer whether services coordinate through public facts, through centrally issued commands, or through both. The decision must be made against the exact Alarm-to-Compliance slice, its failure modes and its ownership rules.

DECISION LEDGER 38-2

Question	Required answer
Who decides?	the service that owns the business invariant
Who observes progress?	the Process Manager in WorkflowDb
What advances the normal path?	the four approved public facts
What may be requested centrally?	nothing in the current contract set
What proves the choice?	ADR, topology, tests and retained evidence

DECISION DISCIPLINE

A durable coordinator is not automatically an orchestrator. Command authority exists only when an approved command contract and target-owned refusal policy exist.

VOCABULARY

Facts, commands and observations carry different authority

The comparison becomes misleading when every message is called an event. A public fact reports owner truth that already committed. A command requests a future target-owned action. A workflow observation is Process-Manager state derived from admitted facts and is not public owner truth.

DECISION LEDGER 38-3

Message	Grammar	Authority
public fact	past tense: VerdictIssued	producer states committed owner truth
public command	imperative: OpenReview	sender requests; target accepts or refuses
workflow observation	AwaitingReview / Overdue	manager describes observed progress
query	GetWorkflow	caller asks without requesting mutation

NAMING RULE

A command is not a disguised event, and an overdue workflow is not evidence that a business decision failed.

ACTUAL RUNTIME

Four owner facts already form the normal coordination path

The current slice starts after AlarmManagement commits its flood decision and ends after Compliance commits one Pending review. RootCause and Compliance act because their own consumers and policies interpret sufficient public facts; the Process Manager observes the same lineage to make progress durable and explainable.

DECISION LEDGER 38-4

Fact	Owner commit	Manager meaning
AlarmFloodDetected.v1	AlarmManagement decision + outbox	start workflow
RootCauseCaseOpened.v1	RootCause case + outbox	bind case
RootCauseVerdictIssued.v1	RootCause verdict + outbox	bind verdict
ComplianceReviewOpened.v1	Compliance review + outbox	complete workflow

CURRENT COMMAND INVENTORY

Zero public commands participate in the approved normal path. This is a fact about the designed contract set, not yet a claim about a deployed broker.

LINEAGE

Correlation groups the journey; causation names each direct hop

All four facts share one CorrelationId so the workflow can group them. Direct causation changes at each hop: CaseOpened is caused by FloodDetected, VerdictIssued by the case/evidence decision, and ReviewOpened by VerdictIssued. The manager must not flatten this lineage into one original cause.

EXECUTABLE ASSET 38-A - FIXED WORKFLOW LINEAGE

```
var lineage = new[]
{
    ("AlarmFloodDetected.v1", "request-am-20482", "request-am-20482"),
    ("RootCauseCaseOpened.v1", "request-am-20482", "flood-message-id"),
    ("RootCauseVerdictIssued.v1", "request-am-20482", "case/evidence-message-id"),
    ("ComplianceReviewOpened.v1", "request-am-20482", "verdict-message-id")
};

Assert.Single(lineage.Select(x => x.Item2).Distinct());
Assert.Equal(4, lineage.Select(x => x.Item3).Distinct().Count());
```

BOUNDARY

Correlation supports lookup. It never proves uniqueness, ordering, direct causation or permission to mutate another service.

PRESERVATION CONSTRAINTS

Any candidate must preserve six already-proved boundaries

The coordination choice is constrained by earlier chapters. It cannot reopen shared-database coupling, hide local commits behind a global transaction, weaken replay safety, move decision policy into WorkflowDb, invent exactly-once delivery or make owner progress depend on Process-Manager availability.

DECISION LEDGER 38-6

Constraint	Reject the candidate if it ...
local decision ownership	decides flood, verdict or review outside the owner
one local commit	requires atomic commit across databases or broker
autonomous owner progress	blocks owner commit on coordinator availability
replay safety	cannot deduplicate or compare exact message identity
repairability	cannot rebuild workflow state from retained facts
honest evidence	labels unexecuted reference code as proved runtime

BLOCKER-FIRST COMPARISON

A model that violates one hard boundary is rejected even if it scores well on convenience, diagram simplicity or perceived control.

CANDIDATE A

Pure choreography removes the durable workflow owner

In the strictest choreography option, each service reacts to public facts and no component owns the end-to-end expectation. This preserves service autonomy and keeps the normal path decentralized, but it leaves workflow phase, missing-fact diagnosis and timeout history scattered across queues, logs and owner databases.

DECISION LEDGER 38-7

Strength	Cost in this runtime
no central command authority	no durable cross-service progress record
owner-local reactions	operators reconstruct state from many surfaces
coordinator outage impossible	no single place owns expected-next-fact
low command coupling	repair logic becomes subscriber-specific

RESULT

Useful facts still choreograph owner work, but removing WorkflowDb would discard the explainability and repair boundary established in Chapter 37.

CANDIDATE A FAILURE REVIEW

A missing fact has no authoritative workflow interpretation

Suppose a verdict exists while Compliance is offline. Owner truth is safe, but an operator needs to know what completed, what is expected next, when it became late and which exact input should be replayed. Pure choreography can infer pieces from telemetry, yet no durable component owns that combined statement.

DECISION LEDGER 38-8

Question	Without WorkflowDb
Which workflow instance?	correlate logs and message properties
What is expected?	infer from consumer topology
When did it become overdue?	recompute with potentially changed policy
Was a late fact observed?	search independent subscriber records
Can state be rebuilt deterministically?	only after inventing a projection contract

HIDDEN COORDINATOR

If an operational projection is later created to answer these questions, the system has recreated a process manager without naming its invariants.

CANDIDATE B

Command-issuing orchestration centralizes the next requested action

A classic orchestrator consumes a start signal, persists state and instructs each participant. This makes sequencing explicit and can centralize recovery, but it also introduces public command contracts, target refusal semantics, stronger security privileges and a temptation to move owner policy into the workflow state machine.

DECISION LEDGER 38-9

Potential benefit	New obligation
explicit requested next step	versioned command and target authorization
central retry policy	expiry, deduplication and refusal semantics
single workflow view	orchestrator availability and scaling
central compensation plan	no fiction that history was undone

NOT INHERENTLY WRONG

Orchestration is valid when the runtime genuinely needs a central request. The current slice must not adopt it merely because a Process Manager exists.

REJECTED COMMAND PATH

OpenCase and IssueVerdict would centralize policy by accident

The tempting orchestrator translates workflow phase into domain commands. That makes Process-Manager availability part of owner progress and lets WorkflowDb decide when a case or verdict should exist. It duplicates knowledge already held by RootCause consumers and policies.

REJECTED SHAPE 38-A - WORKFLOW STATE BECOMES BUSINESS AUTHORITY

```
switch (workflow.Phase)
{
    case Phase.Started:
        await commands.Send(new OpenRootCauseCase(workflow.FloodId));
        break;
    case Phase.CaseOpened:
        await commands.Send(new IssueRootCauseVerdict(workflow.CaseId));
        break;
    case Phase.VerdictIssued:
        await commands.Send(new OpenComplianceReview(workflow.VerdictId));
        break;
}
```

WHY REJECTED

The manager has insufficient private evidence to make these decisions. Retrying such commands would also require public refusal, expiry and idempotency contracts that do not exist.

CANDIDATE B FAILURE REVIEW

Coordinator loss must not stop facts that owners can interpret directly

If the command orchestrator is unavailable after FloodDetected, RootCause receives no OpenCase request even though the public fact already contains sufficient input for its owner-local policy. The extra hop increases coupling without adding needed authority.

DECISION LEDGER 38-11

Failure	Command orchestrator impact	Current fact path
manager offline	next command not issued	owner consumer may progress
duplicate command	target needs command inbox	existing fact inbox applies
stale command	target must expire/refuse	owner evaluates current fact policy
compromised manager	can request privileged mutations	observer cannot decide
version mismatch	command and result coupling	facts evolve independently

BLOCKER

The current command model fails autonomy and least-authority constraints. It is rejected for this runtime, not universally.

CANDIDATE C

Fact-driven choreography plus an observational Process Manager

The chosen model keeps owner reactions on public facts and retains WorkflowDb as a durable observer of progress. The manager binds identities, records expected facts and deadlines, exposes workflow status and supports replay comparison; it does not issue domain commands.

DECISION LEDGER 38-12

Surface	Authority
AlarmManagement	detect flood; publish owner fact
RootCause	open case; issue or abstain from verdict
Compliance	open and later resolve review
Process Manager	observe, correlate, expect, mark overdue, expose progress
RabbitMQ	transport deliveries; never decide business meaning

CHOSEN NAME

ADR-038 calls this fact-driven choreography with an observational Process Manager. The durable observer is central for evidence, not central for business decisions.

STRUCTURAL HYBRID

The model is centralized for progress and decentralized for decisions

Calling the result either pure choreography or full orchestration would hide an important distinction. Owner work is choreographed by facts. Workflow evidence is centrally projected. Commands remain absent. The architecture is therefore structurally hybrid without granting hybrid command authority.

DECISION LEDGER 38-13

Dimension	Chosen placement
business decision	decentralized to owner service
normal trigger	public owner fact
progress model	central WorkflowDb projection
deadline observation	central Process Manager policy
public command issuance	none approved
operator query	central workflow read model

PRECISION

The word hybrid describes placement of responsibilities. It is not permission to add commands later without a new decision record.

DECISION MATRIX

The candidates are compared by preserved boundaries

The matrix records blocking outcomes rather than weighted preferences. A check means the model can satisfy the requirement with the current contract set. A conditional result means additional contracts or authority are required. A block means the model contradicts an established boundary.

DECISION LEDGER 38-14

Requirement	Pure choreography	Command orchestration	Observed choreography
owner decisions remain local	yes	conditional	yes
durable workflow explanation	block	yes	yes
manager outage isolates owner progress	n/a	block	yes
current contracts sufficient	conditional	block	yes
least mutation authority	yes	block	yes
deterministic rebuild	conditional	yes	yes

OUTCOME

Observed choreography is the only candidate that preserves all hard constraints with the contracts already justified by Chapters 31-37.

HARD BLOCKERS

Two alternatives fail for different reasons

Pure choreography fails because the book explicitly requires distributed partial progress to be explainable and repairable. Command orchestration fails because the current runtime has sufficient owner facts and no justified central request. The chosen model avoids both failures.

DECISION LEDGER 38-15

Candidate	Blocking defect	Could revisit when ...
pure choreography	no durable expectation owner	another proved component owns equivalent state
command orchestrator	unnecessary mutation authority	a narrow command passes the admissibility gate
observed choreography	none against current constraints	facts or ownership model materially change

NO UNIVERSAL WINNER

The ADR decides for NEXUS-1's current Alarm-to-Compliance runtime. It is not a blanket claim that choreography always outperforms orchestration.

DECISION OWNERSHIP

Workflow state may describe an owner decision but never replace it

WorkflowDb stores foreign identities and public fact digests so the operator can see progress. Those fields are evidence references. They are not editable substitutes for the Alarm decision, RootCause verdict or Compliance review.

DECISION LEDGER 38-16

WorkflowDb field	Meaning	Forbidden inference
SourceFloodMessageld	exact admitted start fact	manager detected a flood
RootCauseCaseld	case identity observed in owner fact	manager may edit case
VerdictId	verdict identity observed	manager issued/corrected verdict
ComplianceReviewId	review identity observed	manager approved review
Phase / Timeliness	projection state	owner business state

FOREIGN IDENTITY RULE

Store an immutable reference plus lineage. Never add cross-database foreign keys or a workflow API that mutates the owner record.

AUTHORITY PROFILE

Allowed workflow actions are observational and owner-local

The authority profile is explicit so code review, DI tests and security policy can reject accidental expansion. Internal WorkflowDb transitions are allowed; external owner mutations are not.

DECISION LEDGER 38-17

Allowed	Forbidden
admit one approved fact	publish OpenCase / IssueVerdict / OpenReview
create or advance WorkflowDb row	write AlarmManagementDb / RootCauseDb / ComplianceDb
record expectation and deadline	convert timeout into owner failure
buffer/drain early delivery	repair owner data silently
expose status and lineage	impersonate an owner producer
rebuild and shadow-compare	rewrite retained owner fact history

SECURITY CONSEQUENCE

The Process-Manager principal needs consume rights for its queue and write rights for WorkflowDb. It needs no owner database credential and no command-publish grant.

NORMAL PROGRESSION

Owner commits do not wait for workflow observation

Each producer completes one owner-local transaction and publishes through its outbox. The manager may observe immediately, late or after restart. Its lag changes the freshness of workflow evidence, not the validity of owner truth.

EXECUTABLE ASSET 38-B - OBSERVATIONAL TRANSITION CONTRACT

```
public interface IWorkflowFactHandler<in TFact>
{
    Task<ObservationOutcome> ObserveAsync(
        AdmittedFact<TFact> fact,
        CancellationToken cancellationToken);
}

public enum ObservationOutcome
{
    Advanced, Buffered, MatchingDuplicate, Rejected
}

// There is deliberately no ICommandPublisher dependency.
```

AVAILABILITY BOUNDARY

If WorkflowDb is unavailable, the manager retries its own delivery. AlarmManagement, RootCause and Compliance continue to own and commit their work independently.

FAILURE ISOLATION

Process-Manager outage creates evidence lag, not domain rollback

A stopped manager leaves deliveries queued and its read model stale. It cannot invalidate already committed cases, verdicts or reviews. Recovery drains the queue through the same inbox and transition rules, then exposes the observed lag interval.

DECISION LEDGER 38-19

During outage	After restart
owner facts continue to publish	manager redeliveries are admitted
WorkflowDb phase is stale	transitions advance or buffer deterministically
queue depth/age grows	lag returns toward zero
no owner transaction rolls back	no owner record is rewritten
operator sees freshness warning	status records recovery evidence

SLO IMPLICATION

Workflow freshness needs its own objective. It must not be conflated with owner commit success or end-to-end business completion.

OWNER OUTAGE

A missing owner fact remains an open expectation

When RootCause or Compliance is unavailable, the manager retains the last observed owner fact and the exact next expectation. Deadline passage marks Overdue while the business phase remains unchanged. A later fact can advance the workflow without erasing the overdue history.

DECISION LEDGER 38-20

Condition	Manager state	Not claimed
RootCause offline after flood	AwaitingCaseOpened + timeliness	case failed
Compliance offline after verdict	AwaitingReviewOpened + timeliness	review rejected
late fact arrives	advance + append late observation	deadline never elapsed
unknown final outcome	expectation stays durable	silent compensation occurred

CHAPTER 39 HANDOFF

How an overdue expectation becomes operator-visible escalation belongs to Chapter 39. ADR-038 only preserves the separation needed for that design.

DELIVERY ANOMALIES

Duplicates and reordering do not require central commands

The manager uses exact MessageId, body digest, owner stream identity and current transition preconditions. Matching duplicates are acknowledged; conflicting duplicates are quarantined; admissible early facts are buffered; impossible transitions fail closed.

DECISION LEDGER 38-21

Observation	Disposition
same MessageId + same digest	matching duplicate; no second transition
same MessageId + different digest	conflict; reject/quarantine
Verdict before CaseOpened	buffer only if exact identities can later bind
Review for different verdict	reject; foreign workflow
unknown event type/version	reject; no guessed meaning

TRANSPORT HONESTY

The design is at-least-once with idempotent owner-local effects. It does not claim exactly-once delivery or global ordering.

CONTRACT EVOLUTION

A new fact is not an automatic workflow transition

Bindings and transition handlers list exact event type and schema version. Forward-compatible JSON parsing does not grant semantic compatibility. A new version must state its mapping, compatibility corpus and rollout order before the manager accepts it.

EXECUTABLE ASSET 38-C - CLOSED TRANSITION REGISTRY

```
var transitions = new WorkflowTransitionRegistry()
    .Add<AlarmFloodDetectedV1>(WorkflowAction.Start)
    .Add<RootCauseCaseOpenedV1>(WorkflowAction.BindCase)
    .Add<RootCauseVerdictIssuedV1>(WorkflowAction.BindVerdict)
    .Add<ComplianceReviewOpenedV1>(WorkflowAction.Complete);

Assert.False(transitions.Accepts(
    "nexus1.root-cause.root-cause-verdict-issued.v2"));
```

EVOLUTION RULE

Deploy tolerant consumers only for explicitly compatible meaning. Unknown workflow semantics fail closed even if deserialization succeeds.

TRUST BOUNDARY

A compromised observer must not gain mutation authority

The selected model reduces blast radius by keeping the manager off owner write paths. It still handles sensitive lineage and can misreport progress if compromised, so producer identity, contract admission, database authorization and audit history remain mandatory.

DECISION LEDGER 38-23

Threat	Required control
forged producer	broker identity + envelope producer allowlist
body tampering	canonical fingerprint and retained body digest
workflow row rewrite	least-privilege principal + append-only transition evidence
command injection	no command exchange, publisher or route
owner database access	no credential in config/secret inventory

RESIDUAL RISK

The manager can delay or distort its read model. It cannot legitimately create the owner fact that the read model is meant to observe.

LEAST PRIVILEGE

The deployment manifest encodes zero command capability

Runtime permissions should make the ADR true even if application code regresses. The service consumes only its dedicated fact queue, writes WorkflowDb and exposes a read/status endpoint. It cannot publish to owner command exchanges.

EXECUTABLE ASSET 38-D - CAPABILITY MANIFEST

```
service: process-manager
rabbitmq:
  consume: [workflow.alarm-to-compliance.v1]
  publish: []
database:
  runtime: [WorkflowDb:read-write]
  migrations: [WorkflowDb:ddl]
  forbidden:
    - AlarmManagementDb
    - RootCauseDb
    - ComplianceDb
httpClient: []
```

DEPLOY-TIME GATE

A non-empty publish list, owner database credential or owner mutation client must fail manifest validation before deployment.

ADR

ADR-038 records the chosen coordination model

The decision record is Accepted because the architecture choice is made. Its evidence status remains PENDING because the repository, provider and broker test artifacts described here have not been retained in this PDF production run.

DECISION LEDGER 38-25

ADR field	Value
Id	ADR-038
Decision	fact-driven choreography + observational Process Manager
Normal path	approved public facts only
Public commands	empty approved set
Decision owners	AlarmManagement, RootCause, Compliance
Evidence	PENDING until exact artifacts pass validation

STATUS DISTINCTION

Accepted describes the architecture decision. PENDING describes the absence of retained runtime proof. These statuses answer different questions.

ADR SCHEMA

The decision record is machine-readable and reviewable

A typed record prevents prose and pipeline checks from drifting apart. The approved command set is a required collection, not an omitted field whose meaning could be guessed differently by tools.

EXECUTABLE ASSET 38-E - ADR CONTRACT

```
public sealed record CoordinationAdr(  
    string AdrId,  
    string Title,  
    AdrStatus Status,  
    EvidenceStatus EvidenceStatus,  
    string ModelIdentity,  
    NormalPath NormalPath,  
    ImmutableArray<string> ApprovedPublicCommands,  
    string WorkflowOwner,  
    ImmutableArray<string> DecisionOwners,  
    ImmutableArray<string> ContractSet,  
    ImmutableArray<string> RevisitTriggers);  
  
public enum NormalPath { PublicFactsOnly }
```

FAIL CLOSED

Missing or null ApprovedPublicCommands is invalid. The current decision is represented by an explicit empty array.

CANONICAL ADR INSTANCE

One canonical JSON body receives a stable identity

The ADR body is serialized with sorted keys, UTF-8 and no insignificant whitespace before hashing. The digest binds review, CI evidence and deployment manifests to the same decision content.

EXECUTABLE ASSET 38-F - IMMUTABLE ADR IDENTITY

```
const string Model = "nexus1/coordination/observed-choreography/1";  
const string ExpectedSha256 = "622787772a9def5ef18bffc6a622d6efc11185d0a696780445a3ee8cc027f22a";  
  
byte[] body = CanonicalJson.Serialize(adr038);  
string actual = Convert.ToHexString(  
    SHA256.HashData(body)).ToLowerInvariant();  
  
Assert.Equal(ExpectedSha256, actual);  
Assert.Empty(adr038.ApprovedPublicCommands);
```

EXACT FIXTURE

ADR-038 canonical body: 695 UTF-8 bytes; SHA-256
622787772a9def5ef18bffc6a622d6efc11185d0a696780445a3ee8cc027f22a.

ADR CONSEQUENCES

The choice creates obligations as well as benefits

The model isolates owner decisions from coordinator availability and minimizes authority. In exchange, it requires every owner fact to carry sufficient stable meaning, WorkflowDb freshness to be visible, and operations to distinguish missing observation from missing business work.

DECISION LEDGER 38-28

Positive consequence	Accepted cost
owner services progress independently	facts require careful compatibility discipline
manager has low mutation authority	cannot centrally force progress
workflow is explainable/rebuildable	projection freshness is eventually consistent
no new command contracts	operator actions need separate future design
failure isolation is clear	multiple evidence surfaces remain

ACCEPTED TRADE-OFF

The system prefers honest partial progress and explicit repair over central control that appears simpler but owns the wrong decisions.

REJECTED ALTERNATIVES

The ADR preserves why the other models were not selected

Rejected alternatives remain first-class record content so a future team does not repeat the comparison from memory. Each rejection names the current blocker and the evidence that could justify revisiting it.

DECISION LEDGER 38-29

Alternative	Why rejected now	Revisit evidence
no process manager	loses durable expectation and repair view	equivalent proved projection elsewhere
command all participants	centralizes unnecessary mutation authority	admissible command with target policy
shared workflow database	breaks service-owned data	not applicable
poll owner databases	schema/credential/availability coupling	not applicable
global transaction	unavailable ownership model	not applicable

DECISION MEMORY

A rejected option is not forbidden forever unless it violates a permanent boundary. Its revisit trigger must be explicit and testable.

MODEL IDENTITY

Runtime surfaces must report the selected model without leaking secrets

Health, diagnostic and evidence endpoints may expose the model identity, ADR digest, contract-set digest and approved command count. They must not reveal broker credentials, full connection strings, message bodies or operator data.

EXECUTABLE ASSET 38-G - VALUE-FREE COORDINATION IDENTITY

```
public sealed record CoordinationIdentity(  
    string ModelIdentity,  
    string AdrId,  
    string AdrSha256,  
    int ApprovedPublicCommandCount,  
    string ContractSetSha256);  
  
var identity = new CoordinationIdentity(  
    "nexus1/coordination/observed-choreography/1", "ADR-038",  
    "622787772a9def5ef18bffc6a622d6efc11185d0a696780445a3ee8cc027f22a", 0, contractSetDigest);
```

OPERATIONAL USE

Environment comparison can prove the same coordination policy is deployed without exporting configuration values or secrets.

COMMAND TAXONOMY

Not every future request would steal a domain decision

The empty command set applies to the current normal path. A future request could be admissible if it asks a target to perform a target-owned, reversible or rejectable action without prescribing the business result. That distinction must be proven, not assumed.

DECISION LEDGER 38-31

Potential request	Current disposition	Reason
OpenRootCauseCase	forbidden	fact already supplies owner input
IssueVerdict	forbidden	would prescribe owner decision timing
OpenComplianceReview	forbidden	Compliance owns opening policy
ApproveReview	forbidden	human/Compliance authority
RequestOperatorAttention	unapproved	Chapter 39 must define refusal/expiry

NO PREMATURE CONTRACT

Chapter 38 does not add RequestOperatorAttention. It only states the gate that any Chapter 39 proposal must pass.

COMMAND ADMISSIBILITY

Seven blockers guard every proposed public command

A command can enter the public contract set only when the target owns the action and can independently accept, refuse, deduplicate and expire it. Security, topology, observability, recovery and compatibility obligations must be closed before activation.

DECISION LEDGER 38-32

Gate	Required proof
target ownership	target decides action and outcome
necessity	existing fact path cannot satisfy the use case
refusal	typed target-owned rejection reasons
idempotency	stable command identity and semantic key
expiry	stale requests cannot mutate current state
authorization	named sender/target capability
evidence	contract, provider, broker and fault tests retained

ALL GATES REQUIRED

One failed gate rejects the command. A weighted score cannot compensate for missing target ownership or authorization.

COMMAND GATE IMPLEMENTATION

The policy returns typed blockers, not a Boolean guess

A typed evaluation makes review evidence explicit. Activation requires an empty blocker set and a separate ADR version; merely registering a handler does not approve the contract.

EXECUTABLE ASSET 38-H - BLOCKER-FIRST COMMAND GATE

```
public static ImmutableArray<CommandBlocker> Evaluate(
    CommandProposal p)
{
    var b = ImmutableArray.CreateBuilder<CommandBlocker>();
    if (!p.TargetOwnsAction) b.Add(CommandBlocker.TargetOwnership);
    if (!p.NecessityProved) b.Add(CommandBlocker.NotNecessary);
    if (!p.HasTypedRefusal) b.Add(CommandBlocker.NoRefusalContract);
    if (!p.HasSemanticIdempotency) b.Add(CommandBlocker.NoIdempotency);
    if (!p.HasExpiry) b.Add(CommandBlocker.NoExpiry);
    if (!p.AuthorizationBound) b.Add(CommandBlocker.NoAuthorization);
    if (!p.EvidencePassed) b.Add(CommandBlocker.NoEvidence);
    return b.ToImmutable();
}
```

ACTIVATION RULE

Approved = no blockers + reviewed ADR revision + exact topology/permission update. No single code flag bypasses those conditions.

GATE TESTS

The current domain commands fail for specific reasons

Tests freeze why commands are absent. If a future change makes one proposal admissible, the test and ADR must change together and the evidence manifest must bind both changes.

EXECUTABLE ASSET 38-I - NEGATIVE COMMAND CORPUS

```
[Theory]
[MemberData(nameof(CurrentRejectedCommands))]
public void Current_commands_have_at_least_one_blocker(
    CommandProposal proposal,
    CommandBlocker expected)
{
    var blockers = CoordinationCommandPolicy.Evaluate(proposal);
    Assert.Contains(expected, blockers);
}

// OpenCase: NotNecessary
// IssueVerdict: TargetOwnership
// OpenReview: NotNecessary
// ApproveReview: TargetOwnership
```

REGRESSION PROTECTION

Removing a blocker from one proposal is not sufficient to approve it; all seven gates and the ADR revision remain mandatory.

APPROVED COMMAND SET

Empty is an executable contract, not missing documentation

The topology and host expose an explicit zero-command inventory. This lets CI distinguish the chosen absence of commands from a scanner that failed to discover any handlers.

EXECUTABLE ASSET 38-J - EXPLICIT EMPTY INVENTORY

```
public static class CoordinationContracts
{
    public static readonly ImmutableArray<Type> Facts =
    [
        typeof(AlarmFloodDetectedV1),
        typeof(RootCauseCaseOpenedV1),
        typeof(RootCauseVerdictIssuedV1),
        typeof(ComplianceReviewOpenedV1)
    ];

    public static readonly ImmutableArray<Type> Commands = [];
}

Assert.Equal(4, CoordinationContracts.Facts.Length);
Assert.Empty(CoordinationContracts.Commands);
```

ZERO-DISCOVERY DISTINCTION

The inventory test also proves the scanner ran and found the expected four facts. Zero commands alone would be ambiguous.

FUTURE COMMAND CONTRACT

If admitted later, a command needs more than a payload

A future command would require an immutable envelope, stable MessageId and semantic key, NotBefore/ExpiresAt, target policy identity, authorized sender, typed accepted/refused outcomes, inbox/outbox behavior and a dead-letter/repair path. None of these are implied by ADR-038.

DECISION LEDGER 38-36

Required surface	Why
identity + semantic key	deduplicate retries and equivalent requests
expiry	prevent stale mutation
target policy identity	bind acceptance semantics
accepted/refused facts	make outcome observable
authorization	constrain sender capability
inbox/outbox	survive duplicates and commit/publish gap
repair evidence	recover without silent operator mutation

DEFERRED DESIGN

This checklist is a gate, not an implementation claim. Chapter 39 may use it when designing escalation and human review.

ARCHITECTURE TEST

The Process Manager references contracts, not owner implementations

Static architecture tests reject domain assembly references, owner DbContexts and mutation clients. The allowed graph keeps Domain independent, Application dependent on Domain/contracts and Infrastructure responsible for WorkflowDb and RabbitMQ adapters.

EXECUTABLE ASSET 38-K - DEPENDENCY BOUNDARY

```
[Fact]
public void Process_manager_has_no_owner_implementation_reference()
{
    Types.InAssembly(typeof(ProcessManagerAnchor).Assembly)
        .ShouldNot().HaveDependencyOnAny(
            "Nexus1.AlarmManagement.Domain",
            "Nexus1.RootCause.Domain",
            "Nexus1.Compliance.Domain",
            "Nexus1.AlarmManagement.Infrastructure",
            "Nexus1.RootCause.Infrastructure",
            "Nexus1.Compliance.Infrastructure")
        .GetResult().ShouldBeSuccessful();
}
```

CONTRACT ALLOWANCE

Versioned public contract assemblies are allowed. Owner Domain and Infrastructure assemblies are not.

COMPOSITION GATE

The DI graph proves the absence of command authority

The production registration inventory must contain fact handlers, WorkflowDb services and query endpoints while excluding any command publisher, owner database context or owner mutation HTTP client. The test inspects the same composition path used by the host.

EXECUTABLE ASSET 38-L - DESCRIPTOR AUDIT

```
var services = new ServiceCollection();
services.AddProcessManager(configuration);

Assert.Contains(services, x =>
    x.ServiceType == typeof(IWorkflowFactHandler<AlarmFloodDetectedV1>));
Assert.Contains(services, x => x.ServiceType == typeof(WorkflowDbContext));

Assert.DoesNotContain(services, x =>
    x.ServiceType == typeof(IPublicCommandPublisher));
Assert.DoesNotContain(services, x =>
    x.ServiceType.Name is "RootCauseDbContext" or "ComplianceDbContext");
```

SAME-PATH REQUIREMENT

A test-only service collection would prove nothing about production authority. The audit calls the production AddProcessManager registration method.

TOPOLOGY CONTRACT

Four fact bindings and zero command routes are declared

Topology is part of the coordination model. The manifest names one durable Process-Manager queue, four exact fact bindings, manual acknowledgement and no command exchange, queue or routing key.

EXECUTABLE ASSET 38-M - TOPOLOGY MODEL

```
var topology = new CoordinationTopology(
    Queue: "workflow.alarm-to-compliance.v1",
    Durable: true,
    AutoDelete: false,
    ManualAcknowledgement: true,
    FactBindings:
    [
        "alarm-management.alarm-flood-detected.v1",
        "root-cause.root-cause-case-opened.v1",
        "root-cause.root-cause-verdict-issued.v1",
        "compliance.compliance-review-opened.v1"
    ],
    CommandRoutes: []);
```

DRIFT GATE

The deployment topology digest must match this approved model. A command route is an ADR change, not an operational tweak.

TOPOLOGY TESTS

The broker must demonstrate the declared absence and presence

A real broker component suite verifies queue properties, exact bindings, routing behavior, manual acknowledgement and the absence of command resources. Passive declarations alone are insufficient if permissions or routing differ.

DECISION LEDGER 38-40

Probe	Pass condition
declare Process-Manager queue passively	durable, not auto-delete
publish each approved fact route	one delivery reaches workflow queue
publish unknown fact route	no workflow delivery
inspect command exchange/queue	resource absent
attempt command publish as manager	permission denied or unroutable
redeliver unacked fact	same body reaches inbox path

EVIDENCE BOUNDARY

These are proposed runtime probes. The chapter remains PENDING until their exact broker version, commands, logs and artifact digests are retained.

HOST INVENTORY

The running process exposes only approved capabilities

A production-path host test starts the real composition root and inventories endpoints, workers, database registrations and messaging capabilities. It proves both expected presence and forbidden absence.

EXECUTABLE ASSET 38-N - CAPABILITY ASSERTION

```
var manifest = await factory.GetCapabilityManifestAsync();

Assert.Equal("ADR-038", manifest.Coordination.AdrId);
Assert.Equal(4, manifest.Messaging.FactBindings.Count);
Assert.Empty(manifest.Messaging.CommandRoutes);
Assert.Empty(manifest.Http.MutationClients);
Assert.Equal(["WorkflowDb"], manifest.Databases.RuntimeWriteTargets);
Assert.Contains("GET /workflows/{workflowId}", manifest.Endpoints);
```

NO SELF-ATTESTATION

The manifest is derived from endpoint, DI and topology inventories. It is not a hand-written JSON claim disconnected from the running host.

OBSERVABILITY

Telemetry explains the choice without becoming workflow truth

Logs and traces carry WorkflowId, MessageId, CorrelationId, CausationId, contract identity, transition and disposition. Metrics aggregate low-cardinality phase, result and producer labels. WorkflowDb remains the durable workflow record; telemetry supports diagnosis.

DECISION LEDGER 38-42

Signal	Required dimension	Forbidden dimension
counter	contract + disposition	WorkflowId
histogram	transition + result	MessageId
gauge	queue age / open expectations	CorrelationId
structured log	all lineage ids	secret or message body
trace	consume -> TX -> ack	invented cross-service span

FRESHNESS

The operator view reports observation lag and last admitted fact. A Completed phase without freshness context can mislead during manager outage.

CAPACITY

The observer is not on the owner commit path

Observed choreography reduces coordination latency coupling, but WorkflowDb and its queue still need explicit capacity. Measure ingress rate, transition cost, queue age, recovery drain rate and database contention using the real contract corpus.

DECISION LEDGER 38-43

Capacity claim	Required measurement
steady-state keeps up	publish rate <= admitted transition rate
restart recovers	drain rate exceeds live ingress with bounded DB load
duplicates are cheap	inbox hit latency and no second transition
buffer remains bounded	early-fact count/age and retention policy
queries do not starve writes	read/write contention under operator load

NO INVENTED THRESHOLD

The ADR fixes responsibility placement, not production numbers. Capacity thresholds require retained workload profiles and results.

DEPLOYMENT

The coordination model rolls out without changing owner truth

Deploy the Process Manager as an observer: declare its queue, apply WorkflowDb migrations, start consumers in shadow/read-only exposure, compare state, then enable the operator query. Owner producers and consumers retain their existing facts and decisions.

DECISION LEDGER 38-44

Stage	Change	Exit evidence
prepare	WorkflowDb migration + queue	digest and permissions
shadow	consume/rebuild without operator reliance	comparison report
expose	enable workflow query	freshness and fault probes
steady	monitor lag/expectations	retained SLO artifacts

NO OWNER CUTOVER

There is no switch that makes the manager the writer of Alarm, RootCause or Compliance truth.

REVERSAL

Removing the observer does not roll back business history

If the manager release is defective, stop its consumers, preserve the queue and WorkflowDb evidence, restore the prior observer version or rebuild a corrected projection. Do not delete owner facts or issue compensating domain actions merely to align WorkflowDb.

DECISION LEDGER 38-45

Reversal action	Preserved artifact
stop new manager	queued deliveries
snapshot evidence	WorkflowDb + manifests + logs
deploy prior/corrected projection	owner facts unchanged
replay or drain	same inbox/transition rules
shadow compare	difference dossier before exposure

REVERSAL SEMANTICS

Reversal changes the read/progress projection. It does not pretend the distributed owner transactions never happened.

REBUILD

A deterministic shadow rebuild tests the architecture choice

Because the manager derives state from approved public facts, a clean WorkflowDb can replay the retained corpus into a shadow projection. Comparison uses workflow identity, phase, owner identities, observation digests, expectation and transition sequence—not mutable timestamps generated by the replay process.

EXECUTABLE ASSET 38-O - SEMANTIC SHADOW COMPARISON

```
var expected = await live.ExportSemanticSnapshotAsync(workflowId, ct);
var rebuilt = await shadow.ReplayAsync(retainedFacts, ct);
```

```
Assert.Equal(expected.WorkflowId, rebuilt.WorkflowId);
Assert.Equal(expected.Phase, rebuilt.Phase);
Assert.Equal(expected.OwnerIdentities, rebuilt.OwnerIdentities);
Assert.Equal(expected.ObservationDigests, rebuilt.ObservationDigests);
Assert.Equal(expected.Expectation, rebuilt.Expectation);
Assert.Equal(expected.TransitionKinds, rebuilt.TransitionKinds);
```

REPAIR BOUNDARY

A mismatch opens a repair case. The rebuild tool must not silently edit owner data or overwrite the live projection without reviewed evidence.

FAILURE LABORATORY

The chosen model must survive the failures it claims to isolate

Fault tests exercise the actual host, WorkflowDb provider and RabbitMQ topology. Each scenario records injected fault, observed local commit, delivery settlement, workflow transition, telemetry and recovery action.

DECISION LEDGER 38-47

Injected fault	Required observation
manager stopped	owners progress; queue lag grows
WorkflowDb unavailable	delivery unacked/retried; no owner rollback
crash after commit before ack	redelivery becomes matching duplicate
facts reordered	buffer/drain or fail-closed rule
forged producer	admission rejection; no transition
attempt command publish	no route/capability
rebuild from corpus	semantic snapshot matches

EVIDENCE REQUIRED

A passing test name is not enough. The dossier binds environment, exact fault point, inputs, outputs and SHA-256 artifacts.

EVIDENCE CATALOGUE

Every ADR claim maps to a proving surface

The claim catalogue prevents an architecture diagram from standing in for runtime proof. Static tests prove dependency shape; provider and broker tests prove their owned semantics; host and fault suites prove composition and recovery.

DECISION LEDGER 38-48

Claim	Proof surface	Minimum artifact
no owner implementation refs	architecture test	TRX + assembly inventory
zero commands	contract/DI/topology tests	manifests + broker probe
owner progress isolated	fault lab	timeline + owner commits
deterministic rebuild	provider/replay test	source corpus + diff
least privilege	deployment probe	value-free permission report
exact ADR	digest validator	canonical body + SHA-256

NO SUBSTITUTE EVIDENCE

An in-memory unit test cannot prove SQL constraints, broker permissions, publisher routing or production DI composition.

EVIDENCE MANIFEST

The dossier fails closed until every required artifact exists

The manifest names the decision identity, repository state, dependency lock, environment, topology, migrations, commands and artifact hashes. A validator rejects missing, stale, zero-discovery or mismatched results.

EXECUTABLE ASSET 38-P - PROPOSED DOSSIER

```
{
  "evidenceId": "NEXUS1-COORDINATION-ADR-038",
  "status": "PENDING",
  "adrId": "ADR-038",
  "adrSha256": "622787772a9def5ef18bffc6a622d6efc11185d0a696780445a3ee8cc027f22a",
  "modelIdentity": "nexus1/coordination/observed-choreography/1",
  "approvedCommandCount": 0,
  "requiredSuites": [
    "architecture", "provider", "broker", "host", "fault", "rebuild"
  ],
  "artifacts": []
}
```

CURRENT TRUTHFUL STATE

The PDF specifies the dossier but does not contain retained repository execution artifacts. Status therefore remains PENDING.

EVIDENCE VALIDATOR

Zero commands are valid only when discovery is demonstrably non-zero

A fail-closed validator checks that all required suites ran, discovered the expected tests, used the same commit and ADR digest, and produced hashed artifacts. The explicit four-fact inventory distinguishes an intentional empty command set from a broken scanner.

EXECUTABLE ASSET 38-Q - FAIL-CLOSED CHECKS

```
Require(manifest.Status == "PROVED");
Require(manifest.AdrSha256 == expectedAdrSha256);
Require(manifest.DiscoveredFacts == 4);
Require(manifest.ApprovedCommands == 0);
Require(manifest.ExecutedTests > 0);
Require(manifest.FailedTests == 0);
RequireExactSuites("architecture", "provider", "broker",
    "host", "fault", "rebuild");
RequireAllArtifactsExistAndMatchSha256();
RequireSingleRepositoryCommitAndCleanTree();
```

NO PDF SELF-CERTIFICATION

The validator may promote evidence to PROVED only from retained execution outputs, never from the existence of the reference listing itself.

CI REPRODUCTION

One pipeline command should build the coordination dossier

The target pipeline restores locked dependencies, builds Release, provisions isolated SQL and RabbitMQ resources, runs the bounded Chapter 38 suites, validates the dossier and archives every artifact with its digest.

EXECUTABLE ASSET 38-R - PROPOSED REPRODUCTION PATH

```
dotnet test tests/Nexus1.ProcessManager.EvidenceTests \
  --configuration Release \
  --no-restore \
  --filter "Category=Chapter38" \
  --logger "trx;LogFileName=chapter38.trx" \
  --results-directory artifacts/evidence/chapter38

dotnet run --project tools/EvidenceValidator -- \
  artifacts/evidence/chapter38/manifest.json
```

REQUIRED BINDING

The dossier records repository URI, commit, dirty flag, package-lock digest, provider/broker versions, migrations, topology digest, commands and artifact SHA-256 values.

CHANGE CONTROL

A new command changes the model identity and ADR

Any approved command alters authority, topology, security, failure modes and operational repair. It therefore requires a superseding ADR, new model identity, contract corpus, permission diff, migration/rollout plan and compatibility evidence before activation.

DECISION LEDGER 38-52

Changed surface	Mandatory update
ADR	new accepted revision and consequences
model identity	new immutable coordination identity
contracts	command + accepted/refused outcomes
topology	exchange/queue/routes and ownership
security	sender and target grants
tests	idempotency, expiry, refusal, fault/recovery
operations	dashboards, DLQ and repair runbook

NO SILENT EVOLUTION

Adding a publisher registration or broker route without the superseding ADR is an architecture-policy violation.

OPERATOR MEANING

The chosen model exposes progress without promising control

The workflow query tells operators which owner facts were observed, what is expected and how fresh the projection is. It does not offer buttons that force a verdict, fabricate a review or silently repair owner state.

DECISION LEDGER 38-53

May display	Must not imply
phase and timeliness	the business outcome succeeded/failed
exact owner identities	WorkflowDb owns those records
last observed fact and lag	all queues are current
open expectation	target accepted a request
repair case/status	automatic owner mutation occurred

UX BOUNDARY

Chapter 41 will design eventual-consistency states in the operator experience. ADR-038 supplies the truth boundaries that UI must preserve.

CHAPTER 39 BOUNDARY

Escalation may request attention, never prescribe a verdict

The next chapter will turn durable overdue observations into operator-visible escalation and human review. It must preserve the distinction between a late service, a failed technical attempt and a refused or pending business decision.

DECISION LEDGER 38-54

Chapter 38 fixes	Chapter 39 must decide
observed choreography normal path	deadline and escalation policy
explicit zero public commands	whether one narrow attention request is necessary
command admissibility gate	target, refusal, expiry and authorization
overdue is not business failure	operator states and recovery action
owner decision boundaries	human review evidence and audit trail

PROTECTED HANDOFF

Chapter 39 may supersede the empty command set only through the recorded gate. It may not make the Process Manager the owner of RootCause or Compliance policy.

OFFICIAL TECHNICAL ANCHORS

The ADR is grounded in explicit coordination trade-offs

- Microsoft Choreography pattern: <https://learn.microsoft.com/azure/architecture/patterns/choreography>
- Microsoft Saga pattern: <https://learn.microsoft.com/azure/architecture/patterns/saga>
- RabbitMQ queues and ordering: <https://www.rabbitmq.com/docs/queues>
- RabbitMQ consumers and acknowledgements: <https://www.rabbitmq.com/docs/consumers>
- RabbitMQ acknowledgements and publisher confirms: <https://www.rabbitmq.com/docs/confirmations>
- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- W3C Trace Context: <https://www.w3.org/TR/trace-context/>
- RFC 8785 JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785>

WHAT THIS CHAPTER HAS PROVED

Chapter 38 compares pure choreography, command-issuing orchestration and observed choreography against the actual NEXUS-1 runtime; accepts ADR-038; preserves local decision ownership; fixes an explicit four-fact, zero-command model; defines authority, security, topology, composition, failure, deployment, rebuild and change-control gates; and specifies a fail-closed evidence dossier. Runtime evidence remains PENDING.

CHAPTER CHECKPOINT

- Why does a durable Process Manager not automatically imply orchestration?
- Which current requirement blocks pure choreography without WorkflowDb?
- Which ownership and availability constraints block OpenCase and IssueVerdict commands?
- How can manager outage leave owner truth valid but workflow evidence stale?
- Why are Accepted ADR status and PENDING evidence status compatible?
- How does CI distinguish an intentional empty command set from zero discovery?
- Which seven blockers must every future public command pass?
- Why does adding one command require a new model identity, topology and permission diff?

NEXT - CHAPTER 39

Timeouts, Escalations, and Human Review will convert durable overdue observations into explicit operator-visible policy. It will distinguish lateness, technical failure and business refusal, and will use the ADR-038 admissibility gate before introducing any narrow request for attention.

PART VII - WORKFLOW, CONSISTENCY, AND REPAIR

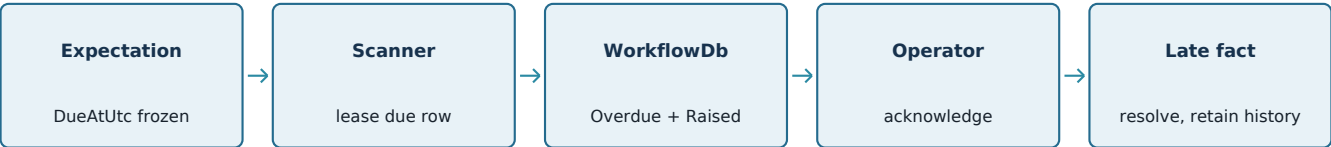
39. Timeouts, Escalations, and Human Review

A durable deadline makes missing progress visible without inventing failure. WorkflowDb raises one operational escalation, an authorized operator may acknowledge and assess it, and a late owner fact may still complete the workflow.

CHAPTER DECISION

Each open workflow expectation stores an immutable DueAtUtc and policy identity. A bounded, multi-replica-safe scanner uses WorkflowDb time to mark Overdue and create one WorkflowEscalation. Human actions acknowledge or assess operational attention only; they cannot force progress, issue a verdict or alter Compliance. Progress, timeliness, processing health and escalation remain separate. ADR-038 retains four facts and zero public commands.

PROOF PATH 39-A - DURABLE ATTENTION WITHOUT STOLEN AUTHORITY



GUARANTEE LEDGER 39-A

Specified	Not granted
durable, policy-bound deadline	process-local timer as truth
one operator-visible escalation	business-failure inference
authorized local acknowledgement	public owner command
race-safe late completion	erasure of overdue history
provider/host/fault evidence plan	runtime proof without artifacts

ORDINARY FAILURE OF LANGUAGE

Timeout is an observation, not a verdict

A distributed workflow can be late because an owner service is unavailable, a queue is delayed, the Process Manager is behind, or the owner has not yet reached a decision. None of those facts authorizes WorkflowDb to report that RootCause failed or Compliance rejected the work.

DECISION LEDGER 39-2

Observed condition	Allowed statement	Forbidden statement
deadline elapsed	expected fact not observed by DueAtUtc	owner decision failed
manager queue lag	workflow evidence may be stale	owner did not commit
owner fact arrives late	workflow completed after overdue	timeout never happened
no negative owner fact	business outcome is unreported	business refusal

CHAPTER RULE

Escalation calls for attention to missing evidence. It never manufactures the missing owner decision.

FOUR-AXIS STATE

Progress, timeliness, health and escalation evolve independently

One status enum would collapse unrelated meanings. The operator model instead exposes four orthogonal axes so a completed workflow can remain historically late, a healthy manager can observe an overdue owner, and a degraded manager can refuse to classify owner timeliness.

DECISION LEDGER 39-3

Axis	Examples	Owner
Progress	AwaitingVerdict / Completed	public owner facts
Timeliness	OnTime / Overdue / CompletedLate	deadline policy + observations
ProcessingHealth	Current / Degraded / Stale	Process Manager runtime
Escalation	None / Raised / Acknowledged / Resolved	WorkflowDb operational state

NO SYNTHETIC TOTAL STATE

Do not create values such as Failed, Error or Success that hide which axis actually changed.

CLASSIFICATION

Late service, technical failure and business outcome require different evidence

The UI may classify lateness from a durable open expectation and authoritative time. It may classify a Process-Manager technical fault from health evidence. It may classify business refusal only from an admitted owner contract whose semantics explicitly report that outcome.

DECISION LEDGER 39-4

Classification	Required evidence	Current path
Late / overdue	open expectation + DueAtUtc elapsed	supported
Manager degraded	queue/database/worker health evidence	supported
Business declined	versioned negative owner fact	not in approved contract set
Completed late	Overdue history + expected fact	supported

FAIL-CLOSED BUSINESS MEANING

Because the current four-fact corpus contains no negative owner outcome, Chapter 39 does not expose BusinessDeclined for this slice.

DURABLE DEADLINE

DueAtUtc is frozen when the expectation is created

The Process Manager persists the policy identity, policy digest, observed start instant, duration and computed due instant beside each expectation. A later configuration reload applies only to new expectations unless a separate, audited migration explicitly re-baselines open work.

DECISION LEDGER 39-5

Field	Invariant
ExpectationId	stable owner-local identity
ExpectedContract	exact event type and schema version
StartedAtUtc	instant the prerequisite fact was admitted
Budget	validated duration snapshot
DueAtUtc	StartedAtUtc + Budget; immutable
PolicyId / digest	exact policy that produced the deadline

REPLAY RULE

A replay uses the retained DueAtUtc and policy identity. It does not recompute history using today's configuration.

POLICY BINDING

Production budgets come from validated configuration, not the book

The reference profile gives deterministic test durations, but it is explicitly marked UNBOUND and test-only. Production must supply reviewed budgets through strict options binding, cross-field validation and a value-free policy digest.

EXECUTABLE ASSET 39-A - STRICT DEADLINE OPTIONS

```
public sealed record DeadlinePolicyOptions
{
    public required string PolicyId { get; init; }
    public required TimeSpan AwaitingCaseOpened { get; init; }
    public required TimeSpan AwaitingVerdict { get; init; }
    public required TimeSpan AwaitingComplianceReview { get; init; }
    public required TimeSpan ScanInterval { get; init; }
    public required TimeSpan LeaseDuration { get; init; }
    public required int BatchSize { get; init; }
}
```

NO INVENTED SLO

PT5M, PT30M and PT15M appear only in the reference test profile. The PDF does not claim that these are production targets.

POLICY VALIDATION

Invalid time relationships fail startup

Validation rejects missing identities, non-positive budgets, a lease shorter than the maximum per-item processing bound, an unbounded batch and a scan interval that defeats the smallest budget. Values remain environment-specific; the rules remain invariant.

EXECUTABLE ASSET 39-B - CROSS-FIELD VALIDATOR

```
public ValidateOptionsResult Validate(string? _, DeadlinePolicyOptions o)
{
    var errors = new List<string>();
    if (string.IsNullOrEmpty(o.PolicyId)) errors.Add("PolicyId required");
    if (Budgets(o).Any(x => x <= TimeSpan.Zero)) errors.Add("Budgets > 0");
    if (o.BatchSize is < 1 or > 500) errors.Add("BatchSize 1..500");
    if (o.LeaseDuration <= o.ScanInterval) errors.Add("Lease > scan interval");
    if (o.ScanInterval >= Budgets(o).Min()) errors.Add("Scan < smallest budget");
    return errors.Count == 0
        ? ValidateOptionsResult.Success
        : ValidateOptionsResult.Fail(errors);
}
```

STARTUP CONTRACT

The host may not silently substitute defaults when the deadline section is absent or malformed.

POLICY IDENTITY

Canonical serialization binds every expectation to one profile

A policy digest lets evidence compare environments without exporting raw configuration. The test fixture includes explicit usage and production status so its durations cannot be mistaken for deployed promises.

EXECUTABLE ASSET 39-C - REFERENCE POLICY IDENTITY

```
const string PolicyId = "workflow/deadline-escalation/reference/1";
const string ExpectedSha256 = "5a6dd3526033f76967761fe972e82a18e9ccd0ead2be00bd1401a4c240a3cfe2";

byte[] body = CanonicalJson.Serialize(referencePolicy);
string actual = Convert.ToHexString(
    SHA256.HashData(body)).ToLowerInvariant();

Assert.Equal(ExpectedSha256, actual);
Assert.Equal("reference-test-profile-only", referencePolicy.Usage);
Assert.Equal("UNBOUND", referencePolicy.ProductionStatus);
```

EXACT FIXTURE

Reference policy body: 455 UTF-8 bytes; SHA-256
5a6dd3526033f76967761fe972e82a18e9ccd0ead2be00bd1401a4c240a3cfe2.

EXPECTATION TYPE

The deadline snapshot is an immutable value object

The Domain receives an already validated DeadlineSnapshot when it opens an expectation. It stores duration and due instant together so later diagnostics can prove the arithmetic that created the deadline.

EXECUTABLE ASSET 39-D - DEADLINE SNAPSHOT

```
public sealed record DeadlineSnapshot
{
    public required string PolicyId { get; init; }
    public required string PolicySha256 { get; init; }
    public required DateTimeOffset StartedAtUtc { get; init; }
    public required TimeSpan Budget { get; init; }
    public required DateTimeOffset DueAtUtc { get; init; }

    public static DeadlineSnapshot Create(
        PolicyIdentity policy, DateTimeOffset started, TimeSpan budget) =>
        new()
        {
            PolicyId = policy.Id, PolicySha256 = policy.Sha256,
            StartedAtUtc = started, Budget = budget,
            DueAtUtc = started.Add(budget)
        };
}
```

INVARIANT

DueAtUtc must equal StartedAtUtc plus Budget and all instants must have UTC offset zero.

DATABASE MODEL

Expectations and escalations remain separate records

The expectation owns what fact is awaited and when it is due. The escalation owns human attention to an overdue expectation. Separation lets a late fact close the expectation while preserving the full escalation acknowledgement and assessment history.

DECISION LEDGER 39-10

Table	Owned meaning
WorkflowExpectations	open/observed/superseded fact expectation
WorkflowEscalations	one attention case per expectation
EscalationActions	append-only human/system action history
WorkflowTransitions	progress/timeliness transition evidence
InboxReceipts	idempotent admitted fact processing

NO SHARED TRUTH

Every table belongs to WorkflowDb. No owner DbContext, cross-database foreign key or private business field is introduced.

EF CORE MAPPING

Constraints encode deadline and escalation invariants

Provider mappings use rowversion for optimistic concurrency, UTC datetime2 values, constrained enums and unique relationships. The database rejects a second escalation for the same expectation even if two workers race.

EXECUTABLE ASSET 39-E - EXPECTATION MAPPING

```
b.HasKey(x => x.ExpectationId);
b.Property(x => x.RowVersion).IsRowVersion();
b.Property(x => x.DueAtUtc).HasPrecision(7);
b.HasCheckConstraint("CK_Expectation_Due",
    "[DueAtUtc] >= [StartedAtUtc]");
b.HasIndex(x => new { x.State, x.DueAtUtc });

escalation.HasKey(x => x.EscalationId);
escalation.HasIndex(x => x.ExpectationId).IsUnique();
escalation.Property(x => x.RowVersion).IsRowVersion();
```

ARBITRATION

Domain checks explain the rule; provider constraints arbitrate concurrent writers and imported data.

DUE-WORK INDEX

The scanner reads a bounded, selective surface

Open expectations are ordered by DueAtUtc and ExpectationId. A filtered index excludes observed and superseded rows, while lease fields prevent multiple replicas from processing the same candidate concurrently.

EXECUTABLE ASSET 39-F - SQL SERVER DUE INDEX

```
CREATE INDEX IX_WorkflowExpectations_DueOpen
ON workflow.WorkflowExpectations
(
    DueAtUtc ASC,
    ExpectationId ASC
)
INCLUDE (WorkflowId, ExpectedEventType, LeaseUntilUtc, RowVersion)
WHERE ClosedAtUtc IS NULL;
```

PLAN EVIDENCE

Index usefulness requires a retained SQL Server execution plan and logical-read measurements. The listing alone is not performance proof.

AUTHORITATIVE TIME

Database time arbitrates due work across replicas

Application TimeProvider supports deterministic Domain and worker tests. The SQL Server adapter compares persisted deadlines and leases to SYSUTCDATETIME so all replicas sharing WorkflowDb use the same arbitration clock.

DECISION LEDGER 39-13

Use	Clock
Domain unit tests	FakeTimeProvider
worker wake-up tests	FakeTimeProvider timer
multi-replica due selection	SQL Server SYSUTCDATETIME
persisted fact occurrence	admitted envelope owner instant
duration measurement	monotonic timestamp where applicable

CLOCK BOUNDARY

Database time is not perfect physical truth; it is the selected shared arbitration source. Clock synchronization and drift remain operational dependencies.

TIMER SEMANTICS

The timer wakes a scan; it never owns the deadline

A process-local timer can be delayed, reset or lost on restart. The worker therefore wakes periodically and asks WorkflowDb for overdue expectations. After downtime, the first scan immediately discovers every still-open row whose DueAtUtc has elapsed.

EXECUTABLE ASSET 39-G - WAKE-UP LOOP

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    using var timer = new PeriodicTimer(_scanInterval, _timeProvider);
    do
    {
        await ScanOneBoundedBatchAsync(stoppingToken);
    }
    while (await timer.WaitForNextTickAsync(stoppingToken));
}
```

RESTART BEHAVIOR

No in-memory callback must be replayed. Durable DueAtUtc plus the next database scan recovers the obligation.

SCOPED WORKER

Each batch receives a new dependency scope

The hosted worker is singleton-lived, while WorkflowDbContext and escalation services are scoped. Every bounded batch creates and disposes one scope, preventing captured DbContexts and uncontrolled change tracking across iterations.

EXECUTABLE ASSET 39-H - PER-BATCH COMPOSITION

```
private async Task ScanOneBoundedBatchAsync(CancellationToken ct)
{
    await using var scope = _scopeFactory.CreateAsyncScope();
    var scanner = scope.ServiceProvider
        .GetRequiredService<IDueExpectationScanner>();

    await scanner.ProcessBatchAsync(
        batchSize: _options.BatchSize,
        cancellationToken: ct);
}
```

SHUTDOWN

Cancellation stops acquisition of new work and bounds completion of the active database operation. Lease expiry recovers any abandoned item.

MULTI-REPLICA LEASE

One SQL statement claims a bounded due batch

The SQL Server adapter selects due, unleased expectations with update locks, skips rows owned by another transaction and stamps one lease token using database UTC. Under READ_COMMITTED_SNAPSHOT, READCOMMITTEDLOCK is required with READPAST.

EXECUTABLE ASSET 39-I - ATOMIC DUE LEASE

```
;WITH due AS
(
    SELECT TOP (@batchSize) *
    FROM workflow.WorkflowExpectations WITH
        (UPDLCK, READPAST, READCOMMITTEDLOCK, ROWLOCK)
    WHERE ClosedAtUtc IS NULL
        AND DueAtUtc <= SYSUTCDATETIME()
        AND (LeaseUntilUtc IS NULL OR LeaseUntilUtc <= SYSUTCDATETIME())
    ORDER BY DueAtUtc, ExpectationId
)
UPDATE due
SET LeaseId = @leaseId,
    LeaseUntilUtc = DATEADD(second, @leaseSeconds, SYSUTCDATETIME())
OUTPUT inserted.ExpectationId, inserted.WorkflowId, inserted.RowVersion;
```

PROVIDER-SPECIFIC PROOF

The exact hint/isolation combination must be tested on the configured SQL Server database options; it is not portable pseudo-SQL.

PER-ITEM TRANSACTION

Overdue, escalation and history commit together

After leasing, each expectation is reloaded and re-evaluated in a fresh transaction. If still open and due, one commit marks timeliness Overdue, creates or reuses the escalation and appends transition/action evidence.

EXECUTABLE ASSET 39-J - LOCAL COMMIT BOUNDARY

```
await execution.ExecuteAsync(async () =>
{
    await using var db = await factory.CreateDbContextAsync(ct);
    await using var tx = await db.Database.BeginTransactionAsync(ct);

    var expectation = await repo.LoadForLeaseAsync(id, leaseId, ct);
    var outcome = expectation.MarkOverdue(await clock.UtcNowAsync(ct));
    await escalations.ApplyAsync(expectation, outcome, ct);
    await db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
});
```

SEMANTIC RETRY

A transient or concurrency retry creates a new DbContext and re-evaluates current state. It never repeats a stale Domain outcome blindly.

DEADLINE OUTCOME

MarkOverdue is deterministic and idempotent

The Domain accepts an authoritative UTC instant and returns a typed outcome. Closed expectations and matching overdue duplicates create no second escalation; an instant before DueAtUtc cannot mark lateness.

EXECUTABLE ASSET 39-K - EXPECTATION TRANSITION

```
public DeadlineOutcome MarkOverdue(DateTimeOffset nowUtc)
{
    RequireUtc(nowUtc);
    if (State is not ExpectationState.Open)
        return DeadlineOutcome.AlreadyClosed;
    if (Timeliness is TimelinessState.Overdue)
        return DeadlineOutcome.MatchingDuplicate;
    if (nowUtc < DueAtUtc)
        return DeadlineOutcome.NotDue;

    Timeliness = TimelinessState.Overdue;
    OverdueAtUtc = nowUtc;
    return DeadlineOutcome.BecameOverdue;
}
```

BOUNDARY INSTANT

The policy uses `nowUtc >= DueAtUtc`. Tests cover the tick immediately before, exactly at and immediately after the deadline.

RACE SEMANTICS

Late fact and deadline scan may both be correct

A fact can commit just before, during or just after the scanner transaction. Rowversion and owner-local constraints serialize the writes; the semantic retry then derives one legal history without deleting the losing observation.

DECISION LEDGER 39-19

Winner	Final state	Retained history
fact before scanner reload	Observed / OnTime	no escalation
scanner before fact	Observed / CompletedLate	Overdue + Raised + Resolved
concurrency collision	fresh retry chooses one path	no duplicate action
same fact redelivery	unchanged	matching inbox duplicate

HISTORICAL TRUTH

A late completion resolves the open attention condition but never erases that the deadline elapsed first.

LATE COMPLETION

The expected fact closes work and resolves attention automatically

When the awaited owner fact is admitted after an escalation, the same WorkflowDb transaction closes the expectation, advances progress, records CompletedLate and resolves the escalation with system reason ExpectedFactObserved.

EXECUTABLE ASSET 39-L - LATE FACT DISPOSITION

```
var result = workflow.Observe(expectedFact, admittedAtUtc);

if (result.WasOverdue)
{
    workflow.MarkCompletedLate();
    escalation.Resolve(
        EscalationResolution.ExpectedFactObserved,
        expectedFact.MessageId,
        admittedAtUtc);
}

await db.SaveChangesAsync(ct); // inbox + transition + resolution
```

NO OPERATOR RACE

If an operator acknowledges concurrently, rowversion retry retains both legal actions in a deterministic order or returns a typed already-resolved result.

ESCALATION AGGREGATE

Human attention is a WorkflowDb operational aggregate

The escalation owns identity, expectation reference, level, attention state, acknowledgement, typed assessments and system resolution. It does not own the RootCause verdict, Compliance review or any repair mutation.

EXECUTABLE ASSET 39-M - ESCALATION AUTHORITY

```
public sealed class WorkflowEscalation
{
    public EscalationId Id { get; private set; }
    public ExpectationId ExpectationId { get; private set; }
    public AttentionState State { get; private set; }
    public EscalationLevel Level { get; private set; }
    public DateTimeOffset RaisedAtUtc { get; private set; }
    public Acknowledgement? Acknowledgement { get; private set; }
    public EscalationResolution? Resolution { get; private set; }

    // No Verdict, ReviewFinding or owner mutation methods.
}
```

NAMING BOUNDARY

This is WorkflowEscalation, not ComplianceReview. The two aggregates answer different questions and live in different owner databases.

ATTENTION STATES

Acknowledged does not mean resolved

An operator can accept responsibility for investigation while the expected fact remains missing. Resolution is automatic only when the expected fact arrives or the expectation is explicitly superseded by a valid workflow transition.

DECISION LEDGER 39-22

State	Meaning	Who may cause
Raised	attention required	deadline policy
Acknowledged	named authorized subject is investigating	operator API
Resolved	underlying expectation cleared/superseded	Process Manager transition
Reopened	new policy step after valid resolution	not implemented in V1

NO DISMISS SHORTCUT

V1 has no operator Close or ForceComplete action. Human acknowledgement cannot make missing owner evidence appear.

ONE ESCALATION

Database uniqueness absorbs scanner duplication

Multiple workers may discover the same due expectation before one commit becomes visible. A unique ExpectationId constraint and semantic insert handling guarantee one escalation identity and one Raised action.

EXECUTABLE ASSET 39-N - DUPLICATE ARBITRATION

```
try
{
    db.WorkflowEscalations.Add(WorkflowEscalation.Raise(expectation, now));
    await db.SaveChangesAsync(ct);
    return RaiseOutcome.Created;
}
catch (DbUpdateException ex) when (classifier.IsUniqueViolation(
    ex, "UX_WorkflowEscalations_ExpectationId"))
{
    var existing = await repo.LoadByExpectationAsync(expectation.Id, ct);
    return existing.Matches(expectation)
        ? RaiseOutcome.MatchingDuplicate
        : RaiseOutcome.Conflict;
}
```

PROVIDER CLASSIFIER

Only the named unique constraint is treated as a duplicate. Other database failures are not swallowed as idempotency.

OPERATOR COMMAND

AcknowledgeEscalation mutates only local operational state

The synchronous Application command targets the Process Manager itself. It records that an authorized operator has taken ownership of attention; it sends no public command to an owner service and does not change workflow progress.

EXECUTABLE ASSET 39-O - LOCAL ACKNOWLEDGEMENT COMMAND

```
public sealed record AcknowledgeEscalation(
    Guid WorkflowId,
    Guid EscalationId,
    string IdempotencyKey,
    string ExpectedVersion);

public sealed record AcknowledgeContext(
    string ActorSubjectHash,
    ImmutableArray<string> AuthorizedScopes,
    DateTimeOffset ReceivedAtUtc);
```

TRUST RULE

Actor identity and scopes come from the authenticated request context, never from caller-supplied JSON fields.

ACKNOWLEDGEMENT TRANSITION

The aggregate returns typed outcomes for retries and races

Acknowledgement is idempotent for the same actor and idempotency key. A different actor or key after acknowledgement produces a conflict; an already-resolved escalation returns Gone/AlreadyResolved without reopening it.

EXECUTABLE ASSET 39-P - ACKNOWLEDGEMENT OUTCOMES

```
public AcknowledgeOutcome Acknowledge(
    ActorSubjectHash actor, IdempotencyKey key, DateTimeOffset now)
{
    if (State is AttentionState.Resolved)
        return AcknowledgeOutcome.AlreadyResolved;
    if (Acknowledgement?.Matches(actor, key) is true)
        return AcknowledgeOutcome.MatchingDuplicate;
    if (Acknowledgement is not null)
        return AcknowledgeOutcome.Conflict;

    Acknowledgement = new(actor, key, now);
    State = AttentionState.Acknowledged;
    return AcknowledgeOutcome.Accepted;
}
```

NO LAST-WRITE-WINS

Two operators racing to claim the same escalation receive one Accepted result and one typed conflict after rowversion arbitration.

HUMAN ASSESSMENT

Assessment codes explain attention without judging the domain

An acknowledged operator may append a typed operational assessment. Free-form notes are excluded from V1 to reduce sensitive data and ambiguous automation; codes describe what the operator knows operationally, not what RootCause or Compliance decided.

DECISION LEDGER 39-26

Assessment code	Meaning	Progress effect
AwaitingOwnerRecovery	owner path known unavailable	none
TransportInvestigation	delivery/queue diagnosis underway	none
ProjectionStale	manager evidence freshness suspect	none
RepairAssessmentNeeded	possible Chapter 42 repair case	none
BusinessDeclined	forbidden without owner fact	not accepted

DATA MINIMIZATION

The action stores code, pseudonymous actor hash, timestamps and lineage. It does not store incident narrative, personal notes or copied owner data.

FORBIDDEN OPERATOR ACTIONS

The console cannot force distributed progress

Operational urgency does not transfer domain authority. The Process Manager exposes no endpoint to issue a verdict, open/approve a Compliance review, rewrite owner facts, skip required evidence or mark the workflow completed.

DECISION LEDGER 39-27

Forbidden action	Correct owner/boundary
ForceCompleteWorkflow	completion follows admitted owner fact
IssueVerdict	RootCause
Open/ApproveReview	Compliance
RetryOwnerTransaction	owner service recovery policy
EditDueAtUtc	separate audited re-baseline policy
DeleteEscalationHistory	retention/governance process

UI GATE

The Operator Console BFF must not synthesize buttons for capabilities absent from the Process-Manager contract.

HTTP BOUNDARY

Queries and acknowledgement have distinct authorization

Read endpoints expose the four-axis state and freshness. Mutation endpoints acknowledge or assess one escalation with optimistic concurrency and idempotency. They do not expose generic PATCH over WorkflowDb records.

EXECUTABLE ASSET 39-Q - ENDPOINT SURFACE

```
app.MapGet("/v1/workflows/{workflowId:guid}", GetWorkflow)
    .RequireAuthorization("WorkflowEscalation.Read");

app.MapPost(
    "/v1/workflows/{workflowId:guid}/escalations/{escalationId:guid}/acknowledgements",
    AcknowledgeEscalation)
    .RequireAuthorization("WorkflowEscalation.Respond");

app.MapPost(
    "/v1/workflows/{workflowId:guid}/escalations/{escalationId:guid}/assessments",
    RecordAssessment)
    .RequireAuthorization("WorkflowEscalation.Respond");
```

CONTRACT LIMIT

No endpoint accepts actor identity, progress phase, owner outcome or resolution state directly from the request body.

PROBLEM DETAILS

HTTP outcomes retain semantic distinctions

The Edge maps Application outcomes to stable status codes and Problem Details types. A concurrency conflict, missing resource, already-resolved escalation and authorization failure are not collapsed into 500 or a generic false body.

DECISION LEDGER 39-29

Application outcome	HTTP	Problem type
Accepted / duplicate	200	acknowledgement representation
NotFound	404	/problems/workflow-escalation-not-found
AlreadyResolved	410	/problems/workflow-escalation-resolved
VersionConflict	409	/problems/workflow-version-conflict
InvalidAssessment	422	/problems/assessment-not-admissible
Forbidden	403	framework authorization response

NO SECRET LEAKAGE

Problem Details carries trace/correlation references and safe codes, not SQL text, connection values, stack traces or message bodies.

AUTHORIZATION

Reader and responder capabilities are separate

Policies require explicit scopes and the correct site/line boundary derived from trusted claims. A responder can acknowledge operational attention but gains no RootCause, Compliance or Workflow administration capability.

EXECUTABLE ASSET 39-R - POLICY REGISTRATION

```
services.AddAuthorization(options =>
{
    options.AddPolicy("WorkflowEscalation.Read", p => p
        .RequireClaim("scope", "workflow.escalations.read"));
    options.AddPolicy("WorkflowEscalation.Respond", p => p
        .RequireClaim("scope", "workflow.escalations.respond")
        .AddRequirements(new MatchingOperationalBoundaryRequirement()));
});
```

LEAST AUTHORITY

Authentication proves identity; authorization proves the specific escalation capability and operational boundary for this request.

IDEMPOTENCY

Operator retries use a local receipt, not transport optimism

The acknowledgement endpoint stores a normalized Idempotency-Key, request fingerprint, actor hash and response outcome in WorkflowDb. The receipt and escalation action commit atomically so a client retry cannot create another acknowledgement.

DECISION LEDGER 39-31

Same key	Same fingerprint	Disposition
yes	yes	return retained semantic result
yes	no	409 idempotency conflict
no	same actor after accepted	409 already acknowledged
expired receipt	n/a	retention policy; action history still authoritative

SCOPE

This is HTTP idempotency inside the Process Manager. It does not add a public broker command or target-service command inbox.

OPERATOR READ MODEL

The representation shows evidence and uncertainty together

The query returns stable identities, four-axis state, exact expected contract, due/overdue instants, escalation summary, projection freshness and permitted actions derived from authorization. It does not return private owner rows.

EXECUTABLE ASSET 39-S - SAFE WORKFLOW VIEW

```
public sealed record WorkflowAttentionView(  
    Guid WorkflowId,  
    string Progress,  
    string Timeliness,  
    string ProcessingHealth,  
    string EscalationState,  
    ExpectedFactView? ExpectedFact,  
    DateTimeOffset? DueAtUtc,  
    DateTimeOffset? OverdueAtUtc,  
    EscalationView? Escalation,  
    FreshnessView Freshness,  
    ImmutableArray<string> PermittedActions,  
    string Version);
```

ETAG

Version is an opaque concurrency token. It is not a timestamp or business sequence that clients may increment.

OPERATOR LANGUAGE

Labels must preserve the four axes

The UI uses precise phrases and visible timestamps. It never turns queue lag into service failure or an acknowledged escalation into a completed workflow.

DECISION LEDGER 39-33

Data	Recommended label	Avoid
OnTime + current	Waiting for verdict	Processing
Overdue + current	Verdict fact overdue since ...	RootCause failed
Overdue + stale	Status stale; lateness not re-evaluated	Still overdue
Acknowledged	Investigation acknowledged by operator	Issue resolved
CompletedLate	Completed after deadline	Completed on time

HONEST UI

Freshness and last-observed fact stay adjacent to status so an operator can judge whether the projection itself is current.

HUMAN REVIEW BOUNDARY

Workflow escalation review is not Compliance review

Two human-facing concepts now coexist. ComplianceReview is a Compliance-owned business aggregate opened from a verdict. WorkflowEscalation is Process-Manager operational attention raised because expected evidence is late. Neither can close or approve the other.

DECISION LEDGER 39-34

Dimension	WorkflowEscalation	ComplianceReview
owner	Process Manager / WorkflowDb	Compliance / ComplianceDb
trigger	deadline elapsed	owner policy from verdict
human action	acknowledge/assess attention	review findings/decision
completion effect	none by operator	Compliance-owned lifecycle
public fact	none activated	ComplianceReviewOpened.v1

TERMINOLOGY GATE

Code, routes and UI use Escalation for operational attention and Review only for the Compliance domain unless fully qualified.

ADR-038 RE-EVALUATION

RequestOperatorAttention does not pass the necessity gate

Operator visibility is satisfied by the Process-Manager query plus its local acknowledgement API. No external target service must be commanded. A public RequestOperatorAttention command would have no identified target-owned consumer or refusal contract.

DECISION LEDGER 39-35

Command gate	Result
target ownership	no external target contract identified
necessity	blocked; local query/API satisfies use case
typed refusal	absent
semantic idempotency	not specified
expiry	not specified
authorization	local HTTP policy exists, public sender grant absent
runtime evidence	PENDING

ADR RESULT

The approved public command set remains empty. ADR-038 and its model identity are not superseded by Chapter 39.

CONTRACT INVENTORY

The four fact bindings remain unchanged

The successful Alarm-to-Compliance path still advances through the same four owner facts. Chapter 39 adds WorkflowDb tables and HTTP contracts, not a fifth integration fact or a command route.

EXECUTABLE ASSET 39-T - UNCHANGED PUBLIC INVENTORY

```
Assert.Equal(4, CoordinationContracts.Facts.Length);
Assert.Empty(CoordinationContracts.Commands);
Assert.Empty(topology.CommandRoutes);
Assert.Equal(
    [ "GET /v1/workflows/{workflowId}",
      "POST .../acknowledgements",
      "POST .../assessments" ],
    processManager.HttpSurface);
```

FUTURE OWNER OUTCOME

If an explicit negative owner fact is later justified, it requires a versioned contract, consumer mapping and compatibility evidence. Timeout does not substitute for it.

COMPOSITION ROOT

Deadline scanning and operator APIs share one owner boundary

Production DI registers one scoped deadline service, one provider-specific lease store, TimeProvider for wake-up/testing, authorization policies and query/command handlers. It still excludes public command publishers and owner mutation clients.

EXECUTABLE ASSET 39-U - PRODUCTION REGISTRATION

```
services.AddSingleton(TimeProvider.System);
services.AddScoped<IDueExpectationScanner, DueExpectationScanner>();
services.AddScoped<IDeadlineLeaseStore, SqlServerDeadlineLeaseStore>();
services.AddScoped<IAcknowledgeEscalationHandler,
    AcknowledgeEscalationHandler>();
services.AddHostedService<DeadlineScannerWorker>();

// Still forbidden:
// IPublicCommandPublisher, RootCauseDbContext,
// ComplianceDbContext, owner mutation HTTP clients.
```

LIFETIME GATE

The hosted singleton receives IServiceScopeFactory and immutable options only; it does not capture WorkflowDbContext.

ARCHITECTURE TESTS

Static gates preserve the authority boundary

Architecture tests inspect assemblies, endpoint metadata and DI descriptors. They prove local operational commands exist while public inter-service commands, owner database dependencies and generic mutation endpoints remain absent.

DECISION LEDGER 39-38

Gate	Expected
public command handlers	0
operator local handlers	2 exact types
owner DbContexts	0
owner mutation HTTP clients	0
generic Workflow PATCH/DELETE	0
deadline worker	1 bounded hosted service
WorkflowDb runtime writer	1 owner database

ZERO-DISCOVERY GUARD

The test also asserts expected fact handlers, endpoints and worker count so zero forbidden types cannot be caused by a scanner failure.

TOPOLOGY GATE

No broker resource changes are required

Escalation remains query-visible from WorkflowDb. The broker manifest is byte-identical to ADR-038: one durable workflow queue, four fact bindings, manual acknowledgement and zero command resources.

EXECUTABLE ASSET 39-V - TOPOLOGY DIGEST CONTINUITY

```
var before = LoadTopology("chapter38/topology.json");
var after  = LoadTopology("chapter39/topology.json");

Assert.Equal(before.CanonicalSha256, after.CanonicalSha256);
Assert.Equal(4, after.FactBindings.Count);
Assert.Empty(after.CommandRoutes);
Assert.DoesNotContain(after.Exchanges,
    x => x.Name.Contains("escalation-command"));
```

NO HIDDEN NOTIFICATION PATH

Email, chat or incident-system delivery is not implemented here. Adding one requires an explicit adapter/contract, target and evidence boundary.

HOST INVENTORY

The production path exposes the exact Chapter 39 surface

A WebApplicationFactory-based test starts the real Program path and inventories endpoints, authorization, workers, databases, fact bindings and forbidden capabilities. It does not register fake production services to make the test pass.

EXECUTABLE ASSET 39-W - HOST CAPABILITY TEST

```
var m = await factory.GetCapabilityManifestAsync();

Assert.Equal("ADR-038", m.Coordination.AdrId);
Assert.Equal(0, m.Messaging.CommandRoutes.Count);
Assert.Equal(1, m.Workers.Count(x => x == "DeadlineScannerWorker"));
Assert.Equal(["WorkflowDb"], m.Databases.RuntimeWriteTargets);
Assert.Contains("WorkflowEscalation.Respond", m.AuthorizationPolicies);
Assert.DoesNotContain(m.Endpoints, x => x.Contains("force-complete"));
```

SAME-PATH EVIDENCE

The manifest is derived from runtime descriptors and endpoint data sources, not from a hand-written expected list.

SECURITY EVIDENCE

Every human action is attributable without storing unnecessary identity

The API authenticates the operator, authorizes scope/boundary, hashes the stable subject with a rotating audit key, records the action and correlation, and emits a safe audit event. Raw tokens, display names and claims are never persisted in WorkflowDb.

DECISION LEDGER 39-41

Persist	Do not persist
pseudonymous subject hash	access/refresh token
authorized scope snapshot	full claims principal
action + reason code	free-form personal notes
request/idempotency fingerprint	request authorization header
UTC instant + lineage	client IP unless separately justified

AUDIT KEY IDENTITY

Evidence records the audit-key identity/version, never the key value. Key rotation must preserve the ability to interpret retained action records.

THREAT REVIEW

Escalation adds a human mutation surface with bounded impact

The new endpoint can falsely acknowledge or misclassify operational attention if compromised. It still cannot alter owner truth or workflow progress. Controls focus on authorization, replay protection, concurrency, data minimization and append-only history.

DECISION LEDGER 39-42

Threat	Control
stolen responder token	short-lived identity + scope/boundary policy
replayed acknowledgement	idempotency receipt + actor binding
two operators race	rowversion + typed conflict
assessment injection	closed enum + no free text
history tampering	least-privilege writes + append-only action evidence
force domain result	no endpoint/client/credential

RESIDUAL RISK

A compromised responder can distort attention workflow. It cannot legitimately create the owner fact required for distributed completion.

ALERT FATIGUE

One expectation creates one durable attention case

Repeated scans do not create repeated escalations. The UI orders open cases by overdue age and level, and an acknowledged case remains visible without producing another Raised action. Any reminder cadence is separately policy-bound.

DECISION LEDGER 39-43

Control	Effect
unique ExpectationId	one escalation aggregate
matching duplicate outcome	no repeated audit/notification
acknowledgement state	attention ownership visible
typed level	stable priority vocabulary
bounded list/query	operator load remains manageable
no implicit reminders	no alert storm after restart

REMINDER BOUNDARY

V1 specifies no outbound reminder adapter. If introduced, reminders must be deduplicated and must not create new escalation identities.

ESCALATION LEVEL

Priority is policy-derived, not manually inflated

The initial level derives from expectation kind and overdue age using the frozen policy snapshot. Operators may acknowledge and assess but cannot arbitrarily promote severity. A later policy step may raise level through an audited system transition.

DECISION LEDGER 39-44

Level	Meaning	Mutation owner
Attention	new overdue expectation	deadline policy
OwnerTeam	policy step requires owner-team visibility	policy worker
IncidentCoordinator	validated escalation ladder step	policy worker
SafetySeverity	not inferred by WorkflowDb	owner domain only

NO SAFETY INFERENCE

Operational escalation level is not nuclear safety significance, incident classification or Compliance severity.

FRESHNESS GATE

A stale Process Manager pauses new lateness claims

If queue or worker freshness exceeds its validated bound, the UI marks ProcessingHealth Stale. The service retains existing overdue history but does not present newly computed timeliness as current until the backlog is drained and freshness recovers.

DECISION LEDGER 39-45

Condition	UI behavior	Automatic action
current evidence	show four-axis state	scan deadlines
queue lag high	show stale banner	continue bounded recovery
WorkflowDb unavailable	status unavailable/degraded	no guessed state
recovered	show current as-of instant	resume due classification

CONSERVATIVE DISPLAY

Staleness does not clear an existing escalation. It qualifies confidence in the current projection.

OBSERVABILITY

Telemetry distinguishes scan, classification and human action

Each scan batch records lease, candidates, outcomes and duration. Each escalation action records safe workflow/expectation references, policy identity, transition, actor hash and disposition. Metrics avoid WorkflowId, EscalationId and actor labels.

DECISION LEDGER 39-46

Signal	Low-cardinality dimensions
deadline_scan_duration	result / provider
expectations_classified_total	expectation kind / outcome
open_escalations	level / state
overdue_age	expectation kind / level
acknowledgement_total	outcome / policy
late_completion_total	expectation kind

TRACE BOUNDARY

Operator HTTP spans link to WorkflowDb actions. They do not fabricate a causal parent inside the earlier owner fact chain.

CAPACITY

Scanning is bounded, indexed and recoverable

Capacity planning measures due-row selectivity, batch transaction time, lock contention, backlog drain and query competition. Batch size, scan interval and lease duration are validated together; the book fixes no production throughput threshold.

DECISION LEDGER 39-47

Claim	Required measurement
steady state	due rate <= classified rate
restart recovery	drain rate > new due rate
lease safety	processing p99 < lease duration with margin
index selectivity	actual plan + logical reads
operator query isolation	read latency without starving scanner writes
history growth	retention and index size over workload horizon

EVIDENCE STATUS

All capacity claims remain PENDING until a retained workload profile, SQL plan and results exist.

DETERMINISTIC TIME TESTS

FakeTimeProvider proves boundary behavior without sleeping

Unit and worker tests set a known instant, create an expectation and advance synthetic time. They assert no early classification, exactly one transition at the deadline, and no additional transition after repeated scans.

EXECUTABLE ASSET 39-X - TIME BOUNDARY TEST

```
var time = new FakeTimeProvider(Instant("2026-07-19T08:00:00Z"));
var e = Fixture.Expectation(dueAt: time.GetUtcNow().AddMinutes(5));

time.Advance(TimeSpan.FromMinutes(5) - TimeSpan.FromTicks(1));
Assert.Equal(DeadlineOutcome.NotDue, e.MarkOverdue(time.GetUtcNow()));

time.Advance(TimeSpan.FromTicks(1));
Assert.Equal(DeadlineOutcome.BecameOverdue, e.MarkOverdue(time.GetUtcNow()));
Assert.Equal(DeadlineOutcome.MatchingDuplicate, e.MarkOverdue(time.GetUtcNow()));
```

NO THREAD.SLEEP

Wall-clock sleeps create slow, flaky tests and do not prove exact boundary ticks.

PROVIDER TESTS

Real SQL Server proves leasing and constraints

A disposable provider database applies the production migrations and runs concurrent scanners. Tests verify one lease, one escalation, rowversion conflicts, filtered-index presence, SQL UTC comparison and lease-expiry recovery.

DECISION LEDGER 39-49

Scenario	Assertion
two scanners / one due row	one lease owner
two overdue commits	one escalation + one Raised action
fact versus scanner	one legal race history
worker crash after lease	row eligible after LeaseUntilUtc
closed expectation	not returned by due query
RCSI database	hint combination executes as designed

PROVIDER OWNERSHIP

InMemory and SQLite tests cannot prove SQL Server locks, rowversion, filtered indexes or provider error numbers.

RACE CORPUS

Every interleaving has one expected semantic history

The concurrency suite uses controllable barriers rather than probabilistic loops. It pauses after load, lease and before commit to force fact/scanner and operator/system interleavings, then compares the append-only action sequence.

DECISION LEDGER 39-50

Interleaving	Expected terminal evidence
fact commits first	Observed OnTime; no escalation
overdue commits first	CompletedLate; Raised then Resolved
ack commits before late fact	Raised, Acknowledged, Resolved
late fact before ack	Raised, Resolved; ack returns AlreadyResolved
two acknowledgers	one Accepted; one VersionConflict
worker dies after lease	no state change; later lease recovery

NO TIMING LUCK

A race test passes only when the intended interleaving was actually reached and recorded.

AUTHORIZATION TESTS

Human review is tested as a capability boundary

Production-path HTTP tests use signed test identities with exact scopes and operational boundaries. They verify challenges, forbids, actor derivation, idempotency and the absence of privileged fields in requests and responses.

DECISION LEDGER 39-51

Identity	Expected response
anonymous	401
reader scope	GET 200; POST 403
responder wrong boundary	403
responder matching boundary	200 Accepted
body attempts actor override	ignored/rejected; trusted actor retained
duplicate key/different body	409 idempotency conflict

NEGATIVE COVERAGE

A single happy-path 200 test does not prove the authorization surface.

FAILURE LABORATORY

The controller action is verified under real faults

Fault injection targets timer wake-up, lease acquisition, WorkflowDb commit, acknowledgement races, clock drift alarms and stale projection recovery. The assertion is the resulting durable action and operator state, not merely an exception or HTTP code.

DECISION LEDGER 39-52

Injected fault	Required recovery evidence
worker restart past deadline	due row classified once
DB unavailable	no guessed overdue; retry after recovery
crash after overdue commit	next scan matching duplicate
lease owner dies	second replica after expiry
late fact during acknowledgement	legal typed race outcome
projection backlog	stale banner then current recovery
unauthorized operator	no action row

NO GLOBAL ROLLBACK

Fault recovery changes WorkflowDb attention state only. It never rolls back or edits an owner transaction.

EVIDENCE CATALOGUE

Each Chapter 39 claim maps to its proving surface

The catalogue separates Domain time semantics, SQL arbitration, host composition, security and failure recovery. No one test layer stands in for another.

DECISION LEDGER 39-53

Claim	Proof surface	Minimum artifact
exact deadline boundary	unit + FakeTimeProvider	TRX + fixture digest
one multi-replica lease	SQL provider test	database log + result
one escalation	constraint/concurrency test	migration + race trace
operator authority	host authorization tests	endpoint/policy inventory
zero public commands	topology/DI tests	unchanged topology digest
late completion history	provider race corpus	semantic action sequence
fault recovery	deployment fault lab	timeline + artifact hashes

BOUNDED CLAIM

Reference code specifies the expected behavior. Only retained outputs from the named surface may prove it.

EVIDENCE MANIFEST

The Chapter 39 dossier starts PENDING

The manifest binds ADR-038, deadline policy identity, repository state, migrations, provider options, topology continuity, test commands and artifact hashes. It cannot become PROVED with missing or zero-discovery suites.

EXECUTABLE ASSET 39-Y - PROPOSED DOSSIER

```
{
  "evidenceId": "NEXUS1-DEADLINE-ESCALATION-039",
  "status": "PENDING",
  "coordinationAdrSha256": "622787772a9def5ef18bffc6a622d6efc11185d0a696780445a3ee8cc027f22a",
  "referencePolicySha256": "5a6dd3526033f76967761fe972e82a18e9ccd0ead2be00bd1401a4c240a3cfe2",
  "productionPolicyStatus": "UNBOUND",
  "approvedPublicCommandCount": 0,
  "requiredSuites": [
    "unit-time", "sql-provider", "race", "authorization",
    "host", "fault", "topology-continuity"
  ],
  "artifacts": []
}
```

TRUTHFUL STATE

This PDF production run did not execute the repository/runtime suite. Evidence status remains PENDING.

EVIDENCE VALIDATOR

The validator rejects stale policy and accidental commands

Validation requires the same repository commit, ADR digest, policy digest, migration set and topology across every suite. It also proves the expected tests and facts were discovered before accepting the zero-command result.

EXECUTABLE ASSET 39-Z - FAIL-CLOSED VALIDATION

```
Require(manifest.Status == "PROVED");
Require(manifest.ExecutedTests > 0 && manifest.FailedTests == 0);
Require(manifest.DiscoveredFacts == 4);
Require(manifest.ApprovedPublicCommands == 0);
Require(manifest.TopologySha256 == chapter38TopologySha256);
Require(manifest.ProductionPolicyStatus == "BOUND");
RequireSingleCommitAndCleanTree();
RequireExactSuitesAndMatchingEnvironment();
RequireAllArtifactsExistAndMatchSha256();
```

REFERENCE PROFILE CANNOT PROVE PRODUCTION

A dossier using the UNBOUND reference-test profile must be rejected as production evidence even if every test passes.

CI REPRODUCTION

One bounded target builds the deadline dossier

The pipeline restores locked dependencies, builds Release, provisions isolated SQL Server resources, runs deterministic time, provider, race, authorization, host and fault suites, validates topology continuity and archives value-free artifacts.

EXECUTABLE ASSET 39-AA - PROPOSED REPRODUCTION PATH

```
dotnet test tests/Nexus1.ProcessManager.EvidenceTests \
  --configuration Release \
  --no-restore \
  --filter "Category=Chapter39" \
  --logger "trx;LogFileName=chapter39.trx" \
  --results-directory artifacts/evidence/chapter39

dotnet run --project tools/EvidenceValidator -- \
  artifacts/evidence/chapter39/manifest.json
```

REQUIRED BINDING

Artifacts record commit, dirty flag, dependency lock, .NET/SQL versions, database options, policy/topology/migration digests, commands, fault points and SHA-256 values.

DEPLOYMENT

Deadline policy rolls out in shadow before operator reliance

Apply WorkflowDb migrations, bind and validate the production policy, run the scanner in classification shadow mode, compare expected due rows, then enable escalation writes and finally the operator endpoints. Existing owner services and broker topology remain unchanged.

DECISION LEDGER 39-57

Stage	Enabled	Exit evidence
prepare	schema/index/policy validation	migration + policy digest
shadow	due query without writes	candidate comparison
classify	Overdue + escalation writes	race/fault probes
expose	read and acknowledgement API	authorization/UX checks
steady	SLO/lag/alert monitoring	retained operational dossier

REVERSAL

Disable endpoints and scanner, preserve WorkflowDb history and leases, correct policy/code, then resume. Do not delete escalations or rewrite owner facts.

CHAPTER 40 BOUNDARY

Escalation records urgency; it does not compensate history

A timeout can motivate investigation or later corrective work, but Chapter 39 never reverses the flood, case, verdict or review. The next chapter will model compensation as new owner facts and actions with retained lineage, not as deletion of prior truth.

DECISION LEDGER 39-58

Chapter 39 owns	Chapter 40 must decide
durable deadline and overdue history	which corrective action is admissible
operator acknowledgement/assessment	which owner receives a request/fact
late completion resolution	how new action preserves old history
zero public commands	whether any compensation contract passes ADR gates
repair assessment hint	separation from Chapter 42 repair jobs

PROTECTED HANDOFF

Compensation cannot be encoded as ForceComplete, row deletion or an operator edit to owner truth.

OFFICIAL TECHNICAL ANCHORS

Time, concurrency and authorization are explicit boundaries

- TimeProvider overview: <https://learn.microsoft.com/dotnet/standard/datetime/timeprovider-overview>
- FakeTimeProvider testing: <https://learn.microsoft.com/dotnet/core/extensions/timeprovider-testing>
- Scoped services in BackgroundService: <https://learn.microsoft.com/dotnet/core/extensions/scoped-service>
- EF Core optimistic concurrency: <https://learn.microsoft.com/ef/core/saving/concurrency>
- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- SQL Server table hints: <https://learn.microsoft.com/sql/t-sql/queries/hints-transact-sql-table>
- SQL Server SYSUTCDATETIME: <https://learn.microsoft.com/sql/t-sql/functions/sysutcdatetime-transact-sql>
- ASP.NET Core claims authorization: <https://learn.microsoft.com/aspnet/core/security/authorization/claims>
- ASP.NET Core API error handling: <https://learn.microsoft.com/aspnet/core/fundamentals/error-handling-api>

WHAT THIS CHAPTER HAS PROVED

Chapter 39 specifies policy-bound durable expectations; validated and immutable deadlines; database-clock arbitration; bounded multi-replica leasing; one atomic Overdue/escalation/history commit; race-safe late completion; a local operational escalation aggregate; authorized, idempotent acknowledgement and assessment APIs; four-axis operator meaning; unchanged four-fact/zero-public-command topology; security, capacity, deployment and fail-closed evidence paths. Runtime evidence remains PENDING.

CHAPTER CHECKPOINT

- Why is a timeout an observation rather than a business verdict?
- Which four state axes must remain independent?
- Why is DueAtUtc frozen instead of recomputed after configuration changes?
- Why does the timer wake a scan rather than own the deadline?
- How do SQL Server leasing and uniqueness arbitrate multiple replicas?
- What history remains when the expected fact arrives after escalation?
- Why can an operator acknowledge but not resolve or force completion?
- Why is WorkflowEscalation different from ComplianceReview?
- Why does ADR-038 still contain zero public commands?
- What artifacts are required before Chapter 39 can be called PROVED?

NEXT - CHAPTER 40

Compensation Without Pretending to Undo History will model corrective work as new, owner-authorized facts and actions. It will preserve the original commits, timeout history and causality while distinguishing compensation from replay, repair and operator acknowledgement.

PART VII - WORKFLOW, CONSISTENCY, AND REPAIR

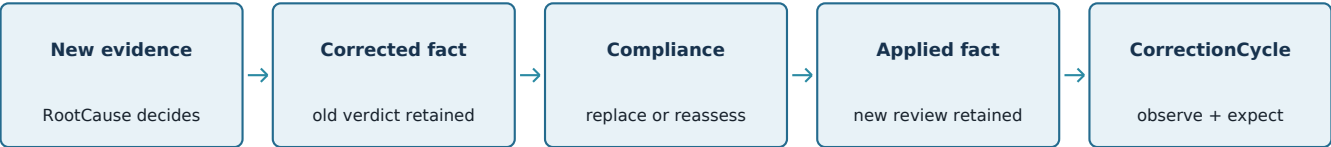
40. Compensation Without Pretending to Undo History

A new RootCause verdict does not erase the old verdict, and a Compliance correction does not delete the old review. Each owner commits a new forward fact, preserves lineage and remains safe under repeated, late and reordered delivery.

CHAPTER DECISION

ADR-040 extends observed choreography with RootCauseVerdictCorrected.v1 and ComplianceReviewCorrectionApplied.v1. RootCause appends a corrected verdict that supersedes its current verdict. Compliance either supersedes an active review and opens a replacement, or preserves a terminal review and opens a reassessment. Inbox, owner effects, history and outbox commit locally. The Process Manager observes a CorrectionCycle; public commands remain zero.

PROOF PATH 40-A - CORRECTION AS NEW OWNER TRUTH



GUARANTEE LEDGER 40-A

Specified	Explicitly rejected
two owner facts with exact lineage	delete or edit published history
active replacement and late reassessment modes	one inverse action for every state
repeated/out-of-order idempotency	exactly-once transport
six fact bindings and zero commands	Process-Manager compensation authority
fail-closed evidence dossier	runtime proof without artifacts

COMPENSATION MEANING

The original commit remains true forever

AlarmManagement detected a flood, RootCause issued a verdict and Compliance opened a review. A later correction can change which verdict is current and which review should receive attention, but it cannot make the earlier commits, public facts or operator observations disappear.

DECISION LEDGER 40-2

Earlier truth	Forward correction	Forbidden fiction
flood was detected	new alarm fact if owner meaning changes	delete flood history
verdict was issued	new verdict supersedes it	edit issued fact in place
review was opened	replace or reassess review	pretend review never existed
deadline elapsed	resolve after corrective fact	erase overdue history

CHAPTER RULE

Compensation adds owner-authorized meaning after prior truth. It does not restore a fictional past.

WHY CORRECTION OCCURS

New evidence can invalidate the current verdict without invalidating history

The concrete NEXUS-1 scenario begins when RootCause admits a new evidence item after its original verdict. RootCause re-evaluates its own policy and issues a corrected verdict. That correction is a business fact, not a timeout inference or a Process-Manager decision.

DECISION LEDGER 40-3

Trigger	Decision owner	Result
new evidence admitted	RootCause	re-evaluate active verdict
correction eligible	RootCause	new verdict + supersession link
review impact	Compliance	replace active review or open reassessment
workflow tracking	Process Manager	observe correction cycle

NO TIMEOUT TRIGGER

Overdue or operator acknowledgement alone cannot correct a verdict. A RootCause-owned evidence decision is required.

COMPENSABILITY MAP

Every completed step has a different correction grammar

The workflow cannot assume that each step has an inverse. The catalog classifies whether the owner can supersede meaning, create a follow-up action or only retain history.

DECISION LEDGER 40-4

Completed step	Classification	Allowed response
AlarmFloodDetected	historical observation	new owner fact; never delete
RootCauseCaseOpened	durable owner identity	retain case; append status/history
VerdictIssued	supersedable conclusion	new corrected verdict
ComplianceReviewOpened	active or terminal work	replace active / reassess terminal
audit evidence	irreversible record	append correction reference

POINT OF NO DELETION

Public facts and audit evidence are irreversible. Only current owner interpretation can move forward.

CHOSEN SLICE

Verdict correction drives one Compliance-owned compensating action

RootCause publishes RootCauseVerdictCorrected.v1 after committing a new verdict. Compliance consumes that owner fact, preserves the prior review, creates one replacement or reassessment review and publishes ComplianceReviewCorrectionApplied.v1 from the same local transaction.

DECISION LEDGER 40-5

Stage	Local commit	Public meaning
RootCause correction	old + new verdict, evidence, outbox	VerdictCorrected.v1
Compliance compensation	inbox, prior review, new review, action, outbox	ReviewCorrectionApplied.v1
Process observation	correction cycle + expectations	queryable progress only

PUBLIC COMMAND INVENTORY

Still empty. Compliance reacts to a RootCause owner fact; the Process Manager issues no ReplaceReview command.

ADR-040**Forward compensation extends the fact profile**

ADR-040 supplements ADR-038 without changing the selected coordination model. The normal four-fact path remains intact. Two exact correction facts are added, the coordination profile advances to revision 2, and public commands remain explicitly empty.

DECISION LEDGER 40-6

ADR field	Value
Decision	append-and-supersede; never delete
Coordination	fact-driven observed choreography
Normal path	four existing facts
Correction path	two new owner facts
Public commands	empty set
Evidence	PENDING

VERSIONED PROFILE

Coordination profile: nexus1/coordination/observed-choreography/2. The ADR-038 architecture choice remains authoritative.

ADR IDENTITY**The compensation decision has a canonical digest**

The ADR binds the contract extension, history policy and zero-command choice to one immutable body. CI, topology and evidence manifests must report the same digest before the chapter can leave PENDING.

EXECUTABLE ASSET 40-A - ADR-040 IDENTITY

```
const string AdrId = "ADR-040";
const string ExpectedSha256 = "65eeede68f69f856376e89bbb01d79c7243a30e6778b7cacf0e57ef4178ea741";

byte[] body = CanonicalJson.Serialize(adr040);
string actual = Convert.ToHexString(
    SHA256.HashData(body)).ToLowerInvariant();

Assert.Equal(ExpectedSha256, actual);
Assert.Empty(adr040.ApprovedPublicCommands);
Assert.Equal("append-and-supersede-never-delete", adr040.HistoryPolicy);
```

EXACT FIXTURE

ADR-040 canonical body: 486 UTF-8 bytes; SHA-256
65eeede68f69f856376e89bbb01d79c7243a30e6778b7cacf0e57ef4178ea741.

ROOTCAUSE AUTHORITY

Only RootCause may decide that its verdict is corrected

The Process Manager, Compliance and an operator may observe new evidence or request investigation through their own boundaries, but only RootCause can interpret its evidence set and commit a corrected verdict.

DECISION LEDGER 40-8

Action	Allowed owner
admit new root-cause evidence	RootCause
decide correction eligibility	RootCause policy
create corrected verdict	RootCause aggregate
publish corrected verdict fact	RootCause outbox
apply Compliance consequence	Compliance
track correction progress	Process Manager

NO BORROWED EVIDENCE

RootCause does not query ComplianceDb, and Compliance does not recompute the verdict.

CORRECTION ELIGIBILITY

A verdict can be superseded only from the current case head

The case verifies that the referenced verdict belongs to it, is the current active verdict, has not already been superseded and is supported by a strictly newer admitted evidence set. A correction of a historical non-head verdict fails closed.

EXECUTABLE ASSET 40-B - ROOTCAUSE CORRECTION POLICY

```
public CorrectionOutcome CorrectVerdict(
    VerdictId supersededId,
    Verdict corrected,
    EvidenceSet evidence,
    CorrectionId correctionId)
{
    if (ActiveVerdictId != supersededId) return CorrectionOutcome.NotCurrent;
    if (!_corrections.Contains(correctionId)) return CorrectionOutcome.Duplicate;
    if (!evidence.IsStrictlyNewerThan(CurrentEvidence))
        return CorrectionOutcome.NoNewEvidence;

    var prior = GetVerdict(supersededId);
    prior.MarkSupersededBy(corrected.Id, correctionId);
    AddCorrectedVerdict(corrected, supersededId, evidence, correctionId);
    ActiveVerdictId = corrected.Id;
    return CorrectionOutcome.Applied;
}
```

CHAIN RULE

A later correction supersedes the latest active verdict, creating a chain. It never rewrites an older link.

VERDICT RECORD

Old and corrected verdicts remain separate durable rows

The corrected verdict carries its own identity, evidence digest, policy identity and issued instant plus an explicit `SupersedesVerdictId`. The prior row gains only a superseded marker and link; its original body remains unchanged.

EXECUTABLE ASSET 40-C - APPEND-AND-SUPERSEDE MODEL

```
public sealed record VerdictCorrectionSnapshot(  
    Guid CorrectionId,  
    Guid RootCauseCaseId,  
    Guid SupersededVerdictId,  
    Guid CorrectedVerdictId,  
    string ReasonCode,  
    string EvidenceSetDigest,  
    string PolicyIdentity,  
    DateTimeOffset CorrectedAtUtc);  
  
// A corrected verdict is a new Verdict row.  
// SupersedesVerdictId is immutable after creation.
```

AUDIT MEANING

Queries may expose `CurrentVerdict` and `VerdictHistory`. They must not overwrite history with only the current result.

ROOTCAUSE TRANSACTION

Correction and publication intent commit atomically

The application handler loads the case, admits the exact new evidence, applies the correction policy and commits evidence receipt, prior/current verdict changes, correction record and outbox intent in one `RootCauseDb` transaction.

EXECUTABLE ASSET 40-D - ROOTCAUSE LOCAL COMMIT

```
await execution.ExecuteAsync(async () =>  
{  
    await using var db = await factory.CreateDbContextAsync(ct);  
    await using var tx = await db.Database.BeginTransactionAsync(ct);  
  
    var rootCauseCase = await cases.LoadAsync(command.CaseId, ct);  
    var outcome = rootCauseCase.CorrectVerdict(  
        command.SupersededVerdictId, corrected, evidence, command.CorrectionId);  
    outbox.Add(mapper.ToVerdictCorrected(rootCauseCase, outcome));  
  
    await db.SaveChangesAsync(ct);  
    await tx.CommitAsync(ct);  
});
```

NO BROKER TRANSACTION

The local commit proves durable publication intent, not broker delivery. The existing outbox dispatcher owns dispatch and confirms.

ROOTCAUSE CONSTRAINTS

Provider uniqueness arbitrates correction races

SQL Server constraints ensure one CorrectionId, one direct correction of a superseded verdict and one current verdict per case. Rowversion protects the case head under concurrent evidence processing.

EXECUTABLE ASSET 40-E - CORRECTION CONSTRAINTS

```
correction.HasKey(x => x.CorrectionId);
correction.HasIndex(x => x.SupersededVerdictId).IsUnique();
correction.HasIndex(x => x.CorrectedVerdictId).IsUnique();

verdict.HasIndex(x => new { x.RootCauseCaseId, x.IsCurrent })
    .IsUnique()
    .HasFilter("[IsCurrent] = 1");

rootCauseCase.Property(x => x.RowVersion).IsRowVersion();
```

SEMANTIC RETRY

After a concurrency conflict, a fresh unit of work re-evaluates which verdict is current. It does not retry the stale correction outcome.

DELETE PROHIBITION

Runtime principals cannot erase verdict or audit history

Application repositories expose append and supersede operations but no generic Remove. The RootCause runtime principal lacks DELETE on verdict, correction, outbox and evidence-history tables; retention remains a separate governed concern.

DECISION LEDGER 40-13

Layer	Gate
Domain	no DeleteVerdict behavior
Application	typed CorrectVerdict command only
Repository	no generic Remove/Delete method
Database	runtime principal lacks DELETE
Audit	append-only correction lineage
Tests	permission and descriptor inventory

NOT EVENT SOURCING

The services keep relational current-state rows plus explicit history. The pattern does not claim that every aggregate is event-sourced.

PUBLIC CORRECTION CONTRACT

RootCauseVerdictCorrected.v1 carries sufficient owner meaning

The V1 payload identifies the correction, case, prior and corrected verdicts, reason code, evidence-set digest, policy identity, time and producer stream. It does not expose private evidence content or Compliance decisions.

EXECUTABLE ASSET 40-F - CORRECTED VERDICT FACT

```
public sealed record RootCauseVerdictCorrectedV1(  
    Guid CorrectionId,  
    Guid RootCauseCaseId,  
    Guid SupersededVerdictId,  
    Guid CorrectedVerdictId,  
    string ReasonCode,  
    string EvidenceSetDigest,  
    string CorrectionPolicyIdentity,  
    DateTimeOffset CorrectedAtUtc,  
    ProducerStreamV1 ProducerStream);
```

COMPATIBILITY BOUNDARY

Changing the meaning of correction, reason grammar or verdict linkage requires a new schema version.

CORRECTION CORPUS

The exact V1 bytes are frozen

The golden document extends the existing correlation while direct causation names the new-evidence message. Its producer stream continues the RootCause case stream at position 3.

EXECUTABLE ASSET 40-0 - ROOTCAUSE CORRECTION FIXTURE

```
EventType      = "nexus1.root-cause.root-cause-verdict-corrected.v1"  
MessageId      = "019bc4d4-b66c-7a7c-a5d4-f0e5386310b7"  
Correlation    = "request-am-20482"  
Causation      = "019bc4d4-b38a-78e1-84d8-a8afc6a3503"  
Correction     = "019bc4d4-b4f1-7a28-b55f-508e7e88fb5c"  
Prior          = "019bc4d4-81c0-7ea1-82d5-4f9b8719d0a1"  
Corrected      = "019bc4d4-b586-738b-953f-e31d9e1a5478"  
Fingerprint    = "16f65e5b1ba65b436c05de7ca11a4b187adb074d8c7fa578e64266d8c7763552"  
BodySha256     = "e22b868cb76c5278abc903f11afc4b11c2c8863d7c2f20b27f0a63ff84e60418"  
BodyLength     = 1039
```

GOLDEN-BODY RULE

Producer, Compliance consumer and Process-Manager consumer must accept the same retained bytes. Re-serialization is not a substitute.

CONTRACT ADMISSION

Compliance authenticates and validates before Domain work

The consumer copies delivery bytes, verifies broker producer identity, required properties, exact event type/version, canonical fingerprint, JSON grammar and stable identifiers, then translates the accepted fact into Compliance language.

DECISION LEDGER 40-16

Admission check	Failure disposition
producer principal/root-cause	reject forged path
exact event type + V1	reject unknown semantics
content fingerprint	quarantine tampered body
non-empty prior/corrected IDs	reject invalid contract
prior != corrected	reject self-supersession
UTC and stream position	reject invalid lineage

TRANSLATION RULE

A valid transport contract is still not permission to mutate any review. Compliance eligibility runs after admission.

COMPLIANCE AUTHORITY

Compliance decides how correction affects its review

RootCause states that its verdict was corrected. It does not command Compliance to delete, close or replace a review. Compliance maps the fact to one of two owner-local modes based on current review state.

DECISION LEDGER 40-17

Prior review state	Compliance mode	Old row
Pending / InReview	ActiveReviewSuperseded	mark Superseded; retain history
Approved / Rejected / Closed	ReassessmentAfterTerminal	leave terminal state unchanged
no review yet	BufferedUntilReviewExists	no guessed review
already corrected	MatchingDuplicate / Conflict	no second effect

OWNER POLICY

The mode is a Compliance decision. It is not encoded as a RootCause field or a Process-Manager command.

ACTIVE REVIEW MODE

An active review is superseded and replaced

When the prior review is still active, Compliance changes its current state to Superseded with an immutable correction link and creates a new Pending review for the corrected verdict. The old review remains queryable with its original opening evidence and action history.

EXECUTABLE ASSET 40-G - ACTIVE-REVIEW COMPENSATION

```
public CompensationResult ApplyToActiveReview(
    VerdictCorrection correction,
    ReviewId replacementId,
    DateTimeOffset now)
{
    EnsureState(ReviewState.Pending, ReviewState.InReview);
    MarkSuperseded(correction.Id, replacementId, now);
    var replacement = ComplianceReview.OpenReassessment(
        replacementId, correction.CorrectedVerdictId, Id, correction.Id, now);
    return CompensationResult.ActiveReviewSuperseded(this, replacement);
}
```

MEANING

Superseded is a new current state of the old review, not deletion or denial that it was once Pending/InReview.

LATE TERMINAL MODE

A terminal review remains terminal and a reassessment is appended

If the correction arrives after the prior review was approved, rejected or closed, Compliance cannot retroactively change that historical decision. It creates a new Pending reassessment linked to both the terminal review and corrected verdict.

EXECUTABLE ASSET 40-H - LATE COMPENSATION

```
public CompensationResult ApplyAfterTerminalReview(
    VerdictCorrection correction,
    ReviewId reassessmentId,
    DateTimeOffset now)
{
    EnsureState(ReviewState.Approved, ReviewState.Rejected, ReviewState.Closed);
    var reassessment = ComplianceReview.OpenReassessment(
        reassessmentId, correction.CorrectedVerdictId, Id, correction.Id, now);
    return CompensationResult.ReassessmentAfterTerminal(this, reassessment);
}
```

LATE DOES NOT MEAN INVALID

The message is late relative to review state, not invalid. Its forward effect differs and the old terminal decision remains visible.

TYPED COMPENSATION RESULT

The handler records which business mode actually applied

A typed result carries prior/replacement identities, before/after states and the exact mode. The outbox mapper uses this committed snapshot rather than reloading mutable rows after the transaction.

EXECUTABLE ASSET 40-I - OWNER-LOCAL RESULT

```
public sealed record CompensationResult(
    CompensationMode Mode,
    ReviewId PriorReviewId,
    ReviewState PriorStateBefore,
    ReviewState PriorStateAfter,
    ReviewId ReplacementReviewId,
    ReviewState ReplacementState,
    CorrectionId CorrectionId);

public enum CompensationMode
{
    ActiveReviewSuperseded,
    ReassessmentAfterTerminal
}
```

CLOSED GRAMMAR

Unknown modes fail contract mapping. Adding a mode requires Domain, contract, compatibility and evidence changes.

COMPLIANCE TRANSACTION

Inbox, reviews, action history and outbox commit once

The consumer performs all Compliance-owned effects in one transaction and acknowledges only a known commit or matching duplicate. A transient retry creates a fresh unit of work and re-evaluates current review state.

EXECUTABLE ASSET 40-J - COMPENSATION COMMIT BOUNDARY

```
await execution.ExecuteAsync(async () =>
{
    await using var db = await factory.CreateDbContextAsync(ct);
    await using var tx = await db.Database.BeginTransactionAsync(ct);

    var admitted = await inbox.AdmitAsync(delivery, ct);
    var prior = await reviews.LoadByVerdictAsync(admitted.SupersededVerdictId, ct);
    var result = compensation.Apply(prior, admitted, ids.NextReviewId(), clock.UtcNow);
    outbox.Add(mapper.ToCorrectionApplied(result, admitted));

    await db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
});
```

SETTLEMENT

The broker delivery is acknowledged after commit. An unknown commit outcome is retried through inbox/constraint arbitration, not assumed successful.

COMPLIANCE PERSISTENCE

Constraints preserve one effect per semantic correction

The model stores one receipt per MessageId, one applied-correction row per CorrectionId, one replacement review per correction and a rowversion on both affected reviews. Provider-specific unique errors are classified by exact constraint name.

EXECUTABLE ASSET 40-K - COMPLIANCE CONSTRAINTS

```

applied.HasKey(x => x.CorrectionId);
applied.HasIndex(x => x.SourceCorrectionMessageId).IsUnique();
applied.HasIndex(x => x.ReplacementReviewId).IsUnique();

review.HasIndex(x => new { x.SourceCorrectionId, x.IsReassessment })
    .IsUnique()
    .HasFilter("[SourceCorrectionId] IS NOT NULL");
review.Property(x => x.RowVersion).IsRowVersion();

```

NO GENERIC DUPLICATE CATCH

A foreign-key, check or unrelated unique violation remains a failure. It is not converted into MatchingDuplicate.

APPLIED PUBLIC FACT

ComplianceReviewCorrectionApplied.v1 reports the forward effect

The output fact identifies the correction, prior and replacement reviews, prior/corrected verdicts, source message, applied mode, exact state transition, policy identity, time and producer stream. It reports what Compliance committed, not what it intends to do later.

EXECUTABLE ASSET 40-L - APPLIED COMPENSATION FACT

```

public sealed record ComplianceReviewCorrectionAppliedV1(
    Guid CorrectionId,
    Guid PriorReviewId,
    Guid ReplacementReviewId,
    Guid SupersededVerdictId,
    Guid CorrectedVerdictId,
    Guid SourceCorrectionMessageId,
    string Mode,
    string PriorReviewStateBefore,
    string PriorReviewStateAfter,
    string ReplacementReviewState,
    string CompensationPolicyIdentity,
    DateTimeOffset AppliedAtUtc,
    ProducerStreamV1 ProducerStream);

```

STABLE FACT

The fact contains no review notes, reviewer identity, findings or private evidence body.

APPLIED CORPUS

The exact Compliance V1 bytes are frozen

The applied fact retains the original workflow correlation and names RootCauseVerdictCorrected MessageId as direct cause. The active-review fixture uses ActiveReviewSuperseded and a new Pending replacement review.

EXECUTABLE ASSET 40-1 - COMPLIANCE APPLIED FIXTURE

```
EventType    = "nexus1.compliance.compliance-review-correction-applied.v1"
MessageId    = "019bc4d4-b81e-75c2-9d63-5da4a5a64eb4"
Correlation  = "request-am-20482"
Causation    = "019bc4d4-b66c-7a7c-a5d4-f0e5386310b7"
Correction   = "019bc4d4-b4f1-7a28-b55f-508e7e88fb5c"
PriorReview  = "019bc4d4-8380-7b4a-9d80-12950da72d31"
Replacement  = "019bc4d4-b789-7c78-93cd-e6ba724df87d"
Fingerprint  = "5977ffff2eb9720c342baae7937d146af203fbd6ed0428e248b5de690fe40e5a"
BodySha256   = "8ccd91c2a7c1fa8fcf7c4b4ada729c2b222fbce603b9e0dc4d88068ca74fdd4b"
BodyLength   = 1206
```

SEPARATE LATE FIXTURE

The late-terminal corpus changes Mode and prior-state fields but preserves the same V1 grammar. It has its own exact body and digest.

REPEATED DELIVERY

The same message never creates a second replacement

A redelivered MessageId with the same retained body digest returns the stored semantic result and may be acknowledged. The handler does not rerun Domain policy or allocate another ReviewId.

DECISION LEDGER 40-25

Delivery	Disposition	Durable change
first valid message	Applied	one replacement + outbox
same id + same body	MatchingDuplicate	none
same id + different body	ConflictingDuplicate	quarantine
redelivery after publish	MatchingDuplicate	none

AT-LEAST-ONCE HONESTY

Idempotent local effects make redelivery safe. The system does not claim exactly-once transport.

SEMANTIC DUPLICATE

Different MessageIds can still name the same correction

If RootCause republishes the same CorrectionId under a new MessageId, Compliance compares the immutable semantic tuple. A matching tuple is recorded as a duplicate receipt without another effect; a conflicting tuple is quarantined.

EXECUTABLE ASSET 40-M - CORRECTION EQUIVALENCE

```
public bool SemanticallyMatches(RootCauseVerdictCorrectedV1 other) =>
    CorrectionId == other.CorrectionId &&
    RootCauseCaseId == other.RootCauseCaseId &&
    SupersededVerdictId == other.SupersededVerdictId &&
    CorrectedVerdictId == other.CorrectedVerdictId &&
    EvidenceSetDigest == other.EvidenceSetDigest &&
    CorrectionPolicyIdentity == other.CorrectionPolicyIdentity;
```

NOT CORRELATION DEDUPLICATION

CorrelationId groups a journey and is not unique. CorrectionId plus the semantic tuple owns correction idempotency.

LATE COMPENSATION TEST

Terminal history produces reassessment, not rewrite

The explicit late test completes the original Compliance review first, delivers the correction and asserts that the terminal row is byte-stable in its business fields while one new reassessment and one applied fact are created.

EXECUTABLE ASSET 40-N - LATE MESSAGE PROOF

```
var prior = await fixture.CompletedReviewAsync(ReviewState.Approved);
var before = prior.BusinessSnapshot();

await consumer.DeliverAsync(Fixtures.VerdictCorrected, ct);

var after = await db.Reviews.SingleAsync(x => x.Id == prior.Id, ct);
Assert.Equal(before, after.BusinessSnapshot());
var reassessment = await db.Reviews.SingleAsync(x =>
    x.SourceCorrectionId == Fixtures.CorrectionId, ct);
Assert.Equal(ReviewState.Pending, reassessment.State);
Assert.Equal(CompensationMode.ReassessmentAfterTerminal, applied.Mode);
```

REQUIRED ASSERTION

Testing only that a new row exists is insufficient. The old terminal decision must also be proven unchanged.

OUT-OF-ORDER CORRECTION

A correction may arrive before Compliance has created the prior review

RabbitMQ does not provide cross-queue global ordering. If Compliance receives the corrected-verdict fact before it has processed the original verdict, it cannot invent the missing prior review. It durably buffers the admitted correction and acknowledges only after that buffer commits.

DECISION LEDGER 40-28

Observation	Action
prior review exists	apply compensation now
prior verdict known but review absent	buffer by SupersededVerdictId
original review later commits	drain matching corrections
buffer expires policy bound	operator-visible unresolved evidence
correction conflicts	quarantine; do not drain

SAFE ACKNOWLEDGEMENT

An early correction may be acknowledged only after sufficient bytes and lineage are durably retained for deterministic drain.

PENDING CORRECTION BUFFER

The buffer is durable, bounded and evidence preserving

The row stores exact MessageId, CorrectionId, prior/corrected verdict IDs, admitted body digest, owner stream, arrival instant and disposition. It does not deserialize and discard fields needed for later admission comparison.

EXECUTABLE ASSET 40-O - PENDING CORRECTION ROW

```
public sealed record PendingVerdictCorrectionRow(  
    Guid MessageId,  
    Guid CorrectionId,  
    Guid SupersededVerdictId,  
    Guid CorrectedVerdictId,  
    string BodySha256,  
    string ProducerStreamId,  
    long ProducerPosition,  
    DateTimeOffset AdmittedAtUtc,  
    string State,  
    byte[] RowVersion);
```

RETENTION BOUNDARY

Expiry does not delete evidence automatically. It changes disposition and surfaces repair attention; Chapter 42 owns reconciliation policy.

DRAIN TRANSACTION

Creating the prior review can drain a pending correction atomically

After Compliance processes the original verdict and creates its first review, the same unit of work loads any exact pending correction, applies the proper mode and creates the applied-fact outbox. A concurrent drain is protected by rowversion and uniqueness.

EXECUTABLE ASSET 40-P - DETERMINISTIC DRAIN

```
var review = ComplianceReview.OpenFromVerdict(verdict, ids.NextReviewId(), now);
db.Reviews.Add(review);

var pending = await pendingCorrections
    .LoadBySupersededVerdictAsync(verdict.Id, ct);
if (pending is not null)
{
    var result = compensation.Apply(review, pending.ToAdmittedFact(),
                                    ids.NextReviewId(), now);
    pending.MarkApplied(result, now);
    outbox.Add(mapper.ToCorrectionApplied(result, pending));
}

await db.SaveChangesAsync(ct);
```

NO SECOND CONSUMER TRANSACTION

The original review and immediate correction effect become one ComplianceDb commit when the pending input already exists.

CRASH WINDOWS

Every crash recovers through durable owner-local evidence

The consumer can crash before commit, after commit before ack, while buffered, or after outbox dispatch before marking publication. Each window has one bounded recovery path and no deletion-based cleanup.

DECISION LEDGER 40-31

Crash point	Recovery
before Compliance commit	redelivery; no effect exists
after commit before ack	inbox/constraint duplicate
after buffer commit	redelivery duplicate; later drain
after publish before dispatcher mark	republish possible; consumers idempotent
during semantic retry	fresh unit of work re-evaluates

NO CLEANUP LIE

Recovery never deletes a prior verdict, review, receipt, buffer or public-fact record to make the retry appear first.

PROCESS MANAGER EXTENSION

A CorrectionCycle makes compensation progress explainable

The Process Manager observes RootCauseVerdictCorrected.v1 and creates one owner-local CorrectionCycle attached to the completed Alarm-to-Compliance workflow. It expects ComplianceReviewCorrectionApplied.v1 and reuses Chapter 39 deadlines/escalation if that fact is late.

DECISION LEDGER 40-32

Observed fact	CorrectionCycle action
VerdictCorrected.v1	Start; bind prior/corrected verdicts
ReviewCorrectionApplied.v1	Complete; bind prior/replacement reviews
duplicate corrected	matching duplicate
applied before corrected	durable early buffer
deadline elapsed	Overdue + WorkflowEscalation
late applied fact	CompletedLate; retain overdue history

AUTHORITY UNCHANGED

The cycle observes and expects facts. It never tells RootCause to correct or Compliance to replace a review.

CORRECTION IDENTITY

One CorrectionId owns one Process-Manager cycle

WorkflowId remains the parent journey identity while CorrectionId is the semantic idempotency key for the forward correction. CorrelationId remains searchable but not unique.

EXECUTABLE ASSET 40-Q - CORRECTION-CYCLE KEY

```
public sealed record CorrectionCycleIdentity(  
    Guid WorkflowId,  
    Guid CorrectionId,  
    Guid SourceCorrectionMessageId,  
    Guid SupersededVerdictId,  
    Guid CorrectedVerdictId);  
  
cycle.HasIndex(x => x.CorrectionId).IsUnique();  
cycle.HasIndex(x => x.SourceCorrectionMessageId).IsUnique();  
cycle.HasIndex(x => x.WorkflowId);
```

MATCHING RULE

The applied fact must match CorrectionId, prior/corrected verdict IDs and direct causation before it may complete the cycle.

CORRECTION CYCLE STATE

Compensation progress is separate from original completion

The original workflow remains Completed because Compliance did open the original review. The attached CorrectionCycle can be AwaitingCompliance, Overdue, Completed or CompletedLate without rewriting the original phase.

DECISION LEDGER 40-34

Original workflow	Correction cycle	Meaning
Completed	AwaitingCompliance	correction observed; effect pending
Completed	Overdue	applied fact late
Completed	Completed	forward effect observed
Completed	CompletedLate	effect observed after deadline

CHAPTER 41 HANDOFF

The BFF must later communicate both original completion and correction progress instead of collapsing them into one badge.

TOPOLOGY EXTENSION

Two fact types and one Compliance queue are added

ADR-040 changes the topology deliberately. RootCauseVerdictCorrected.v1 routes to the Compliance compensation queue and the Process-Manager queue. ComplianceReviewCorrectionApplied.v1 routes to the Process Manager. No command exchange or route is introduced.

DECISION LEDGER 40-35

Queue	New binding	Consumer
compliance.verdict-corrections.v1	root-cause-verdict-corrected.v1	Compliance
workflow.alarm-to-compliance.v1	root-cause-verdict-corrected.v1	Process Manager
workflow.alarm-to-compliance.v1	compliance-review-correction-applied.v1	Process Manager

PROFILE COUNT

The Process-Manager fact inventory grows from four to six. The public command inventory remains zero.

TOPOLOGY MANIFEST

The new routes are exact and versioned

A declarative manifest binds queue properties, routing keys, producer identities and consumers. Wildcards are forbidden because a future correction fact must not become an accidental workflow transition.

EXECUTABLE ASSET 40-R - CORRECTION TOPOLOGY

```
bindings:
- queue: compliance.verdict-corrections.v1
  route: root-cause.root-cause-verdict-corrected.v1
  producer: root-cause
- queue: workflow.alarm-to-compliance.v1
  route: root-cause.root-cause-verdict-corrected.v1
  producer: root-cause
- queue: workflow.alarm-to-compliance.v1
  route: compliance.compliance-review-correction-applied.v1
  producer: compliance
commandRoutes: []
```

PERMISSION DIFF

Compliance gains consume rights only on its correction queue. The Process Manager gains two consume bindings and no publish capability.

COMMAND GATE

ReplaceComplianceReview remains rejected

The public fact contains sufficient RootCause owner meaning, and Compliance owns the effect/refusal mode. A central ReplaceComplianceReview command would prescribe a target action, duplicate fact flow and expand Process-Manager authority without necessity.

DECISION LEDGER 40-37

ADR-038 gate	Result
target ownership	Compliance owns local consequence
necessity	blocked; owner fact is sufficient
refusal	handled as Compliance outcome, not command refusal
idempotency	fact inbox + CorrectionId
expiry	late fact has explicit forward mode
authorization	producer identity on fact route
evidence	PENDING

INVENTORY

Approved public commands: zero. Correction facts: two exact V1 contracts.

ARCHITECTURE GATES

Correction code preserves service dependency direction

RootCause references no Compliance implementation. Compliance consumes only the RootCause public contract. The Process Manager references both public contracts but no owner Domain or Infrastructure assembly.

DECISION LEDGER 40-38

Assembly	Allowed dependency	Forbidden dependency
RootCause	its Domain/Application/Infrastructure	Compliance / WorkflowDb
Compliance	public correction contract	RootCause Domain/Db
Process Manager	both public contracts	owner Domain/Db
Operator BFF	query contracts	direct database access

STATIC PROOF

Architecture tests inspect actual production assemblies and fail on owner implementation references or command publishers.

COMPOSITION GATES

Each host exposes only its owner-local compensation capability

Production descriptor tests inventory handlers, DbContexts, outbox/inbox services, workers and endpoints. The Process Manager gains two fact handlers; Compliance gains one fact consumer; RootCause gains one local correction handler.

EXECUTABLE ASSET 40-S - DESCRIPTOR INVENTORY

```
Assert.Contains(rootCause, Service<CorrectVerdictHandler>());
Assert.Contains(compliance,
    Service<IIntegrationFactHandler<RootCauseVerdictCorrectedV1>>());
Assert.Contains(processManager,
    Service<IWorkflowFactHandler<RootCauseVerdictCorrectedV1>>());
Assert.Contains(processManager,
    Service<IWorkflowFactHandler<ComplianceReviewCorrectionAppliedV1>>());

Assert.DoesNotContain(processManager, Service<IPublicCommandPublisher>());
Assert.DoesNotContain(compliance, Service<RootCauseDbContext>());
```

SAME-PATH REQUIREMENT

The tests call the production registration methods. A test-only container cannot prove deployment authority.

SECURITY BOUNDARY**Correction authority is narrower than operator attention**

Chapter 39 responders can acknowledge workflow attention but cannot correct a verdict. RootCause correction requires a separate owner authorization and validated evidence reference. Broker consumers authenticate producer service identities.

DECISION LEDGER 40-40

Capability	Authorized principal
acknowledge escalation	workflow escalation responder
submit new RootCause evidence	RootCause evidence contributor
correct verdict	RootCause correction policy/handler
apply review correction	Compliance service consumer
observe correction cycle	Process Manager consumer
delete history	no runtime principal

NO PRIVILEGE INHERITANCE

Workflow responder scope does not imply RootCause or Compliance mutation rights.

DATA MINIMIZATION**Public correction facts carry digests, not evidence bodies**

RootCause retains private evidence content. The public correction fact carries an evidence-set digest and policy identity sufficient for lineage. Compliance stores only the correction fields required to apply its review policy.

DECISION LEDGER 40-41

May cross boundary	Must remain owner-private
correction and verdict identities	raw evidence payload
reason code	analyst narrative
evidence-set digest	attachments and source secrets
policy identity	private scoring intermediates
timestamps/producer stream	operator personal data

DIGEST LIMIT

A digest supports identity and comparison. It does not prove evidence quality or authorize Compliance to interpret private evidence.

OBSERVABILITY

Original and corrective lineage remain connected but distinct

Logs and traces carry WorkflowId, CorrectionId, MessageId, CorrelationId, direct CausationId, prior/corrected VerdictIds, prior/replacement ReviewIds, mode and disposition. Metrics avoid all high-cardinality identities.

DECISION LEDGER 40-42

Signal	Low-cardinality dimensions
verdict_corrections_total	reason / result
review_corrections_total	mode / disposition
pending_corrections	state / age band
correction_cycles	phase / timeliness
compensation_conflicts_total	consumer / conflict kind
late_compensation_total	mode

TRACE CAUSALITY

Compliance applied-fact spans link to the correction delivery. They do not rewrite the original VerdictIssued or ReviewOpened causal chain.

PERFORMANCE

Correction is exceptional work, not the normal throughput path

The new queues and indexes must be measured under a workload that includes normal traffic, bursts of repeated corrections, late-terminal reviews and buffer drains. The chapter fixes no production volume or latency target.

DECISION LEDGER 40-43

Claim	Required measurement
duplicates are cheap	inbox/unique-hit latency
late mode remains bounded	terminal lookup + new review commit
buffer drains safely	drain rate and lock contention
normal path unaffected	baseline versus correction traffic
history remains queryable	index size/read plan over retention

EVIDENCE STATUS

Performance remains PENDING until retained workload, execution plans, logical reads and results exist.

FAILURE MATRIX

Forward compensation can fail and resume without rollback

Every failure leaves a durable point from which work can continue. The system may remain in a visible partial correction state, but it never removes previously committed truth to obtain consistency.

DECISION LEDGER 40-44

Failure	Durable state	Recovery
RootCause outbox delayed	corrected verdict committed	dispatcher retry
Compliance offline	correction queued	consumer recovery
correction before review	pending buffer	drain after review
Compliance commit/ack crash	applied effect + inbox	matching duplicate
Process Manager offline	owner facts queued	correction-cycle replay
applied fact late	cycle Overdue	CompletedLate

NO AUTOMATIC DELETE

A failed compensation opens retry, escalation or repair work. It does not trigger deletion of the original case, verdict or review.

ROOTCAUSE UNIT TESTS

Correction policy proves append-and-supersede invariants

Deterministic Domain tests cover successful correction, no new evidence, historical non-head verdict, repeated CorrectionId, concurrent head changes and a second correction that extends the chain.

DECISION LEDGER 40-45

Scenario	Assertion
new evidence/current verdict	new verdict; prior Superseded
same evidence digest	NoNewEvidence; no mutation
old non-head verdict	NotCurrent; no mutation
same CorrectionId	Duplicate; one correction
second correction	chain length +1; one current head
failed invariant	no outbox fact

NEGATIVE EFFECTS

Tests assert both the expected result and absence of unintended verdict, correction and outbox rows.

COMPLIANCE UNIT TESTS

Active, terminal and missing-review modes are explicit

The policy suite freezes the two applied modes and the buffer decision. It also proves that terminal business fields remain unchanged in the late path.

DECISION LEDGER 40-46

Prior state	Expected result
Pending	ActiveReviewSuperseded + replacement Pending
InReview	ActiveReviewSuperseded + replacement Pending
Approved	ReassessmentAfterTerminal; Approved unchanged
Rejected	ReassessmentAfterTerminal; Rejected unchanged
Closed	ReassessmentAfterTerminal; Closed unchanged
missing	BufferedUntilReviewExists

NO GENERIC FAILURE

Each legal state has a typed result. Unknown/corrupt states fail closed instead of falling through to a default compensation.

PROVIDER TESTS

Real databases prove atomicity, uniqueness and no deletion

Disposable SQL Server databases apply production migrations and run component tests for both services. They verify transaction rollback, unique CorrectionId, one current verdict, one replacement review, rowversion races and runtime DELETE denial.

DECISION LEDGER 40-47

Provider scenario	Required assertion
RootCause commit fault	no partial verdict/outbox
Compliance commit fault	no partial review/action/outbox
two correction writers	one current head
two Compliance consumers	one replacement
late terminal correction	terminal row unchanged
runtime DELETE	permission denied

PROVIDER BOUNDARY

InMemory and SQLite cannot prove SQL Server rowversion, filtered uniqueness, permissions or provider error classification.

CONTRACT TESTS**Golden bytes and negative corpora guard meaning**

Producer and both consumers use the same retained V1 bodies. Negative fixtures cover unknown versions, forged producers, invalid stream positions, self-supersession, conflicting CorrectionId and unsupported compensation mode.

DECISION LEDGER 40-48

Corpus	Expected
exact RootCause correction	accepted by Compliance + Process Manager
exact Compliance applied	accepted by Process Manager
unknown V2	rejected; no guessed mapping
tampered digest	quarantined
same id/conflicting body	conflict
late mode V1	accepted; terminal grammar retained

ZERO DISCOVERY

The suite asserts the expected positive and negative fixture counts before interpreting results.

BROKER TESTS**Routing, redelivery and confirms are component evidence**

A real RabbitMQ test environment declares the production topology, publishes exact bodies, forces consumer redelivery and publisher-confirm ambiguity, and verifies that correction effects remain idempotent.

DECISION LEDGER 40-49

Probe	Pass condition
publish corrected route	Compliance + Process queues receive
publish applied route	Process queue receives
unknown correction route	no accidental binding
redeliver corrected body	one Compliance effect
republish after confirm ambiguity	consumer duplicate
manager publish attempt	no command capability

RETAINED ENVIRONMENT

Evidence records RabbitMQ version, queue type, properties, permissions, commands, deliveries, confirms and artifact digests.

HOST TESTS

The production composition exposes revision 2 exactly

Production-path hosts inventory fact handlers, queues, databases, endpoints and forbidden capabilities. The Process Manager reports six fact bindings and zero commands under coordination profile revision 2.

EXECUTABLE ASSET 40-T - CAPABILITY MANIFEST TEST

```
var m = await demonstrator.GetCapabilityManifestAsync();

Assert.Equal("nexus1/coordination/observed-choreography/2",
    m.ProcessManager.CoordinationProfile);
Assert.Equal(6, m.ProcessManager.FactBindings.Count);
Assert.Empty(m.ProcessManager.CommandRoutes);
Assert.Contains("compliance.verdict-corrections.v1",
    m.Compliance.ConsumerQueues);
Assert.Empty(m.ProcessManager.OwnerMutationClients);
```

SAME-PATH EVIDENCE

The manifest is derived from production endpoint, DI and topology inventories, not maintained separately as a claim.

REPEATED AND LATE FAULT LAB

The required message cases run against the assembled runtime

The deployment lab exercises the exact repeated, semantic-duplicate, conflicting, out-of-order and late-terminal messages while injecting crashes at each local commit/ack boundary.

DECISION LEDGER 40-51

Scenario	Durable expected result
same body repeated 10x	one replacement and applied fact
new MessageId/same CorrectionId	semantic duplicate
same CorrectionId/conflict	quarantine; no second effect
correction before original review	buffer then one drain
correction after Approved	new reassessment; Approved unchanged
applied fact after cycle deadline	CompletedLate + history
crash after Compliance commit	redelivery duplicate

REQUIRED ASSERTION

The lab compares owner rows, inbox/outbox, buffers, workflow transitions and public bodies - not only test exit codes.

DEPLOYMENT**Correction facts roll out with consumers before producers**

Apply RootCause, Compliance and WorkflowDb migrations; deploy tolerant consumers and bindings; verify shadow admission; then activate the RootCause correction producer. This ordering prevents unroutable correction facts and unknown workflow semantics.

DECISION LEDGER 40-52

Stage	Change	Exit evidence
prepare	migrations + permissions	schema/principal digests
consume	Compliance and PM handlers	contract corpus
bind	three exact topology bindings	passive/routing probes
shadow	admission without owner reliance	comparison report
produce	RootCause correction enabled	end-to-end fault lab

NO DUAL SEMANTICS

Feature activation is bound to one contract/policy digest. Environments may not mix correction modes silently.

REVERSAL**A bad release is reversed without removing correction history**

Stop new correction initiation, preserve queues and database evidence, fix consumers/policy, replay or drain through the same idempotent paths and shadow-compare. Corrections already committed remain historical facts even if a later correction supersedes them.

DECISION LEDGER 40-53

Reversal action	Preserved truth
disable new RootCause corrections	committed corrected verdicts
pause consumers	queued messages
snapshot evidence	inbox/outbox/buffer/actions
deploy corrected code	contract identities
replay/drain	same semantic idempotency
issue later correction	full supersession chain

NO ROLLBACK FICTION

A software rollback changes which code runs next. It does not erase business actions already committed by the prior version.

EVIDENCE CATALOGUE

Every compensation claim maps to an owned proof surface

The catalogue distinguishes Domain eligibility, provider atomicity, contract meaning, broker delivery, topology, assembled-host authority and repeated/late failure behavior.

DECISION LEDGER 40-54

Claim	Proof surface	Minimum artifact
old verdict retained	RootCause provider test	before/after rows
terminal review unchanged	Compliance late test	business snapshots
one repeated effect	broker + provider	deliveries + row counts
out-of-order drain	provider race test	buffer/action sequence
two new facts	contract/topology	golden bodies + bindings
zero commands	DI/topology/security	capability manifests
cycle recovery	fault lab	timeline + hashes

NO SUBSTITUTE

An in-memory unit test cannot prove broker routing, SQL constraints, permissions or crash-after-commit recovery.

EVIDENCE MANIFEST

ADR-040 remains PENDING until exact artifacts exist

The dossier binds both ADRs, contract bodies, topology revision, migrations, provider/broker versions, commands, tests and artifact hashes. It rejects stale Chapter 39 topology evidence because revision 2 intentionally adds bindings.

EXECUTABLE ASSET 40-U - PROPOSED DOSSIER

```
{
  "evidenceId": "NEXUS1-FORWARD-COMPENSATION-040",
  "status": "PENDING",
  "adr038Sha256": "622787772a9def5ef18bffc6a622d6efc11185d0a696780445a3ee8cc027f22a",
  "adr040Sha256": "65eeede68f69f856376e89bbb01d79c7243a30e6778b7cacf0e57ef4178ea741",
  "coordinationProfile": "nexus1/coordination/observed-choreography/2",
  "factBindingCount": 6,
  "approvedPublicCommandCount": 0,
  "correctionBodySha256": "e22b868cb76c5278abc903f11afc4b11c2c8863d7c2f20b27f0a63ff84e60418",
  "appliedBodySha256": "8ccd91c2a7c1fa8fcf7c4b4ada729c2b222fbce603b9e0dc4d88068ca74fdd4b",
  "artifacts": []
}
```

TRUTHFUL STATE

The PDF specifies evidence but did not execute the repository/runtime suite. Status remains PENDING.

EVIDENCE VALIDATOR

The validator requires repeated and late suites explicitly

The dossier cannot become PROVED from only happy-path contract tests. It must include repeated transport delivery, semantic duplicate, conflict, out-of-order buffer/drain, late-terminal compensation and late Process-Manager completion.

EXECUTABLE ASSET 40-V - FAIL-CLOSED VALIDATION

```
Require(manifest.Status == "PROVED");
Require(manifest.FactBindings == 6);
Require(manifest.ApprovedPublicCommands == 0);
RequireExactSuites("domain", "sql-provider", "contracts", "broker",
    "repeated", "late-terminal", "out-of-order",
    "host", "fault");
Require(manifest.ExecutedTests > 0 && manifest.FailedTests == 0);
RequireSingleCommitAndCleanTree();
RequireMatchingAdrContractsTopologyAndMigrations();
RequireAllArtifactsExistAndMatchSha256();
```

ZERO-DISCOVERY GATE

Every suite reports discovered fixture/scenario counts. Missing repeated or late cases fail validation even when the test process exits zero.

CI REPRODUCTION

One target builds the forward-compensation dossier

The pipeline restores locked dependencies, builds Release, provisions isolated RootCauseDb, ComplianceDb, WorkflowDb and RabbitMQ resources, runs the bounded suites, validates the dossier and archives value-free artifacts.

EXECUTABLE ASSET 40-W - PROPOSED REPRODUCTION PATH

```
dotnet test tests/Nexus1.Compensation.EvidenceTests \
  --configuration Release \
  --no-restore \
  --filter "Category=Chapter40" \
  --logger "trx;LogFileName=chapter40.trx" \
  --results-directory artifacts/evidence/chapter40

dotnet run --project tools/EvidenceValidator -- \
  artifacts/evidence/chapter40/manifest.json
```

REQUIRED BINDING

Artifacts record repository commit, dirty flag, locks, .NET/SQL/RabbitMQ versions, migrations, contracts, topology, permissions, fault points and SHA-256 values.

CHAPTER 41 BOUNDARY

The operator now needs two timelines, not one simplified status

After compensation begins, the original workflow may be Completed while a CorrectionCycle is Pending, Delayed, Completed or Failed. Chapter 41 must communicate that uncertainty through the BFF and recover correctly after refresh.

DECISION LEDGER 40-58

Owner truth	Projection question for Chapter 41
original review opened	show original completion
corrected verdict observed	show correction pending
applied fact overdue	show delayed, not failed business decision
applied fact observed	show corrected completion
consumer/contract conflict	show failed projection with evidence
BFF refresh	recompose without losing prior timeline

PROTECTED HANDOFF

The UI may summarize, but it must preserve the difference between original completion, correction progress and projection freshness.

OFFICIAL TECHNICAL ANCHORS

Compensation is domain-specific, durable and repeatable

- Microsoft Compensating Transaction pattern: <https://learn.microsoft.com/azure/architecture/patterns/compensating-transaction>
- Microsoft Saga pattern: <https://learn.microsoft.com/azure/architecture/patterns/saga>
- Microsoft Event Sourcing pattern: <https://learn.microsoft.com/azure/architecture/patterns/event-sourcing>
- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- EF Core optimistic concurrency: <https://learn.microsoft.com/ef/core/saving/concurrency>
- RabbitMQ acknowledgements and confirms: <https://www.rabbitmq.com/docs/confirms>
- RabbitMQ queues and ordering: <https://www.rabbitmq.com/docs/queues>
- RabbitMQ reliability guide: <https://www.rabbitmq.com/docs/reliability>
- RFC 8785 JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785>

WHAT THIS CHAPTER HAS PROVED

Chapter 40 specifies ADR-040; a concrete RootCause verdict-correction fact; active-review replacement and late-terminal reassessment modes in Compliance; atomic inbox/owner-history/outbox commits; exact V1 corpora; transport and semantic idempotency; durable out-of-order buffering; crash recovery; an observed CorrectionCycle with Chapter 39 deadlines; a six-fact, zero-command topology; security, deployment and fail-closed evidence paths. Runtime evidence remains PENDING.

CHAPTER CHECKPOINT

- Why can compensation not restore the exact original state?
- Which service owns verdict correction and which owns review consequences?
- How does an active-review correction differ from a late terminal correction?
- Which rows and facts remain visible after compensation?
- How are repeated MessageIds and repeated CorrectionIds distinguished?
- When may an out-of-order correction delivery be acknowledged?
- Why does the Process Manager open a CorrectionCycle but issue no command?
- Why does the fact inventory grow to six while commands remain zero?
- Which repeated and late artifacts are mandatory before status becomes PROVED?

NEXT - CHAPTER 41

Eventual Consistency in the Operator Experience will represent original completion and correction progress as separate pending, delayed, completed and failed projection states. The BFF will communicate freshness and uncertainty and the UI will recover those states after refresh.

PART VII - WORKFLOW, CONSISTENCY, AND REPAIR

41. Eventual Consistency in the Operator Experience

The console makes partial progress visible without inventing a global present. Original and correction cycles keep separate Pending, Delayed, Completed or Failed projection states, and a browser refresh reconstructs them through one authorized BFF GET.

CHAPTER DECISION

ADR-041 makes uncertainty part of the V1 operator contract. Reporting derives four closed projection states from retained fact slots, immutable expectations and durable failure evidence. The BFF preserves business, projection, freshness and transport as separate axes. The Angular client performs bounded GET refresh, rejects stale responses and rebuilds from the route after a hard reload. No refresh issues a command or edits owner truth.

PROOF PATH 41-A - DELAYED TRUTH TO RECOVERABLE EXPERIENCE



GUARANTEE LEDGER 41-A

Specified	Explicitly rejected
Pending/Delayed/Completed/Failed union	null/default success
original and correction timelines	one simplified status
server-owned time classification	browser clock authority
hard-refresh recovery	localStorage owner truth
GET-only observation	refresh or BFF command replay

CHAPTER BOUNDARY

Eventual consistency becomes an operator contract

Chapter 35 made Reporting delay measurable. Chapter 22 made the BFF an experience boundary. Chapters 38-40 added a durable original workflow and an independent correction cycle. Chapter 41 combines those decisions into one honest operator view without moving business ownership into Reporting, the BFF or the browser.

DECISION LEDGER 41-2

Inherited boundary	Chapter 41 extension	Still forbidden
Reporting owns delayed copies	cycle projection states	Reporting decides verdict/review
BFF owns UI composition	versioned experience contract	BFF becomes workflow engine
Process Manager observes	separate original/correction progress	operator forces completion
owners publish facts	refresh observes newer evidence	refresh repeats commands

SCOPE

The chapter changes how uncertainty is represented and recovered. It does not change who can make a domain decision.

INHERITED GUARANTEES

Six facts remain owner truth; the console remains a reader

ADR-038 selected observed choreography, ADR-040 added two correction facts, and the approved public command set stayed empty. ADR-041 consumes that fact profile into an operator projection. It does not add a seventh fact merely to announce that the UI refreshed.

DECISION LEDGER 41-3

Coordinate	Authoritative owner	Operator use
four normal-path facts	AlarmManagement/RootCause/Compliance	original timeline
two correction facts	RootCause/Compliance	correction timeline
deadline/policy identity	Reporting projection policy	pending versus delayed
projection failure	Reporting	safe failed state
refresh request	BFF Edge	new observation only

TOPOLOGY INVARIANT

Process Manager remains six facts and zero public commands. ADR-041 changes a read contract, not coordination authority.

TWO TIMELINES

Original completion and correction progress coexist

A completed original cycle does not become incomplete when a correction begins. The operator sees the original sequence as retained history and a second correction sequence with its own progress. This is the UI counterpart of Chapter 40's append-and-supersede rule.

DECISION LEDGER 41-4

Timeline	Possible current projection state	Meaning
original	Completed	four required facts observed
correction	Pending	corrected verdict observed; application expected
correction	Delayed	application fact overdue; outcome unknown
correction	Completed	application fact projected
correction	Failed	projection cannot interpret safely

NO SIMPLIFIED STATUS

The headline may summarize, but the response retains both cycles and their separate evidence.

FOUR INDEPENDENT AXES

One badge cannot carry four kinds of truth

The operator experience separates business outcome, projection progress, freshness and transport. A Compliance review can be Approved while its correction projection is Delayed; the BFF can be temporarily unavailable while the stored projection remains Completed.

DECISION LEDGER 41-5

Axis	Examples	Question answered
business outcome	Pending/Approved/Declined/Superseded	what owner decided
projection progress	Pending/Delayed/Completed/Failed	what this view assembled
freshness	current-as-observed/stale/unknown	quality of observation
transport	loading/ready/unavailable/denied	can this request return

TYPE-SYSTEM RULE

No mapper may derive one axis by parsing another axis's display label.

PROJECTION VOCABULARY

The four states have closed and testable meanings

Pending and Delayed describe incomplete evidence at different times. Completed describes a satisfied required-fact set. Failed describes a durable projection defect that prevents safe interpretation. None is a synonym for a domain rejection.

DECISION LEDGER 41-6

State	Required evidence	Operator meaning
Pending	missing required fact; before ExpectedByUtc	work may still converge normally
Delayed	missing required fact; deadline elapsed	late, not failed
Completed	all cycle-required facts applied	projection complete for stated version
Failed	active blocking failure record	view cannot advance safely

CLOSED UNION

Unknown JSON enum values fail contract validation. They do not fall back to Pending or Completed.

STATE MACHINE

A cycle never moves backward to make history fit

Each cycle has its own monotonic evidence set. Time can expose delay; facts can complete; a blocking projection defect can fail. A later repair is a new audited projection generation owned by Chapter 42, not an unrecorded Failed-to-Completed edit.

EXECUTABLE ASSET 41-A - CLOSED TRANSITION MODEL

```
public enum ProjectionStateV1 { Pending, Delayed, Completed, Failed }

public static readonly IReadOnlyDictionary<ProjectionStateV1,
    ProjectionStateV1[]> Allowed = new Dictionary<ProjectionStateV1,
    ProjectionStateV1[]>
{
    [ProjectionStateV1.Pending] =
        [ProjectionStateV1.Delayed, ProjectionStateV1.Completed,
        ProjectionStateV1.Failed],
    [ProjectionStateV1.Delayed] =
        [ProjectionStateV1.Completed, ProjectionStateV1.Failed],
    [ProjectionStateV1.Completed] = [],
    [ProjectionStateV1.Failed] = []
};
```

REPAIR BOUNDARY

Chapter 42 may create a new generation from retained inputs. It may not mutate the terminal history of this generation.

CLASSIFICATION PRECEDENCE**Blocking evidence wins before optimistic interpretation**

The classifier is fail closed. An active blocking failure wins. Otherwise a satisfied fact set completes. Only incomplete cycles are classified by the policy deadline. This order prevents a clock from hiding a reducer failure or a completed set from being relabeled merely because time passed.

DECISION LEDGER 41-8

Precedence	Predicate	Result
1	blocking failure attached to cycle/generation	Failed
2	required fact set satisfied	Completed
3	incomplete and now $\geq$ ExpectedByUtc	Delayed
4	incomplete and before ExpectedByUtc	Pending

BUSINESS ISOLATION

A Declined review remains a valid owner outcome. It does not create ProjectionState.Failed.

TIME SEMANTICS**Expected time is policy evidence, not a promise of completion**

ExpectedByUtc is frozen when a cycle begins from one versioned projection policy. The query samples one ObservedAtUtc through TimeProvider and derives Pending or Delayed. It never extends the deadline on refresh and never uses the browser clock.

DECISION LEDGER 41-9

Time	Owner	Use
source OccurredAtUtc	fact producer	timeline evidence
LastAppliedAtUtc	Reporting transaction	projection progress
ExpectedByUtc	frozen projection policy	pending/delayed boundary
ObservedAtUtc	BFF query clock	one classification instant
browser time	device	display countdown only

CLOCK RULE

Client clock skew can change a countdown label, never the projection state returned by the server.

ADR-041**Uncertainty is part of the public experience contract**

ADR-041 accepts a four-state projection union, separate original and correction cycles, server-owned classification and GET-only rehydration. It explicitly rejects UI inference from missing fields, automatic command retry and browser-persisted owner truth.

DECISION LEDGER 41-10

ADR field	Decision
states	Pending, Delayed, Completed, Failed
axes	business, projection, freshness, transport
refresh	authorized GET reconstructs representation
history	original and correction cycles retained
commands	empty public command addition
evidence	PENDING

ADR LINEAGE

ADR-041 extends ADR-040 and retains coordination profile `nexus1/coordination/observed-choreography/2`.

ADR IDENTITY**The experience decision is a canonical artifact**

The ADR digest binds the state vocabulary, axis separation, refresh rule and zero-command decision. Contract, UI and evidence validators compare the same body instead of relying on prose equivalence.

EXECUTABLE ASSET 41-B - ADR-041 IDENTITY

```
const string AdrId = "ADR-041";
const string ProjectionPolicy = "reporting/operator-workflow-experience/1";
const string CoordinationProfile = "nexus1/coordination/observed-choreography/2";
const string AdrSha256 = "0e8d710a99dcdb8f2edc01322520322922647803b2e57e324b4a218463689a8";
```

```
ProjectionStateV1[] states =
[
    ProjectionStateV1.Pending,
    ProjectionStateV1.Delayed,
    ProjectionStateV1.Completed,
    ProjectionStateV1.Failed
];

string[] approvedPublicCommands = [];
```

CANONICAL BODY

ADR-041 is 539 UTF-8 bytes with SHA-256

`0e8d710a99dcdb8f2edc01322520322922647803b2e57e324b4a218463689a8`.

INPUT CONTRACT

The projection consumes the six already-approved facts

The experience projection does not inspect owner databases. It translates the normal four-fact story and the two correction facts. Every slot retains MessageId, body digest, producer, occurrence time and owner identifier.

DECISION LEDGER 41-12

Cycle	Required fact	Projection contribution
original	AlarmFloodDetected.v1	seed experience
original	RootCauseCaseOpened.v1	case timeline
original	RootCauseVerdictIssued.v1	verdict timeline
original	ComplianceReviewOpened.v1	complete original
correction	RootCauseVerdictCorrected.v1	seed correction
correction	ComplianceReviewCorrectionApplied.v1	complete correction

NO SYNTHETIC COMPLETION

A Process-Manager row, operator acknowledgement or low queue depth cannot substitute for a missing owner fact.

SUBSCRIPTION ROUTES

Reporting receives independent copies without joining owners

The existing Reporting integration queue keeps explicit contract admission. Its RootCause binding already covers case, verdict and correction facts; an exact Compliance binding is added for the two approved review facts. AlarmManagement remains an exact binding. These are Reporting routes, not Process-Manager commands.

DECISION LEDGER 41-13

Binding	Accepted contracts	Failure isolation
alarm-management.alarm-flood-detected.v1	FloodDetected	Reporting backlog
root-cause.#	CaseOpened/VerdictIssued/VerdictCorrected	Reporting backlog
compliance.review-opened.v1	ReviewOpened	Reporting backlog
compliance.review-correction-applied.v1	CorrectionApplied	Reporting backlog

ROUTING IS NOT ADMISSION

The queue binding may deliver a message, but only the versioned contract set may translate it into projection work.

PROJECTION CONTRACT SET

Six descriptors freeze translation behavior

Every input has an exact type, schema and mapper identity. Adding a schema or changing a reducer advances the contract-set identity and requires a rebuild decision; it is not an in-place parser tweak.

EXECUTABLE ASSET 41-C - EXPLICIT CONTRACT SET

```
public static readonly ProjectionContract[] Contracts =
[
    V1("alarm-flood-detected", "AlarmFloodDetectedMapper/1"),
    V1("root-cause-case-opened", "CaseOpenedMapper/1"),
    V1("root-cause-verdict-issued", "VerdictIssuedMapper/1"),
    V1("compliance-review-opened", "ReviewOpenedMapper/1"),
    V1("root-cause-verdict-corrected", "VerdictCorrectedMapper/1"),
    V1("compliance-review-correction-applied",
        "ReviewCorrectionAppliedMapper/1")
];

public const string Identity =
    "reporting/operator-workflow-experience/1";
```

DIGEST GATE

The ordered descriptor list, reducer identity and deadline policy are hashed into the projection generation manifest.

ADMISSION

Projection work starts only after envelope proof

Reporting copies the delivery body, validates broker principal and route, verifies envelope identity and fingerprint, selects an exact contract descriptor and then translates. Unsupported or conflicting meaning is durable failure evidence; transient database unavailability is not.

DECISION LEDGER 41-15

Gate	Failure class	Projection effect
principal/route	unauthorized	reject + security evidence
envelope/fingerprint	identity conflict	quarantine + Failed
contract/schema	unsupported	quarantine + Failed
payload invariants	semantic invalid	failure row + Failed
provider unavailable	transient	no ack; remain Pending/Delayed

FAILURE DISCIPLINE

ProjectionState.Failed requires durable classified evidence. An exception or timeout alone does not earn that state.

REPORTING-OWNED EXPERIENCE ROW

The row is a delayed view with explicit generation

One experience row identifies the correlation and active generation. It stores representation version and latest applied coordinates, but not a mutable universal status. Each original or correction cycle owns its own progress record.

EXECUTABLE ASSET 41-D - EXPERIENCE ROW

```
internal sealed class OperatorWorkflowExperienceRow
{
    public Guid ExperienceId { get; init; }
    public string WorkflowId { get; init; } = null!;
    public string CorrelationId { get; init; } = null!;
    public Guid ProjectionGenerationId { get; init; }
    public long RepresentationVersion { get; set; }
    public DateTimeOffset LastAppliedAtUtc { get; set; }
    public Guid LastAppliedMessageId { get; set; }
    public byte[] RowVersion { get; set; } = null!;
    public List<OperatorCycleRow> Cycles { get; } = [];
}
```

READ-MODEL LIMIT

This row may drive the console. It may not authorize owner commands or prove source databases current.

CYCLE ROW

Original and correction cycles carry independent evidence

A cycle freezes its required evidence mask, policy and expected time at creation. CompletedAtUtc is Reporting's completion observation; it is not the time the distributed business world became globally atomic.

EXECUTABLE ASSET 41-E - CYCLE RECORD

```
internal sealed class OperatorCycleRow
{
    public Guid ProjectionGenerationId { get; init; }
    public Guid ExperienceId { get; init; }
    public string CycleId { get; init; } = null!;
    public string CycleKind { get; init; } = null!; // Original | Correction
    public int RequiredMask { get; init; }
    public int ObservedMask { get; set; }
    public string PolicyIdentity { get; init; } = null!;
    public DateTimeOffset StartedAtUtc { get; init; }
    public DateTimeOffset ExpectedByUtc { get; init; }
    public DateTimeOffset? CompletedAtUtc { get; set; }
    public long CycleVersion { get; set; }
}
```

NO DEADLINE REFRESH

ExpectedByUtc is immutable. A browser refresh cannot move it into the future.

EVIDENCE SLOTS

Fact identity is retained per required slot

A bit mask is convenient for classification but insufficient for audit. Each observed slot retains the exact producer fact coordinates. Duplicate delivery resolves against this record; a different body under the same semantic slot becomes conflict evidence.

EXECUTABLE ASSET 41-F - DURABLE SLOT

```
internal sealed class ProjectionEvidenceSlotRow
{
    public Guid ProjectionGenerationId { get; init; }
    public string CycleId { get; init; } = null!;
    public string Slot { get; init; } = null!;
    public Guid MessageId { get; init; }
    public byte[] EnvelopeSha256 { get; init; } = null!;
    public string Producer { get; init; } = null!;
    public string EventType { get; init; } = null!;
    public DateTimeOffset OccurredAtUtc { get; init; }
    public DateTimeOffset AppliedAtUtc { get; init; }
}
```

SEMANTIC KEY

ProjectionGenerationId + CycleId + Slot is unique; MessageId remains separately unique per consumer receipt.

FAILURE EVIDENCE

Failed is backed by a safe, durable blocker

A failure row records which admitted input and reducer/generation could not be interpreted safely. Public output exposes only a reviewed safe code and evidence reference. Raw payload, stack, queue and database detail remain protected.

EXECUTABLE ASSET 41-G - BLOCKING FAILURE ROW

```
internal sealed class ProjectionFailureRow
{
    public Guid FailureId { get; init; }
    public Guid ProjectionGenerationId { get; init; }
    public string CycleId { get; init; } = null!;
    public Guid? SourceMessageId { get; init; }
    public string SafeCode { get; init; } = null!;
    public string InternalClassification { get; init; } = null!;
    public byte[]? BodySha256 { get; init; }
    public DateTimeOffset RecordedAtUtc { get; init; }
    public DateTimeOffset? ResolvedAtUtc { get; set; }
    public string EvidenceRef { get; init; } = null!;
}
```

TERMINAL FOR ONE GENERATION

Resolution is performed by an audited Chapter 42 repair generation; the original failure evidence remains retained.

SQL CONSTRAINTS

The provider arbitrates duplicate and conflicting races

Named constraints make race outcomes classifiable. The transaction cannot create two cycles for one correction, two bodies for one evidence slot or two active failure meanings for the same input.

EXECUTABLE ASSET 41-H - CONSTRAINT CORE

```
CREATE UNIQUE INDEX UX_Experience_Generation_Workflow
ON reporting.OperatorWorkflowExperience
(ProjectionGenerationId, WorkflowId);

CREATE UNIQUE INDEX UX_Cycle_Generation_Kind_Id
ON reporting.OperatorCycle
(ProjectionGenerationId, CycleKind, CycleId);

CREATE UNIQUE INDEX UX_Evidence_Generation_Cycle_Slot
ON reporting.ProjectionEvidenceSlot
(ProjectionGenerationId, CycleId, Slot);

CREATE UNIQUE INDEX UX_Failure_Generation_Message
ON reporting.ProjectionFailure
(ProjectionGenerationId, SourceMessageId)
WHERE SourceMessageId IS NOT NULL;
```

NO LAST-WRITE WINS

A constraint violation is classified by exact provider code and constraint name, then resolved by a fresh read.

EF CORE MAPPING

Relationships preserve history and forbid cascaded erasure

The experience is the aggregate of projection rows only. Evidence and failure records use restrictive deletion. Rowversion protects concurrent reducers, while generation and semantic keys protect replay.

EXECUTABLE ASSET 41-I - MAPPING GATES

```
experience.ToTable("OperatorWorkflowExperience", "reporting");
experience.HasKey(x => x.ExperienceId);
experience.Property(x => x.RowVersion).IsRowVersion();

cycle.ToTable("OperatorCycle", "reporting");
cycle.HasKey(x => new { x.ProjectionGenerationId,
    x.ExperienceId, x.CycleId });
cycle.HasOne<OperatorWorkflowExperienceRow>()
    .WithMany(x => x.Cycles).HasForeignKey(x => x.ExperienceId)
    .OnDelete(DeleteBehavior.Restrict);

failure.ToTable("ProjectionFailure", "reporting");
failure.Property(x => x.SafeCode).HasMaxLength(80);
```

PROVIDER EVIDENCE

SQLite and EF InMemory cannot prove SQL Server rowversion, filtered uniqueness or provider error classification.

ATOMIC REDUCER COMMIT

Receipt, evidence, projection and version move together

One ReportingDb transaction stores the inbox outcome, immutable ledger entry, evidence slot, cycle update, experience version and checkpoint. Broker acknowledgement follows the known commit outcome.

DECISION LEDGER 41-22

TX-UX addition	Count	Atomic condition
InboxReceipt	1	same delivery body
ProjectionEventLedger	1	immutable translated input
EvidenceSlot	0 or 1	semantic slot first writer
Cycle/Experience update	0 or 1	monotonic version
Checkpoint	1	contiguous stream application

ACK RULE

Ack only after committed, duplicate-confirmed or durably quarantined outcome. Never ack an unknown database result.

ORIGINAL-CYCLE REDUCER

Four facts satisfy four named slots

The normal cycle begins with AlarmFloodDetected and advances only through exact fact identities. ReviewOpened completes the experience projection of the original path; it does not assert the review's later business outcome.

EXECUTABLE ASSET 41-J - ORIGINAL EVIDENCE MASK

```
[Flags]
public enum OriginalEvidence
{
    None = 0,
    FloodDetected = 1,
    CaseOpened = 2,
    VerdictIssued = 4,
    ReviewOpened = 8,
    Required = FloodDetected | CaseOpened | VerdictIssued | ReviewOpened
}

cycle.Observe(slot, delivery.MessageId, delivery.BodySha256);
if ((cycle.ObservedMask & (int)OriginalEvidence.Required) ==
    (int)OriginalEvidence.Required)
    cycle.MarkCompleted(appliedAtUtc);
```

COMPLETION MEANING

Completed means the original four-fact projection is assembled, not that all future corrections are impossible.

CORRECTION-CYCLE REDUCER

Two facts create a new progress line

VerdictCorrected creates or resolves the Correction cycle with an immutable ExpectedByUtc. ReviewCorrectionApplied satisfies the second slot and completes that cycle. The original cycle row remains unchanged.

EXECUTABLE ASSET 41-K - CORRECTION REDUCER

```
[Flags]
public enum CorrectionEvidence
{
    None = 0,
    VerdictCorrected = 1,
    ReviewCorrectionApplied = 2,
    Required = VerdictCorrected | ReviewCorrectionApplied
}

var cycle = await experience.GetOrStartCorrectionAsync(
    correction.CorrectionId,
    startedAtUtc: correction.CorrectedAtUtc,
    expectedByUtc: policy.ExpectedBy(correction.CorrectedAtUtc, ct);
cycle.Observe(CorrectionEvidence.VerdictCorrected, correction.MessageId);
```

AUTHORITY PRESERVED

Reporting records that Compliance applied a correction; it does not apply or approve the Compliance action itself.

APPLIED-CORRECTION REDUCER

Completion preserves mode and review lineage

The applied fact carries active-replacement or late-reassessment mode from Compliance. Reporting copies that public meaning into the timeline and closes only the matching CorrectionId.

EXECUTABLE ASSET 41-L - APPLY SECOND SLOT

```
var cycle = experience.RequireCorrection(applied.CorrectionId);
cycle.DemandExpectedVerdicts(
    applied.SupersededVerdictId,
    applied.CorrectedVerdictId);
cycle.Observe(
    CorrectionEvidence.ReviewCorrectionApplied,
    applied.MessageId,
    applied.BodySha256);

experience.AddTimelineEntry(new(
    Kind: "compliance-correction-applied",
    Owner: "Compliance",
    Mode: applied.Mode,
    OccurredAtUtc: applied.OccurredAtUtc));
```

LATE REMAINS VISIBLE

LateTerminalReassessment and ActiveReviewSuperseded are not collapsed into the same display sentence.

SEMANTIC IDEMPOTENCY

Repeated delivery cannot create a second progress effect

Transport duplicate, semantic duplicate and identity conflict remain distinct. A repeated MessageId returns the stored receipt. A new MessageId for the same slot and same body meaning is a semantic duplicate. Different meaning for the same slot fails closed.

DECISION LEDGER 41-26

Input	Outcome	Mutation
same MessageId/body	transport duplicate	none
new MessageId/same slot/same semantic digest	semantic duplicate	receipt only
same MessageId/different body	identity conflict	failure/quarantine
same slot/different owner identity	semantic conflict	failure/quarantine

VERSION INVARIANT

RepresentationVersion advances only when visible projection meaning changes.

OUT-OF-ORDER DELIVERY

A later slot waits; it does not invent its predecessor

ComplianceReviewCorrectionApplied may arrive before VerdictCorrected on a different producer stream. Reporting retains the admitted event in a bounded gap buffer keyed by CorrectionId and drains it after the first slot is committed.

DECISION LEDGER 41-27

Arrival	Durable action	Ack
Applied before Corrected	buffer exact admitted event	after buffer commit
Corrected arrives	create cycle + first slot	after projection commit
drain buffered Applied	second slot + complete	no broker ack involved
buffer policy expires	operator-visible unresolved evidence	Chapter 42 candidate

NO CROSS-STREAM ORDERING CLAIM

The buffer proves local recovery from reordering. It does not claim RabbitMQ globally orders RootCause and Compliance streams.

CRASH RECOVERY**Redelivery repeats admission, not the visible effect**

A crash before commit leaves no projection effect and the message is redelivered. A crash after commit but before ack resolves the inbox receipt and evidence slot. A crash during buffer drain resumes from durable buffer state.

DECISION LEDGER 41-28

Fault point	After restart	Expected count
before TX-UX commit	redeliver and commit	one effect
after commit/before ack	receipt duplicate	one effect
after first slot/before drain	drain buffer	one completion
after failure row/before ack	quarantine duplicate	one failure

NO EXACTLY-ONCE CLAIM

Delivery may repeat; the Reporting effect and representation version remain idempotent.

EXPECTED-BY POLICY**The cycle freezes one named latency expectation**

The deadline is a user-experience expectation derived from a versioned policy, not an estimate recalculated by the browser. Different cycle kinds may use different windows, but every response exposes the exact deadline and policy identity.

EXECUTABLE ASSET 41-M - IMMUTABLE EXPECTATION

```
public sealed record ProjectionExpectationPolicy(  
    string Identity,  
    TimeSpan OriginalWindow,  
    TimeSpan CorrectionWindow,  
    TimeSpan PendingRefresh,  
    TimeSpan DelayedRefresh);  
  
DateTimeOffset ExpectedBy(  
    CycleKind kind, DateTimeOffset startedAtUtc) =>  
    startedAtUtc + (kind == CycleKind.Original  
        ? policy.OriginalWindow  
        : policy.CorrectionWindow);
```

NO SLO INVENTION

Candidate durations remain policy proposals until measured runtime evidence accepts them.

SERVER CLASSIFIER

One pure function derives the four-state union

The classifier receives the frozen cycle row, active failure and one server observation time. It returns state, stable reason code and missing slots. The browser never reimplements this decision.

EXECUTABLE ASSET 41-N - FAIL-CLOSED CLASSIFIER

```
public ProjectionProgress Classify(
    OperatorCycle cycle,
    ProjectionFailure? blocker,
    DateTimeOffset observedAtUtc)
{
    if (blocker is not null)
        return Failed(blocker.SafeCode, blocker.EvidenceRef);
    if (cycle.HasAllRequiredEvidence)
        return Completed(cycle.CompletedAtUtc!.Value);

    var missing = cycle.MissingSlots();
    return observedAtUtc >= cycle.ExpectedByUtc
        ? Delayed("required-fact-overdue", missing, cycle.ExpectedByUtc)
        : Pending("awaiting-required-fact", missing, cycle.ExpectedByUtc);
}
```

DETERMINISTIC CLOCK

Tests inject FakeTimeProvider and prove the exact boundary at ExpectedByUtc.

PENDING

Pending says evidence is incomplete but still inside policy

The UI names the missing owner fact, the time at which delay begins and the last projected event. It does not show a spinner without meaning and does not claim that the missing business action has failed.

DECISION LEDGER 41-31

Field	Pending value	Forbidden inference
reasonCode	awaiting-required-fact	consumer crashed
missingFacts	stable public fact names	owner rejected
expectedByUtc	frozen server time	guaranteed completion
refresh	bounded automatic + manual GET	repeat command

OPERATOR WORDING

Waiting for Compliance observation. No failure has been reported.

DELAYED

Delayed preserves uncertainty after the expectation expires

Delayed is earned only when required evidence is still absent at or after ExpectedByUtc. The representation may still be fresh as an observation: Reporting can be completely current about the fact that the expected owner fact has not arrived.

DECISION LEDGER 41-32

Axis	Delayed fixture	Meaning
projection progress	Delayed	required fact overdue
freshness	current-as-observed	status evidence is recent
business outcome	unknown	Compliance decision not inferred
transport	ready	BFF request succeeded

CRITICAL DISTINCTION

Delayed projection is not Failed business outcome and not Unavailable transport.

COMPLETED

Completed is bounded by cycle, generation and fact set

A completed cycle lists every required evidence slot and the representation version that included it. It remains a Reporting observation, not a global snapshot. A later correction creates a new cycle rather than reopening the original one.

DECISION LEDGER 41-33

Completed claim	Bound	Not claimed
required slots present	cycle + generation	all future facts known
timeline rendered	representation version	owner databases simultaneous
original retained	immutable cycle history	correction impossible
auto refresh may slow	server advice	screen is authoritative forever

VISIBLE LINEAGE

Completion keeps source MessageIds and occurrence times available to authorized evidence views, even when the standard UI displays friendly labels.

FAILED**Failed names a projection defect, not an owner verdict**

Failed requires an active durable blocker such as an unsupported contract, identity conflict, invalid sequence or reducer invariant violation. Transient SQL or HTTP failure keeps transport unavailable or progress delayed; it does not manufacture a terminal projection diagnosis.

DECISION LEDGER 41-34

Condition	Projection state	Business outcome
unsupported schema	Failed	unchanged/unknown
same MessageId changed bytes	Failed	unchanged/unknown
ReportingDb temporarily offline	not reclassified	unchanged
Compliance Declined	Completed projection	Declined

OPERATOR ACTION

Show a safe evidence reference and support path. Never invite the operator to edit owner truth or force projection completion.

CYCLE COMPOSITION**A correction never overwrites the original headline evidence**

The BFF maps two cycle snapshots. If no correction fact has been observed, correctionCycle is absent with an explicit not-started relationship; once started, its state is exhaustive. The original cycle's Completed state and history remain visible.

DECISION LEDGER 41-35

Original	Correction	Headline
Completed	absent	Original workflow completed
Completed	Pending	Correction in progress
Completed	Delayed	Correction delayed
Completed	Completed	Correction applied
Completed	Failed	Correction projection failed

NO WORST-BADGE COLLAPSE

The headline does not erase original completion merely because the correction projection is delayed or failed.

GOLDEN DELAYED VIEW

One exact fixture proves the difficult mixed state

The canonical fixture intentionally combines an original Completed cycle, a correction Delayed cycle, current-as-observed freshness and ready transport. This prevents test suites from passing only the easy all-green case.

EXECUTABLE ASSET 41-O - READABLE DELAYED FIXTURE

```
{
  "experienceId": "019bc4d4-c1b0-7e4e-ae6c-4e3af2e7b331",
  "workflowId": "019bc4d4-61a8-76f1-a935-b953af1dd9c4",
  "correlationId": "request-am-20482",
  "representationVersion": 17,
  "projectionGenerationId": "019bc4d4-c21a-7527-8078-5be0476e77bf",
  "observedAtUtc": "2026-07-19T08:17:11.000Z",
  "originalCycle": {
    "cycleId": "019bc4d4-61a8-76f1-a935-b953af1dd9c4",
    "projectionState": "completed",
    "reasonCode": "required-facts-observed",
    "completedAtUtc": "2026-07-19T07:46:04.012Z",
    "missingFacts": []
  },
  "correctionCycle": {
    "cycleId": "019bc4d4-b4f1-7a28-b55f-508e7e88fb5c",
    "projectionState": "delayed",
    "reasonCode": "correction-application-overdue",
    "expectedByUtc": "2026-07-19T08:16:00.000Z",
    "missingFacts": [
      "ComplianceReviewCorrectionApplied.v1"
    ]
  },
  "freshness": {
    "updateStatus": "current-as-observed",
    "projectionName": "operator-workflow-experience",
    "contractSetIdentity": "reporting/operator-workflow-experience/1",
    "lastAppliedAtUtc": "2026-07-19T08:14:27.033Z",
    "statusObservedAtUtc": "2026-07-19T08:17:10.810Z"
  },
  "refresh": {
    "automatic": true,
    "afterMs": 5000,
    "manual": true,
    "href": "/ui-api/v1/workflow-experiences/019bc4d4-61a8-76f1-a935-b953af1dd9c4"
  }
}
```

READABLE VERSUS CANONICAL

The displayed JSON is for review. Contract tests compare the RFC 8785 canonical UTF-8 body and its exact digest.

FIXTURE IDENTITY

The mixed-state response has one exact digest

The body digest detects accidental field loss, enum renaming, time rewriting and refresh-policy drift. It identifies one representation fixture; it is not a global distributed version or business signature.

EXECUTABLE ASSET 41-P - GOLDEN BODY GATE

```
const string MediaType = "application/vnd.nexus1.operator-workflow-experience.v1+json";
const string ExperienceId = "019bc4d4-c1b0-7e4e-ae6c-4e3af2e7b331";
const string GenerationId = "019bc4d4-c21a-7527-8078-5be0476e77bf";
const long RepresentationVersion = 17;
const int CanonicalBodyLength = 1111;
const string CanonicalBodySha256 =
  "026654b93a21cd97f1d8069927016a7c33e914a45655e57e222b2660213891dd";

actual.Bytes.Should().Equal(File.ReadAllBytes(
  "contracts/operator-workflow-experience-delayed.v1.json"));
actual.Sha256.Should().Be(CanonicalBodySha256);
```

EVIDENCE BOUNDARY

The fixture is internally frozen in the manuscript. Cross-language serializer and deployed endpoint execution remain PENDING.

BFF CONTRACT**The response carries cycles, freshness and refresh advice**

The BFF reads an authorized Reporting projection and maps it into an experience-specific V1 contract. It preserves the Chapter 35 freshness vocabulary and adds cycle progress. It does not query owner databases to manufacture a synchronous truth.

EXECUTABLE ASSET 41-Q - V1 RESPONSE TYPES

```
public sealed record OperatorWorkflowExperienceV1(
    string ExperienceId,
    string WorkflowId,
    string CorrelationId,
    long RepresentationVersion,
    string ProjectionGenerationId,
    DateTimeOffset ObservedAtUtc,
    CycleProgressV1 OriginalCycle,
    CycleProgressV1? CorrectionCycle,
    ProjectionFreshnessV1 Freshness,
    RefreshAdviceV1 Refresh);

public sealed record CycleProgressV1(
    string CycleId, string ProjectionState, string ReasonCode,
    DateTimeOffset? ExpectedByUtc, DateTimeOffset? CompletedAtUtc,
    IReadOnlyList<string> MissingFacts, string? SafeFailureRef);
```

CONTRACT RULE

State controls legal fields: Failed requires SafeFailureRef; Completed requires CompletedAtUtc and no missing facts.

BFF MAPPING**The mapper is exhaustive and side-effect free**

One ObservedAtUtc is sampled before mapping. The BFF retains Reporting's generation/version and maps stable codes to UI wording resources. Unknown states or illegal field combinations fail closed to a versioned problem rather than silently defaulting.

EXECUTABLE ASSET 41-R - EXHAUSTIVE CYCLE MAPPING

```
CycleProgressV1 Map(CycleObservation cycle) => cycle.Progress switch
{
    ProjectionProgress.Pending x =>
        PendingV1(cycle, x.ExpectedByUtc, x.Missing),
    ProjectionProgress.Delayed x =>
        DelayedV1(cycle, x.ExpectedByUtc, x.Missing),
    ProjectionProgress.Completed x =>
        CompletedV1(cycle, x.CompletedAtUtc),
    ProjectionProgress.Failed x =>
        FailedV1(cycle, x.SafeCode, x.EvidenceRef),
    _ => throw new UnsafeProjectionState()
};
```

NO LABEL PARSING

The BFF maps typed states and stable codes. It never parses a Reporting display string.

HTTP EDGE

Refresh is one authorized, cache-safe GET

The route returns 200 for every legal projection state, because Pending, Delayed and Failed are representation content rather than HTTP transport failures. Invalid media, denied scope and BFF/Reporting unavailability use the established Problem Details boundary.

EXECUTABLE ASSET 41-S - MINIMAL API ROUTE

```
app.MapGet("/ui-api/v1/workflow-experiences/{workflowId}",
    async (string workflowId, ClaimsPrincipal user,
        IOperatorExperienceQueries queries,
        HttpContext http, CancellationToken ct) =>
    {
        var scope = SiteScope.FromClaims(user);
        var view = await queries.GetAsync(workflowId, scope, ct);
        http.Response.Headers.CacheControl = "private, no-store";
        return Results.Json(view, contentType: "application/vnd.nexus1.operator-workflow-experience.v1+json");
    })
    .RequireAuthorization("operator-workflow-experience-read-v1")
    .Produces<OperatorWorkflowExperienceV1>(200, "application/vnd.nexus1.operator-workflow-experience.v1+json")
    .ProducesProblem(400).ProducesProblem(403).ProducesProblem(503);
```

REFRESH INVARIANT

The route has no command side effect, no idempotency key and no owner write capability.

HTTP OUTCOME MATRIX

Projection content and request failure remain distinct

The HTTP status describes whether the BFF request produced a usable authorized representation. The body state describes what the projection knows. This prevents 500, 404 or 202 from being overloaded as a distributed workflow state.

DECISION LEDGER 41-41

Condition	HTTP	Body meaning
Pending/Delayed/Completed/Failed row	200	typed experience representation
not observed but authorized anchor	200	Pending observation contract
invalid route/media	400/406	versioned Problem Details
scope denied	403/safe policy	no existence leak
Reporting query unavailable	503	transport problem + bounded retry

NO 206

HTTP 206 describes byte-range partial content, not application-level eventual consistency.

TRANSPORT VERSUS PROJECTION FAILURE

The UI keeps the last known meaning only with an explicit stale label

A 503 means the current refresh could not obtain a representation. It does not change a prior Completed projection into Failed. If policy permits an in-memory last-known view during a transient request failure, the screen labels it stale and retains its original observation time; a hard browser reload starts without that memory.

DECISION LEDGER 41-42

Prior view	Refresh outcome	UI result
none	503	transport unavailable; no projection claim
Completed	503	optional stale Completed + old observed time
Delayed	503	optional stale Delayed + old observed time
any	200 Failed	fresh projection Failed representation

NO PERSISTENCE SHORTCUT

The reference UI does not place authorized projection bodies in localStorage or shared browser caches.

AUTHORIZATION AND MINIMIZATION

Uncertainty cannot become a disclosure side channel

The BFF authorizes the operator and site scope before reading the projection. The response exposes stable missing-fact names and safe evidence references, never raw broker bodies, queue names, stack traces, cross-site identifiers or internal policy diagnostics.

DECISION LEDGER 41-43

Layer	Allowed	Denied
Reporting consumer	queue read + ReportingDb write	owner databases
BFF	scoped Reporting read	projection mutation
operator	authorized cycle/freshness view	raw evidence/payload
support reviewer	protected evidence reference	credential/token material

EXISTENCE POLICY

Denied and hidden-nonexistent resources follow one reviewed external oracle; state detail cannot reveal cross-scope existence.

REFRESH ADVICE

The server suggests cadence; the client enforces bounds

Pending refreshes relatively quickly, Delayed backs off, Completed stops rapid polling and Failed waits for manual/support action. Advice is a UI policy hint, not a guarantee. The client clamps values and adds bounded jitter to avoid synchronized polling.

DECISION LEDGER 41-44

State	Automatic	Candidate cadence
Pending	yes	2-5 seconds
Delayed	yes	5-15 seconds with backoff
Completed	slow/disabled	manual or low-frequency observation
Failed	no rapid loop	manual after evidence/support

CAPACITY GATE

Actual intervals remain candidates until concurrency, BFF capacity and recovery-time evidence accept them.

ANGULAR CONTRACT

The UI consumes discriminated states, not nullable guesses

The TypeScript contract mirrors the server union and keeps transport state separate. Exhaustive mapping makes a newly introduced server state a compile/test failure rather than an accidental green badge.

EXECUTABLE ASSET 41-T - CLIENT TYPES

```
export type ProjectionState =
  | "pending" | "delayed" | "completed" | "failed";

export interface CycleProgressV1 {
  cycleId: string;
  projectionState: ProjectionState;
  reasonCode: string;
  expectedByUtc: string | null;
  completedAtUtc: string | null;
  missingFacts: readonly string[];
  safeFailureRef: string | null;
}

export type FetchState =
  | { kind: "loading" }
  | { kind: "ready"; value: OperatorWorkflowExperienceV1 }
  | { kind: "unavailable"; retryAfterMs: number | null }
  | { kind: "denied" };
```

TWO UNIONS

FetchState answers whether this request worked. ProjectionState answers what the returned projection knows.

ANGULAR DATA SERVICE

Every subscription creates one GET and no hidden fan-out

The browser calls only the BFF route fixed by the current URL. HttpClient returns a cold Observable, so the store owns one subscription per refresh intent and cancels obsolete work through switchMap.

EXECUTABLE ASSET 41-U - TYPED BFF CLIENT

```
@Injectable({ providedIn: "root" })
export class OperatorExperienceClient {
  private readonly http = inject(HttpClient);

  get(workflowId: string) {
    const id = encodeURIComponent(workflowId);
    return this.http.get<OperatorWorkflowExperienceV1>(
      `/ui-api/v1/workflow-experiences/${id}`,
      { headers: { Accept:
        "application/vnd.nexus1.operator-workflow-experience.v1+json" } }
    );
  }
}
```

BROWSER BOUNDARY

JavaScript never calls AlarmManagement, RootCause, Compliance or Reporting separately.

EXPERIENCE STORE

Route identity and refresh intent drive one reactive stream

The store keeps only current in-memory presentation state. A route change or refresh counter triggers a new BFF GET. switchMap prevents an obsolete request from remaining authoritative after a newer intent.

EXECUTABLE ASSET 41-V - SIGNAL-BACKED STORE CORE

```
private readonly workflowId = signal<string | null>(null);
private readonly refreshTick = signal(0);
readonly state = signal<FetchState>({ kind: "loading" });

readonly request = computed(() => ({
  workflowId: this.workflowId(),
  tick: this.refreshTick()
}));

connect(id: string): void { this.workflowId.set(id); }
refresh(): void { this.refreshTick.update(x => x + 1); }

effect(() => {
  const request = this.request();
  if (request.workflowId) this.load(request.workflowId);
});
```

REFERENCE STATUS

The listing is compile-oriented shape. Exact Angular package versions and a built UI artifact belong in the evidence manifest.

REFRESH SCHEDULING

Only nonterminal advice creates a bounded timer

The scheduler cancels its prior timer when a new representation, route or component lifetime arrives. It clamps server advice, adds bounded jitter and never schedules a command. Manual refresh uses the same GET path.

EXECUTABLE ASSET 41-W - BOUNDED REFRESH

```
schedule(view: OperatorWorkflowExperienceV1): void {
  this.timer?.unsubscribe();
  if (!view.refresh.automatic) return;

  const base = clamp(view.refresh.afterMs, 1000, 30000);
  const delay = jitter(base, 0.10);
  this.timer = timer(delay)
    .pipe(takeUntilDestroyed(this.destroyRef))
    .subscribe() => this.refresh();
}

manualRefresh(): void { this.refresh(); } // GET only
```

NO POLLING STORM

Visibility changes, offline state and repeated button presses are coalesced by the store's one-active-request policy.

RESPONSE RACES

An older request cannot overwrite a newer representation

Cancellation reduces wasted work but is not sufficient proof: a response may already be in flight. The store compares projection generation and representation version before replacing Ready state. A generation change is accepted only with a BFF-declared replacement relationship.

EXECUTABLE ASSET 41-X - MONOTONIC ACCEPTANCE

```
function mayReplace(
  current: OperatorWorkflowExperienceV1 | null,
  incoming: OperatorWorkflowExperienceV1): boolean {
  if (!current) return true;
  if (incoming.projectionGenerationId ===
    current.projectionGenerationId)
    return incoming.representationVersion >=
      current.representationVersion;

  return incoming.replacesGenerationId ===
    current.projectionGenerationId;
}
```

CHAPTER 42 SEAM

A repaired generation must explicitly name the generation it replaces; lexical GUID order is never treated as recency.

BROWSER REFRESH

Reload reconstructs state from the BFF

The workflow identifier remains in the authorized route. On a hard refresh, a new component and store instance issue the same GET, receive the latest representation and render it. No in-memory timeline, timer or push message is required for correctness.

DECISION LEDGER 41-50

Before reload	After reload	Invariant
Pending view in memory	BFF may return Delayed	server reclassifies
Delayed view in memory	BFF may return Completed	facts drive convergence
Completed original + Pending correction	same two cycles restored	history retained
lost optional push hint	GET still recovers	push is not truth

RECOVERY RULE

A page reload may lose presentation memory. It must not lose any durable owner or projection evidence.

ACCESSIBLE RENDERING

Status is conveyed by text, structure and live-region policy

The screen renders original and correction sections with explicit labels. Progress updates use a polite status region; blocking Failed evidence may use an alert only when immediate attention is justified. Color is never the sole distinction.

EXECUTABLE ASSET 41-Y - EXHAUSTIVE TEMPLATE SHAPE

```
<section aria-labelledby="original-cycle-heading">
  <h2 id="original-cycle-heading">Original workflow</h2>
  <app-cycle-progress [cycle]="view.originalCycle" />
</section>

@if (view.correctionCycle; as correction) {
  <section aria-labelledby="correction-cycle-heading">
    <h2 id="correction-cycle-heading">Correction</h2>
    <app-cycle-progress [cycle]="correction" />
  </section>
}

<p role="status" aria-live="polite">{{ statusText() }}</p>
<button type="button" (click)="manualRefresh()">Refresh status</button>
```

ACCESSIBILITY INVARIANT

State changes remain understandable without color, animation or pointer interaction.

OPERATOR COPY

Wording preserves uncertainty instead of hiding it

Every state has approved visible language and a forbidden misleading phrase. The UI may localize text, but stable machine states and reason codes remain unchanged.

DECISION LEDGER 41-52

State	Visible wording	Forbidden wording
Pending	Waiting for required observation	Everything is fine
Delayed	Update delayed; outcome not yet known	Failed
Completed	Projection complete for version N	Globally committed
Failed	Projection could not advance safely	Business decision failed
transport unavailable	Unable to refresh now	Projection failed

TWO-TIMELINE COPY

Original workflow completed. Correction delayed. Both statements may appear together.

UI MAPPING TESTS

Every projection state has a positive and negative assertion

Component tests verify visible wording, semantic role and forbidden copy. A default branch is prohibited so a new state breaks the test suite and client generation instead of borrowing Completed styling.

EXECUTABLE ASSET 41-Z - TRUTHFUL STATE MATRIX

```
describe.each([
  ["pending", "Waiting", "Complete"],
  ["delayed", "delayed", "Failed"],
  ["completed", "complete for version", "Globally"],
  ["failed", "could not advance", "decision failed"]
])("projection %s", (state, visible, forbidden) => {
  const screen = renderExperience(golden({ state }));
  expect(screen.getByRole("status")).toHaveTextContent(visible);
  expect(screen.getByRole("status")).not.toHaveTextContent(forbidden);
  expect(screen.getByText("Original workflow")).toBeVisible();
});
```

COLOR GATE

Tests also inspect accessible names and icon labels; CSS color snapshots alone are insufficient.

REFRESH RECOVERY TESTS

A new store instance converges without client persistence

The essential UI proof destroys the component and store, advances server evidence, recreates the route and observes the newer state. It also proves that no POST or owner call occurred.

EXECUTABLE ASSET 41-AA - HARD-REFRESH PROOF

```
it("recovers delayed to completed after hard refresh", async () => {
  bff.enqueue(delayedView({ version: 17 }));
  const first = await mountRoute(WORKFLOW_ID);
  expect(first.status()).toBe("delayed");
  first.destroy();

  bff.enqueue(completedView({ version: 18 }));
  const reloaded = await mountRoute(WORKFLOW_ID);
  expect(reloaded.status()).toBe("completed");
  expect(bff.getCount()).toBe(2);
  expect(bff.postCount).toBe(0);
  expect(storage.ownerTruthWrites).toBe(0);
});
```

REQUIRED RECOVERY

The test starts with a fresh client process model; retaining the old in-memory store would not prove browser refresh recovery.

RESPONSE-ORDER TESTS

Slow old responses cannot repaint the screen

The test holds version 17, completes version 18 first and then releases the older response. The store must retain version 18. A second test changes projection generation and requires an explicit replacement link.

EXECUTABLE ASSET 41-AB - STALE RESPONSE REJECTION

```
it("ignores an older response that arrives last", async () => {
  const old = bff.defer(delayedView({ version: 17 }));
  store.refresh();
  const current = bff.defer(completedView({ version: 18 }));
  store.refresh();

  current.resolve();
  await expectStoreVersion(18, "completed");
  old.resolve();
  await expectStoreVersion(18, "completed");
});
```

CANCELLATION PLUS VERSIONING

Request cancellation is a resource optimization. Monotonic acceptance is the correctness guard.

BFF CONTRACT TESTS

Illegal field combinations fail before the UI

The contract corpus enumerates every state and illegal shape. Completed with missing facts, Failed without evidence reference, Pending after its declared deadline fixture and unknown enum values are rejected.

DECISION LEDGER 41-56

Fixture	Expected
Pending + ExpectedBy + missing	accept
Delayed + ExpectedBy + missing	accept
Completed + CompletedAt + empty missing	accept
Failed + safe evidence reference	accept
Completed + missing fact	reject
unknown state or raw failure detail	reject

COMPATIBILITY

Old and new generated clients must preserve the four meanings; silently defaulting a new enum is a breaking change.

PROVIDER-BACKED TESTS

SQL Server proves atomic progress and race arbitration

Real-provider tests execute TX-UX, rowversion conflicts, filtered uniqueness and named error classification. They compare rows before and after injected failures rather than trusting SaveChanges return values.

EXECUTABLE ASSET 41-AC - PROVIDER ORACLE

```
[Fact]
public async Task delayed_cycle_completes_once_after_applied_fact()
{
    await lab.ApplyAsync(CORRECTION_MESSAGE_ID);
    lab.Clock.AdvancePastCorrectionDeadline();
    (await lab.QueryAsync()).Correction.State.Should().Be("Delayed");

    await lab.DeliverConcurrentlyAsync(APPLIED_MESSAGE_ID, copies: 8);
    var snapshot = await lab.ReportingDb.ReadSnapshotAsync();
    snapshot.Correction.State.Should().Be("Completed");
    snapshot.Correction.EvidenceSlots.Should().HaveCount(2);
    snapshot.RepresentationVersionDelta.Should().Be(1);
}
```

ROLLBACK PROBE

Failure after evidence insert but before checkpoint commit must leave no partial slot, version or inbox outcome.

BROKER TESTS

Repeated, reordered and delayed facts converge through one queue

The RabbitMQ lab declares production bindings, publishes exact golden bodies, pauses Reporting, reorders correction inputs, loses an acknowledgement and verifies one final projection effect. Queue recovery is measured separately from business completion.

DECISION LEDGER 41-58

Scenario	Pass condition
Reporting offline	facts durable; view remains delayed
Applied before Corrected	buffer then one completion
same body redelivered	one evidence slot
confirm/ack ambiguity	idempotent recovery
unknown schema	durable failure; no reducer mutation

ACCEPTANCE PATH

The complete slice must converge after one service was temporarily offline, with every transition explainable from retained evidence.

PRODUCTION-PATH HOST TESTS

The assembled route proves policy, media and no-store behavior

A WebApplicationFactory built from the production composition path exercises authentication, authorization, Reporting query, mapping, Problem Details and response headers. It also inventories the absence of owner mutation clients.

EXECUTABLE ASSET 41-AD - HOST BOUNDARY

```
var response = await host.GetAuthorizedAsync(
    $"/ui-api/v1/workflow-experiences/{WORKFLOW_ID}", MEDIA_TYPE);

response.StatusCode.Should().Be(HttpStatusCode.OK);
response.Content.Headers.ContentType!.MediaType.Should().Be(MEDIA_TYPE);
response.Headers.CacheControl!.Private.Should().BeTrue();
response.Headers.CacheControl.NoStore.Should().BeTrue();

host.Capabilities.OwnerMutationClients.Should().BeEmpty();
host.Capabilities.PublicCommandRoutes.Should().BeEmpty();
host.Capabilities.ReportingReadRoutes.Should().ContainSingle();
```

SAME-PATH PROOF

A controller unit test cannot prove production middleware order, authorization, headers or DI capability inventory.

FAULT LABORATORY

The difficult combinations are first-class scenarios

The deployment lab combines broker lag, database interruption, browser reload and response reordering. It asserts owner rows, projection rows, BFF bodies and UI text at every checkpoint rather than waiting only for an eventual green result.

DECISION LEDGER 41-60

Scenario	Intermediate view	Final view
Compliance offline	Correction Delayed	Completed after recovery
Reporting offline	transport unavailable/stale	latest state after GET
bad contract	Projection Failed	new repair generation in Ch42
browser hard reload	Loading	server-current state
old HTTP response late	newer version retained	unchanged

NO SHORTCUT ASSERTION

A successful test exit without observed intermediate Pending/Delayed/Failed semantics is insufficient evidence.

OBSERVABILITY

Metrics describe state without leaking workflow identity

Reporting and BFF emit bounded state/reason metrics, transition counters and refresh latency. Logs carry trace ID, generation, stable code and protected evidence reference under access control. Workflow, case, verdict and operator identifiers are excluded from metric labels.

DECISION LEDGER 41-61

Signal	Allowed labels	Forbidden labels
projection_state_total	state/cycle-kind/reason	workflowId/caseld
projection_transition_total	from/to/generation-status	correctionId
bff_refresh_duration	route/status class	operator identity
ui_refresh_result	ready/unavailable/denied	token/resource ID

TRACE MEANING

One BFF refresh span describes the request path; it is not used as business causation or projection identity.

CAPACITY AND BACKPRESSURE

Refresh policy must survive the incident it reports

Thousands of consoles polling at one fixed interval can become a recovery incident. The design clamps cadence, adds jitter, stops rapid polling for terminal states, applies per-actor limits and serves one indexed Reporting query without cross-service fan-out.

DECISION LEDGER 41-62

Risk	Control	Evidence
synchronized polling	bounded jitter	arrival distribution
button hammering	coalesce + rate limit	request count
slow ReportingDb	deadline + circuit	latency/fault matrix
large timelines	bounded page/summary	payload/heap profile
recovery stampede	server advice + backoff	offline recovery run

NO INVENTED SCALE CLAIM

Candidate intervals and limits remain PENDING until tested against a named concurrency and latency profile.

DEPLOYMENT AND REVERSAL

Consumers and UI compatibility precede activation

Deploy the Reporting schema/generation, tolerant reducers, BFF V1 route and UI exhaustive mapping before enabling new bindings. Reversal stops activation and preserves queues, projection generations and failure evidence; it does not delete delayed or failed history.

DECISION LEDGER 41-63

Stage	Change	Exit evidence
prepare	schema + principals	migration/permission digest
consume	six-contract generation	golden corpus
expose	BFF V1 route	host/security matrix
render	four-state UI	accessibility/refresh tests
activate	bindings + policy	fault lab
reverse	pause activation; retain evidence	replay plan for Ch42

GENERATION DISCIPLINE

A rollback changes code used next. It does not erase projection transitions already shown to operators.

EVIDENCE CATALOGUE

Every operator claim maps to a proof surface

The catalogue separates classifier, provider, broker, HTTP, browser and accessibility proof. One green UI snapshot cannot substitute for delayed delivery, hard reload, provider atomicity or authorization evidence.

DECISION LEDGER 41-64

Claim	Proof surface	Minimum artifact
four state meanings	domain/contract	classifier + schema corpus
one projection effect	SQL Server + broker	row/delivery evidence
honest BFF	production host	HTTP/security snapshots
hard refresh recovery	browser E2E	new-process timeline
accessible uncertainty	component/a11y	role/name/copy report
offline convergence	deployment lab	full transition dossier

FAIL-CLOSED RULE

Zero-discovery, skipped matrices, stale builds or missing environment identities keep status PENDING.

EVIDENCE MANIFEST

ADR-041 remains PENDING until the exact runtime exists

The dossier binds ADR lineage, contract set, delayed fixture, database/broker/browser versions, discovered tests, commands and artifact digests. The PDF specifies the proof path but does not claim that the NEXUS-1 repository or deployment executed it.

EXECUTABLE ASSET 41-AE - PROPOSED DOSSIER

```
{
  "evidenceId": "NEXUS1-OPERATOR-EVENTUAL-041",
  "status": "PENDING",
  "adr040Sha256": "65eeede68f69f856376e89bbb01d79c7243a30e6778b7cacf0e57ef4178ea741",
  "adr041Sha256": "0e8d710a99dcdbd8f2edc01322520322922647803b2e57e324b4a218463689a8",
  "projectionPolicy": "reporting/operator-workflow-experience/1",
  "projectionStates": ["Pending", "Delayed", "Completed", "Failed"],
  "delayedViewSha256": "026654b93a21cd97f1d8069927016a7c33e914a45655e57e222b2660213891dd",
  "publicCommandCountAdded": 0,
  "artifacts": []
}
```

TRUTHFUL VERDICT

Reference listings are compile-oriented. Runtime/provider/broker/browser evidence remains PENDING.

EVIDENCE VALIDATOR

CI rejects partial or mismatched proof

The validator checks that all four golden states exist, hard refresh starts a new client instance, the offline scenario converges, the provider and broker identities match, no public command was introduced and every artifact belongs to the same build.

EXECUTABLE ASSET 41-AF - FAIL-CLOSED GATES

```
manifest.DemandExactStates("Pending", "Delayed", "Completed", "Failed");
manifest.DemandTestDiscovery(minimum: expectedTestCount);
manifest.DemandScenario("ui-hard-refresh-new-instance");
manifest.DemandScenario("service-offline-delayed-then-completed");
manifest.DemandArtifact("sqlserver-provider-results.trx");
manifest.DemandArtifact("rabbitmq-redelivery-and-reorder.trx");
manifest.DemandArtifact("bff-host-security-and-cache.trx");
manifest.DemandArtifact("angular-refresh-and-accessibility.json");
manifest.DemandSameBuildIdentity();
manifest.DemandPublicCommandCountAdded(0);
```

NO EVIDENCE AGGREGATION TRICK

A complete browser test cannot make a missing provider, broker or contract artifact implicitly pass.

CHAPTER 42 BOUNDARY

Observation names repair candidates but does not repair them

Chapter 41 may expose missing slots, gap evidence, a failed generation and a safe support reference. It does not replay a message, clear a dead-letter queue, rewrite a checkpoint or mark a failure resolved. Chapter 42 owns those actions with authorization, idempotency and audit.

DECISION LEDGER 41-67

Chapter 41 exposes	Chapter 42 must implement
missing projection slot	comparison against retained facts
stuck outbox symptom	owner-safe dispatch/replay tool
dead-letter evidence	authorized re-admission decision
failed generation	new rebuild/repair generation
safe evidence reference	auditable repair action/result

PROTECTED HANDOFF

Every repair must be idempotent and auditable. The operator UI may request inspection, never silently perform repair on refresh.

OFFICIAL TECHNICAL ANCHORS

Refresh, time, accessibility and HTTP semantics stay explicit

- Angular HttpClient requests: <https://angular.dev/guide/http/making-requests>
- Angular Signals: <https://angular.dev/guide/signals>
- Angular Router lifecycle: <https://angular.dev/guide/routing/lifecycle-and-events>
- .NET TimeProvider: <https://learn.microsoft.com/dotnet/standard/datetime/timeprovider-overview>
- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- EF Core optimistic concurrency: <https://learn.microsoft.com/ef/core/saving/concurrency>
- HTTP caching semantics: <https://www.rfc-editor.org/rfc/rfc9111>
- Problem Details for HTTP APIs: <https://www.rfc-editor.org/rfc/rfc9457>
- WAI-ARIA 1.2 status and alert roles: <https://www.w3.org/TR/wai-aria-1.2/>
- OpenTelemetry HTTP semantic conventions: <https://opentelemetry.io/docs/specs/semconv/http/>

WHAT THIS CHAPTER HAS PROVED

Chapter 41 specifies ADR-041; a six-fact Reporting projection with retained slots, cycles and failure evidence; separate original and correction timelines; exact Pending, Delayed, Completed and Failed semantics; immutable expectations and server-clock classification; atomic/idempotent reducers; an honest BFF V1 contract; transport/freshness separation; bounded Angular refresh, monotonic response acceptance and hard-reload recovery; accessibility, security, capacity, deployment and fail-closed evidence paths. Runtime evidence remains PENDING.

CHAPTER CHECKPOINT

- Why can original completion and correction delay both be true?
- Which four axes must never be collapsed into one badge?
- What exact evidence distinguishes Pending from Delayed?
- Why is a failed projection not a failed business decision?
- Which transaction moves evidence, cycle and representation version together?
- Why can a fresh observation truthfully report a delayed cycle?
- How does the UI prevent an older HTTP response from replacing a newer version?
- What proves that a hard browser refresh recovers without localStorage?
- Why must refresh remain GET-only and command-free?
- Which artifacts are required before ADR-041 can become PROVED?

NEXT - CHAPTER 42

Reconciliation and Repair Jobs will compare retained owner facts with projection slots, inspect stuck outbox and dead-letter evidence, and execute only authorized, idempotent and auditable repair actions. It will create new repair evidence or projection generations rather than hiding the Pending, Delayed or Failed history specified here.

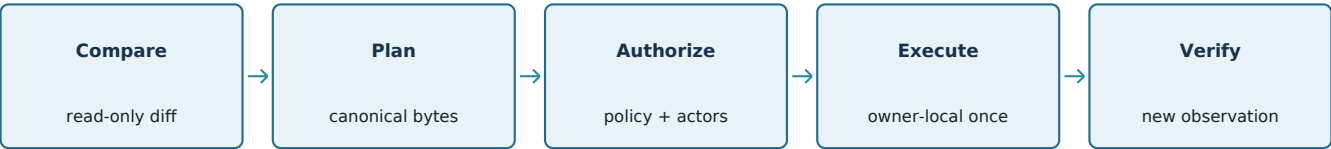
PART VII - WORKFLOW, CONSISTENCY, AND REPAIR

42. Reconciliation and Repair Jobs

Repair begins with a read-only comparison and ends only with a new verification. Between them, an immutable plan, explicit authorization, owner-local idempotency and bounded execution prevent recovery tooling from becoming a second source of truth.

CHAPTER DECISION
ADR-042 defines repair as compare-plan-authorize-execute-verify. RepairControlDb owns administrative intent and audit only. Reporting rebuilds missing projections in new generations; an outbox owner redispaches the exact retained message; the messaging replay authority re-admits approved poison evidence through the normal route. Every action uses a deterministic repair key, fresh-evidence precondition, blast limits and an independent post-repair oracle. Public business command additions remain zero.

PROOF PATH 42-A - EVIDENCE TO VERIFIED CONVERGENCE



GUARANTEE LEDGER 42-A

Specified	Explicitly rejected
three typed repair kinds	generic RepairAnything
new projection generation	edit failed generation
same MessageId and frozen bytes	new business fact
dual control for high risk	self-approved replay
independent verification	owner HTTP 200 means fixed

CHAPTER BOUNDARY

Repair restores convergence; it does not rewrite truth

Chapters 26, 29, 35 and 41 already retain the ingredients needed for safe recovery: frozen outbox bytes, poison evidence, an event ledger, explicit projection generations and honest failure states. Chapter 42 adds the administrative workflow that may act on that evidence without borrowing business authority.

DECISION LEDGER 42-2

Inherited evidence	Permitted repair	Forbidden shortcut
retained owner facts	rebuild a derived generation	edit owner decision
frozen outbox bytes	owner-local redispach	manufacture a new fact
poison-message record	approved normal-route replay	invoke Domain directly
failed projection generation	retain and supersede	mark failure Completed

SCOPE INVARIANT

Repair control owns intent and evidence. Domain services continue to own business decisions and their databases.

FAILURE VOCABULARY

Missing, stuck and dead-lettered are observations, not diagnoses

A scanner may identify a suspicious coordinate, but it cannot infer the correct remedy from absence alone. A projection can be late, an outbox row can have an unknown confirm, and a dead-letter can reflect a permanently invalid contract. Classification therefore stays read-only and fail closed.

DECISION LEDGER 42-3

Observation	Possible benign cause	Unsafe assumption
missing projection	lag or gap buffer	republish immediately
old Pending outbox	publisher confirm unknown	insert duplicate row
dead-letter record	unsupported/hostile bytes	all poison is transient
failed generation	deterministic reducer defect	retry same generation

CANDIDATE IS NOT PERMISSION

Only a stable comparison result can create a candidate; only an authorized plan can create an execution attempt.

REPAIR DOCTRINE

Five gates separate evidence from side effects

The workflow is deliberately longer than a replay button. Comparison establishes the observed mismatch. Planning freezes target, basis and limits. Authorization binds actors and policy. Execution delegates to the owner-local capability. Verification independently asks whether the declared invariant now holds.

DECISION LEDGER 42-4

Gate	Durable output	Must not do
compare	snapshot + classified diff	write target
plan	canonical plan + digest	execute
authorize	approval record	change plan
execute	owner receipt + attempt	claim convergence
verify	new comparison + verdict	hide original failure

PLAIN-LANGUAGE RULE

See what differs, freeze what you intend, prove who allowed it, act once, then measure again.

AUTHORITY MAP

The administrative host is not a fourth business service

Nexus1.Operations.Repair stores repair cases, plans, approvals, attempts and evidence references in RepairControlDb. It has no credentials that can mutate AlarmManagementDb, RootCauseDb, ComplianceDb or ReportingDb directly. Owner-local administrative endpoints validate and execute narrowly typed actions.

DECISION LEDGER 42-5

Component	Owns	Credential must not allow
Repair control	plan and audit state	business-table writes
Reporting repair port	projection generations	owner-fact edits
outbox owner port	redispatch scheduling	payload replacement
messaging replay port	poison re-admission	direct Domain calls
verifier	read-only comparison	execution

CREDENTIAL PROOF

The intended permission split is specified here; environment-specific grants remain PENDING until retained grant inventories prove it.

ADR-042

Repair is compare-plan-authorize-execute-verify

ADR-042 accepts a separate administrative control plane, owner-local execution, canonical plan identity, fresh-evidence preconditions, bounded blast radius and post-action reconciliation. It adds no public business command to the Chapter 38-41 choreography.

DECISION LEDGER 42-6

ADR field	Decision
candidate kinds	MissingProjection, StuckOutbox, DeadLetteredEvent
workflow	compare, plan, authorize, execute, verify
business ownership	unchanged
public commands	zero additions
history	append attempts and generations
evidence	PENDING

ADR DIGEST

sha-256+jcs 5982cb3f37a3d5f90ebfd13d188d77c0ffab7f56a5e44449a1cac33acf162e19

CONTROL PLANES

Read, decide and act through different capabilities

The comparison reader, approval service and execution adapters are intentionally distinct dependencies. A bug or compromised credential in one surface cannot silently acquire every repair capability. The production composition root must prove the split.

DECISION LEDGER 42-7

Plane	Operation	Trust level
observation	read safe evidence contracts	read-only
decision	freeze plan and approvals	administrative
execution	one owner-local repair action	privileged and narrow
verification	re-run comparison	read-only independent
business	normal commands/facts	outside repair authority

CHAPTER 43 HANDOFF

These privileged edges become explicit trust boundaries in the next chapter; this chapter does not claim production security maturity.

READ-ONLY COMPARISON

A typed snapshot freezes the basis of every diff

Comparison does not run arbitrary SQL across service databases. Each owner exposes a safe operational evidence contract with a watermark, contract identity, captured time and digest. The scanner stores references and hashes, not secret connection strings or unbounded payloads.

EXECUTABLE ASSET 42-A - COMPARISON SNAPSHOT

```
public sealed record EvidenceSnapshot(
    string Owner,
    string ContractIdentity,
    string SnapshotId,
    long Watermark,
    DateTimeOffset CapturedAtUtc,
    string ContentSha256,
    IReadOnlyList<EvidenceCoordinate> Coordinates);

public interface IRepairEvidenceReader
{
    Task<EvidenceSnapshot> ReadAsync(
        RepairTarget target, EvidenceBudget budget,
        CancellationToken cancellationToken);
}
```

NO CROSS-DATABASE JOIN

The comparer joins bounded owner-published coordinates in memory; it never treats network reachability as permission to borrow a database.

COMPARISON BUDGETS

Observation is bounded before it becomes expensive

Every scan carries a tenant/site scope, key range, time window, maximum item count, maximum byte count and deadline. A truncated snapshot is never equivalent to a complete snapshot and cannot authorize execution.

EXECUTABLE ASSET 42-B - FAIL-CLOSED BUDGET

```
public sealed record EvidenceBudget(
    SiteId SiteId,
    Instant FromInclusive,
    Instant ToExclusive,
    int MaxItems,
    long MaxBytes,
    TimeSpan Timeout)
{
    public void DemandValid()
    {
        if (ToExclusive <= FromInclusive) throw Invalid("window");
        if (MaxItems is < 1 or > 10_000) throw Invalid("max-items");
        if (MaxBytes is < 1 or > 64 * 1024 * 1024) throw Invalid("max-bytes");
        if (Timeout <= TimeSpan.Zero || Timeout > TimeSpan.FromMinutes(2))
            throw Invalid("timeout");
    }
}
```

TRUNCATION RULE

BudgetExceeded and Timeout produce UnsafeUnknown, never InSync and never an executable candidate.

STABLE COORDINATES

Compare meaning by identity, not by row position

A coordinate names the producer, message or aggregate identity, contract version, semantic slot and digest. Database surrogate keys, queue delivery tags and local row ordering are excluded because they cannot prove cross-boundary meaning.

EXECUTABLE ASSET 42-C - SEMANTIC COORDINATE

```
public sealed record EvidenceCoordinate(  
    string Owner,  
    string Kind,  
    string SemanticKey,  
    string ContractIdentity,  
    string ContentSha256,  
    DateTimeOffset ObservedAtUtc,  
    string SafeReference);  
  
public sealed record CoordinateDiff(  
    EvidenceCoordinate Expected,  
    EvidenceCoordinate? Actual,  
    DiffKind Kind,  
    string ReasonCode);  
  
public enum DiffKind { Missing, DigestMismatch, Extra, InSync }
```

MEANING OVER SHAPE

Equal row counts do not prove equal histories; unequal local row identifiers do not prove a mismatch.

CLASSIFICATION

Four verdicts keep incomplete evidence from becoming action

The comparer returns InSync only when all required layers are complete. Candidate means one recognized and bounded mismatch has a supported repair strategy. Blocked means a known policy or hold forbids action. UnsafeUnknown means the evidence is incomplete, inconsistent or unsupported.

DECISION LEDGER 42-11

Verdict	Execution allowed	Typical cause
InSync	no	all declared invariants hold
Candidate	only after plan + approval	supported bounded mismatch
Blocked	no	legal hold, active owner work, expired plan
UnsafeUnknown	no	truncation, drift, unsupported contract

FAIL-CLOSED DEFAULT

Unknown enum values, missing digests and unavailable owners map to UnsafeUnknown.

COMPARISON LAYERS

Coverage is only the first reconciliation question

Chapter 13's reconciliation discipline still applies after service extraction. The tool checks coverage, per-key meaning, invariants, deletes and corrections, history, freshness and operational completeness. A candidate names exactly which layer failed.

DECISION LEDGER 42-12

Layer	Question	Example repair blocker
coverage	all expected keys represented?	snapshot truncated
meaning	digests/contracts agree?	unsupported schema
invariants	owner rules preserved?	duplicate active generation
corrections	supersession retained?	correction chain broken
history	original evidence retained?	ledger gap
freshness	watermark expectation met?	normal lag window
operations	holds/leases/health safe?	active publisher lease

NO ROW-COUNT SHORTCUT

A passing count with a failed meaning or history layer remains UnsafeUnknown or Blocked.

CANDIDATE IDENTITY

The mismatch digest makes detection repeatable

A RepairCandidate is an immutable observation. Repeating the same comparison basis returns the same candidate key; a newer snapshot creates a new candidate or proves the old one obsolete. Candidates never carry executable credentials.

EXECUTABLE ASSET 42-D - IMMUTABLE CANDIDATE

```
public sealed record RepairCandidate(  
    RepairCandidateId Id,  
    RepairKind Kind,  
    RepairTarget Target,  
    string ExpectedSnapshotId,  
    string ActualSnapshotId,  
    string ComparisonSha256,  
    ComparisonVerdict Verdict,  
    string ReasonCode,  
    DateTimeOffset ObservedAtUtc,  
    DateTimeOffset ExpiresAtUtc);  
  
public enum RepairKind  
{  
    MissingProjection, StuckOutbox, DeadLetteredEvent  
}
```

EXPIRY

An expired candidate can be inspected but cannot be planned; the operator must compare again.

CANONICAL PLAN

The authorized object is exact bytes, not a mutable form

The plan freezes candidate identity, owner, target, action, source snapshot, precondition digest, blast limits, policy and expiry using RFC 8785 canonical JSON. Approval signs the plan digest; any later mutation invalidates the approval.

EXECUTABLE ASSET 42-E - PLAN ENVELOPE

```
public sealed record RepairPlanV1(
    Guid RepairPlanId,
    Guid RepairCaseId,
    RepairKind Kind,
    OwnerId Owner,
    RepairTarget Target,
    EvidenceBasis Basis,
    RepairLimits Limits,
    RepairMode Mode,
    string Policy);

public sealed record CanonicalRepairPlan(
    byte[] Utf8Json,
    string Sha256,
    string MediaType);
```

FIXTURE DIGEST

The chapter fixture hashes to 41d43184923631699b2d6ea069da502a2f8f86cd7bcfd db3a8cabf4a4a537592 under sha-256+jcs.

IDEMPOTENCY KEY

Same target and same plan converge on one job

The repair key binds kind, owning capability, semantic target and canonical plan digest. A repeated POST with the same key and same plan returns the existing job. Reusing the key with different bytes returns conflict and creates no second action.

EXECUTABLE ASSET 42-F - DETERMINISTIC REPAIR KEY

```
public static string ComputeRepairKey(
    RepairKind kind, OwnerId owner,
    RepairTarget target, string planSha256)
{
    var material = string.Join('|',
        kind, owner.Value, target.SemanticKey, planSha256);
    return Convert.ToHexString(
        SHA256.HashData(Encoding.UTF8.GetBytes(material)))
        .ToLowerInvariant();
}
```

REFERENCE KEY

011300e2b8b0a6ef0a153d1eaae021f37f4628d1dd4301c1b9d5962d4ac9898b

REPAIR AGGREGATE

One state machine owns authorization and execution history

RepairCase never performs a business mutation itself. It validates transitions, binds approvals to an exact plan digest and records attempts and verification verdicts. Owner adapters execute only after the aggregate emits an authorized execution intent.

EXECUTABLE ASSET 42-G - AGGREGATE SURFACE

```
public sealed class RepairCase
{
    public RepairCaseId Id { get; private init; }
    public RepairState State { get; private set; }
    public string? PlanSha256 { get; private set; }
    public int Attempt { get; private set; }

    public void RecordComparison(RepairCandidate candidate);
    public void FreezePlan(CanonicalRepairPlan plan);
    public void Approve(Approval approval);
    public RepairExecutionIntent Begin(Instant now);
    public void RecordOwnerReceipt(OwnerRepairReceipt receipt);
    public void RecordVerification(VerificationResult result);
}
```

AGGREGATE LIMIT

The aggregate cannot mark Succeeded from an owner receipt alone; an independent verification result is required.

REPAIR STATE MACHINE

Terminal means evidenced, not merely attempted

Compared and Planned are still read-only. Approved authorizes a bounded attempt. Executing records an owner call. Verifying re-runs reconciliation. Succeeded requires the target invariant. Failed preserves an attempt error. ManualReview preserves uncertainty that automation cannot safely resolve.

DECISION LEDGER 42-17

State	Allowed next	Durable prerequisite
Draft	Compared	case identity
Compared	Planned/Closed	candidate digest
Planned	Approved/Expired	canonical plan
Approved	Executing/Expired	valid approvals
Executing	Verifying/Failed/ManualReview	owner receipt or uncertainty
Verifying	Succeeded/Failed/ManualReview	new comparison
Succeeded/Failed	terminal	audit verdict

NO HISTORY REWRITE

Retry creates another Attempt under the same repair case; it never deletes the previous attempt.

TRANSITION GUARD

The state machine is closed in code

Transitions are enumerated and tested. Cancellation is permitted only before an owner acknowledges execution. Once a publish or generation build may have occurred, the job moves to verification or manual review instead of pretending that cancellation undid the world.

EXECUTABLE ASSET 42-H - ALLOWED TRANSITIONS

```
static readonly IReadOnlyDictionary<RepairState, RepairState[]> Allowed =
    new Dictionary<RepairState, RepairState[]>
{
    [RepairState.Draft] = [RepairState.Compared],
    [RepairState.Compared] = [RepairState.Planned, RepairState.Closed],
    [RepairState.Planned] = [RepairState.Approved, RepairState.Expired],
    [RepairState.Approved] = [RepairState.Executing, RepairState.Expired],
    [RepairState.Executing] =
        [RepairState.Verifying, RepairState.Failed, RepairState.ManualReview],
    [RepairState.Verifying] =
        [RepairState.Succeeded, RepairState.Failed, RepairState.ManualReview],
    [RepairState.Succeeded] = [],
    [RepairState.Failed] = [],
    [RepairState.ManualReview] = []
};
```

CANCELLATION TRUTH

Stop scheduling is possible; undoing an already published delivery is not.

FRESH-EVIDENCE PRECONDITION

Authorization expires when the observed world changes

Immediately before execution, the adapter re-reads the minimal target evidence. The plan's snapshot identity, content digest, hold status and owner generation must still match. Drift blocks the attempt and returns the case to comparison through a new candidate.

EXECUTABLE ASSET 42-I - COMPARE-AND-ACT GUARD

```
var current = await evidence.ReadPreconditionAsync(
    plan.Target, cancellationToken);

if (!FixedTimeEquals(current.ContentSha256,
    plan.Basis.ExpectedDigest) ||
    current.SnapshotId != plan.Basis.SourceSnapshot ||
    current.HasActiveHold)
{
    return ExecutionDecision.Blocked(
        "repair-plan-stale", current.SafeReference);
}

return ExecutionDecision.Proceed(plan.Sha256);
```

TOCTOU BOUNDARY

The check narrows the race but cannot create a distributed transaction. Owner-local idempotency closes the remaining retry window.

DRY RUN

Preview and execution share one planner

DryRun computes the exact target set, preconditions, expected owner calls, bytes and estimated load, then produces a plan digest. ExecuteAfterApproval must consume those same canonical bytes. A second planner run that changes the digest requires new approval.

DECISION LEDGER 42-20

Dry-run output	Why retained
target coordinates	prove scope
source/actual snapshot IDs	prove comparison basis
planned owner actions	review side effects
item/byte estimate	enforce blast limits
holds and blockers	fail before approval
plan digest + expiry	bind later execution

NO PREVIEW-ONLY LOGIC

Dry run and execution do not use separate selection implementations; only the final side-effect port differs.

BLAST RADIUS

Limits are executable policy, not a warning label

A plan declares maximum targets, bytes, age, sites and wall-clock duration. The executor stops before exceeding any limit. Partial completion is represented as an attempt with per-item receipts, then verified; it is never summarized as all repaired.

EXECUTABLE ASSET 42-J - LIMITS

```
public sealed record RepairLimits(
    int MaxItems,
    long MaxBytes,
    IReadOnlySet<SiteId> Sites,
    TimeSpan MaxDuration,
    Instant ExpiresAt)
{
    public void DemandAllows(RepairBatch batch, Instant now)
    {
        if (now >= ExpiresAt) throw Blocked("plan-expired");
        if (batch.Count > MaxItems) throw Blocked("item-limit");
        if (batch.Bytes > MaxBytes) throw Blocked("byte-limit");
        if (!batch.Sites.IsSubsetOf(Sites)) throw Blocked("site-limit");
    }
}
```

DEFAULT POSTURE

The reference policy defaults to one site and one semantic target. Broad jobs need elevated policy and dual control.

RISK POLICY

Approval strength follows consequence, not UI convenience

A single derived projection rebuild can use one privileged approver when policy permits. Re-admitting dead-lettered bytes, touching multiple sites, exceeding the default item limit or acting on sensitive tenants requires two distinct approvers and a change reference.

DECISION LEDGER 42-22

Risk class	Example	Minimum authorization
R1	one projection generation	requester + authorized approver
R2	one exact outbox redispach	distinct approver
R3	dead-letter replay	dual control
R4	multi-site/bulk or unknown	manual change window; dual control

NO SELF-APPROVAL

The requester, executor service identity and required human approver roles are checked independently.

APPROVAL MODEL

An approval is bound to digest, role and expiry

Free-text comments are audit context, not authority. The approval record binds subject, plan digest, policy, scope, authentication context and expiry. A deactivated actor or expired plan blocks execution even when an old approval row exists.

EXECUTABLE ASSET 42-K - APPROVAL RECORD

```
public sealed record Approval(  
    ApprovalId Id,  
    SubjectId Subject,  
    string Role,  
    string PlanSha256,  
    string Policy,  
    string ScopeSha256,  
    Instant ApprovedAt,  
    Instant ExpiresAt,  
    string AuthenticationContext,  
    string ReasonCode,  
    string ChangeReference);  
  
static bool IsDistinct(Approval a, RepairRequested r) =>  
    a.Subject != r.RequestedBy && a.Subject != r.ExecutorIdentity;
```

SECURITY STATUS

Identity provider, MFA and role binding evidence are PENDING and become Chapter 43-47 concerns.

DUAL CONTROL

Two approvals mean two independent identities

The policy engine rejects duplicated subjects, delegated service identities masquerading as humans and approvals over different plan digests. Both approvals must remain valid at execution time.

EXECUTABLE ASSET 42-L - DUAL-CONTROL GATE

```
public static void DemandDualControl(
    RepairPlan plan, RepairRequested request,
    IReadOnlyList<Approval> approvals, Instant now)
{
    var valid = approvals.Where(a =>
        a.PlanSha256 == plan.Sha256 &&
        a.ExpiresAt > now &&
        a.Subject != request.RequestedBy &&
        a.Subject != request.ExecutorIdentity)
        .DistinctBy(a => a.Subject)
        .ToArray();

    if (valid.Length < 2)
        throw AuthorizationDenied("dual-control-required");
}
```

AUDIT EXPECTATION

The record proves which identities approved which immutable plan; it does not copy authentication secrets.

REPAIRCONTROLDB

Control state is append-oriented and locally transactional

RepairControlDb contains no business aggregates and no message payload archive. One local transaction persists the case transition, immutable audit event and any execution intent. A worker leases that intent after commit.

DECISION LEDGER 42-25

Table	Key	Invariant
RepairCases	RepairCaseld	one current state/version
RepairCandidates	CandidateId	immutable comparison digest
RepairPlans	PlanId	unique plan SHA-256
RepairApprovals	ApprovalId	subject + exact plan
RepairAttempts	Caseld + attempt	append only
RepairAudit	AuditEventId	append only; hash chained
RepairIntents	IntentId	leased owner dispatch

LOCAL ATOMICITY

A state transition and its audit event commit together. The owner repair remains a separate idempotent operation.

PERSISTENCE MAPPING

Database constraints defend the repair key

The application checks idempotency before insert, but the database owns the final uniqueness guarantee. Concurrency tokens prevent two workers from advancing one case version. Plan and audit blobs are immutable after insertion.

EXECUTABLE ASSET 42-M - EF CORE CONSTRAINTS

```
modelBuilder.Entity<RepairCaseRow>(b =>
{
    b.ToTable("RepairCases");
    b.HasKey(x => x.RepairCaseId);
    b.Property(x => x.Version).IsConcurrencyToken();
    b.HasIndex(x => x.RepairKey).IsUnique();
    b.Property(x => x.RepairKey).HasMaxLength(64).IsRequired();
});

modelBuilder.Entity<RepairPlanRow>(b =>
{
    b.HasKey(x => x.RepairPlanId);
    b.HasIndex(x => x.PlanSha256).IsUnique();
    b.Property(x => x.CanonicalJson).IsRequired();
});
```

PROVIDER PROOF

Uniqueness and concurrency claims require the production database provider, not only EF's in-memory provider.

AUDIT LEDGER

Every meaningful transition becomes immutable evidence

Audit events name actor, authentication context, action, case, plan digest, owner receipt, before/after state and safe evidence references. A per-case hash chain makes omission or reordering detectable when exported.

EXECUTABLE ASSET 42-N - CHAINED AUDIT EVENT

```
public sealed record RepairAuditEvent(
    Guid AuditEventId,
    Guid RepairCaseId,
    long Sequence,
    string EventType,
    string Actor,
    string PlanSha256,
    string? OwnerReceipt,
    string PreviousSha256,
    string EventSha256,
    DateTimeOffset RecordedAtUtc,
    IReadOnlyList<string> SafeEvidenceReferences);

// EventSha256 = SHA256(JCS(event fields excluding EventSha256))
```

AUDIT IS NOT LOGGING

Ordinary logs help operate the job; the durable repair audit is the authoritative administrative history.

AUDIT EVENTS

The ledger records decisions as well as attempts

A repair that never executes still needs evidence explaining why. Comparison, plan expiry, authorization denial, stale precondition and manual review are first-class audit outcomes rather than missing log lines.

DECISION LEDGER 42-28

Event	Required evidence
CandidateObserved	snapshot IDs + comparison digest
PlanFrozen	canonical plan digest + limits
ApprovalRecorded	subject + policy + expiry
ExecutionBlocked	fresh precondition + reason
OwnerActionAttempted	idempotency key + receipt/uncertainty
VerificationRecorded	new comparison digest + verdict
CaseClosed	terminal state + safe summary

SENSITIVE-DATA BOUNDARY

Evidence references are allow-listed identifiers and digests; message bodies, tokens and connection strings are excluded.

OWNER-LOCAL PORTS

Repair control calls capabilities, never tables

Each target owner exposes one narrow port with the common repair key and exact owner-specific request. The response is a durable receipt or an explicit unknown outcome. No generic ExecuteSql or ReplayAnything endpoint exists.

EXECUTABLE ASSET 42-O - TYPED OWNER CAPABILITIES

```
public interface IReportingRepairPort
{
    Task<OwnerRepairReceipt> StartGenerationAsync(
        StartProjectionGeneration request, RepairContext context, CancellationToken ct);
}
public interface IOutboxRepairPort
{
    Task<OwnerRepairReceipt> ScheduleRedispatchAsync(
        ScheduleExactRedispatch request, RepairContext context, CancellationToken ct);
}
public interface IPoisonReplayPort
{
    Task<OwnerRepairReceipt> RequestApprovedReplayAsync(
        ReplayPoisonEvidence request, RepairContext context, CancellationToken ct);
}
```

NO GENERIC ESCAPE HATCH

Adding a new repair kind requires a new typed contract, threat review, policy and verification oracle.

OWNER RECEIPT

A durable owner receipt closes retries, not verification

The owner stores a receipt under the repair key in the same local transaction that schedules or applies its repair action. Repeating the call returns the same receipt. The control plane still verifies the declared invariant afterward.

EXECUTABLE ASSET 42-P - RECEIPT CONTRACT

```
public sealed record OwnerRepairReceipt(  
    string RepairKey,  
    string Owner,  
    string Action,  
    int Generation,  
    OwnerRepairOutcome Outcome,  
    string OwnerStateVersion,  
    string ReceiptSha256,  
    DateTimeOffset RecordedAtUtc,  
    string SafeReference);  
  
public enum OwnerRepairOutcome  
{  
    Accepted, AlreadyAccepted, Rejected, Unknown  
}
```

UNKNOWN STAYS UNKNOWN

A timeout after the owner commit triggers receipt lookup with the same repair key; it does not generate a second key.

MISSING PROJECTION

Detection compares retained input evidence with projection slots

A MissingProjection candidate requires proof that the authoritative input exists, the Reporting ledger or archive retained it, the active generation lacks the corresponding slot after its normal lateness budget, and no active gap or rebuild already owns the coordinate.

DECISION LEDGER 42-31

Predicate	Required result
owner fact coordinate	present and digest-valid
Reporting retained input	present or safely fetchable
active projection slot	absent or blocking failure
normal lateness window	elapsed
gap/rebuild ownership	none active
projection contract	supported by selected reducer

REPAIRABLE BOUNDARY

If the retained input is missing or its digest is invalid, the verdict is UnsafeUnknown; projection repair cannot invent source history.

PROJECTION COMPARISON

Expected slots are derived from the declared policy

The verifier computes the expected evidence-slot set from the policy and retained ledger, then compares it with one generation. It does not infer completion from a headline status field.

EXECUTABLE ASSET 42-Q - SLOT ORACLE

```
public ProjectionComparison Compare(
    RetainedWorkflowLedger source,
    OperatorExperienceGeneration actual,
    ProjectionPolicy policy)
{
    var expected = policy.RequiredSlots(source.Cycles)
        .ToDictionary(x => x.Key, x => x.ContentSha256);
    var observed = actual.Slots
        .ToDictionary(x => x.Key, x => x.ContentSha256);

    return ProjectionComparison.Create(
        Missing: expected.Keys.Except(observed.Keys),
        Changed: expected.Keys.Intersect(observed.Keys)
            .Where(k => expected[k] != observed[k]),
        Extra: observed.Keys.Except(expected.Keys));
}
```

NO STATUS ORACLE

Completed is an output of satisfied slot evidence, not an input trusted by the repair tool.

NEW GENERATION

Projection repair rebuilds beside the failed generation

Reporting creates a replacement generation with the repair key, policy identity, source snapshot and parent failure reference. It replays retained inputs into isolated tables. The failed generation remains addressable and unchanged.

EXECUTABLE ASSET 42-R - OWNER-LOCAL GENERATION START

```
await using var tx = await reporting.Database.BeginTransactionAsync(ct);

var prior = await generations.RequireAsync(request.FailedGenerationId, ct);
var receipt = await repairReceipts.FindAsync(context.RepairKey, ct);
if (receipt is not null) return receipt.ToContract();

var next = ProjectionGeneration.StartRepair(
    request.ReplacementGenerationId,
    prior.Id,
    request.Policy,
    request.SourceSnapshot,
    context.RepairKey);

reporting.Add(next);
reporting.Add(RepairReceipt.Accepted(context, next.Id));
await reporting.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

APPEND AND SUPERSEDE

The original Failed generation stays terminal. Only an independently verified generation can later become active.

REBUILD WORKER

Retained inputs pass through the production reducer

The rebuild worker reads the bounded retained ledger in canonical order and invokes the same versioned reducer used by live projection. It writes only the replacement generation and checkpoints every batch under the repair key.

EXECUTABLE ASSET 42-S - DETERMINISTIC REBUILD

```
await foreach (var batch in ledger.ReadAsync(
    request.SourceSnapshot, request.Policy, limits, ct))
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var reducer = scope.ServiceProvider
        .GetRequiredKeyedService<IProjectionReducer>(request.Policy);
    var unit = scope.ServiceProvider.GetRequiredService<ReportingUnit>();

    foreach (var retained in batch.OrderBy(CanonicalEventOrder.Instance))
        await reducer.ApplyAsync(unit, request.GenerationId, retained, ct);

    await unit.CommitCheckpointAsync(
        request.GenerationId, batch.LastCoordinate, context.RepairKey, ct);
}
```

NO ALTERNATE REPAIR REDUCER

A special script that bypasses the production reducer would create an untested second semantics path.

SHADOW VERIFICATION

The replacement stays invisible until reconciliation passes

Reads continue from the current active generation while the replacement is built. Verification compares slot coverage, semantic digests, invariants, correction history and watermark. Only an InSync verdict permits activation.

DECISION LEDGER 42-35

Check	Activation requirement
required slots	complete
slot digests	equal retained inputs
cycle invariants	valid original/correction histories
blocking failures	none active
watermark	declared snapshot fully consumed
repair lineage	parent + repair key retained

SERVING ISOLATION

Building or failing a shadow generation cannot replace the active operator experience.

GENERATION CUTOVER

Activation is one Reporting transaction

After verification, Reporting atomically marks the replacement active, records the prior active generation as superseded for serving, and stores the verification digest. It does not delete either generation or their failure evidence.

EXECUTABLE ASSET 42-T - ATOMIC ACTIVATION

```
await using var tx = await db.Database.BeginTransactionAsync(ct);
var current = await generations.LockActiveAsync(experienceId, ct);
var replacement = await generations.LockAsync(replacementId, ct);

replacement.DemandVerified(verificationSha256);
current.StopServing(replacement.Id, now);
replacement.Activate(now, repairKey);

await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

SERVING POINTER ONLY

Cutover changes which derived generation serves reads. It does not alter any owner fact or the terminal history of the failed generation.

PROJECTION ROLLBACK

A bad cutover is corrected by another generation decision

If post-cutover monitoring discovers a defect, operators stop new repair jobs and select a verified serving generation through another audited plan. They do not edit checkpoints or delete the bad generation. Any source correction remains an owner fact, not a projection rollback.

DECISION LEDGER 42-37

Situation	Safe response	Forbidden
replacement fails before cutover	mark generation Failed	activate partially
verification inconclusive	ManualReview	assume equal
defect after cutover	audited serving-generation plan	delete evidence
source fact wrong	owner correction workflow	projection edits owner meaning

FORWARD-ONLY HISTORY

Rollback restores a serving choice; it never pretends the repair attempt did not occur.

PROJECTION COMPONENT TESTS

A real provider proves generation isolation and cutover

The component suite uses the production database provider to exercise unique active-generation constraints, checkpoints, crash recovery and transaction rollback. It compares the rebuilt generation with the golden retained ledger.

EXECUTABLE ASSET 42-U - GENERATION PROOF

```
[Fact]
public async Task PROJ_REPAIR_001_rebuilds_new_generation_only()
{
    await lab.SeedFailedGenerationAsync(GENERATION_ID);
    var first = await lab.StartRepairAsync(REPAIR_IDEMPOTENCY_KEY);
    var duplicate = await lab.StartRepairAsync(REPAIR_IDEMPOTENCY_KEY);

    first.ReceiptSha256.Should().Be(duplicate.ReceiptSha256);
    (await lab.GenerationAsync(GENERATION_ID)).State.Should().Be("Failed");
    (await lab.GenerationAsync(REPLACEMENT_GENERATION_ID))
        .ParentGenerationId.Should().Be(GENERATION_ID);
    (await lab.ActiveCountAsync(EXPERIENCE_ID)).Should().Be(1);
}
```

EVIDENCE STATUS

The listing specifies the proof. It remains PENDING until provider output and retained database evidence exist.

STUCK OUTBOX

Old Pending is not enough to justify redispach

A StuckOutbox candidate requires a due row, expired or absent lease, intact frozen envelope bytes, supported contract, no administrative hold and a publisher state that is not already confirmed. Ambiguous confirm status triggers owner reconciliation before any send.

DECISION LEDGER 42-39

Outbox evidence	Candidate rule
status Pending + due	necessary, not sufficient
lease	expired and fencing token not active
frozen bytes	present and digest matches
publisher outcome	not confirmed; ambiguity classified
message identity	original MessageId retained
hold/supersession	none active

NO DUPLICATE BUSINESS FACT

Repair schedules another transport attempt for the existing outbox record; it never inserts a replacement public event.

OUTBOX CLASSIFICATION

Publisher state determines the next safe action

The owner classifies the record from durable outbox state and broker-confirm evidence. Confirmed means no repair. Due with no active lease can be scheduled. Unknown confirm must first pass the Chapter 26 reconciliation path. Invalid bytes or unsupported contracts block permanently.

DECISION LEDGER 42-40

Classification	Repair action
Confirmed	close candidate InSync
DueUnleased	schedule exact redispach
ConfirmUnknown	reconcile owner publisher attempt
ActiveLease	Blocked; wait or investigate lease
InvalidFrozenEnvelope	ManualReview; never mutate bytes
Held	Blocked by administrative policy

DELIVERY LIMIT

A publisher confirm can prove broker acceptance; it cannot prove every consumer completed its local effect.

OWNER-LOCAL REDISPATCH

Scheduling and receipt commit in the outbox owner's database

The owner locks the existing row, validates the precondition version and frozen-body digest, inserts a repair receipt and moves the row to a due state under a new transport attempt generation. Message identity and public bytes remain unchanged.

EXECUTABLE ASSET 42-V - EXACT REDISPATCH SCHEDULE

```
await using var tx = await ownerDb.Database.BeginTransactionAsync(ct);
var existing = await receipts.FindAsync(context.RepairKey, ct);
if (existing is not null) return existing.ToContract();

var row = await outbox.LockAsync(request.OutboxId, ct);
row.DemandVersion(request.ExpectedVersion);
row.DemandFrozenDigest(request.EnvelopeSha256);
row.ScheduleRedispach(
    nextAttemptGeneration: row.AttemptGeneration + 1,
    dueAtUtc: clock.GetUtcNow(),
    repairKey: context.RepairKey);

var receipt = RepairReceipt.Accepted(context, row.Id, row.Version);
ownerDb.Add(receipt);
await ownerDb.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

SAME ENVELOPE

Only transport-attempt metadata changes. MessageId, routing key, content type and frozen body bytes remain identical.

PUBLISHER ATTEMPT

Repair reuses the normal confirm-aware publisher

The dispatcher does not get a privileged broker shortcut. It acquires the normal owner lease, publishes the retained properties and body, waits for confirm, and records Confirmed, Rejected or Unknown using the same Chapter 26 state machine.

EXECUTABLE ASSET 42-W - FROZEN PUBLISH

```
var publication = new FrozenPublication(  
    row.Exchange,  
    row.RoutingKey,  
    row.MessageId,  
    row.ContentType,  
    row.Headers.WithTransportAttempt(  
        row.AttemptGeneration, row.RepairKey),  
    row.EnvelopeBytes);  
  
var outcome = await publisher.PublishWithConfirmAsync(publication, ct);  
await attempts.RecordAsync(  
    row.Id, row.LeaseToken, row.AttemptGeneration,  
    outcome, clock.UtcNow(), ct);
```

TRANSPORT METADATA

Repair identifiers live in allow-listed transport headers and audit records; they do not alter the signed/canonical public envelope.

OUTBOX VERIFICATION

Verification proves the declared transport invariant only

The post-repair oracle checks that one frozen message identity remains, a confirm-aware attempt is durably recorded and the outbox state converged. Consumer completion requires separate inbox or workflow evidence and is never inferred from the publisher row.

DECISION LEDGER 42-43

Claim	Evidence	Not proved
same public intent	same outbox ID, MessageId and bytes	consumer effect
broker accepted	publisher confirm receipt	all queues retained
owner state converged	outbox state/version	projection completed
retry idempotent	same repair receipt	exactly-once transport

BOUNDED VERDICT

The repair case may succeed for RedispatchOutbox while the end-to-end workflow remains Delayed and visible as such.

OUTBOX FAULT TESTS

Unknown confirms and worker crashes preserve one public intent

The broker-backed laboratory disconnects after publish, crashes after owner receipt, races two repair workers and redelivers the same control request. The assertions count business message identities, frozen-body digests and repair receipts rather than method calls.

EXECUTABLE ASSET 42-X - OUTBOX FAULT PROOF

```
[Fact]
public async Task OUTBOX_REPAIR_004_unknown_confirm_reconciles_before_retry()
{
    broker.DisconnectAfterAcceptingPublish();
    await repair.ExecuteAsync(plan, Cancellation.Token.None);
    await worker.RestartAsync();
    await repair.ExecuteAsync(plan, Cancellation.Token.None);

    (await owner.OutboxRowsAsync(messageId)).Should().ContainSingle();
    (await owner.DistinctEnvelopeDigestsAsync(messageId)).Should().Equal(expectedDigest);
    (await owner.RepairReceiptsAsync(REPAIR_IDEMPOTENCY_KEY)).Should().ContainSingle();
    (await broker.PublicMessageIdsAsync(messageId)).Distinct().Should().ContainSingle();
}
```

AT-LEAST-ONCE TRUTH

More than one delivery attempt may be observed; downstream inbox idempotency remains mandatory.

DEAD-LETTER EVIDENCE

The terminal record is an input to diagnosis, not a replay queue

Chapter 29 already retains PoisonMessage, ReplayRequest and ReplayDispatch. Chapter 42 consumes that evidence through a safe contract. It does not drain a queue by copying arbitrary broker messages or erase the original terminal record.

DECISION LEDGER 42-45

Required poison evidence	Purpose
original MessageId + body digest	identity
frozen bytes/properties reference	exact replay input
consumer + contract identity	routing and compatibility
terminal reason and attempts	diagnosis
first/last observed time	age and incident scope
existing replay generations	idempotency/history

RETENTION RULE

A successful replay appends a disposition and outcome; it does not delete poison evidence.

REPLAY ELIGIBILITY

The cause must be addressed before bytes are re-admitted

A dead-letter candidate is executable only when the contract is supported, the consumer defect or configuration issue has an identified remediation, the normal route is healthy, the bytes still pass immutable admission checks and policy permits replay.

DECISION LEDGER 42-46

Condition	If false
reason is replayable	Blocked/ManualReview
remediation reference approved	Blocked
contract accepted by current consumer	UnsafeUnknown
frozen digest intact	Failed evidence integrity
normal exchange/route available	Delayed; no direct invocation
no hold or prior success	Blocked/InSync

HOSTILE INPUT

Authentication, authorization, schema or integrity rejection is not made replayable merely because an operator asks.

REUSE CHAPTER 29

Repair control delegates to the established replay aggregate

The repair plan maps to the existing ReplayRequest and dual-approval workflow. It does not introduce another replay state machine. The common repair key becomes external idempotency context; the messaging subsystem remains owner of poison and dispatch records.

EXECUTABLE ASSET 42-Y - REPLAY ADAPTER

```
public async Task<OwnerRepairReceipt> RequestApprovedReplayAsync(
    ReplayPoisonEvidence request,
    RepairContext context,
    CancellationToken ct)
{
    var existing = await receipts.FindAsync(context.RepairKey, ct);
    if (existing is not null) return existing;

    var replay = await replayRequests.CreateOrGetAsync(
        request.PoisonMessageId,
        request.RemediationReference,
        request.Approvals,
        externalIdempotencyKey: context.RepairKey,
        ct);
    return await receipts.StoreAcceptedAsync(context, replay.Id, ct);
}
```

ONE REPLAY AUTHORITY

Chapter 42 coordinates the decision; Chapter 29's replay engine still owns re-admission mechanics and provenance.

NORMAL-ROUTE DISPATCH

Replay publishes exact bytes through the production exchange

After approvals, ReplayDispatch republishes the retained body and public identity to the normal exchange/routing key. A new replay generation and repair reference are transport provenance. Consumers execute their normal admission and inbox paths.

EXECUTABLE ASSET 42-Z - RE-ADMISSION PUBLICATION

```
var publication = poison.ToFrozenPublication() with
{
    Headers = poison.TransportHeaders
        .With("x-nexus-replay-generation", dispatch.Generation)
        .With("x-nexus-repair-id", repairCaseId.ToString())
};

publication.MessageId.Should().Be(poison.OriginalMessageId);
SHA256.HashData(publication.Body)
    .Should().Equal(poison.BodySha256);

await publisher.PublishWithConfirmAsync(publication, ct);
```

NO DIRECT HANDLER CALL

Normal routing, admission, authorization, inbox idempotency, transaction and acknowledgement remain in force.

DEAD-LETTER VERIFICATION

A confirm is necessary but not the final repair result

Verification follows the replay into the intended consumer through safe receipt/outcome evidence. Duplicate receipt is a valid idempotent success. Another terminal rejection is a failed repair attempt with new evidence, not permission for an unbounded replay loop.

DECISION LEDGER 42-49

Observed outcome	Repair verdict
consumer durable success	Succeeded
matching inbox receipt already exists	Succeeded/AlreadyApplied
publisher confirm; no consumer evidence yet	Verifying/Delayed
new terminal poison record	Failed; retain both generations
outcome unavailable after deadline	ManualReview
digest/identity differs	FailedIntegrity; stop

BOUNDED RETRY

A failed replay requires a new comparison and plan; the worker never loops poison indefinitely.

DEAD-LETTER FAULT TESTS

Replay proves provenance, bytes and inbox convergence

The broker-backed suite injects duplicate dispatch, publisher disconnect, consumer crash after commit and a still-invalid contract. It asserts exact frozen bytes, one consumer effect and retained poison/replay generations.

EXECUTABLE ASSET 42-AA - REPLAY PROOF

```
[Fact]
public async Task DLQ_REPAIR_006_duplicate_dispatch_has_one_effect()
{
    await replay.ApproveAndDispatchTwiceAsync(plan);
    await consumer.DrainAsync();

    var attempts = await broker.DeliveriesAsync(originalMessageId);
    attempts.Should().HaveCountGreaterThanOrEqualTo(2);
    attempts.Select(x => x.BodySha256).Distinct().Should().Equal(originalDigest);
    (await consumer.InboxReceiptsAsync(originalMessageId)).Should().ContainSingle();
    (await consumer.BusinessEffectsAsync(originalMessageId)).Should().ContainSingle();
    (await poison.EvidenceGenerationsAsync(originalMessageId)).Should().NotBeEmpty();
}
```

EVIDENCE STATUS

The scenario remains PENDING until broker, consumer and database artifacts are retained together.

EXACTLY-ONCE BOUNDARY

Repair idempotency does not create exactly-once transport

The control plane prevents duplicate repair intent, owners deduplicate repair actions, and consumers deduplicate message effects. Networks can still duplicate or obscure attempts. The system converges through layered idempotency and reconciliation, not a global exactly-once claim.

DECISION LEDGER 42-51

Layer	Idempotency coordinate	Residual uncertainty
control	RepairKey	response loss
owner action	RepairKey + owner receipt	owner timeout
publication	MessageId + frozen digest	confirm unknown
consumer	consumer + MessageId	ack loss
projection	generation + evidence slot	freshness lag

HONEST GUARANTEE

Repeated attempts are expected; repeated business meaning is prevented and measured at owner-local boundaries.

REPEATED CONTROL REQUEST

Same key and same plan return the existing resource

The administrative POST is asynchronous. A network retry with identical Idempotency-Key and canonical plan returns the existing case location and current representation. It does not enqueue another job.

EXECUTABLE ASSET 42-AB - IDEMPOTENT CREATE

```
var existing = await cases.FindByRepairKeyAsync(repairKey, ct);
if (existing is not null)
{
    if (!FixedTimeEquals(existing.PlanSha256, submitted.PlanSha256))
        return Results.Conflict(Problems.KeyReusedWithDifferentPlan());

    return Results.Accepted(
        $"/admin-api/v1/repair-jobs/{existing.Id}",
        mapper.ToV1(existing));
}

var created = RepairCase.Create(repairKey, submitted, actor, clock.UtcNow());
await unit.CommitAsync(created, ct);
return Results.Accepted($"/admin-api/v1/repair-jobs/{created.Id}", mapper.ToV1(created));
```

HTTP BOUNDARY

POST is made retry-safe by an application idempotency contract; HTTP itself does not make POST idempotent.

CONFLICTING REQUEST

A reused key cannot silently authorize a new target

If any canonical plan byte differs, the server returns a stable problem type and the existing repair reference. It does not overwrite the plan, merge scopes or choose the newer request.

DECISION LEDGER 42-53

Condition	HTTP outcome	Side effects
new key + valid dry-run plan	202 Accepted	case stored
same key + same plan	202 existing resource	none new
same key + different plan	409 key-reused	none
expired candidate	412 stale-evidence	none
unauthorized actor	403	none
unsupported contract	422	none

PROBLEM DETAILS

Machine-readable problem types carry safe references, never payload bodies or credential details.

CRASH RECOVERY

Every uncertain edge resolves from durable local evidence

A worker crash before owner call leaves the intent leased and recoverable. A crash after an owner commit but before receipt storage triggers lookup by RepairKey. A crash during verification reruns the read-only oracle. None requires inventing a new plan.

DECISION LEDGER 42-54

Crash point	Recovery source	Next action
before intent commit	RepairControlDb	no job exists
after intent commit	leased intent	resume after lease expiry
after owner commit	owner receipt by RepairKey	record same receipt
after broker publish	confirm/owner attempt evidence	reconcile first
during verification	snapshots + plan	re-run comparer

NO BLIND RETRY

Unknown owner or broker outcomes enter resolution; they do not immediately repeat the side effect.

LEASES AND FENCING

At most one live executor generation owns a case

Repair workers lease execution intents with a monotonically increasing fencing token. Every state update and owner call carries that token. An expired worker cannot overwrite a newer executor's result.

EXECUTABLE ASSET 42-AC - FENCED LEASE

```
var lease = await intents.TryLeaseAsync(
    intentId, workerId, now, duration: TimeSpan.FromSeconds(30), ct);
if (lease is null) return;

var context = new RepairContext(
    repairKey, repairCaseId, attempt, lease.FencingToken, planSha256);

var receipt = await owner.ExecuteAsync(action, context, ct);
await intents.CompleteAsync(
    intentId, lease.FencingToken, receipt.ReceiptSha256, ct);
```

LEASE LIMIT

A lease coordinates workers; owner-local RepairKey uniqueness remains the side-effect idempotency boundary.

CANCELLATION AND PAUSE

Stop scheduling without claiming to undo effects

Operators can pause a batch, expire an unexecuted plan or close a compared candidate. After an owner action becomes possible, cancellation becomes a request to stop future work and verify what happened. Published bytes and committed generations are not recalled.

DECISION LEDGER 42-56

Current state	Permitted control
Compared	close without plan
Planned	expire/cancel plan
Approved	pause before lease
Executing	request stop; resolve owner outcome
Verifying	pause polling; retain status
Succeeded/Failed/ManualReview	no mutation; create new case if needed

NO UNDO BUTTON

Repair is forward recovery through explicit evidence, consistent with Chapter 40's compensation model.

ADMINISTRATIVE API

Read, plan, approve and execute are separate routes

The API keeps read-only inspection apart from privileged transitions. Planning consumes a candidate, approval binds a digest, and execution requires both the plan digest and an Idempotency-Key. GET status never triggers work.

EXECUTABLE ASSET 42-AD - ROUTE INVENTORY

```
GET /admin-api/v1/repair-candidates/{candidateId}
POST /admin-api/v1/repair-plans:dry-run
POST /admin-api/v1/repair-jobs
POST /admin-api/v1/repair-jobs/{jobId}:approve
POST /admin-api/v1/repair-jobs/{jobId}:execute
POST /admin-api/v1/repair-jobs/{jobId}:pause
GET /admin-api/v1/repair-jobs/{jobId}
GET /admin-api/v1/repair-jobs/{jobId}/audit
```

Required create/execute headers:
Idempotency-Key: <64 lowercase hex>
If-Match: "<plan-sha256>"
Content-Type: application/vnd.nexus1.repair-plan.v1+json

NOT A PUBLIC BUSINESS API

These routes are administrative control surfaces and add zero public commands to the business choreography.

ADMINISTRATIVE CONTRACT

The representation separates state, evidence and available actions

The response exposes safe identifiers, plan identity, attempts, approvals, verification and policy-derived actions. It does not return message bodies, secrets or raw exception text.

EXECUTABLE ASSET 42-AE - V1 RESPONSE

```
public sealed record RepairJobV1(
    Guid RepairJobId,
    Guid RepairCaseId,
    string Kind,
    string State,
    string RepairKey,
    string PlanSha256,
    CandidateSummaryV1 Candidate,
    IReadOnlyList<ApprovalSummaryV1> Approvals,
    IReadOnlyList<AttemptSummaryV1> Attempts,
    VerificationSummaryV1? Verification,
    IReadOnlyList<ActionLinkV1> Actions,
    DateTimeOffset RepresentationObservedAtUtc,
    string RepresentationEtag);
```

MEDIA TYPE

application/vnd.nexus1.repair-job.v1+json

AUTHORIZATION MATRIX

Scopes are narrow and action-specific

Viewing candidate metadata, preparing a plan, approving a plan and executing it are distinct permissions. The API enforces site/tenant scope in addition to role. Service identities cannot substitute for human approvals when policy demands humans.

DECISION LEDGER 42-59

Route action	Required scope	Additional gate
read	Repair.Read	site scope
dry run	Repair.Plan	candidate visibility
create job	Repair.Request	reason + change reference
approve R1/R2	Repair.Approve	distinct actor
approve R3/R4	Repair.Approve.HighRisk	dual control
execute	Repair.Execute	valid plan + approvals
audit export	Repair.Audit.Read	purpose recorded

DENY BY DEFAULT

Unknown repair kinds, sites or policies have no route action and produce no owner call.

HTTP CONCURRENCY

ETags prevent stale administrative transitions

The representation ETag covers case version and plan digest. Approval and execute requests carry If-Match. A stale operator screen receives 412 and must GET the current state before deciding again.

EXECUTABLE ASSET 42-AF - CONDITIONAL TRANSITION

```
app.MapPost("/admin-api/v1/repair-jobs/{id}:execute",
    async (Guid id, HttpRequest http, ExecuteRepair handler, CancellationToken ct) =>
    {
        var expected = EntityTag.RequireStrong(http.Headers.IfMatch);
        var key = RepairKey.Require(http.Headers["Idempotency-Key"]);
        var outcome = await handler.HandleAsync(id, expected, key, ct);
        return RepairHttp.Map(outcome);
    })
    .RequireAuthorization("Repair.Execute")
    .Accepts<ExecuteRepairV1>(ADMIN_MEDIA_TYPE)
    .Produces<RepairJobV1>(StatusCodes.Status202Accepted);
```

NO GET SIDE EFFECT

Polling or refreshing a job can never renew approval, acquire a lease or execute a plan.

HTTP OUTCOMES

Problems preserve the difference between denial, drift and failure

The API maps domain outcomes explicitly. A stale plan is not an authorization failure; an unavailable owner is not an invalid contract; an unknown execution result is not success.

DECISION LEDGER 42-61

Outcome	Status	Problem type suffix
accepted/existing	202	none; Location returned
unauthenticated	401	authentication-required
not authorized	403	repair-not-authorized
not found/hidden	404	repair-not-found
stale ETag/precondition	412	repair-precondition-failed
key reused differently	409	idempotency-key-reused
unsupported plan	422	repair-plan-invalid
owner unavailable	503	repair-owner-unavailable

SAFE DETAIL

Problem instances include correlation and repair references but not broker payloads, SQL or stack traces.

ADMINISTRATIVE EXPERIENCE

The console displays gates, not a tempting replay button

The operator view presents candidate basis, expiry, dry-run digest, blast limits, approval requirements, owner action and verification separately. High-risk execution requires a fresh confirmation step that names the exact scope.

DECISION LEDGER 42-62

Panel	Required content
candidate	kind, reason, snapshot age, evidence references
plan	targets, bytes, limits, digest, expiry
approval	required roles, distinct identities, remaining expiry
execution	owner receipt or explicit unknown
verification	new diff and bounded verdict
history	all attempts and generations

HUMAN FACTORS

The UI never defaults a broad scope, hides dry-run changes or relabels ManualReview as Failed/Complete for visual simplicity.

REFRESH SEMANTICS

Administrative polling observes; it never repairs

Job status uses bounded GET polling with ETags and server-owned Retry-After. The client rejects older representation versions and stops on terminal states. A hard reload recovers from the route and RepairControlDb, not local browser state.

EXECUTABLE ASSET 42-AG - STATUS POLLING

```
load(id: string, etag?: string) {
  return this.http.get<RepairJobV1>({
    url: `/admin-api/v1/repair-jobs/${id}`,
    headers: { observe: 'response', etag: etag ? { 'If-None-Match': etag } : {} }
  });
}

// UI rule:
// GET may refresh state and permitted action links.
// Only an explicit, authorized POST can approve or execute.
```

CHAPTER 41 CONTINUITY

The same GET-only refresh principle now protects the privileged repair console.

OBSERVABILITY

Metrics show pressure and outcomes without unbounded identities

Repair telemetry separates candidates, plans, attempts and verification. Repair IDs, message IDs and actor IDs belong in traces or safe logs, not metric labels. Alerts distinguish a backlog of candidates from a backlog of approved executions.

DECISION LEDGER 42-64

Signal	Bounded dimensions
repair_candidates_total	kind, verdict, reasonCode
repair_jobs_current	kind, state, riskClass
repair_attempts_total	kind, owner, outcome
repair_verification_seconds	kind, verdict
repair_plan_expired_total	kind, riskClass
repair_owner_unknown_total	owner, action

NO VANITY METRIC

Repaired count alone can reward unsafe activity; success rate is paired with recurrence, manual-review and stale-plan rates.

TRACE CONTINUITY

Repair causality is explicit but separate from business causality

Administrative trace context carries RepairCaseld, PlanSha256 and owner action. Replayed public messages keep the original business correlation and causation coordinates; repair provenance is added as transport/audit context rather than replacing them.

DECISION LEDGER 42-65

Coordinate	Retained from	Purpose
business correlation	original envelope	workflow history
business causation	original envelope	fact lineage
RepairCaseld	control plane	administrative history
PlanSha256	canonical plan	authorization identity
repair attempt	control/owner receipt	retry history
trace/span IDs	current execution	diagnostics only

NO LINEAGE OVERWRITE

An operator action cannot become the business cause of an already existing fact.

PRIVACY AND REDACTION

Evidence is sufficient without duplicating sensitive payloads

The control plane stores semantic keys, digests, contract identities, safe references and bounded reason codes. Raw event bodies remain in their owning retention system. Audit exports apply allow-list serialization and purpose-bound access.

DECISION LEDGER 42-66

Data	RepairControlDb	Telemetry
message body	reference only	never
message/repair ID	yes	trace/log only
actor subject	audit pseudonymous ID	no metric label
approval reason	bounded code + protected note	code only
connection/token	never	never
payload digest	yes	trace/log when needed

ERASURE BOUNDARY

Retention and deletion policy for administrative evidence must preserve required audit while honoring applicable data obligations; legal decisions remain environment-specific.

CAPACITY

Repair traffic cannot starve the normal path

Owner-local repair endpoints use separate concurrency limits, low-priority queues where supported and explicit budgets. A circuit breaker pauses repair when normal-path latency, broker pressure or database health crosses policy thresholds.

DECISION LEDGER 42-67

Resource	Repair limit
comparison readers	bounded page size and deadlines
projection rebuild	batch size + checkpoint + throttle
outbox redispach	owner rate limit below normal publishing
dead-letter replay	single/bounded batch; consumer health gate
RepairControlDb	lease batch and retention partitions
operator API	per-actor and per-site quotas

BACKPRESSURE RULE

When normal traffic is unhealthy, automated repair pauses and remains visible; it does not compete more aggressively.

SCHEDULER

A bounded supervisor advances durable steps

The scheduler selects due intents, acquires a lease, validates the plan, invokes one owner action, stores its outcome and enqueues verification. Each batch creates a dependency-injection scope and honors shutdown cancellation.

EXECUTABLE ASSET 42-AH - BOUNDED WORKER

```
protected override async Task ExecuteAsync(CancellationTokens stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
        await using var scope = scopeFactory.CreateAsyncScope();
        var runner = scope.ServiceProvider.GetRequiredService<RepairBatchRunner>();
        var result = await runner.RunDueAsync(maxItems: 20, stoppingToken);

        var delay = result.HasMore ? TimeSpan.Zero : TimeSpan.FromSeconds(2);
        await timeProvider.Delay(delay, stoppingToken);
    }
}

public override Task StopAsync(CancellationTokens cancellationToken) =>
    base.StopAsync(cancellationToken);
```

SHUTDOWN BOUND

The host stops taking new leases and lets the platform cancellation bound the current owner call; unresolved outcomes remain durable.

RETRY POLICY

Only explicitly transient control operations retry automatically

Comparison reads and status lookups may retry bounded transient faults. Owner execution first resolves by RepairKey before repeating. Invalid plans, authorization failures, integrity failures and policy blocks are permanent for that attempt.

DECISION LEDGER 42-69

Failure	Automatic behavior
comparison 503/timeout	bounded jittered retry
owner response lost	lookup receipt by RepairKey
publisher confirm unknown	owner reconciliation
409/412 drift	new comparison required
401/403	stop; no credential refresh loop
integrity/contract failure	Failed/ManualReview

RETRY BUDGET

The plan expiry and attempt budget cap automation even when individual failures appear transient.

VERIFICATION ORACLE

Success is a new observation against the declared invariant

Each repair kind supplies a verification oracle. The oracle is read-only, versioned and returns its own snapshot/digest. The executor cannot fabricate success from its own receipt.

EXECUTABLE ASSET 42-AI - VERIFICATION CONTRACT

```
public interface IRepairVerificationOracle
{
    RepairKind Kind { get; }
    Task<VerificationResult> VerifyAsync(
        CanonicalRepairPlan plan,
        OwnerRepairReceipt ownerReceipt,
        VerificationBudget budget,
        CancellationToken ct);
}

public sealed record VerificationResult(
    ComparisonVerdict Verdict,
    string SnapshotId,
    string ComparisonSha256,
    string ReasonCode,
    IReadOnlyList<string> SafeEvidenceReferences);
```

INDEPENDENT EVIDENCE

A Succeeded transition requires Candidate/InSync semantics from the oracle, not merely HTTP 200 from an owner adapter.

VERIFICATION DELAY

Asynchronous convergence remains visible after execution

A successful owner receipt can precede the observable invariant. The job remains Verifying with a policy deadline and bounded polling advice. It becomes ManualReview when evidence stays unavailable beyond the deadline; it does not silently time out to Succeeded.

DECISION LEDGER 42-71

Phase	Displayed state	Operator meaning
owner accepted	Executing -> Verifying	action durable
target not yet converged	Verifying/Delayed	wait within policy
target InSync	Succeeded	declared invariant proved
new mismatch	Failed	repair did not converge
evidence unavailable	ManualReview	outcome unknown

NO PREMATURE GREEN

Owner acceptance and repair success are separate fields in both storage and API contracts.

BATCH SEMANTICS

Bulk work is a set of independently evidenced items

A batch plan freezes a bounded list of item plans and a Merkle-like root digest. Each item has its own RepairKey, receipt and verification. The batch summary never erases partial failure or manual review.

EXECUTABLE ASSET 42-AJ - BATCH SUMMARY

```
public sealed record RepairBatchResult(  
    Guid BatchId,  
    string BatchPlanSha256,  
    int Planned,  
    int Succeeded,  
    int Failed,  
    int ManualReview,  
    int Blocked,  
    IReadOnlyList<RepairItemResult> Items)  
{  
    public bool FullyVerified =>  
        Planned > 0 && Succeeded == Planned &&  
        Failed == 0 && ManualReview == 0 && Blocked == 0;  
}
```

ZERO-WORK GATE

A plan that discovers zero executable items is Closed/NoAction, never a successful repair run.

CONCURRENCY

Overlapping plans conflict on semantic target

RepairControlDb prevents two active cases for the same repair kind, owner and semantic target. An operator may still inspect multiple historical candidates. A new plan waits until the active case is terminal or is explicitly superseded before execution.

DECISION LEDGER 42-73

Overlap	Decision
same target + same plan	return existing case
same target + different active plan	409 active-repair-conflict
same target + terminal prior case	new comparison allowed
different target, shared resource	capacity policy may serialize
bulk plan overlaps single case	exclude or block entire plan by policy

OWNER BACKSTOP

Owner-local receipt keys and target-generation constraints protect against races the control plane cannot see.

RETENTION

Repair evidence outlives transient telemetry

Plans, approvals, attempts, receipts, verification digests and audit hashes follow an administrative retention schedule. Large snapshots and payloads stay in owner stores with durable references. Expired operational logs do not erase the repair decision trail.

DECISION LEDGER 42-74

Artifact	Retention posture
candidate summary/digest	administrative evidence
canonical plan bytes	immutable with case
approvals	immutable subject/policy/expiry
owner receipts	digest + safe reference
verification snapshots	digest + owner reference
payload bytes	owner retention only
metrics/traces	operational schedule

POLICY STATUS

Concrete retention periods depend on deployment obligations and remain VERIFY/PENDING.

PROVIDER-BACKED COMPONENT SUITE

Database behavior is tested where constraints actually live

The suite boots RepairControlDb and one owner database using production providers. It proves unique RepairKey, optimistic concurrency, attempt ordering, transaction rollback, immutable plan bytes and hash-chain continuity.

DECISION LEDGER 42-75

Test ID	Fault/claim	Assertion
REP-DB-001	duplicate RepairKey	one case
REP-DB-002	same key/different plan	conflict; unchanged bytes
REP-DB-003	concurrent state advance	one version wins
REP-DB-004	audit insert fails	transition rolls back
REP-DB-005	worker crash after owner receipt	receipt lookup resumes
REP-DB-006	audit event altered	chain validator fails

NO IN-MEMORY SUBSTITUTE

Provider-specific locking, uniqueness and isolation claims remain unproved by collection-backed test doubles.

ARCHITECTURE TESTS

Static gates prohibit borrowed repair authority

Architecture tests inspect references, registrations, endpoint metadata and EF models. They prevent the repair host from referencing Domain projects, registering business DbContexts or exposing generic database/broker clients to handlers.

EXECUTABLE ASSET 42-AK - AUTHORITY GATE

```
[Fact]
public void ARCH_REPAIR_001_control_plane_has_no_business_dbcontext()
{
    RepairControlAssembly.ShouldNotReference("Nexus.AlarmManagement.Domain");
    RepairControlAssembly.ShouldNotReference("Nexus.RootCause.Domain");
    RepairControlAssembly.ShouldNotReference("Nexus.Compliance.Domain");

    services.ShouldNotContainDbContext("AlarmManagementDbContext");
    services.ShouldNotContainDbContext("RootCauseDbContext");
    services.ShouldNotContainDbContext("ComplianceDbContext");
    services.ShouldNotContainDbContext("ReportingDbContext");
    endpoints.ShouldNotExposeGeneric("sql", "publish", "replay-any");
}
```

COMPOSITION PROOF

The production registration path, not a test-only service collection, is the architecture-test subject.

CONTRACT TESTS

Every owner capability has a frozen request and outcome corpus

Consumer-driven contract tests cover supported repair kind, target grammar, canonical plan digest, receipt outcomes and safe problem mappings. Unknown members and enum values are rejected rather than ignored.

DECISION LEDGER 42-77

Corpus	Required cases
plan V1	golden, unknown member, invalid limits, expired basis
owner request	golden, stale precondition, key conflict
owner receipt	Accepted, AlreadyAccepted, Rejected, Unknown
verification	InSync, Candidate, Blocked, UnsafeUnknown
problem details	401/403/409/412/422/503

VERSION RULE

A new repair semantics requires a new media type or explicit backward-compatible contract decision.

PRODUCTION-PATH HOST TESTS

The real host proves route, policy and DI composition

WebApplicationFactory starts the production repair host with controlled external adapters. Tests inventory every administrative endpoint, authorization policy, content type, idempotency requirement, DbContext lifetime and hosted worker.

EXECUTABLE ASSET 42-AL - ROUTE INVENTORY GATE

```
[Fact]
public async Task HOST_REPAIR_001_admin_surface_is_closed()
{
    await using var factory = new RepairHostFactory();
    var inventory = await factory.DescribeEndpointsAsync();

    inventory.Should().ContainExact(AdminRepairContract.ExpectedEndpoints);
    inventory.Should().OnlyContain(e => e.Route.StartsWith("/admin-api/v1/"));
    inventory.Where(e => e.Mutates)
        .Should().OnlyContain(e => e.RequiresIdempotencyKey &&
            e.AuthorizationPolicy is not null);
    factory.ServiceGraph.Should().HaveNoCaptiveDependencies();
    factory.Workers.Should().ContainSingle("RepairExecutionWorker");
}
```

ZERO DISCOVERY FAILS

An empty endpoint or test inventory is a build failure, never a passing architecture proof.

FAULT LABORATORY

The chapter's claims are tested across the real seams

The deployment laboratory combines production database providers, a broker, owner repair endpoints and the repair host. Fault injection occurs at commit, response, confirm, consumer acknowledgement, verification and restart boundaries.

DECISION LEDGER 42-79

Scenario	Required observation
control commit then response loss	same job returned
owner commit then timeout	same owner receipt recovered
publish accept then disconnect	confirm reconciliation before retry
consumer commit then crash	one inbox effect
projection rebuild crash	checkpointed same generation resumes
source changes after approval	stale plan blocked
two workers lease	fencing rejects stale worker
verification unavailable	ManualReview, never Succeeded

EVIDENCE BUNDLE

One run ID links database snapshots, broker observations, test results, configuration identity and audit export.

GOLDEN END-TO-END REPAIR

A missing correction projection becomes a verified new generation

The reference scenario starts from Chapter 41's failed correction projection. The comparer proves retained correction facts and a missing slot, the operator approves one-target R1 repair, Reporting rebuilds a new generation, verification passes and activation switches only the serving pointer.

DECISION LEDGER 42-80

Step	Reference evidence
compare	experience 019bc4d4-c1b0-7e4e-ae6c-4e3af2e7b331; generation 019bc4d4-c21a-7527-8078-5be0476e77bf
plan	019bc4d4-d311-7f42-9238-337bfebcbdd63; digest 41d4318492363169...
authorize	distinct privileged approver; plan unexpired
execute	replacement generation 019bc4d4-d401-7ced-802f-90464f1f50d9
verify	slot/digest/history/watermark InSync
activate	old Failed retained; one active generation

REFERENCE STATE

This is the normative cross-chapter fixture. Its runtime execution evidence remains PENDING.

NEGATIVE END-TO-END REPAIR

A stale plan performs zero owner actions

The negative scenario approves a plan, then advances the source snapshot before execution. The fresh-evidence guard detects drift, records ExecutionBlocked, creates no owner receipt and requires a new comparison.

EXECUTABLE ASSET 42-AM - STALE-PLAN PROOF

```
[Fact]
public async Task E2E_REPAIR_010_stale_plan_is_side_effect_free()
{
    var plan = await lab.ComparePlanApproveAsync(Target.MissingProjection);
    await lab.AdvanceSourceSnapshotAsync(plan.Target);

    var result = await lab.ExecuteAsync(plan);

    result.ProblemType.Should().EndWith("repair-precondition-failed");
    (await lab.OwnerCallsAsync(plan.RepairKey)).Should().BeEmpty();
    (await lab.OwnerReceiptsAsync(plan.RepairKey)).Should().BeEmpty();
    (await lab.AuditAsync(plan.RepairCaseId))
        .Should().ContainSingle(x => x.EventType == "ExecutionBlocked");
}
```

TOCTOU EVIDENCE

The plan can be approved and still correctly become non-executable when evidence changes.

OFFLINE CONVERGENCE

A bounded replay laboratory proves no permanent divergence

The laboratory stops Reporting, accumulates owner facts, restarts it, induces one deterministic projection failure and repairs through a new generation. It separately delays outbox dispatch and dead-letters one event. All three paths converge or terminate with explicit ManualReview evidence.

DECISION LEDGER 42-82

Phase	Expected invariant
owners continue	facts/outbox durable
Reporting offline	operator view Delayed, not false Complete
restart	normal catch-up attempted before repair
repair	bounded owner-local action
verify	per-kind oracle records result
post-run	no hidden Failed/poison/outbox history

NORMAL PATH FIRST

Repair is not a substitute for ordinary retry, catch-up, gap buffering or health recovery.

DEPLOYMENT

Enable observation before privileged execution

Rollout begins with read-only candidate scans and shadow verification. Dry-run planning follows. Owner repair endpoints are enabled one kind at a time. Execution remains disabled until policy, grants, audit export, limits, dashboards and rollback procedures are evidenced.

DECISION LEDGER 42-83

Stage	Capability	Exit gate
0	schema + host deployed	no execution routes enabled
1	read-only comparison	false-positive review
2	dry-run plans	digest/limits reviewed
3	projection R1 only	provider/fault evidence
4	outbox R2	confirm reconciliation evidence
5	dead-letter R3	dual-control/security evidence
6	bounded automation	SLO and recurrence review

FEATURE FLAG LIMIT

A flag can disable execution quickly; it cannot replace authorization or owner-local idempotency.

ROLLBACK AND KILL SWITCH

Disable new attempts while preserving resolution

The kill switch prevents new leases by repair kind and site. In-flight attempts continue to a durable receipt, Unknown or ManualReview. Read-only status, audit export and verification remain available so operators can resolve the consequences of already-started work.

DECISION LEDGER 42-84

Control	Behavior
global execute off	no new owner actions
kind/site off	targeted lease suppression
pause batch	no new item leases
owner circuit open	attempt remains Scheduled/Delayed
verification only	continues read-only checks
audit access	always retained under policy

NO DESTRUCTIVE ROLLBACK

Disabling the feature never deletes repair rows, poison evidence, outbox attempts or projection generations.

RUNBOOK

The operator sequence is executable and conservative

The runbook begins by preserving evidence and checking normal recovery. It ends only after verification and evidence export. Any uncertainty pauses execution rather than widening scope.

DECISION LEDGER 42-85

Order	Action
1	identify incident/site/owner; freeze broad automation
2	capture health, config identity and safe evidence references
3	run bounded read-only comparison
4	confirm normal retry/catch-up cannot resolve
5	dry-run exact plan; review limits and cause remediation
6	collect policy-required approvals
7	execute one bounded plan and monitor owner receipt
8	verify independently; export audit/evidence
9	review recurrence; fix underlying defect

STOP CONDITION

Stop on stale evidence, integrity failure, active hold, unknown contract, unexpected scope or unavailable verification.

OPERATIONAL DECISION TABLE

The tool answers 'what next' without pretending certainty

Operators need a stable disposition for common observations. The decision table keeps wait, compare, repair and escalate separate.

DECISION LEDGER 42-86

Observation	Disposition	Reason
projection within lag budget	Wait	normal convergence
projection overdue; retained fact/slot mismatch	Plan rebuild	supported candidate
outbox due; lease active	Wait/inspect	owner work in flight
outbox confirm unknown	Reconcile	duplicate/loss ambiguity
dead letter; defect unresolved	Blocked	replay would repeat failure
dead letter; remediated and approved	Replay	normal route restored
evidence incomplete	ManualReview	unsafe to automate

NO GENERIC REPAIR

The action is specific to the diagnosed invariant and owning capability.

EVIDENCE CATALOGUE

Every runtime claim names a retained artifact

The evidence catalogue binds claims to exact outputs rather than screenshots or prose. A run is admissible only when configuration identity, test discovery counts, database/broker observations, audit chain and verification result refer to one run ID.

DECISION LEDGER 42-87

Claim	Required retained artifact
idempotent control request	API transcript + RepairControlDb case/plan
owner action once	owner receipt + local rows
exact outbox replay	frozen body digest + publish attempts
one consumer effect	inbox + business rows
new projection generation	generation/checkpoint/slot rows
dual control	approval records + identity/policy evidence
audit integrity	export + hash-chain validation
post-repair convergence	verification snapshot + digest

SAME-RUN RULE

Artifacts from different deployments, policies or runs cannot be assembled into one PROVED verdict.

EVIDENCE MANIFEST

The reference record stays honestly PENDING

The chapter supplies a machine-readable manifest shape with expected artifacts and verdict rules. Because no repository execution bundle is attached to this manuscript, every runtime-dependent claim remains PENDING.

EXECUTABLE ASSET 42-AN - EVIDENCE RECORD

```
{
  "chapter": 42,
  "adr": "ADR-042",
  "adrSha256": "5982cb3f37a3d5f90ebfd13d188d77c0ffab7f56a5e44449a1cac33acf162e19",
  "planSha256": "41d43184923631699b2d6ea069da502a2f8f86cd7bcfd7b3a8cabf4a4a537592",
  "status": "PENDING",
  "runId": null,
  "configurationIdentity": null,
  "artifacts": {
    "providerTests": null,
    "brokerFaultLab": null,
    "routeInventory": null,
    "ownerReceipts": null,
    "auditExport": null,
    "verificationSnapshots": null
  },
  "publicCommandCount": 0
}
```

VERDICT

SPECIFIED | RUNTIME PENDING

MANIFEST VALIDATOR

Zero discovery, stale runs and missing artifacts fail closed

The validator checks run identity, artifact existence and digests, configuration identity, test discovery counts, timestamps, expected scenario IDs and cross-artifact repair keys. It cannot promote a listing or a screenshot to runtime proof.

EXECUTABLE ASSET 42-AO - FAIL-CLOSED EVIDENCE GATE

```
public EvidenceVerdict Validate(RepairEvidenceManifest m)
{
  Require(m.Status == "CANDIDATE");
  RequireNotEmpty(m.RunId, m.ConfigurationIdentity);
  Require(m.Artifacts.All(a => a.Exists && a.Sha256Matches));
  Require(m.Tests.Discovered > 0 && m.Tests.Failed == 0);
  Require(m.Tests.RequiredIds.SetEquals(RequiredRepairScenarios));
  Require(m.Artifacts.Select(a => a.RunId).Distinct().Single() == m.RunId);
  Require(m.OwnerReceipts.All(r => r.RepairKey == m.RepairKey));
  Require(m.AuditChain.Valid && m.Verification.Verdict == "InSync");
  return EvidenceVerdict.Proved(m.RunId);
}
```

NO SELF-ATTESTATION

The build script cannot declare PROVED merely because it generated the expected manifest schema.

PART VII ACCEPTANCE

The distributed workflow now has an explicit recovery boundary

Chapters 37-42 define the end-to-end slice from workflow identity through choreography, deadlines, compensation, operator uncertainty and repair. The acceptance ledger keeps business truth, projection truth, administrative authority and runtime evidence separate.

DECISION LEDGER 42-90

Part VII concern	Accepted boundary
identity	workflow/correlation/cycle coordinates retained
coordination	observed choreography; Process Manager emits no public commands
time	timeouts/escalations are explicit evidence
compensation	forward correction; history retained
operator experience	Pending/Delayed/Completed/Failed projection states
repair	compare-plan-authorize-execute-verify
evidence	PENDING unless same-run artifacts prove execution

NO GLOBAL PRESENT

The book still refuses a single status or database that claims to contain all current truth.

PART VII INVARIANTS

Repair does not weaken the earlier chapters

The repair mechanism is accepted only if it preserves the invariants established across the part. A repair feature that bypasses one of these rules is a regression, not operational maturity.

DECISION LEDGER 42-91

Invariant	Chapter 42 enforcement
service-owned truth	no business DbContext in repair host
zero public command additions	administrative routes stay separate
append/supersede history	new generations and attempts
at-least-once delivery	RepairKey + inbox idempotency
honest uncertainty	Unknown/ManualReview remain visible
bounded operation	budgets, limits, expiry and kill switch
evidence discipline	same-run fail-closed manifest

ACCEPTANCE TEST

Any proposed convenience must show which invariant it preserves and which executable proof demonstrates that claim.

REJECTED SHORTCUTS

Convenient repair buttons create hidden ownership

The chapter rejects direct SQL, blind replay, checkpoint edits, body mutation, self-approval, unbounded scans and success inferred from HTTP acceptance. Each shortcut removes the evidence needed to distinguish recovery from corruption.

DECISION LEDGER 42-92

Shortcut	Failure introduced	Required design
UPDATE projection row	history/semantics hidden	new generation
duplicate outbox insert	second public intent	exact redispach
drain DLQ	hostile/permanent failures replayed	eligibility + approval
invoke consumer handler	transport/inbox bypassed	normal route
approve mutable form	authorization ambiguity	canonical digest
retry on timeout	duplicate side effect	receipt lookup/reconcile
200 means fixed	no convergence proof	verification oracle

DESIGN SMELL

A generic RepairAnything endpoint is evidence that ownership and verification have not been designed.

REVIEW QUESTIONS

A senior review can challenge every consequential edge

The review checklist is organized around authority, identity, preconditions, idempotency, verification and evidence. An answer that depends on 'the operator will know' is not an executable control.

DECISION LEDGER 42-93

Question	Expected artifact
who owns the target action?	typed owner port + credential map
what exact mismatch exists?	comparison snapshot/digest
what bytes were approved?	canonical plan + SHA-256
what changed after approval?	fresh precondition result
why can retry be safe?	RepairKey + owner receipt
what proves convergence?	independent verification
what remains uncertain?	explicit ManualReview/PENDING fields

REVIEW STANDARD

The correct answer points to a durable artifact or a testable invariant, not an intention.

CHAPTER 43 BOUNDARY

Repair paths expose the system's most privileged trust edges

Chapter 42 specifies authorization records, scopes, credential separation and safe evidence, but it does not claim that identities, networks, secrets or administrative workstations are trustworthy. Chapter 43 must draw those boundaries before selecting mechanisms.

DECISION LEDGER 42-94

Chapter 42 exposes	Chapter 43 must analyze
human request/approval	identity, device and session trust
admin API	BFF/admin ingress and service authentication
owner repair ports	service-to-service authorization
broker replay	publisher identity and route permissions
database/audit	credential and tamper boundaries
telemetry/evidence export	confidentiality and integrity

PROTECTED HANDOFF

Security mechanisms follow the threat and trust-boundary ledger; they are not retrofitted from product defaults.

OFFICIAL TECHNICAL ANCHORS

Repair semantics are anchored in owner transactions and reliable delivery

- EF Core transactions: <https://learn.microsoft.com/ef/core/saving/transactions>
- EF Core optimistic concurrency: <https://learn.microsoft.com/ef/core/saving/concurrency>
- RabbitMQ reliability guide: <https://www.rabbitmq.com/docs/reliability>
- RabbitMQ publisher confirms: <https://www.rabbitmq.com/docs/confirmations>
- RabbitMQ dead-letter exchanges: <https://www.rabbitmq.com/docs/dlx>
- RabbitMQ quorum queue dead-lettering: <https://www.rabbitmq.com/docs/quorum-queues#dead-lettering>
- Azure Retry pattern: <https://learn.microsoft.com/azure/architecture/patterns/retry>
- Azure Scheduler Agent Supervisor pattern:
<https://learn.microsoft.com/azure/architecture/patterns/scheduler-agent-supervisor>
- JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785>
- HTTP Semantics: <https://www.rfc-editor.org/rfc/rfc9110>
- Problem Details for HTTP APIs: <https://www.rfc-editor.org/rfc/rfc9457>
- OpenTelemetry HTTP semantic conventions: <https://opentelemetry.io/docs/specs/semconv/http/>

WHAT THIS CHAPTER HAS PROVED

Chapter 42 specifies ADR-042; typed comparison snapshots and candidates; canonical plans and deterministic repair keys; fresh-evidence guards, risk classes, dual control and blast limits; an append-only RepairControlDb and audit chain; owner-local idempotent repair ports; new-generation projection rebuild, exact outbox redispach and normal-route poison replay; independent verification, failure recovery, administrative contracts, observability, capacity, deployment, runbook and fail-closed evidence validation. Runtime evidence remains PENDING.

CHAPTER CHECKPOINT

- Why is a repair candidate not permission to execute?
- Which data may RepairControlDb own, and which business state must it never write?
- What exact bytes and coordinates does an approval bind?
- How do fresh-evidence preconditions and owner receipts close different race windows?
- Why does projection repair create a new generation?
- Which fields remain identical during outbox redispach?
- Why must a dead-letter replay use the normal route and inbox path?
- What does an owner receipt prove, and what does it not prove?
- When must execution stop in ManualReview?
- Which same-run artifacts are required before ADR-042 can become PROVED?

NEXT - CHAPTER 43

Threat and Trust Boundaries Before Mechanisms will map users, the BFF, service APIs, owner-local repair ports, the broker, databases, telemetry, evidence exports and administrative paths. It will identify who and what is trusted at each crossing before the book selects authentication, authorization, secret or network mechanisms.

PART VIII

Security and Trust Across Services

The Phase-0 boundary crosses the network without claiming production security maturity. Trust is explicit, local, bounded and evidenced.

CHAPTER 43

Threat and trust boundaries

assets -> crossings -> threats -> requirements

CHAPTER 44

Edge identity and authority

user -> BFF -> owner policy

CHAPTER 45

Workload identity

service -> service / broker / repair port

CHAPTER 46

Minimal claims

external context -> local capability

CHAPTER 47

Secrets and certificates

issue -> use -> rotate -> recover

CHAPTER 48

Security evidence

attack -> refusal -> audit -> proof

ENTRY RULE

A successful connection proves neither identity nor authority. Every crossing must name the principal, resource, capability, evidence and failure behavior.

BEGIN WITH CHAPTER 43 / DRAW THE BOUNDARY BEFORE CHOOSING THE MECHANISM
